

Traducción al español de Python Programming and Numerical Methods

A Guide for Engineers and Scientists

Qingkai Kong, Timmy Siau, Alexandre M. Bayen

Traducción: **Nahuel Villafañe**

Año: **2026**

Edición: **1.0**

FUNDAMENTOS DE PYTHON

Comenzando con Python

Configuración de tu entorno de trabajo

El primer paso para usar Python es configurar el entorno de trabajo en la computadora. Esta sección presenta los procesos iniciales para ponerlo en marcha.

Hay diferentes maneras de instalar Python y los paquetes relacionados, y recomendamos usar **Anaconda**¹ o **Miniconda**² para instalar y administrar tus paquetes. Dependiendo del sistema operativo que estés usando (por ejemplo, Windows, Mac OS X o Linux), necesitarás descargar un instalador específico para tu máquina. Tanto Anaconda como Miniconda están diseñadas para proporcionar formas fáciles de administrar el entorno de trabajo de Python en computación científica y ciencia de datos.

En este ejemplo, usaremos Mac OS X para mostrarte cómo instalar Miniconda (el proceso es muy similar a la instalación en Linux). Para usuarios de Windows, por favor, omitan el resto de esta sección y lean el Apéndice A sobre las instrucciones de instalación. Las principales diferencias entre Anaconda y Miniconda son las siguientes:

- **Anaconda** es un marco de distribución completo que incluye el intérprete de Python, el administrador de paquetes y los paquetes de uso común en computación científica.

¹<https://www.anaconda.com/download/>

²<https://conda.io/miniconda.html>

- **Miniconda** es una versión "ligera" de Anaconda que no incluye los paquetes de uso común. Necesitas instalar todos los paquetes diferentes tú mismo, pero sí incluye el intérprete de Python y el administrador de paquetes.

La opción que hemos elegido aquí es Miniconda, e instalaremos solo los paquetes que necesitaremos. El proceso de instalación de Miniconda se describe a continuación:

Paso 1. Descarga el instalador de Miniconda desde el sitio web³. La página de descarga se muestra en la Fig. 1. Aquí puedes elegir un instalador diferente según tu sistema operativo. En este ejemplo, elegimos Mac OS X y Python 3.7.

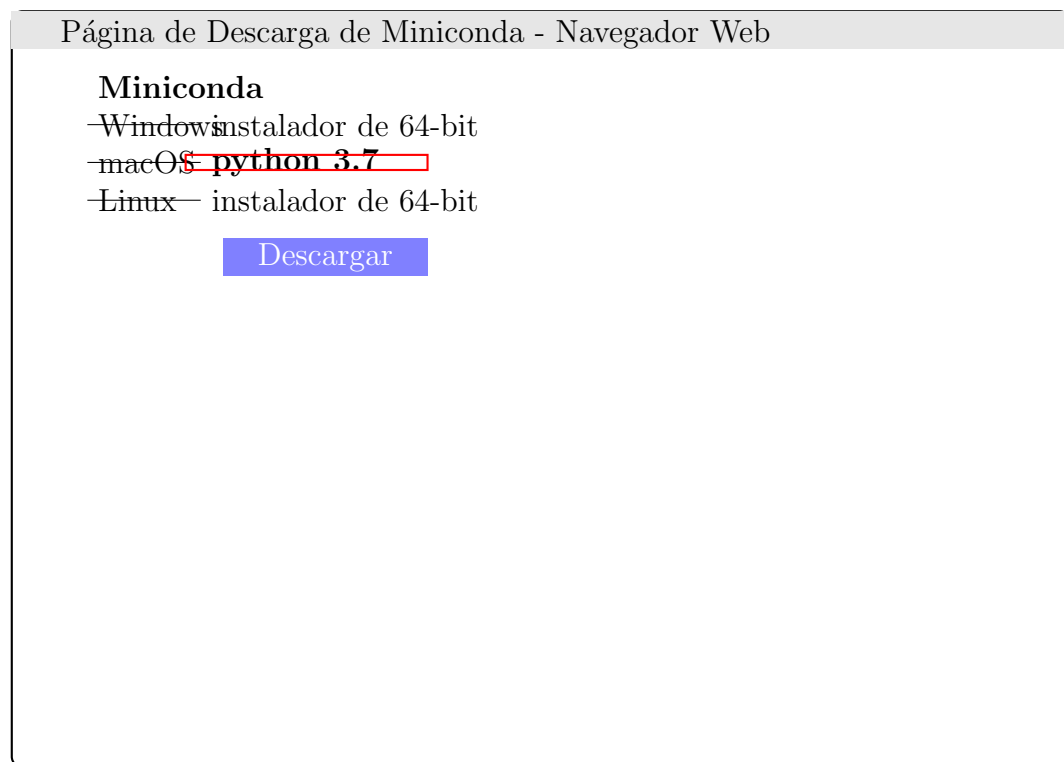
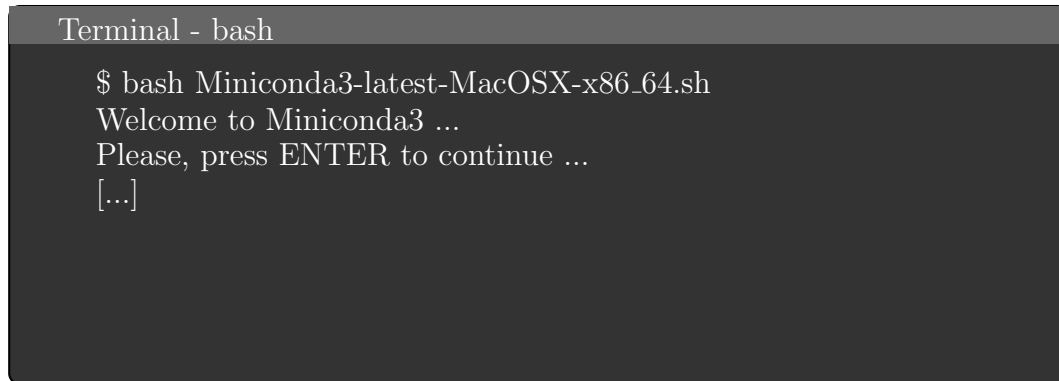


Figura 1: La página de descarga de Miniconda; elige el instalador según tu sistema operativo.

³<https://conda.io/miniconda.html>

Paso 2. Abre una terminal (en una Mac, puedes buscar "terminal" en la búsqueda de Spotlight). Ejecuta el instalador desde la terminal usando los comandos que se muestran en la Fig. 2. Después de ejecutar el instalador, sigue la guía para finalizar la instalación.

A screenshot of a terminal window titled "Terminal - bash". The terminal shows the command "\$ bash Miniconda3-latest-MacOSX-x86_64.sh" being executed. Below the command, the text "Welcome to Miniconda3 ..." is displayed, followed by "Please, press ENTER to continue ...". The prompt "[...]" is visible at the end of the line.

```
Terminal - bash
$ bash Miniconda3-latest-MacOSX-x86_64.sh
Welcome to Miniconda3 ...
Please, press ENTER to continue ...
[...]
```

Figura 2: Captura de pantalla de la ejecución del instalador en una terminal.

Ten en cuenta que, aunque puedes cambiar la ubicación de la instalación dándole una ubicación alternativa en tu máquina, la ubicación predeterminada es tu directorio de inicio (Fig. 3).

A screenshot of a terminal window titled "Terminal - bash". The terminal shows the prompt "[/Users/nahuel] \$\$\$" indicating the current directory is the user's home directory.

```
Terminal - bash
[/Users/nahuel] $$$
```

Figura 3: La ubicación de instalación predeterminada de tu sistema de archivos.

Después de la instalación, puedes verificar los paquetes instalados escribiendo los comandos que se muestran en la Fig. 4.

```
Terminal - bash

$ conda -version
conda 4.8.3

$ python -version
Python 3.7.6
```

Figura 4: Forma rápida de comprobar si Miniconda se instaló correctamente y los programas se ejecutan correctamente.

Paso 3. Como se muestra en la Fig. 5, instala los paquetes básicos utilizados en este libro: `ipython`, `numpy`, `scipy`, `pandas`, `matplotlib` y `jupyter notebook`. En una sección posterior, hablaremos sobre la gestión de los paquetes usando `pip` y `conda`.

```
Terminal - bash

$ conda install ipython numpy scipy pandas matplotlib jupyter

Proceed ([y]/n)? y
Downloading and Extracting Packages ...
```

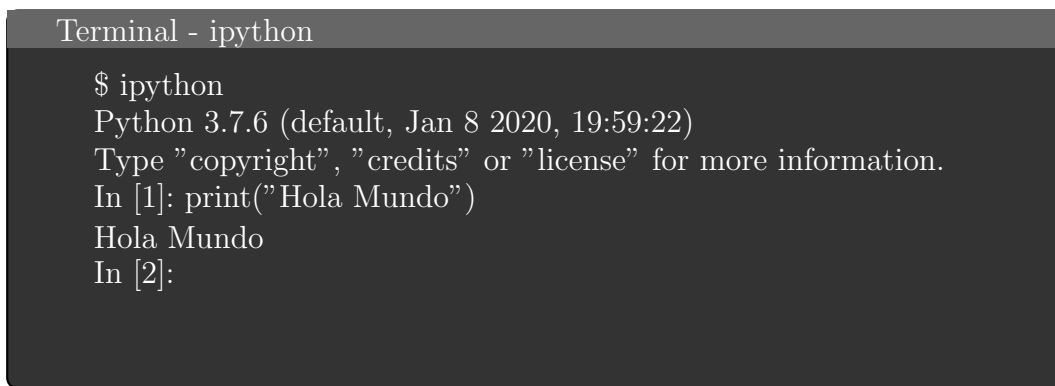
Figura 5: Proceso de instalación de los paquetes que se utilizarán en el resto del libro.

Tres formas de ejecutar código Python

Hay diferentes maneras de ejecutar código Python y todas tienen diferentes usos. Esta sección introducirá las tres formas diferentes para empezar.

Uso del shell de Python o IPython.

La forma más fácil de ejecutar código Python es a través del shell de Python o el shell de IPython (que significa Python Interactivo). El shell de IPython es más potente que el shell de Python, e incluye características como autocompletado con la tecla TAB, mensajes de error resaltados en color, integración básica con el shell de UNIX, etc. Dado que acabamos de instalar IPython, intentemos ejecutar el ejemplo de "Hola Mundo" usando el shell de IPython. Para iniciar el shell de IPython, comenzamos escribiendo `ipython` en una terminal (ver Fig. 6). Luego ejecutamos un comando de Python escribiéndolo en el shell y presionando Enter. Inmediatamente vemos los resultados del comando. Por ejemplo, podemos imprimir "Hola Mundo" usando la función `print` en Python, como se muestra en la Fig. 6.



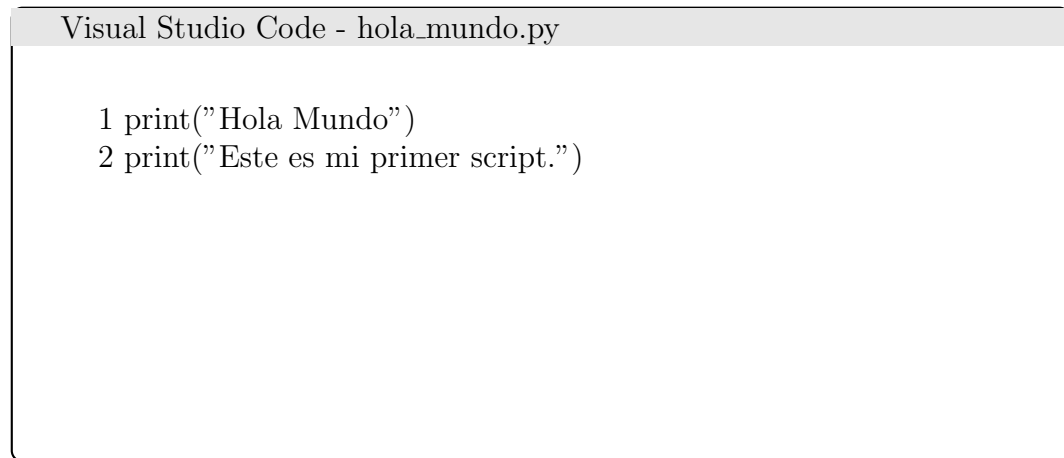
```
Terminal - ipython
$ ipython
Python 3.7.6 (default, Jan 8 2020, 19:59:22)
Type "copyright", "credits" or "license" for more information.
In [1]: print("Hola Mundo")
Hola Mundo
In [2]:
```

Figura 6: Ejecuta "Hola Mundo" en el shell de IPython escribiendo el comando. "print" es una función de la que se hablará más adelante en el libro y que imprimirá cualquier cosa entre paréntesis.

Ejecutar un script/archivo de Python desde la línea de comandos.

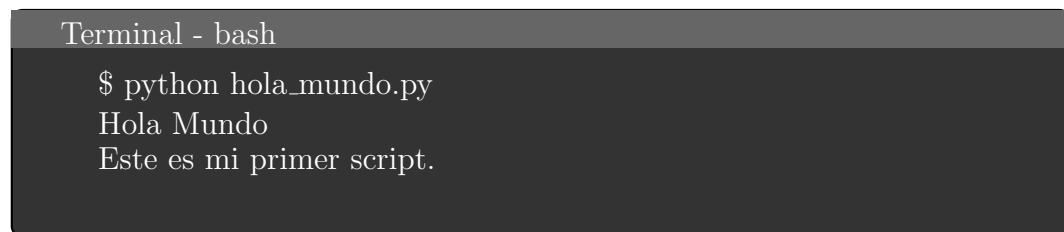
La segunda forma de ejecutar código Python es poner todos los comandos en un archivo y guardarlo con la extensión `.py`; la extensión del archivo puede ser cualquier cosa, pero por convención, suele ser `.py`. Por ejemplo, usa tu editor de texto favorito (aquí usamos Visual Studio Code⁴) para escribir los comandos que deseas ejecutar en un archivo llamado `hola_mundo.py`, como se muestra en la Fig. 7, que luego se ejecuta desde la terminal (ver Fig. 8).

⁴<https://code.visualstudio.com>

A screenshot of the Visual Studio Code editor interface. The title bar at the top reads "Visual Studio Code - hola_mundo.py". The main editing area contains two lines of Python code:

```
1 print("Hola Mundo")  
2 print("Este es mi primer script.")
```

Figura 7: Ejemplo de un archivo de script de Python usando Visual Studio Code. Escribe los comandos que quieras ejecutar y guarda el archivo con un nombre apropiado.

A screenshot of a terminal window with a dark background. The title bar reads "Terminal - bash". The terminal shows the command to run a Python script and its output:

```
$ python hola_mundo.py  
Hola Mundo  
Este es mi primer script.
```

Figura 8: Para ejecutar el script de Python desde la línea de comandos, escribe "python hola_mundo.py". Esta línea le dice a Python que ejecute los comandos guardados en este archivo.

Uso de Jupyter Notebook.

La tercera forma de ejecutar Python es a través de Jupyter Notebook, que es un entorno de Python muy potente basado en el navegador. Discutiremos esto en detalle más adelante en este capítulo. El ejemplo presentado aquí es para demostrar la rapidez con la que podemos ejecutar código usando Jupyter notebook. Si escribes `jupyter notebook` en la terminal, aparecerá una página web local; usa el botón de la parte superior derecha para crear un nuevo notebook de Python 3, como se muestra en la Fig. 9.

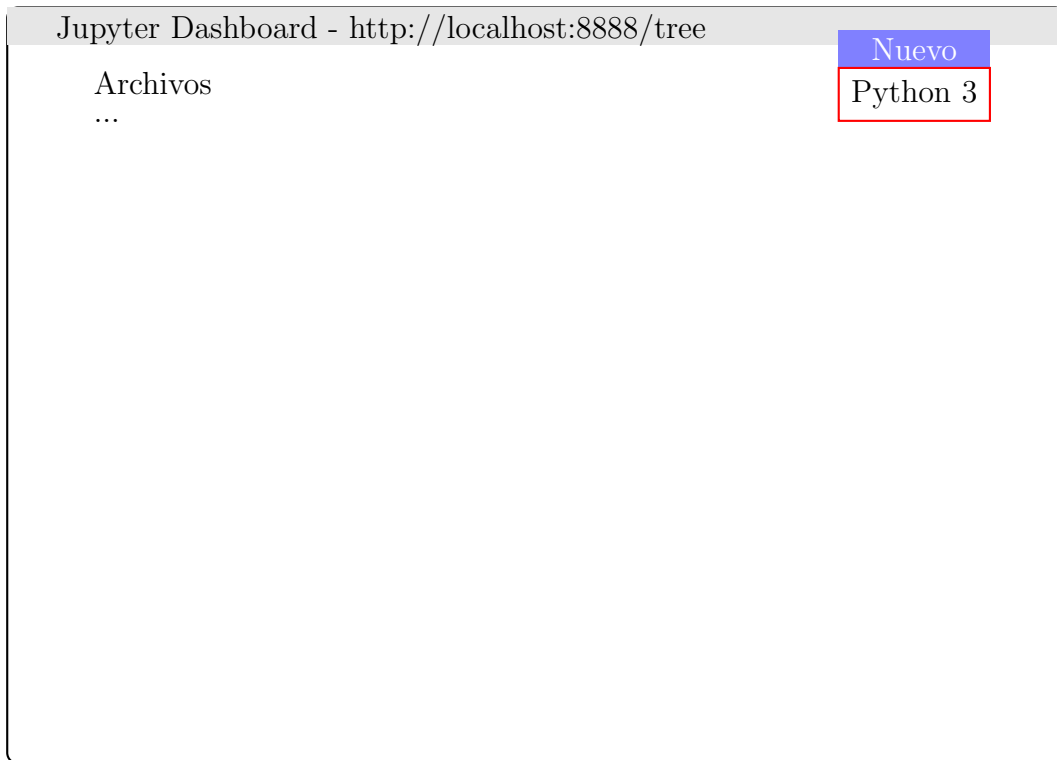


Figura 9: Para lanzar un servidor de Jupyter notebook, escribe `jupyter notebook` en la línea de comandos, lo que abrirá una página del navegador como se muestra aquí. Haz clic en el botón "Nuevo" en la parte superior derecha y elige "Python3". Esto creará un notebook de Python desde el cual ejecutar código Python.

Ejecutar código en Jupyter notebook es fácil. Escribe tu código en la celda y presiona **Shift + Enter** para ejecutar la celda; los resultados se mostrarán debajo del código (Fig. 10).

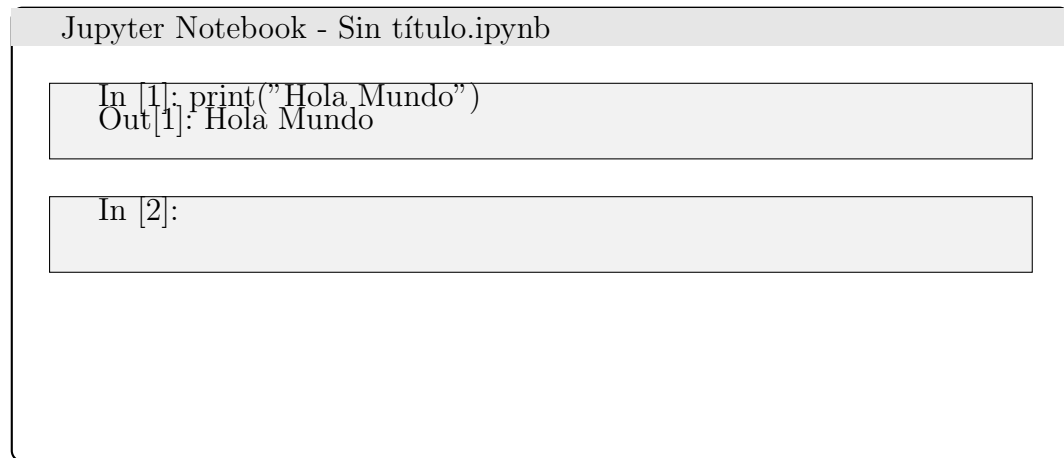


Figura 10: Para ejecutar el ejemplo de Hola Mundo dentro de Jupyter notebook, escribe el comando en la celda de código (los cuadros grises) y presiona Shift + Enter para ejecutarlo.

Python como calculadora

Python contiene funciones que se encuentran en cualquier calculadora gráfica estándar. Una **operación aritmética** es la suma, resta, multiplicación, división o potencias entre dos números. Un **operador aritmético** es un símbolo que Python tiene reservado para significar una de las operaciones mencionadas. Estos símbolos son + para la suma, - para la resta, * para la multiplicación, / para la división y ** para la exponenciación.

Una instrucción u operación se **ejecuta** cuando es resuelta por la computadora. Una instrucción se ejecuta en el prompt de comandos escribiéndola donde veas el símbolo

¿ en el shell de Python (o el signo In [1]: en IPython) y luego presionando Enter. En el caso de Jupyter notebook, escribe la operación en la celda de código y presiona Shift + Enter. Dado que usaremos Jupyter notebook para el resto del libro, para familiarizarte con todas las diferentes opciones, todos los ejemplos de esta sección se mostrarán en el shell de IPython; consulta la sección anterior para saber cómo empezar a trabajar en IPython. Para usuarios de Windows, usarán el prompt de Anaconda que se muestra en el Apéndice A en lugar de usar la terminal.

```
1 In [1]: 1 + 2
```

```
2 Out [1]: 3
```

Listing 1: Calculando la suma de 1 y 2.

Un **orden de operaciones** es un orden estándar de precedencia que tienen las diferentes operaciones en relación con las demás. Python utiliza el mismo orden de operaciones que aprendiste en la escuela. Las potencias se ejecutan antes que la multiplicación y la división, que se ejecutan antes que la suma y la resta. Los paréntesis () también se pueden usar en Python para anular el orden de operaciones estándar.

```
1 In [2]: (3*4) / (2**2 + 4/2)
2 Out [2]: 2.0
```

Listing 2: Calculando $\frac{3 \times 4}{(2^2 + 4/2)}$.

Consejo

Observa que `Out [2]` es el valor resultante de la última operación ejecutada. Usa el símbolo de subrayado `_` para representar este resultado y así dividir expresiones complicadas en comandos más simples.

```
1 In [3]: 3/4
2 Out [3]: 0.75
3
4 In [4]: _*2
5 Out [4]: 1.5
6
7 In [5]: _**3
8 Out [5]: 3.375
```

Listing 3: Calculando 3 dividido por 4, luego multiplicando el resultado por 2, y luego elevando el resultado a la 3^a potencia.

Python tiene muchas funciones aritméticas básicas como `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `exp`, `log`, `log10` y `sqrt` almacenadas en un módulo (explicado más adelante en este capítulo) llamado `math`. Primero, importa este módulo para acceder a estas funciones.

```
1 In [6]: import math
```

Listing 4: Importando el módulo `math`.

Consejo

En Jupyter notebook e IPython, puedes obtener una vista rápida de lo que hay en el módulo escribiendo el nombre del módulo + punto + TAB. Además, si escribes las primeras letras de la función y presionas TAB, completará automáticamente la función, lo que se conoce como “autocompletado con TAB” (se muestra un ejemplo en la Fig. 11).



Figura 11: Ejemplo que demuestra la búsqueda interactiva de funciones dentro de IPython escribiendo TAB después del punto. El cuadro gris muestra todas las funciones disponibles.

Estas funciones matemáticas se ejecutan a través de `modulo.funcion`. Las entradas a ellas siempre se colocan dentro de paréntesis que están conectados al nombre de la función. Para las funciones trigonométricas, es útil tener el valor de π disponible. Puedes llamar a este valor en cualquier momento escribiendo `math.pi` en el shell de IPython.

```
1 In [7]: math.sqrt(4)
2 Out[7]: 2.0
```

Listing 5: Encontrando la raíz cuadrada de 4.

```
1 In [8]: math.sin(math.pi/2)
2 Out[8]: 1.0
```

Listing 6: Calculando $\sin(\pi/2)$.

Python compone funciones como se espera, ejecutándose primero la función más interna. Lo mismo ocurre con las llamadas a funciones que se combinan con operaciones aritméticas.

```
1 In [9]: math.exp(math.log(10))
2 Out[9]: 10.000000000000002
```

Listing 7: Calculando $e^{\log 10}$.

Observa que la función `log` en Python es $\log_e$, o el logaritmo natural. No es $\log_{10}$. Para usar $\log_{10}$, utiliza la función `math.log10`.

Consejo

Puedes ver que el resultado anterior debería ser 10, pero en Python, aparece como 10.000000000000002. Esto se debe a la aproximación numérica de Python, que se discute en el Capítulo 9.

```
1 In [10]: math.exp(3/4)
2 Out[10]: 2.117000016612675
```

Listing 8: Calculando $e^{\frac{3}{4}}$.

Consejo

Usar la FLECHA HACIA ARRIBA en el prompt de comandos recupera los comandos ejecutados previamente. Si accidentalmente escribes un comando incorrecto, puedes usar la FLECHA HACIA ARRIBA para recuperarlo y luego editarlo en lugar de volver a escribir la línea completa.

A menudo, cuando se usa una función en Python, necesitas ayuda específica para el contexto de la función. En IPython o Jupyter notebook, la descripción de cualquier función está disponible escribiendo `función?`; el signo de interrogación es un atajo para la ayuda. Si ves una función con la que no estás familiarizado, es una buena práctica usar el signo de interrogación antes de preguntar a tus instructores qué hace una función específica.

```
1 In [11]: math.factorial?
2 Signature: math.factorial(x, /)
3 Docstring:
4 Find x!.
5 Raise a ValueError if x is negative or non-integral.
6 Type: builtin_function_or_method
```

Listing 9: Usando el signo de interrogación para encontrar la definición de la función factorial.

Python generará un error `ZeroDivisionError` cuando aparezca la expresión $1/0$ (que es infinito), para recordártelo.

```

1 In [12]: 1/0
2 -----
3 ZeroDivisionError Traceback (most recent call last)
4 <ipython-input-12-9e1622b385b6> in <module>()
5 ----> 1 1/0
6 ZeroDivisionError: division by zero

```

Listing 10: Intentando 1/0.

Puedes escribir `math.inf` en el prompt de comandos para denotar infinito o `math.nan` para denotar algo que no es un número pero que deseas que se maneje como un número. Si esto es confuso, esta distinción puede omitirse por ahora; se explicará en detalle más adelante. Finalmente, Python también puede manejar números imaginarios.

```

1 In [13]: 1/math.inf
2 Out[13]: 0.0
3
4 In [14]: math.inf * 2
5 Out[14]: inf
6
7 In [15]: math.inf/math.inf
8 Out[15]: nan

```

Listing 11: Probando infinito.

```

1 In [16]: 2 + 5j
2 Out[16]: (2+5j)

```

Listing 12: Calculando la suma $2 + 5i$.

Observa que en Python la parte imaginaria se representa con `j` en lugar de `i`. Otra forma de representar un número complejo en Python es usar la función `complex`.

```

1 In [17]: complex(2,5)
2 Out[17]: (2+5j)

```

Listing 13: Usando la función `complex`.

Python también puede manejar notación científica usando la letra `e` entre dos números. Por ejemplo, $1e6 = 1000000$ y $1e-3 = 0.001$.

```

1 In [18]: 3e0*3.65e2*2.4e1*3.6e3
2 Out[18]: 94608000.0

```

Listing 14: Calculando el número de segundos en 3 años usando notación científica.

Consejo

Cada vez que se escribe una función del módulo `math`, siempre se escribe `math.nombre_de_la_funcion`. Alternativamente, hay una forma más sencilla. Por ejemplo, si queremos usar `sin` y `log` del módulo `math`, podemos importarlos de la siguiente manera: `from math import sin, log`. Con esta declaración de importación modificada, al usar estas funciones, las usamos directamente, por ejemplo, `sin(20)` o `log(10)`.

Los ejemplos anteriores demostraron cómo usar Python como calculadora para tratar con diferentes valores de datos. En Python, hay tipos de datos adicionales necesarios para los valores numéricos: `int`, `float` y `complex` son los tipos asociados con estos valores.

- `int`: Enteros, como 1, 2, 3, ...
- `float`: Números de punto flotante, como 3.2, 6.4, ...
- `complex`: Números complejos, como $2 + 5j$, $3 + 2j$, ...

Usa la función `type` para verificar el tipo de datos de diferentes valores.

```
1 In [19]: type(1234)
2 Out[19]: int
```

Listing 15: Comprobando el tipo de dato de 1234.

```
1 In [20]: type(3.14)
2 Out[20]: float
```

Listing 16: Comprobando el tipo de dato de 3.14.

```
1 In [21]: type(2 + 5j)
2 Out[21]: complex
```

Listing 17: Comprobando el tipo de dato de $2 + 5j$.

Por supuesto, hay otros tipos de datos, como `boolean`, `string`, etc.; estos se introducen en el Capítulo 2.

Esta sección demostró cómo usar Python como una calculadora ejecutando comandos en el shell de IPython. Antes de pasar a una codificación más compleja, aprendamos más sobre la gestión de paquetes, es decir, cómo instalar, actualizar y eliminar los paquetes.

Gestión de paquetes

Una característica que hace que Python sea realmente genial son los diversos paquetes/módulos desarrollados por la comunidad de usuarios. La mayoría de las veces, cuando quieres aplicar algunas funciones o algoritmos, a menudo encontrarás varios paquetes ya disponibles. Todo lo que necesitas hacer es instalar los paquetes y usarlos en tu código. Gestionar paquetes es una de las habilidades más importantes que necesitas aprender para aprovechar al máximo Python. Esta sección te mostrará cómo gestionar paquetes en Python.

Gestión de paquetes usando gestores de paquetes

Al principio de este libro, instalamos algunos paquetes usando `pip` escribiendo `pip install nombre_del_paquete`. Esta es actualmente la forma más común y fácil de instalar paquetes de Python. **Pip** es un gestor de paquetes que automatiza el proceso de instalación, actualización y eliminación de los paquetes. Puede instalar paquetes publicados en el Python Package Index (PyPI)⁵. Si instalaste Miniconda, `pip` también estará disponible para ti.

Usa `pip help` para obtener ayuda sobre los diferentes comandos, como se muestra en la Fig. 12.

```
Terminal - pip help

$ pip help
Usage: pip [command] [options]

Commands:
install Install packages.
uninstall Uninstall packages.
list List installed packages.
show Show information about installed packages.
```

Figura 12: El documento de ayuda de `pip` después de ejecutar `pip help`.

⁵<https://pypi.org/>

Los comandos más utilizados suelen incluir: instalar, actualizar y desinstalar un paquete.

Instalar un paquete

Para instalar la última versión de un paquete:

```
1 pip install nombre_del_paquete
```

Listing 18: Instalando la última versión.

Para instalar una versión específica, por ejemplo, instalar la versión 1.5:

```
1 pip install nombre_del_paquete==1.5
```

Listing 19: Instalando una versión específica.

Pip instalará el paquete, así como los otros paquetes de los que depende, para que los uses.

Actualizar un paquete

Para actualizar un paquete instalado a la última versión desde PyPI.

```
1 pip install --upgrade nombre_del_paquete
```

Listing 20: Actualizando un paquete.

o simplemente

```
1 pip install -U nombre_del_paquete
```

Desinstalar un paquete

```
1 pip uninstall nombre_del_paquete
```

Listing 21: Desinstalando un paquete.

Otros comandos útiles

Otros comandos útiles que proporcionan información sobre los paquetes instalados están disponibles. Como se muestra en la Fig. 13, si quieres obtener una lista de todos los paquetes instalados, puedes usar el comando:

```
1 pip list
```



```
Terminal - pip list

$ pip list
Package Version
-----
ipython 7.5.0
jupyter 1.0.0
matplotlib 3.1.0
numpy 1.16.4
```

Figura 13: Usando `pip list` para mostrar todos los paquetes instalados en tu máquina.

Si quieres saber más sobre un paquete instalado, como la ubicación del paquete, los otros paquetes dependientes requeridos, etc., puedes usar el siguiente comando como se muestra en la Fig. 14:

```
1 pip show nombre_del_paquete
```

```
Terminal - pip show numpy

$ pip show numpy
Name: numpy
Version: 1.16.4
Location: /usr/local/lib/python3.7/site-packages
Requires:
```

Figura 14: Usando `pip show` para obtener información detallada sobre un paquete instalado.

Existen otros gestores de paquetes, por ejemplo, **conda** (que viene incluido con la distribución de Anaconda y es similar a **pip** en términos de capacidad); por lo tanto, no se discute más, y puedes encontrar más información leyendo la documentación⁶.

⁶<https://conda.io/docs/user-guide/getting-started.html>

Instalar paquetes desde la fuente

Ocasionalmente, necesitarás descargar el archivo fuente de algún proyecto que no está en PyPI. En ese caso, necesitas instalar el paquete de una manera diferente. En la instalación estándar, después de descomprimir el archivo que descargaste, normalmente puedes ver que la carpeta contiene un script de configuración `setup.py` y un archivo llamado `README`, que documenta cómo construir e instalar el módulo. Para la mayoría de los casos, solo necesitas ejecutar un comando desde la terminal para instalar el paquete:

```
1 python setup.py install
```

Listing 22: Instalando desde el código fuente.

Observa que los usuarios de Windows necesitarán ejecutar el siguiente comando desde una ventana de prompt de comandos:

```
1 setup.py install
```

Ahora ya sabes cómo gestionar los paquetes en Python, lo cual es un gran paso adelante para usar Python correctamente. En la próxima sección, hablaremos más sobre el Jupyter notebook que usamos para el resto del libro.

Introducción a Jupyter Notebook

Ya has usado el shell de IPython para ejecutar código línea por línea. ¿Qué pasa si tienes más líneas y quieres ejecutarlas bloque por bloque, y compartirlas fácilmente con otros? En este caso, el shell de IPython no es una buena opción. Esta sección te presentará otra opción, Jupyter notebook, que usaremos para el resto del libro. Desde el sitio web de Jupyter notebook⁷:

El Jupyter notebook es una aplicación web de código abierto que te permite crear y compartir documentos que contienen código en vivo, ecuaciones, visualizaciones y texto narrativo. Los usos incluyen: limpieza y transformación de datos, simulación numérica, modelado estadístico, visualización de datos, aprendizaje automático y mucho más.

Jupyter notebook se ejecuta usando tu navegador; puede ejecutarse localmente en tu máquina como un servidor local o de forma remota en un

⁷<http://jupyter.org/>

servidor. La razón por la que se llama notebook es porque puede contener código en vivo, elementos de texto enriquecido como ecuaciones, enlaces, imágenes, tablas, etc. Por lo tanto, puedes tener un notebook muy agradable para describir tu idea y el código en vivo todo en un documento. Así, Jupyter notebook se ha convertido en una forma popular de probar ideas, escribir blogs, artículos e incluso libros. De hecho, este libro fue escrito enteramente dentro de Jupyter notebook y convertido a L^AT_EX posteriormente. Aunque Jupyter notebook tiene muchas otras ventajas, para empezar, solo cubriremos lo básico aquí.

Iniciando Jupyter Notebook

Como se vio anteriormente, podemos iniciar Jupyter notebook escribiendo el siguiente comando en nuestra terminal (para usuarios de Windows, escriban este comando en el prompt de Anaconda) en la carpeta donde queremos ubicar los notebooks:

```
1 jupyter notebook
```

Listing 23: Iniciando Jupyter Notebook.

El panel de control (dashboard) de Jupyter notebook aparecerá en el navegador como se muestra en la Fig. 15. La dirección predeterminada es: <http://localhost:8888>, que está en `localhost` con el puerto 8888, como se muestra en la Fig. 15 (si el puerto 8888 está ocupado por otros Jupyter notebooks, entonces usará automáticamente otro puerto). Esto es esencialmente crear un servidor local para ejecutarlo en tu navegador. Cuando navegues al navegador, verás un panel de control. En este panel, verás algunas características importantes etiquetadas en rojo (gris claro en la versión impresa). Para crear un nuevo notebook de Python, selecciona "Python 3", que generalmente se llama kernel de Python. Puedes usar Jupyter para ejecutar otros kernels también. Por ejemplo, en la Fig. 15, hay kernels de Bash y Julia que puedes ejecutar como notebook, pero necesitarás instalarlos primero. Usaremos el kernel de Python, así que elige el kernel Python 3.

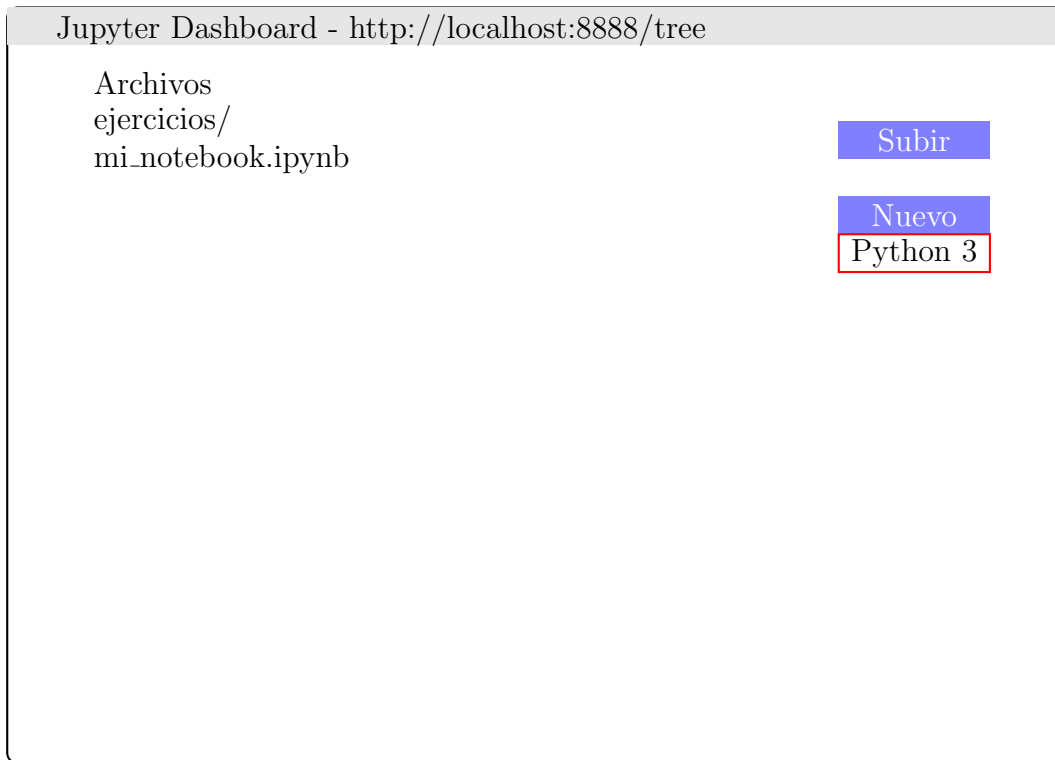


Figura 15: El panel de control de Jupyter notebook después de lanzar el servidor. Las flechas rojas (flechas gris claro en la versión impresa) te señalan las características más comunes en el panel de control.

Dentro del Notebook

Después de crear un nuevo notebook de Python, se verá como el que se muestra en la Fig. 16. La barra de herramientas y el menú se explican por sí mismos. Si pasas el cursor sobre la barra de herramientas, te mostrará la función de la herramienta. Cuando presiones el menú, te mostrará la lista desplegable. Algunas cosas importantes que debes saber sobre Jupyter notebook son las siguientes: una **celda** es el lugar donde puedes escribir tu código o texto. Si ejecutas esta celda, solo ejecuta el código dentro de este bloque de celda. Dos tipos de celda importantes son **code** (código) y **markdown** (rebaja): (a) la celda de código es donde escribir tu código y donde ejecutar el código; (b) la celda de markdown es donde escribir la descripción en formato de texto enriquecido (ver la Fig. 16 como ejemplo). Busca la "hoja de trucos

de Markdown” para empezar rápidamente con markdown. Para ejecutar el código o renderizar el markdown en el notebook es simple: presiona **Shift + Enter**.

El notebook te permite mover la celda hacia arriba o hacia abajo, insertar o eliminar la celda, etc. Hay muchas otras características interesantes disponibles usando Jupyter notebook, y te animamos a acceder a los muchos tutoriales en línea disponibles para ampliar tu conocimiento.

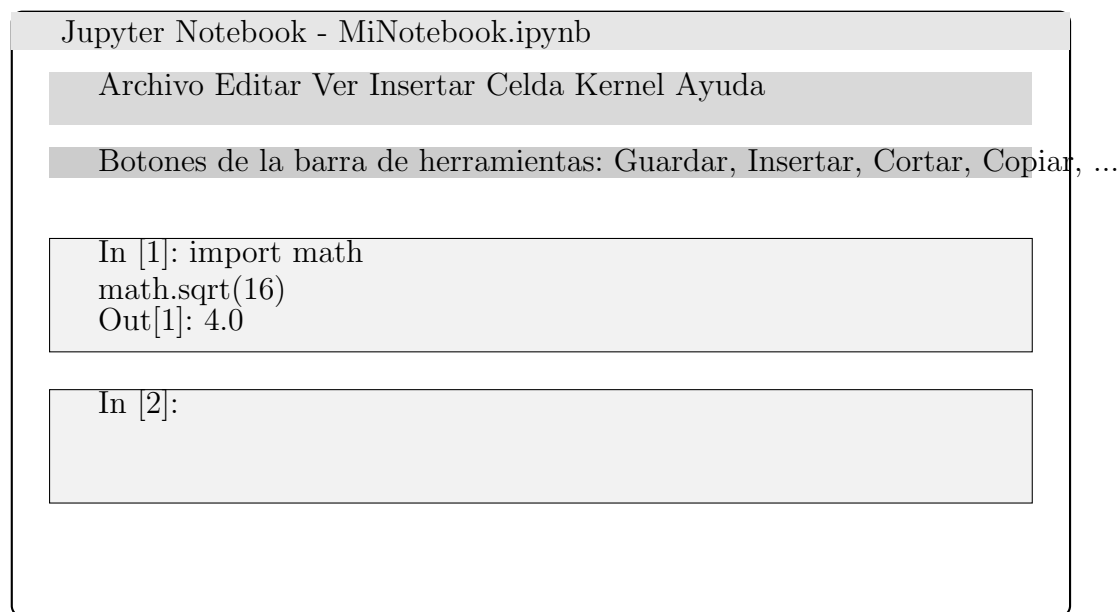


Figura 16: Una vista rápida de un notebook. El encabezado del notebook muestra el nombre del notebook. El menú tiene varias listas desplegables que te permiten acceder a todas las funcionalidades del notebook. La barra de herramientas te proporciona algunos atajos para las funcionalidades de uso común.

¿Cómo cierro un Notebook?

Cuando cierras la pestaña del navegador del notebook, el notebook en realidad no se cierra; todavía se está ejecutando en segundo plano. Para cerrar completamente un notebook, ve al panel de control (dashboard) y marca la casilla antes del notebook. Hay una opción de apagado (shutdown) en la barra de herramientas superior; esta es la forma correcta de cerrar un

notebook por completo.

Apagando el servidor de Jupyter Notebook

Cerrar el navegador no apagará el servidor del notebook, ya que todavía se está ejecutando. Puedes volver a abrir la dirección anterior en un navegador. Para apagarlo completamente, necesitamos cerrar la terminal asociada que usaste para lanzar el notebook.

Ahora que tienes el conocimiento básico para lanzar y ejecutar un Jupyter notebook, estás equipado para continuar aprendiendo Python.

Expresiones lógicas y operadores

Una **expresión lógica** es una declaración que puede ser verdadera o falsa. Por ejemplo, $a < b$ es una expresión lógica. Puede ser verdadera o falsa dependiendo de los valores que se den a a y b . Observa que esto difiere de una **expresión matemática**, que denota una declaración de verdad. En el ejemplo anterior, la expresión matemática $a < b$ significa que a es menor que b , y los valores de a y b donde $a \geq b$ no están permitidos. Las expresiones lógicas forman la base de la computación. En este libro se asume que todas las declaraciones son lógicas en lugar de matemáticas, a menos que se indique lo contrario.

En Python, una expresión lógica que es verdadera se evaluará como el valor `True`. Una expresión falsa se evaluará como el valor `False`. Este es un nuevo tipo de dato conocido como **booleano**, que tiene los valores incorporados `True` y `False`. En este libro, "True" es equivalente a 1, y "False" es equivalente a 0. Las expresiones lógicas se usan para plantear preguntas a Python. Por ejemplo, " $3 < 4$ " es equivalente a "¿Es 3 menor que 4?" Dado que esta declaración es verdadera, Python la evaluará como 1; sin embargo, si escribimos $3 > 4$, esto es falso, y Python lo evaluará como 0.

Los **operadores de comparación** comparan el valor de dos números, que se utilizan para construir expresiones lógicas. Python reserva los símbolos `>`, `>=`, `<`, `<=`, `!=`, `==` para denotar "mayor que", "mayor o igual que", "menor que", "menor o igual que", "diferente de", "igual a", respectivamente; ver la Tabla 1. Comencemos con un ejemplo, $a = 4$, $b = 2$:

Cuadro 1: Operadores de comparación.

Operador	Descripción	Ejemplo	Resultados
>	mayor que	a > b	True
>=	mayor o igual que	a >= b	True
<	menor que	a < b	False
<=	menor o igual que	a <= b	False
!=	diferente de	a != b	True
==	igual a	a == b	False

```

1 In [1]: 5 == 4
2 Out[1]: False
3
4 In [2]: 2 < 3
5 Out[2]: True

```

Listing 24: Calculando las expresiones lógicas "¿Es 5 igual a 4?" "¿Es 2 menor que 3?"

Los **operadores lógicos**, como se muestra en la Tabla 2, son operaciones entre dos expresiones lógicas que, para facilitar la discusión, llamaremos P y Q . Los operadores lógicos fundamentales que usaremos aquí son **and**, **or** y **not**.

Cuadro 2: Operadores lógicos.

Operador	Descripción	Resultados
and	P and Q	Verdadero si tanto P como Q son Verdaderos. Falso en caso contrario.
or	P or Q	Verdadero si P o Q es Verdadero. Falso en caso contrario.
not	not P	Verdadero si P es Falso. Falso si P es Verdadero.

La **tabla de verdad**, como se muestra en la Fig. 17, de un operador o expresión lógica da el resultado de cada combinación de verdad de P y Q . La Fig. 17 muestra las tablas de verdad para **and** y **or**.

P	Q	P and Q	P	Q	P or Q
T	T	T	T	T	T
T	F	F	T	F	T
F	T	F	F	T	T
F	F	F	F	F	F

Figura 17: Tablas de verdad para los operadores lógicos and/or.

```

1 In [3]: (1 and not 1) or (1 and 1)
2 Out[3]: 1
3
4 In [4]: (1 and not 0) or (1 and 0)
5 Out[4]: True

```

Listing 25: Probando una expresión lógica compleja.

Al igual que con los operadores aritméticos, los operadores lógicos tienen un orden de operaciones entre sí y en relación con los operadores aritméticos. Todas las operaciones aritméticas se ejecutarán antes que las operaciones de comparación, que se ejecutarán antes que las operaciones lógicas. Los paréntesis se pueden usar para cambiar el orden de las operaciones.

```

1 In [5]: 1 + 3 > 2 + 5
2 Out[5]: False

```

Listing 26: Calculando $(1 + 3) > (2 + 5)$.

Consejo

Incluso cuando se conoce el orden de las operaciones, suele ser útil para ti y para quienes lean tu código usar paréntesis para hacer más claras tus intenciones. En el ejemplo anterior, $(1 + 3) > (2 + 5)$ es más claro.

Advertencia

En la implementación de la lógica en Python, se usa 1 para denotar verdadero y 0 para falso. Pero como 1 y 0 siguen siendo números, Python permitirá abusos como: $(3 > 2) + (5 > 4)$, que se resolverá como 2.

```
1 In [6]: (3 > 2) + (5 > 4)
2 Out[6]: 2
```

Listing 27: Un “abuso” de los valores booleanos.

Advertencia

Aunque en la lógica formal se usa 1 para denotar verdadero y 0 para denotar falso, el sistema de notación de Python es diferente, y tomará cualquier número distinto de 0 como verdadero cuando se use en una operación lógica. Por ejemplo, `3 and 1` se evaluará como verdadero. No utilices esta característica de Python. Usa siempre 1 para denotar una declaración verdadera.

Consejo

Una quincena es un periodo de tiempo que consta de 14 días. Usa una expresión lógica para determinar si hay más de 100,000 segundos en una quincena.

```
1 In [7]: (14*24*60*60) > 100000
2 Out[7]: True
```

Listing 28: Comprobando los segundos en una quincena.

Resumen y Problemas

Resumen

1. Ahora has aprendido los fundamentos de Python, lo que debería permitirte configurar el entorno de trabajo y experimentar con formas de ejecutar Python.

2. Python puede usarse como una calculadora. Tiene todas las funciones y operaciones aritméticas comúnmente utilizadas con una calculadora científica.
3. Puedes gestionar los paquetes de Python utilizando gestores de paquetes.
4. Aprendiste cómo interactuar con Jupyter notebook.
5. También puedes usar Python para realizar operaciones lógicas.
6. Ahora se te han presentado los tipos de datos `int`, `float`, `complex`, `string` y `boolean` en Python.

Problemas y Soluciones

A continuación, se presentan las soluciones paso a paso para los problemas del capítulo. Se asume que ya tienes Python y Jupyter Notebook instalados y configurados.

1. **Problema 1:** Imprime "I love Python" usando el shell de Python.

Solución: Abre una terminal y escribe `python` para iniciar el shell de Python. Luego, escribe el comando `print("I love Python")` y presiona Enter.

```
1 >>> print("I love Python")
2 I love Python
3
```

Listing 29: Solución Problema 1 - Shell de Python

2. **Problema 2:** Imprime "I love Python" escribiéndolo en un archivo `.py` y ejecútalo desde la línea de comandos.

Solución: Paso 1: Crea un archivo llamado, por ejemplo, `amor.py` con un editor de texto y escribe dentro: `print("I love Python")`. **Paso 2:** Guarda el archivo. Abre una terminal, navega hasta el directorio donde guardaste el archivo y ejecuta `python amor.py`.

```
1 $ python amor.py
2 I love Python
3
```

Listing 30: Solución Problema 2 - Línea de comandos

3. **Problema 3:** Escribe `import antigavity` en el shell de IPython.

Solución: Abre una terminal, escribe `ipython` y luego `import antigavity`. Esto abrirá una página web de xkcd en tu navegador predeterminado. Es un "huevo de pascua" de Python.

```
1 In [1]: import antigavity
2
```

Listing 31: Solución Problema 3 - IPython

(La salida es que se abre una página web.)

4. **Problema 4:** Lanza un nuevo servidor de Jupyter Notebook en una carpeta llamada `.exercisez` crea un nuevo notebook de Python con el nombre `.exercise_1`. Pon el resto de los problemas dentro de este notebook.

Solución: Paso 1: Abre una terminal. Navega hasta el directorio donde quieres crear la carpeta `.exercise`. Crea la carpeta con `mkdir exercise` y luego entra en ella con `cd exercise`. **Paso 2:** Lanza el servidor con `jupyter notebook`. Se abrirá el dashboard en tu navegador. **Paso 3:** Haz clic en "Nuevo» "Python 3". Aparecerá un nuevo notebook. En la parte superior, junto al título "Untitled", haz clic y renómbralo a `.exercise_1`. Ahora puedes resolver los problemas en las celdas de este notebook.

Las soluciones de los problemas 5 al 24 se pueden escribir en las celdas de código de `exercise_1.ipynb`.

5. **Problema 5:** Calcula el área de un triángulo con base 10 y altura 12. El área de un triángulo es la mitad de la base por la altura.

Solución: Definimos las variables `base` y `altura`, y aplicamos la fórmula.

```
1 base = 10
2 altura = 12
3 area = (base * altura) / 2
4 print(f"El area del triangulo es: {area}")
5 # Salida: El area del triangulo es: 60.0
6
```

Listing 32: Solución Problema 5

6. **Problema 6:** Calcula el área superficial y el volumen de un cilindro con radio 5 y altura 3.

Solución: Las fórmulas son: Área superficial $A = 2\pi r^2 + 2\pi rh$, Volumen $V = \pi r^2 h$. Usamos `math.pi`.

```
1 import math
2 radio = 5
3 altura = 3
4 area_superficial = 2 * math.pi * radio**2 + 2 * math.pi *
   radio * altura
5 volumen = math.pi * radio**2 * altura
6 print(f"Area superficial: {area_superficial:.2f}")
7 print(f"Volumen: {volumen:.2f}")
8 # Salida: Area superficial: 251.33, Volumen: 235.62
9
```

Listing 33: Solución Problema 6

7. **Problema 7:** Calcula la pendiente entre los puntos (3, 4) y (5, 9). La pendiente entre puntos (x_1, y_1) y (x_2, y_2) es $\frac{y_2 - y_1}{x_2 - x_1}$.

Solución: Asignamos los puntos a variables y aplicamos la fórmula.

```
1 x1, y1 = 3, 4
2 x2, y2 = 5, 9
3 pendiente = (y2 - y1) / (x2 - x1)
4 print(f"La pendiente es: {pendiente}")
5 # Salida: La pendiente es: 2.5
6
```

Listing 34: Solución Problema 7

8. **Problema 8:** Calcula la distancia entre los puntos (3, 4) y (5, 9). La distancia en dos dimensiones es $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.

Solución: Usamos la función `math.sqrt` para la raíz cuadrada.

```
1 import math
2 x1, y1 = 3, 4
3 x2, y2 = 5, 9
4 distancia = math.sqrt((x2 - x1)**2 + (y2 - y1)**2)
5 print(f"La distancia es: {distancia:.4f}")
6 # Salida: La distancia es: 5.3852
7
```

Listing 35: Solución Problema 8

9. **Problema 9:** Usa la función factorial de Python para calcular 6!.

Solución: La función factorial está en el módulo `math`.

```
1 import math
2 factorial_6 = math.factorial(6)
3 print(f"6! = {factorial_6}")
4 # Salida: 6! = 720
5
```

Listing 36: Solución Problema 9

10. **Problema 10:** Calcula el número de años bisiestos entre los años 1500 y 2010.

Solución: Un año es bisiesto si es divisible por 4, excepto si es divisible por 100, a menos que también sea divisible por 400. Iteramos sobre los años y contamos los que cumplen la condición.

```
1 contador_bisiestos = 0
2 for año in range(1500, 2011): # 2010 inclusive
3     if (año % 4 == 0):
4         if (año % 100 == 0):
5             if (año % 400 == 0):
6                 contador_bisiestos += 1
```

```

7         else:
8             contador_bisiestos += 1
9     print(f"Numero de años bisiestos entre 1500 y 2010: {
10         contador_bisiestos}")
11 # Salida: Numero de años bisiestos entre 1500 y 2010: 124

```

Listing 37: Solución Problema 10

11. **Problema 11:** Aproxima π usando la fórmula de Ramanujan para $N = 0$ y $N = 1$. Compara con el valor de Python.

Solución: La fórmula es $\frac{1}{\pi} \approx \frac{2\sqrt{2}}{9801} \sum_{k=0}^N \frac{(4k)!(1103+26390k)}{(k!)^4 396^{4k}}$.
Calculamos la suma y luego invertimos.

```

1 import math
2
3 def ramanujan_pi(N):
4     suma = 0
5     for k in range(N+1):
6         numerador = math.factorial(4*k) * (1103 + 26390*k)
7         denominador = (math.factorial(k)**4) * (396**(4*k))
8         suma += numerador / denominador
9     factor = (2 * math.sqrt(2)) / 9801
10    pi_aprox = 1 / (factor * suma)
11    return pi_aprox
12
13 pi_aprox_N0 = ramanujan_pi(0)
14 pi_aprox_N1 = ramanujan_pi(1)
15 print(f"Aproximacion de pi (N=0): {pi_aprox_N0}")
16 print(f"Aproximacion de pi (N=1): {pi_aprox_N1}")
17 print(f"Valor de math.pi: {math.pi}")
18 # Salida: Aproximacion de pi (N=0): 3.1415927300133055
19 # Salida: Aproximacion de pi (N=1): 3.141592653589793
20 # Salida: Valor de math.pi: 3.141592653589793
21

```

Listing 38: Solución Problema 11

12. **Problema 12:** Calcula $\sinh(2)$ usando exponenciales y verifica con `math.sinh`.

Solución: $\sinh(x) = \frac{e^x - e^{-x}}{2}$.

```

1 import math
2 x = 2
3 sinh_exp = (math.exp(x) - math.exp(-x)) / 2
4 sinh_math = math.sinh(x)
5 print(f"sinh(2) calculado: {sinh_exp}")
6 print(f"sinh(2) de math.sinh: {sinh_math}")
7 # Salida: sinh(2) calculado: 3.6268604078470186
8 # Salida: sinh(2) de math.sinh: 3.626860407847019
9

```

Listing 39: Solución Problema 12

13. **Problema 13:** Verifica $\sin^2(x) + \cos^2(x) = 1$ para $x = \pi, \frac{\pi}{2}, \frac{\pi}{4}, \frac{\pi}{6}$.

Solución: Calculamos cada lado de la identidad.

```

1 import math
2 valores_x = [math.pi, math.pi/2, math.pi/4, math.pi/6]
3 for x in valores_x:
4     resultado = math.sin(x)**2 + math.cos(x)**2
5     print(f"Para x = {x:.4f}, sin^2+cos^2 = {resultado}")
6 # Salida: Para x = 3.1416, sin^2+cos^2 = 1.0
7 # Para x = 1.5708, sin^2+cos^2 = 1.0
8 # Para x = 0.7854, sin^2+cos^2 = 1.0
9 # Para x = 0.5236, sin^2+cos^2 = 1.0
10

```

Listing 40: Solución Problema 13

14. **Problema 14:** Calcula el $\sin 87^\circ$.

Solución: Las funciones trigonométricas de Python usan radianes. Convertimos grados a radianes ($\text{rad} = \text{grad} \times \pi/180$).

```

1 import math
2 grados = 87
3 radianes = math.radians(grados)
4 seno_87 = math.sin(radianes)
5 print(f"sin(87 ) = {seno_87}")
6 # Salida: sin(87 ) = 0.9986295347545738
7

```

Listing 41: Solución Problema 14

15. **Problema 15:** Genera el error `.AttributeError: module 'math' has no attribute 'sni'.`

Solución: Intentamos acceder a una función que no existe, como `math.sni`.

```
1 import math
2 math.sni
3 # Salida: AttributeError: module 'math' has no attribute
  'sni'
4
```

Listing 42: Solución Problema 15

16. **Problema 16:** Genera el error `"TypeError: sin() takes exactly one argument (0 given)."`

Solución: Llamamos a `math.sin` sin ningún argumento.

```
1 import math
2 math.sin()
3 # Salida: TypeError: sin() takes exactly one argument (0
  given)
4
```

Listing 43: Solución Problema 16

17. **Problema 17:** Verifica la ley de no contradicción: `P AND (NOT P)` es siempre falso.

Solución: Probamos con `P` verdadero (1) y `P` falso (0). Usamos la equivalencia de Python (`True=1`, `False=0`) o los valores booleanos.

```
1 P_true = 1
2 P_false = 0
3 print(f"P=1: {P_true and not P_true}")
4 print(f"P=0: {P_false and not P_false}")
5 # Salida: P=1: 0, P=0: 0
6
```

Listing 44: Solución Problema 17

18. **Problema 18:** Verifica las leyes de De Morgan.

Solución: Creamos una tabla de verdad para todas las combinaciones de P y Q .

```

1 print("P Q | NOT(P OR Q) | (NOT P) AND (NOT Q)")
2 for P in [0, 1]:
3     for Q in [0, 1]:
4         izquierda = not (P or Q)
5         derecha = (not P) and (not Q)
6         print(f"{P} {Q} | {izquierda} | {derecha}")
7 # Salida: P Q | NOT(P OR Q) | (NOT P) AND (NOT Q)
8 # 0 0 | 1 | 1
9 # 0 1 | 0 | 0
10 # 1 0 | 0 | 0
11 # 1 1 | 0 | 0
12

```

Listing 45: Solución Problema 18

La segunda ley se verifica de manera análoga.

19. **Problema 19:** ¿Bajo qué condiciones para P y Q es $(P \wedge Q) \vee (P \wedge \neg Q)$ falso?

Solución: La expresión se simplifica a $P \wedge (Q \vee \neg Q) = P \wedge \text{True} = P$. Por lo tanto, la expresión es falsa solo cuando P es falso (0), independientemente de Q .

```

1 print("P Q | Resultado")
2 for P in [0, 1]:
3     for Q in [0, 1]:
4         resultado = (P and Q) or (P and (not Q))
5         print(f"{P} {Q} | {resultado}")
6 # Salida: P Q | Resultado
7 # 0 0 | 0
8 # 0 1 | 0
9 # 1 0 | 1
10 # 1 1 | 1
11

```

Listing 46: Solución Problema 19

20. **Problema 20:** Construye una expresión equivalente para OR usando solo AND y NOT.

Solución: Usando la ley de De Morgan, $P \vee Q = \neg(\neg P \wedge \neg Q)$.

```

1 P = 1
2 Q = 0
3 or_equivalente = not ( (not P) and (not Q) )
4 print(f"OR original: {P or Q}, Equivalente: {
    or_equivalente}")
5 # Salida: OR original: 1, Equivalente: 1
6

```

Listing 47: Solución Problema 20

21. **Problema 21:** Construye una expresión equivalente para AND usando solo OR y NOT.

Solución: Usando la ley de De Morgan, $P \wedge Q = \neg(\neg P \vee \neg Q)$.

```

1 P = 1
2 Q = 0
3 and_equivalente = not ( (not P) or (not Q) )
4 print(f"AND original: {P and Q}, Equivalente: {
    and_equivalente}")
5 # Salida: AND original: 0, Equivalente: 0
6

```

Listing 48: Solución Problema 21

22. **Problema 22:** Construye una expresión para XOR usando AND, OR y NOT.

Solución: La expresión común es $(P \wedge \neg Q) \vee (\neg P \wedge Q)$.

```

1 print("P Q | XOR Equivalente")
2 for P in [0, 1]:
3     for Q in [0, 1]:
4         xor = (P and (not Q)) or ((not P) and Q)
5         print(f"{P} {Q} | {xor}")

```

```

6 # Salida: P Q | XOR Equivalente
7 # 0 0 | 0
8 # 0 1 | 1
9 # 1 0 | 1
10 # 1 1 | 0
11

```

Listing 49: Solución Problema 22

(La tabla de verdad generada coincide con la Fig. 1.18.)

23. **Problema 23:** Realiza el cálculo: $e^2 \sin(\pi/6) + \log_e(3) \cos(\pi/9) - 5^3$.

Solución: Usamos `math.exp`, `math.sin`, `math.log`, `math.cos`.

```

1 import math
2 resultado = math.exp(2) * math.sin(math.pi/6) + math.log
   (3) * math.cos(math.pi/9) - 5**3
3 print(f"El resultado es: {resultado}")
4 # Salida: El resultado es: -115.41119679087638
5

```

Listing 50: Solución Problema 23

24. **Problema 24:** Realiza operaciones lógicas. Para $P = 1, Q = 1$, computa $\neg(P) \wedge \neg(Q)$. Para $a = 10, b = 25$, computa $(a < b) \wedge (a = b)$.

Solución: Se evalúan las expresiones directamente.

```

1 P = 1
2 Q = 1
3 resultado1 = (not P) and (not Q)
4 print(f"NOT(P) AND NOT(Q) con P=1,Q=1: {resultado1}")
5
6 a = 10
7 b = 25
8 resultado2 = (a < b) and (a == b)
9 print(f"(a<b) and (a==b) con a=10,b=25: {resultado2}")
10 # Salida: NOT(P) AND NOT(Q) con P=1,Q=1: 0
11 # (a<b) and (a==b) con a=10,b=25: False
12

```

Listing 51: Solución Problema 24

VARIABLES Y ESTRUCTURAS DE DATOS BÁSICAS

Variables y Asignación

Cuando se programa, es útil poder almacenar información en **variables**. Una variable es una cadena de caracteres y números asociada con una pieza de información. El **operador de asignación**, denotado por el símbolo "=", es el operador que se utiliza para asignar valores a variables en Python. La línea `x=1` toma el valor conocido, 1, y asigna ese valor a la variable con nombre "x". Después de ejecutar esta línea, este número se almacenará en esta variable. Hasta que el valor se cambie o la variable se elimine, el carácter `x` se comporta como el valor 1.

```
1 In [1]: x = 1
2         x
3 Out [1]: 1
```

Listing 52: Asignando un valor a una variable.

```
1 In [2]: y = 2
2         y
3 Out [2]: 2
4
5 In [3]: y*3
6 Out [3]: 6
```

Listing 53: Asignando el valor 2 a la variable `y` y multiplicándolo por 3.

Una variable es como un "contenedor" que se utiliza para almacenar los datos en la memoria de la computadora. El nombre de la variable le dice

a la computadora dónde encontrar este valor en la memoria. Por ahora, es suficiente saber que el notebook tiene su propio espacio de memoria para almacenar todas las variables en el notebook. Como resultado del ejemplo anterior, verás las variables `x` e `y` en la memoria. Puedes ver una lista de todas las variables en el notebook usando el comando mágico `%whos` (los comandos mágicos son un conjunto especializado de comandos alojados por el kernel de IPython y requieren el prefijo `%` para especificar los comandos).

```

1 In [4]: %whos
2 Variable      Type      Data/Info
3 -----
4 x             int       1
5 y             int       2

```

Listing 54: Listando todas las variables en el notebook.

Nota

El signo igual en programación no es lo mismo que una declaración de verdad en matemáticas. En matemáticas, la declaración `x = 2` declara la verdad universal dentro del marco dado, `x` es 2. En programación, la declaración `x=2` significa que se está asociando un valor conocido con un nombre de variable, almacena 2 en `x`. Aunque es perfectamente válido decir `1 = x` en matemáticas, las asignaciones en Python siempre van hacia la izquierda, lo que significa que el valor a la derecha del signo igual se asigna a la variable a la izquierda del signo igual. Por lo tanto, `1=x` generará un error en Python. El operador de asignación es siempre el último en el orden de las operaciones en relación con los operadores matemáticos, lógicos y de comparación.

```

1 In [5]: x = x + 1
2         x
3 Out [5]: 2

```

Listing 55: Reasignando una variable.

Hay algunas restricciones sobre los nombres que pueden tomar las variables. Las variables solo pueden contener caracteres alfanuméricos (letras y números) así como guiones bajos; sin embargo, el primer carácter de un nombre de variable debe ser una letra o un guión bajo. No se permiten espacios dentro de un nombre de variable, y los nombres de las variables distinguen

entre mayúsculas y minúsculas (por ejemplo, `x` y `X` se consideran variables diferentes).

Consejo

A diferencia de las matemáticas puras, las variables en programación casi siempre representan algo tangible. Puede ser la distancia entre dos puntos en el espacio o el número de conejos en una población. Por lo tanto, a medida que tu código se vuelve más complicado, es muy importante que tus variables tengan un nombre que pueda asociarse fácilmente con lo que representan. Por ejemplo, la distancia entre dos puntos en el espacio está mejor representada por la variable `dist` que por `x`, y el número de conejos en una población está mejor representado por `n_conejos` que por `y`.

Ten en cuenta que cuando se asigna una variable, no tiene memoria de cómo se asignó. Es decir, si el valor de una variable, `y`, se construye a partir de otras variables, como `x`, reasignar el valor de `x` no cambiará el valor de `y`.

```
1 In [7]: x = 1
2         y = x + 1
3         x = 2
4         y
5 Out [7]: 2
```

Listing 56: Demostrando que las variables no recuerdan su origen.

Advertencia

Puedes sobrescribir variables o funciones que se han almacenado en Python. Por ejemplo, el comando `help = 2` almacenará el valor 2 en la variable con nombre `help`. Después de esta asignación, `help` se comportará como el valor 2 en lugar de la función `help`. Por lo tanto, siempre debes tener cuidado de no dar a tus variables el mismo nombre que las funciones o valores incorporados.

Consejo

Ahora que sabes cómo asignar variables, es importante que recuerdes nunca dejar comandos sin asignar. Un comando sin asignar es una operación que tiene un resultado, pero ese resultado no se asigna a una variable. Por ejemplo, nunca debes usar `2+2`. En su lugar, debes asignarlo a alguna variable `x=2+2`. Esto te permite conservar los resultados de comandos anteriores y hará que tu interacción con Python sea mucho menos confusa.

Puedes eliminar una variable del notebook usando la función `del`. Escribir `del x` eliminará la variable `x` del espacio de trabajo. Si deseas eliminar todas las variables en el notebook, puedes usar el comando mágico `%reset`.

En matemáticas, las variables suelen asociarse con números desconocidos; en programación, las variables se asocian con un valor de un cierto tipo. Hay muchos **tipos de datos** que se pueden asignar a las variables. Un tipo de dato es una clasificación del tipo de información que se está almacenando en una variable. Los tipos de datos básicos que utilizarás a lo largo de este libro son `boolean`, `int`, `float`, `string`, `list`, `tuple`, `dictionary` y `set`. En las siguientes secciones se ofrece una descripción formal de estos tipos de datos.

Estructura de Datos – String (Cadena)

Hemos introducido los diferentes tipos de datos, como `int`, `float` y `boolean`; todos estos están relacionados con un solo valor. El resto de este capítulo te presentará más tipos de datos para que podamos almacenar múltiples valores. Las estructuras de datos relacionadas con estos nuevos tipos son *strings* (cadenas), *lists* (listas), *tuples* (tuplas), *sets* (conjuntos) y *dictionaries* (diccionarios). Comenzaremos con las cadenas.

Una **string** (cadena) es una secuencia de caracteres, como "Hello World" que vimos en el Capítulo 1. Las cadenas están rodeadas por comillas simples o dobles. Podemos usar la función `print` para mostrar las cadenas en la pantalla.

```
1 In [1]: print("I love Python!")
```

Listing 57: Imprimiendo una cadena.

```
1 In [2]: s = "S"
2         w = "Hello World"
```

```

3
4 In [3]: type(s)
5 Out[3]: str
6
7 In [4]: type(w)
8 Out[4]: str

```

Listing 58: Asignando cadenas y verificando su tipo.

Nota

Un espacio en blanco, , entre "Hello" "World" también es de tipo `str`. Cualquier símbolo puede ser un carácter, incluso aquellos que han sido reservados para operadores. Ten en cuenta que como `str`, no realizan la misma función. Aunque se ven iguales, Python los interpreta de manera completamente diferente.

```

1 In [5]: s = " "
2         type(s)
3 Out[5]: str

```

Listing 59: Creando una cadena vacía.

Debido a que una cadena es un arreglo de caracteres, tiene una *length* (longitud) para indicar el tamaño de la cadena. Por ejemplo, podemos verificar el tamaño de la cadena usando la función incorporada `len`.

```

1 In [6]: len(w)
2 Out[6]: 11

```

Listing 60: Longitud de una cadena.

Las cadenas también tienen **índices** que nos permiten encontrar la ubicación de cada carácter, como se muestra en la Fig. 18. El índice de la posición comienza con 0.

H	e	l	l	o		W	o	r	l	d
0	1	2	3	4	5	6	7	8	9	10

Figura 18: Índices de cadena para el ejemplo "Hello World".

Podemos acceder a un carácter usando corchetes y el índice de la posición. Por ejemplo, si queremos acceder al carácter "W", escribimos lo siguiente:


```
1 In [7]: w[6]
2 Out[7]: "W"
```

Listing 61: Accediendo a un carácter por índice.

También podemos seleccionar una secuencia usando el **slicing** (segmentación) de cadenas. Por ejemplo, si queremos acceder a "World", escribimos el siguiente comando.

```
1 In [8]: w[6:11]
2 Out[8]: "World"
```

Listing 62: Segmentando una cadena.

[6:11] significa que la posición de inicio es el índice 6 y la posición final es el índice 10. En el rango de segmentación de cadenas de Python, el límite superior es exclusivo; esto significa que [6:11] "segmentará" los caracteres del índice 6 al 10. La sintaxis para el segmentado en Python es [inicio:fin:paso], el tercer argumento, **paso**, es opcional. Si se ignora el argumento **paso**, el valor predeterminado será 1.

Puedes ignorar la posición final si deseas segmentar hasta el final de la cadena. Por ejemplo, el siguiente comando es el mismo que el anterior:

```
1 In [9]: w[6:]
2 Out[9]: "World"
```

Listing 63: Segmentando hasta el final.

```
1 In [10]: w[:5]
2 Out[10]: "Hello"
```

Listing 64: Recuperando la palabra "Hello".

También puedes usar un índice negativo al segmentar cadenas, lo que significa contar desde el final de la cadena. Por ejemplo, -1 significa el último carácter, -2 significa el penúltimo, y así sucesivamente.

```
1 In [11]: w[6:-2]
2 Out[11]: "Wor"
```

Listing 65: Usando índices negativos.

```
1 In [12]: w[::2]
2 Out[12]: "HlWrd"
```

Listing 66: Recuperando cada dos caracteres.

Las cadenas no se pueden usar en las operaciones matemáticas.

```
1 In [13]: 1 "+" 2
2     File "<ipython-input-13-46b54f731e00>", line 1
3         1 "+" 2
4
5 SyntaxError: invalid syntax
```

Listing 67: Intentando "sumar" cadenas incorrectamente.

Advertencia

Los números también se pueden expresar como `str`. Por ejemplo, `x = '123'` significa que `x` es la cadena "123", no el número 123. Sin embargo, las cadenas representan palabras o texto, por lo que no se debe definir la suma en ellas.

Consejo

Puede que te encuentres en una situación en la que desees usar un apóstrofe dentro de una `str`. Esto es problemático ya que un apóstrofe se usa para denotar cadenas. Afortunadamente, se puede usar un apóstrofe en una cadena de la siguiente manera. La barra invertida (`\`) es una forma de decirle a Python que esto es parte de la cadena, no que denota cadenas. El carácter de barra invertida se usa para escapar caracteres que de otro modo tienen un significado especial, como nueva línea, la propia barra invertida o la comilla. Si una comilla simple o doble es parte de la cadena en sí, en Python hay una manera fácil de hacerlo: simplemente puedes colocar la cadena entre comillas dobles o simples respectivamente, como se muestra en el siguiente ejemplo.

```
1 In [14]: "don't"
2 Out[14]: "don't"
```

Listing 68: Manejando apóstrofes en cadenas.

Una cadena se puede **concatenar** con otra cadena. Por ejemplo:

```
1 In [15]: str_a = "I love Python! "
2         str_b = "You too!"
3         print(str_a + str_b)
4 I love Python! You too!
```

Listing 69: Concatenando cadenas.

Podemos convertir otros tipos de datos a cadenas también usando la función incorporada `str`. Esto es útil, por ejemplo, si tenemos la variable `x` que ha almacenado 1 como un tipo entero, y queremos imprimirlo directamente como una cadena, obtendremos un error que dice que no podemos concatenar una cadena con un entero.

```

1 In [16]: x = 1
2         print("x = " + x)
3 -----
4 TypeError Traceback (most recent call last)
5 <ipython-input-16-3e562ba0dd83> in <module>()
6     1 x = 1
7 ----> 2 print("x = " + x)
8 TypeError: can only concatenate str (not "int") to str

```

Listing 70: Error al concatenar tipos diferentes.

La forma correcta de hacerlo es convertir el entero a cadena primero y luego imprimirlo.

```

1 In [17]: print("x = " + str(x))
2 x = 1
3
4 In [18]: type(str(x))
5 Out[18]: str

```

Listing 71: Conversión correcta a cadena.

En Python, una cadena es un objeto que tiene varios **métodos** que se pueden usar para manipularla (esto es la llamada programación orientada a objetos y se discutirá más adelante). Para acceder a estos diversos métodos, usa este patrón `cadena.nombre_del_metodo`.

```

1 In [19]: w.upper()
2 Out[19]: "HELLO WORLD"

```

Listing 72: Usando el método `upper`.

```

1 In [20]: w.count("l")
2 Out[20]: 3

```

Listing 73: Usando el método `count`.

```

1 In [21]: w.replace("World", "Berkeley")
2 Out[21]: "Hello Berkeley"

```

Listing 74: Usando el método `replace`.

Hay diferentes maneras de preformatear una cadena. Aquí introducimos dos formas de hacerlo. Por ejemplo, si tenemos dos variables `nombre` y `pais`, y queremos imprimirlas en una oración, pero no queremos usar la concatenación de cadenas que usamos antes, ya que usaría muchos signos `+` en la cadena, podemos hacer lo siguiente:

```
1 In [22]: nombre = "UC Berkeley"
2         pais = "USA"
3         print("%s is a great school in %s!" % (nombre, pais))
4 UC Berkeley is a great school in USA!
```

Listing 75: Formato de cadena con %.

¿QUÉ ESTÁ PASANDO?

En el ejemplo anterior, el `%s` dentro de las comillas dobles le está diciendo a Python que queremos insertar algunas cadenas en esta ubicación (`s` significa cadena en este caso). El `%(nombre, pais)` es la ubicación desde donde se deben insertar las dos cadenas.

NUEVO!

Hay una forma diferente que solo se introdujo en Python 3.6 y versiones posteriores, se llama **f-string**, que significa cadena formateada. Puedes formatear fácilmente una cadena con la siguiente línea:

```
1 In [23]: print(f"{nombre} is a great school in {pais}.")
2 UC Berkeley is a great school in USA.
```

Listing 76: Usando f-string.

Incluso puedes imprimir una expresión numérica sin convertir el tipo de dato como hicimos antes.

```
1 In [24]: print(f"{3*4}")
2 12
```

Listing 77: f-string con expresión numérica.

En este punto, hemos aprendido sobre la estructura de datos `string`; esta es nuestra primera estructura de datos secuencial. Aprendamos más ahora.

Estructura de Datos – List (Lista)

En la sección anterior, aprendimos que las cadenas pueden contener una secuencia de caracteres. Ahora, presentamos una estructura de datos secuencial más versátil en Python: la **lista**. La forma de definirla es usar un par de corchetes `[ ]`, y los elementos dentro de ella están separados por comas. Una lista puede contener cualquier tipo de dato: numérico, cadenas u otros tipos. Por ejemplo,

```

1 In [1]: list_1 = [1, 2, 3]
2         list_1
3 Out[1]: [1, 2, 3]
4
5 In [2]: list_2 = ["Hello", "World"]
6         list_2
7 Out[2]: ["Hello", "World"]

```

Listing 78: Creando listas.

Podemos poner tipos mixtos en la lista también:

```

1 In [3]: list_3 = [1, 2, 3, "Apple", "orange"]
2         list_3
3 Out[3]: [1, 2, 3, "Apple", "orange"]

```

Listing 79: Lista con tipos mixtos.

También podemos anidar listas, por ejemplo:

```

1 In [4]: list_4 = [list_1, list_2]
2         list_4
3 Out[4]: [[1, 2, 3], ["Hello", "World"]]

```

Listing 80: Listas anidadas.

La forma de recuperar el elemento en la lista es muy similar a cómo se hace para las cadenas, ver Fig. 19.

1	2	3	"Apple"	"orange"
0	1	2	3	4

Figura 19: Ejemplo de índices de lista.

```
1 In [5]: list_3[2]
2 Out[5]: 3
3
4 In [6]: list_3[:3]
5 Out[6]: [1, 2, 3]
6
7 In [7]: list_3[-1]
8 Out[7]: "orange"
9
10 In [8]: list_4[0]
11 Out[8]: [1, 2, 3]
```

Listing 81: Accediendo a elementos de la lista.

De manera similar, podemos obtener la longitud de la lista usando la función `len`.

```
1 In [9]: len(list_3)
2 Out[9]: 5
```

Listing 82: Longitud de una lista.

También podemos concatenar dos listas simplemente usando un signo `+`.

```
1 In [10]: list_1 + list_2
2 Out[10]: [1, 2, 3, "Hello", "World"]
```

Listing 83: Concatenando listas.

Se pueden agregar nuevos elementos a una lista existente usando el método `append` de la lista.

```
1 In [11]: list_1.append(4)
2          list_1
3 Out[11]: [1, 2, 3, 4]
```

Listing 84: Agregando elementos con `append`.

Nota

La función `append` opera sobre la lista misma, como se muestra en el ejemplo anterior, se agrega 4 a la lista. Pero en el ejemplo `list_1 + list_2`, `list_1` y `list_2` no cambiarán. Puedes verificar `list_2` para comprobarlo.

También podemos insertar o eliminar elementos de la lista usando los métodos `insert` y `remove`, pero también operan directamente sobre la lista.

```
1 In [12]: list_1.insert(2, "center")
2         list_1
3 Out[12]: [1, 2, "center", 3, 4]
```

Listing 85: Insertando elementos con `insert`.

Nota

Usar el método `remove` solo eliminará la primera aparición del elemento (lee la documentación del método). Hay otra forma de eliminar un elemento usando su índice: la función `del`.

```
1 In [13]: del list_1[2]
2         list_1
3 Out[13]: [1, 2, 3, 4]
```

Listing 86: Eliminando elementos con `del`.

También podemos definir una lista vacía y agregar nuevos elementos más tarde usando el método `append`. Se usa mucho en Python cuando tienes que recorrer una secuencia de elementos; aprenderemos más sobre este método en el Capítulo 5.

```
1 In [14]: list_5 = []
2         list_5.append(5)
3         list_5
4 Out[14]: [5]
5
6 In [15]: list_5.append(6)
7         list_5
8 Out[15]: [5, 6]
```

Listing 87: Creando y llenando una lista vacía.

También podemos comprobar rápidamente si un elemento está en la lista usando el operador `in`.

```
1 In [16]: 5 in list_5
2 Out[16]: True
```

Listing 88: Comprobando pertenencia a la lista.

Usando la función `list`, podemos convertir otros elementos de secuencia en una lista.

```
1 In [17]: list("Hello World")
```

```
2 Out [17]: ["H", "e", "l", "l", "o", " ", "W", "o", "r", "l", "d"]
```

Listing 89: Convirtiendo una cadena en lista.

Las listas se usan con frecuencia en Python cuando se trabaja con datos, con muchas aplicaciones posibles diferentes, como se discutirá en secciones posteriores.

Estructura de Datos – Tuple (Tupla)

Aprendamos una estructura de datos secuencial diferente en Python: la **tuple**. Por lo general, se define usando un par de paréntesis (), y sus elementos están separados por comas. Por ejemplo,

```
1 In [1]: tuple_1 = (1, 2, 3, 2)
2         tuple_1
3 Out [1]: (1, 2, 3, 2)
```

Listing 90: Creando una tupla.

Al igual que con las cadenas y las listas, hay una forma de indexar tuplas, segmentar los elementos e incluso algunos métodos son muy similares a los que vimos antes.

```
1 In [2]: len(tuple_1)
2 Out [2]: 4
3
4 In [3]: tuple_1[1:4]
5 Out [3]: (2, 3, 2)
6
7 In [4]: tuple_1.count(2)
8 Out [4]: 2
```

Listing 91: Operaciones con tuplas.

Puedes preguntarte, ¿cuál es la diferencia entre listas y tuplas? Si son similares entre sí, ¿por qué necesitamos otra estructura de datos secuencial?

Las tuplas se crean por una razón. De la documentación de Python⁸:

Aunque las tuplas pueden parecer similares a las listas, a menudo se usan en diferentes situaciones y para diferentes propósitos.

⁸<https://docs.python.org/3/tutorial/datastructures.html#tuples-and-sequences>

Las tuplas son **inmutables** y, por lo general, contienen una secuencia **heterogénea** de elementos a los que se accede mediante *unpacking* (desempaquetado) (ver más adelante en esta sección) o indexación (o incluso por atributo en el caso de tuplas con nombre). Las listas son **mutables**, y sus elementos suelen ser **homogéneos** y se accede a ellos iterando sobre la lista.

¿Qué significa **immutable**? Significa que los elementos de la tupla, una vez definidos, no se pueden cambiar. En contraste, los elementos de una lista se pueden cambiar sin ningún problema. Por ejemplo,

```

1 In [5]: list_1 = [1, 2, 3]
2         list_1[2] = 1
3         list_1
4 Out[5]: [1, 2, 1]
5
6 In [6]: tuple_1[2] = 1
7 -----
8 TypeError Traceback (most recent call last)
9 <ipython-input-6-76fb6b169c14> in <module>()
10 ----> 1 tuple_1[2] = 1
11 TypeError: 'tuple' object does not support item assignment

```

Listing 92: Mutabilidad de listas vs. inmutabilidad de tuplas.

¿Qué significa **heterogéneo**? Las tuplas suelen contener una secuencia heterogénea de elementos, mientras que las listas suelen contener una secuencia homogénea. Por ejemplo, tenemos una lista que contiene diferentes frutas. Por lo general, los nombres de las frutas se pueden almacenar en una lista, ya que son homogéneos. Ahora queremos tener una estructura de datos para almacenar cuántas piezas de fruta tenemos de cada tipo. Aquí es donde suelen entrar las tuplas, ya que el nombre de la fruta y el número son heterogéneos. Por ejemplo, ("manzana", 3) significa que tenemos 3 manzanas.

```

1 In [7]: # una lista de frutas
2         ["manzana", "banana", "naranja", "pera"]
3 Out[7]: ["manzana", "banana", "naranja", "pera"]
4
5 In [8]: # una lista de pares (fruta, número)
6         [("manzana", 3), ("banana", 4), ("naranja", 1), ("pera", 4)]
7 Out[8]: [("manzana", 3), ("banana", 4), ("naranja", 1), ("pera", 4)]

```

Listing 93: Homogeneidad vs. heterogeneidad.

Se puede acceder a las tuplas o listas mediante **desempaquetado** como se muestra en el siguiente ejemplo, que requiere que el número de variables en el lado izquierdo del signo igual sea igual al número de elementos en la secuencia.

```
1 In [9]: a, b, c = list_1
2         print(a, b, c)
3 1 2 1
```

Listing 94: Desempaquetando una lista.

Nota

La operación opuesta al desempaquetado es el **empaquetado**, como se muestra en el siguiente ejemplo. Podemos ver que no necesitamos los paréntesis para definir una tupla, pero se considera una buena práctica hacerlo.

```
1 In [10]: list_2 = 2, 4, 5
2          list_2
3 Out [10]: (2, 4, 5)
```

Listing 95: Empaquetando una tupla.

Estructura de Datos – Set (Conjunto)

Otro tipo de dato en Python es el **set** (conjunto). Es un tipo que puede almacenar una colección desordenada sin elementos duplicados. También puede soportar operaciones matemáticas como unión, intersección, diferencia y diferencia simétrica. Se define usando un par de llaves, { }, y sus elementos están separados por comas.

```
1 In [1]: {3, 3, 2, 3, 1, 4, 5, 6, 4, 2}
2 Out [1]: {1, 2, 3, 4, 5, 6}
```

Listing 96: Creando un conjunto.

Usar conjuntos.^{es} una forma rápida de determinar los elementos únicos en una cadena, lista o tupla.

```
1 In [2]: set_1 = set([1, 2, 2, 3, 2, 1, 2])
2         set_1
3 Out [2]: {1, 2, 3}
```

```
4
5 In [3]: set_2 = set((2, 4, 6, 5, 2))
6         set_2
7 Out[3]: {2, 4, 5, 6}
8
9 In [4]: set("Banana")
10 Out[4]: {"B", "a", "n"}
```

Listing 97: Encontrando elementos únicos con `set`.

Mencionamos anteriormente que los conjuntos soportan operaciones matemáticas como unión, intersección, diferencia y diferencia simétrica.

```
1 In [5]: print(set_1)
2         print(set_2)
3 {1, 2, 3}
4 {2, 4, 5, 6}
5
6 In [6]: set_1.union(set_2)
7 Out[6]: {1, 2, 3, 4, 5, 6}
8
9 In [7]: set_1.intersection(set_2)
10 Out[7]: {2}
11
12 In [8]: set_1.issubset({1, 2, 3, 4, 5})
13 Out[8]: True
```

Listing 98: Operaciones con conjuntos.

Estructura de Datos – Dictionary (Diccionario)

Presentamos varios tipos de datos secuenciales en las secciones anteriores. Ahora te presentaremos un tipo nuevo y útil: el **dictionary** (diccionario), que es un tipo de dato totalmente diferente a los que presentamos anteriormente. En lugar de usar una secuencia de números para indexar los elementos (como listas o tuplas), los diccionarios se indexan por **claves**, que pueden ser una cadena, un número o incluso una tupla (pero no una lista). Un diccionario comprende pares **clave-valor**, y cada clave se asigna a un valor correspondiente. Se define usando un par de llaves `{ }`, mientras que los elementos son una lista de pares **clave:valor** separados por comas (observa que el par

`clave:valor` está separado por dos puntos, con la clave al frente y el valor al final).

```
1 In [1]: dict_1 = {"manzana":3, "naranja":4, "pera":2}
2         dict_1
3 Out[1]: {"manzana": 3, "naranja": 4, "pera": 2}
```

Listing 99: Creando un diccionario.

Dentro de un diccionario, debido a que los elementos se almacenan sin orden, no puedes acceder a un diccionario basándote en una secuencia de números de índice. Para acceder a un diccionario, necesitamos usar la clave del elemento: `diccionario[clave]`.

```
1 In [2]: dict_1["manzana"]
2 Out[2]: 3
```

Listing 100: Accediendo a un valor por clave.

Podemos obtener todas las claves en un diccionario usando el método `keys`, o todos los valores usando el método `values`.

```
1 In [3]: dict_1.keys()
2 Out[3]: dict_keys(["manzana", "naranja", "pera"])
3
4 In [4]: dict_1.values()
5 Out[4]: dict_values([3, 4, 2])
```

Listing 101: Obteniendo claves y valores.

También podemos obtener el tamaño de un diccionario usando la función `len`.

```
1 In [5]: len(dict_1)
2 Out[5]: 3
```

Listing 102: Tamaño del diccionario.

Podemos definir un diccionario vacío y luego llenar el elemento más tarde. O podemos convertir una lista de tuplas con pares (clave, valor) en un diccionario.

```
1 In [6]: school_dict = {}
2         school_dict["UC Berkeley"] = "USA"
3         school_dict
4 Out[6]: {"UC Berkeley": "USA"}
5
6 In [7]: school_dict["Oxford"] = "UK"
7         school_dict
```

```
8 Out [7]: {"UC Berkeley": "USA", "Oxford": "UK"}
9
10 In [8]: dict([("UC Berkeley", "USA"), ("Oxford", "UK")])
11 Out [8]: {"UC Berkeley": "USA", "Oxford": "UK"}
```

Listing 103: Creando y llenando un diccionario.

También podemos comprobar si un elemento pertenece a un diccionario usando el operador `in`.

```
1 In [9]: "UC Berkeley" in school_dict
2 Out [9]: True
3
4 In [10]: "Harvard" not in school_dict
5 Out [10]: True
```

Listing 104: Comprobando pertenencia en diccionario.

También podemos usar la función `list` para convertir un diccionario en una lista de sus claves. Por ejemplo,

```
1 In [11]: list(school_dict)
2 Out [11]: ["UC Berkeley", "Oxford"]
```

Listing 105: Convirtiendo claves de diccionario en lista.

Introducción a los Arrays de NumPy

La segunda parte de este libro introdujo métodos numéricos usando Python. Usaremos mucho la construcción de arrays/matrices más adelante en el libro. Para que estés preparado para esa sección, aquí vamos a introducir la forma más común de manejar arrays en Python usando el módulo **NumPy**⁹. NumPy es probablemente el módulo de computación numérica más fundamental en Python.

NumPy está codificado tanto en Python como en C (por velocidad). En su sitio web, se enumeran algunas características importantes de NumPy:

- Un poderoso objeto de array N-dimensional
- Funciones sofisticadas (de difusión)
- Herramientas para integrar código C/C++ y Fortran

⁹<http://www.numpy.org>

- Capacidades útiles de álgebra lineal, transformada de Fourier y números aleatorios

Aquí, solo te presentaremos la parte del array de NumPy que está relacionada con la estructura de datos. Poco a poco, tocaremos otros aspectos de NumPy en capítulos posteriores.

Para usar el módulo NumPy, necesitamos importarlo primero. Una forma convencional de importarlo es usar `np` como nombre abreviado.

```
1 In [1]: import numpy as np
```

Listing 106: Importando NumPy.

Advertencia

Por supuesto, puedes llamarlo con cualquier nombre, pero "`np`" se considera una convención y es aceptado por toda la comunidad, y es una buena práctica usarlo.

Para definir un array en Python, puedes usar la función `np.array` para convertir una lista.

```
1 In [2]: x = np.array([1, 4, 3])
2         x
3 Out[2]: array([1, 4, 3])
4
5 In [3]: y = np.array([[1, 4, 3], [9, 2, 7]])
6         y
7 Out[3]: array([[1, 4, 3],
8               [9, 2, 7]])
```

Listing 107: Creando arrays de NumPy.

Nota

Un array 2D puede usar listas anidadas para representarse, donde la lista interna representa cada fila.

Conocer el tamaño o la longitud de un array suele ser útil. El **atributo** `shape` de un array se llama sobre un array `M` y devuelve una tupla donde el primer elemento es el número de filas en la matriz `M` y el segundo elemento es el número de columnas en `M`. El **atributo** `size` se llama sobre un array `M` y devuelve el número total de elementos en la matriz `M`.

```
1 In [4]: y.shape
2 Out[4]: (2, 3)
3
4 In [5]: y.size
5 Out[5]: 6
```

Listing 108: Atributos `shape` y `size`.

Nota

Puedes notar la diferencia de que usamos `y.shape` en lugar de `y.shape()`; esto se debe a que `shape` es un atributo en lugar de un método en este objeto array. Presentaremos más sobre la programación orientada a objetos en un capítulo posterior. Por ahora, solo recuerda que cuando llamamos a un método en un objeto, necesitamos usar los paréntesis, mientras que con un atributo no.

Muy a menudo nos gustaría generar arrays que tengan una estructura o patrón. Por ejemplo, podemos desear crear el array `z = [1 2 3 ... 2000]`. Sería muy tedioso escribir toda la descripción de `z` en Python. Para generar arrays que estén en orden y espaciados uniformemente, es útil usar la función `arange` en NumPy.

```
1 In [6]: z = np.arange(1, 2000, 1)
2         z
3 Out[6]: array([ 1,  2,  3, ..., 1997, 1998, 1999])
```

Listing 109: Usando `np.arange`.

Usando `np.arange`, podemos crear `z` fácilmente. Los primeros dos números son el inicio y el final de la secuencia, y el último es el incremento. Dado que es muy común tener un incremento de 1, si no se especifica un incremento, Python usará un valor predeterminado de 1. Por lo tanto, `np.arange(1, 2000)` tendrá el mismo resultado que `np.arange(1, 2000, 1)`. También se pueden usar incrementos negativos o no enteros. Si el incremento "se pasa" del último valor, solo se extenderá hasta el valor justo antes del valor final. Por ejemplo, `x = np.arange(1,8,2)` sería `[1, 3, 5, 7]`.

```
1 In [7]: np.arange(0.5, 3, 0.5)
2 Out[7]: array([0.5, 1. , 1.5, 2. , 2.5])
```

Listing 110: Generando un array con incremento de 0.5.

A veces queremos garantizar un punto de inicio y final para un array, pero aún así tener elementos espaciados uniformemente. Por ejemplo, podemos querer un array que comience en 1, termine en 8 y tenga exactamente 10 elementos. Para hacer esto, usa la función `np.linspace`. La función `linspace` toma tres valores de entrada separados por comas; por lo tanto, `A = linspace(a,b,n)` genera un array de `n` elementos igualmente espaciados desde `a` hasta `b`.

```
1 In [8]: np.linspace(3, 9, 10)
2 Out[8]: array([3., 3.66666667, 4.33333333, 5., 5.66666667,
3            6.33333333, 7., 7.66666667, 8.33333333, 9.])
```

Listing 111: Usando `np.linspace`.

Acceder a un array 1D de NumPy es similar a lo que describimos para listas o tuplas: tiene un índice para indicar la ubicación. Por ejemplo,

```
1 In [9]: # obtener el segundo elemento de x
2         x[1]
3 Out[9]: 4
4
5 In [10]: # obtener todos los elementos después del segundo
6           elemento de x
7           x[1:]
8 Out[10]: array([4, 3])
9
10 In [11]: # obtener el último elemento de x
11          x[-1]
12 Out[11]: 3
```

Listing 112: Indexando arrays 1D.

Para arrays 2D, es ligeramente diferente, ya que tenemos filas y columnas. Para acceder a los datos en un array 2D `M`, necesitamos usar `M[f, c]`, donde la fila `f` y la columna `c` están separadas por una coma. Esto se conoce como "indexado de arrays". `f` y `c` pueden ser un solo número, una lista, etc. Si solo piensas en el índice de la fila o el índice de la columna, entonces es similar al array 1D. Usemos la matriz `y` como ejemplo.

```
1 In [12]: y[0, 1]
2 Out[12]: 4
3
4 In [13]: y[0, :]
5 Out[13]: array([1, 4, 3])
6
7 In [14]: y[:, -1]
```



```
8 Out[14]: array([3, 7])
9
10 In [15]: y[:, [0, 2]]
11 Out[15]: array([[1, 3],
12               [9, 7]])
```

Listing 113: Indexando arrays 2D.

Aquí hay algunos arrays predefinidos que son realmente útiles: las funciones `np.zeros`, `np.ones` y `np.empty` son tres funciones útiles. Ve ejemplos de estos arrays predefinidos a continuación:

```
1 In [16]: np.zeros((3, 5))
2 Out[16]: array([[0., 0., 0., 0., 0.],
3               [0., 0., 0., 0., 0.],
4               [0., 0., 0., 0., 0.]])
5
6 In [17]: np.ones((5, 3))
7 Out[17]: array([[1., 1., 1.],
8               [1., 1., 1.],
9               [1., 1., 1.],
10              [1., 1., 1.],
11              [1., 1., 1.]])
```

Listing 114: Arrays de ceros y unos.

Nota

La forma del array se define en una tupla con el número de filas como primer elemento y el número de columnas como segundo. Si solo necesitas un array 1D, entonces usa solo un número como entrada: `np.ones(5)`.

```
1 In [18]: np.empty(3)
2 Out[18]: array([-3.10503618e+231, -3.10503618e+231,
3               -3.10503618e+231])
```

Listing 115: Array vacío.

Nota

El array vacío no está realmente vacío; está lleno de números aleatorios muy pequeños.

Puedes reasignar un valor de un array usando el indexado de arrays y el operador de asignación. Puedes reasignar múltiples elementos a un solo número usando el indexado de arrays en el lado izquierdo. También puedes reasignar múltiples elementos de un array siempre que tanto el número de elementos que se asignan como el número de elementos asignados sean los mismos. Puedes crear un array usando el indexado de arrays.

```
1 In [19]: a = np.arange(1, 7)
2         a
3 Out[19]: array([1, 2, 3, 4, 5, 6])
4
5 In [20]: a[3] = 7
6         a
7 Out[20]: array([1, 2, 3, 7, 5, 6])
8
9 In [21]: a[:3] = 1
10        a
11 Out[21]: array([1, 1, 1, 7, 5, 6])
12
13 In [22]: a[1:4] = [9, 8, 7]
14        a
15 Out[22]: array([1, 9, 8, 7, 5, 6])
```

Listing 116: Reasignando valores en arrays.

```
1 In [23]: b = np.zeros((2, 2))
2         b[0, 0] = 1
3         b[0, 1] = 2
4         b[1, 0] = 3
5         b[1, 1] = 4
6         b
7 Out[23]: array([[1., 2.],
8                [3., 4.]])
```

Listing 117: Creando un array desde cero.

Los arrays se definen usando aritmética básica; sin embargo, hay operaciones entre un escalar (un solo número) y un array, y operaciones entre dos arrays. Comenzaremos con las operaciones entre un escalar y un array. Para ilustrar, sea c un escalar, y b una matriz. Entonces $b + c$, $b - c$, $b * c$ y b / c suman c a cada elemento de b , restan c de cada elemento de b , multiplican cada elemento de b por c y dividen cada elemento de b por c , respectivamente.

```
1 In [24]: b + 2
```

```
2 Out [24]: array([[3., 4.],
3              [5., 6.]])
4
5 In [25]: b - 2
6 Out [25]: array([[ -1.,  0.],
7              [ 1.,  2.]])
8
9 In [26]: 2 * b
10 Out [26]: array([[2., 4.],
11               [6., 8.]])
12
13 In [27]: b / 2
14 Out [27]: array([[0.5, 1. ],
15               [1.5, 2. ]])
16
17 In [28]: b**2
18 Out [28]: array([[ 1.,  4.],
19               [ 9., 16.]])
```

Listing 118: Operaciones escalar-array.

Describir operaciones entre dos matrices es más complicado. Sean **b** y **d** dos matrices del mismo tamaño. Entonces **b - d** toma cada elemento de **b** y le resta el elemento correspondiente de **d**. De manera similar, **b + d** suma cada elemento de **d** al elemento correspondiente de **b**.

```
1 In [29]: b = np.array([[1, 2], [3, 4]])
2          d = np.array([[3, 4], [5, 6]])
3
4 In [30]: b + d
5 Out [30]: array([[ 4,  6],
6               [ 8, 10]])
7
8 In [31]: b - d
9 Out [31]: array([[ -2, -2],
10               [-2, -2]])
```

Listing 119: Operaciones array-array (suma/resta).

Hay dos tipos diferentes de multiplicación (y división) para matrices. Está la multiplicación de matrices elemento por elemento y la multiplicación de matrices estándar. Esta sección solo demostrará cómo funcionan la multiplicación y división de matrices elemento por elemento. La multiplicación de matrices estándar se describirá en el capítulo posterior sobre Álgebra Lineal. Python toma el símbolo ***** para significar la multiplicación elemento por elemento. Para matrices **b** y **d** del mismo tamaño, **b * d** toma cada elemento

de `b` y lo multiplica por el elemento correspondiente de `d`. Lo mismo es cierto para `/` y `**`.

```
1 In [32]: b * d
2 Out[32]: array([[ 3,  8],
3              [15, 24]])
4
5 In [33]: b / d
6 Out[33]: array([[0.33333333, 0.5 ],
7              [0.6 , 0.66666667]])
8
9 In [34]: b**d
10 Out[34]: array([[ 1,  16],
11              [243, 4096]])
```

Listing 120: Multiplicación y división elemento por elemento.

La **transposición** de un array, `b`, es un array, `d`, donde `b[i, j] = d[j, i]`. En otras palabras, la transposición intercambia las filas y las columnas de `b`. Puedes transponer un array en Python usando el método `T` del array.

```
1 In [35]: b.T
2 Out[35]: array([[1, 3],
3              [2, 4]])
```

Listing 121: Transposición de un array.

NumPy tiene muchas funciones aritméticas, como `sin`, `cos`, etc., que pueden tomar arrays como argumentos de entrada. La salida es la función evaluada para cada elemento del array de entrada. Se dice que una función que toma un array como entrada y realiza la función sobre él está **vectorizada**.

```
1 In [36]: x = [1, 4, 9, 16]
2          np.sqrt(x)
3 Out[36]: array([1., 2., 3., 4.])
```

Listing 122: Funciones vectorizadas.

Las operaciones lógicas se definen solo entre un escalar y un array y entre dos arrays del mismo tamaño. Entre un escalar y un array, la operación lógica se realiza entre el escalar y cada elemento del array. Entre dos arrays, la operación lógica se realiza elemento por elemento.

```
1 In [37]: x = np.array([1, 2, 4, 5, 9, 3])
2          y = np.array([0, 2, 3, 1, 2, 3])
3
4 In [38]: x > 3
5 Out[38]: array([False, False,  True,  True,  True, False])
```

```
6
7 In [39]: x > y
8 Out[39]: array([ True, False,  True,  True,  True, False])
```

Listing 123: Operaciones lógicas con arrays.

Python puede indexar elementos de un array que satisfacen una expresión lógica.

```
1 In [40]: y = x[x > 3]
2          y
3 Out[40]: array([4, 5, 9])
4
5 In [41]: x[x > 3] = 0
6          x
7 Out[41]: array([1, 2, 0, 0, 0, 3])
```

Listing 124: Indexado booleano.

Resumen y Problemas

Resumen

1. Almacenar, recuperar y manipular información y datos es importante en cualquier campo científico y de ingeniería.
2. Asignar variables es una herramienta importante para manejar valores de datos.
3. Hay diferentes tipos de datos para almacenar información en Python: `int`, `float` y `boolean` para valores individuales, y `string`, `list`, `tuple`, `set` y `dict` para datos secuenciales.
4. El array de NumPy es una poderosa estructura de datos que se usa mucho en computación científica.

Problemas y Soluciones

1. **Problema 1:** Asigna el valor 2 a la variable `x` y el valor 3 a la variable `y`. Limpia solo la variable `x`.

Solución: Usamos `del` para eliminar `x`.

```
1 x = 2
2 y = 3
3 del x
4 # Ahora, si intentas usar x, obtendrías un NameError.
5
```

Listing 125: Solución Problema 1

2. **Problema 2:** Escribe una línea de código que genere el error `NameError`: `name 'x' is not defined`.

Solución: Intentamos imprimir o usar una variable que no ha sido definida.

```
1 print(x)
2 # Salida: NameError: name 'x' is not defined
3
```

Listing 126: Solución Problema 2

3. **Problema 3:** Sean $x = 10$ e $y = 3$. Escribe una línea de código para cada una de las siguientes asignaciones: $u = x + y$, $v = x*y$, $w = x/y$, $z = \sin(x)$, $r = 8*\sin(x)$, $s = 5*\sin(x*y)$, $p = x**y$.

Solución: Importamos `math` para las funciones trigonométricas. Usamos los operadores aritméticos estándar.

```
1 import math
2 x = 10
3 y = 3
4
5 u = x + y
6 v = x * y
7 w = x / y
8 z = math.sin(x)
9 r = 8 * math.sin(x)
10 s = 5 * math.sin(x * y)
11 p = x ** y
12
13 print(f"u={u}, v={v}, w={w:.2f}, z={z:.2f}, r={r:.2f}, s={s:.2f}, p={p}")
14 # Salida: u=13, v=30, w=3.33, z=-0.54, r=-4.34, s=4.37, p=1000
```

15

Listing 127: Solución Problema 3

4. **Problema 4:** Muestra todas las variables en el Jupyter notebook después de terminar el Problema 3.

Solución: Usamos el comando mágico `%whos`.

```
1 %whos
2
```

Listing 128: Solución Problema 4

(La salida mostrará variables como `x`, `y`, `u`, `v`, etc., junto con sus tipos y valores).

5. **Problema 5:** Asigna la cadena "123" a la variable `S`. Convierte la cadena en un tipo `float` y asigna la salida a la variable `N`. Verifica que `S` es una cadena y `N` es un float usando `type`.

Solución: Usamos `float(S)` para la conversión.

```
1 S = "123"
2 N = float(S)
3 print(f"Tipo de S: {type(S)}")
4 print(f"Tipo de N: {type(N)}")
5 # Salida: Tipo de S: <class 'str'>, Tipo de N: <class '
   float'>
6
```

Listing 129: Solución Problema 5

6. **Problema 6:** Asigna "HELLO" a `s1` y "hello" a `s2`. Muestra que no son iguales. Luego muestra que `s1` y `s2` son iguales si se usa el método `lower` en `s1` y si se usa el método `upper` en `s2`.

Solución: Comparamos directamente y después de aplicar los métodos.

```
1 s1 = "HELLO"
2 s2 = "hello"
3 print(s1 == s2)
4 print(s1.lower() == s2)
5 print(s1 == s2.upper())
6 # Salida: False, True, True
7
```

Listing 130: Solución Problema 6

7. **Problema 7:** Usa la función `print` para generar las siguientes cadenas: "The word 'Engineering' has 11 letters.z "The word 'Book' has 4 letters.".

Solución: Podemos usar concatenación o f-strings.

```
1 palabra1 = "Engineering"
2 len1 = len(palabra1)
3 print(f"The word '{palabra1}' has {len1} letters.")
4
5 palabra2 = "Book"
6 len2 = len(palabra2)
7 print(f"The word '{palabra2}' has {len2} letters.")
8 # Salida: The word 'Engineering' has 11 letters.
9 #           The word 'Book' has 4 letters.
10
```

Listing 131: Solución Problema 7

8. **Problema 8:** Verifica si "Python" está en "Python is great!".

Solución: Usamos el operador `in`.

```
1 print("Python" in "Python is great!")
2 # Salida: True
3
```

Listing 132: Solución Problema 8

9. **Problema 9:** Obtén la última palabra "great" de "Python is great!".

Solución: Usamos segmentación de cadenas. Podemos encontrar el índice de "gz" segmentar hasta el final.


```
1 frase = "Python is great!"
2 # Sabemos que "great" comienza en el índice 10 (después
  de "Python is ")
3 palabra_great = frase[10:]
4 print(palabra_great)
5 # Salida: great!
6
```

Listing 133: Solución Problema 9

(Una solución más robusta usaría `frase.split()[-1]`).

10. **Problema 10:** Asigna la lista `[1, 8, 9, 15]` a `lista_a` e inserta 2 en el índice 1 usando `insert`. Agrega 4 a `lista_a` usando `append`.

Solución: Aplicamos los métodos directamente.

```
1 lista_a = [1, 8, 9, 15]
2 lista_a.insert(1, 2)
3 lista_a.append(4)
4 print(lista_a)
5 # Salida: [1, 2, 8, 9, 15, 4]
6
```

Listing 134: Solución Problema 10

11. **Problema 11:** Ordena la `lista_a` del problema 10 en orden ascendente.

Solución: Usamos el método `sort` que ordena la lista in-place.

```
1 lista_a.sort()
2 print(lista_a)
3 # Salida: [1, 2, 4, 8, 9, 15]
4
```

Listing 135: Solución Problema 11

12. **Problema 12:** Convierte `"Python is great!"` en una lista.

Solución: Usamos la función `list()`.

```
1 print(list("Python is great!"))
2 # Salida: ['P', 'y', 't', 'h', 'o', 'n', ' ', 'i', 's', ' ',
3           ', 'g', 'r', 'e', 'a', 't', '!']
```

Listing 136: Solución Problema 12

13. **Problema 13:** Crea una tupla con los elementos `"One"`, 1 y asígnala a `tupla_a`.

Solución: Usamos paréntesis.

```
1 tupla_a = ("One", 1)
2 print(tupla_a)
3 # Salida: ('One', 1)
4
```

Listing 137: Solución Problema 13

14. **Problema 14:** Obtén el segundo elemento en la tupla `tupla_a` del Problema 13.

Solución: Indexamos con `[1]` (recordando que el índice 0 es el primero).

```
1 print(tupla_a[1])
2 # Salida: 1
3
```

Listing 138: Solución Problema 14

15. **Problema 15:** Obtén el elemento único de la tupla `(2, 3, 2, 3, 1, 2, 5)`.

Solución: Convertimos la tupla a un conjunto para eliminar duplicados.

```
1 tupla_b = (2, 3, 2, 3, 1, 2, 5)
2 conjunto_unico = set(tupla_b)
3 print(conjunto_unico)
4 # Salida: {1, 2, 3, 5}
5
```

Listing 139: Solución Problema 15

16. **Problema 16:** Asigna (2, 3, 2) a `conjunto_a`, y (1, 2, 3) a `conjunto_b`. Obtén: unión, intersección y diferencia de `conjunto_a` a `conjunto_b` usando el método `difference`.

Solución: Primero creamos conjuntos a partir de las tuplas y luego aplicamos los métodos.

```
1 conjunto_a = set((2, 3, 2))
2 conjunto_b = set((1, 2, 3))
3 print(f"Unión: {conjunto_a.union(conjunto_b)}")
4 print(f"Interseccion: {conjunto_a.intersection(conjunto_b)}")
5 print(f"Diferencia (A - B): {conjunto_a.difference(conjunto_b)}")
6 # Salida: Unión: {1, 2, 3}, Interseccion: {2, 3},
   Diferencia (A - B): set()
7
```

Listing 140: Solución Problema 16

17. **Problema 17:** Crea un diccionario con las claves `.a`, `"B"`, `ζ` y los valores `.a`, `"b"`, `ζ` respectivamente. Imprime todas las claves en el diccionario.

Solución: Usamos el método `keys()`.

```
1 dict_abc = {"A": "a", "B": "b", "C": "c"}
2 print(dict_abc.keys())
3 # Salida: dict_keys(['A', 'B', 'C'])
4
```

Listing 141: Solución Problema 17

18. **Problema 18:** Verifica si la clave `"B"` está en el diccionario definido en el Problema 17.

Solución: Usamos el operador `in`.

```
1 print("B" in dict_abc)
2 # Salida: True
3
```

Listing 142: Solución Problema 18

19. **Problema 19:** Crea arrays `x` e `y` con `x = [1, 4, 3, 2, 9, 4]` e `y = [2, 3, 4, 1, 2, 3]`. Calcula las asignaciones del Problema 3 (asumiendo que las operaciones son elemento por elemento).

Solución: Usamos NumPy para operaciones vectorizadas.

```
1 import numpy as np
2 x = np.array([1, 4, 3, 2, 9, 4])
3 y = np.array([2, 3, 4, 1, 2, 3])
4
5 u = x + y
6 v = x * y
7 w = x / y
8 z = np.sin(x)
9 r = 8 * np.sin(x)
10 s = 5 * np.sin(x * y)
11 p = x ** y
12
13 print("u:", u)
14 print("v:", v)
15 print("w:", w)
16 print("z:", z)
17 print("r:", r)
18 print("s:", s)
19 print("p:", p)
20
```

Listing 143: Solución Problema 19

20. **Problema 20:** Genera un array de tamaño 100 espaciado uniformemente entre -10 y 10 usando la función `linspace` de NumPy.

Solución: Usamos `np.linspace(-10, 10, 100)`.

```
1 array_100 = np.linspace(-10, 10, 100)
2 print(array_100)
3
```

Listing 144: Solución Problema 20

21. **Problema 21:** Sea `array_a` un array `[-1, 0, 1, 2, 0, 3]`. Escribe un comando que devuelva un array que consista en todos los elementos de `array_a` que sean mayores que cero.

Solución: Usamos indexado booleano.

```
1 array_a = np.array([-1, 0, 1, 2, 0, 3])
2 mayores_que_cero = array_a[array_a > 0]
3 print(mayores_que_cero)
4 # Salida: [1 2 3]
5
```

Listing 145: Solución Problema 21

22. **Problema 22:** Crea una matriz y y calcula su transpuesta.

Solución: Creamos un array 2D y usamos el atributo T.

```
1 y = np.array([[3, 5, 3],
2               [2, 2, 5],
3               [3, 8, 9]])
4 y_transpuesta = y.T
5 print("Matriz original:\n", y)
6 print("Transpuesta:\n", y_transpuesta)
7
```

Listing 146: Solución Problema 22

23. **Problema 23:** Crea un array de 2 filas y 4 columnas lleno de ceros.

Solución: Usamos `np.zeros((2,4))`.

```
1 array_ceros = np.zeros((2, 4))
2 print(array_ceros)
3
```

Listing 147: Solución Problema 23

24. **Problema 24:** Cambia la segunda columna del array anterior a 1.

Solución: Indexamos todas las filas y la columna con índice 1 (la segunda columna) y les asignamos 1.

```
1 array_ceros[:, 1] = 1
2 print(array_ceros)
3 # Salida: [[0. 1. 0. 0.]
4 #          [0. 1. 0. 0.]]
5
```

Listing 148: Solución Problema 24

25. **Problema 25:** Escribe un comando mágico para limpiar todas las variables en el Jupyter notebook.

Solución: El comando es `%reset`. Preguntará confirmación.

```
1 %reset
2
```

Listing 149: Solución Problema 25

FUNCIONES

Fundamentos de las Funciones

En programación, una **función** es una secuencia de instrucciones que realiza una tarea específica. Una función es un bloque de código que puede ejecutarse cuando es **llamada**. Una función puede tener **argumentos de entrada**, que son proporcionados por el usuario (la entidad que llama a la función). Las funciones también tienen **parámetros de salida**. Estos son los resultados de la función una vez que ha completado su tarea. Por ejemplo, la función `math.sin` tiene un argumento de entrada (un ángulo en radianes) y un argumento de salida (una aproximación de la función seno calculada en el ángulo de entrada). La secuencia de instrucciones para calcular esta aproximación constituye el **cuerpo** de la función, que se está introduciendo aquí.

Funciones Incorporadas en Python

Ya se han presentado muchas funciones incorporadas de Python, como `type`, `len`, etc. Además, hemos introducido varias funciones disponibles en diferentes paquetes, por ejemplo, `math.sin`, `np.array`, etc. ¿Recuerdas cómo llamar y usar estas funciones?

```
1 In [1]: type(len)
2 Out[1]: builtin_function_or_method
```

Listing 150: Verificando que `len` es una función incorporada.

```
1 In [2]: import numpy as np
2         type(np.linspace)
3 Out[2]: function
4
```



```
5 In [3]: np.linspace?
```

Listing 151: Verificando que `np.linspace` es una función.

Definiendo tu Propia Función

Podemos definir nuestras propias funciones. Una función se puede especificar de varias maneras. La forma más común de definir una función es usando la palabra clave `def`, como se muestra a continuación:

```
1 def nombre_de_la_funcion(parametro_1, parametro_2, ...):  
2     """  
3     Cadena descriptiva  
4     """  
5     # comentarios sobre las declaraciones  
6     declaraciones_de_la_funcion  
7     return parametros_de_salida (opcional)
```

Listing 152: Sintaxis para definir una función.

Definir una función en Python requiere los siguientes dos componentes:

1. **Cabecera de la función:** comienza con la palabra clave `def`, seguida de un par de paréntesis con los parámetros de entrada dentro, y termina con dos puntos (`:`).
2. **Cuerpo de la función:** es un bloque indentado (generalmente cuatro espacios en blanco) que indica el cuerpo principal de la función. Consta de tres partes:
 - **Cadena descriptiva:** una cadena que describe la función y a la que se puede acceder mediante la función `help()` o el signo de interrogación. Las comillas triples simples o dobles indican dónde colocar las cadenas descriptivas. Puedes escribir cualquier cadena dentro de las comillas, ya sea en una línea o en varias líneas.
 - **Declaraciones de la función:** son las instrucciones paso a paso que ejecutará la función cuando se la llame. Observa que hay una línea que comienza con `#`; esto es un comentario de una sola línea, lo que significa que no es parte de la función y no se ejecuta.

- **Declaraciones de retorno:** pueden contener algunos parámetros que serán devueltos después de que se llame a la función. Como se discutirá más adelante, cualquier tipo de dato puede ser devuelto, incluso una función.

Nota

Parámetro de entrada vs. argumento. Un *parámetro* es una variable definida por una función que recibe un valor cuando la función es llamada. Un *argumento* es un valor que se pasa a un método cuando es invocado. Por ejemplo, si definimos una función `hola(nombre)`, entonces `nombre` es un parámetro de entrada. Cuando llamamos a la función y le pasamos un valor 'Juan', entonces este valor es un argumento de entrada. Debido a la diferencia muy sutil, en el resto del libro, usaremos parámetros y argumentos indistintamente.

Consejo

Cuando tu código se vuelve más largo y complicado, los comentarios pueden ayudarte a ti y a quienes lean tu código a navegar por los comandos y proporcionar un "mapa de ruta" lógico para entender lo que estás tratando de hacer. Acostumbrarse a comentar con frecuencia evitará errores de codificación, te ayudará a entender hacia dónde se dirige tu código cuando lo escribes y te ayudará a encontrar errores cuando los cometes. Aunque es opcional, también es costumbre poner una descripción de la función, el autor y la fecha de creación en la cadena descriptiva debajo de la cabecera de la función (puedes omitir la cadena descriptiva). Recomendamos encarecidamente que comentes abundantemente tu propio código.

```
1 In [4]: def my_adder(a, b, c):
2         """
3         function to sum the 3 numbers
4         Input: 3 numbers a, b, c
5         Output: the sum of a, b, and c
6         author:
7         date:
8         """
9         # this is the summation
10        out = a + b + c
```

```
11     return out
```

Listing 153: Definiendo una función `my_adder`.

Advertencia

Si no indentas tu código al definir una función, obtendrás un `IndentationError`.

```
1 In [5]: def my_adder(a, b, c):
2         """
3         function to sum the 3 numbers
4         Input: 3 numbers a, b, c
5         Output: the sum of a, b, and c
6         author:
7         date:
8         """
9         # this is the summation
10        out = a + b + c
11        return out
12 File "<ipython-input-5-e6a61721f00e>", line 8
13        """
14
15 IndentationError: expected an indented block
```

Listing 154: Error de indentación.

Consejo

Escribir manualmente cuatro espacios en blanco es un nivel de indentación. Se requieren niveles más profundos de indentación cuando tienes funciones anidadas o declaraciones `if`, que discutiremos en el siguiente capítulo. Observa que a veces necesitas indentar o desindentar un bloque de código. Puedes hacerlo seleccionando primero todas las líneas en el bloque de código y luego presionando `Tab` y `Shift+Tab` para aumentar o disminuir un nivel de indentación.

Consejo

Desarrolla buenas prácticas de codificación dando a las variables y funciones nombres descriptivos, comentando a menudo y evitando líneas de código superfluas.

Para contrastar, considera la siguiente función que realiza la misma tarea que `my_adder` pero no está construida siguiendo las mejores prácticas. Como puedes ver, es extremadamente difícil seguir la lógica del código y las intenciones del autor.

```
1 In [6]: def abc(a, s2, d):
2         z = a + s2
3         z = z + d
4         x = z
5         return x
```

Listing 155: Mala representación de `my_adder`.

Las funciones deben cumplir con un esquema de nomenclatura similar al de las variables. Solo pueden contener caracteres alfanuméricos y guiones bajos, y el primer carácter debe ser una letra.

Consejo

Al igual que la convención para los nombres de las variables, los nombres de las funciones deben estar en minúsculas, con palabras separadas por guiones bajos según sea necesario para mejorar la legibilidad.

Consejo

Es una buena práctica de programación guardar a menudo mientras escribes tu función. De hecho, ¡muchos programadores guardan su código usando el atajo `Ctrl+s` (PC) o `cmd+s` (Mac) cada vez que dejan de escribir!

```
1 In [7]: d = my_adder(1, 2, 3)
2         d
3 Out [7]: 6
4
5 In [8]: d = my_adder(4, 5, 6)
6         d
7 Out [8]: 15
8
9 In [9]: help(my_adder)
10 Help on function my_adder in module __main__:
11
12 my_adder(a, b, c)
13     function to sum the 3 numbers
```

```
14 Input: 3 numbers a, b, c
15 Output: the sum of a, b, and c
16 author:
17 date:
```

Listing 156: Usando `my_adder`.

¿QUÉ ESTÁ PASANDO?

Primero, recuerda que el operador de asignación funciona de derecha a izquierda. Esto significa que `my_adder(1,2,3)` se resuelve antes de la asignación a `d`.

1. Python encuentra la función `my_adder`.
2. `my_adder` toma el primer valor del argumento de entrada, 1, y lo asigna a la variable con nombre `a` (primer nombre de variable en la lista de argumentos de entrada).
3. `my_adder` toma el segundo valor del argumento de entrada, 2, y lo asigna a la variable con nombre `b` (segundo nombre de variable en la lista de argumentos de entrada).
4. `my_adder` toma el tercer valor del argumento de entrada, 3, y lo asigna a la variable con nombre `c` (tercer nombre de variable en la lista de argumentos de entrada).
5. `my_adder` calcula la suma de `a`, `b` y `c`, que es $1 + 2 + 3 = 6$.
6. `my_adder` asigna el valor 6 a la variable `out`.
7. `my_adder` devuelve el valor contenido en la variable de salida `out`, que es 6.
8. `my_adder(1,2,3)` es equivalente al valor 6, y este valor se asigna a la variable con nombre `d`.

Python le da al usuario una tremenda libertad para asignar variables a diferentes tipos de datos. Por ejemplo, es posible dar a la variable `x` un valor de diccionario o de tipo `float`. En otros lenguajes de programación, este no es siempre el caso. En esos programas, debes declarar al comienzo de una sesión si `x` será de tipo diccionario o `float`, y una vez que decides qué tipo

es, no puedes cambiarlo. Esto puede ser tanto un beneficio como un inconveniente. Por ejemplo, `my_adder` se construyó asumiendo que los argumentos de entrada eran de tipo numérico, ya sea `int` o `float`; sin embargo, el usuario puede ingresar accidentalmente una lista o una cadena en `my_adder`, lo cual no es correcto. Si intentas ingresar un argumento de entrada de tipo no numérico en `my_adder`, Python continuará ejecutando la función hasta que algo salga mal.

```
1 In [10]: d = my_adder("1", 2, 3)
2 -----
3 TypeError Traceback (most recent call last)
4 <ipython-input-10-245d0f4254a9> in <module>
5 ----> 1 d = my_adder("1", 2, 3)
6 <ipython-input-4-72d064c3ba7a> in my_adder(a, b, c)
7     9
8     10 # this is the summation
9 ----> 11 out = a + b + c
10     12
11     13 return out
12 TypeError: must be str, not int
13
14 In [11]: d = my_adder(1, 2, [2, 3])
15 -----
16 TypeError Traceback (most recent call last)
17 <ipython-input-11-04f0428ffc51> in <module>
18 ----> 1 d = my_adder(1, 2, [2, 3])
19 <ipython-input-4-72d064c3ba7a> in my_adder(a, b, c)
20     9
21     10 # this is the summation
22 ----> 11 out = a + b + c
23     12
24     13 return out
25 TypeError: unsupported operand type(s) for +: "int" and "list"
```

Listing 157: Errores con tipos de datos incorrectos.

Consejo

Recuerda leer los mensajes de error que proporciona Python. Por lo general, te dicen exactamente dónde estaba el problema. En este caso, el error dice `--->11 out = a + b + c`, lo que significa que hubo un error en `my_adder` en la línea 11. La razón del error es `TypeError`, porque `unsupported operand type(s) for +: int and "list"`, lo que significa que no podemos sumar `int` y `list`.

En este punto, no tienes ningún control sobre lo que el usuario asigna como argumentos de entrada a tu función y si corresponden a lo que pretendías que fueran esos argumentos de entrada. Así que, por el momento, escribe tus funciones asumiendo que serán utilizadas correctamente. Puedes ayudarte a ti mismo y a otros usuarios a usar tu función correctamente proporcionando comentarios que detallen tu código.

Puedes componer funciones asignando llamadas a funciones como entrada a otras funciones. En el orden de las operaciones, Python ejecutará primero la llamada a la función más interna. También puedes asignar expresiones matemáticas como entrada a las funciones. En este caso, Python ejecutará primero las expresiones matemáticas.

```

1 In [12]: d = my_adder(np.sin(np.pi), np.cos(np.pi), np.tan(np
      .pi))
2         d
3 Out[12]: -1.0
4
5 In [13]: d = my_adder(5 + 2, 3 * 4, 12 / 6)
6         d
7 Out[13]: 21.0
8
9 In [14]: d = (5 + 2) + 3 * 4 + 12 / 6
10        d
11 Out[14]: 21.0

```

Listing 158: Componiendo funciones.

Las funciones de Python pueden tener múltiples parámetros de salida. Al llamar a una función con múltiples parámetros de salida, puedes desempaquetar los resultados con múltiples variables, que debes separar por comas. La función, esencialmente, devolverá los múltiples parámetros de resultado en una tupla, lo que luego te permite desempaquetar la tupla devuelta. Ve el siguiente ejemplo (observa que tiene múltiples parámetros de salida):

```
1 In [15]: def my_trig_sum(a, b):
2         """
3         function to demo return multiple
4         author
5         date
6         """
7         out1 = np.sin(a) + np.cos(b)
8         out2 = np.sin(b) + np.cos(a)
9         return out1, out2, [out1, out2]
10
11 In [16]: c, d, e = my_trig_sum(2, 3)
12         print(f"c = {c}, d = {d}, e = {e}")
13 c = -0.0806950697747637, d = -0.2750268284872752,
14 e = [-0.0806950697747637, -0.2750268284872752]
```

Listing 159: Función con múltiples salidas.

Si asignas los resultados a una sola variable, obtendrás una tupla que incluye todos los parámetros de salida.

```
1 In [17]: c = my_trig_sum(2, 3)
2         print(f"c={c}, and the returned type is {type(c)}")
3 c=(-0.0806950697747637, -0.2750268284872752,
4    [-0.0806950697747637, -0.2750268284872752]),
5    and the returned type is <class 'tuple'>
```

Listing 160: Asignando múltiples salidas a una sola variable.

Se puede definir una función sin argumento de entrada y sin devolver ningún valor. Por ejemplo,

```
1 In [18]: def print_hello():
2         print("Hello")
3
4 In [19]: print_hello()
5 Hello
```

Listing 161: Función sin entradas ni salidas.

Nota

Incluso si no hay argumento de entrada, cuando llamas a la función, aún necesitas incluir los paréntesis.

Para los argumentos de entrada, podemos incluir un valor predeterminado también. Ve el siguiente ejemplo:


```
1 In [20]: def print_greeting(day = "Monday", name = "Qingkai")
2         :
3         print(f"Greetings, {name}, today is {day}")
4
5 In [21]: print_greeting()
6 Greetings, Qingkai, today is Monday
7
8 In [22]: print_greeting(name = "Timmy", day = "Friday")
9 Greetings, Timmy, today is Friday
10
11 In [23]: print_greeting(name = "Alex")
12 Greetings, Alex, today is Monday
```

Listing 162: Argumentos con valores predeterminados.

Podemos ver que si asignamos un valor al argumento cuando definimos la función, este valor será el valor predeterminado de la función. Si el usuario no proporciona una entrada para este argumento, entonces este valor predeterminado se usará durante la llamada a la función. Observa que el orden de los argumentos no es importante al llamar a la función si proporcionas el nombre del argumento.

Variables Locales y Variables Globales

El Capítulo 2 introdujo la idea de un bloque de memoria asociado con el notebook, donde se almacenan las variables creadas en el notebook. Una función también tiene su propio bloque de memoria que está reservado para las variables creadas dentro de esa función. Este bloque de memoria no se comparte con el bloque de memoria de todo el notebook. Por lo tanto, se puede asignar una variable con un nombre dado dentro de una función sin cambiar una variable con el mismo nombre fuera de la función. El bloque de memoria asociado con la función se abre cada vez que se usa una función.

```
1 In [1]: def my_adder(a, b, c):
2         out = a + b + c
3         print(f"The value out within the function is {out}")
4         return out
5
6 out = 1
7 d = my_adder(1, 2, 3)
8 print(f"The value out outside the function is {out}")
9 The value out within the function is 6
```

```
10 The value out outside the function is 1
```

Listing 163: Variables locales vs. globales.

En `my_adder`, la variable `out` es una **variable local**. Esto se debe a que solo está definida dentro de la función `my_adder`, no puede afectar a las variables fuera de la función. Las acciones realizadas en el notebook fuera de la función no pueden afectarla, incluso si tienen el mismo nombre. Así que en el ejemplo anterior, está la variable `out`, que ha sido definida en la celda del notebook. Cuando se llama a `my_adder` en la siguiente línea, Python abre un nuevo bloque de memoria para las variables de esa función. Una de las variables creadas dentro de la función es otra variable, `out`. Debido a que están ubicadas en diferentes bloques de memoria, la asignación a `out` dentro de `my_adder` no cambia el valor asignado a `out` fuera de la función.

¿Por qué tener bloques de memoria de función separados en lugar de un solo bloque de memoria? Aunque puede no parecer lógico que Python separe los bloques de memoria, es muy eficiente para proyectos grandes que constan de muchas funciones que trabajan juntas. Si un programador es responsable de codificar una función y otro es responsable de codificar una función diferente, tener bloques de memoria separados permite que cada programador trabaje de forma independiente y tenga la confianza de que su codificación no producirá errores al considerar el código de otro programador, y viceversa. Los bloques de memoria separados protegen una función de influencias externas. Las únicas cosas del exterior del bloque de memoria de la función que pueden afectar lo que sucede dentro de una función son los argumentos de entrada, y las únicas cosas que pueden escapar al mundo exterior desde el bloque de memoria de una función cuando la función termina son los argumentos de salida.

Los siguientes ejemplos están diseñados para ser ejercicios para ganar experiencia con el concepto de variables locales. Son intencionalmente muy confusos, pero si puedes desenredarlos, habrás dominado el concepto de variables locales dentro de una función. Concéntrate en exactamente lo que Python está haciendo y en el orden en que Python lo hace.

```
1 In [2]: def my_test(a, b):
2     x = a + b
3     y = x * b
4     z = a + b
5     m = 2
6     print(f"Within function: x={x}, y={y}, z={z}")
```

```
7     return x, y
```

Listing 164: Función de prueba `my_test`.

```
1 In [3]: a = 2
2         b = 3
3         z = 1
4         y, x = my_test(b, a)
5         print(f"Outside function: x={x}, y={y}, z={z}")
6 Within function: x=5, y=10, z=5
7 Outside function: x=10, y=5, z=1
```

Listing 165: Primera prueba con `my_test`.

```
1 In [4]: x = 5
2         y = 3
3         b, a = my_test(x, y)
4         print(f"Outside function: x={x}, y={y}, z={z}")
5 Within function: x=8, y=24, z=8
6 Outside function: x=5, y=3, z=1
```

Listing 166: Segunda prueba con `my_test`.

```
1 In [5]: m
2 -----
3 NameError Traceback (most recent call last)
4 <ipython-input-5-9a40b379906c> in <module>
5 ----> 1 m
6 NameError: name "m" is not defined
```

Listing 167: Intentando acceder a una variable local fuera de la función.

Observa que el valor `m` no está definido fuera de la función porque se definió dentro de ella. Lo mismo ocurre si defines una variable fuera de una función; usarla dentro de la función cambiará el valor, y aparecerá el mismo mensaje de error.

```
1 In [6]: n = 42
2
3         def func():
4             print(f"Within function: n is {n}")
5             n = 3
6             print(f"Within function: change n to {n}")
7
8         func()
9         print(f"Outside function: Value of n is {n}")
10 -----
```

```

11 UnboundLocalError Traceback (most recent call last)
12 <ipython-input-6-85f3215553ae> in <module>
13     6         print(f"Within function: change n to {n}")
14     7
15 ----> 8 func()
16     9 print(f"Outside function: Value of n is {n}")
17 <ipython-input-6-85f3215553ae> in func()
18     2
19     3 def func():
20 ----> 4     print(f"Within function: n is {n}")
21     5     n = 3
22     6     print(f"Within function: change n to {n}")
23 UnboundLocalError: local variable "n" referenced before
    assignment

```

Listing 168: Problema al usar una variable global dentro de una función.

La solución es usar la palabra clave `global` para que Python sepa que esta variable es una **variable global** y puede usarse tanto fuera como dentro de la función.

```

1 In [7]: n = 42
2
3     def func():
4         global n
5         print(f"Within function: n is {n}")
6         n = 3
7         print(f"Within function: change n to {n}")
8
9     func()
10    print(f"Outside function: Value of n is {n}")
11 Within function: n is 42
12 Within function: change n to 3
13 Outside function: Value of n is 3

```

Listing 169: Usando la palabra clave `global`.

Funciones Anidadas

Una vez que has creado y guardado una nueva función, se comporta como cualquier otra función incorporada de Python. Puedes llamar a la función desde cualquier lugar del notebook, y cualquier otra función puede llamar a la función también. Una **función anidada** es una función que se define dentro de otra función, llamada **función padre**. Solo la función padre puede

llamar a la función anidada. Recuerda que la función anidada conserva un bloque de memoria separado de su función padre.

```

1 In [1]: import numpy as np
2
3         def my_dist_xyz(x, y, z):
4             """
5             x, y, z are 2D coordinates contained in a tuple
6             output:
7             d - list, where
8             d[0] is the distance between x and y
9             d[1] is the distance between x and z
10            d[2] is the distance between y and z
11            """
12            def my_dist(x, y):
13                """
14                subfunction for my_dist_xyz
15                Output is the distance between x and y,
16                computed using the distance formula
17                """
18                out = np.sqrt((x[0]-y[0])**2 + (x[1]-y[1])
19                               **2)
20
21                return out
22
23            d0 = my_dist(x, y)
24            d1 = my_dist(x, z)
25            d2 = my_dist(y, z)
26            return [d0, d1, d2]

```

Listing 170: Ejemplo de función anidada.

Observa que las variables `x` e `y` aparecen tanto en `my_dist_xyz` como en `my_dist`. Esto es permisible porque una función anidada tiene un bloque de memoria separado de su función padre. Las funciones anidadas son útiles cuando una tarea debe realizarse muchas veces dentro de la función pero no fuera de ella. De esta manera, las funciones anidadas ayudan a la función padre a realizar su tarea mientras se ocultan dentro de ella.

```

1 In [2]: d = my_dist_xyz((0, 0), (0, 1), (1, 1))
2         print(d)
3         d = my_dist((0, 0), (0, 1))
4 [1.0, 1.4142135623730951, 1.0]
5 -----
6 NameError Traceback (most recent call last)
7 <ipython-input-2-1bec838581d7> in <module>
8     1 d = my_dist_xyz((0, 0), (0, 1), (1, 1))

```

```
9      2 print(d)
10 ----> 3 d = my_dist((0, 0), (0, 1))
11 NameError: name "my_dist" is not defined
```

Listing 171: Llamando a la función anidada (y su error).

El siguiente ejemplo es el código repetido sin usar la función anidada. Observa cuán más recargada y desordenada se ve la función y cuánto más difícil es entender lo que está sucediendo. Esta versión es mucho más propensa a errores porque tienes tres oportunidades de escribir mal la fórmula de la distancia. Observa que esta función se puede escribir de manera más compacta usando operaciones vectoriales. Dejamos esto como un ejercicio.

```
1 import numpy as np
2
3 def my_dist_xyz(x, y, z):
4     """
5     x, y, z are 2D coordinates contained in a tuple
6     output:
7     d - list, where
8     d[0] is the distance between x and y
9     d[1] is the distance between x and z
10    d[2] is the distance between y and z
11    """
12    d0 = np.sqrt((x[0]-y[0])**2 + (x[1]-y[1])**2)
13    d1 = np.sqrt((x[0]-z[0])**2 + (x[1]-z[1])**2)
14    d2 = np.sqrt((y[0]-z[0])**2 + (y[1]-z[1])**2)
15    return [d0, d1, d2]
```

Listing 172: Versión sin función anidada.

Funciones Lambda

A veces no es óptimo definir una función de la manera habitual, especialmente si nuestra función es solo una línea. En este caso, usamos la función anónima en Python, que es una función que se define sin nombre. Este tipo de función también se llama **función lambda**, ya que se definen usando la palabra clave `lambda`. Una función lambda típica se define de la siguiente manera:

CONSTRUCCIÓN

```
1 lambda argumentos: expresion
```

Puede tener cualquier número de argumentos pero solo una expresión.

```
1 In [1]: square = lambda x: x**2
2
3         print(square(2))
4         print(square(5))
5 4
6 25
```

Listing 173: Definiendo y usando una función lambda.

En la función lambda anterior, `x` es el argumento y `x**2` es la expresión que se evalúa y se devuelve. La función en sí no tiene nombre y devuelve un objeto de función (discutido en un capítulo posterior) para elevarlo al cuadrado. Después de definirse, podemos llamarla como una función normal. La función lambda es equivalente a:

```
1 def square(x):
2     return x**2

1 In [2]: my_adder = lambda x, y: x + y
2
3         print(my_adder(2, 4))
4 6
```

Listing 174: Función lambda con dos argumentos.

Las funciones lambda pueden ser útiles en muchos casos, y proporcionaremos otros ejemplos en capítulos posteriores. Aquí solo mostramos un caso de uso común para la función lambda.

```
1 In [3]: sorted([(1, 2), (2, 0), (4, 1)], key=lambda x: x[1])
2 Out[3]: [(2, 0), (4, 1), (1, 2)]
```

Listing 175: Usando lambda para ordenar.

¿Qué está pasando? La función `sorted` tiene un argumento `key`, donde se puede proporcionar una función clave personalizada para personalizar el orden de clasificación. Usamos la función lambda como un atajo para esta función clave personalizada.

Funciones como Argumentos de Funciones

Hasta ahora, has asignado varias estructuras de datos a nombres de variables. Poder asignar una estructura de datos a una variable permite pasar información a las diversas funciones y recuperar información de ellas de una manera ordenada y metódica. A veces es útil poder pasar una función y hacer que actúe como una variable para otra función. En otras palabras, la entrada de algunas funciones pueden ser otras funciones. En la última sección, vimos la función `lambda` devolviendo un objeto de función a la variable. En esta sección, veremos ejemplos adicionales de cómo se puede usar el objeto de función como entrada para otra función.

```
1 In [1]: f = max
2         print(type(f))
3 <class "builtin_function_or_method">
```

Listing 176: Asignando la función `max` a una variable.

En el ejemplo anterior, `f` ahora es equivalente a la función `max`. Debido a que `x = 1` significa que `x` y `1` son intercambiables, `f` y la función `max` ahora son intercambiables.

```
1 In [2]: print(f([2, 3, 5]))
2         print(max([2, 3, 5]))
3 5
4 5
```

Listing 177: Usando la variable de función.

```
1 In [3]: import numpy as np
2
3         def my_fun_plus_one(f, x):
4             return f(x) + 1
5
6         print(my_fun_plus_one(np.sin, np.pi/2))
7         print(my_fun_plus_one(np.cos, np.pi/2))
8         print(my_fun_plus_one(np.sqrt, 25))
9 2.0
10 1.0
11 6.0
```

Listing 178: Escribiendo una función que toma otra función como argumento.

En el ejemplo anterior, se utilizan diferentes funciones como entradas para la función. Por supuesto, también podemos usar las funciones `lambda`.


```
1 In [4]: print(my_fun_plus_one(lambda x: x + 2, 2))
2 5
```

Listing 179: Usando una función lambda como argumento.

Resumen y Problemas

Resumen

1. Una función es un conjunto de instrucciones autocontenido diseñado para realizar una tarea específica.
2. Una función tiene su propio bloque de memoria para sus variables. La información se puede agregar al bloque de memoria de una función solo a través de sus variables de entrada. La información puede salir del bloque de memoria de una función solo a través de sus variables de salida.
3. Se puede definir una función dentro de otra función, lo que se denomina función anidada. Solo la función padre puede acceder a esta función anidada.
4. Puedes definir una función anónima usando la palabra clave `lambda`, que es la llamada función lambda.
5. Puedes asignar funciones a variables utilizando referencias a funciones (handles).

Problemas y Soluciones

1. **Problema 1:** Escribe una función `my_sinh(x)` que devuelva el seno hiperbólico de `x`, dado por $\frac{e^x - e^{-x}}{2}$. Asume que `x` es un float.

Solución: Usamos `math.exp` para la exponencial.

```
1 import math
2
3 def my_sinh(x):
4     y = (math.exp(x) - math.exp(-x)) / 2
5     return y
```

```
6
7 print(f"my_sinh(0) = {my_sinh(0)}")
8 print(f"my_sinh(1) = {my_sinh(1):.4f}")
9 print(f"my_sinh(2) = {my_sinh(2):.4f}")
10 # Salida: my_sinh(0) = 0.0, my_sinh(1) = 1.1752, my_sinh
    (2) = 3.6269
11
```

Listing 180: Solución Problema 1

2. **Problema 2:** Escribe una función `my_checker_board(n)` que devuelva una matriz $n \times n$ con un patrón de tablero de ajedrez (1 y 0), comenzando con 1 en la esquina superior izquierda.

Solución: Creamos una matriz de ceros y luego asignamos 1 a las posiciones donde la suma de los índices de fila y columna es par.

```
1 import numpy as np
2
3 def my_checker_board(n):
4     m = np.zeros((n, n), dtype=int)
5     for i in range(n):
6         for j in range(n):
7             if (i + j) % 2 == 0:
8                 m[i, j] = 1
9     return m
10
11 print(my_checker_board(1))
12 print(my_checker_board(2))
13 print(my_checker_board(5))
14
```

Listing 181: Solución Problema 2

3. **Problema 3:** Escribe una función `my_triangle(b, h)` que devuelva el área de un triángulo con base b y altura h .

Solución: Aplicamos la fórmula $\text{área} = \frac{1}{2} \times \text{base} \times \text{altura}$.

```
1 def my_triangle(b, h):
2     area = 0.5 * b * h
3     return area
```

```

4
5 print(my_triangle(1, 1))
6 print(my_triangle(2, 1))
7 print(my_triangle(12, 5))
8 # Salida: 0.5, 1.0, 30.0
9

```

Listing 182: Solución Problema 3

4. **Problema 4:** Escribe una función `my_split_matrix(m)` que divida una matriz por la mitad (o aproximadamente) en dos matrices: la mitad izquierda y la mitad derecha.

Solución: Calculamos el punto medio de las columnas. Si el número de columnas es impar, la columna del medio va a la izquierda.

```

1 import numpy as np
2
3 def my_split_matrix(m):
4     filas, columnas = m.shape
5     mitad = (columnas + 1) // 2 # Redondeo hacia arriba
6     para la izquierda
7     m1 = m[:, :mitad]
8     m2 = m[:, mitad:]
9     return [m1, m2]
10
11 m = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
12 m1, m2 = my_split_matrix(m)
13 print("m1:\n", m1)
14 print("m2:\n", m2)

```

Listing 183: Solución Problema 4

5. **Problema 5:** Escribe una función `my_cylinder(r, h)` que devuelva una lista con el área superficial y el volumen de un cilindro.

Solución: Usamos `math.pi` y las fórmulas: $A = 2\pi r^2 + 2\pi rh$, $V = \pi r^2 h$.

```
1 import math
2
3 def my_cylinder(r, h):
4     area_sup = 2 * math.pi * r**2 + 2 * math.pi * r * h
5     volumen = math.pi * r**2 * h
6     return [area_sup, volumen]
7
8 print(my_cylinder(1, 5))
9 print(my_cylinder(2, 4))
10
```

Listing 184: Solución Problema 5

6. **Problema 6:** Escribe una función `my_n_odds(a)` que cuente el número de números impares en un array unidimensional `a`.

Solución: Iteramos sobre el array y usamos el operador módulo `%` para verificar si un número es impar.

```
1 import numpy as np
2
3 def my_n_odds(a):
4     contador = 0
5     for num in a:
6         if num % 2 != 0:
7             contador += 1
8     return contador
9
10 print(my_n_odds(np.arange(100)))           # 50 (0..99,
11       los impares son 50)
12 print(my_n_odds(np.arange(2, 100, 2)))     # 0 (todos
13       pares)
```

Listing 185: Solución Problema 6

7. **Problema 7:** Escribe una función `my_twos(m, n)` que devuelva una matriz `m x n` llena de doses.

Solución: Usamos `np.ones` y multiplicamos por 2.

```
1 import numpy as np
2
3 def my_twos(m, n):
4     return 2 * np.ones((m, n), dtype=int)
5
6 print(my_twos(3, 2))
7 print(my_twos(1, 4))
8
```

Listing 186: Solución Problema 7

8. **Problema 8:** Escribe una función lambda que tome x e y y devuelva $x - y$.

Solución: La función lambda es directa.

```
1 resta = lambda x, y: x - y
2 print(resta(10, 3))
3 # Salida: 7
4
```

Listing 187: Solución Problema 8

9. **Problema 9:** Escribe una función `add_string(s1, s2)` que concatene dos cadenas.

Solución: Usamos el operador `+` para concatenar.

```
1 def add_string(s1, s2):
2     return s1 + s2
3
4 s1 = add_string("Programming", " ")
5 s2 = add_string("is ", "fun!")
6 print(add_string(s1, s2))
7 # Salida: Programming is fun!
8
```

Listing 188: Solución Problema 9

10. **Problema 10:** Genera los errores `TypeError` (por argumento faltante) e `IndentationError`.

Solución: Llamamos a una función que espera argumentos sin proporcionarlos. Para el error de indentación, escribimos código con indentación incorrecta.

```
1 def mi_funcion(a):
2     print(a)
3
4 mi_funcion() # Esto genera TypeError: mi_funcion()
               missing 1 required positional argument: 'a'
5
```

Listing 189: Solución Problema 10 (TypeError)

```
1 def mi_funcion():
2     print("Hola") # Esta línea no está indentada
                   correctamente
3
```

Listing 190: Solución Problema 10 (IndentationError)

11. **Problema 11:** Escribe una función `greeting(name, age)` que devuelva un saludo formateado.

Solución: Usamos una f-string para formatear la cadena.

```
1 def greeting(name, age):
2     return f"Hi, my name is {name} and I am {age} years
           old."
3
4 print(greeting("John", 26))
5 print(greeting("Kate", 19))
6 # Salida: Hi, my name is John and I am 26 years old.
7 #         Hi, my name is Kate and I am 19 years old.
8
```

Listing 191: Solución Problema 11

12. **Problema 12:** Escribe una función vectorizada `my_donut_area(r1, r2)` que calcule el área de un anillo circular (donut). Asume que `r1` y `r2` son arrays de la misma longitud.

Solución: El área es $\pi(r_2^2 - r_1^2)$. En NumPy, las operaciones son vectorizadas por defecto.

```

1 import numpy as np
2
3 def my_donut_area(r1, r2):
4     area = np.pi * (r2**2 - r1**2)
5     return area
6
7 r1 = np.arange(1, 4)
8 r2 = np.arange(2, 7, 2) # Esto da [2,4,6]
9 print(my_donut_area(r1, r2))
10 # Salida: [ 9.42477796 37.69911184 84.82300165]
11

```

Listing 192: Solución Problema 12

13. **Problema 13:** Escribe una función `my_within_tolerance(A, a, tol)` que devuelva los índices de los elementos en `A` que están dentro de `tol` de `a` ($|A - a| < tol$).

Solución: Iteramos sobre el array y verificamos la condición. Guardamos los índices que la cumplen.

```

1 def my_within_tolerance(A, a, tol):
2     indices = []
3     for i, valor in enumerate(A):
4         if abs(valor - a) < tol:
5             indices.append(i)
6     return indices
7
8 print(my_within_tolerance([0, 1, 2, 3], 1.5, 0.75)) #
9 # Salida: [1, 2]
10
11 import numpy as np
12 A = np.arange(0, 1.01, 0.01)
13 print(my_within_tolerance(A, 0.5, 0.03))
14 # Salida: [47, 48, 49, 50, 51, 52]
15

```

Listing 193: Solución Problema 13

14. **Problema 14:** Escribe una función `bounding_array(A, top, bottom)` que limite los valores de un array entre `bottom` y `top`.

Solución: Usamos indexado booleano de NumPy para modificar los elementos que están fuera del rango.

```
1 import numpy as np
2
3 def bounding_array(A, top, bottom):
4     A = A.copy() # Para no modificar el original
5     A[A < bottom] = bottom
6     A[A > top] = top
7     return A
8
9 A = np.arange(-5, 6, 1)
10 print(bounding_array(A, 3, -3))
11 # Salida: [-3 -3 -3 -2 -1  0  1  2  3  3  3]
12
```

Listing 194: Solución Problema 14

DECLARACIONES DE RAMIFICACIÓN

Declaraciones If-Else

Una **declaración de ramificación**, también conocida como declaración If-Else o simplemente declaración If, es una construcción de código que ejecuta bloques de código solo si se cumplen ciertas condiciones. Estas condiciones se representan como expresiones lógicas.

CONSTRUCCIÓN: Sintaxis de una declaración If simple

```
1 if expresion_logica:
2     bloque_de_codigo
```

CONSTRUCCIÓN: Sintaxis de una declaración If-Else simple

```
1 if expresion_logica:
2     bloque_de_codigo_1
3 else:
4     bloque_de_codigo_2
```

La palabra `if` es una palabra clave. Cuando Python encuentra una declaración `if`, determinará si la expresión lógica asociada es verdadera. Si es verdadera, entonces se ejecutará el código en el `bloque_de_codigo`. Si es falsa, entonces el código dentro de la declaración `if` no se ejecutará. La forma de leer esto es "Si la expresión lógica es verdadera, entonces haz el bloque de código". De manera similar, si la expresión lógica en la declaración `if-else`

es verdadera, entonces se ejecutará el código en el `bloque_de_codigo_1`. De lo contrario, se ejecutará el `bloque_de_codigo_2`.

Cuando hay varias condiciones a considerar, puedes incluir declaraciones `elif`; si deseas una condición que cubra cualquier otro caso, entonces puedes usar una declaración `else`. Sean P, Q y R tres expresiones lógicas en Python. El siguiente ejemplo muestra múltiples ramas.

Nota

Python da el mismo nivel de indentación a cada línea de código dentro de una declaración condicional.

CONSTRUCCIÓN: Sintaxis de una declaración If-Else extendida

```
1 if expresion_logica_P:
2     bloque_de_codigo_1
3 elif expresion_logica_Q:
4     bloque_de_codigo_2
5 elif expresion_logica_R:
6     bloque_de_codigo_3
7 else:
8     bloque_de_codigo_4
```

En el código anterior, Python primero verificará si P es verdadero. Si P es verdadero, entonces se ejecutará el `bloque_de_codigo_1` y luego la declaración `if` terminará. En otras palabras, Python no verificará el resto de las declaraciones una vez que alcanza una declaración verdadera. Si P es falso, entonces Python verificará si Q es verdadero. Si Q es verdadero, entonces se ejecutará el `bloque_de_codigo_2` y la declaración `if` terminará. Si es falso, entonces se verificará R, y así sucesivamente. Si P, Q y R son todos falsos, entonces se ejecutará el `bloque_de_codigo_4`. Puedes tener cualquier número de declaraciones `elif` (o ninguna) siempre que haya al menos una declaración `if` (la primera declaración). No necesitas una declaración `else`, pero puedes tener, como máximo, una declaración `else`. Las expresiones lógicas después de `if` y `elif` (es decir, como P, Q y R) se denominarán **declaraciones condicionales**.

```
1 In [1]: def my_thermo_stat(temp, desired_temp):
2         """
3         Changes the status of the thermostat based on
```

```
4     temperature and desired temperature
5     author
6     date
7     :type temp: Int
8     :type desiredTemp: Int
9     :rtype: String
10    """
11    if temp < desired_temp - 5:
12        status = "Heat"
13    elif temp > desired_temp + 5:
14        status = "AC"
15    else:
16        status = "off"
17    return status
18
19 In [2]: status = my_thermo_stat(65,75)
20         print(status)
21 Heat
22
23 In [3]: status = my_thermo_stat(75,65)
24         print(status)
25 AC
26
27 In [4]: status = my_thermo_stat(65,63)
28         print(status)
29 off
```

Listing 195: Función que simula un termostato.

```
1 In [5]: x = 3
2         if x > 1:
3             y = 2
4         elif x > 2:
5             y = 4
6         else:
7             y = 0
8         print(y)
9 2
```

Listing 196: Ejemplo de múltiples condiciones.

También podemos insertar declaraciones condicionales más complicadas usando operadores lógicos.

```
1 In [6]: x = 3
2         if x > 1 and x < 2:
3             y = 2
```

```
4 elif x > 2 and x < 4:
5     y = 4
6 else:
7     y = 0
8 print(y)
9 4
```

Listing 197: Usando operadores lógicos.

Observa que si quieres la declaración lógica $a < x < b$, esto se considera como dos declaraciones condicionales, $a < x$ y $x < b$. Python también te permite escribir $a < x < b$. Por ejemplo,

```
1 In [7]: x = 3
2         if 1 < x < 2:
3             y = 2
4         elif 2 < x < 4:
5             y = 4
6         else:
7             y = 0
8         print(y)
9 4
```

Listing 198: Sintaxis simplificada para rangos.

Una declaración se llama **anidada** si está completamente contenida dentro de otra declaración del mismo tipo. Por ejemplo, una declaración `if` anidada es una declaración `if` que está completamente contenida dentro de una cláusula de otra declaración `if`.

```
1 In [8]: def my_nested_branching(x,y):
2         """
3         Nested Branching Statement Example
4         author
5         date
6         :type x: Int
7         :type y: Int
8         :rtype: Int
9         """
10        if x > 2:
11            if y < 2:
12                out = x + y
13            else:
14                out = x - y
15        else:
16            if y > 2:
17                out = x * y
```

```
18         else:
19             out = 0
20     return out
```

Listing 199: Ejemplo de ramificación anidada.

Nota

Como antes, Python da el mismo nivel de indentación a cada línea de código dentro de una declaración condicional. La declaración `if` anidada debe estar indentada con cuatro espacios adicionales. Obtendrás un `IndentationError` si la indentación no es correcta, como vimos anteriormente al discutir cómo definir funciones.

```
1 In [9]: import numpy as np
2
3 In [10]: all([1, 1, 0])
4 Out[10]: False
```

Listing 200: Funciones lógicas `all` y `any`.

Hay muchas funciones lógicas diseñadas para ayudarte a construir declaraciones de ramificación. Por ejemplo, puedes preguntar si una variable tiene un cierto tipo de dato con la función `isinstance`. También hay funciones que pueden darte información sobre arrays de valores lógicos, como `any`, que se evalúa como verdadero si algún elemento en un array es verdadero, y falso en caso contrario, y `all`, que se evalúa como verdadero solo si todos los elementos en un array son verdaderos.

A veces quieres diseñar tu función para verificar las entradas de una función y asegurarte de que se usará correctamente. Por ejemplo, la función `my_adder` del capítulo anterior espera números como entrada. Si el usuario ingresa una lista o una cadena como una de las variables de entrada, entonces la función lanzará un error o tendrá resultados inesperados. Para prevenir esto, puedes poner una verificación para informar al usuario que la función no se ha utilizado correctamente. Esta y otras técnicas para controlar errores se exploran más a fondo en el Capítulo 10. Por el momento, solo necesitas saber que podemos usar la declaración `raise` con una excepción `TypeError` para detener la ejecución de una función y lanzar un error con un texto específico.

```
1 In [11]: def my_adder(a, b, c):
2         """
```

```

3     Calculate the sum of three numbers
4     author
5     date
6     """
7     # Check for erroneous input
8     if not (isinstance(a, (int, float)) \
9             or isinstance(b, (int, float)) \
10            or isinstance(c, (int, float))):
11         raise TypeError("Inputs must be numbers.")
12     # Return output
13     return a + b + c
14
15 In [12]: x = my_adder(1,2,3)
16         print(x)
17 6
18
19 In [13]: x = my_adder("1","2","3")
20         print(x)
21 -----
22 TypeError Traceback (most recent call last)
23 <ipython-input-13-c3e353c636b0> in <module>
24 ----> 1 x = my_adder("1","2","3")
25       2 print(x)
26 <ipython-input-11-0f3d29ecee0> in my_adder(a, b, c)
27     10         or isinstance(b, (int, float)) \
28     11         or isinstance(c, (int, float))):
29 ----> 12         raise TypeError("Inputs must be numbers.")
30     13     # Return output
31     14     return a + b + c
32 TypeError: Inputs must be numbers.

```

Listing 201: Verificación de tipos de entrada con `raise`.

Hay una gran variedad de entradas erróneas que tu función puede encontrar por parte de los usuarios, y no es razonable esperar que tu función las detecte todas. Por lo tanto, a menos que se indique lo contrario, escribe tus funciones asumiendo que se usarán correctamente.

El resto de la sección da algunos ejemplos más de declaraciones de ramificación.

```

1 In [14]: def is_odd(number):
2         """
3         function returns "odd" if the input is odd,
4         "even" otherwise
5         author
6         date

```

```
7      :type number: Int
8      :rtype: String
9      """
10     # use modulo to check if the input divisible by 2
11     if number % 2 == 0:
12         # if divisible by 2, then input is not odd
13         return "even"
14     else:
15         return "odd"
16
17 In [15]: is_odd(11)
18 Out[15]: "odd"
19
20 In [16]: is_odd(2)
21 Out[16]: "even"
```

Listing 202: Función que verifica si un número es impar o par.

```
1 In [17]: np.pi
2 Out[17]: 3.141592653589793
3
4 In [18]: def my_circ_calc(r, calc):
5     """
6     Calculate various circle measurements
7     author
8     date
9     :type r: Int or Float
10    :type calc: String
11    :rtype: Int or Float
12    """
13    if calc == "area":
14        return np.pi * r**2
15    elif calc == "circumference":
16        return 2 * np.pi * r
17
18 In [19]: my_circ_calc(2.5, "area")
19 Out[19]: 19.634954084936208
20
21 In [20]: my_circ_calc(3, "circumference")
22 Out[20]: 18.84955592153876
```

Listing 203: Función para cálculos de círculos.

Nota

La función aquí no se limita a una entrada de un solo valor, sino que también se puede ejecutar usando arrays de NumPy (es decir, la misma operación se aplicará a cada elemento del array). Observa el siguiente ejemplo donde calculamos las circunferencias para radios [2, 3, 4] usando un array de NumPy.

```
1 In [21]: my_circ_calc(np.array([2, 3, 4]), "circumference")
2 Out [21]: array([12.56637061, 18.84955592, 25.13274123])
```

Listing 204: Usando la función con arrays de NumPy.

Operadores Ternarios

La mayoría de los lenguajes de programación tienen **operadores ternarios**, que generalmente se conocen como **expresiones condicionales**, que proporcionan un mecanismo que utiliza una línea de código para evaluar la primera expresión si la condición es verdadera; de lo contrario, evalúa la segunda expresión. Para implementar el operador ternario en Python, utiliza la construcción que se presenta a continuación:

CONSTRUCCIÓN: operador ternario en Python

```
1 expresion_si_verdadero if condicion else
  expresion_si_falso
```

```
1 In [1]: is_student = True
2         person = "student" if is_student else "not student"
3         print(person)
4 student
```

Listing 205: Ejemplo de operador ternario.

En el ejemplo anterior, podemos ver que este código de una línea es equivalente al siguiente bloque de códigos.

```
1 In [2]: is_student = True
2         if is_student:
3             person = "student"
```

```
4     else:
5         person = "not student"
6     print(person)
7 student
```

Listing 206: Equivalente con if-else.

Los operadores ternarios proporcionan una forma simple de ramificación y pueden hacer que nuestros códigos sean concisos. En el próximo capítulo, presentamos su papel en las comprensiones de listas y demostraremos su utilidad.

Resumen y Problemas

Resumen

1. Las declaraciones de ramificación (if-else) permiten que las funciones tomen diferentes acciones bajo diferentes circunstancias.
2. Los operadores ternarios permiten declaraciones de ramificación en una sola línea.

Problemas y Soluciones

1. **Problema 1:** Calculadora de propinas.

Solución: Usamos una serie de condiciones `if-elif-else` para determinar el porcentaje de propina según el tamaño del grupo.

```
1 def my_tip_calc(bill, party):
2     if party < 6:
3         tip_percent = 0.15
4     elif party < 8:
5         tip_percent = 0.18
6     elif party < 11:
7         tip_percent = 0.20
8     else:
9         tip_percent = 0.25
10    tips = bill * tip_percent
11    return tips
```

```
12
13 print(f"{my_tip_calc(109.29, 3):.4f}")
14 print(f"{my_tip_calc(109.29, 7):.4f}")
15 print(f"{my_tip_calc(109.29, 9):.4f}")
16 print(f"{my_tip_calc(109.29, 12):.4f}")
17 # Salida: 16.3935, 19.6722, 21.8580, 27.3225
18
```

Listing 207: Solución Problema 1

2. Problema 2: Operaciones matemáticas con un parámetro de selección.

Solución: Usamos una serie de condiciones `if-elif-else` para la cadena `operation`. La función es vectorizada porque usa operaciones de NumPy que lo son.

```
1 import numpy as np
2
3 def my_mult_operation(a, b, operation):
4     if operation == "plus":
5         out = a + b
6     elif operation == "minus":
7         out = a - b
8     elif operation == "mult":
9         out = a * b
10    elif operation == "div":
11        out = a / b
12    elif operation == "pow":
13        out = a ** b
14    else:
15        raise ValueError("Operación no válida")
16    return out
17
18 x = np.array([1,2,3,4])
19 y = np.array([2,3,4,5])
20 print(my_mult_operation(x, y, "plus"))
21 print(my_mult_operation(x, y, "minus"))
22 print(my_mult_operation(x, y, "mult"))
23 print(my_mult_operation(x, y, "div"))
24 print(my_mult_operation(x, y, "pow"))
25
```

Listing 208: Solución Problema 2

3. **Problema 3:** Determinar si un punto está dentro de un triángulo rectángulo.

Solución: El triángulo con vértices (0,0), (1,0), (0,1) tiene hipotenusa la línea $x + y = 1$. Las condiciones para los bordes son $x = 0$ con $0 \leq y \leq 1$, $y = 0$ con $0 \leq x \leq 1$, y $x + y = 1$ con $0 \leq x \leq 1$. Un punto está dentro si $x > 0$, $y > 0$ y $x + y < 1$. Usamos tolerancia para considerar los bordes.

```
1 def my_inside_triangle(x, y):
2     tol = 1e-10
3     # Verificar bordes
4     if (abs(x) < tol and 0 <= y <= 1) or \
5         (abs(y) < tol and 0 <= x <= 1) or \
6         (abs(x + y - 1) < tol and 0 <= x <= 1):
7         return "border"
8     elif x > 0 and y > 0 and x + y < 1:
9         return "inside"
10    else:
11        return "outside"
12
13 print(my_inside_triangle(0.5, 0.5))
14 print(my_inside_triangle(0.25, 0.25))
15 print(my_inside_triangle(5, 5))
16 # Salida: border, inside, outside
17
```

Listing 209: Solución Problema 3

4. **Problema 4:** Ajustar un array a un tamaño de 10.

Solución: Usamos una condición para ver la longitud. Si es mayor o igual a 10, tomamos los primeros 10. Si es menor, creamos un array de ceros de tamaño 10 y llenamos los primeros elementos con los de x.

```
1 import numpy as np
2
3 def my_make_size10(x):
4     x = np.array(x)
5     if len(x) >= 10:
```

```

6         size10 = x[:10]
7     else:
8         size10 = np.zeros(10, dtype=x.dtype)
9         size10[:len(x)] = x
10    return size10
11
12 print(my_make_size10(range(1, 2)))    #
13     [1,2,0,0,0,0,0,0,0,0]
14 print(my_make_size10(range(1, 15)))   #
15     [1,2,3,4,5,6,7,8,9,10]

```

Listing 210: Solución Problema 4

5. Problema 5: Versión sin if-statement del Problema 4.

Solución: Podemos crear un array de resultado de tamaño 10 con ceros y luego asignar los valores de x hasta el mínimo entre la longitud de x y 10. Esto usa indexado pero no un if explícito.

```

1 import numpy as np
2
3 def my_make_size10(x):
4     x = np.array(x)
5     size10 = np.zeros(10, dtype=x.dtype)
6     size10[:min(len(x), 10)] = x[:min(len(x), 10)]
7     return size10
8

```

Listing 211: Solución Problema 5

6. Problema 6: Convertir porcentaje a calificación con letra.

Solución: Usamos una larga cadena de if-elif-else. Incluimos los límites en la categoría superior.

```

1 def my_letter_grader(percent):
2     if percent > 97:
3         return "A+"
4     elif percent > 93:
5         return "A"
6     elif percent > 90:

```

```
7         return "A-"
8     elif percent > 87:
9         return "B+"
10    elif percent > 83:
11        return "B"
12    elif percent > 80:
13        return "B-"
14    elif percent > 77:
15        return "C+"
16    elif percent > 73:
17        return "C"
18    elif percent > 70:
19        return "C-"
20    elif percent > 67:
21        return "D+"
22    elif percent > 63:
23        return "D"
24    elif percent > 60:
25        return "D-"
26    else:
27        return "F"
28
29 print(my_letter_grader(97))
30 print(my_letter_grader(84))
31 # Salida: A+, B
32
```

Listing 212: Solución Problema 6

7. Problema 7: Alarma nuclear basada en tres sensores.

Solución: Verificamos cada par de sensores para ver si la diferencia absoluta es mayor que 10. Si alguno de los tres pares cumple la condición, se activa la alarma.

```
1 def my_nuke_alarm(s1, s2, s3):
2     if abs(s1 - s2) > 10 or abs(s1 - s3) > 10 or abs(s2 -
3         s3) > 10:
4         return "alarm!"
5     else:
6         return "normal"
7
8 print(my_nuke_alarm(94, 96, 90))
9 print(my_nuke_alarm(94, 96, 80))
```

```

9 print(my_nuke_alarm(100, 96, 90))
10 # Salida: normal, alarm!, normal
11

```

Listing 213: Solución Problema 7

8. Problema 8: Calcular raíces de una ecuación cuadrática.

Solución: Calculamos el discriminante $d = b^2 - 4ac$. Usamos `complex` para manejar raíces imaginarias.

```

1 import math
2
3 def my_n_roots(a, b, c):
4     d = b**2 - 4*a*c
5     if d > 0:
6         n_roots = 2
7         r1 = (-b + math.sqrt(d)) / (2*a)
8         r2 = (-b - math.sqrt(d)) / (2*a)
9         r = [r1, r2]
10    elif d == 0:
11        n_roots = 1
12        r = [-b / (2*a)]
13    else:
14        n_roots = -2
15        real = -b / (2*a)
16        imag = math.sqrt(-d) / (2*a)
17        r = [complex(real, imag), complex(real, -imag)]
18    return n_roots, r
19
20 print(my_n_roots(1, 0, -9))
21 print(my_n_roots(3, 4, 5))
22 print(my_n_roots(2, 4, 2))
23 # Salida: (2, [3.0, -3.0]), (-2, [(-0.666... + 1.105...j)
24           , (-0.666... - 1.105...j)]), (1, [-1.0])

```

Listing 214: Solución Problema 8

9. Problema 9: Función por partes.

Solución: Usamos condiciones `if-elif-else` para determinar qué función evaluar.

```
1 import numpy as np
2
3 def my_split_function(f, g, a, b, x):
4     if x <= a:
5         return f(x)
6     elif x >= b:
7         return g(x)
8     else:
9         return 0.0
10
11 print(f"{my_split_function(np.exp, np.sin, 2, 4, 1):.3f}"
12       )
13 print(f"{my_split_function(np.exp, np.sin, 2, 4, 3):.3f}"
14       )
15 print(f"{my_split_function(np.exp, np.sin, 2, 4, 5):.4f}"
16       )
17 # Salida: 2.718, 0.000, -0.9589
```

Listing 215: Solución Problema 9

ITERACIÓN

Bucles For

Un **bucle for** es un conjunto de instrucciones que se repite, o itera, para cada valor en una secuencia. A veces, los bucles for se denominan **bucles definidos** porque tienen un principio y un fin predefinidos, delimitados por la secuencia.

La sintaxis general de un bloque de bucle for es la siguiente.

CONSTRUCCIÓN: Bucle for

```
1 for variable_de_iteracion in secuencia:  
2     bloque_de_codigo
```

Un bucle for asigna la `variable_de_iteracion` al primer elemento de la secuencia. Ejecuta todo en el `bloque_de_codigo`. Luego asigna la `variable_de_iteracion` al siguiente elemento de la secuencia y ejecuta el `bloque_de_codigo` nuevamente. Continúa hasta que no haya más elementos en la secuencia para asignar.

```
1 In [1]: n = 0  
2         for i in range(1, 4):  
3             n = n + i  
4         print(n)  
5 6
```

Listing 216: Suma de enteros del 1 al 3 con un bucle for.

¿QUÉ ESTÁ PASANDO?

- Primero, la función `range(1, 4)` genera una secuencia de números que comienzan en 1 y terminan en 3. Consulta la descripción de la función `range` y familiarízate con cómo usarla. En su forma más simple, es `range(inicio, parada, paso)`, y el `paso` es opcional con 1 como valor predeterminado.

1. La variable `n` se asigna con el valor 0.
2. La variable `i` se asigna con el valor 1.
3. La variable `n` se asigna con el valor `n + i` ($0 + 1 = 1$).
4. La variable `i` se asigna con el valor 2.
5. La variable `n` se asigna con el valor `n + i` ($1 + 2 = 3$).
6. La variable `i` se asigna con el valor 3.
7. La variable `n` se asigna con el valor `n + i` ($3 + 3 = 6$).
8. Al no haber más valores para asignar en la lista, el bucle `for` termina con `n = 6`.

A continuación, se presentan varios ejemplos más para darte una idea de cómo funcionan los bucles `for`. Otros ejemplos de secuencias sobre las que podemos iterar incluyen los elementos de una tupla, los caracteres de una cadena y otros tipos de datos secuenciales.

```
1 In [2]: for c in "banana":
2         print(c)
3 b
4 a
5 n
6 a
7 n
8 a
9
10 In [3]: s = "banana"
11         for i in range(len(s)):
12             print(s[i])
13 b
```

```
14 a
15 n
16 a
17 n
18 a
```

Listing 217: Imprimiendo todos los caracteres de una cadena.

```
1 In [4]: s = 0
2         a = [2, 3, 1, 3, 3]
3         for i in a:
4             s += i # nota que esto es equivalente a s = s +
5             i
6         print(s)
12
```

Listing 218: Sumando todos los elementos de una lista.

La función de Python `sum` ya está escrita para manejar el ejemplo anterior. ¿Qué pasa si quieres sumar solo los números en índices pares? ¿Qué cambios harías en el bloque de bucle `for` anterior para manejar esta restricción?

```
1 In [5]: s = 0
2         for i in range(0, len(a), 2):
3             s += a[i]
4         print(s)
5 6
```

Listing 219: Sumando elementos en índices pares.

Nota

Usamos el paso 2 en la función `range` para obtener los índices pares de la lista `a`. Un atajo de Python de uso común es el operador `+=`. En Python y muchos otros lenguajes de programación, una declaración como `i += 1` es equivalente a `i = i + 1` y es lo mismo para otros operadores como `-=`, `*=`, `/=`.

```
1 In [6]: dict_a = {"One":1, "Two":2, "Three":3}
2         for key in dict_a.keys():
3             print(key, dict_a[key])
4 One 1
5 Two 2
6 Three 3
```

Listing 220: Iterando sobre un diccionario.

En el ejemplo anterior, usamos el método `keys` primero para recuperar todas las claves. Luego, usamos la clave para acceder al valor. Alternativamente, podemos usar el método `items` en un diccionario, que devuelve un objeto que contiene una lista de pares clave-valor en tuplas. Podemos asignarlos simultáneamente a dos variables (asignación de tuplas); ve el ejemplo a continuación.

```
1 In [7]: for key, value in dict_a.items():
2         print(key, value)
3 One 1
4 Two 2
5 Three 3
```

Listing 221: Usando el método `items`.

Observa que podemos asignar dos variables de iteración diferentes al mismo tiempo. Hay otros casos en los que podemos asignar tareas simultáneamente. Por ejemplo, si tenemos dos listas con la misma longitud y queremos recorrerlas simultáneamente, usamos la función `zip`. Ve el ejemplo a continuación. Esta función agrega elementos de dos iterables y devuelve un iterador de tuplas, donde el *i*-ésimo elemento de la tupla contiene el *i*-ésimo elemento de cada uno de los iterables.

```
1 In [8]: a = ["One", "Two", "Three"]
2         b = [1, 2, 3]
3         for i, j in zip(a, b):
4             print(i, j)
5 One 1
6 Two 2
7 Three 3
```

Listing 222: Iterando sobre dos listas simultáneamente con `zip`.

```
1 In [9]: def have_digits(s):
2         out = 0
3         # loop through the string
4         for c in s:
5             # check if the character is a digit
6             if c.isdigit():
7                 out = 1
8                 break
9         return out
10
11 In [10]: out = have_digits("only4you")
12          print(out)
```

```
13 1
14
15 In [11]: out = have_digits("only for you")
16         print(out)
17 0
```

Listing 223: Función que verifica si una cadena contiene dígitos.

El primer paso en la función `have_digits` asume que no hay dígitos en la cadena `s` (es decir, la salida es 0 o Falso).

Observa la nueva palabra clave `break`. Si se ejecuta, la palabra clave `break` detiene inmediatamente el bucle `for` más inmediato que la contiene; es decir, si está contenida en un bucle `for` anidado, entonces solo detendrá el bucle `for` más interno. En este caso particular, el comando `break` se ejecuta si alguna vez encontramos un dígito en la cadena. El código seguirá funcionando correctamente sin esta declaración, pero dado que la tarea es determinar si hay algún dígito en `s`, no tenemos que seguir buscando si encontramos uno. De manera similar, si a un humano se le diera la misma tarea para una cadena larga de caracteres, esa persona no continuaría buscando dígitos si ya encontró uno. Las declaraciones `break` se utilizan cuando ocurre algo en un bucle `for` que haría que desees detener la ejecución temprano. Un comando menos intrusivo es la palabra clave `continue`, que omite el código restante en la iteración actual del bucle `for` y continúa con el siguiente elemento del array de iteración. Ve el siguiente ejemplo donde usamos la palabra clave `continue` para omitir la función `print` al imprimir 2:

```
1 In [12]: for i in range(5):
2         if i == 2:
3             continue
4         print(i)
5 0
6 1
7 3
8 4
```

Listing 224: Usando `continue` para omitir una iteración.

```
1 In [13]: import math
2
3         def my_dist_2_points(xy_points, xy):
4             """
5             Returns an array of distances between xy and the
6             points contained in the rows of xy_points
7             author
```

```

8         date
9         """
10        d = []
11        for xy_point in xy_points:
12            dist = math.sqrt((xy_point[0]-xy[0])**2 + (
xy_point[1]-xy[1])**2)
13            d.append(dist)
14        return d
15
16 In [14]: xy_points = [[3,2], [2, 3], [2, 2]]
17         xy = [1, 2]
18         my_dist_2_points(xy_points, xy)
19 Out[14]: [2.0, 1.4142135623730951, 1.0]

```

Listing 225: Calculando distancias desde un punto a una lista de puntos.

Al igual que las declaraciones if, los bucles for pueden anidarse.

```

1 In [15]: x = np.array([[5, 6], [7, 8]])
2         n, m = x.shape
3         s = 0
4         for i in range(n):
5             for j in range(m):
6                 s += x[i, j]
7         print(s)
8 26

```

Listing 226: Sumando todos los elementos de un array 2D con bucles anidados.

¿QUÉ ESTÁ PASANDO?

1. `s`, que representa la suma total en ejecución, se establece en 0.
2. El bucle `for` externo comienza con la variable de iteración `i` establecida en 0.
3. El bucle `for` interno comienza con la variable de iteración `j` establecida en 0.
4. `s` se incrementa en $x[i, j] = x[0, 0] = 5$. Por lo tanto, `s = 5`.
5. El bucle `for` interno establece `j = 1`.
6. `s` se incrementa en $x[i, j] = x[0, 1] = 6$; por lo tanto, `s = 11`.
7. El bucle `for` interno termina.
8. El bucle `for` externo establece `i = 1`.
9. El bucle `for` interno comienza con la variable de iteración `j` establecida en 0.
10. `s` se incrementa en $x[i, j] = x[1, 0] = 7$; por lo tanto, `s = 18`.
11. El bucle `for` interno establece `j = 1`.
12. `s` se incrementa en $x[i, j] = x[1, 1] = 8$. Por lo tanto, `s = 26`.
13. El bucle `for` interno termina.
14. El bucle `for` externo termina con `s = 26`.

Advertencia

Aunque es posible hacerlo, no intentes cambiar la variable de iteración dentro del bucle `for`. Hará que tu código sea muy complicado y probablemente resultará en errores.

Bucles While

Un **bucle while** o **bucle indefinido** es un conjunto de instrucciones que se repite mientras la expresión lógica asociada sea verdadera. La siguiente es la sintaxis abstracta de un bloque de bucle while.

CONSTRUCCIÓN: Bucle while

```
1 while <expresion_logica>:  
2     # Bloque de código a repetir hasta que la declaraci  
  n lógica sea falsa  
3     bloque_de_codigo
```

Cuando Python llega a un bloque de bucle `while`, primero determina si la expresión lógica del bucle `while` es verdadera o falsa. Si la expresión es verdadera, se ejecutará el bloque de código. Después de ejecutarse, el programa regresa a la expresión lógica al comienzo de la declaración `while`. Si es falsa, entonces el bucle `while` terminará.

```
1 In [1]: i = 0  
2         n = 8  
3         while n >= 1:  
4             n /= 2  
5             i += 1  
6         print(f"n = {n}, i = {i}")  
7 n = 0.5, i = 4
```

Listing 227: Dividiendo 8 por 2 hasta que el resultado sea menor que 1.

¿QUÉ ESTÁ PASANDO?

1. Primero, la variable `i` (recuento en ejecución de divisiones de `n` por 2) se establece en 0.
2. `n` se establece en 8 y representa el valor actual que estamos dividiendo por 2.
3. Comienza el bucle `while`.
4. Python evalúa la expresión `n >= 1` o `8 >= 1`, que es verdadera; por lo tanto, se ejecuta el bloque de código.
5. `n` se asigna a $n/2 = 8/2 = 4$.
6. `i` se incrementa a 1.
7. Python evalúa la expresión `n >= 1` o `4 >= 1`, que es verdadera; por lo tanto, se ejecuta el bloque de código.
8. `n` se asigna a $n/2 = 4/2 = 2$.
9. `i` se incrementa a 2.
10. Python evalúa la expresión `n >= 1` o `2 >= 1`, que es verdadera; por lo tanto, se ejecuta el bloque de código.
11. `n` se asigna a $n/2 = 2/2 = 1$.
12. `i` se incrementa a 3.
13. Python evalúa la expresión `n >= 1` o `1 >= 1`, que es verdadera; por lo tanto, se ejecuta el bloque de código.
14. `n` se asigna a $n/2 = 1/2 = 0.5$.
15. `i` se incrementa a 4.
16. Python evalúa la expresión `n >= 1` o `0.5 >= 1`, que es falsa; por lo tanto, el bucle `while` termina con `i = 4`.

Puedes preguntarte: "¿Qué pasa si la expresión lógica es verdadera y

nunca cambia?"; una excelente pregunta. Si la expresión lógica es verdadera y nada en el código del bucle **while** cambia la expresión, entonces el resultado se conoce como un **bucle infinito**. Los bucles infinitos se ejecutan para siempre, o hasta que tu computadora se rompa o se quede sin memoria.

```

1 In [ ]: n = 0
2         while n > -1:
3             n += 1

```

Listing 228: Bucle infinito (no ejecutar).

Dado que **n** siempre será mayor que -1 sin importar cuántas veces se ejecute el bucle, este código nunca terminará.

Puedes terminar el bucle **while** infinito manualmente presionando el botón de interrumpir el kernel – el botón cuadrado negro en la barra de herramientas que se muestra en la Fig. 20, o en el menú desplegable Kernel - Interrupt en el notebook. O si estás usando el shell de Python, presiona **cmd + c** en Mac o **Ctrl + c** en PC.

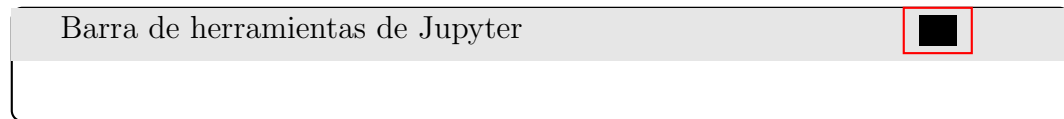


Figura 20: Interrumpe el kernel presionando el pequeño cuadrado.

¿Puedes cambiar un solo carácter para que el bucle **while** se ejecute al menos una vez pero no sea infinito?

Los bucles infinitos no siempre son fáciles de detectar. Considera los siguientes dos ejemplos: uno realiza bucles infinitos y el otro no. ¿Puedes determinar cuál es cuál? A medida que tu código se vuelva más complicado, será más difícil de detectar.

```

1 In [ ]: # Ejemplo 1
2         n = 1
3         while n > 0:
4             n /= 2
5
6 In [ ]: # Ejemplo 2
7         n = 2
8         while n > 0:
9             if n % 2 == 0:
10                n += 1

```

```
11         else:
12             n -= 1
```

Listing 229: ¿Cuál bucle while causa un bucle infinito?

Respuesta: El primer ejemplo no causará un bucle infinito porque eventualmente `n` será tan pequeño que Python no podrá distinguir la diferencia entre `n` y 0. Esto se discutirá con más detalle en el Capítulo 9. El segundo ejemplo causará un bucle infinito porque `n` oscilará entre 2 y 3 indefinidamente.

Ahora conocemos dos tipos de bucles: `for` y `while`. En algunos casos, cualquiera de los dos es apropiado, pero a veces uno es más adecuado para la tarea que el otro. En general, usa bucles `for` cuando el número de iteraciones a realizar está bien definido; usa bucles `while` cuando el número de iteraciones a realizar es indefinido o no se conoce bien.

Comprensiones

En Python, hay otras formas de hacer iteraciones; las comprensiones de listas, diccionarios y conjuntos son formas muy populares. Una vez que te familiarices con ellas, te encontrarás usándolas mucho. Las comprensiones permiten crear secuencias a partir de otras secuencias usando una sintaxis muy compacta. Primero veamos la comprensión de listas.

Comprensión de Listas

CONSTRUCCIÓN: Comprensión de listas

```
1 [Salida Secuencia_de_entrada Condiciones]
```

```
1 In [1]: x = range(5)
2
3         # Sin comprension de listas
4         y = []
5         for i in x:
6             y.append(i**2)
7         print(y)
8 [0, 1, 4, 9, 16]
9
```

```

10 In [2]: # Con comprension de listas
11         y = [i**2 for i in x]
12         print(y)
13 [0, 1, 4, 9, 16]

```

Listing 230: Cuadrando números con y sin comprensión de listas.

Además, también podemos incluir condiciones en la comprensión de listas. Por ejemplo, si solo queremos almacenar los números pares en el ejemplo anterior, simplemente agregamos una condición en la comprensión de listas.

```

1 In [3]: y = [i**2 for i in x if i%2 == 0]
2         print(y)
3 [0, 4, 16]

```

Listing 231: Comprensión de listas con condición.

Si tenemos dos niveles de bucles anidados, también podemos usar las comprensiones de listas. Por ejemplo, tenemos los siguientes dos niveles de bucles `for` que podemos realizar usando la comprensión de listas.

```

1 In [4]: # Sin comprension de listas
2         y = []
3         for i in range(5):
4             for j in range(2):
5                 y.append(i + j)
6         print(y)
7 [0, 1, 1, 2, 2, 3, 3, 4, 4, 5]
8
9 In [5]: # Con comprension de listas
10        y = [i + j for i in range(5) for j in range(2)]
11        print(y)
12 [0, 1, 1, 2, 2, 3, 3, 4, 4, 5]

```

Listing 232: Bucles anidados con y sin comprensión de listas.

Comprensión de Dicionarios

De manera similar, también podemos hacer la comprensión de diccionarios. Ve el siguiente ejemplo.

```

1 In [6]: x = {"a": 1, "b": 2, "c": 3}
2         {key: v**3 for (key, v) in x.items()}
3 Out [6]: {"a": 1, "b": 8, "c": 27}

```

Listing 233: Comprensión de diccionarios.

Realizar la comprensión de conjuntos también es posible en Python, pero no se explorará aquí. Esta es una tarea que deberías explorar por tu cuenta.

Resumen y Problemas

Resumen

1. Los bucles proporcionan un mecanismo para que un código realice tareas repetitivas, es decir, iteración.
2. Hay dos tipos de bucles: bucles `for` y bucles `while`.
3. Los bucles son importantes para construir soluciones iterativas a problemas.

Problemas y Soluciones

1. **Problema 1:** Valor de `y` después del código.

Solución: El código incrementa `y` cada vez que `i` es igual a `j`. Dado que `i` y `j` toman valores de 0 a 999, la condición `i == j` será verdadera exactamente 1000 veces (una por cada valor). Por lo tanto, `y` será 1000.

```
1 y = 0
2 for i in range(1000):
3     for j in range(1000):
4         if i == j:
5             y += 1
6 print(y) # Salida: 1000
7
```

Listing 234: Solución Problema 1

2. **Problema 2:** Función `my_max(x)` que devuelve el máximo sin usar `max`.

Solución: Inicializamos el máximo con el primer elemento y luego iteramos comparando.

```

1 def my_max(x):
2     if not x: # si la lista esta vacia
3         return None
4     max_val = x[0]
5     for item in x[1:]:
6         if item > max_val:
7             max_val = item
8     return max_val
9
10 print(my_max([3, 7, 2, 9, 1])) # Salida: 9
11

```

Listing 235: Solución Problema 2

3. **Problema 3:** Función `my_n_max(x, n)` para los `n` elementos más grandes.

Solución: Hacemos una copia de la lista, encontramos el máximo `n` veces, lo añadimos al resultado y lo eliminamos de la copia.

```

1 def my_n_max(x, n):
2     x_copy = x[:]
3     out = []
4     for _ in range(n):
5         max_val = max(x_copy)
6         out.append(max_val)
7         x_copy.remove(max_val)
8     return out
9
10 x = [7, 9, 10, 5, 8, 3, 4, 6, 2, 1]
11 print(my_n_max(x, 3)) # Salida: [10, 9, 8]
12

```

Listing 236: Solución Problema 3

4. **Problema 4:** Función `my_trig_odd_even(m)` que aplica `sin` o `cos` según la paridad.

Solución: Iteramos sobre los índices de la matriz. Usamos `%` para verificar paridad.

```
1 import numpy as np
2
3 def my_trig_odd_even(m):
4     m = np.array(m)
5     q = np.zeros(m.shape)
6     for i in range(m.shape[0]):
7         for j in range(m.shape[1]):
8             if m[i, j] % 2 == 0:
9                 q[i, j] = np.sin(m[i, j])
10            else:
11                q[i, j] = np.cos(m[i, j])
12    return q
13
14 m = np.array([[1, 2], [3, 4]])
15 print(my_trig_odd_even(m))
16
```

Listing 237: Solución Problema 4

5. Problema 5: Multiplicación de matrices con bucles anidados.

Solución: Implementamos la fórmula de la multiplicación de matrices con tres bucles anidados.

```
1 import numpy as np
2
3 def my_mat_mult(P, Q):
4     m = P.shape[0]
5     p = P.shape[1]
6     n = Q.shape[1]
7     M = np.zeros((m, n))
8     for i in range(m):
9         for j in range(n):
10             suma = 0
11             for k in range(p):
12                 suma += P[i, k] * Q[k, j]
13             M[i, j] = suma
14    return M
15
16 P = np.ones((3, 3))
17 print(my_mat_mult(P, P))
18
19 P = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
20 Q = np.array([[1, 1, 1], [2, 2, 2], [3, 3, 3], [4, 4, 4]])
```



```
21 print(my_mat_mult(P, Q))
22
```

Listing 238: Solución Problema 5

6. **Problema 6:** Plan de ahorro con interés compuesto.

Solución: Usamos un bucle **while** para incrementar el capital hasta alcanzar la meta.

```
1 def my_saving_plan(P0, i, goal):
2     years = 0
3     while P0 < goal:
4         P0 = P0 * (1 + i)
5         years += 1
6     return years
7
8 print(my_saving_plan(1000, 0.05, 2000))
9 print(my_saving_plan(1000, 0.07, 2000))
10 print(my_saving_plan(500, 0.07, 2000))
11 # Salida: 15, 11, 21
12
```

Listing 239: Solución Problema 6

7. **Problema 7:** Encontrar índices donde el valor es 1.

Solución: Iteramos sobre la lista y guardamos los índices.

```
1 def my_find(M):
2     indices = []
3     for i, val in enumerate(M):
4         if val == 1:
5             indices.append(i)
6     return indices
7
8 M = [1, 0, 1, 1, 0]
9 print(my_find(M)) # Salida: [0, 2, 3]
10
```

Listing 240: Solución Problema 7

8. **Problema 8:** Simulación de dados de Monopoly.

Solución: Usamos un bucle `while` para tirar los dados repetidamente mientras salgan iguales.

```
1 import random
2
3 def my_monopoly_dice():
4     total = 0
5     while True:
6         d1 = random.randint(1, 6)
7         d2 = random.randint(1, 6)
8         total += d1 + d2
9         if d1 != d2:
10             break
11     return total
12
13 # Ejemplo de uso
14 print(my_monopoly_dice())
15
```

Listing 241: Solución Problema 8

9. **Problema 9:** Verificar si un número es primo.

Solución: Verificamos divisibilidad desde 2 hasta la raíz cuadrada del número.

```
1 import math
2
3 def my_is_prime(n):
4     if n <= 1:
5         return 0
6     for i in range(2, int(math.sqrt(n)) + 1):
7         if n % i == 0:
8             return 0
9     return 1
10
11 print(my_is_prime(17)) # Salida: 1
12 print(my_is_prime(20)) # Salida: 0
13
```

Listing 242: Solución Problema 9

10. **Problema 10:** Lista de los primeros `n` números primos.

Solución: Buscamos números primos hasta que tengamos n de ellos.

```
1 def my_n_primes(n):
2     primes = []
3     num = 2
4     while len(primes) < n:
5         if my_is_prime(num):
6             primes.append(num)
7         num += 1
8     return primes
9
10 print(my_n_primes(5)) # Salida: [2, 3, 5, 7, 11]
11
```

Listing 243: Solución Problema 10

11. **Problema 11:** Primeros n números que son Fibonacci y primos.

Solución: Generamos números de Fibonacci y verificamos si son primos. Usamos una función iterativa para Fibonacci.

```
1 def fibonacci(n):
2     fibs = [1, 1]
3     for i in range(2, n):
4         fibs.append(fibs[i-1] + fibs[i-2])
5     return fibs
6
7 def my_n_fib_primes(n):
8     fib_primes = []
9     fib_nums = []
10    a, b = 1, 1
11    while len(fib_primes) < n:
12        if my_is_prime(b) and b not in fib_primes:
13            fib_primes.append(b)
14        a, b = b, a + b
15    return fib_primes
16
17 print(my_n_fib_primes(3)) # Salida: [3, 5, 13] (puede
18     variar)
```

Listing 244: Solución Problema 11

12. Problema 12: Función trigonométrica basada en paridad de enteros.

Solución: Similar al Problema 4, pero con `sin(pi/M)` y `cos(pi/M)`.

```
1 import numpy as np
2
3 def my_trig_odd_even(M):
4     M = np.array(M)
5     Q = np.zeros(M.shape)
6     for i in range(M.shape[0]):
7         for j in range(M.shape[1]):
8             if M[i, j] % 2 == 0:
9                 Q[i, j] = np.sin(np.pi / M[i, j])
10            else:
11                Q[i, j] = np.cos(np.pi / M[i, j])
12    return Q
13
14 M = [[3, 4], [6, 7]]
15 print(my_trig_odd_even(M))
16
```

Listing 245: Solución Problema 12

13. Problema 13: Convertir matriz de conectividad a diccionario.

Solución: Iteramos sobre la matriz y, donde hay un 1, añadimos el índice (1-based según la salida esperada) a la lista del punto correspondiente.

```
1 def my_connectivity_mat_2_dict(C, names):
2     node = {}
3     for i, name in enumerate(names):
4         connections = []
5         for j, val in enumerate(C[i]):
6             if val == 1:
7                 connections.append(j + 1) # indices 1-
8     based
9     node[name] = connections
10    return node
11
12 C = [[0, 1, 0, 1], [1, 0, 0, 1], [0, 0, 0, 1], [1, 1, 1,
13    0]]
```

```
12 names = ["Los Angeles", "New York", "Miami", "Dallas"]
13 node = my_connectivity_mat_2_dict(C, names)
14 print(node)
15 # Salida esperada: {'Los Angeles': [2, 4], 'New York':
    [1, 4], 'Miami': [4], 'Dallas': [1, 2, 3]}
16
```

Listing 246: Solución Problema 13

14. **Problema 14:** Convertir palabras a mayúsculas con comprensión de listas.

Solución: Aplicamos el método `upper()` a cada palabra en la lista.

```
1 words = ["test", "data", "analyze"]
2 upper_words = [word.upper() for word in words]
3 print(upper_words) # Salida: ['TEST', 'DATA', 'ANALYZE']
4
```

Listing 247: Solución Problema 14

RECURSIÓN

Funciones Recursivas

Una **función recursiva** es una función que se llama a sí misma. Funciona como los bucles que describimos antes. Sin embargo, en algunos casos, es preferible usar recursión en lugar de bucles.

Toda función recursiva tiene dos componentes: un **caso base** y un **paso recursivo**. El caso base suele ser la entrada más pequeña y tiene una solución fácilmente verificable. Este es también el mecanismo que evita que la función se llame a sí misma para siempre. El paso recursivo es el conjunto de todos los casos donde se realiza una **llamada recursiva**, o una llamada de la función a sí misma.

Como ejemplo, demostraremos cómo se puede usar la recursión para definir y calcular el factorial de un número entero. El factorial de un entero n es $1 \times 2 \times 3 \times \dots \times (n-1) \times n$. La definición recursiva se puede escribir de la siguiente manera:

$$f(n) = \begin{cases} 1 & \text{si } n = 1, \\ n \times f(n-1) & \text{en otro caso.} \end{cases}$$

El caso base es $n = 1$, que es trivial de calcular: $f(1) = 1$. En el paso recursivo, n se multiplica por el resultado de una llamada recursiva al factorial de $n-1$.

```
1 In [1]: def factorial(n):
2     """Computes and returns the factorial of n,
3     a positive integer.
4     """
5     if n == 1: # Base case!
6         return 1
7     else:      # Recursive step
```

```
8         return n * factorial(n - 1) # Recursive call
9
10 In [2]: factorial(3)
11 Out[2]: 6
```

Listing 248: Función factorial recursiva.

¿QUÉ ESTÁ PASANDO?

Primero, recuerda que cuando Python ejecuta una función, crea un espacio de trabajo para las variables creadas en esa función. Siempre que una función llama a otra función, esperará hasta que esa función devuelva una respuesta antes de continuar. En programación, este espacio de trabajo se llama **pila**. Similar a una pila de platos en nuestro armario de cocina, los elementos en una pila se añaden o eliminan desde la parte superior de la pila hasta la inferior, en un orden de "último en entrar, primero en salir". Por ejemplo, en `np.sin(np.tan(x))`, `sin` debe esperar a que `tan` devuelva una respuesta antes de poder ser evaluada. Aunque una función recursiva se llama a sí misma, se aplican las mismas reglas.

1. Se realiza una llamada a `factorial(3)`, por lo que se abre un nuevo espacio de trabajo para calcular `factorial(3)`.
2. El valor del argumento de entrada 3 se compara con 1. Como no son iguales, se ejecuta la declaración `else`.
3. Se debe calcular `3 * factorial(2)`. Se abre un nuevo espacio de trabajo para calcular `factorial(2)`.
4. El valor del argumento de entrada 2 se compara con 1. Como no son iguales, se ejecuta la declaración `else`.
5. Se debe calcular `2 * factorial(1)`. Se abre un nuevo espacio de trabajo para calcular `factorial(1)`.
6. El valor del argumento de entrada 1 se compara con 1. Como son iguales, se ejecuta la declaración `if`.
7. La variable de retorno se asigna con el valor 1. `factorial(1)` termina con la salida 1.
8. `2 * factorial(1)` se puede resolver a `2 * 1 = 2`. La salida se asigna con el valor 2. `factorial(2)` termina con la salida 2.
9. `3 * factorial(2)` se puede resolver a `3 * 2 = 6`. La salida se asigna con el valor 6. Por lo tanto, `factorial(3)` termina con la salida 6.

El orden de las llamadas recursivas se puede representar mediante un **árbol de recursión** que se muestra en la Fig. 21 para `factorial(3)`. Un árbol de recursión es un diagrama de las llamadas a funciones conectadas por flechas numeradas para representar el orden en que se realizaron las llamadas.

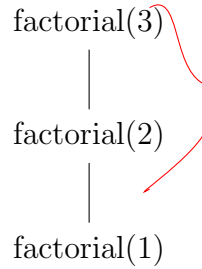


Figura 21: Árbol de recursión para `factorial(3)`.

Los números de Fibonacci se desarrollaron originalmente para modelar el crecimiento idealizado de la población de conejos. Desde entonces, se ha descubierto que son significativos en muchos fenómenos naturales. Los números de Fibonacci se pueden generar utilizando la siguiente fórmula recursiva. Observa que el paso recursivo contiene dos llamadas recursivas y que también hay dos casos base (es decir, dos casos que hacen que la recursión se detenga):

$$F(n) = \begin{cases} 1 & \text{si } n = 1, \\ 1 & \text{si } n = 2, \\ F(n-1) + F(n-2) & \text{en otro caso.} \end{cases}$$

```

1 In [3]: def fibonacci(n):
2         """Computes and returns the Fibonacci of n,
3         a positive integer.
4         """
5         if n == 1: # first base case
6             return 1
7         elif n == 2: # second base case
8             return 1
9         else: # Recursive step
10            return fibonacci(n-1) + fibonacci(n-2)
11
12 In [4]: print(fibonacci(1))

```

```
13     print(fibonacci(2))
14     print(fibonacci(3))
15     print(fibonacci(4))
16     print(fibonacci(5))
17 1
18 1
19 2
20 3
21 5
```

Listing 249: Función de Fibonacci recursiva.

Como ejercicio, considera la siguiente modificación de `fibonacci`, donde los resultados de cada llamada recursiva se muestran en la pantalla (ver Fig. ??).

```
1 In [5]: def fib_disp(n):
2     """Computes and returns the Fibonacci of n,
3     a positive integer, and prints intermediate results.
4     """
5     if n == 1: # first base case
6         out = 1
7         print(out)
8         return out
9     elif n == 2: # second base case
10        out = 1
11        print(out)
12        return out
13    else: # Recursive step
14        out = fib_disp(n-1) + fib_disp(n-2)
15        print(out)
16        return out
17
18 In [6]: fib_disp(5)
19 1
20 1
21 2
22 1
23 3
24 1
25 1
26 2
27 5
28 5
```

Listing 250: Función de Fibonacci con impresión.

Observa que el número de llamadas recursivas se vuelve muy grande incluso para entradas relativamente pequeñas de n . Si no estás de acuerdo, intenta dibujar el árbol de recursión para `fibonacci(10)`. Si pruebas tu función sin modificar para entradas alrededor de 35, experimentarás tiempos de cálculo significativos.

```
1 In [7]: fibonacci(35)
2 Out[7]: 9227465
```

Listing 251: Tiempo de cálculo para Fibonacci(35).

Existe un método iterativo para calcular el n -ésimo número de Fibonacci que requiere solo un espacio de trabajo.

```
1 In [8]: import numpy as np
2
3     def iter_fib(n):
4         fib = np.ones(n)
5         for i in range(2, n):
6             fib[i] = fib[i - 1] + fib[i - 2]
7         return fib
```

Listing 252: Implementación iterativa de Fibonacci.

```
1 In [9]: %timeit iter_fib(25)
2 7.22 s      171 ns per loop (mean      std. dev. of 7 runs,
   100000 loops each)
3
4 In [10]: %timeit fibonacci(25)
5 16.7 ms     910 s per loop (mean      std. dev. of 7 runs, 100
   loops each)
```

Listing 253: Comparación de tiempos entre recursivo e iterativo.

Puedes ver en el ejemplo anterior que la versión iterativa se ejecuta mucho más rápido que su contraparte recursiva. En general, las funciones iterativas son más rápidas que las funciones recursivas que realizan la misma tarea. Entonces, ¿por qué usar funciones recursivas? Hay algunos métodos de solución que tienen una estructura naturalmente recursiva. En estos casos, suele ser muy difícil escribir una contraparte usando bucles. El valor principal de escribir funciones recursivas es que generalmente se pueden escribir de manera mucho más compacta que las funciones iterativas. El costo de la compacidad mejorada es un tiempo de ejecución adicional.

La relación entre los argumentos de entrada y el tiempo de ejecución se discute con más detalle más adelante en el capítulo sobre complejidad.

Consejo

Intenta escribir funciones de forma iterativa siempre que sea conveniente hacerlo. Tus funciones se ejecutarán más rápido.

Nota

Cuando se usa una llamada recursiva como se muestra arriba, debemos asegurarnos de que pueda alcanzar el caso base; de lo contrario, entra en recursión infinita. En Python, ejecutar una función recursiva con una entrada grande que no puede alcanzar el caso base resultará en un error de "máxima profundidad de recursión excedida". Prueba el siguiente ejemplo para producir el `RecursionError`.

```
1 factorial(5000)
```

Listing 254: Error de recursión infinita.

Podemos evitar el límite de recursión usando el módulo `sys` estableciendo un límite más alto en Python. Si ejecutas el siguiente código, el mensaje de error no ocurrirá.

```
1 import sys
2 sys.setrecursionlimit(10**5)
3 factorial(5000)
```

Listing 255: Aumentando el límite de recursión.

Divide y Vencerás

Divide y vencerás es una estrategia útil para resolver problemas difíciles. Usando divide y vencerás, los problemas difíciles se resuelven a partir de soluciones a muchos problemas fáciles similares. De esta manera, los problemas difíciles se dividen para que sean más manejables. Esta sección presentará dos ejemplos clásicos de divide y vencerás: el problema de la Torre de Hanoi y el algoritmo Quicksort.

Torre de Hanoi

El problema de la **Torre de Hanoi** consta de tres varillas verticales (o torres) y N discos de diferentes tamaños, cada uno con un agujero en el

centro para que la varilla pueda deslizarse a través de él. La configuración original de los discos es que están apilados en una de las torres en orden de tamaño descendente (es decir, el disco más grande está en la parte inferior). El objetivo del problema es mover todos los discos a una varilla diferente cumpliendo con las siguientes tres reglas:

1. Solo se puede mover un disco a la vez.
2. Solo se puede mover el disco que está en la parte superior de una pila.
3. Un disco no puede colocarse encima de un disco más pequeño.

La Fig. 22 muestra los pasos de la solución al problema de la Torre de Hanoi con tres discos.

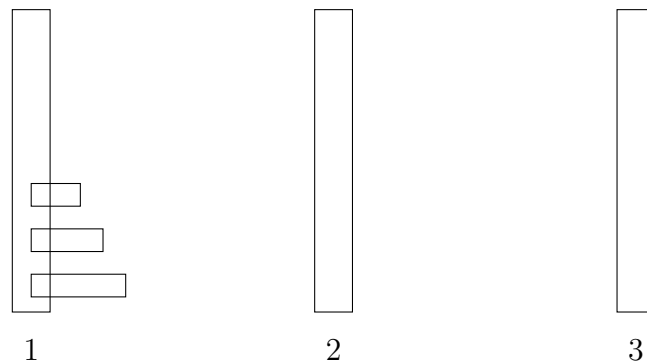


Figura 22: Ilustración de la Torre de Hanoi: En ocho pasos, todos los discos se transportan de la torre 1 a la torre 3, uno a la vez, moviendo solo el disco en la parte superior de la pila actual y colocando solo discos más pequeños encima de discos más grandes.

Hay una leyenda que dice que un grupo de monjes indios están en un monasterio trabajando para completar un problema de la Torre de Hanoi con 64 discos. Cuando completen el problema, el mundo terminará. Afortunadamente, el número de movimientos requeridos es $2^{64} - 1$, por lo que incluso si pudieran mover un disco por milisegundo, les llevaría más de 584 millones de años terminar.

La clave del problema de la Torre de Hanoi es dividirlo en problemas más pequeños y fáciles de manejar que llamaremos **subproblemas**. Para este problema, es relativamente fácil ver que mover un disco es fácil (lo que tiene

solo tres reglas), pero mover una torre es difícil y la solución no es obvia. Así que asignaremos mover una pila de tamaño N a varios subproblemas de mover una pila de tamaño $N-1$.

Considera una pila de N discos que deseamos mover de la torre 1 a la torre 3, y dejemos que `my_tower(N)` mueva una pila de tamaño N a la torre deseada (es decir, muestre los movimientos). Cómo escribir `my_tower` puede no ser inmediatamente claro. Si pensamos en el problema en términos de subproblemas, podemos ver que necesitamos mover los $N-1$ discos superiores a la torre del medio, luego el disco inferior a la torre derecha, y luego los $N-1$ discos en la torre del medio a la torre derecha. `my_tower` puede mostrar la instrucción para mover el disco N y luego hacer llamadas recursivas a `my_tower(N-1)` para manejar el movimiento de las torres más pequeñas. Las llamadas a `my_tower(N-1)` hacen llamadas recursivas a `my_tower(N-2)`, y así sucesivamente. Un desglose de los tres pasos se representa en la Fig. 23.

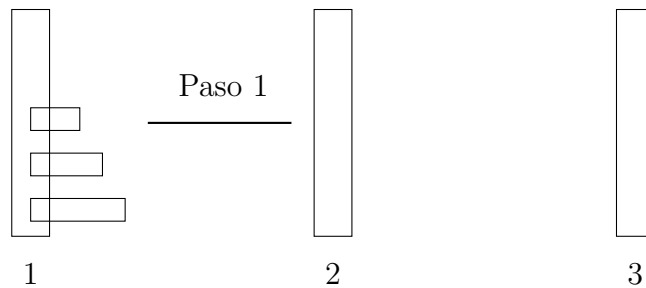


Figura 23: Desglose de una iteración de la solución recursiva del problema de la Torre de Hanoi.

El código que se reproduce a continuación es una solución recursiva al problema de la Torre de Hanoi. Observa su compacidad y simplicidad. El código refleja exactamente nuestra intuición sobre la naturaleza recursiva de la solución: primero movemos una pila de tamaño $N-1$ desde la "torre_origen.original" a la "torre_alternativa". Esta es una tarea difícil, así que en su lugar hacemos una llamada recursiva que hará llamadas recursivas subsiguientes, pero al final, moverá la pila como se desea. Luego movemos el disco inferior a la "torre_destino.objetivo". Finalmente, movemos la pila de tamaño $N-1$ a la torre de destino haciendo otra llamada recursiva.

```

1 In [1]: def my_towers(N, from_tower, to_tower, alt_tower):
2         """
3         Displays the moves required to move a tower of size

```

```

4     N from the "from_tower" to the "to_tower".
5     "from_tower", "to_tower" and "alt_tower" are
6     either 1, 2, 3 referring to tower 1, 2, and 3.
7     """
8     if N != 0:
9         # recursive call that moves N-1 stack from
10        # starting tower to alternate tower
11        my_towers(N-1, from_tower, alt_tower, to_tower)
12
13        # display to screen movement of bottom disk from
14        # starting tower to final tower
15        print("Move disk %d from tower %d to tower %d." \
16              % (N, from_tower, to_tower))
17
18        # recursive call that moves N-1 stack from
19        # alternate tower to final tower
20        my_towers(N-1, alt_tower, to_tower, from_tower)
21
22 In [2]: my_towers(3, 1, 3, 2)
23 Move disk 1 from tower 1 to tower 3.
24 Move disk 2 from tower 1 to tower 2.
25 Move disk 1 from tower 3 to tower 2.
26 Move disk 3 from tower 1 to tower 3.
27 Move disk 1 from tower 2 to tower 1.
28 Move disk 2 from tower 2 to tower 3.
29 Move disk 1 from tower 1 to tower 3.

```

Listing 256: Solución recursiva de la Torre de Hanoi.

Al usar divide y vencerás, hemos resuelto el problema de la Torre de Hanoi haciendo llamadas recursivas a los problemas de Torre de Hanoi ligeramente más pequeños que, a su vez, hacen llamadas recursivas a problemas aún más pequeños de la Torre de Hanoi. Juntas, las soluciones forman la solución a todo el problema. El trabajo real realizado por una sola llamada a la función es en realidad bastante pequeño: dos llamadas recursivas y mover un disco. En otras palabras, una llamada a la función hace muy poco trabajo (mover un disco) y luego pasa el resto del trabajo a otras llamadas, una habilidad que te será muy útil a lo largo de tu carrera de ingeniería.

Quicksort

Una lista de números, **A**, está **ordenada** si los elementos están dispuestos en orden ascendente o descendente. Aunque hay muchas formas de ordenar

una lista, **quicksort** es un enfoque de divide y vencerás, que es un algoritmo muy rápido para ordenar usando un solo procesador (hay algoritmos más rápidos para múltiples procesadores).

El algoritmo quicksort comienza con la observación de que, aunque ordenar una lista es difícil, una comparación es relativamente fácil. Así que, en lugar de ordenar una lista, separamos la lista comparando con un **pivote**. En cada llamada recursiva a quicksort, la lista de entrada se divide en tres partes: elementos que son más pequeños que el pivote, elementos que son iguales al pivote y elementos que son más grandes que el pivote. Luego se hace una llamada recursiva a quicksort en los dos subproblemas: la lista de elementos más pequeños que el pivote y la lista de elementos más grandes que el pivote. Finalmente, los subproblemas son lo suficientemente pequeños (es decir, lista de tamaño 1 o 0) como para que ordenar la lista sea ahora trivial.

Considera la siguiente implementación recursiva de quicksort.

```
1 In [3]: def my_quicksort(lst):
2     if len(lst) <= 1:
3         # list of length 1 is easiest to sort
4         # because it is already sorted
5         sorted_list = lst
6     else:
7         # select pivot as the first element of the list
8         pivot = lst[0]
9
10        # initialize lists for bigger and smaller
11        # elements as well those equal to the pivot
12        bigger = []
13        smaller = []
14        same = []
15
16        # put elements into appropriate array
17        for item in lst:
18            if item > pivot:
19                bigger.append(item)
20            elif item < pivot:
21                smaller.append(item)
22            else:
23                same.append(item)
24
25        sorted_list = my_quicksort(smaller) + same +
26        my_quicksort(bigger)
```



```
27     return sorted_list
28
29 In [4]: my_quicksort([2, 1, 3, 5, 6, 3, 8, 10])
30 Out[4]: [1, 2, 3, 3, 5, 6, 8, 10]
```

Listing 257: Implementación recursiva de Quicksort.

Como hicimos con el problema de la Torre de Hanoi, hemos dividido el problema de ordenar (difícil) en muchas comparaciones (fáciles).

Resumen y Problemas

Resumen

1. Una función recursiva es una función que se llama a sí misma.
2. Las funciones recursivas son útiles cuando los problemas tienen una estructura jerárquica en lugar de una estructura iterativa.
3. Divide y vencerás es una poderosa estrategia de resolución de problemas que se puede utilizar para resolver problemas difíciles.

Problemas y Soluciones

1. **Problema 1:** Suma recursiva de una lista.

Solución: El caso base es una lista vacía (suma 0). El paso recursivo suma el primer elemento con la suma del resto de la lista.

```
1 def my_sum(lst):
2     if not lst: # lista vacia
3         return 0
4     else:
5         return lst[0] + my_sum(lst[1:])
6
7 print(my_sum([1, 2, 3]))
8 print(my_sum(range(1, 101)))
9 # Salida: 6, 5050
10
```

Listing 258: Solución Problema 1

2. Problema 2: Polinomios de Chebyshev de primera especie.

Solución: Implementamos la relación de recurrencia $T_0(x) = 1$, $T_1(x) = x$, $T_n(x) = 2xT_{n-1}(x) - T_{n-2}(x)$. Para manejar listas, iteramos sobre los elementos.

```

1 def my_chebyshev_poly1(n, x):
2     if n == 0:
3         return [1] * len(x)
4     elif n == 1:
5         return x[:]
6     else:
7         T_prev2 = [1] * len(x) # T_0
8         T_prev1 = x[:]          # T_1
9         for k in range(2, n+1):
10             T_curr = [2 * x[i] * T_prev1[i] - T_prev2[i]
11                     for i in range(len(x))]
12             T_prev2, T_prev1 = T_prev1, T_curr
13         return T_prev1
14
15 x = [1, 2, 3, 4, 5]
16 print(my_chebyshev_poly1(0, x))
17 print(my_chebyshev_poly1(1, x))
18 print(my_chebyshev_poly1(3, x))

```

Listing 259: Solución Problema 2

3. Problema 3: Función de Ackermann.

Solución: Implementamos directamente la definición recursiva. Debemos tener cuidado con la profundidad de recursión para valores grandes.

```

1 def my_ackermann(m, n):
2     if m == 0:
3         return n + 1
4     elif n == 0:
5         return my_ackermann(m - 1, 1)
6     else:
7         return my_ackermann(m - 1, my_ackermann(m, n - 1))
8

```

```

9 print(my_ackermann(1, 1))
10 print(my_ackermann(1, 2))
11 print(my_ackermann(2, 3))
12 print(my_ackermann(3, 3))
13 print(my_ackermann(3, 4))
14 # Salida: 3, 4, 9, 61, 125
15

```

Listing 260: Solución Problema 3

4. Problema 4: Coeficiente binomial "n choose k".

Solución: Implementamos la relación de recurrencia $C(n, k) = C(n - 1, k) + C(n - 1, k - 1)$ con casos base $C(n, 0) = 1$ y $C(n, n) = 1$.

```

1 def my_n_choose_k(n, k):
2     if k == 0 or k == n:
3         return 1
4     else:
5         return my_n_choose_k(n - 1, k) + my_n_choose_k(n
6             - 1, k - 1)
7
8 print(my_n_choose_k(10, 1))
9 print(my_n_choose_k(10, 10))
10 print(my_n_choose_k(10, 3))
11 # Salida: 10, 1, 120

```

Listing 261: Solución Problema 4

5. Problema 5: Algoritmo recursivo para dar cambio.

Solución: Definimos una lista de denominaciones. El caso base es cuando la diferencia es cero. El paso recursivo resta la denominación más grande posible y llama de nuevo.

```

1 def my_change(cost, paid):
2     denominations = [100, 50, 20, 10, 5, 1, 0.25, 0.10,
3         0.05, 0.01]
4     change_due = round(paid - cost, 2)
5     if change_due == 0:
6         return []

```

```

6     for d in denominations:
7         if change_due >= d:
8             return [d] + my_change(cost + d, paid)
9
10    print(my_change(27.57, 100))
11    # Salida: [50, 20, 1, 1, 0.25, 0.10, 0.05, 0.01, 0.01,
12           0.01]

```

Listing 262: Solución Problema 5

6. **Problema 6:** Aproximación recursiva de la razón áurea mediante fracción continua.

Solución: La fracción continua se puede definir como $\phi_n = 1 + \frac{1}{\phi_{n-1}}$ con $\phi_1 = 1$.

```

1    def my_golden_ratio(n):
2        if n == 1:
3            return 1
4        else:
5            return 1 + 1 / my_golden_ratio(n - 1)
6
7    print(my_golden_ratio(10))
8    # Salida: 1.618181818181818
9

```

Listing 263: Solución Problema 6

7. **Problema 7:** Máximo común divisor (GCD) recursivo.

Solución: Implementamos el algoritmo de Euclides: $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$ hasta que $b = 0$.

```

1    def my_gcd(a, b):
2        if b == 0:
3            return a
4        else:
5            return my_gcd(b, a % b)
6
7    print(my_gcd(10, 4))
8    print(my_gcd(33, 121))
9    print(my_gcd(18, 1))

```

```

10 # Salida: 2, 11, 1
11

```

Listing 264: Solución Problema 7

8. Problema 8: Fila del triángulo de Pascal.

Solución: Usamos la relación recursiva $R_m(i) = R_{m-1}(i-1) + R_{m-1}(i)$. La fila se construye a partir de la fila anterior.

```

1 def my_pascal_row(m):
2     if m == 1:
3         return [1]
4     else:
5         prev_row = my_pascal_row(m - 1)
6         row = [1]
7         for i in range(1, m - 1):
8             row.append(prev_row[i - 1] + prev_row[i])
9         row.append(1)
10        return row
11
12 print(my_pascal_row(1))
13 print(my_pascal_row(2))
14 print(my_pascal_row(3))
15 print(my_pascal_row(4))
16 print(my_pascal_row(5))
17 # Salida: [1], [1,1], [1,2,1], [1,3,3,1], [1,4,6,4,1]
18

```

Listing 265: Solución Problema 8

9. Problema 9: Matriz de espiral recursiva.

Solución: La función recursiva rellena el borde exterior y luego llama recursivamente para la submatriz interior rota-

```

1 import numpy as np
2
3 def my_spiral_ones(n):
4     A = np.zeros((n, n), dtype=int)
5     def fill_spiral(start, size, offset):
6         if size <= 0:

```

```

7         return
8     if size == 1:
9         A[start, start] = 1
10        return
11    # top row
12    for j in range(size):
13        A[start, start + j] = 1
14    # right column
15    for i in range(1, size):
16        A[start + i, start + size - 1] = 1
17    # bottom row (if size > 2)
18    if size > 2:
19        for j in range(size - 2, 0, -1):
20            A[start + size - 1, start + j] = 1
21    # left column (if size > 3)
22    if size > 3:
23        for i in range(size - 2, 1, -1):
24            A[start + i, start] = 1
25    fill_spiral(start + 2, size - 4, offset + 1)
26
27    fill_spiral(0, n, 0)
28    return A
29
30 print(my_spiral_ones(5))
31

```

Listing 266: Solución Problema 9

10. Problema 10: Versión iterativa de la matriz espiral.

Solución: Usamos un enfoque de llenado por capas con bucles.

```

1 import numpy as np
2
3 def my_spiral_ones_iterative(n):
4     A = np.zeros((n, n), dtype=int)
5     start = 0
6     size = n
7     while size > 0:
8         if size == 1:
9             A[start, start] = 1
10            break
11        # top row

```

```

12     for j in range(size):
13         A[start, start + j] = 1
14     # right column
15     for i in range(1, size):
16         A[start + i, start + size - 1] = 1
17     # bottom row
18     if size > 2:
19         for j in range(size - 2, 0, -1):
20             A[start + size - 1, start + j] = 1
21     # left column
22     if size > 3:
23         for i in range(size - 2, 1, -1):
24             A[start + i, start] = 1
25     start += 2
26     size -= 4
27     return A
28
29 print(my_spiral_ones_iterative(5))
30

```

Listing 267: Solución Problema 10

11. **Problema 11:** Árbol de recursión para `my_towers(4)`.

Solución: El árbol muestra cómo se descompone el problema. (Solución conceptual/dibujo).

12. **Problema 12:** Versión iterativa de la Torre de Hanoi.

Solución: Existe un algoritmo iterativo basado en la paridad del número de discos, pero es más complejo. Una solución común usa una pila explícita para simular la recursión.

```

1 def my_towers_iterative(N, from_tower, to_tower,
2   alt_tower):
3     # Implementacion iterativa usando una pila
4     stack = [(N, from_tower, to_tower, alt_tower, False)]
5     while stack:
6         n, f, t, a, moved = stack.pop()
7         if n == 0:
8             continue
9         if not moved:
10            stack.append((n, f, t, a, True))

```

```

10         stack.append((n-1, a, t, f, False))
11     else:
12         print(f"Move disk {n} from tower {f} to tower
13         {t}")
14         stack.append((n-1, f, a, t, False))

```

Listing 268: Solución Problema 12 (esqueleto)

13. **Problema 13:** Árbol de recursión para `my_quicksort([5,4,6,2,9,1,7,3])`.

Solución: El árbol muestra cómo se particiona la lista en cada paso. (Solución conceptual/dibujo).

14. **Problema 14:** Versión iterativa de Quicksort.

Solución: Usamos una pila explícita para simular las llamadas recursivas.

```

1 def my_quicksort_iterative(lst):
2     if len(lst) <= 1:
3         return lst
4     stack = [(0, len(lst) - 1)]
5     arr = lst[:]
6     while stack:
7         low, high = stack.pop()
8         if low < high:
9             pivot = arr[low]
10            left = low + 1
11            right = high
12            while True:
13                while left <= right and arr[left] <=
pivot:
14                    left += 1
15                while left <= right and arr[right] >=
pivot:
16                    right -= 1
17                if left <= right:
18                    arr[left], arr[right] = arr[right],
arr[left]
19            else:
20                break
21            arr[low], arr[right] = arr[right], arr[low]

```



```
22         stack.append((low, right - 1))
23         stack.append((right + 1, high))
24     return arr
25
```

Listing 269: Solución Problema 14

PROGRAMACIÓN ORIENTADA A OBJETOS

Introducción a la POO

Hasta ahora, todos los códigos que hemos escrito pertenecen a la categoría de **programación orientada a procedimientos (POP)**, que consiste en una lista de instrucciones para decirle a la computadora qué hacer; estas instrucciones luego se organizan en funciones. El programa se divide en una colección de variables, estructuras de datos y rutinas para realizar diferentes tareas. Python es un lenguaje de programación multiparadigma, es decir, soporta diferentes enfoques de programación. Una forma de programar en Python es usar la **programación orientada a objetos (POO)**. La curva de aprendizaje es más pronunciada, pero es extremadamente poderosa y vale la pena el tiempo invertido en dominarla. Ten en cuenta que no tienes que usar POO cuando programas en Python. Aún puedes escribir programas muy potentes usando POP. Dicho esto, la POP es buena para programas simples y pequeños, mientras que la POO es más adecuada para programas grandes. Echemos un vistazo más de cerca a la programación orientada a objetos.

La programación orientada a objetos divide la tarea de programación en **objetos**, que combinan datos (conocidos como **atributos**) y comportamientos/funciones (conocidos como **métodos**). Por lo tanto, hay dos componentes principales de la POO: la **clase** y el **objeto**.

La **clase** es un modelo o plano (blueprint) para definir una agrupación lógica de datos y funciones. Proporciona una forma de crear estructuras de datos que modelan entidades del mundo real. Por ejemplo, podemos crear una clase **Persona** que contenga datos como nombre, edad y algunas funciones de comportamiento para imprimir edades y géneros de un grupo de personas. Mientras que la clase es el modelo, un **objeto** es una **instancia** de la clase con

valores reales. Por ejemplo, una persona llamada Iron Man con edad 35. Dicho de otra manera, una clase es como una plantilla para definir la información necesaria, y un objeto es una copia específica que completa la plantilla. Además, los objetos instanciados de la misma clase son independientes entre sí. Por ejemplo, si tenemos otra persona, digamos Batman con edad 33, puede ser instanciada desde la clase `Persona`, pero sigue siendo una instancia independiente.

Para implementar el ejemplo anterior en Python, mira el código a continuación. No te preocupes si no entiendes la sintaxis; la siguiente sección proporciona ejemplos más útiles.

```
1 In [1]: class People():
2         def __init__(self, name, age):
3             self.name = name
4             self.age = age
5
6         def greet(self):
7             print("Greetings, " + self.name)
8
9 In [2]: person1 = People(name = "Iron Man", age = 35)
10         person1.greet()
11         print(person1.name)
12         print(person1.age)
13 Greetings, Iron Man
14 Iron Man
15 35
16
17 In [3]: person2 = People(name = "Batman", age = 33)
18         person2.greet()
19         print(person2.name)
20         print(person2.age)
21 Greetings, Batman
22 Batman
23 33
```

Listing 1: Definición básica de una clase y creación de objetos.

En el ejemplo de código anterior, primero definimos una clase, `People`, con `name` y `age` como datos, y un método `greet`. Luego instanciamos un objeto, `person1` con el nombre y la edad específicos. Podemos ver claramente que la clase define toda la estructura, mientras que el objeto es solo una instancia de la clase.

Los muchos beneficios de usar POO son los siguientes: Proporciona una estructura modular clara para los programas que mejora la reutilización del

código. Proporciona una forma simple de resolver problemas complejos. Ayuda a definir tipos de datos más abstractos para modelar escenarios del mundo real. Oculta los detalles de implementación, dejando una interfaz claramente definida. Combina datos y operaciones.

También hay otras ventajas de usar POO en un proyecto grande. Te animamos a buscar en línea para descubrir más. En este punto, es posible que aún no comprendas completamente las ventajas de la POO hasta que te involucres en proyectos grandes y complejos. Continuaremos aprendiendo más sobre POO durante el transcurso de este libro, y su utilidad se hará evidente.

Clase y Objeto

La sección anterior introdujo los dos componentes principales de la POO: la **clase**, que es un modelo utilizado para definir una agrupación lógica de datos y funciones, y el **objeto**, que es una instancia de la clase definida con valores reales. En esta sección, profundizaremos en ambos componentes.

Clase

Una clase es una definición de la estructura que queremos. Similar a una función, se define como un bloque de código, comenzando con la declaración `class`. La sintaxis para definir una clase es:

```
1 class NombreDeLaClase(superclase):  
2     def __init__(self, argumentos):  
3         # definir o asignar atributos del objeto  
4  
5     def otros_metodos(self, argumentos):  
6         # cuerpo del metodo
```

Listing 2: Sintaxis para definir una clase.

Observa que la definición de una clase es muy similar a una función. Necesita ser instanciada primero antes de poder usarla. Para el nombre de la clase, es una convención estándar usar "PalabrasEnMayusculas" (CapWords). La **superclase** se usa cuando quieres crear una nueva clase para heredar los atributos y métodos de otra clase ya definida. Hablaremos más sobre la herencia en la próxima sección. El método `__init__` es uno de los métodos especiales en las clases de Python que se ejecuta tan pronto como se instancia (crea) un objeto de una clase. Asigna valores iniciales al objeto antes de que

esté listo para ser usado. Observa los dos guiones bajos al principio y al final de `init`, lo que indica que este es un método especial reservado para uso especial en el lenguaje. En este método `init`, puedes asignar atributos directamente cuando creas el objeto. Las funciones `otros_metodos` se utilizan para definir los métodos de instancia que se aplicarán a los atributos, al igual que las funciones que discutimos antes. Puedes notar que hay un argumento `self` para definir este método en la clase. ¿Por qué? Un método de instancia de clase debe tener este argumento adicional como primer argumento cuando lo defines. Este argumento particular se refiere al objeto mismo; convencionalmente, usamos `self` para nombrarlo. Los métodos de instancia pueden acceder libremente a los atributos y otros métodos en el mismo objeto usando este parámetro `self`. Ve el ejemplo a continuación.

```
1 In [1]: class Student():
2     def __init__(self, sid, name, gender):
3         self.sid = sid
4         self.name = name
5         self.gender = gender
6         self.type = "learning"
7
8     def say_name(self):
9         print("My name is " + self.name)
```

Listing 3: Definiendo una clase `Student`.

Del ejemplo anterior, podemos ver que esta clase simple contiene todas las partes necesarias mencionadas anteriormente. El método `__init__` inicializará los atributos cuando creamos un objeto. Necesitamos pasar el valor inicial para `sid`, `name` y `gender`, mientras que el atributo `type` es un valor fijo como `"learning"`. Estos atributos pueden ser accedidos por todos los demás métodos definidos en la clase con `self.atributo`, por ejemplo, en el método `say_name`, podemos usar el atributo `name` con `self.name`. Los métodos definidos en la clase también pueden ser accedidos y utilizados en otros métodos diferentes usando `self.metodo`. Veamos el siguiente ejemplo.

```
1 In [2]: class Student():
2     def __init__(self, sid, name, gender):
3         self.sid = sid
4         self.name = name
5         self.gender = gender
6         self.type = "learning"
7
8     def say_name(self):
```

```
9         print("My name is " + self.name)
10
11     def report(self, score):
12         self.say_name()
13         print("My id is: " + self.sid)
14         print("My score is: " + str(score))
```

Listing 4: Añadiendo un método `report` a la clase `Student`.

Objeto

Como se mencionó anteriormente, un **objeto** es una instancia de la clase definida con valores reales. Son posibles muchas instancias de diferentes valores asociados con la clase, y cada una de estas instancias será independiente entre sí (como se vio anteriormente). Además, después de crear un objeto y llamar a este método de instancia desde el objeto, no necesitamos darle valor al parámetro `self` porque Python lo proporciona automáticamente; ve el siguiente ejemplo.

```
1 In [3]: student1 = Student("001", "Susan", "F")
2         student2 = Student("002", "Mike", "M")
3
4         student1.say_name()
5         student2.say_name()
6         print(student1.type)
7         print(student1.gender)
8 My name is Susan
9 My name is Mike
10 learning
11 F
```

Listing 5: Creando objetos de la clase `Student`.

En el código anterior, creamos dos objetos, `student1` y `student2`, con dos conjuntos de valores diferentes. Cada objeto es una instancia de la clase `Student` y tiene un conjunto diferente de atributos. Escribe `student1. + TAB` para ver los atributos y métodos definidos. Para acceder a un atributo, escribe `objeto.atributo`, por ejemplo, `student1.type`. En contraste, para llamar a un método, necesitas los paréntesis porque estás llamando a una función, como `student1.say_name()`.

```
1 In [4]: student1.report(95)
2         student2.report(90)
3 My name is Susan
```

```
4 My id is: 001
5 My score is: 95
6 My name is Mike
7 My id is: 002
8 My score is: 90
```

Listing 6: Llamando al método `report`.

Podemos ver que ambos métodos llaman para imprimir los datos asociados con los dos objetos. Observa que el valor de `score` que pasamos solo está disponible para el método `report` (dentro del alcance de este método). También podemos ver que la llamada al método `say_name` en `report` también funciona, siempre y cuando llames al método con el `self` en él.

Atributos de Clase vs. Atributos de Instancia

Los atributos que presentamos anteriormente se llaman en realidad **atributos de instancia**, lo que significa que solo pertenecen a una instancia específica; cuando los usas, necesitas usar `self.atributo` dentro de la clase. Hay otros atributos llamados **atributos de clase**, que serán compartidos con todas las instancias creadas a partir de esta clase. Veamos un ejemplo de cómo definir y usar un atributo de clase.

```
1 In [5]: class Student():
2         n = 0
3
4         def __init__(self, sid, name, gender):
5             self.sid = sid
6             self.name = name
7             self.gender = gender
8             self.type = "learning"
9             Student.n += 1
10
11        def say_name(self):
12            print("My name is " + self.name)
13
14        def report(self, score):
15            self.say_name()
16            print("My id is: " + self.sid)
17            print("My score is: " + str(score))
18
19        def num_instances(self):
20            print(f"We have {Student.n}-instance in total")
```

Listing 7: Añadiendo un atributo de clase a `Student`.

Al definir un atributo de clase, debemos definirlo fuera de todos los otros métodos sin usar `self`. Para usar los atributos de clase, usamos `NombreClase.atributo`, que en este caso es `Student.n`. Este atributo será compartido con todas las instancias que se creen a partir de esta clase. Veamos el siguiente código para mostrar la idea.

```
1 In [6]: student1 = Student("001", "Susan", "F")
2         student1.num_instances()
3         student2 = Student("002", "Mike", "M")
4         student1.num_instances()
5         student2.num_instances()
6 We have 1-instance in total
7 We have 2-instance in total
8 We have 2-instance in total
```

Listing 8: Usando el atributo de clase.

Como antes, creamos dos objetos, los atributos de instancia `sid`, `name` y `gender` solo pertenecen al objeto específico. Por ejemplo, `student1.name` es "Susan" `student2.name` es "Mike". Pero cuando imprimimos el atributo de clase `Student.n` después de haber creado el objeto `student2`, el valor en `student1` también cambia. Esta es la expectativa que tenemos para el atributo de clase porque se comparte entre todos los objetos creados.

Ahora que entendemos la diferencia entre clase e instancia, estamos en buena forma para usar la POO básica en Python. Antes de poder aprovechar al máximo la POO, aún necesitamos entender el concepto de herencia, encapsulación y polimorfismo. ¡Comencemos la próxima sección!

Herencia, Encapsulación y Polimorfismo

Ya hemos visto el poder de modelado de la POO utilizando las funciones de clase y objeto combinando datos y métodos. Hay tres conceptos más importantes: (1) **herencia**, que hace que el código POO sea más modular, más fácil de reutilizar y capaz de construir una relación entre clases; (2) **encapsulación**, que puede ocultar algunos de los detalles privados de una clase de otros objetos; y (3) **polimorfismo**, que nos permite usar una operación común de diferentes maneras. A continuación se presenta una breve discusión de estos conceptos.

Herencia

La **herencia** nos permite definir una clase que hereda todos los métodos y atributos de otra clase. La convención denota la nueva clase como **clase hija**, y la clase de la que hereda se llama **clase padre** o **superclase**. Si volvemos a la definición de la estructura de la clase, podemos ver que la estructura para la herencia básica es `class NombreClase(superclase)`, lo que significa que la nueva clase puede acceder a todos los atributos y métodos de la superclase. La herencia construye una relación entre las clases hija y padre. Por lo general, la clase padre es un tipo general mientras que la clase hija es un tipo específico. A continuación se presenta un ejemplo.

```
1 In [1]: class Sensor():
2     def __init__(self, name, location, record_date):
3         self.name = name
4         self.location = location
5         self.record_date = record_date
6         self.data = {}
7
8     def add_data(self, t, data):
9         self.data["time"] = t
10        self.data["data"] = data
11        print(f"We have {len(data)} points saved")
12
13    def clear_data(self):
14        self.data = {}
15        print("Data cleared!")
```

Listing 9: Definiendo una clase `Sensor`.

Ahora que tenemos una clase para almacenar información general del sensor, podemos crear un objeto sensor para almacenar datos.

```
1 In [2]: import numpy as np
2
3         sensor1 = Sensor("sensor1", "Berkeley", "2019-01-01")
4         data = np.random.randint(-10, 10, 10)
5         sensor1.add_data(np.arange(10), data)
6         sensor1.data
7 We have 10 points saved
8 Out[2]: {"time": array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]),
9         "data": array([ 3, 2, 5, -1, 2, -2, 6, -1, 5, 4])}
```

Listing 10: Creando un objeto `Sensor`.

Heredando y Extendiendo un Nuevo Método

Supongamos que tenemos un tipo diferente de sensor, un acelerómetro. Comparte los mismos atributos y métodos que la clase `Sensor`, pero también tiene diferentes atributos o métodos que deben ser añadidos o modificados respecto a la clase original. ¿Qué debemos hacer? ¿Creamos una clase diferente desde cero? Aquí es donde la herencia se puede usar para facilitar la vida. Esta nueva clase heredará de la clase `Sensor` con todos los atributos y métodos. Podemos pensar si queremos extender los atributos o métodos. Primero creamos esta nueva clase, `Accelerometer`, y añadamos un nuevo método, `show_type`, para informar qué tipo de sensor es.

```
1 In [3]: class Accelerometer(Sensor):
2         def show_type(self):
3             print("I am an accelerometer!")
4
5         acc = Accelerometer("acc1", "Oakland", "2019-02-01")
6         acc.show_type()
7         data = np.random.randint(-10, 10, 10)
8         acc.add_data(np.arange(10), data)
9         acc.data
10 I am an accelerometer!
11 We have 10 points saved
12 Out[3]: {"time": array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]),
13         "data": array([-1, 4, 7, -10, -2, -6, 2, -8, 9, 3])
14         }
```

Listing 11: Herencia y extensión de métodos.

Crear esta nueva clase `Accelerometer` es muy simple. Hereda de `Sensor` (denotada como una superclase), y la nueva clase realmente contiene todos los atributos y métodos de la superclase. Luego añadimos un nuevo método, `show_type`, que no existe en la clase `Sensor`, pero podemos extender con éxito la clase hija añadiendo el nuevo método. Esto muestra el poder de la herencia: hemos reutilizado la mayor parte de la clase `Sensor` en una nueva clase y hemos extendido la funcionalidad. Básicamente, la herencia establece una relación lógica para el modelado de entidades del mundo real: la clase `Sensor` como clase padre es más general y pasa todas las características a la clase hija `Accelerometer`.

Heredando y Sobrescribiendo Métodos (Method Overriding)

Cuando heredamos de una clase padre, podemos cambiar la implementación de un método proporcionado por la clase padre; esto se llama **sobrescritura de métodos (method overriding)** y se muestra en el ejemplo a continuación.

```
1 In [4]: class UCBacc(Accelerometer):
2         def show_type(self):
3             print(f"I am {self.name}, created at Berkeley!")
4
5         acc2 = UCBacc("UCBacc", "Berkeley", "2019-03-01")
6         acc2.show_type()
7 I am UCBacc, created at Berkeley!
```

Listing 12: Sobrescritura de métodos.

Vemos que nuestra nueva clase `UCBacc` realmente sobrescribe el método `show_type` con nuevas características. En este ejemplo, no solo estamos heredando características de nuestra clase padre, sino que también estamos modificando/mejorando algunos métodos.

Heredando y Actualizando Atributos con super

Creemos una clase `NewSensor` que hereda de la clase `Sensor`, pero actualiza los atributos añadiendo un nuevo atributo `brand`. Por supuesto, podemos redefinir todo el método `__init__` como se muestra a continuación, que es capaz de sobrescribir la función padre.

```
1 In [5]: class NewSensor(Sensor):
2         def __init__(self, name, location, record_date, brand):
3             self.name = name
4             self.location = location
5             self.record_date = record_date
6             self.brand = brand
7             self.data = {}
8
9         new_sensor = NewSensor("OK", "SF", "2019-03-01", "XYZ")
10        new_sensor.brand
11 Out [5]: "XYZ"
```

Listing 13: Redefiniendo `__init__` sin usar `super`.

Hay una mejor manera de lograr el mismo resultado. Si usamos el método `super`, podemos evitar referirnos a la clase padre explícitamente, como se muestra en el siguiente ejemplo:

```
1 In [6]: class NewSensor(Sensor):
2         def __init__(self, name, location, record_date, brand):
3             super().__init__(name, location, record_date)
4             self.brand = brand
5
6         new_sensor = NewSensor("OK", "SF", "2019-03-01", "XYZ
7         ")
8         new_sensor.brand
8 Out [6]: "XYZ"
```

Listing 14: Usando `super` para llamar al `__init__` de la clase padre.

Ahora podemos ver que con el método `super`, hemos evitado enumerar toda la definición de los atributos; esto ayuda a mantener tu código mantenible en el futuro previsible. Debido a que la clase hija no llama implícitamente al `__init__` de la clase padre, debemos usar `super().__init__`, como se muestra arriba.

Encapsulación

La **encapsulación** es uno de los conceptos fundamentales en la POO. Describe la idea de restringir el acceso a métodos y atributos en una clase. La encapsulación oculta los detalles complejos de los usuarios y evita que los datos sean modificados accidentalmente. En Python, esto se logra utilizando métodos o atributos privados con un guion bajo como prefijo, es decir, un solo guion bajo `"_"` o doble guion bajo `"__"`, como se muestra en el siguiente ejemplo.

```
1 In [7]: class Sensor():
2         def __init__(self, name, location):
3             self.name = name
4             self._location = location
5             self.__version = "1.0"
6
7         # a getter function
8         def get_version(self):
9             print(f"The sensor version is {self.__version}")
10
11        # a setter function
12        def set_version(self, version):
```

```

13         self.__version = version
14
15 In [8]: sensor1 = Sensor("Acc", "Berkeley")
16         print(sensor1.name)
17         print(sensor1.__location)
18         print(sensor1.__version)
19 Acc
20 Berkeley
21 -----
22 AttributeError Traceback (most recent call last)
23 <ipython-input-8-ca9b481690ba> in <module>
24     2 print(sensor1.name)
25     3 print(sensor1.__location)
26 ----> 4 print(sensor1.__version)
27 AttributeError: 'Sensor' object has no attribute '__version'

```

Listing 15: Encapsulación con atributos privados.

El ejemplo anterior muestra cómo funciona la encapsulación. Con un solo guion bajo, definimos una variable privada a la que no se debe acceder directamente. Observa que esto es una convención y nada te impide acceder a ella realmente. Con dobles guiones bajos, podemos ver que el atributo `__version` no se puede acceder ni modificar directamente. Para acceder a los atributos con doble guion bajo, necesitamos usar funciones "getter" y "setter" para acceder internamente. A continuación se muestra un ejemplo de una función "getter".

```

1 In [9]: sensor1.get_version()
2 The sensor version is 1.0
3
4 In [10]: sensor1.set_version("2.0")
5           sensor1.get_version()
6 The sensor version is 2.0

```

Listing 16: Usando getter y setter.

El guion bajo simple y doble también se aplican a los métodos privados, pero no se discuten porque son similares a los atributos privados.

Polimorfismo

El **polimorfismo** es otro concepto fundamental en la POO, que significa "múltiples formas". El polimorfismo permite el uso de una única interfaz con diferentes formas subyacentes, como tipos de datos o clases. Por ejemplo,

podemos tener métodos con nombres comunes en distintas clases o clases hijas. Ya hemos visto un ejemplo arriba cuando sobrescribimos el método `show_type` en `UCBAcc`. Tanto la clase padre `Accelerometer` como la clase hija `UCBAcc` tienen un método llamado `show_type`, pero se implementan de manera diferente. Esta habilidad de usar un solo nombre con muchas formas que actúan de manera diferente en distintas situaciones reduce en gran medida nuestras complejidades. No ampliaremos esta discusión sobre el polimorfismo; si estás interesado, busca más en línea para obtener una comprensión más profunda.

Resumen y Problemas

Resumen

1. La POO y la POP son diferentes. La POO tiene muchos beneficios y a menudo es más apropiada para su uso en proyectos de gran escala.
2. La clase es el modelo de la estructura que nos permite agrupar datos y métodos, mientras que el objeto es una instancia de la clase.
3. El concepto de "herencia" es clave en la POO, que nos permite referirnos a atributos o métodos de la superclase.
4. El concepto de "encapsulación" nos permite ocultar algunos de los detalles privados de una clase de otros objetos.
5. El concepto de "polimorfismo" nos permite usar una operación común de diferentes maneras para diferentes entradas de datos.

Problemas y Soluciones

1. **Problema 1:** Describe las diferencias entre clases y objetos.

Solución: Una clase es un modelo o plantilla que define la estructura y el comportamiento (atributos y métodos) que tendrán los objetos. Un objeto es una instancia concreta de una clase, con valores específicos para los atributos definidos por la clase.

2. **Problema 2:** Describe por qué usamos "self" como primer argumento en un método.

Solución: El parámetro `self` es una referencia a la instancia actual de la clase. Se utiliza para acceder a los atributos y métodos de esa instancia específica desde dentro de la clase. Cuando se llama a un método de instancia (ej. `objeto.metodo()`), Python pasa automáticamente el objeto como primer argumento (el `self`).

3. **Problema 3:** ¿Qué es un constructor? ¿Y por qué lo usamos?

Solución: En Python, el constructor es el método `__init__`. Es un método especial que se llama automáticamente cuando se crea una instancia de una clase. Se utiliza para inicializar los atributos de la nueva instancia, a menudo con valores proporcionados como argumentos al crear el objeto.

4. **Problema 4:** Describe las diferencias entre atributos de clase y atributos de instancia.

Solución: Los atributos de instancia son específicos de cada objeto (instancia). Se definen típicamente dentro del método `__init__` usando `self.nombre_atributo` y su valor puede variar de una instancia a otra. Los atributos de clase son compartidos por todas las instancias de la clase. Se definen directamente dentro de la clase, fuera de cualquier método, y se accede a ellos mediante el nombre de la clase (o incluso mediante una instancia, aunque no es recomendable). Cambiar un atributo de clase afecta a todas las instancias.

5. **Problema 5:** Añade un método `plot_point` a la clase `Point` para graficar el punto.

Solución: Importamos `matplotlib.pyplot` y usamos `plt.plot` para dibujar el punto como un marcador.

```
1 import matplotlib.pyplot as plt
2
3 class Point():
4     def __init__(self, x, y):
```

```
5         self.x = x
6         self.y = y
7
8     def plot_point(self):
9         plt.plot(self.x, self.y, 'ro')
10        plt.xlabel('X')
11        plt.ylabel('Y')
12        plt.title('Punto')
13        plt.grid(True)
14        plt.show()
15
16 p = Point(3, 4)
17 p.plot_point()
18
```

Listing 17: Solución Problema 5

6. **Problema 6:** Añade un método `calculate_dist` a la clase `Point` que calcule la distancia a otro punto.

Solución: Usamos la fórmula de la distancia euclidiana. El método toma las coordenadas de otro punto como argumento.

```
1 import math
2
3 class Point():
4     def __init__(self, x, y):
5         self.x = x
6         self.y = y
7
8     def calculate_dist(self, other_x, other_y):
9         dist = math.sqrt((self.x - other_x)**2 + (self.y
10        - other_y)**2)
11         return dist
12
13 p1 = Point(0, 0)
14 dist = p1.calculate_dist(3, 4)
15 print(f"Distancia: {dist}")
16 # Salida: Distancia: 5.0
```

Listing 18: Solución Problema 6

7. **Problema 7:** ¿Qué es la herencia?

Solución: La herencia es un mecanismo de la POO que permite crear una nueva clase (clase hija) basada en una clase existente (clase padre o superclase). La clase hija hereda todos los atributos y métodos de la clase padre, pudiendo también añadir nuevos atributos o métodos, o modificar (sobrescribir) los existentes. Esto promueve la reutilización del código y la creación de jerarquías de clases.

8. **Problema 8:** ¿Cómo heredamos de una superclase y añadimos nuevos métodos?

Solución: Al definir la clase hija, se coloca el nombre de la superclase entre paréntesis. Luego, dentro de la clase hija, se pueden definir nuevos métodos normalmente. La clase hija tendrá acceso a los métodos de la superclase y también a sus propios métodos.

```
1 class Animal:
2     def hacer_sonido(self):
3         print("Sonido generico")
4
5 class Perro(Animal): # Perro hereda de Animal
6     def moverCola(self): # Nuevo metodo especifico de
7         print("El perro mueve la cola")
8
```

Listing 19: Solución Problema 8 (Ejemplo)

9. **Problema 9:** ¿Cómo reemplazamos un método de la superclase en la clase hija?

Solución: Para reemplazar (sobrescribir) un método, simplemente se define un método en la clase hija con el mismo nombre que el método de la superclase. Cuando se llame a ese método desde una instancia de la clase hija, se ejecutará la versión definida en la hija en lugar de la de la clase padre.

```
1 class Animal:
2     def hacer_sonido(self):
3         print("Sonido generico")
4
```

```
5 class Perro(Animal):
6     def hacer_sonido(self): # Sobrescritura del metodo
7         print("Guau!")
8
```

Listing 20: Solución Problema 9 (Ejemplo)

10. **Problema 10:** ¿Qué es el método `super`? ¿Por qué lo necesitamos?

Solución: `super()` es una función incorporada que devuelve un objeto proxy que permite llamar a métodos de la superclase (clase padre) desde una clase hija. Es útil cuando se quiere extender la funcionalidad de un método de la superclase en lugar de reemplazarlo por completo, por ejemplo, para llamar al constructor `__init__` de la clase padre desde el constructor de la clase hija para inicializar los atributos heredados correctamente.

11. **Problema 11:** Crear una clase para modelar un objeto del mundo real y otra clase que herede de ella.

Solución: Vamos a modelar una clase `Vehiculo` y una clase hija `Coche`. Incluiremos encapsulación (atributos privados) y sobrescritura de métodos.

```
1 class Vehiculo:
2     def __init__(self, marca, modelo):
3         self.marca = marca
4         self.modelo = modelo
5         self.__encendido = False # Atributo privado
6
7     def encender(self):
8         self.__encendido = True
9         print(f"{self.marca} {self.modelo} encendido.")
10
11    def apagar(self):
12        self.__encendido = False
13        print(f"{self.marca} {self.modelo} apagado.")
14
15    def estado(self):
16        return "encendido" if self.__encendido else "
17    apagado"
```

```
18 class Coche(Vehiculo):
19     def __init__(self, marca, modelo, num_puertas):
20         super().__init__(marca, modelo) # Llamada al
           constructor padre
21         self.num_puertas = num_puertas
22
23     def encender(self): # Sobrescritura del metodo
24         print("Insertando llave y girando...")
25         super().encender() # Llamada al metodo de la
           clase padre
26
27     def abrir_maletero(self): # Nuevo metodo especifico
           de Coche
28         print("Maletero abierto.")
29
30 # Uso de las clases
31 mi_coche = Coche("Toyota", "Corolla", 4)
32 print(f"Estado inicial: {mi_coche.estado()}")
33 mi_coche.encender()
34 print(f"Estado actual: {mi_coche.estado()}")
35 mi_coche.abrir_maletero()
36 print(f"El coche tiene {mi_coche.num_puertas} puertas.")
37
```

Listing 21: Solución Problema 11

COMPLEJIDAD

Complejidad y Notación Big-O

La **complejidad** de una función es la relación entre el tamaño de la entrada y la dificultad de ejecutar la función hasta su finalización. El tamaño de la entrada generalmente se denota por n . Sin embargo, n usualmente describe algo más tangible, como la longitud de un array. La dificultad de un problema se puede medir de varias maneras. Una forma adecuada de describir la dificultad del problema es usar **operaciones básicas**: sumas, restas, multiplicaciones, divisiones, asignaciones y llamadas a funciones. Aunque cada operación básica toma una cantidad diferente de tiempo, el número de operaciones básicas necesarias para completar una función está suficientemente relacionado con el tiempo de ejecución para ser útil, y es mucho más fácil de contar.

```
1 In [1]: def f(n):  
2     out = 0  
3     for i in range(n):  
4         for j in range(n):  
5             out += i*j  
6     return out
```

Listing 22: Función con bucles anidados.

Calculemos el número de operaciones:

- n^2 sumas (por la operación `out += i*j`);
- 0 restas;
- n^2 multiplicaciones (por `i*j`);
- 0 divisiones;

- $2n^2 + 1$ asignaciones (la línea `out += i*j` se evalúa n^2 veces, j se asigna n^2 veces, i se asigna n veces, y la línea `out = 0` se asigna una vez).
- 0 llamadas a función;
- Total: $4n^2 + n + 1$ operaciones.

Por lo tanto, la complejidad de la función f se puede describir como $4n^2 + n + 1$.

Una notación común para la complejidad se llama **notación Big-O**. La notación Big-O establece la relación en el crecimiento del número de operaciones básicas con respecto al tamaño de la entrada a medida que el tamaño de la entrada se vuelve muy grande. Debido a que el hardware puede ser diferente en cada máquina, no podemos calcular con precisión cuánto tiempo tomará completar una función sin evaluar también el hardware, lo cual solo es válido para esa máquina específica. Cuánto tiempo lleva calcular un conjunto específico de entrada en una máquina específica no es relevante. Lo que es relevante es el "tiempo hasta la finalización". Debido a que este tipo de análisis es independiente del hardware, las operaciones básicas crecen en respuesta directa al aumento en el tamaño de la entrada. A medida que n se vuelve grande, la potencia más alta domina; por lo tanto, solo el término de la potencia más alta se incluye en la notación Big-O. Adicionalmente, los coeficientes no son necesarios para caracterizar el crecimiento, por lo que también se eliminan. En el ejemplo anterior, contamos $4n^2 + n + 1$ operaciones básicas para completar la función. En notación Big-O diríamos que la función es $O(n^2)$ (pronunciado ".º de n al cuadrado"). Decimos que cualquier algoritmo con complejidad $O(n^c)$ donde c es alguna constante con respecto a n es de **tiempo polinomial**.

```

1 In [2]: def my_fib_iter(n):
2         out = [1, 1]
3         for i in range(2, n):
4             out.append(out[i - 1] + out[i - 2])
5         return out

```

Listing 23: Función de Fibonacci iterativa.

Dado que las únicas líneas de código que toman más tiempo a medida que n crece son las del bucle `for`, podemos restringir nuestra atención al bucle `for` y al bloque de código dentro de él. El código dentro del bucle `for` no crece con respecto a n (es decir, es constante). Por lo tanto, el número de operaciones básicas es Cn , donde C es alguna constante que representa el número de

operaciones básicas que ocurren en el bucle `for`, y estas C operaciones se ejecutan n veces. Esto da una complejidad de $O(n)$ para `my_fib_iter`.

Evaluar la complejidad exacta de una función puede ser difícil. En estos casos, podría ser suficiente dar una cota superior o incluso una aproximación de la complejidad.

```
1 In [3]: def my_fib_rec(n):  
2     if n < 2:  
3         out = 1  
4     else:  
5         out = my_fib_rec(n-1) + my_fib_rec(n-2)  
6     return out
```

Listing 24: Función de Fibonacci recursiva.

A medida que n se vuelve grande, podemos decir que la gran mayoría de las llamadas a función hacen otras dos llamadas a función: una suma y una asignación a la salida. La suma y la asignación no crecen con n por llamada a función, por lo que podemos ignorarlas en la notación Big-O. Sin embargo, el número de llamadas a función crece aproximadamente en 2^n , por lo que la complejidad de `my_fib_rec` está acotada superiormente por $O(2^n)$.

Dado que el número de llamadas recursivas crece exponencialmente con n , no hay manera de que la función de Fibonacci recursiva pueda ser polinomial. Es decir, para cualquier c , existe una n tal que `my_fib_rec` toma más de $O(n^c)$ operaciones básicas para completar. Cualquier función que sea $O(c^n)$ para alguna constante c se dice que es de **tiempo exponencial**.

```
1 In [4]: def my_divide_by_two(n):  
2     out = 0  
3     while n > 1:  
4         n /= 2  
5         out += 1  
6     return out
```

Listing 25: Función que divide por dos hasta que el resultado es menor que 1.

Nuevamente, solo el bucle `while` se ejecuta más tiempo para n más grandes, por lo que podemos restringir nuestra atención allí. Dentro del bucle `while`, hay dos asignaciones: una división y una suma, ambas de tiempo constante con respecto a n . Por lo tanto, la complejidad depende solo de cuántas veces se ejecuta el bucle `while`.

El bucle `while` divide n por la mitad en cada iteración hasta que n es menor que 1. Por lo tanto, el número de iteraciones, I , es la solución a la ecuación $n/2^I = 1$. Con algo de manipulación, la solución es $I = \log_2 n$, por

lo que la complejidad de `my_divide_by_two` es $O(\log n)$. Si recordamos las reglas de los logaritmos, no importa cuál sea la base del logaritmo porque todos los logaritmos son un múltiplo escalar entre sí. Cualquier función con complejidad $O(\log n)$ se dice que es de **tiempo logarítmico**.

La Importancia de la Complejidad

Entonces, ¿por qué es importante la complejidad? Porque las diferentes complejidades requieren diferentes cantidades de tiempo para completar la tarea. La Fig. 1 es un bosquejo rápido que muestra cómo cambia el tiempo con diferentes tamaños de entrada para las complejidades $\log(n)$, n y n^2 .

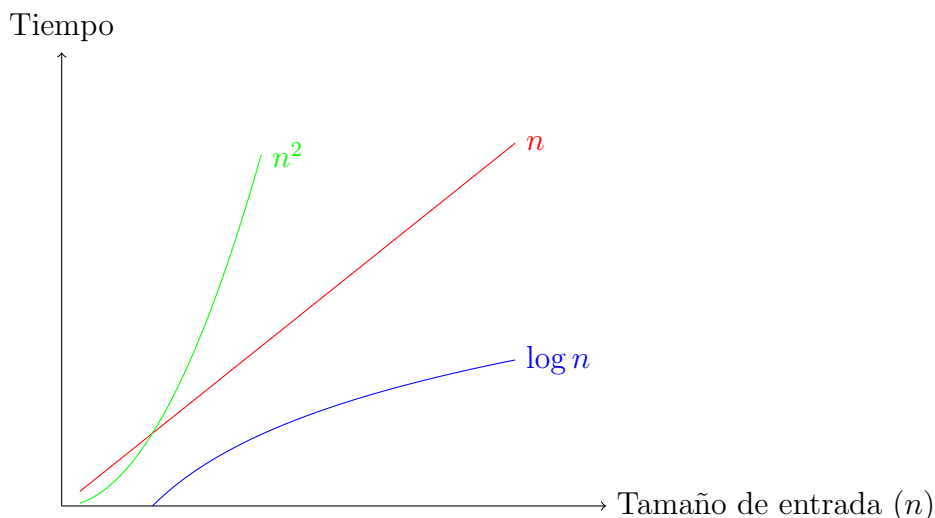


Figura 1: Ilustración del tiempo de ejecución para las complejidades $\log n$, n y n^2 .

Veamos otro ejemplo. Supongamos que tienes un algoritmo que se ejecuta en tiempo exponencial, digamos $O(2^n)$, y sea N el problema más grande que puedes resolver con este algoritmo usando los recursos computacionales que tienes, denotados por R . Este R puede ser la cantidad de tiempo que estás dispuesto a esperar a que la función termine, o R puede ser el número de operaciones básicas que observas que la computadora ejecuta antes de cansarte de esperar. Usando el mismo algoritmo, ¿qué tan grande de un problema puedes resolver con una nueva computadora que es el doble de rápida?

Si establecemos $R = 2^N$, usando nuestra computadora vieja, con nuestra nueva computadora tenemos $2R$ recursos computacionales; por lo tanto, queremos encontrar N' tal que $2R = 2^{N'}$. Con algo de sustitución, podemos llegar a $2 \times 2^N = 2^{N'} \rightarrow 2^{N+1} = 2^{N'} \rightarrow N' = N + 1$. Así que con un algoritmo de tiempo exponencial, duplicar tus recursos computacionales te permitirá resolver un problema una unidad más grande de lo que podías con tu computadora vieja. Esta es una diferencia muy pequeña. De hecho, a medida que N se vuelve grande, la mejora relativa tiende a 0.

Con un algoritmo de tiempo polinomial, puedes hacerlo mucho mejor. Esta vez supongamos que $R = N^c$, donde c es alguna constante mayor que uno. Entonces $2R = (N')^c$, que, si usas sustituciones similares a las anteriores, resultará en $N' = 2^{1/c}N$. Así que con un algoritmo de tiempo polinomial con potencia c , puedes resolver un problema $2^{1/c}$ veces más grande de lo que podías con tu computadora vieja. Cuando c es pequeño, digamos menor que 5, esta es una diferencia mucho mayor que con el algoritmo exponencial.

Finalmente, consideremos un algoritmo de tiempo logarítmico. Sea $R = \log N$. Entonces $2R = \log N'$; nuevamente con algo de sustitución obtenemos $N' = N^2$. ¡Así que con el doble de recursos, podemos elevar al cuadrado el tamaño del problema que podemos resolver!

La moraleja de la historia es que los algoritmos de tiempo exponencial no escalan bien. Es decir, a medida que aumentas el tamaño de la entrada, pronto encontrarás que la función toma más tiempo (mucho más) del que estás dispuesto a esperar. Para un ejemplo final, `my_fib_rec(100)` tomaría del orden de 2^{100} operaciones básicas para completar el cálculo. Si tu computadora puede hacer 100 billones de operaciones básicas por segundo (mucho más rápido que la computadora más rápida de la tierra), le tomaría a tu computadora unos 400 millones de años completarlo; sin embargo, usar `my_fib_iter(100)` tomaría menos de 1 nanosegundo para completar la misma tarea.

Existen tanto un algoritmo de tiempo exponencial (recursivo) como un algoritmo de tiempo polinomial (iterativo) para calcular los números de Fibonacci. Dada la elección, claramente elegiríamos el algoritmo de tiempo polinomial. Sin embargo, hay una clase de problemas para los cuales nadie ha descubierto un algoritmo de tiempo polinomial. En otras palabras, solo se conocen algoritmos de tiempo exponencial para ellos. Estos problemas se conocen como **NP-completos**; una investigación en curso trata de determinar si existen algoritmos de tiempo polinomial para estos problemas. Ejemplos de problemas NP-completos incluyen el problema del Viajante (Traveling Salesman), el de Cubierta de Conjuntos (Set Cover) y el de Empaquetamiento

(Set Packing). Aunque son de construcción teórica, las soluciones a estos problemas tienen numerosas aplicaciones en logística e investigación de operaciones. De hecho, algunos algoritmos de encriptación que mantienen seguras las aplicaciones web y bancarias se basan en la NP-completitud de romperlos. Una discusión más profunda de los problemas NP-completos y la teoría de la complejidad está fuera del alcance de este libro, pero estos problemas son muy interesantes e importantes para muchas aplicaciones de ingeniería.

El Perfilador

Usando el Comando Mágico

Incluso si no cambia la complejidad Big-O de un programa, muchos programadores pasarán largas horas para hacer que su código se ejecute el doble de rápido o para obtener mejoras aún más pequeñas.

Hay formas de verificar el tiempo de ejecución del código en Jupyter notebook. A continuación se presentan los comandos mágicos para hacerlo:

- `%time`: Obtiene el tiempo de ejecución de una sola declaración.
- `%timeit`: Obtiene el tiempo de ejecución repetido de una sola declaración.
- `%%time`: Obtiene el tiempo de ejecución de todo el código en la celda.
- `%%timeit`: Obtiene el tiempo de ejecución repetido de una celda.

Observa que el comando mágico de doble porcentaje medirá el tiempo de ejecución de todo el código en una celda, mientras que el comando de un solo porcentaje solo funciona para una sola declaración.

```
1 In [1]: %time sum(range(200))
2 CPU times: user 6 us, sys: 1 us, total: 7 us, wall time: 9.06
  us
3 Out[1]: 19900
4
5 In [2]: %timeit sum(range(200))
6 1.24 us \textpm 70.6 ns per loop (mean \textpm std. dev. of 7
  runs, 1000000 loops each)
7
8 In [3]: %%time
```

```

9         s = 0
10        for i in range(200):
11            s += i
12 CPU times: user 15 us, sys: 0 ns, total: 15 us, wall time:
13         17.9 us
14 In [4]: %%timeit
15         s = 0
16         for i in range(200):
17             s += i
18 7.06 us \textpm 414 ns per loop (mean \textpm std. dev. of 7
19     runs, 100000 loops each)

```

Listing 26: Usando %time y %timeit.

Advertencia

A veces puede no ser apropiado usar `timeit`, ya que ejecutará muchos bucles para el código, tomando una cantidad excesiva de tiempo para completar la tarea.

Usando el Perfilador de Python

También puedes usar el perfilador de Python (para una discusión adicional, lee más en la documentación de Python) para perfilar el código que escribes. En Jupyter notebook,

- `%prun`: Ejecuta una sola declaración a través del perfilador de código de Python.
- `%%prun`: Ejecuta una celda a través del perfilador de código de Python.

El siguiente ejemplo suma números aleatorios una y otra vez:

```

1 In [6]: import numpy as np
2
3 In [7]: def slow_sum(n, m):
4         for i in range(n):
5             # we create a size m array of random numbers
6             a = np.random.rand(m)
7             s = 0
8             # in this loop we iterate through the array
9             # and add elements to the sum one by one

```

```

10     for j in range(m):
11         s += a[j]
12
13 In [8]: %prun slow_sum(1000, 10000)

```

Listing 27: Definiendo una función lenta para sumar.

Los resultados se muestran en la Fig. 2.

Resultado del perfilador %prun					
	ncalls	tottime	percall	cumtime	percallname:lineno(function)
1000					
...					

Figura 2: El resultado del perfilado de prun.

La tabla muestra las siguientes columnas (del perfilador de Python):

- **ncalls** es el número de llamadas;
- **tottime** es el tiempo total dedicado a realizar la función dada (excluyendo el tiempo dedicado a llamadas a subfunciones);
- **percall** es el cociente de dividir **tottime** por **ncalls**;
- **cumtime** es el tiempo total dedicado a esta y todas las subfunciones (desde la invocación hasta la salida). Esta cifra es precisa incluso para funciones recursivas;
- **percall** es el cociente de dividir **cumtime** por llamadas primitivas.

Usando el Perfilador de Líneas

Muchas veces queremos determinar qué línea en un script de código toma mucho tiempo para poder reescribir esa línea y hacerla más eficiente. Esto se puede hacer usando `line_profiler`, que perfilará el código línea por línea. Esta función no viene incluida con Python; por lo tanto, necesitamos instalarla, y luego podemos usar el comando mágico:

- `%lprun`: Ejecuta el perfilado línea por línea en una sola declaración.

```
1 In [9]: # Note, you only need run this once.
2         !conda install line_profiler
3
4 In [10]: %load_ext line_profiler
```

Listing 28: Instalando y cargando `line_profiler`.

La forma en que usamos `line_profiler` para perfilar el código es la siguiente:

```
1 In [11]: %lprun -f slow_sum slow_sum(1000, 10000)
```

Listing 29: Usando `%lprun`.

Ejecutar el comando anterior realizará un perfilado línea por línea, como se muestra en la Fig. 3.

Resultado del perfilador de línea				
Línea	Veces	Tiempo	Por llamada	% Tiempo
3				
4				
5				
7				
10				
11				

Figura 3: El resultado del perfilado línea por línea de `line_profiler`.

Los resultados incluyen un resumen para cada línea de la función. Observa que las líneas 10 y 11 toman la mayoría del tiempo total de ejecución.

Por lo general, cuando el código tarda más de lo que te gustaría en ejecutarse, hay un **cuello de botella** donde se pasa la mayor parte del tiempo. Es decir, hay una línea de código que tarda mucho más en ejecutarse que las otras líneas en el programa. Abordar el cuello de botella en un programa generalmente conducirá a la mayor mejora en el rendimiento, incluso si hay otras áreas de tu código que se pueden mejorar más fácilmente.

Consejo

Comienza por el cuello de botella cuando mejores el rendimiento de un código.

Resumen y Problemas

Resumen

1. La complejidad de un algoritmo es la relación entre el tamaño del problema de entrada y el tiempo que tarda el algoritmo en terminar.
2. La notación Big-O es un método estándar para clasificar la complejidad algorítmica de una manera que es independiente de la computadora y del sistema operativo.
3. Los algoritmos con complejidad logarítmica se ejecutan más rápido que los algoritmos con complejidad polinomial, que son más rápidos que los algoritmos con complejidad exponencial.
4. El perfilador de Python es una herramienta útil para determinar dónde tu código se está ejecutando lentamente, para que puedas mejorar su rendimiento.

Problemas y Soluciones

1. **Problema 1:** Define el tamaño de las siguientes tareas.

Solución:

- **Resolver un rompecabezas:** El tamaño podría ser el número de piezas.
- **Repartir un folleto en una clase:** El tamaño podría ser el número de estudiantes.
- **Caminar a clase:** El tamaño podría ser la distancia (en metros) o el número de pasos.
- **Encontrar un nombre en un diccionario:** El tamaño podría ser el número de entradas (palabras) en el diccionario.

2. **Problema 2:** Complejidad Big-O de las tareas del problema anterior.

Solución:

- **Rompecabezas:** $O(n^2)$ o peor, dependiendo de la estrategia (probar combinaciones).
- **Repartir folleto:** $O(n)$ (lineal, pasas por cada estudiante una vez).
- **Caminar a clase:** $O(n)$ (lineal, cada paso/unidad de distancia toma tiempo constante).
- **Diccionario (búsqueda binaria):** $O(\log n)$. Si se usara un índice (hash), podría ser $O(1)$.

3. **Problema 3:** Búsqueda en tiempo constante $O(1)$.

Solución: La búsqueda en tiempo constante se puede lograr usando una **función hash** y una tabla hash. Una función hash toma una clave (por ejemplo, una palabra) y calcula un índice único (o casi único) en un array donde se almacena el valor. Acceder a un elemento en un array por su índice es $O(1)$. Los diccionarios de Python (`dict`) están implementados internamente como tablas hash y proporcionan búsqueda en tiempo promedio $O(1)$.

4. **Problema 4:** Complejidad de las relaciones recursivas Tribonacci y Timmynacci.

Solución:

- **Tribonacci:** Similar a Fibonacci pero con 3 llamadas. El árbol de recursión tiene una tasa de ramificación de 3, por lo que el número de llamadas crece como $O(3^n)$. Por lo tanto, la versión recursiva es **exponencial** ($O(3^n)$). La versión iterativa (con un bucle y almacenamiento de los últimos 3 valores) sería **polinomial** ($O(n)$).
- **Timmynacci:** La relación $t(n) = t(n/2) + t(n/4)$. Aunque es recursiva, el tamaño del problema se reduce drásticamente en cada nivel. La profundidad de la recursión es $O(\log n)$. La versión recursiva probablemente sería **polinomial** o incluso **logarítmica** en la práctica, no exponencial como Fibonacci. Requiere un análisis más detallado (como el Teorema Maestro), pero es mucho más eficiente. La versión iterativa también sería eficiente.

5. **Problema 5:** Complejidad Big-O de las Torres de Hanoi.

Solución: La solución recursiva de las Torres de Hanoi requiere $2^n - 1$ movimientos para n discos. Por lo tanto, su complejidad Big-O es $O(2^n)$ (tiempo exponencial). Esta complejidad es exacta, no solo una cota superior, ya que se requiere ese número de movimientos para resolver el problema.

6. **Problema 6:** Complejidad Big-O de Quicksort.

Solución: En el caso promedio, Quicksort tiene una complejidad de $O(n \log n)$. En el peor de los casos (cuando el pivote es siempre el elemento más pequeño o más grande), puede degradarse a $O(n^2)$.

7. **Problema 7:** Perfilado de dos implementaciones de Fibonacci.

Solución: A continuación se muestran las dos funciones para que el lector las profile. La primera (`my_fib_iter1`) preasigna memoria con NumPy, lo que suele ser más rápida. La segunda (`my_fib_iter2`) usa una lista de Python que crece dinámicamente.

```
1 import numpy as np
2
```

```
3 def my_fib_iter1(n):
4     out = np.zeros(n)
5     out[:2] = 1
6     for i in range(2, n):
7         out[i] = out[i-1] + out[i-2]
8     return out
9
10 def my_fib_iter2(n):
11     out = [1, 1]
12     for i in range(2, n):
13         out.append(out[i-1] + out[i-2])
14     return np.array(out)
15
16 # Para perfilar, ejecutar en una celda de Jupyter:
17 # %prun my_fib_iter1(10000)
18 # %prun my_fib_iter2(10000)
19 # %load_ext line_profiler
20 # %lprun -f my_fib_iter1 my_fib_iter1(10000)
21 # %lprun -f my_fib_iter2 my_fib_iter2(10000)
22
```

Listing 30: Implementaciones para perfilado.

Análisis esperado: Se espera que `my_fib_iter1` sea más rápida porque evita el redimensionamiento dinámico de la lista. El perfilador de línea mostrará que la línea que consume más tiempo en ambas es la suma dentro del bucle. La diferencia principal estará en el tiempo dedicado a la gestión de la memoria (`append` vs. asignación en array preasignado).

REPRESENTACION DE NUMEROS

9.1 Base-N y Binario

El sistema decimal es una forma de representar números que aprendiste en la escuela primaria. En el sistema decimal, un número se representa mediante una lista de dígitos del 0 al 9, donde cada dígito representa el coeficiente de una potencia de 10.

EJEMPLO: Muestra la expansión decimal de 147.3.

$$147,3 = 1 \cdot 10^2 + 4 \cdot 10^1 + 7 \cdot 10^0 + 3 \cdot 10^{-1}.$$

Dado que cada dígito está asociado a una potencia de 10, el sistema decimal también se conoce como base 10 porque se basa en 10 dígitos (0 al 9). No hay nada especial en los números en base 10, salvo que estás más acostumbrado a usarlos. Por ejemplo, en base 3 tenemos los dígitos 0, 1 y 2, y el número

$$121_{(\text{base } 3)} = 1 \cdot 3^2 + 2 \cdot 3^1 + 1 \cdot 3^0 = 9 + 6 + 1 = 16_{(\text{base } 10)}.$$

Para los propósitos de este capítulo, es útil denotar la representación de un número; es decir, cada número irá seguido de su representación entre paréntesis (por ejemplo, $11_{(\text{base } 10)}$ significa 11 en base 10) a menos que el contexto sea claro.

En las computadoras, los números suelen representarse en base 2 o números binarios. En binario, los únicos dígitos disponibles son 0 y 1, y cada dígito es el coeficiente de una potencia de 2. Los dígitos de un número binario también se conocen como *bits*. Nótese que los números binarios siguen siendo números, y los procesos de suma y multiplicación son exactamente como los aprendiste en la escuela.

¡PRUÉBALO! Convierte el número $11_{(\text{base } 10)}$ a binario.

$$11_{(\text{base } 10)} = 8 + 2 + 1 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 1011_{(\text{base } 2)}.$$

¡PRUÉBALO! Convierte $37_{(\text{base } 10)}$ y $17_{(\text{base } 10)}$ a binario. Suma y multiplica los números resultantes en binario. Verifica que el resultado sea correcto en base 10.

Conversión a binario:

$$37_{(\text{base } 10)} = 32 + 4 + 1 = 1 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 100101_{(\text{base } 2)},$$

$$17_{(\text{base } 10)} = 16 + 1 = 1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 10001_{(\text{base } 2)}.$$

Resultados de la suma y multiplicación en decimal:

$$37 + 17 = 54, \quad 37 \times 17 = 629.$$

Realizando la suma en binario (ver Fig. 9.1).

$$\begin{array}{r} 100101 \\ + 10001 \\ \hline 111000 \end{array}$$

Figura 9.1: Suma binaria.

Realizando la multiplicación en binario (ver Fig. 9.2).

$$\begin{array}{r} 100101 \\ \times 10001 \\ \hline 100101 \quad (\times 1) \\ 000000 \quad (\times 0) \\ 000000 \quad (\times 0) \\ 000000 \quad (\times 0) \\ 100101 \quad (\times 1) \\ \hline 1011011101 \end{array}$$

Figura 9.2: Multiplicación binaria.

Los números binarios son útiles para las computadoras porque las operaciones aritméticas con los dígitos 0 y 1 pueden representarse mediante AND, OR y NOT, que pueden calcularse rápidamente.

A diferencia de los humanos, que pueden abstraer números a valores arbitrariamente grandes, las computadoras tienen un número fijo de bits que pueden almacenar a la vez. Por ejemplo, una computadora de 32 bits puede representar y procesar números binarios de 32 dígitos, y no más. Si se usan los 32 bits para representar números enteros positivos en binario, entonces hay $\sum_{n=0}^{31} 2^n = 4\,294\,967\,296$ números que la computadora puede representar. Esto no es una gran cantidad de números y sería completamente insuficiente para realizar algo más que cálculos básicos. Por ejemplo, no se puede calcular la suma perfectamente razonable $0,5 + 1,25$ usando esta representación porque todos los bits están dedicados solo a enteros.

9.2 Números de Punto Flotante

El número de bits suele ser fijo para cualquier computadora. Usar la representación binaria nos da un rango y una precisión insuficientes para hacer cálculos de ingeniería relevantes. Para lograr el rango de valores necesario con la misma cantidad de bits, usamos números de punto flotante, o *floats* para abreviar. En lugar de utilizar cada bit como coeficiente de una potencia de 2, los floats asignan bits a tres partes diferentes: el indicador de signo, s , que indica si un número es positivo o negativo; la característica o exponente, e , que es la potencia de 2; y la fracción, f , que es el coeficiente del exponente. Casi todas las plataformas asignan los floats de Python a la doble precisión IEEE754, con un total de 64 bits. Se asigna un bit al signo, 11 bits al exponente y 52 bits a la fracción.

Con 11 bits asignados al exponente, hay 2048 valores que este número puede usar. Dado que queremos poder hacer números muy precisos, queremos que algunos de estos valores representen exponentes negativos (es decir, para permitir números entre 0 y 1 en base 10). Para lograrlo, se resta 1023 al exponente para normalizarlo. El valor restado del exponente se conoce comúnmente como *sesgo*. La fracción es un número entre 1 y 2. En binario, esto significa que el término principal siempre será 1 y, por lo tanto, es un desperdicio de bits almacenarlo. Para ahorrar espacio, se omite el 1 inicial. En Python, podemos obtener la información de los floats usando el paquete

Muestra que 2^{-1075} subdesborda a cero, y que el resultado no puede distinguirse de cero. Muestra que 2^{-1074} no lo hace.

```

1 In [11]: 2**(-1075)
2 Out[11]: 0.0
3 In [12]: 2**(-1075) == 0
4 Out[12]: True
5 In [13]: 2**(-1074)
6 Out[13]: 5e-324

```

¿Qué hemos ganado al usar IEEE754 frente al binario? Usar binario de 64 bits nos da 2^{64} números. Como el número de bits no cambia entre binario e IEEE754, IEEE754 también debe darnos 2^{64} números. En binario, los números tienen un espaciado constante entre ellos. Como resultado, no se puede tener simultáneamente rango (es decir, gran distancia entre el mínimo y el máximo número representable) y precisión (es decir, pequeño espaciado entre números). Controlar estos parámetros dependería de dónde coloques el punto decimal en tu número. IEEE754 supera esta limitación usando una precisión muy alta en números pequeños y una precisión muy baja en números grandes. Esta limitación suele ser aceptable porque la brecha en números grandes sigue siendo pequeña en relación con el tamaño del número en sí. Por lo tanto, incluso si la brecha es de millones, es irrelevante para cálculos normales si el número considerado está en los billones o más.

Esta sección introdujo la representación de números de punto flotante. Este concepto se describe en detalle en *Computer Organization and Design* de David Patterson y John Hennessy.

9.3 Errores de Redondeo

En la sección anterior, hablamos sobre cómo los números de punto flotante se representan en las computadoras como fracciones en base 2. Esto tiene un efecto secundario: los números de punto flotante no pueden almacenarse con precisión perfecta; en cambio, los números se aproximan mediante un número finito de bytes. Por lo tanto, la diferencia entre una aproximación de un número usado en un cálculo y su valor correcto (verdadero) se denomina *error de redondeo*. Es uno de los errores comunes que se encuentran habitualmente en los cálculos numéricos. El otro es el *error de truncamiento*, que se presentará en el Capítulo 18. La diferencia es que el error de truncamiento es el error cometido al truncar una suma infinita y aproximarla mediante una suma

finita.

9.3.1 Error de Representación

La forma más común de error de redondeo es el error de representación en los números de punto flotante. Un ejemplo sencillo es: ¿cómo representamos π ? Sabemos que π es un número infinito (es decir, tiene un número infinito de dígitos); normalmente, solo usamos un número finito de dígitos. Por ejemplo, si solo usas 3.14159265, habrá un error entre esta aproximación y el verdadero número infinito. Otro ejemplo es $1/3$; el valor verdadero será $0,333333333\ldots$. No importa cuántos dígitos decimales elijamos, también hay un error de redondeo.

Cuando redondeamos los números varias veces, el error se acumula. Por ejemplo, si 4.845 se redondea a dos decimales, es 4.85. Luego, si lo redondeamos nuevamente a un decimal, es 4.9; el error total será 0.055. Pero si redondeamos una sola vez a un decimal, es 4.8, y el error es 0.045.

9.3.2 Error de Redondeo por Aritmética de Punto Flotante

Del ejemplo anterior, el error entre 4.845 y 4.8 debería ser 0.055. Pero si lo calculas en Python, verás que $4,9 - 4,845$ no es igual a 0.055.

```
1 In [1]: 4.9 - 4.845 == 0.055
2 Out[1]: False
```

¿Por qué ocurre esto? Si observamos nuevamente $4,9 - 4,845$, en realidad obtenemos 0,055000000000000604. Esto se debe a que el número de punto flotante no puede representar el número exacto porque es una aproximación; cuando se usa en aritmética, causa un pequeño error.

```
1 In [2]: 4.9 - 4.845
2 Out[2]: 0.055000000000000604
3 In [3]: 4.8 - 4.845
4 Out[3]: -0.04499999999999993
```

Otro ejemplo muestra que $0,1 + 0,2 + 0,3$ no es igual a 0.6; el error se debe a la misma causa.

```
1 In [4]: 0.1 + 0.2 + 0.3 == 0.6
2 Out[4]: False
```

Aunque los números no pueden acercarse más a sus valores exactos deseados, la función `round` puede ser útil para redondear después de los cálculos, de modo que los resultados con valores inexactos sean comparables entre sí:

```
1 In [5]: round(0.1 + 0.2 + 0.3, 5) == round(0.6, 5)
2 Out[5]: True
```

9.3.3 Acumulación de Errores de Redondeo

Cuando realizamos una secuencia de cálculos sobre una entrada inicial con errores de redondeo debidos a una representación inexacta, los errores pueden magnificarse o acumularse. Como ejemplo, si tenemos el número 1, sumamos y luego restamos $1/3$, debería darnos 1. Pero si sumamos $1/3$ y restamos la misma cantidad de veces $1/3$ durante varias iteraciones, ¿seguimos obteniendo el mismo número 1? No, no es así. En el siguiente ejemplo, cuantas más veces lo hagas, más se acumula el error.

```
1 In [6]: # Si lo hacemos solo una vez
2         1 + 1/3 - 1/3
3 Out[6]: 1.0
4
5 In [7]: def add_and_subtract(iterations):
6         result = 1
7         for i in range(iterations):
8             result += 1/3
9         for i in range(iterations):
10            result -= 1/3
11        return result
12
13 In [8]: # Si lo hacemos 100 veces
14         add_and_subtract(100)
15 Out[8]: 1.0000000000000002
16
17 In [9]: # Si lo hacemos 1000 veces
18         add_and_subtract(1000)
19 Out[9]: 1.00000000000000064
20
21 In [10]: # Si lo hacemos 10000 veces
22          add_and_subtract(10000)
23 Out[10]: 1.00000000000001166
```

9.4 Resumen y Problemas

9.4.1 Resumen

1. Los números pueden representarse de varias formas diferentes; cada esquema de representación tiene ventajas y desventajas.
2. Las computadoras deben representar números usando un número finito de dígitos (es decir, bits).
3. El binario y el IEEE754 son representaciones finitas de números usadas por las computadoras.
4. El error de redondeo es un error importante asociado a los métodos numéricos.

9.4.2 Problemas

1. Escribe una función `my_bin_2_dec(b)` donde `b` es un número binario representado por una lista de unos y ceros. El último elemento de `b` representa el coeficiente de 2^0 , el penúltimo elemento de `b` representa el coeficiente de 2^1 , y así sucesivamente. La variable de salida, `d`, debe ser la representación decimal de `b`. Se proporcionan casos de prueba a continuación.
2. Escribe una función `my_dec_2_bin(d)` donde `d` es un entero positivo en decimal, y `b` es la representación binaria de `d`. La salida `b` debe ser una lista de unos y ceros, y el término principal debe ser 1 a menos que el valor de entrada decimal sea 0. Se proporcionan casos de prueba.
3. Usa las dos funciones que escribiste en los Problemas 1 y 2 para calcular `d = my_bin_2_dec(my_dec_2_bin(12654))`. ¿Obtienes el mismo número?
4. Escribe una función `my_bin_adder(b1, b2)` donde `b1`, `b2` y la variable de salida `b` son números binarios representados como en el Problema 1. La variable de salida debe calcularse como `b = b1 + b2`. No uses tus funciones de los Problemas 1 y 2 para escribir esta función (es decir, no conviertas `b1` y `b2` a decimal; súmalos y luego convierte el resultado de vuelta a binario). Esta función debe ser capaz de aceptar entradas `b1` y `b2` de cualquier longitud (es decir, números binarios muy largos), y `b1` y `b2` pueden no tener necesariamente la misma longitud.

5. ¿Cuál es el efecto de asignar más bits a la fracción frente a la característica, y viceversa? ¿Cuál es el efecto de asignar más bits al signo?
6. Escribe una función `my_ieee_2_dec(ieee)` donde `ieee` es una cadena que contiene 64 caracteres de unos y ceros, que representa un número IEEE754 de 64 bits. La salida debe ser `d`, que es la representación decimal equivalente de `ieee`. La variable de entrada `ieee` siempre será una cadena de 64 elementos de unos y ceros que define un float de 64 bits.
7. Escribe una función `my_dec_2_ieee(d)` donde `d` es un número en decimal, y la variable de salida `ieee` es una cadena con 64 caracteres de unos y ceros, que representa el número IEEE754 de 64 bits más cercano a `d`. Supón que `d` no causará desbordamiento para números IEEE754 de 64 bits.
8. Define `ieee_baby` como una representación de números usando 6 bits, donde el primer bit es el bit de signo, el segundo y tercer bit se asignan a la característica, y el cuarto, quinto y sexto bit se asignan a la fracción. La normalización para la característica es 1.

Escribe todos los números decimales que pueden representarse mediante `ieee_baby`. ¿Cuál es la brecha más grande/más pequeña en `ieee_baby`?
9. Usa la función `np.spacing` para determinar el número más pequeño tal que la brecha sea 1.
10. ¿Cuáles son algunas de las ventajas y desventajas de usar binario frente a decimal?
11. Escribe el número $13_{(\text{base } 10)}$ en base 1. ¿Cómo sumarías y multiplicarías en base 1?
12. ¿Qué tan alto puedes contar con los dedos si cuentas en binario?
13. Sea `b` un número binario con `n` dígitos. ¿Puedes pensar en formas de multiplicar y dividir `b` por 2 que no impliquen aritmética? Pista: Piensa en cómo multiplicas y divides un número decimal por 10.

Soluciones a los Problemas

Problema 1

```
1 def my_bin_2_dec(b):
2     d = 0
3     n = len(b)
4     for i, bit in enumerate(b):
5         d += bit * (2 ** (n - 1 - i))
6     return d
7
8 # Casos de prueba
9 print(my_bin_2_dec([1, 1, 1]))           # 7
10 print(my_bin_2_dec([1, 0, 1, 0, 1, 0, 1])) # 85
11 print(my_bin_2_dec([1]*25))             # 33554431
```

Problema 2

```
1 def my_dec_2_bin(d):
2     if d == 0:
3         return [0]
4     b = []
5     while d > 0:
6         b.insert(0, d % 2)
7         d //= 2
8     return b
9
10 # Casos de prueba
11 print(my_dec_2_bin(0))           # [0]
12 print(my_dec_2_bin(23))          # [1, 0, 1, 1, 1]
13 print(my_dec_2_bin(2097))        # [1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0,
                                     1]
```

Problema 3

```
1 d = my_bin_2_dec(my_dec_2_bin(12654))
2 print(d) # 12654, sí se obtiene el mismo número.
```

Problema 4

```

1 def my_bin_adder(b1, b2):
2     # Asegurar que ambas listas tengan la misma longitud
   agregando ceros a la izquierda
3     max_len = max(len(b1), len(b2))
4     b1 = [0] * (max_len - len(b1)) + b1
5     b2 = [0] * (max_len - len(b2)) + b2
6
7     carry = 0
8     result = []
9     for i in range(max_len - 1, -1, -1):
10         total = b1[i] + b2[i] + carry
11         result.insert(0, total % 2)
12         carry = total // 2
13     if carry:
14         result.insert(0, 1)
15     return result
16
17 # Casos de prueba
18 print(my_bin_adder([1, 1, 1, 1, 1], [1])) # [1, 0, 0, 0, 0,
   0]
19 print(my_bin_adder([1, 1, 1, 1, 1], [1, 0, 1, 0, 1, 0, 0]))
   # [1, 1, 1, 0, 0, 1, 1]
20 print(my_bin_adder([1, 1, 0], [1, 0, 1])) # [1, 0, 1, 1]

```

Problema 5

- Asignar más bits a la fracción aumenta la precisión (más dígitos significativos), pero reduce el rango del exponente (menor rango de valores representables).
- Asignar más bits a la característica (exponente) aumenta el rango de valores, pero disminuye la precisión (mayor espaciado entre números).
- Asignar más bits al signo solo sirve para distinguir entre positivo y negativo; con un bit es suficiente, más bits serían redundantes.

Problema 6

```

1 def my_ieee_2_dec(ieee):
2     sign_bit = int(ieee[0])
3     exponent_bits = ieee[1:12]
4     fraction_bits = ieee[12:]

```



```

6     packed = struct.pack('>d', d) # big-endian, doble
    precisi3n
7     # Convertir cada byte a binario de 8 bits
8     bits = ''.join(f'{byte:08b}' for byte in packed)
9     return bits
10
11 # Casos de prueba
12 d1 = 1.518484199625
13 print(my_dec_2_ieee(d1))
14 # Debería imprimir:
    0100000000101110010111101010001110011100001100011010010001101000
15
16 d2 = -309.141740
17 print(my_dec_2_ieee(d2))
18 #
    1100000001110011010100100100010010010001001010011000100010010000
19
20 d3 = -25252
21 print(my_dec_2_ieee(d3))
22 #
    1100000011011000101010010000000000000000000000000000000000000000

```

Problema 8

- Bits: signo (1 bit), característica (2 bits), fracci3n (3 bits). Sesgo: $2^{2-1} - 1 = 1$.
- Exponentes posibles: 0,1,2,3 (en binario 00,01,10,11). Aplicando sesgo: $e - 1$:
 - $e=0$ (00) → usado para subnormales y cero.
 - $e=1$ (01) → exponente -1.
 - $e=2$ (10) → exponente 0.
 - $e=3$ (11) → exponente 1 (reservado para infinitos/NaN si fracci3n=0? pero como es baby, probablemente se usa normal).
- Fracci3n de 3 bits: valores f de 0 a 7 (en binario 000 a 111). Para n3meros normalizados ($e \neq 0$ y $e \neq 3$), el valor es $(-1)^s \times 2^{e-1} \times (1 + f/8)$.

Para subnormales ($e=0$), es $(-1)^s \times 2^{-1} \times (0 + f/8)$. Para $e=3$, si $f=0$, infinito; si $f \neq 0$, NaN (pero para baby se pueden considerar como valores normalizados también, aunque IEEE754 los reserva; asumamos que se usan normalizados con exponente 1, pero cuidado).

- Construyamos todos los valores posibles (64 combinaciones). Listamos en orden:

Vamos a generar sistemáticamente todos los números representables en `ieee_baby` (6 bits). Incluyendo ceros, subnormales, normales, infinitos y NaN. Como no se especifica qué hacer con los casos especiales, asumiremos la misma semántica que IEEE754: cuando $e=3$ (11) y $f \neq 0$, es NaN; cuando $e=3$ y $f=0$, es infinito (signo según s). Cuando $e=0$, son subnormales (incluyendo el cero cuando $f=0$). El resto son normalizados.

- Bits: s (bit5), $e1$ (bit4), $e0$ (bit3), $f2$ (bit2), $f1$ (bit1), $f0$ (bit0). Numeramos de izquierda a derecha (más significativo a menos).

Lista de valores decimales (solo algunos ejemplos, la lista completa es larga). La brecha más pequeña ocurre cerca de cero (subnormales y números pequeños). La brecha más grande ocurre cerca de los números grandes.

Para calcular la brecha máxima y mínima, necesitamos el espaciado entre números consecutivos. En general, para normalizados con exponente e , el espaciado es $2^{e-1} \times 2^{-3} = 2^{e-4}$. Para $e=2$ (exponente 0), espaciado = $2^{-4} = 0,0625$. Para $e=1$ (exponente -1), espaciado = $2^{-5} = 0,03125$. Para $e=0$ (subnormales), el espaciado es constante $2^{-1} \times 2^{-3} = 2^{-4} = 0,0625$ también? Pero cuidado: los subnormales no tienen el 1 implícito, entonces el valor es $(-1)^s 2^{-1} (f/8)$. El incremento de f en 1 aumenta el valor en $2^{-1} \cdot 1/8 = 2^{-4} = 0,0625$, igual que para $e=2$ normalizado. Sin embargo, el primer subnormal ($f=1$) es $2^{-1} \cdot 1/8 = 0,0625$, mientras que el número normalizado más pequeño con $e=1$ (exponente -1) es $2^{-1} \cdot (1 + 0) = 0,5$. Entonces hay un hueco entre el subnormal más grande ($f=7$) = $2^{-1} \cdot 7/8 = 0,4375$ y el normalizado más pequeño con $e=1$ ($f=0$) = $0,5$, que es $0,0625$. Luego los espaciados son uniformes en cada región.

La brecha más grande será para el mayor exponente normalizado, $e=2$ (exponente 0), el máximo número antes de infinito es con $f=7$: $2^0 \cdot (1 + 7/8) = 1,875$. El siguiente número (si consideramos infinito) no es finito, pero el último espaciado antes de infinito es entre $1,875$ y el anterior: para $f=6$: $1,75$, espaciado = $0,125$? Calculemos: $2^0 \cdot (1 + 6/8) = 1,75$, $2^0 \cdot (1 + 7/8) = 1,875$, diferencia = $0,125$. Ese es el mayor espaciado entre números finitos. También para

$e=2$ normalizados, el espaciado es constante 0.125. Para $e=1$ normalizados, espaciado 0.0625; para subnormales, espaciado 0.0625. Entonces la brecha máxima es 0.125, la mínima es 0.0625.

Sin embargo, también hay que considerar los números negativos, que son simétricos. La brecha entre -0 y +0? El cero tiene dos representaciones (+0 y -0), pero son el mismo valor. No hay brecha entre ellos porque son iguales.

Entonces:

- Mayor brecha: 0.125 (entre números normalizados con exponente 0).
- Menor brecha: 0.0625 (entre subnormales y normalizados con exponente -1).

Problema 9

Usando `np.spacing`, queremos el menor número x tal que la brecha en x sea 1. Es decir, $\text{spacing}(x) = 1$. Sabemos que $\text{spacing}(x) \approx 2^{e-52} \cdot 2^{-52}$? En realidad, para IEEE754 de 64 bits, la brecha es $2^{e-1023} \cdot 2^{-52} = 2^{e-1075}$. Queremos $2^{e-1075} = 1 \Rightarrow e-1075 = 0 \Rightarrow e = 1075$. El número correspondiente es $2^{e-1023} \cdot (1 + f)$ con $f = 0$, es decir, 2^{52} . Pero 2^{52} es aproximadamente $4,5 \times 10^{15}$. Verifiquemos con numpy:

```
1 import numpy as np
2 x = 2**52
3 print(np.spacing(x)) # 1.0
4 print(np.spacing(x-1)) # menor que 1
```

El número más pequeño con espaciado 1 es 2^{52} .

Problema 10

Ventajas del binario:

- Simplicidad en circuitos electrónicos (dos estados).
- Operaciones aritméticas y lógicas rápidas.
- Directamente compatible con la lógica digital.

Desventajas:

- Los números son largos de escribir para humanos.

- No es intuitivo para las personas.
- Algunos números decimales no tienen representación binaria exacta, causando errores de redondeo.

Ventajas del decimal:

- Natural para humanos.
- Representación exacta de fracciones decimales.

Desventajas:

- Requiere más estados (0-9) en hardware, más complejo.
- Menos eficiente en computadoras digitales.

Problema 11

Base 1 es un sistema unario, donde el número n se representa con n símbolos (generalmente palotes). Así, $13_{(\text{base } 10)}$ en base 1 sería una cadena de 13 unos: 1111111111111. Sumar en base 1 es concatenar las representaciones. Multiplicar en base 1 es repetir una cadena tantas veces como el otro factor; por ejemplo, 3×2 sería concatenar “111” dos veces: “111111” (seis unos). Es ineficiente para números grandes.

Problema 12

Con 10 dedos, contando en binario (cada dedo arriba/abajo representa un bit), se pueden representar números del 0 al $2^{10} - 1 = 1023$. Por lo tanto, se puede contar hasta 1023.

Problema 13

Sí: multiplicar por 2 en binario equivale a desplazar todos los bits una posición a la izquierda y agregar un 0 al final (similar a multiplicar por 10 en decimal). Dividir por 2 equivale a desplazar una posición a la derecha (división entera), perdiendo el bit menos significativo. Si se quiere división exacta, se puede considerar el resto.

Capítulo: Errores, Buenas Prácticas de Programación y Depuración

10.1 Tipos de Errores

Hemos notado errores antes pero aún no hemos hablado de ellos en detalle. Hay tres tipos básicos de errores que los programadores deben tener en cuenta: errores de sintaxis, errores de tiempo de ejecución y errores lógicos. La sintaxis es el conjunto de reglas que gobiernan un lenguaje. En el lenguaje escrito y hablado, las reglas pueden ser flexionadas o incluso rotas para acomodar al hablante o escritor. Sin embargo, en un lenguaje de programación las reglas son rígidas. Un error de sintaxis ocurre cuando el programador escribe una instrucción usando sintaxis incorrecta; Python no puede entender lo que estás diciendo. Por ejemplo, `1 = x` no es legal en el lenguaje de programación Python porque los números no pueden ser asignados como variables. Si el programador intenta ejecutar una de estas instrucciones o cualquier otra declaración sintácticamente incorrecta, Python devolverá un mensaje de error al programador y señalará la ubicación donde ocurrió el error. Nótese que aunque Python puede identificar la ubicación donde el analizador sintáctico detecta el error de sintaxis, es posible que el error de sintaxis que causa el problema pueda estar lejos de la línea específica identificada.

EJEMPLO: Ejemplos de errores de sintaxis.

```
1 In [1]: 1 = x
```

```
File "<ipython-input-1-7a7b257d8e3d>", line 1
  1 = x
    ^
```

SyntaxError: can't assign to literal

```
1 In [2]: (1)
```

```
File "<ipython-input-2-800df0a5e99c>", line 1
      (1)
      ^
```

SyntaxError: invalid syntax

```
1 In [3]: if True
2         print("Here")
```

```
File "<ipython-input-3-025e9fcee1ee3>", line 1
      if True
      ^
```

SyntaxError: invalid syntax

La última línea del mensaje de error muestra lo que sucedió – `SyntaxError`, y las líneas anteriores indican dónde ocurre el error en el contexto del código. En general, los errores de sintaxis suelen ser fácilmente detectables, localizables y corregibles.

Incluso si toda la sintaxis es correcta en tu código, aún puede causar un error durante la ejecución del código. Los errores que ocurren durante la ejecución se llaman *excepciones* o *errores de tiempo de ejecución*. Las excepciones son más difíciles de encontrar y solo son detectables cuando se ejecuta un programa. Nótese que las excepciones no son fatales. Aprenderemos más adelante cómo manejarlas en Python. Si no las manejamos, Python terminará el programa. Veamos algunos ejemplos a continuación.

```
1 In [4]: 1/0
```

```
-----
ZeroDivisionError Traceback (most recent call last)
<ipython-input-4-9e1622b385b6> in <module>
----> 1 1/0
ZeroDivisionError: division by zero
```

```
1 In [5]: x = [2]
2         x + 2
```

```
-----
TypeError Traceback (most recent call last)
<ipython-input-5-29a14b9fefb9> in <module>
      1 x = [2]
----> 2 x + 2
TypeError: can only concatenate list (not "int") to list
```

```
1 In [6]: print(a)
```

```
-----
NameError Traceback (most recent call last)
<ipython-input-6-bca0e2660b9f> in <module>
----> 1 print(a)
NameError: name "a" is not defined
```

Como se muestra en los ejemplos anteriores, hay diferentes excepciones incorporadas: `ZeroDivisionError`, `TypeError` y `NameError`. Puedes encontrar una lista completa de las excepciones incorporadas en la documentación de Python¹. Además, puedes definir tus propios tipos de excepción, pero no trataremos esto aquí; si estás interesado en cómo definir excepciones personalizadas, consulta la documentación².

La mayoría de las excepciones son fáciles de localizar porque Python dejará de ejecutarse y te dirá dónde está el problema. Después de programar una función, los programadores experimentados suelen ejecutar la función varias veces, permitiendo que la función "lance cualquier error para poder corregirlos. Nótese que la ausencia de excepciones no significa que la función funcione correctamente.

Uno de los errores más difíciles de encontrar se llaman *errores lógicos*. Un error lógico no lanza un error. Aunque el programa se ejecutará sin problemas, es un error porque la salida obtenida no es la solución que esperas. Por ejemplo, considera la siguiente implementación incorrecta de la función factorial.

```
1 In [7]: def my_bad_factorial(n):
2         out = 0
3         for i in range(1, n+1):
4             out = out*i
5         return out
```

¹<https://docs.python.org/3/library/exceptions.html>

²<https://docs.python.org/3/tutorial/errors.html>

```
6  
7 In [8]: my_bad_factorial(4)  
8 Out[8]: 0
```

Esta función no producirá un error de tiempo de ejecución para ninguna entrada que sea válida para una función factorial correctamente implementada; sin embargo, si intentas usar `my_bad_factorial`, encontrarás que la respuesta es siempre 0 porque `out` se inicializa a 0 en lugar de 1. Por lo tanto, la línea `out = 0` es un error lógico. No produce un error de tiempo de ejecución por Python, pero conduce a un cálculo incorrecto.

Aunque los errores lógicos parecen poco probables de ocurrir —o al menos tan fáciles de encontrar como otros tipos de errores— cuando los programas se vuelven más largos y complicados, tales errores son muy fáciles de generar y notoriamente difíciles de encontrar. Cuando ocurren errores lógicos, no tienes más opción que revisar meticulosamente cada línea de tu código hasta encontrar el problema. Para estos casos, es importante saber exactamente cómo responderá Python a cada comando que des y no hacer suposiciones. También puedes usar el depurador de Python, que se describirá en la última sección de este capítulo.

10.2 Evitando Errores

Hay muchas técnicas que pueden ayudar a prevenir errores y facilitar su localización cuando ocurren. Debido a que familiarizarse con los tipos de errores comunes en programación es un proceso de “aprender sobre la marcha”; no podemos enumerarlos todos aquí. Dicho esto, presentamos algunos de ellos en la siguiente sección para ayudarte a construir buenos hábitos.

10.2.1 Planifica tu Programa

Al escribir un ensayo, es importante tener una estructura y una dirección que pretendes seguir. Para hacer tu estructura más tangible, escribir un ensayo generalmente comienza con un esquema que contiene los puntos principales que deseas abordar en tu trabajo. Esto es aún más importante hacerlo al programar porque las computadoras no interpretan lo que escribes. Al codificar programas complicados, debes comenzar con un esquema de tu programa que aborde todas las tareas que deseas que realice tu programa y en el orden en que debe realizarlas.

Muchos programadores novatos, ansiosos por terminar sus tareas, se apresuran a la parte de programación sin planificar adecuadamente las tareas necesarias para llevar a cabo la tarea dada. La planificación apresurada da como resultado un código igualmente apresurado y lleno de errores. El tiempo dedicado a planificar lo que intentas hacer es tiempo bien empleado. La planificación previa asegura que terminarás tu programa mucho más rápido que si juntas un programa sin mucho pensamiento.

Entonces, ¿en qué consiste planificar un programa? Recuerda en el Capítulo 3 que una función se define como una secuencia de instrucciones diseñadas para realizar una cierta tarea. Un módulo es una función o grupo de funciones que realizan una cierta tarea. Es importante diseñar tu programa en términos de módulos, especialmente para tareas que necesitan repetirse una y otra vez. Cada módulo debe realizar una tarea pequeña y bien definida y saber la menor cantidad de información posible sobre otras funciones (es decir, tener un conjunto muy limitado de entradas y salidas).

Una buena regla general es planificar de arriba a abajo, y luego programar de abajo a arriba. Es decir, decide qué se supone que debe hacer el programa en general, determina qué código es necesario para completar las tareas principales, y luego divide las tareas principales en componentes hasta que el módulo sea lo suficientemente pequeño como para estar seguro de que puedes escribirlo sin errores.

10.2.2 Prueba Todo a Menudo

Al codificar en módulos, debes probar cada módulo usando casos de prueba para los cuales conoces la respuesta, y codificar suficientes casos para estar seguro de que la función está funcionando correctamente (incluyendo casos extremos). Por ejemplo, si estás escribiendo una función que te dice si un número es primo o no, debes probar la función para entradas de 0 (caso extremo), 1 (caso extremo), 2 (sí simple), 4 (no simple) y 97 (no complicado). Si todos tus casos de prueba proceden sin error, puedes pasar a otros módulos, confiando de que el módulo actual funciona correctamente. Esto es especialmente importante si módulos subsiguientes dependen del módulo en el que estás trabajando o lo llaman. Si no pruebas tu código y asumes que tu código incorrecto es correcto, cuando comiences a obtener mensajes de error en módulos posteriores, no sabrás si el error está en el módulo en el que estás trabajando o en un módulo anterior; esto hará que encontrar el error sea exponencialmente más difícil.

Debes probar a menudo, incluso dentro de un solo módulo o función. Cuando estés trabajando en un módulo particular que tiene varios pasos, debes realizar pruebas intermedias para asegurarte de que es correcto hasta ese punto. Entonces, si alguna vez obtienes un error, probablemente estará en la parte de tu código escrita desde la última vez que lo probaste. Esta tendencia a apresurar la codificación es un error del que incluso los programadores experimentados son culpables; escribirán páginas y páginas de código sin probar y luego pasarán horas tratando de encontrar un pequeño error en alguna parte.

10.2.3 Mantén tu Código Limpio

Así como los buenos artesanos mantienen su área de trabajo libre de desorden innecesario, también debes mantener tu código lo más limpio posible. Hay muchas estrategias que puedes implementar para mantener tu código limpio. Primero, debes escribir tu código en la menor cantidad de instrucciones posible. Por ejemplo,

```
1 y = x**2 + 2*x+1
```

es mejor que

```
1 y=x**2
2 y=y+2*x
3 y=y+1
```

Incluso si el resultado es el mismo, cada carácter que escribes es una oportunidad de cometer un error; por lo tanto, reducir la cantidad de código que escribes reducirá tu riesgo de introducir errores. Además, escribir una expresión completa te ayudará a ti y a otras personas a entender lo que estás haciendo. En el ejemplo anterior, en el primer caso está claro que estás calculando el valor de un cuadrático en x , mientras que en el segundo caso no está claro. También puedes mantener tu código "limpio" usando variables en lugar de valores.

EJEMPLO: Mala implementación de sumar 10 números aleatorios.

```
1 In [1]: import numpy as np
2         s = 0
3         a = np.random.rand(10)
4         for i in range(10):
5             s = s + a[i]
```

EJEMPLO: Buena implementación de sumar 10 números aleatorios.

```

1 In [2]: n = 10
2         s = 0
3         a = np.random.rand(n)
4         for i in range(n):
5             s = s + a[i]

```

La segunda implementación es mejor por dos razones: primero, es más fácil para cualquiera que lea tu código que `n` representa el número de números aleatorios que deseas sumar, y aparece racionalmente donde se supone que debe estar en el código (es decir, al crear el arreglo de números aleatorios y al indexar el arreglo en el bucle `for`); y segundo, si alguna vez quisieras cambiar el número de números aleatorios que deseas sumar, solo tendrías que cambiarlo en un lugar al principio. Esto reduce las posibilidades de cometer errores al escribir el código y al cambiar el valor de `n`.

Nuevamente, esto no es crítico para un fragmento de código tan pequeño, pero se volverá muy importante cuando tu código se vuelva más complicado y los valores deban reutilizarse muchas veces.

EJEMPLO: Una implementación aún mejor de sumar 10 números aleatorios.

```

1 In [3]: s = sum(np.random.rand(10))

```

Cuando te familiarices más con Python, querrás usar la menor cantidad de líneas posible para hacer el mismo trabajo; por lo tanto, la familiaridad con las funciones comunes y cómo usarlas hará que tu código sea más conciso y eficiente.

También puedes mantener tu código limpio asignando a tus variables nombres cortos y descriptivos. Por ejemplo, como se señaló anteriormente, `n` es una variable suficiente para una tarea tan simple. El nombre de variable `x` probablemente sea un buen nombre ya que `x` generalmente contiene un valor de posición más que un número; pero `theNumberOfRandomNumbersToBeAdded` es un mal nombre de variable aunque sea descriptivo.

Finalmente, puedes mantener tu código limpio comentando con frecuencia. Aunque no comentar es ciertamente una mala práctica, comentar en exceso puede ser igualmente una mala práctica. Diferentes programadores estarán en desacuerdo sobre exactamente cuánto comentario es apropiado. Dependerá de ti decidir qué nivel de comentario es apropiado.

10.3 Try/Except

A menudo es importante escribir programas que puedan manejar ciertos tipos de errores o excepciones de manera elegante. Más específicamente, el error o excepción no debe causar un error crítico que haga que tu programa se detenga. Una declaración Try-Except es un bloque de código que permite a tu programa tomar acciones alternativas en caso de que ocurra un error.

CONSTRUCCIÓN: Declaración Try-Except

```
1 try:
2     bloque de codigo 1
3 except NombreExcepcion:
4     bloque de codigo 2
```

Python intentará primero ejecutar el código en la declaración try (bloque de código 1). Si no ocurre ninguna excepción, la declaración except se omite y la ejecución de la declaración try finaliza. Si ocurre cualquier excepción, el resto de la cláusula se omite. Luego, si el tipo de excepción coincide con la excepción nombrada después de la palabra clave except (*NombreExcepcion*), se ejecutará el código en la declaración except (bloque de código 2). Si nada en este bloque detiene el programa, continuará ejecutando el resto del código fuera de los bloques de código try-except. Si la excepción no coincide con *NombreExcepcion*, se pasa a declaraciones try externas. Si no se encuentra otro manejador, entonces la ejecución se detiene con un mensaje de error.

EJEMPLO: Capturar la excepción.

```
1 In [1]: x = "6"
2         try:
3             if x > 3:
4                 print("X is larger than 3")
5         except TypeError:
6             print("Oops! x is not a valid number. Try again
7             ...")
```

Oops! x is not a valid number. Try again...

EJEMPLO: Si tu manejador está intentando capturar otro tipo de excepción que el except no captura, entonces ocurre un error y la ejecución se detiene.

```
1 In [2]: x = "6"
2         try:
3             if x > 3:
```

```

4         print("X is larger than 3")
5     except ValueError:
6         print("Oops! x is not a valid number. Try again
    ...")

```

```

-----
TypeError Traceback (most recent call last)
<ipython-input-2-899d928e7a1f> in <module>
      1 x = "6"
      2 try:
----> 3     if x > 3:
      4         print("X is larger than 3")
      5 except ValueError:
TypeError: ">" not supported between instances of "str"
and "int"

```

Por supuesto, una declaración try puede tener más de una declaración except para manejar diferentes excepciones o puedes no especificar el tipo de excepción para que el except capture cualquier excepción.

```

1 In [3]: x = "s"
2         try:
3             if x > 3:
4                 print(x)
5         except:
6             print(f"Something is wrong with x = {x}")

```

Something is wrong with x = s

EJEMPLO: Manejando múltiples excepciones.

```

1 In [4]: def test_exceptions(x):
2         try:
3             x = int(x)
4             if x > 3:
5                 print(x)
6         except TypeError:
7             print("x was not a valid number. Try again...")
8         except ValueError:
9             print("Cannot convert x to int. Try again...")

```

```

10         except:
11             print("Unexpected error")
12
13 In [5]: x = [1, 2]
14         test_exceptions(x)

```

x was not a valid number. Try again...

```

1 In [6]: x = "s"
2         test_exceptions(x)

```

Cannot convert x to int. Try again...

Otra cosa útil en Python es que podemos generar algunas excepciones en ciertos casos usando **raise**. Por ejemplo, si necesitamos que x sea menor o igual a 5, podemos usar el siguiente código para generar una excepción si x es mayor que 5. El programa mostrará nuestra excepción y detendrá la ejecución.

```

1 In [7]: x = 10
2         if x > 5:
3             raise(Exception("x should be <= 5"))

```

```

-----
Exception Traceback (most recent call last)
<ipython-input-7-99b32b52c4f8> in <module>
      2
      3 if x > 5:
----> 4     raise(Exception("x should be <= 5"))
Exception: x should be <= 5

```

¡ADVERTENCIA! Las declaraciones Try-except nunca deben usarse en lugar de buenas prácticas de programación. Por ejemplo, no debes codificar descuidadamente y luego encerrar tu programa en una declaración try-except hasta que hayas tomado todas las medidas que puedas pensar para asegurar que tu función esté funcionando correctamente.

10.4 Comprobación de Tipos

Python es un lenguaje de programación tanto fuertemente tipado como dinámicamente tipado. Esto significa que cualquier variable puede adoptar cualquier tipo de datos en cualquier momento (esta es la parte dinámicamente tipada); sin embargo, una vez que una variable se asigna a un tipo, no puede cambiar. Por ejemplo, puedes escribir `x = 1` inmediatamente seguido de `x = "s"` porque Python es un lenguaje dinámicamente tipado. No puedes ejecutar `3 + 5` porque es un lenguaje fuertemente tipado, es decir, la cadena `"3"` no puede convertirse en tiempo de ejecución a un entero. En lenguajes de programación estáticamente tipados, debes declarar qué tipo de datos tendrá tu variable para ejecutarla antes de usarla, y el uso del tipo de datos de tu variable no puede cambiar dentro del alcance de una función.

En el caso de Python, no hay forma de asegurar que el usuario de tu función esté ingresando variables del tipo de datos que esperas. Por ejemplo, la función `my_adder` en el Capítulo 3 está diseñada para sumar tres números. Sin embargo, el usuario puede ingresar cadenas, listas, diccionarios o funciones, cada uno de los cuales causará diferentes niveles de problemas. Puedes hacer que tu función verifique el tipo de las variables de entrada antes de continuar y forzar un error usando la función de error.

¡PRUÉBALO! Modifica `my_adder` para comprobar que las variables de entrada son flotantes. Si alguna de las variables de entrada no es flotante, la función debe devolver un error apropiado al usuario usando la función `raise`. Prueba tu función con argumentos de entrada erróneos para verificar que se comprueban.

```

1 In [1]: def my_adder(a, b, c):
2         # type check
3         if isinstance(a, float) and isinstance(b, float)
4         and isinstance(c, float):
5             pass
6         else:
7             raise(TypeError("Inputs must be floats"))
8         out = a + b + c
9         return out
10
11 In [2]: my_adder(1.0, 2.0, 3.0)
12 Out[2]: 6.0
13
14 In [3]: my_adder(1.0, 2.0, "3.0")

```

```

-----
TypeError Traceback (most recent call last)
<ipython-input-3-14e4b71b8c1d> in <module>
----> 1 my_adder(1.0, 2.0, "3.0")
<ipython-input-1-c2a54d39e3d9> in my_adder(a, b, c)
----> 6         raise(TypeError("Inputs must be floats"))
      7
      8     out = a + b + c
      9     return out
TypeError: Input arguments must be floats

```

```

1 In [4]: my_adder(1, 2, 3)

```

```

-----
Exception Traceback (most recent call last)
<ipython-input-4-fc54adcab3d7> in <module>
----> 1 my_adder(1, 2, 3)
<ipython-input-1-c2a54d39e3d9> in my_adder(a, b, c)
      4     pass
      5     else:
----> 6         raise(TypeError("Inputs must be floats"))
      7
      8     out = a + b + c
TypeError: Inputs must be floats

```

Nótese que 1, 2, 3 son enteros en lugar de flotantes, por lo tanto, lanzó un mensaje de error, y necesitamos cambiar la función para asegurar que cualquier número será sumado.

```

1 In [5]: def my_adder(a, b, c):
2         # type check
3         if isinstance(a, (float, int, complex)) and
4           isinstance(b, (float, int, complex)) and isinstance(c, (
5           float, int, complex)):
6             pass
7         else:
8             raise(TypeError("Inputs must be numbers"))
9         out = a + b + c
10        return out

```

```

10 In [6]: my_adder(1, 2, 3)
11 Out[6]: 6
12
13 In [7]: my_adder(1.0, 2, 3)
14 Out[7]: 6.0
15
16 In [8]: my_adder(1j, 2+2j, 3+2j)
17 Out[8]: (5+5j)

```

10.5 Depuración

La depuración es el proceso de eliminar sistemáticamente errores, o *bugs*, de tu código. Python tiene funcionalidades que pueden asistirte al depurar. La herramienta de depuración estándar en Python es `pdb` (Python DeBugger) para depuración interactiva. Te permite avanzar paso a paso a través del código para descubrir qué podría estar causando un error difícil. La versión IPython de esto es `ipdb` (IPython DeBugger). No cubriremos demasiado sobre esto aquí; consulta la documentación³ para más detalles. En esta sección, se introducirán pasos básicos de depuración en el cuaderno Jupyter. Te mostraremos cómo usar dos comandos mágicos realmente útiles `%debug` y `%pdb` para encontrar el código que causa problemas.

Hay dos formas de depurar tu código: (1) activar el depurador cuando te encuentres con una excepción; y (2) activar el depurador antes de ejecutar el código.

10.5.1 Activar el Depurador Después de Encontrar una Excepción

Si ejecutamos el código que se detiene en una excepción, podemos llamar a `%debug`. Por ejemplo, tenemos una función que eleva al cuadrado el número de entrada y luego lo suma a sí mismo, como se muestra a continuación:

```

1 In [1]: def square_number(x):
2         sq = x**2
3         sq += x
4         return sq
5
6 In [2]: square_number("10")

```

³<https://docs.python.org/3/library/pdb.html>


```

-----
TypeError Traceback (most recent call last)
<ipython-input-2-e0b77a2957d5> in <module>
----> 1 square_number("10")
<ipython-input-1-3fc6a3900214> in square_number(x)
      1 def square_number(x):
      2
----> 3     sq = x**2
      4     sq += x
      5
TypeError: unsupported operand type(s) for ** or pow(): "str" and "int"

```

Después de localizar esta excepción, podemos activar el depurador usando el comando mágico `%debug`, que abre un depurador interactivo, momento en el cual puedes escribir comandos en el depurador para obtener información útil.

```
1 In [3]: %debug
```

```

> <ipython-input-1-3fc6a3900214>(3)square_number()
      1 def square_number(x):
      2
----> 3     sq = x**2
      4     sq += x
      5

```

```
ipdb> h
```

```
Documented commands (type help <topic>):
```

```
=====
```

EOF	cl	disable	interact	next	psource	rv	unt
a	clear	display	j	p	q	s	until
alias	commands	down	jump	pdef	quit	source	up
args	condition	enable	l	pdoc	r	step	w
b	cont	exit	list	pfile	restart	tbreak	whatis
break	continue	h	ll	pinfo	return	u	where
bt	d	help	longlist	pinfo2	retval	unalias	
c	debug	ignore	n	pp	run	undisplay	

Miscellaneous help topics:

=====

exec pdb

ipdb> p x

'10'

ipdb> type(x)

<class 'str'>

ipdb> p locals()

{'x': '10'}

ipdb> q

Puedes ver que después de activar el ipdb, podemos escribir comandos para obtener la información del código. En el ejemplo anterior, escribimos los siguientes comandos:

- h para obtener una lista de funciones de ayuda
- p x para imprimir el valor de x
- type(x) para obtener el tipo de x
- p locals() para imprimir todas las variables locales

Hay algunos comandos más frecuentes que puedes escribir en el pdb, como:

- n(ext) línea y ejecutar esta
- c(ontinue) ejecutar hasta el próximo punto de interrupción
- p(rint) imprimir variables
- l(ist) dónde estás
- Enter repetir el comando anterior
- s(tep) entrar en una subrutina
- r(eturn) salir de una subrutina
- h(elp) obtener ayuda
- q(uit) salir del depurador

10.5.2 Activar el Depurador Antes de Ejecutar el Código

También podemos activar el depurador antes de ejecutar el código y luego desactivarlo una vez que hayamos terminado de ejecutar el código.

```
1 In [4]: %pdb on
```

Automatic pdb calling has been turned ON

```
1 In [5]: square_number("10")
```

```
-----
TypeError Traceback (most recent call last)
<ipython-input-5-e0b77a2957d5> in <module>
----> 1 square_number("10")
<ipython-input-1-3fc6a3900214> in square_number(x)
      1 def square_number(x):
      2
----> 3     sq = x**2
      4     sq += x
      5
TypeError: unsupported operand type(s) for ** or pow(): "str" and "int"

> <ipython-input-1-3fc6a3900214>(3)square_number()
      1 def square_number(x):
      2
----> 3     sq = x**2
      4     sq += x
      5

ipdb> p x
"10"
ipdb> c
```

```
1 In [6]: # let's turn off the debugger
2         %pdb off
```

Automatic pdb calling has been turned OFF

10.5.3 Añadir un Punto de Interrupción

A menudo es muy útil insertar un punto de interrupción en tu código. Un punto de interrupción es una línea en tu código en la que Python se detendrá cuando se ejecute la función.

```

1 In [7]: import pdb
2
3 In [8]: def square_number(x):
4         sq = x**2
5         # we add a breakpoint here
6         pdb.set_trace()
7         sq += x
8         return sq
9
10 In [9]: square_number(3)

```

```

> <ipython-input-8-e48ec2675aea>(8)square_number()
-> sq += x
(Pdb) l
      3      sq = x**2
      4
      5      # we add a breakpoint here
      6      pdb.set_trace()
      7
      8 -> sq += x
      9
     10      return sq
[EOF]
(Pdb) p x
3
(Pdb) p sq
9
(Pdb) c
Out[9]: 12

```

Después de añadir `pdb.set_trace()`, el programa se detuvo en esta línea y activó el depurador `pdb`. Ahora podemos comprobar todos los valores de las variables que se asignaron antes de esta línea y usar el comando `c` para continuar la ejecución.

Usar el depurador de Python puede ser extremadamente útil para encontrar y corregir errores en tu código. Te animamos a usar el depurador para programas grandes.

10.6 Resumen y Problemas

10.6.1 Resumen

1. Los errores son inevitables al codificar. Los errores son importantes porque te dicen que algo no está funcionando como pretendías.
2. Hay tres tipos de errores: errores de sintaxis, excepciones y errores lógicos.
3. Puedes reducir la cantidad de errores en tu codificación con buenas prácticas de programación.
4. Las declaraciones Try-except se pueden usar para manejar excepciones sin detener tu código.
5. El Depurador es una herramienta de Python para ayudarte a encontrar errores.

10.6.2 Problemas

Ninguno, ¡diviértete con lo que aprendimos en este capítulo!

LECTURA Y ESCRITURA DE DATOS

Archivos TXT

Hasta ahora, hemos usado la función `print` para mostrar los datos en la pantalla. Pero hay muchas maneras de almacenar datos en tu disco y compartirlos con otros programas o colegas. Por ejemplo, si tenemos algunas cadenas en este notebook pero queremos usarlas en otro notebook, la forma más fácil es almacenar las cadenas en un archivo de texto primero y luego abrirlo en otro notebook. Un **archivo de texto**, que a menudo aparece con una extensión `.txt`, es un archivo que contiene solo texto plano, que se define como una secuencia pura de códigos de caracteres. Ten en cuenta que los programas que escribes y los programas que leen tus archivos de texto generalmente esperarán que el archivo de texto esté en un formato determinado, es decir, organizado de una manera específica.

Para trabajar con archivos de texto, necesitamos usar la función `open`, que devuelve un **objeto archivo**. Se usa comúnmente con dos argumentos:

```
1 f = open(nombre_archivo, modo)
```

La `f` anterior es el objeto archivo devuelto. El `nombre_archivo` toma una cadena que le dice a la computadora la ubicación del archivo que quieres abrir, y `modo` es otra cadena que contiene algunos caracteres, que describe la forma en que se utilizará el archivo. Los modos comunes son:

- `r`", este es el modo predeterminado, que abre un archivo para lectura.
- `w`", este modo abre un archivo para escritura. Si el archivo no existe, crea un nuevo archivo.

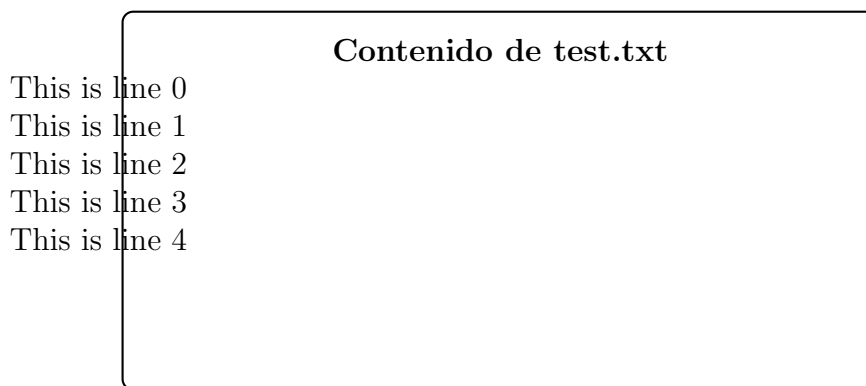
- `.a`", abre un archivo en modo añadir (append), para que puedas añadir datos al final del archivo. Si el archivo no existe, crea un nuevo archivo.
- `"b"`, abre un archivo en modo binario.
- `r+`", abre un archivo (no lo crea) para lectura y escritura.
- `"w+"`, abre o crea un archivo para escritura y lectura, descarta el contenido existente.
- `.a+`", abre o crea un archivo para lectura y escritura, y añade datos al final del archivo.

Escribiendo en un Archivo

```
1 In [1]: f = open("test.txt", "w")
2         for i in range(5):
3             f.write(f"This is line {i}\n")
4         f.close()
```

Listing 31: Creando y escribiendo en un archivo de texto.

En el código anterior, primero abrimos un objeto archivo `f` con el nombre de archivo `"test.txt"`. Usamos `"w"` para el modo, lo que indica que queremos escribir código. Escribimos cinco líneas (observa la nueva línea `\n` al final de la cadena), y luego cerramos el objeto archivo. El contenido del archivo se muestra en la Fig. 4.



Contenido de test.txt

This is line 0
This is line 1
This is line 2
This is line 3
This is line 4

Figura 4: El contenido en el archivo de texto que escribimos.

Nota

Es una buena práctica cerrar el archivo usando `f.close()` al final. Si no los cierras tú mismo, Python eventualmente los cerrará por ti. Ten en cuenta que a veces, al escribir en un archivo, es posible que los datos no se escriban en el disco hasta que cierres el archivo. Cuanto más tiempo mantengas el archivo abierto, mayor será la probabilidad de perder tus datos.

Añadiendo a un Archivo

A continuación, añadiremos una cadena al archivo `test.txt`. Es muy similar a cómo escribimos el archivo, con una sola diferencia: cambiamos el modo a `"a"` en su lugar. Ve los resultados en la Fig. 5.

Contenido de test.txt después de añadir

This is line 0
This is line 1
This is line 2
This is line 3
This is line 4
This is another line

Figura 5: Añadir una línea al final de un archivo existente.

```
1 In [2]: f = open("test.txt", "a")
2         f.write("This is another line\n")
3         f.close()
```

Listing 32: Añadiendo a un archivo de texto.

Leyendo un Archivo

Podemos leer un archivo del disco y almacenar todo su contenido en una variable. Leamos el archivo `test.txt` que creamos arriba y almacenemos todo su contenido en una variable `content`.

```
1 In [3]: f = open("./test.txt", "r")
2         content = f.read()
3         f.close()
4         print(content)
5 This is line 0
6 This is line 1
7 This is line 2
8 This is line 3
9 This is line 4
10 This is another line
```

Listing 33: Leyendo un archivo de texto.

De esta manera, podemos almacenar todas las líneas del archivo en una variable de una sola cadena. Para probar que esto es una cadena, podemos verificar el tipo de la variable `content`.

```
1 In [4]: type(content)
2 Out[4]: str
```

Listing 34: Verificando el tipo de contenido.

A veces queremos leer el contenido de los archivos línea por línea y almacenarlo en una lista: usar `f.readlines()` logrará esto.

```
1 In [5]: f = open("./test.txt", "r")
2         contents = f.readlines()
3         f.close()
4         print(contents)
5 ["This is line 0\n", "This is line 1\n", "This is line 2\n",
6 "This is line 3\n", "This is line 4\n", "This is another line\n"]
7
8 In [6]: type(contents)
9 Out[6]: list
```

Listing 35: Leyendo un archivo línea por línea.

Manejando Números y Arrays

Dado que trabajaremos con métodos numéricos más adelante y a menudo trabajaremos con números o arrays, podemos usar los métodos anteriores para guardar los números o arrays en un archivo y volver a leerlos en la memoria. Aunque es viable, es un poco torpe. En cambio, los programadores suelen usar el paquete NumPy para guardar/leer directamente un array. A continuación se presenta un ejemplo.

```

1 In [7]: import numpy as np
2
3 In [8]: arr = np.array([[1.20, 2.20, 3.00], [4.14, 5.65,
4           6.42]])
5           arr
6 Out[8]: array([[1.2 , 2.2 , 3.  ],
7           [4.14, 5.65, 6.42]])
8 In [9]: np.savetxt("my_arr.txt", arr, fmt="%.2f", header = "
           Col1 Col2 Col3")

```

Listing 36: Guardando y leyendo un array de NumPy en un archivo de texto.

El ejemplo anterior demuestra cómo guardar un array 2D en un archivo de texto usando `np.savetxt`. El primer argumento es el nombre del archivo, el segundo argumento es el objeto que deseamos guardar, y el tercer argumento es para definir el formato de la salida (usamos `"%.2f"` para indicar que queremos los números de salida con dos decimales). El cuarto argumento es el encabezado que deseamos escribir en el archivo. Ve los resultados en la Fig. 6.

Contenido de my_arr.txt			
#	Col1	Col2	Col3
	1.20	2.20	3.00
	4.14	5.65	6.42

Figura 6: El array de NumPy que guardamos en el archivo.

```

1 In [10]: my_arr = np.loadtxt("my_arr.txt")
2           my_arr
3 Out[10]: array([[1.2 , 2.2 , 3.  ],

```

```
4 [4.14, 5.65, 6.42]])
```

Listing 37: Leyendo el archivo de texto con NumPy.

Leer el archivo directamente en un array es muy simple usando la función `np.loadtxt`, que omite el primer encabezado. Hay muchos argumentos diferentes que pueden controlar cómo se lee un archivo. Se explorará una discusión adicional sobre este tema en la próxima sección; sin embargo, no entraremos en muchos detalles aquí más allá de lo presentado arriba. Consulta la documentación o usa el signo de interrogación si necesitas ayuda.

Archivos CSV

A menudo, los datos científicos se almacenan en el formato de archivo de **valores separados por comas (CSV)**, que es un archivo de texto delimitado que usa una coma para separar valores. Es un formato muy útil que puede almacenar grandes tablas de datos (números y texto) en texto plano. Cada línea (fila) en los datos es un registro de datos, y cada registro consta de uno o más campos separados por comas. También se puede abrir usando Microsoft Excel para visualizar las filas y columnas.

Python tiene su propio módulo `csv` para leer y escribir un archivo CSV; no vamos a discutir esto más a fondo. Los detalles de este módulo están disponibles en la documentación⁴. En su lugar, usaremos el paquete **NumPy** para manejar archivos CSV, porque es preferible leer un archivo CSV directamente en un array de NumPy.

Escribiendo y Abriendo un Archivo CSV

A continuación se presenta un ejemplo simple de generación de 100 filas y 5 columnas de datos.

```
1 In [1]: import numpy as np
2
3 In [2]: data = np.random.random((100, 5))
4
5 In [3]: np.savetxt("test.csv", data, fmt="%.2f",
6               delimiter=",", header="c1, c2, c3, c4, c5"
               )
```

Listing 38: Generando y guardando datos CSV con NumPy.

⁴<https://docs.python.org/3/library/csv.html>

Primero, generamos algunos datos aleatorios para 100 filas y 5 columnas usando la función `np.random` y los asignamos a una variable `data`. Luego usamos la función `np.savetxt` para guardar los datos en un archivo CSV. Observa que los primeros tres argumentos son los mismos que los utilizados en la sección anterior, pero en este caso establecemos el argumento `delimiter` en `","`, lo que indica que queremos separar los datos usando una coma.

Ahora, podemos abrir el archivo CSV usando Microsoft Excel; ver Fig. 7. También podemos abrir el archivo CSV usando un editor de texto; observa que los valores están separados por comas (Fig. 8).

Archivo CSV abierto en Excel				
A	B	C	D	E
0.84	0.99	0.56	0.24	0.71
...				

Figura 7: Abriendo el archivo csv usando Microsoft Excel.

Archivo CSV abierto en un editor de texto				
c1, c2, c3, c4, c5				
0.84, 0.99, 0.56, 0.24, 0.71				
0.33, 0.80, 0.32, 0.28, 0.83				

Figura 8: Abriendo el archivo csv usando un editor de texto.

Leyendo un Archivo CSV

Como antes, podemos leer el archivo CSV usando la función `np.loadtxt`. Si leemos el archivo CSV que acabamos de guardar en el disco en una variable

`my_csv` y mostramos las primeras 5 filas, necesitamos usar el delimitador nuevamente para especificar que los datos en el archivo están separados por comas.

```
1 In [4]: my_csv = np.loadtxt("./test.csv", delimiter=",")
2         my_csv[:5, :]
3 Out[4]: array([[0.84, 0.99, 0.56, 0.24, 0.71],
4               [0.33, 0.8 , 0.32, 0.28, 0.83],
5               [0.89, 0.19, 0.25, 0.63, 0.84],
6               [0.08, 0.49, 0.76, 0.34, 0.69],
7               [0.66, 0.65, 0.73, 0.48, 0.12]])
```

Listing 39: Leyendo un archivo CSV con NumPy.

Más Allá de NumPy

Aunque NumPy es muy conveniente cuando se trata de archivos CSV, existen numerosos paquetes capaces de manejar archivos CSV. El paquete **Pandas** es popular y puede manejar fácilmente datos tabulares en un **DataFrame**. Te animamos a explorar por tu cuenta estas diferentes formas de manejar archivos CSV.

Archivos Pickle

Esta sección introducirá otra forma de almacenar datos en el disco, llamada **pickle**. Hemos hablado sobre guardar datos en archivos de texto o CSV, pero en ciertos casos, queremos almacenar diccionarios, tuplas, listas o cualquier otro tipo de dato en el disco y usarlos más tarde o enviarlos a colegas. Aquí es donde entra pickle; puede **serializar** objetos para que puedan guardarse en un archivo y cargarse nuevamente más tarde.

Pickle se puede usar para serializar estructuras de objetos de Python, lo que se refiere al proceso de convertir un objeto en la memoria en un flujo de bytes que puede almacenarse como un archivo binario en el disco. Cuando lo cargamos de nuevo en un programa de Python, este archivo binario puede ser **deserializado** de vuelta a un objeto de Python.

Escribiendo en un Archivo Pickle

```
1 In [1]: import pickle
2
3 In [2]: dict_a = {"A":0, "B":1, "C":2}
4         pickle.dump(dict_a, open("test.pkl", "wb"))
```

Listing 40: Guardando un diccionario con pickle.

Para usar pickle y serializar un objeto, usamos la función `pickle.dump` que toma dos argumentos: el primero es el objeto, y el segundo argumento es un objeto archivo devuelto por la función `open`. Observa que el modo de la función `open` es `"wb"` lo que indica que está escribiendo en un archivo binario.

Leyendo un Archivo Pickle

A continuación, cargamos el archivo pickle que acabamos de guardar en el disco usando la función `pickle.load`.

```
1 In [3]: my_dict = pickle.load(open("./test.pkl", "rb"))
2         my_dict
3 Out[3]: {"A": 0, "B": 1, "C": 2}
```

Listing 41: Cargando un archivo pickle.

Podemos ver que la carga de un archivo pickle es muy similar al proceso de guardado, pero aquí el modo de la función `open` es `"rb"`, lo que indica que está leyendo el archivo binario. Esta función deserializa el archivo binario de vuelta al objeto original, que en este caso es un diccionario. Esta es también una de las razones por las que el formato "pickle" es popular; es muy fácil almacenar y cargar una estructura de datos de Python sin agregar código extra para cambiarla.

Leyendo un Archivo Pickle de Python 2

A veces, es posible que necesites abrir un archivo pickle de algún colega que lo generó usando Python 2 en lugar de Python 3. Puedes o bien abrirlo (unpickle) usando Python 2, o usar Python 3 con `encoding="latin1"` en la función `pickle.load`.

```
1 infile = open(filename, "rb")
2 new_dict = pickle.load(infile, encoding="latin1")
```

Listing 42: Leyendo un pickle de Python 2 en Python 3.

Advertencia

Una desventaja de los archivos pickle es que no están en un formato de archivo universal, lo que significa que no es fácil para otros lenguajes de programación usarlo. Los archivos TXT y CSV se pueden compartir fácilmente con otros colegas que no usan Python, y pueden abrirlos usando R, Matlab®, Java, etc. Pero los archivos pickle están especialmente diseñados para Python y los datos no están diseñados para funcionar con otros lenguajes de programación.

Archivos JSON

JSON es otro formato que estamos introduciendo. Significa *JavaScript Object Notation*. Un archivo JSON generalmente termina con la extensión “.json”. A diferencia de pickle, que depende de Python, JSON es un formato de datos independiente del lenguaje, lo que lo hace atractivo de usar. Además, generalmente ocupa menos espacio en el disco y manipular un archivo JSON es más rápido en comparación con pickle; busca en línea más detalles sobre las ventajas y desventajas de usar JSON versus pickle. Por lo tanto, es una buena práctica almacenar tus datos usando JSON. Esta sección explora brevemente cómo manejar archivos JSON en Python.

Formato JSON

El texto en JSON se representa usando cadenas entre comillas que contienen valores en pares clave-valor dentro de {}; la estructura debería parecer muy similar, ya que es casi idéntica al diccionario usado en Python. Por ejemplo,

```
1 {  
2     "school": "UC Berkeley",  
3     "address": {  
4         "city": "Berkeley",  
5         "state": "California",  
6         "postal": "94720"  
7     },  
8     "list": [  
9         "student 1",  
10        "student 2",  
11        "student 3"
```

```
12 ]  
13 }
```

Listing 43: Ejemplo de estructura JSON.

Escribiendo un Archivo JSON

La forma más fácil de manejar JSON en Python es usar la biblioteca `json`. Hay varias otras bibliotecas disponibles, como `simplejson`, `jyson`, etc. Esta sección se limitará a usar `json`, que es compatible de forma nativa con Python para permitirte escribir y cargar archivos JSON.

```
1 In [1]: import json  
2  
3 In [2]: school = {  
4     "school": "UC Berkeley",  
5     "address": {  
6         "city": "Berkeley",  
7         "state": "California",  
8         "postal": "94720"  
9     },  
10    "list": [  
11        "student 1",  
12        "student 2",  
13        "student 3"  
14    ],  
15    "array": [1, 2, 3]  
16 }  
17 json.dump(school, open("school.json", "w"))
```

Listing 44: Guardando un diccionario como JSON.

Para serializar un objeto usando JSON, usamos la función `json.dump` que toma dos argumentos: el primero es el objeto, y el segundo argumento es un objeto archivo devuelto por la función `open`. Observa que el modo de la función `open` es `"w"`, lo que indica que es un archivo de “escritura”.

Leyendo un Archivo JSON

Ahora cargamos el archivo JSON que acabamos de guardar en el disco usando la función `json.load`.

```
1 In [3]: my_school = json.load(open("./school.json", "r"))
```



```
2 my_school
```

Listing 45: Leyendo un archivo JSON.

Podemos ver que el uso de `json` es en realidad muy similar a `pickle` en la última sección. JSON admite cadenas y números, así como listas, tuplas y objetos anidados. Sugerimos explorar estas diferentes opciones por tu cuenta.

Archivos HDF5

La computación científica a menudo necesita almacenar grandes cantidades de datos con acceso rápido; los formatos de archivo que introdujimos anteriormente no son aplicables. Para almacenar grandes cantidades de datos, **HDF5** (Formato de Datos Jerárquico) es la solución. Es un potente formato de datos binario sin límite en el tamaño del archivo. Proporciona E/S (entrada/salida) paralela y lleva a cabo un montón de optimizaciones de bajo nivel "bajo el capó" para acelerar las consultas y minimizar los requisitos de almacenamiento.

Un archivo HDF5 guarda dos tipos de objetos: **datasets** (conjuntos de datos), que son colecciones de datos similares a arrays (como los arrays de NumPy), y **groups** (grupos), que son contenedores similares a carpetas que contienen conjuntos de datos y otros grupos. También hay **atributos** que se pueden asociar con los conjuntos de datos y los grupos para describir algunas propiedades. El llamado *jerárquico* en HDF5 se refiere al hecho de que los datos se pueden guardar como un sistema de archivos, con estructuras similares a carpetas, como carpeta, subcarpeta (en HDF5, se llama grupo, subgrupo). Los grupos funcionan como diccionarios con claves y valores: las claves son los nombres de los grupos y los valores son los subgrupos o conjuntos de datos.

Para poder leer/escribir HDF5 en Python, hay algunos paquetes o envoltorios que sirven para este propósito. Los dos paquetes más comunes son PyTables⁵ y h5py⁶. Aquí solo introduciremos h5py. Puedes instalar h5py usando conda (consulta el Capítulo 1 si necesitas un repaso).

Una vez que h5py esté instalado, sigue la Guía de Inicio Rápido en la documentación de h5py⁷. Para tu información, a continuación se presenta

⁵<https://www.pytables.org>

⁶<https://www.h5py.org>

⁷<http://docs.h5py.org/en/latest/quick.html>

un ejemplo para demostrar cómo crear y leer un archivo HDF5. Primero importaremos NumPy y h5py.

```
1 In [1]: import numpy as np
2         import h5py
```

Listing 46: Importando módulos para HDF5.

```
1 In [2]: # Generate random data for recording
2         acc_1 = np.random.random(1000)
3         station_number_1 = "1"
4         # unix timestamp
5         start_time_1 = 1542000276
6         # time interval for recording
7         dt_1 = 0.04
8         location_1 = "Berkeley"
9
10        acc_2 = np.random.random(500)
11        station_number_2 = "2"
12        start_time_2 = 1542000576
13        dt_2 = 0.01
14        location_2 = "Oakland"
15
16 In [3]: hf = h5py.File("station.hdf5", "w")
17
18 In [4]: hf["/acc/1/data"] = acc_1
19         hf["/acc/1/data"].attrs["dt"] = dt_1
20         hf["/acc/1/data"].attrs["start_time"] = start_time_1
21         hf["/acc/1/data"].attrs["location"] = location_1
22
23         hf["/acc/2/data"] = acc_2
24         hf["/acc/2/data"].attrs["dt"] = dt_2
25         hf["/acc/2/data"].attrs["start_time"] = start_time_2
26         hf["/acc/2/data"].attrs["location"] = location_2
27
28         hf["/gps/1/data"] = np.random.random(100)
29         hf["/gps/1/data"].attrs["dt"] = 60
30         hf["/gps/1/data"].attrs["start_time"] = 1542000000
31         hf["/gps/1/data"].attrs["location"] = "San Francisco"
32
33 In [5]: hf.close()
```

Listing 47: Creando y guardando datos en un archivo HDF5.

El código anterior muestra los conceptos centrales en HDF5: los grupos, conjuntos de datos y atributos. Primero, creamos un objeto HDF5 para escritura: `station.hdf5`. Luego almacenamos los datos en dos grupos de nivel

superior: `acc` y `gps`. Ambos grupos de nivel superior contienen subgrupos etiquetados como 1 o 2 para indicar los nombres de las estaciones. Cada estación contendrá el subgrupo de siguiente nivel, es decir, `data`, que se utiliza para almacenar los datos del array que recopilamos. A continuación, añadimos atributos a los grupos o a los datos. En este caso, hemos añadido `dt`, `start_time` y `location` como atributos a los conjuntos de datos almacenados aquí. Puedes ver que es bastante similar a una estructura de carpetas, con los datos `acc_1` guardados en `/acc/1/data`. Por último, cerramos el objeto archivo.

Guardar datos en HDF5 es fácil. También podemos usar las funciones `create_dataset` y `create_group` como se muestra en la guía de inicio rápido⁸.

Leyendo un Archivo HDF5

Ahora supongamos que envías `station.hdf5` a un colega, que quiere acceder a los datos. Así es como lo hará.

```

1 In [6]: hf_in = h5py.File("station.hdf5", "r")
2
3 In [7]: list(hf_in.keys())
4 Out[7]: ["acc", "gps"]
5
6 In [8]: acc = hf_in["acc"]
7
8 In [9]: list(acc.keys())
9 Out[9]: ["1", "2"]
10
11 In [10]: data_1 = hf_in["acc/1/data"]
12
13 In [11]: data_1[:10] # Nota: .value está obsoleto, usar
14 Out[11]: array([0.41820889, 0.89832446, 0.40229251,
15               0.41287538,
16               0.16173359, 0.75855904, 0.89288185,
17               0.82944522,
18               0.84228139, 0.50365515])
19
20 In [12]: list(data_1.attrs)
21 Out[12]: ["dt", "start_time", "location"]
22
23 In [13]: data_1.attrs["dt"]

```

⁸<http://docs.h5py.org/en/latest/quick.html>

```
22 Out [13]: 0.04
23
24 In [14]: data_1.attrs["location"]
25 Out [14]: "Berkeley"
```

Listing 48: Leyendo un archivo HDF5.

Leer un HDF5 resulta ser fácil también usando `h5py`. Después de leer el HDF5 en `hf_in`, podemos ver qué grupos hay en el HDF5 usando la función `keys`. Luego podemos acceder a los miembros del grupo y ver qué hay en los subgrupos como `hf_in[.acc]`, o especificar directamente la ruta a los conjuntos de datos como `hf_in[.acc/1/data]` y obtener los datos del array. Recuerda que los atributos asociados con los datos también se pueden acceder como un diccionario.

Resumen y Problemas

Resumen

1. Los datos a menudo deben almacenarse en un disco para una sesión posterior de Python o para ser leídos por otros programas.
2. Es posible que otros programas tengan que leer los datos creados por Python.
3. Python tiene funciones para leer y escribir datos en varias formas estándar: TXT, CSV, pickle y HDF5.

Problemas y Soluciones

1. **Problema 1:** Crea una lista y guárdala en un archivo de texto para que cada elemento de la lista ocupe una línea.

Solución: Iteramos sobre la lista y escribimos cada elemento seguido de un salto de línea.

```
1 mi_lista = ["manzana", "banana", "cereza", "dátil"]
2 with open("lista.txt", "w") as f:
3     for item in mi_lista:
4         f.write(item + "\n")
5
```

```
6 # Para verificar:
7 with open("lista.txt", "r") as f:
8     print(f.read())
9
```

Listing 49: Solución Problema 1

2. **Problema 2:** Guarda la misma lista del Problema 1 en un archivo CSV.

Solución: Usamos el módulo `csv` de Python o simplemente escribimos los elementos separados por comas. Usaremos `csv.writer` para mayor claridad.

```
1 import csv
2 mi_lista = ["manzana", "banana", "cereza", "dátil"]
3 with open("lista.csv", "w", newline="") as f:
4     writer = csv.writer(f)
5     # Escribir cada elemento como una fila de un solo
6     # campo, o como una sola fila con todos los elementos
7     for item in mi_lista:
8         writer.writerow([item]) # Cada elemento en su
9         # propia fila
10
11 # Para una sola fila con todos los elementos separados
12 # por comas:
13 with open("lista_fila.csv", "w", newline="") as f:
14     writer = csv.writer(f)
15     writer.writerow(mi_lista)
```

Listing 50: Solución Problema 2

3. **Problema 3:** Crea un array 2D de NumPy, guárdalo en un archivo CSV y vuelve a leerlo en un array 2D.

Solución: Usamos `np.savetxt` con el delimitador de coma y `np.loadtxt` para leer.

```
1 import numpy as np
2
3 arr_2d = np.array([[1.5, 2.3, 4.1], [5.2, 6.8, 7.4],
4                   [8.1, 9.0, 10.5]])
5 np.savetxt("array_2d.csv", arr_2d, delimiter=",", fmt="
```

```
5
6 arr_leido = np.loadtxt("array_2d.csv", delimiter=",")
7 print(arr_leido)
8
```

Listing 51: Solución Problema 3

4. **Problema 4:** Guarda el mismo array del Problema 3 en un archivo pickle y cárgalo de vuelta.

Solución: Usamos `pickle.dump` y `pickle.load`.

```
1 import pickle
2 import numpy as np
3
4 arr_2d = np.array([[1.5, 2.3, 4.1], [5.2, 6.8, 7.4],
5                   [8.1, 9.0, 10.5]])
6
7 with open("array.pkl", "wb") as f:
8     pickle.dump(arr_2d, f)
9
10 with open("array.pkl", "rb") as f:
11     arr_cargado = pickle.load(f)
12 print(arr_cargado)
13
```

Listing 52: Solución Problema 4

5. **Problema 5:** Crea un diccionario y guárdalo en un archivo JSON.

Solución: Usamos `json.dump`.

```
1 import json
2
3 datos = {
4     "nombre": "Ana",
5     "edad": 30,
6     "ciudad": "Madrid",
7     "habilidades": ["Python", "Análisis de datos"]
8 }
9
10 with open("datos.json", "w") as f:
11     json.dump(datos, f, indent=4) # indent para que sea
    legible
```

12

Listing 53: Solución Problema 5

6. **Problema 6:** Crea un array 1D de NumPy, guárdalo en un archivo JSON con la clave “data”. Luego cárgalo de vuelta.

Solución: Convertimos el array de NumPy a una lista de Python antes de guardarlo en JSON, ya que JSON no maneja arrays de NumPy directamente. Al cargar, reconstruimos el array.

```
1 import numpy as np
2 import json
3
4 arr_1d = np.array([10, 20, 30, 40, 50])
5 datos_json = {"data": arr_1d.tolist()} # Convertir a
   lista
6
7 with open("datos_array.json", "w") as f:
8     json.dump(datos_json, f)
9
10 with open("datos_array.json", "r") as f:
11     datos_cargados = json.load(f)
12
13 arr_cargado = np.array(datos_cargados["data"])
14 print(arr_cargado)
15
```

Listing 54: Solución Problema 6

VISUALIZACIÓN Y TRAZADO DE GRÁFICAS

Trazado 2D

En Python, `matplotlib` es el paquete más importante para trazar datos. Visita la galería de `matplotlib`⁹ para hacerte una idea de lo que se puede hacer. Por lo general, lo primero que debes hacer es importar el paquete `matplotlib`. En Jupyter notebook, podemos mostrar la figura directamente dentro del notebook y también tener operaciones interactivas como desplazar, acercar/alejar, etc., usando el comando mágico `%matplotlib notebook`. A continuación, se muestran algunos ejemplos.

```
1 In [1]: import numpy as np
2         import matplotlib.pyplot as plt
3         %matplotlib notebook
```

Listing 55: Importando módulos para graficar.

La función básica para graficar es `plot(x, y)`. La función `plot` toma dos listas/arrays, `x` e `y`, y produce una visualización de los puntos respectivos en `x` e `y`.

```
1 In [2]: x = [0, 1, 2, 3]
2         y = [0, 1, 4, 9]
3         plt.plot(x, y)
4         plt.show()
```

Listing 56: Gráfica simple de `x` vs `y`.

⁹<https://matplotlib.org/gallery/index.html>

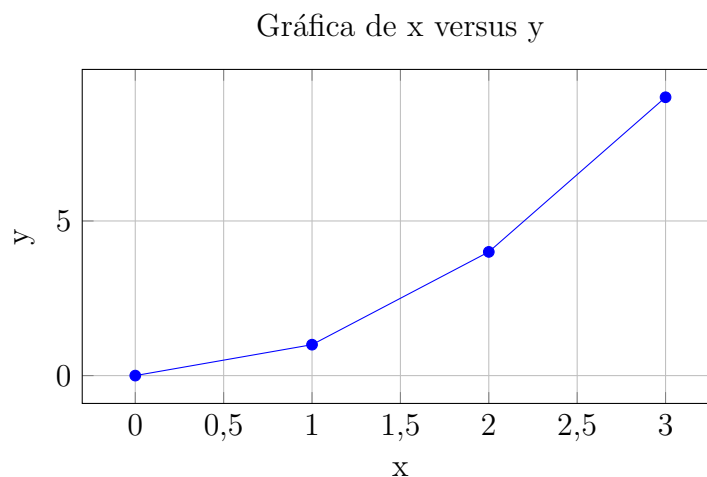


Figura 9: Gráfica de x versus y.

Observa en la Fig. 9 que, por defecto, la función `plot` conecta cada punto con una línea azul. Para hacer que la función parezca suave, puedes usar puntos de discretización más finos. La función `plt.plot` hizo el trabajo principal para trazar la figura, y `plt.show()` le dice a Python que hemos terminado de trazar y le pide que muestre la figura. Los botones debajo de la gráfica te permiten mover la línea, acercar o alejar, o guardar la figura. Ten en cuenta que antes de trazar la Fig. 10, debes desactivar la gráfica interactiva presionando el botón de detener interacción en la parte superior derecha de la figura, de lo contrario, la siguiente figura se trazaré en el mismo marco, o usa la función mágica `%matplotlib inline` para desactivar las funciones interactivas.

```

1 In [3]: %matplotlib inline
2
3 In [4]: x = np.linspace(-5, 5, 100)
4         plt.plot(x, x**2)
5         plt.show()

```

Listing 57: Trazando la función $f(x) = x^2$.

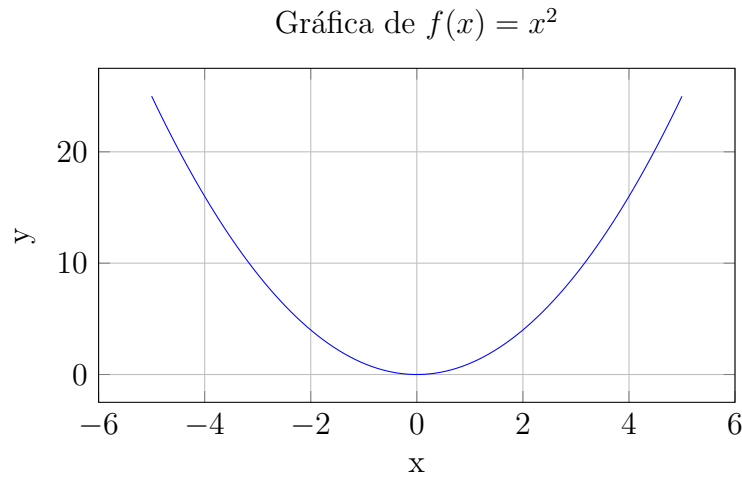


Figura 10: Gráfica de la función $f(x) = x^2$ para $-5 \leq x \leq 5$.

Para cambiar el marcador o la línea, agrega un tercer argumento de entrada a `plot`, que es una cadena que especifica el color y el estilo de línea que se usará en la gráfica. Por ejemplo, `plot(x, y, 'ro')` trazará los elementos de `x` contra los elementos de `y` usando círculos rojos `ro`". Las especificaciones posibles se muestran a continuación en la tabla (ver también Fig. 11).

Cuadro 1: Especificaciones de color y estilo de línea.

Símbolo	Descripción	Símbolo	Descripción
b	azul	s	cuadrado
g	verde	d	diamante
r	rojo	v	triángulo (abajo)
c	cian	^	triángulo (arriba)
m	magenta	!	triángulo (izquierda)
y	amarillo	!	triángulo (derecha)
k	negro	p	pentagrama
w	blanco	h	hexagrama
.	punto	-	sólida
o	círculo	:	punteada
x	marca x	-.	guiones-punto
+	más	--	rayada
*	estrella		

```
1 In [5]: x = np.linspace(-5, 5, 100)
2         plt.plot(x, x**2, "g--")
3         plt.show()
```

Listing 58: Usando una línea verde discontinua.

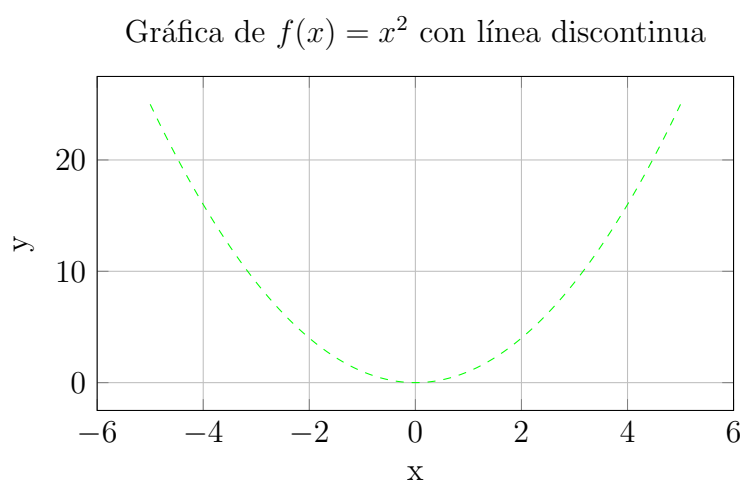


Figura 11: Gráfica de la función $f(x) = x^2$ para $-5 \leq x \leq 5$ usando línea discontinua.

Antes de la declaración `plt.show()`, puedes añadir y trazar más conjuntos de datos dentro de una misma figura (ver también Fig. 12).

```
1 In [6]: x = np.linspace(-5, 5, 20)
2         plt.plot(x, x**2, "ko")
3         plt.plot(x, x**3, "r*")
4         plt.show()
```

Listing 59: Trazando múltiples funciones.

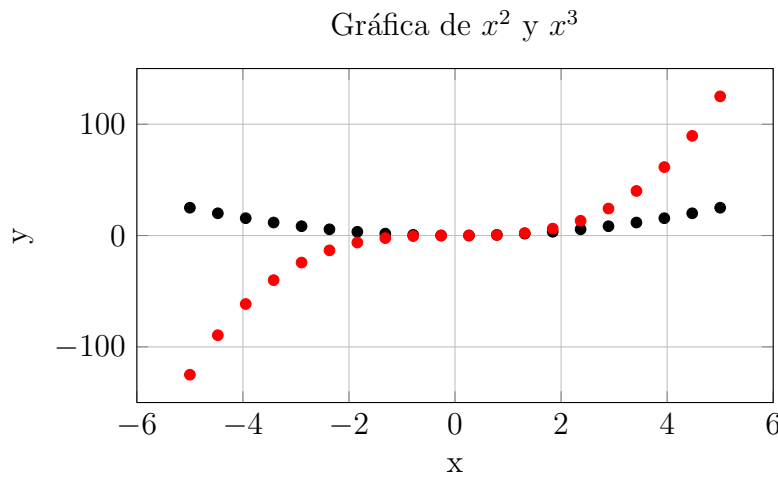


Figura 12: Usando diferentes colores y marcadores.

Es costumbre en ingeniería y ciencia proporcionar siempre a tu gráfica un título y etiquetas en los ejes para que la gente sepa de qué trata tu gráfica. A veces también quieres cambiar el tamaño de la figura. Puedes añadir un título a tu gráfica usando la función `title`, que toma como entrada una cadena y pone esa cadena como título de la gráfica. Las funciones `xlabel` y `ylabel` funcionan de la misma manera para nombrar las etiquetas de tus ejes. Para cambiar el tamaño de la figura, crea un objeto `figure` y redimensiona. Observa que cada vez que llamamos a la función `plt.figure`, creamos un nuevo objeto `figure` y dibujamos en él.

```

1 In [7]: plt.figure(figsize=(10, 6))
2         x = np.linspace(-5, 5, 20)
3         plt.plot(x, x**2, "ko")
4         plt.plot(x, x**3, "r*")
5         plt.title(f"Plot of Various Polynomials from {x[0]}
6         to {x[-1]}")
7         plt.xlabel("X axis", fontsize=18)
8         plt.ylabel("Y axis", fontsize=18)
9         plt.show()

```

Listing 60: Añadiendo título y etiquetas a los ejes.

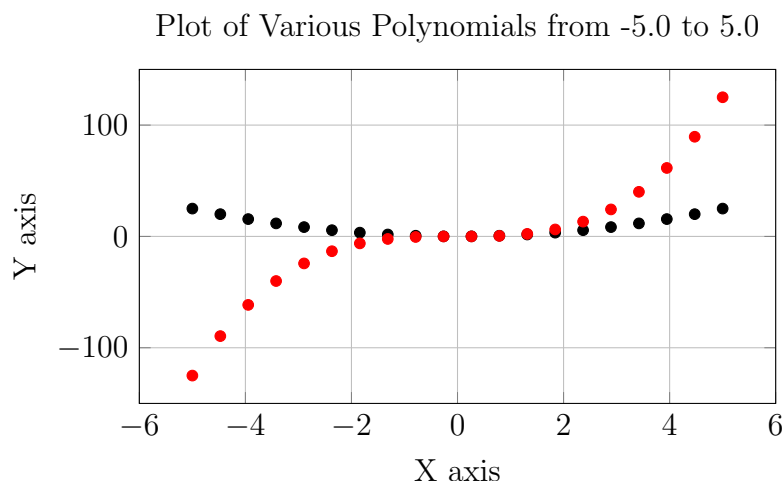


Figura 13: Añadiendo título y etiquetas a los ejes a una figura.

Como se demuestra en la Fig. 13, podemos cambiar cualquier parte de la figura, como el tamaño de la etiqueta de los ejes x e y, especificando un argumento `fontsize` en la función `plt.xlabel`. Hay algunos estilos predefinidos que cambian automáticamente el estilo. Aquí está la lista de los estilos.

```

1 In [8]: print(plt.style.available)
2 ["seaborn-dark", "seaborn-darkgrid", "seaborn-ticks",
3  "fivethirtyeight", "seaborn-whitegrid", "classic",
4  "_classic_test", "fast", "seaborn-talk",
5  "seaborn-dark-palette", "seaborn-bright",
6  "seaborn-pastel", "grayscale", "seaborn-notebook",
7  "ggplot", "seaborn-colorblind", "seaborn-muted",
8  "seaborn", "Solarize_Light2", "seaborn-paper",
9  "bmh", "tableau-colorblind10", "seaborn-white",
10 "dark_background", "seaborn-poster", "seaborn-deep"]

```

Listing 61: Lista de estilos disponibles.

Uno de mis estilos predefinidos favoritos es el estilo `seaborn`; podemos cambiarlo usando la función `plt.style.use`. Si lo cambiamos a `"seaborn-poster"`, hará que todo sea más grande (ver también Fig. 14).

```

1 In [9]: plt.style.use("seaborn-poster")
2
3 In [10]: plt.figure(figsize=(10, 6))
4          x = np.linspace(-5, 5, 20)
5          plt.plot(x, x**2, "ko")

```

```

6     plt.plot(x, x**3, "r*")
7     plt.title(f"Plot of Various Polynomials from {x[0]}
  to {x[-1]}")
8     plt.xlabel("X axis")
9     plt.ylabel("Y axis")
10    plt.show()

```

Listing 62: Usando el estilo seaborn-poster.

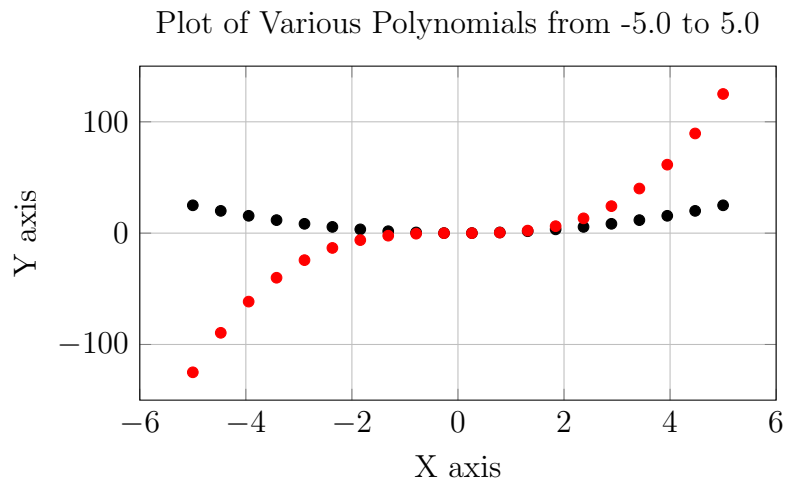


Figura 14: Usando el estilo seaborn-poster.

Puedes crear una leyenda para tu gráfica usando la función `legend`, añadiendo el argumento `label` en la función `plot`. La función `legend` también toma el argumento `loc` para indicar dónde colocar la leyenda; cámbialo de 0 a 10 y observa qué sucede (ver también Fig. 15).

```

1 In [11]: plt.figure(figsize=(10, 6))
2           x = np.linspace(-5, 5, 20)
3           plt.plot(x, x**2, "ko", label="quadratic")
4           plt.plot(x, x**3, "r*", label="cubic")
5           plt.title(f"Plot of Various Polynomials from {x[0]}
  to {x[-1]}")
6           plt.xlabel("X axis")
7           plt.ylabel("Y axis")
8           plt.legend(loc=2)
9           plt.show()

```

Listing 63: Añadiendo una leyenda.

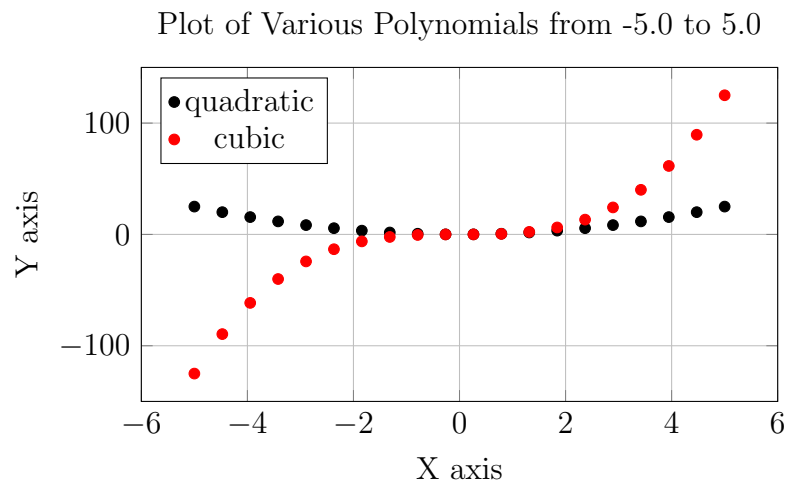


Figura 15: Usando una leyenda en la figura.

Finalmente, puedes personalizar aún más la apariencia de tus gráficas cambiando los límites de cada eje usando las funciones `xlim` o `ylim`. Usar la función `grid` activará la cuadrícula de la Fig. 16.

```

1 In [12]: plt.figure(figsize=(10, 6))
2           x = np.linspace(-5, 5, 100)
3           plt.plot(x, x**2, "ko", label="quadratic")
4           plt.plot(x, x**3, "r*", label="cubic")
5           plt.title(f"Plot of Various Polynomials from {x[0]}
6           to {x[-1]}")
7           plt.xlabel("X axis")
8           plt.ylabel("Y axis")
9           plt.legend(loc=2)
10          plt.xlim(-6, 6)
11          plt.ylim(-10, 10)
12          plt.grid()
          plt.show()

```

Listing 64: Cambiando los límites de los ejes y activando la cuadrícula.

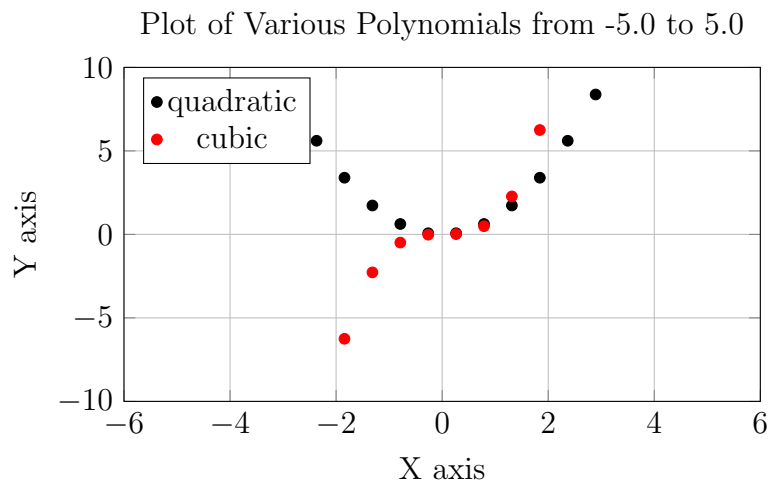


Figura 16: Cambiando los límites de la figura y activando la cuadrícula.

Podemos crear una tabla de gráficas en una sola figura usando la función `subplot`. La función `subplot` toma tres entradas: el número de filas de gráficas, el número de columnas de gráficas, y designa qué gráfica deben dibujar todas las llamadas a las funciones de trazado. Puedes moverte a una subgráfica diferente llamando a `subplot` nuevamente con una entrada diferente para la ubicación de la gráfica.

Hay varias otras funciones de trazado que grafican datos de x versus y . Algunas de ellas son `scatter`, `bar`, `loglog`, `semilogx` y `semilogy`. La función `scatter` funciona exactamente igual que `plot` excepto que por defecto usa círculos rojos (es decir, `plot(x, y, 'ro')` es equivalente a `scatter(x, y)`). La función `bar` traza barras centradas en x con altura y . Las funciones `loglog`, `semilogx` y `semilogy` trazan los datos en x e y , con los ejes x e y en escala logarítmica, el eje x en escala logarítmica y el eje y en escala lineal, y el eje y en escala logarítmica y el eje x en escala lineal, respectivamente (ver también Fig. 17).

```

1 In [13]: x = np.arange(11)
2           y = x**2
3
4           plt.figure(figsize=(14, 8))
5           plt.subplot(2, 3, 1)
6           plt.plot(x, y)
7           plt.title("Plot")
8           plt.xlabel("X")

```



```
9      plt.ylabel("Y")
10     plt.grid()
11
12     plt.subplot(2, 3, 2)
13     plt.scatter(x, y)
14     plt.title("Scatter")
15     plt.xlabel("X")
16     plt.ylabel("Y")
17     plt.grid()
18
19     plt.subplot(2, 3, 3)
20     plt.bar(x, y)
21     plt.title("Bar")
22     plt.xlabel("X")
23     plt.ylabel("Y")
24     plt.grid()
25
26     plt.subplot(2, 3, 4)
27     plt.loglog(x, y)
28     plt.title("Loglog")
29     plt.xlabel("X")
30     plt.ylabel("Y")
31     plt.grid(which="both")
32
33     plt.subplot(2, 3, 5)
34     plt.semilogx(x, y)
35     plt.title("Semilogx")
36     plt.xlabel("X")
37     plt.ylabel("Y")
38     plt.grid(which="both")
39
40     plt.subplot(2, 3, 6)
41     plt.semilogy(x, y)
42     plt.title("Semilogy")
43     plt.xlabel("X")
44     plt.ylabel("Y")
45     plt.grid()
46
47     plt.tight_layout()
48     plt.show()
```

Listing 65: Creando múltiples subgráficas.

[Figura simulada con múltiples subgráficas]

Figura 17: Diferentes tipos de gráficas.

Podemos ver que al final de nuestra gráfica, usamos `plt.tight_layout` para asegurar que las subfiguras

A veces, quieres guardar las figuras en un formato específico, como pdf, jpeg, png, etc. Puedes hacerlo con la función `plt.savefig` (ver también Fig. 18).

```
1 In [14]: plt.figure(figsize=(8, 6))
2         plt.plot(x, y)
3         plt.xlabel("X")
4         plt.ylabel("Y")
5         plt.savefig("image.pdf")
```

Listing 66: Guardando una figura.

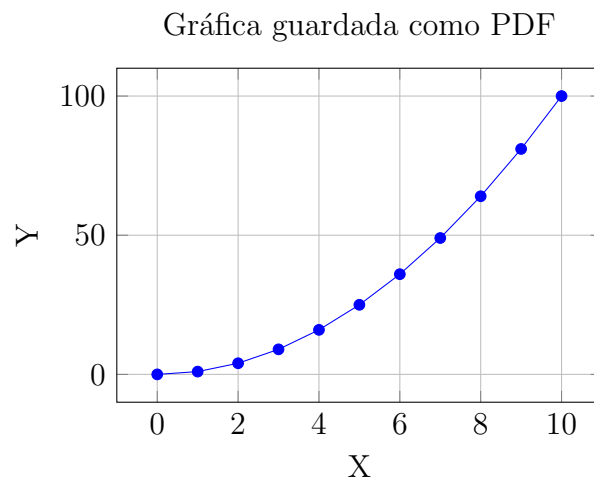


Figura 18: Guardando una figura.

Finalmente, hay otras funciones para trazar datos en 2D. La función `errorbar` traza datos de x versus y pero con barras de error para cada elemento. La función `polar` traza versus r en lugar de x versus y. La función `stem` traza líneas verticales en x con altura y. La función `hist` hace un histograma de un conjunto de datos; `boxplot` da un resumen estadístico de un conjunto de datos; y `pie` hace un

gráfico circular. El uso de estas funciones se deja para que lo explores por tu cuenta. Recuerda consultar los ejemplos en la galería de matplotlib¹⁰.

Trazado 3D

Para trazar figuras tridimensionales (3D), primero necesitamos importar el toolkit mplot3d, que añade las capacidades simples de trazado 3D a matplotlib.

```
1 In [1]: import numpy as np
2         from mpl_toolkits import mplot3d
3         import matplotlib.pyplot as plt
4         plt.style.use("seaborn-poster")
```

Listing 67: Importando módulos para gráficas 3D.

Una vez que hemos importado el toolkit mplot3d, podemos crear ejes 3D y añadir datos a los ejes. Primero creemos ejes 3D (ver también Fig. 19).

```
1 In [2]: fig = plt.figure(figsize=(10, 10))
2         ax = plt.axes(projection="3d")
3         plt.show()
```

Listing 68: Creando ejes 3D.

[Figura simulada de ejes 3D]

Figura 19: Ejes 3D.

El comando `ax = plt.axes(projection="3d")` creó un objeto de ejes 3D. Para añadir datos a él, usa la función `plot3D`. También es posible cambiar el título, establecer las etiquetas de los ejes x, y, z para la gráfica.

```
1 In [3]: %matplotlib notebook
2
3 In [4]: fig = plt.figure(figsize=(8, 8))
4         ax = plt.axes(projection="3d")
5         ax.grid()
6         t = np.arange(0, 10*np.pi, np.pi/50)
```

¹⁰<https://matplotlib.org/gallery/index.html>

```

7     x = np.sin(t)
8     y = np.cos(t)
9     ax.plot3D(x, y, t)
10    ax.set_title("3D Parametric Plot")
11    ax.set_xlabel("x", labelpad=20)
12    ax.set_ylabel("y", labelpad=20)
13    ax.set_zlabel("t", labelpad=20)
14    plt.show()

```

Listing 69: Gráfica paramétrica 3D.

Observa los ejemplos interactivos en el notebook. Intenta rotar la figura anterior para obtener una vista 3D de la gráfica. Puedes notar que también configuramos `labelpad=20` para las etiquetas de los 3 ejes, lo que hará que la etiqueta no se superponga con los textos de las marcas.

También podemos trazar un gráfico de dispersión 3D usando la función `scatter` (ver también Fig. 20).

```

1 In [5]: %matplotlib inline
2
3 In [6]: x = np.random.random(50)
4         y = np.random.random(50)
5         z = np.random.random(50)
6         fig = plt.figure(figsize=(10, 10))
7         ax = plt.axes(projection="3d")
8         ax.grid()
9         ax.scatter(x, y, z, c="r", s=50)
10        ax.set_title("3D Scatter Plot")
11        ax.set_xlabel("x", labelpad=20)
12        ax.set_ylabel("y", labelpad=20)
13        ax.set_zlabel("z", labelpad=20)
14        plt.show()

```

Listing 70: Gráfica de dispersión 3D.

[Figura simulada de dispersión 3D]

Figura 20: Gráfica de dispersión 3D.

Al trazar en tres dimensiones, a veces es deseable usar una gráfica de superficie en lugar de una gráfica de líneas. En el trazado de superficies 3D, deseamos hacer un gráfico de alguna relación $f(x, y)$.

En el trazado de superficies, todos los pares (x, y) deben estar dados. Esto no es sencillo de hacer usando vectores; por lo tanto, en el trazado de superficies, la primera estructura de datos que debes crear se llama malla. Dadas listas/arrays de valores de x e y , una malla es una lista de todas las combinaciones posibles de x e y . En Python, la malla se da como dos arrays X e Y , donde $X[i, j]$ e $Y[i, j]$ definen los posibles pares (x, y) . Luego se puede crear un tercer array, Z , tal que $Z[i, j] = f(X[i, j], Y[i, j])$. Se puede crear una malla usando la función `np.meshgrid` en Python. La función `meshgrid` tiene como entradas x e y , que son listas que contienen el conjunto de datos independiente. Las variables de salida X e Y se describieron anteriormente.

```

1 In [7]: x = [1, 2, 3, 4]
2         y = [3, 4, 5]
3         X, Y = np.meshgrid(x, y)
4         print(X)
5 [[1 2 3 4]
6  [1 2 3 4]
7  [1 2 3 4]]
8
9 In [8]: print(Y)
10 [[3 3 3 3]
11  [4 4 4 4]
12  [5 5 5 5]]

```

Listing 71: Creando una malla con `meshgrid`.

Además, podemos trazar superficies 3D en Python: la función para trazar superficies 3D es `plot_surface(X, Y, Z)`, donde X e Y son los arrays de salida de `meshgrid`, y $Z = f(X, Y)$ o $Z[i, j] = f(X[i, j], Y[i, j])$ (ver Fig. 21).

```

1 In [9]: fig = plt.figure(figsize=(12, 10))
2         ax = plt.axes(projection="3d")
3         x = np.arange(-5, 5.1, 0.2)
4         y = np.arange(-5, 5.1, 0.2)
5         X, Y = np.meshgrid(x, y)
6         Z = np.sin(X) * np.cos(Y)
7         surf = ax.plot_surface(X, Y, Z, cmap=plt.cm.cividis)
8         ax.set_xlabel("x", labelpad=20)
9         ax.set_ylabel("y", labelpad=20)
10        ax.set_zlabel("z", labelpad=20)
11        fig.colorbar(surf, shrink=0.5, aspect=8)

```

```
12 plt.show()
```

Listing 72: Gráfica de superficie 3D.

[Figura simulada de superficie 3D]

Figura 21: Gráfica de superficie 3D.

Observa que la gráfica de superficie muestra diferentes colores para diferentes elevaciones, amarillo para las más altas y azul para las más bajas (gris claro para las más altas y gris oscuro para las más bajas en la versión impresa); esto se debe a que usamos el mapa de colores `plt.cm.cividis` en la gráfica de superficie. Puedes cambiar a diferentes esquemas de color para la gráfica de superficie. Estos se dejan para que los completes como ejercicios adicionales. También trazamos una barra de color para mostrar los colores correspondientes a diferentes valores.

Podemos tener subgráficas de diferentes gráficas 3D también (ver Fig. 22). Para hacer eso, usa la función `add_subplot` del objeto figura que creamos para generar las subgráficas para casos 3D.

```
1 In [10]: fig = plt.figure(figsize=(12, 6))
2         ax = fig.add_subplot(1, 2, 1, projection="3d")
3         ax.plot_wireframe(X, Y, Z)
4         ax.set_title("Wireframe plot")
5
6         ax = fig.add_subplot(1, 2, 2, projection="3d")
7         ax.plot_surface(X, Y, Z)
8         ax.set_title("Surface plot")
9
10        plt.tight_layout()
11        plt.show()
```

Listing 73: Subgráficas 3D: wireframe y superficie.

[Figura simulada con wireframe y superficie]

Figura 22: Gráfica 3D de wireframe y superficie.

Dado que hay muchas más funciones relacionadas con el trazado en Python, esta no es de ninguna manera una lista exhaustiva; sin

embargo, debería ser suficiente para que empieces, para que puedas descubrir qué funciones de trazado en Python se adaptan mejor a ti y proporcionarte suficiente base para aprender a usarlas correctamente. Puedes encontrar más ejemplos de diferentes tipos de gráficas 3D en el sitio web del tutorial de mplot3d¹¹.

Trabajando con Mapas

A menudo, en ingeniería y ciencia, tenemos que interactuar con mapas o usar representaciones geográficas de datos. Hay muchos paquetes diferentes de Python que pueden dibujar mapas, como basemap¹², cartopy¹³, folium¹⁴, etc. El paquete folium te permite trazar mapas interactivos para páginas web. Pero la mayoría de las veces, solo necesitamos trazar un mapa estático para mostrar características espaciales, y basemap y cartopy harán el trabajo. Esta sección es una breve introducción a cómo dibujar mapas con datos usando cartopy. Primero, instala cartopy con conda install cartopy.

Los fundamentos de un mapa son simples: es una gráfica 2D con proyecciones específicas¹⁵. El eje x es la longitud, que va de -180 a 180 , y especifica la posición este-oeste de un punto en la superficie de la Tierra. El eje y es la latitud, que va de -90 a 90 , y describe la posición sur-norte de un punto. Si especificas un par de latitud y longitud, puedes determinar de manera única la ubicación del punto en la Tierra.

El paquete cartopy tiene una API muy agradable para interactuar con matplotlib y trazar un mapa; solo necesitamos decirle a matplotlib que use una proyección de mapa específica, y luego podemos añadir otras características del mapa a la gráfica (ver también Fig. 23).

```
1 In [1]: import cartopy.crs as ccrs
2         import matplotlib.pyplot as plt
3         %matplotlib inline
4
5 In [2]: plt.figure(figsize=(12, 8))
```

¹¹https://matplotlib.org/mpl_toolkits/mplot3d/tutorial.html

¹²<https://matplotlib.org/basemap/>

¹³<https://scitools.org.uk/cartopy/docs/latest/>

¹⁴<https://github.com/python-visualization/folium>

¹⁵https://en.wikipedia.org/wiki/Map_projection

```
6 ax = plt.axes(projection=ccrs.PlateCarree())
7 ax.coastlines()
8 ax.gridlines(draw_labels=True)
9 plt.show()
```

Listing 74: Trazando un mapa mundial simple con cartopy.

[Figura simulada de mapa mundial]

Figura 23: Mapa mundial simple.

El ejemplo anterior trazó el mapa con la proyección Plate Carree. Además, activamos las líneas de cuadrícula y dibujamos las etiquetas en los mapas. Te sugerimos que consultes otras proyecciones soportadas por cartopy¹⁶.

El fondo del mapa que dibujamos arriba está en blanco; sin embargo, podemos agregar fácilmente un buen fondo de mapa en cartopy usando `stock_img` (ver también Fig. 24).

```
1 In [3]: plt.figure(figsize=(12, 8))
2         ax = plt.axes(projection=ccrs.PlateCarree())
3         ax.coastlines()
4         ax.stock_img()
5         ax.gridlines(draw_labels=True)
6         plt.show()
```

Listing 75: Añadiendo una imagen de fondo al mapa.

[Figura simulada de mapa mundial con fondo]

Figura 24: Mapa mundial con fondo.

Podemos acercar el mapa a cualquier lugar de la Tierra usando la función `ax.set_extent`, que toma una lista con los primeros dos números que son los límites del eje x y los últimos dos números son los límites del eje y (ver también Fig. 25).

```
1 In [4]: plt.figure(figsize=(10, 5))
2         ax = plt.axes(projection=ccrs.PlateCarree())
```

¹⁶<https://scitools.org.uk/cartopy/docs/v0.16/crs/projections.html>


```

3     ax.coastlines()
4     ax.set_extent([-125, -75, 25, 50])
5     ax.gridlines(draw_labels=True)
6     plt.show()

```

Listing 76: Acercando el mapa a Estados Unidos.

[Figura simulada de mapa de EE. UU.]

Figura 25: Mapa simple de Estados Unidos.

¿Notaste que no hay características añadidas en el mapa, como límites de países, límites de estados, lagos/agua, etc.? En cartopy, estas características deben especificarse si queremos añadirlas a nuestro mapa (ver también Fig. 26).

```

1 In [5]: import cartopy.feature as cfeature
2
3 In [6]: plt.figure(figsize=(10, 5))
4         ax = plt.axes(projection=ccrs.PlateCarree())
5         ax.coastlines()
6         ax.set_extent([-125, -75, 25, 50])
7         ax.add_feature(cfeature.LAND)
8         ax.add_feature(cfeature.OCEAN)
9         ax.add_feature(cfeature.STATES, linestyle=":")
10        ax.add_feature(cfeature.BORDERS)
11        ax.add_feature(cfeature.LAKES, alpha=0.5)
12        ax.add_feature(cfeature.RIVERS)
13        plt.show()

```

Listing 77: Añadiendo características al mapa de EE. UU.

[Figura simulada de mapa de EE. UU. con características]

Figura 26: Mapa de EE. UU. con varias características.

Es posible acercarse aún más a un área más pequeña, pero necesitaríamos descargar y usar las costas y tierra de alta resolución para tener un mapa de aspecto decente (ver también Fig. 27).

```

1 In [7]: plt.figure(figsize=(10, 8))
2         ax = plt.axes(projection=ccrs.PlateCarree())

```

```

3     ax.coastlines(resolution="10m")
4     ax.set_extent([-122.8, -122, 37.3, 38.3])
5     # we can add high-resolution land and water
6     LAND = cfeature.NaturalEarthFeature("physical", "land
    ", "10m",
7                                         edgecolor="face",
8                                         facecolor=
9     cfeature.COLORS["land"],
10                                         linewidth=.1)
11     OCEAN = cfeature.NaturalEarthFeature("physical", "
    ocean", "10m",
12                                         edgecolor="face"
13                                         ,
14                                         facecolor=
15     cfeature.COLORS["water"],
16                                         linewidth=.1)
17     ax.add_feature(LAND, zorder=0)
18     ax.add_feature(OCEAN, zorder=0)
19     plt.show()

```

Listing 78: Mapa del Área de la Bahía con alta resolución.

[Figura simulada del mapa del Área de la Bahía]

Figura 27: Mapa del Área de la Bahía con mayor resolución.

En muchos casos, queremos trazar nuestros datos en el mapa y mostrar la ubicación espacial de diferentes entidades. Los datos se pueden agregar exactamente de la misma manera que con los ejes normales de matplotlib. Por defecto, el sistema de coordenadas de cualquier dato adicional es el mismo que el sistema de coordenadas de los ejes que definimos al principio de la gráfica. Primero intentemos agregar algunos datos al mapa anterior (ver también Fig. 28).

```

1 In [8]: plt.figure(figsize=(10, 8))
2         # plot the map related stuff
3         ax = plt.axes(projection=ccrs.PlateCarree())
4         ax.coastlines(resolution="10m")
5         ax.set_extent([-122.8, -122, 37.3, 38.3])
6         ax.add_feature(LAND, zorder=0)
7         ax.add_feature(OCEAN, zorder=0)
8
9         # plot the data related stuff

```

```

10     berkeley_lon, berkeley_lat = -122.2585, 37.8719
11     stanford_lon, stanford_lat = -122.1661, 37.4241
12
13     # plot the two universities as blue dots
14     ax.plot([berkeley_lon, stanford_lon],
15             [berkeley_lat, stanford_lat],
16             color="blue", linewidth=2, marker="o")
17
18     # add labels for the two universities
19     ax.text(berkeley_lon + 0.16, berkeley_lat - 0.02,
20            "UC Berkeley", horizontalalignment="right")
21     ax.text(stanford_lon + 0.02, stanford_lat - 0.02,
22            "Stanford", horizontalalignment="left")
23     plt.show()

```

Listing 79: Agregando datos al mapa.

[Figura simulada del mapa con datos de universidades]

Figura 28: Agregando más entidades en el mapa.

El paquete `cartopy` es extremadamente versátil. Los ejemplos oficiales están disponibles en la galería en línea¹⁷. Sugerimos explorar esto por tu cuenta para aumentar tus habilidades en la creación de mapas.

Animaciones y Películas

Una animación es una secuencia de fotogramas fijos, o gráficas, que se muestran en una sucesión lo suficientemente rápida como para crear la ilusión de movimiento continuo. Las animaciones y películas a menudo transmiten información mejor que las gráficas individuales. Puedes crear animaciones en Python llamando a una función de trazado dentro de un bucle (generalmente un bucle `for`). La principal herramienta para hacer animaciones en Python es la clase base `matplotlib.animation.Animation` que proporciona un marco sobre el cual se construye la funcionalidad de animación. Ve el ejemplo a continuación.

```

1 In [1]: import numpy as np
2         import matplotlib.pyplot as plt

```

¹⁷<https://scitools.org.uk/cartopy/docs/latest/gallery/index.html>

```

3         import matplotlib.animation as manimation
4
5 In [2]: n = 1000
6         x = np.linspace(0, 6*np.pi, n)
7         y = np.sin(x)
8
9         # Define the meta data for the movie
10        FFMpegWriter = manimation.writers["ffmpeg"]
11        metadata = dict(title="Movie Test", artist="
Matplotlib",
12                        comment="a red circle following a
blue sine wave")
13        writer = FFMpegWriter(fps=15, metadata=metadata)
14
15        # Initialize the movie
16        fig = plt.figure()
17        sine_line, = plt.plot(x, y, "b")
18        red_circle, = plt.plot([], [], "ro", markersize=10)
19        plt.xlabel("x")
20        plt.ylabel("sin(x)")
21
22        # Update the frames for the movie
23        with writer.saving(fig, "writer_test.mp4", 100):
24            for i in range(n):
25                x0 = x[i]
26                y0 = y[i]
27                red_circle.set_data(x0, y0)
28                writer.grab_frame()

```

Listing 80: Creando una animación simple.

Antes de hacer una película, se recomienda seguir los siguientes tres pasos:

- Definir los metadatos de la película.
- Decidir qué en el fondo no necesita cambiar.
- Decidir qué objetos necesitan cambiar en cada fotograma de la película.

Una vez que se determinan estos parámetros, es relativamente fácil hacer la película usando Python siguiendo los siguientes tres pasos:

- Definir los metadatos de la película.

- Inicializar la figura de fondo de la película.
- Actualizar los fotogramas de la película.

Usando el ejemplo anterior, podemos ver claramente cómo se relaciona el código con estos tres pasos.

1. Definiendo los metadatos de la película

```
1 FFMpegWriter = manimation.writers["ffmpeg"]
2 metadata = dict(title="Movie Test", artist="Matplotlib",
3               comment="a red circle following a blue sine
4               wave")
5 writer = FFMpegWriter(fps=15, metadata=metadata)
```

Este bloque de código le dice a Python que queremos crear una película usando un escritor de películas y necesitamos proporcionar el título, el artista y cualquier comentario necesario. Además, necesitamos decirle a Python la velocidad de los fotogramas en la película, es decir, `fps=15`, lo que significa que queremos mostrar 15 fotogramas consecutivos en 1 segundo (`fps` significa fotogramas por segundo).

2. Inicializando la figura de fondo de la película

```
1 fig = plt.figure()
2 sine_line, = plt.plot(x, y, "b")
3 red_circle, = plt.plot([], [], "ro", markersize=10)
4 plt.xlabel("x")
5 plt.ylabel("sin(x)")
```

Primero necesitamos inicializar la figura de fondo para la película. La razón por la que la llamamos figura de fondo es que los gráficos que trazamos aquí no cambiarán durante la película. En este ejemplo, la curva de la onda sinusoidal no cambiará. Al mismo tiempo, trazamos un punto rojo vacío (que no aparecerá en la figura). Sirve como un marcador de posición para las cosas que cambiarán más adelante en la película; esto es equivalente a decirle a Python que tendremos un punto rojo y que actualizaremos la ubicación del punto más tarde. En este caso, las etiquetas de los ejes `x` e `y` no cambiarán y, por lo tanto, se trazan aquí.

3. Actualizando los fotogramas de la película

```
1 with writer.saving(fig, "writer_test.mp4", 100):
2     for i in range(n):
3         x0 = x[i]
4         y0 = y[i]
```

```
5     red_circle.set_data(x0, y0)
6     writer.grab_frame()
```

Este bloque de código especifica el nombre del archivo de salida, el formato y la resolución de la figura (dpi { puntos por pulgada). En este caso, queremos que el archivo de salida tenga el nombre "writer_test" con el formato "mp4" y queremos un dpi de 100 para esta figura. A continuación, codificamos la parte central de la película: la actualización repetida de la figura, es decir, para crear el "movimiento". Usamos un bucle for para actualizar la figura, y en cada bucle, cambiamos la ubicación (ubicación x, y) del círculo rojo. La función `writer.grab_frame` capturará este cambio en cada fotograma y lo mostrará según el fps que configuramos.

Así es como se hace una película simple.

Numerosos ejemplos sobre cómo hacer una película están disponibles en el tutorial de películas de matplotlib¹⁸. Ejecuta algunos de los ejemplos para comprender mejor cómo hacer películas en Python.

Resumen y Problemas

Resumen

1. Visualizar datos es una herramienta esencial en aplicaciones de ingeniería y ciencia.
2. Python tiene muchos paquetes diferentes de herramientas de trazado que se pueden usar para visualizar datos.
3. Las gráficas y mapas bidimensionales y tridimensionales se utilizan generalmente en ingeniería y ciencia para comunicar resultados de investigación.
4. Los videos son una secuencia de imágenes estáticas que se muestran a una cierta velocidad.

Problemas y Soluciones

1. Problema 1: Trazar una cicloide.

¹⁸https://matplotlib.org/api/animation_api.html

Solución: Generamos los valores de ϕ y calculamos x e y usando las ecuaciones paramétricas.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 phi = np.linspace(0, 2*np.pi, 1000)
5 r = 3
6 x = r * (phi - np.sin(phi))
7 y = r * (1 - np.cos(phi))
8
9 plt.figure(figsize=(10, 6))
10 plt.plot(x, y)
11 plt.title("Cycloid (r=3)")
12 plt.xlabel("x")
13 plt.ylabel("y")
14 plt.grid(True)
15 plt.xlim(0, max(x))
16 plt.ylim(0, max(y)+1)
17 plt.show()
18

```

Listing 81: Solución Problema 1

2. Problema 2: Subgráficas de una función con diferentes escalas.

Solución: Definimos la función y creamos una figura 2x2 con los cuatro tipos de gráfica.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(0, 100, 1000)
5 y = np.sqrt(100*(1-0.01*x**2)**2 + 0.02*x**2) / ((1-x**2)
6         **2 + 0.1*x**2)
7
8
9 plt.figure(figsize=(12, 8))
10
11 plt.subplot(2, 2, 1)
12 plt.plot(x, y)
13 plt.title("Plot")
14 plt.xlabel("x")
15 plt.ylabel("y")
16 plt.grid()
17

```

```
16 plt.subplot(2, 2, 2)
17 plt.semilogx(x, y)
18 plt.title("Semilogx")
19 plt.xlabel("x (log scale)")
20 plt.ylabel("y")
21 plt.grid()
22
23 plt.subplot(2, 2, 3)
24 plt.semilogy(x, y)
25 plt.title("Semilogy")
26 plt.xlabel("x")
27 plt.ylabel("y (log scale)")
28 plt.grid()
29
30 plt.subplot(2, 2, 4)
31 plt.loglog(x, y)
32 plt.title("Loglog")
33 plt.xlabel("x (log scale)")
34 plt.ylabel("y (log scale)")
35 plt.grid(which="both")
36
37 plt.tight_layout()
38 plt.show()
39
```

Listing 82: Solución Problema 2

El gráfico loglog probablemente transmite más información, ya que comprime ambos ejes y permite visualizar mejor el comportamiento en un rango amplio de valores.

3. Problema 3: Trazar dos funciones en un mismo eje.

Solución: Calculamos los valores de ambas funciones y las graficamos en la misma figura.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(0, 5, 500)
5 y1 = 3 + np.exp(-x) * np.sin(6*x)
6 y2 = 4 + np.exp(-x) * np.cos(6*x)
7
8 plt.figure(figsize=(10, 6))
```



```

9 plt.plot(x, y1, label=r"$y_1(x)=3+e^{-x}\sin(6x)$")
10 plt.plot(x, y2, label=r"$y_2(x)=4+e^{-x}\cos(6x)$")
11 plt.title("Funciones con decaimiento exponencial y
    oscilaciones")
12 plt.xlabel("x")
13 plt.ylabel("y")
14 plt.legend()
15 plt.grid(True)
16 plt.show()
17

```

Listing 83: Solución Problema 3

4. Problema 4: Histograma y gráfico de barras de números aleatorios.

Solución: Generamos números aleatorios y usamos `plt.hist` y `plt.bar` para visualizarlos.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 data = np.random.randn(1000)
5
6 plt.figure(figsize=(12, 5))
7
8 plt.subplot(1, 2, 1)
9 counts, bins, _ = plt.hist(data, bins=10, edgecolor='
    black')
10 plt.title("Histograma (plt.hist)")
11 plt.xlabel("Valor")
12 plt.ylabel("Frecuencia")
13
14 plt.subplot(1, 2, 2)
15 plt.bar(bins[:-1], counts, width=np.diff(bins), edgecolor
    ='black', align='edge')
16 plt.title("Gráfico de barras (plt.bar)")
17 plt.xlabel("Valor")
18 plt.ylabel("Frecuencia")
19
20 plt.tight_layout()
21 plt.show()
22

```

Listing 84: Solución Problema 4

`np.random.randn` parece ser una buena aproximación de una distribución normal, ya que el histograma muestra la forma de campana característica.

5. Problema 5: Gráfico circular de calificaciones.

Solución: Usamos `plt.pie` con etiquetas y un título.

```
1 import matplotlib.pyplot as plt
2
3 grade_dist = [42, 85, 67, 20, 5]
4 labels = ["A's", "B's", "C's", "D's", "F's"]
5 colors = ["gold", "yellowgreen", "lightcoral", "lightskyblue", "red"]
6
7 plt.figure(figsize=(8, 8))
8 plt.pie(grade_dist, labels=labels, colors=colors, autopct="%1.1f%%", startangle=90)
9 plt.title("Distribución de Calificaciones")
10 plt.legend(labels, loc="best")
11 plt.axis("equal")
12 plt.show()
13
```

Listing 85: Solución Problema 5

6. Problema 6: Superficie y wireframe de una función 2D.

Solución: Creamos una malla, calculamos $z(x,y)$ y graficamos.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from mpl_toolkits.mplot3d import Axes3D
4
5 x = np.linspace(-4, 4, 100)
6 y = np.linspace(-3, 3, 100)
7 X, Y = np.meshgrid(x, y)
8 Z = X * Y * (X**2 - Y**2) / (X**2 + Y**2)
9
10 fig = plt.figure(figsize=(12, 5))
11
12 ax1 = fig.add_subplot(1, 2, 1, projection="3d")
13 surf = ax1.plot_surface(X, Y, Z, cmap="viridis")
14 ax1.set_title("Surface Plot")
15 ax1.set_xlabel("x")
```

```
16 ax1.set_ylabel("y")
17 ax1.set_zlabel("z")
18 fig.colorbar(surf, ax=ax1, shrink=0.5)
19
20 ax2 = fig.add_subplot(1, 2, 2, projection="3d")
21 wire = ax2.plot_wireframe(X, Y, Z)
22 ax2.set_title("Wireframe Plot")
23 ax2.set_xlabel("x")
24 ax2.set_ylabel("y")
25 ax2.set_zlabel("z")
26
27 plt.tight_layout()
28 plt.show()
29
```

Listing 86: Solución Problema 6

7. Problema 7: Función para dibujar un polígono regular.

Solución: Usamos coordenadas polares para calcular los vértices.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def my_polygon(n):
5     angles = np.linspace(0, 2*np.pi, n+1)
6     x = np.cos(angles)
7     y = np.sin(angles)
8
9     plt.figure(figsize=(6, 6))
10    plt.plot(x, y, "b-")
11    plt.fill(x, y, "lightblue", alpha=0.3)
12    plt.title(f"Poligono regular de {n} lados")
13    plt.xlabel("x")
14    plt.ylabel("y")
15    plt.axis("equal")
16    plt.grid(True)
17    plt.show()
18
19 my_polygon(5)
20
```

Listing 87: Solución Problema 7

8. Problema 8: Función para graficar cualquier función lambda.

Solución: La función toma una función y un array x y grafica f(x).

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def my_fun_plotter(f, x):
5     y = f(x)
6     plt.figure(figsize=(8, 5))
7     plt.plot(x, y)
8     plt.xlabel("x")
9     plt.ylabel("f(x)")
10    plt.title("Gráfica de la función proporcionada")
11    plt.grid(True)
12    plt.show()
13
14 my_fun_plotter(lambda x: np.sqrt(x) + np.exp(np.sin(x)),
15                np.linspace(0, 2*np.pi, 100))
```

Listing 88: Solución Problema 8

9. Problema 9: Graficar múltiples polinomios x^k .

Solución: Iteramos sobre k de 1 a n y graficamos cada polinomio.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def my_poly_plotter(n, x):
5     plt.figure(figsize=(10, 6))
6     for k in range(1, n+1):
7         plt.plot(x, x**k, label=f"$x^{k}$")
8     plt.title(f"Polinomios $x^k$ para k=1..{n}")
9     plt.xlabel("x")
10    plt.ylabel("y")
11    plt.legend()
12    plt.grid(True)
13    plt.show()
14
15 my_poly_plotter(5, np.linspace(-1, 1, 200))
```

16

Listing 89: Solución Problema 9

10. Problema 10: Triángulo de Sierpinski.

Solución: Generamos puntos recursivamente eligiendo un vértice aleatorio y moviéndose a la mitad de la distancia desde el punto actual a ese vértice.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def my_sierpinski(n):
5     vertices = np.array([[0, 0], [0.5, np.sqrt(3)/2], [1,
6         0]])
7     points = np.zeros((n, 2))
8     points[0] = [0, 0]
9     for i in range(1, n):
10         vertex = vertices[np.random.randint(0, 3)]
11         points[i] = (points[i-1] + vertex) / 2
12     plt.figure(figsize=(8, 8))
13     plt.plot(points[:, 0], points[:, 1], "b.", markersize
14         =0.5)
15     plt.title(f"Triangulo de Sierpinski ({n} puntos)")
16     plt.axis("equal")
17     plt.axis("off")
18     plt.show()
19 my_sierpinski(10000)

```

Listing 90: Solución Problema 10

11. Problema 11: Helecho de Barnsley (fractal estocástico).

Solución: Aplicamos las transformaciones lineales con las probabilidades dadas.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def my_fern(n):
5     points = np.zeros((n, 2))
6     for i in range(1, n):

```

```

7         r = np.random.random()
8         x, y = points[i-1]
9         if r < 0.01:
10             points[i] = [0, 0.16*y]
11         elif r < 0.08:
12             points[i] = [0.2*x - 0.26*y, 0.23*x + 0.22*y
+ 1.6]
13         elif r < 0.15:
14             points[i] = [-0.15*x + 0.28*y, 0.26*x + 0.24*
y + 0.44]
15         else:
16             points[i] = [0.85*x + 0.04*y, -0.04*x + 0.85*
y + 1.6]
17         plt.figure(figsize=(8, 10))
18         plt.plot(points[:, 0], points[:, 1], "g.", markersize
=0.5)
19         plt.title(f"Helecho de Barnsley ({n} puntos)")
20         plt.axis("equal")
21         plt.axis("off")
22         plt.show()
23
24 my_fern(10000)
25

```

Listing 91: Solución Problema 11

12. Problema 12: Gráfica paramétrica 3D.

Solución: Usamos plot3D para trazar la curva $(x(t), y(t), t)$.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from mpl_toolkits.mplot3d import Axes3D
4
5 def my_parametric_plotter(x, y, t):
6     fig = plt.figure(figsize=(10, 8))
7     ax = fig.add_subplot(111, projection="3d")
8     ax.plot3D(x(t), y(t), t)
9     ax.set_title("Gráfica Paramétrica 3D")
10    ax.set_xlabel("x(t)")
11    ax.set_ylabel("y(t)")
12    ax.set_zlabel("t")
13    plt.show()
14
15 f = lambda t: np.sin(t)

```

```

16 g = lambda t: t**2
17 t = np.linspace(0, 6*np.pi, 100)
18 my_parametric_plotter(f, g, t)
19

```

Listing 92: Solución Problema 12

13. Problema 13: Función para trazar superficie o contorno.

Solución: Dependiendo de option, usamos plot_surface o contour.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from mpl_toolkits.mplot3d import Axes3D
4
5 def my_surface_plotter(f, x, y, option):
6     X, Y = np.meshgrid(x, y)
7     Z = f(X, Y)
8     if option == "surface":
9         fig = plt.figure(figsize=(10, 8))
10        ax = fig.add_subplot(111, projection="3d")
11        surf = ax.plot_surface(X, Y, Z, cmap="viridis")
12        fig.colorbar(surf, shrink=0.5, aspect=5)
13        ax.set_title("Surface Plot")
14        ax.set_xlabel("x")
15        ax.set_ylabel("y")
16        ax.set_zlabel("z")
17    elif option == "contour":
18        plt.figure(figsize=(8, 6))
19        contour = plt.contour(X, Y, Z, 20, cmap="viridis"
20    )
21        plt.clabel(contour, inline=True, fontsize=8)
22        plt.title("Contour Plot")
23        plt.xlabel("x")
24        plt.ylabel("y")
25        plt.colorbar(contour)
26    else:
27        print("Option not recognized")
28    plt.show()
29
30 f = lambda x, y: np.cos(y) * np.sin(np.exp(x))
31 x = np.linspace(-1, 1, 20)
32 y = np.linspace(-2, 2, 40)
33 my_surface_plotter(f, x, y, "surface")

```

```
33 my_surface_plotter(f, x, y, "contour")
34
```

Listing 93: Solución Problema 13

14. Problema 14: Generar un error de dimensiones.

Solución: Intentamos graficar arrays de diferentes longitudes.

```
1 import matplotlib.pyplot as plt
2 x = [1, 2, 3]
3 y = [1, 2, 3, 4]
4 plt.plot(x, y) # Genera ValueError: x and y must have
   same first dimension
5
```

Listing 94: Solución Problema 14

15. Problema 15: Mapa nocturno de América del Norte con cartopy.

Solución: Usamos WMTS para obtener una imagen de la Tierra de noche.

```
1 import matplotlib.pyplot as plt
2 import cartopy.crs as ccrs
3 from cartopy.io.img_tiles import MapboxStyleTiles
4
5 # Usar una fuente de teselas de mapa nocturno (Earth at
   Night)
6 plt.figure(figsize=(12, 8))
7 ax = plt.axes(projection=ccrs.PlateCarree())
8 ax.set_extent([-128.75583, -68.01197, 19.50139,
   64.85694], crs=ccrs.PlateCarree())
9
10 # Nota: Esto requiere una clave de API de Mapbox o usar
   una fuente alternativa.
11 # Como alternativa, se puede usar ax.stock_img() o una
   imagen estática.
12 ax.stock_img() # Imagen de stock (día)
13 ax.coastlines(resolution="50m")
14 ax.gridlines(draw_labels=True)
15 plt.title("America del Norte - Mapa Nocturno")
16 plt.show()
17
```

Listing 95: Solución Problema 15

PARALELIZA TU PYTHON

Fundamentos de Computación Paralela

Ahora tenemos un conocimiento práctico de Python, y pronto comenzaremos a usarlo para analizar datos y realizar análisis numérico. Antes de profundizar, debemos cubrir la computación paralela en Python. Esto significa que podrás ejecutar tu código simultáneamente en múltiples núcleos de tu procesador CPU (o en múltiples procesadores CPU) o aumentar la velocidad aprovechando los ciclos de CPU desperdiciados mientras tu programa espera recursos externos (por ejemplo, descargar archivos, llamadas a API, etc.). La idea fundamental de la computación paralela se basa en hacer múltiples tareas al mismo tiempo para reducir el tiempo de ejecución de tu programa. La Fig. 2 ilustra la idea simple de hacer computación paralela versus computación en serie, que es lo que hemos discutido hasta ahora. Por ejemplo, si tienes 1 millón de archivos de datos y necesitas aplicar las mismas operaciones a cada uno de ellos, puedes hacerlo un archivo a la vez, o puedes hacerlo con múltiples archivos al mismo tiempo; o si estás descargando 1 millón de sitios web, puedes aprovechar la descarga de 10 a la vez para reducir el tiempo total de descarga. Por lo tanto, aprender los fundamentos de la computación paralela te ayudará a diseñar código más eficiente.

La mayoría de las computadoras modernas usan un diseño de **múltiples núcleos (multicore)**, lo que significa que en un solo componente de computación hay múltiples unidades de procesamiento independientes, los llamados **núcleos**, disponibles para realizar diferentes tareas. Por ejemplo, en la laptop del autor, hay un procesador CPU, con seis núcleos físicos (ver Fig. 1). Cada uno de ellos tiene dos núcleos lógicos, lo que eleva el número total de núcleos a 12 (ver la próxima sección cuando imprimamos el número total de CPU en tu máquina).

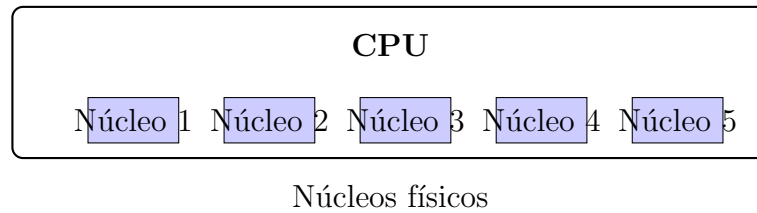


Figura 1: El hardware en la laptop del autor con procesador de múltiples núcleos.

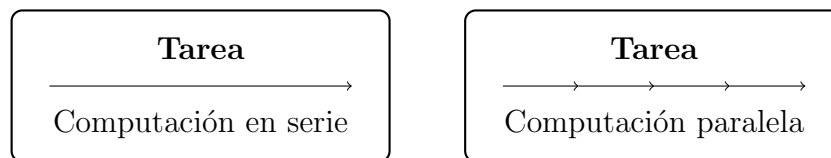


Figura 2: Computación paralela vs. computación en serie.

En Python, hay dos enfoques básicos cuando deseas realizar computación paralela: puedes emplear la biblioteca `multiprocessing` o `threading`. Primero, echemos un vistazo a las diferencias entre un **proceso** y un **hilo**.

Proceso e Hilo

Un **proceso** es una instancia de un programa (como el intérprete de Python, Jupyter notebook, etc.). Un proceso es creado por el sistema operativo para ejecutar un programa, y cada proceso tiene su propio bloque de memoria. Un **hilo** es un subproceso que reside dentro del proceso. Cada proceso puede tener múltiples hilos; estos hilos compartirán el mismo bloque de memoria dentro del proceso. Por lo tanto, debido a la memoria compartida, las variables u objetos para múltiples hilos en un proceso son todos compartidos. Si cambias una variable en un hilo, cambiará para todos los otros hilos. Este no es el caso para diferentes procesos. Si cambias una variable en un proceso, no cambiará esa variable en otros procesos. Tanto los procesos como los hilos tienen ventajas o desventajas, y se pueden usar en diferentes tareas para maximizar los beneficios de cada uno.

El Problema del GIL en Python

Python fue diseñado antes de que los procesadores de múltiples núcleos estuvieran disponibles en computadoras personales (esto te muestra lo antiguo que es el lenguaje). Contiene una limitación inherente llamada **GIL** (*Global Interpreter Lock*), por la cual solo un hilo nativo puede ejecutarse a la vez, es decir, evita que múltiples hilos se ejecuten simultáneamente. Aunque existen soluciones en Python para hacer multihilo, a continuación solo se cubrirá la biblioteca `multiprocessing`.

Desventajas de Usar Computación Paralela

Hay desventajas al usar computación paralela: el código se vuelve más complicado debido a los **overheads** (gastos generales) requeridos para lanzar y mantener nuevos procesos. Por lo tanto, si tu tarea es pequeña, usar computación paralela en realidad tomará más tiempo, ya que el sistema tarda en inicializar nuevos procesos y mantenerlos.

Multiprocessing

La biblioteca `multiprocessing`, la biblioteca estándar de Python para soportar su computación paralela usando procesos, tiene muchas características diferentes que son demasiado numerosas para discutir aquí. Te sugerimos consultar la documentación oficial¹. Aquí, presentamos solo los conceptos básicos para que comiences con la computación paralela. Comencemos importando la biblioteca e imprimiendo el número total de CPU en tu máquina que se pueden usar para computación paralela.

```
1 In [1]: import multiprocessing as mp
2
3 In [2]: print(f"Number of cpu: {mp.cpu_count()}")
4 Number of cpu: 12
```

Listing 1: Obteniendo el número de CPUs.

El siguiente ejemplo te muestra cómo usar múltiples núcleos en una máquina para reducir el tiempo de ejecución.

```
1 In [3]: import numpy as np
```

¹<https://docs.python.org/3/library/multiprocessing.html>

```
2     import time
3
4     def random_square(seed):
5         np.random.seed(seed)
6         random_num = np.random.randint(0, 10)
7         return random_num**2
8
9 In [4]: t0 = time.time()
10         results = []
11         for i in range(10000000):
12             results.append(random_square(i))
13         t1 = time.time()
14         print(f"Execution time {t1 - t0} s")
15 Execution time 38.20956087112427 s
```

Listing 2: Versión en serie para generar y elevar al cuadrado números aleatorios.

La forma más simple de hacer computación paralela usando `multiprocessing` es usar la clase `Pool`. Hay cuatro métodos comunes en esta clase que se usan a menudo: `apply`, `map`, `apply_async` y `map_async`. Consulta la documentación para conocer las diferencias entre ellos para tu propio aprendizaje. Solo usaremos la función `map` en paralelo con el ejemplo anterior. La función `map(func, iterable)` toma dos argumentos. Aplica la función `func` a cada elemento en el `iterable` y luego recopila los resultados.

```
1 In [5]: t0 = time.time()
2         n_cpu = mp.cpu_count()
3         pool = mp.Pool(processes=n_cpu)
4         results = [pool.map(random_square, range(10000000))]
5         t1 = time.time()
6         print(f"Execution time {t1 - t0} s")
7 Execution time 7.130078077316284 s
```

Listing 3: Versión paralela con `multiprocessing`.

Usar la versión paralela del código anterior redujo el tiempo de ejecución de aproximadamente 38 s a aproximadamente 7 s. Esta es una gran ganancia en velocidad, especialmente si estuviéramos ejecutando un código que demandara mucha computación.

La función `pool.apply` es similar, excepto que puede aceptar más argumentos. `pool.map` y `pool.apply` bloquearán el programa principal hasta que todos los procesos terminen, lo cual es bastante útil si queremos obtener resultados en un orden particular para algunas aplicaciones. En contraste, si no necesitamos los resultados en un orden particular, también podemos usar

`pool.apply_async` o `pool.map_async`, que enviarán todos los procesos a la vez y recuperarán los resultados tan pronto como terminen. Consulta en línea para aprender más.

Visualizando el Tiempo de Ejecución

Visualicemos los cambios en el tiempo de ejecución versus el número de puntos de datos usando tanto la versión en serie como la paralela. Verás que hasta cierto punto, es mejor usar la versión en serie.

```
1 In [6]: import matplotlib.pyplot as plt
2         plt.style.use("seaborn-poster")
3         %matplotlib inline
4
5         def serial(n):
6             t0 = time.time()
7             results = []
8             for i in range(n):
9                 results.append(random_square(i))
10            t1 = time.time()
11            exec_time = t1 - t0
12            return exec_time
13
14        def parallel(n):
15            t0 = time.time()
16            n_cpu = mp.cpu_count()
17            pool = mp.Pool(processes=n_cpu)
18            results = [pool.map(random_square, range(n))]
19            t1 = time.time()
20            exec_time = t1 - t0
21            return exec_time
22
23 In [7]: n_run = np.logspace(1, 7, num=7)
24         t_serial = [serial(int(n)) for n in n_run]
25         t_parallel = [parallel(int(n)) for n in n_run]
26
27 In [8]: plt.figure(figsize=(10, 6))
28         plt.plot(n_run, t_serial, "-o", label="serial")
29         plt.plot(n_run, t_parallel, "-o", label="parallel")
30         plt.loglog()
31         plt.legend()
32         plt.ylabel("Execution time (s)")
33         plt.xlabel("Number of random points")
```

34 `plt.show()`

Listing 4: Funciones para medir tiempo de ejecución en serie y paralelo.

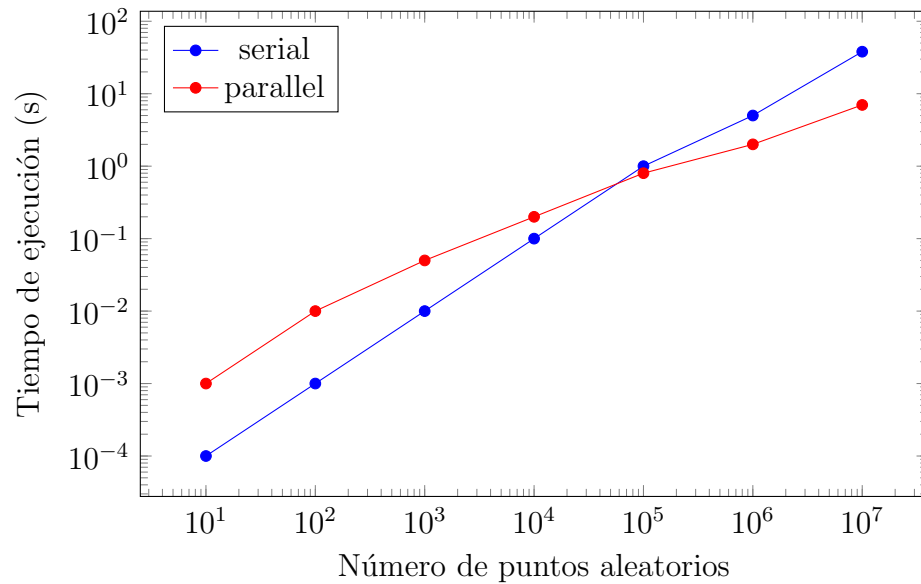


Figura 3: Tiempo de ejecución para versiones en serie y paralela.

Como se muestra en la figura, cuando el número de puntos de datos es pequeño (por debajo de 10000), el tiempo de ejecución para la versión en serie es más rápido debido al overhead de la versión paralela necesario para lanzar y mantener los nuevos procesos. Después de ese punto, la versión paralela sería una mejor opción. Por ejemplo, cuando tenemos 10^7 puntos de datos, la versión paralela usa menos de 10 s para terminar la tarea mientras que la versión en serie toma aproximadamente 50 s.

Usando Joblib

Python también tiene otros paquetes de terceros que pueden facilitar la computación paralela, especialmente para algunas tareas cotidianas, por ejemplo, el paquete `joblib`² proporciona una forma simple de realizar computación paralela (y también tiene muchos otros usos).

²<https://joblib.readthedocs.io/en/latest/index.html>

Primero, instala este paquete ejecutando `pip install joblib`; luego ejecuta el ejemplo anterior usando este nuevo paquete.

```
1 In [1]: from joblib import Parallel, delayed
2         import numpy as np
3
4         def random_square(seed):
5             np.random.seed(seed)
6             random_num = np.random.randint(0, 10)
7             return random_num**2
8
9 In [2]: results = Parallel(n_jobs=8)(delayed(random_square)(i
   ) for i in range(1000000))
```

Listing 5: Usando joblib para paralelizar.

Observa que la parte paralela del código se convierte en una sola línea al usar el paquete `joblib`, lo cual es muy conveniente. `Parallel` es una clase auxiliar que esencialmente proporciona una interfaz conveniente para el módulo `multiprocessing` que vimos anteriormente. La función `delayed` se usa para capturar los argumentos de la función objetivo, que en este caso es `random_square`. Ejecutamos el código anterior con ocho CPUs. Si quieres usar toda la potencia computacional en tu máquina, puedes usar todos los CPUs disponibles estableciendo `n_jobs=-1`. Si lo estableces en `-2`, se usarán todos los CPUs excepto uno. Además, activa el argumento `verbose` si quieres mostrar los mensajes de estado.

```
1 In [3]: results = Parallel(n_jobs=-1, verbose=1)(delayed(
   random_square)(i) for i in range(1000000))
2 [Parallel(n_jobs=-1)]: Using backend LokyBackend with 12
   concurrent workers.
3 [Parallel(n_jobs=-1)]: Done 60 tasks | elapsed: 0.1s
4 [Parallel(n_jobs=-1)]: Done 176056 tasks | elapsed: 3.0s
5 [Parallel(n_jobs=-1)]: Done 787056 tasks | elapsed: 12.4s
6 [Parallel(n_jobs=-1)]: Done 1000000 out of 1000000 | elapsed:
   15.5s finished
```

Listing 6: joblib con todos los CPUs y verbose.

Hay múltiples **backends** en `joblib`, lo que significa usar diferentes formas de hacer la computación paralela. Si configuras el backend como `multiprocessing`, es decir, bajo el capó, crea un grupo de multiprocesamiento que utiliza procesos de trabajo Python separados para ejecutar tareas concurrentemente en CPUs separadas.

```
1 In [4]: results = Parallel(n_jobs=-1, backend="
      multiprocessing", verbose=1)(delayed(random_square)(i) for
      i in range(1000000))
2 [Parallel(n_jobs=-1)]: Using backend MultiprocessingBackend
      with 12 concurrent workers.
3 [Parallel(n_jobs=-1)]: Done 220 tasks | elapsed: 0.0s
4 [Parallel(n_jobs=-1)]: Done 457032 tasks | elapsed: 1.9s
5 [Parallel(n_jobs=-1)]: Done 1000000 out of 1000000 | elapsed:
      3.8s finished
```

Listing 7: joblib con backend multiprocessing.

Resumen y Problemas

Resumen

1. La computación paralela puede reducir el tiempo de ejecución utilizando múltiples núcleos en tu computadora.
2. Hay una diferencia entre proceso e hilo; es más fácil usar un enfoque basado en procesos en Python para lograr paralelismo.
3. Usar el paquete `multiprocessing` es una forma fácil de resolver tus problemas cuando usas múltiples núcleos.
4. Usar `joblib` simplificará el código de computación paralela para muchas tareas comunes.

Problemas y Soluciones

1. **Problema 1:** ¿Qué es la computación paralela?

Solución: La computación paralela es un tipo de computación en la que muchos cálculos o la ejecución de procesos se llevan a cabo simultáneamente. Se basa en el principio de que los problemas grandes a menudo se pueden dividir en problemas más pequeños, que luego se resuelven concurrentemente para reducir el tiempo total de procesamiento.

2. **Problema 2:** Especifica la diferencia entre proceso e hilo.

Solución: Un **proceso** es una instancia de un programa en ejecución, con su propio espacio de memoria independiente. Un **hilo** es la unidad más pequeña de ejecución dentro de un proceso. Los hilos de un mismo proceso comparten el espacio de memoria de ese proceso (incluyendo código, datos y recursos), lo que facilita la comunicación entre ellos pero requiere sincronización para evitar conflictos. Los procesos están aislados entre sí, mientras que los hilos no.

3. **Problema 3:** Encuentra el número de procesadores en tu computadora usando el paquete `multiprocessing`.

Solución: Usamos `mp.cpu_count()`.

```
1 import multiprocessing as mp
2 print(f"Número de CPUs: {mp.cpu_count()}")
3
```

Listing 8: Solución Problema 3

4. **Problema 4:** Usa el paquete `multiprocessing` para paralelizar el código dado y registra el tiempo de ejecución.

Solución: Definimos la función y usamos `mp.Pool` con `map` o `starmap` (ya que la función toma dos argumentos). Usamos `zip` para combinar las dos listas en una sola de tuplas.

```
1 import multiprocessing as mp
2 import time
3
4 def plus_cube(x, y):
5     return (x + y) ** 3
6
7 # Version serial
8 def serial():
9     results = []
10    for x, y in zip(range(100), range(100)):
11        results.append(plus_cube(x, y))
12    return results
13
14 # Version paralela
15 def parallel():
16    with mp.Pool() as pool:
```

```

17         results = pool.starmap(plus_cube, zip(range(100),
18         range(100)))
19         return results
20
21 if __name__ == "__main__":
22     t0 = time.time()
23     serial()
24     t1 = time.time()
25     print(f"Serial: {t1-t0:.4f} s")
26
27     t0 = time.time()
28     parallel()
29     t1 = time.time()
30     print(f"Paralelo: {t1-t0:.4f} s")

```

Listing 9: Solución Problema 4

5. **Problema 5:** Proporciona un ejemplo para ilustrar la diferencia entre `pool.map` y `pool.map_async`.

Solución: `map` bloquea la ejecución del programa principal hasta que todos los resultados estén listos. `map_async` no bloquea; devuelve un objeto `AsyncResult` inmediatamente y puedes obtener los resultados más tarde con `get()`. Esto permite que el programa principal continúe mientras se ejecutan las tareas en segundo plano.

```

1 import multiprocessing as mp
2 import time
3
4 def tarea(x):
5     time.sleep(1)
6     return x*x
7
8 if __name__ == "__main__":
9     with mp.Pool(4) as pool:
10         # map: bloquea
11         t0 = time.time()
12         resultados_map = pool.map(tarea, range(5))
13         t1 = time.time()
14         print(f"map: {t1-t0:.2f} s")
15
16         # map_async: no bloquea

```

```
17     t0 = time.time()
18     resultados_async = pool.map_async(tarea, range(5)
19 )
19     # Podemos hacer otras cosas mientras se ejecuta
20     print("Haciendo otra cosa mientras...")
21     resultados_async.wait() # Esperar a que terminen
22     t1 = time.time()
23     print(f"map_async: {t1-t0:.2f} s")
24
```

Listing 10: Solución Problema 5

6. Problema 6: ¿Qué es el GIL de Python?

Solución: El GIL (*Global Interpreter Lock*) es un mecanismo en el intérprete estándar de Python (CPython) que permite que solo un hilo ejecute código Python a la vez, incluso en sistemas con múltiples núcleos. Esto simplifica la gestión de memoria pero limita el rendimiento de los programas multihilo para tareas vinculadas a la CPU. Para tareas vinculadas a E/S, el GIL se libera durante las operaciones de E/S, permitiendo cierta concurrencia. La biblioteca `multiprocessing` evita el GIL usando procesos separados en lugar de hilos.

7. Problema 7: Usa `joblib` para paralelizar el ejemplo del Problema 4 con backend `multiprocessing`.

Solución: Usamos `Parallel` y `delayed` con el backend especificado.

```
1 from joblib import Parallel, delayed
2 import time
3
4 def plus_cube(x, y):
5     return (x + y) ** 3
6
7 if __name__ == "__main__":
8     t0 = time.time()
9     results = Parallel(n_jobs=-1, backend="
10 multiprocessing")(
11     delayed(plus_cube)(x, y) for x, y in zip(range
12 (100), range(100))
13 )
```

```
12 t1 = time.time()  
13 print(f"Tiempo con joblib: {t1-t0:.4f} s")  
14
```

Listing 11: Solución Problema 7

ÁLGEBRA LINEAL Y SISTEMAS DE ECUACIONES LINEALES

Fundamentos de Álgebra Lineal

Primero, introducimos algunos conceptos básicos de álgebra lineal, que se utilizarán para describir y resolver ecuaciones lineales. Solo cubriremos lo más básico en este capítulo. Si deseas orientación adicional, te sugerimos estudiar un libro de texto de álgebra lineal.

Conjuntos

Ya hemos discutido la estructura de datos `set` en el Capítulo 2. Aquí lo veremos desde un punto de vista matemático. En matemáticas, un **conjunto** es una colección de objetos. Como se definió anteriormente, los conjuntos suelen denotarse con llaves $\{\}$. Por ejemplo, $S = \{\text{naranja, manzana, plátano}\}$ significa que S es el conjunto que contiene "naranja", "manzana" y "plátano".

El **conjunto vacío** es el conjunto que no contiene objetos y normalmente se denota con llaves vacías como $\{\}$ o por $\emptyset$. Dados dos conjuntos A y B , la **unión** de A y B se denota por $A \cup B$ y es igual al conjunto que contiene todos los elementos de A y B . La **intersección** de A y B se denota por $A \cap B$ y es igual al conjunto que contiene todos los elementos que pertenecen tanto a A como a B . En la notación de conjuntos, se usan dos puntos para significar "tal que". El uso de estos términos se aclarará en breve. El símbolo $\in$ se usa para denotar que un objeto está contenido en un conjunto. Por ejemplo, $a \in A$ significa ". a es un miembro de A ". " a está en A ". Una barra invertida, $\setminus$, en notación de conjuntos significa *conjunto menos*. Así que si $a \in A$ entonces

$A \setminus a$ significa “ A menos el elemento a ”.

Hay varios conjuntos estándar relacionados con los números, por ejemplo, los **números naturales**, **números enteros no negativos**, **números enteros**, **números racionales**, **números irracionales**, **números reales** y **números complejos**. Una descripción de cada conjunto y el símbolo utilizado para denotarlo se muestra en la siguiente tabla.

Cuadro 1: Conjuntos numéricos estándar.

Nombre del Conjunto	Símbolo	Descripción
Naturales	$\mathbb{N}$	$\mathbb{N} = \{1, 2, 3, 4, \dots\}$
Enteros no negativos	$\mathbb{W}$	$\mathbb{W} = \mathbb{N} \cup \{0\}$
Enteros	$\mathbb{Z}$	$\mathbb{Z} = \mathbb{W} \cup \{-1, -2, -3, \dots\}$
Racionales	$\mathbb{Q}$	$\mathbb{Q} = \left\{ \frac{p}{q} : p \in \mathbb{Z}, q \in \mathbb{Z} \setminus \{0\} \right\}$
Irracionales	$\mathbb{I}$	$\mathbb{I}$ es el conjunto de números reales no expresables como fra
Reales	$\mathbb{R}$	$\mathbb{R} = \mathbb{Q} \cup \mathbb{I}$
Complejos	$\mathbb{C}$	$\mathbb{C} = \{a + bi : a, b \in \mathbb{R}, i = \sqrt{-1}\}$

```

1 In []: # Sea S el conjunto de todos los pares (x, y) reales
        tales que x^2 + y^2 = 1.
2 # En notacion de conjuntos: S = {(x, y) : x, y      R, x^2 + y
        ^2 = 1}

```

Listing 12: Representando un conjunto en notación matemática.

Vectores

El conjunto $\mathbb{R}^n$ es el conjunto de todas las n -tuplas de números reales. En notación de conjuntos, esto es $\mathbb{R}^n = \{(x_1, x_2, x_3, \dots, x_n) : x_1, x_2, x_3, \dots, x_n \in \mathbb{R}\}$. Por ejemplo, el conjunto $\mathbb{R}^3$ representa el conjunto de tripletes reales, coordenadas (x, y, z) , en el espacio 3D.

Un **vector** en $\mathbb{R}^n$ es una n -tupla, o punto, en $\mathbb{R}^n$. Los vectores se pueden escribir horizontalmente (es decir, con los elementos del vector uno al lado del otro) en un **vector fila**, o verticalmente (es decir, con los elementos del vector uno encima del otro) en un **vector columna**. Si el contexto de un vector es ambiguo, generalmente significa que el vector es un vector columna. El i -ésimo elemento de un vector $\mathbf{v}$ se denota por v_i . La **transpuesta** de un vector columna es un vector fila de la misma longitud, y la transpuesta de un

vector fila es un vector columna. En matemáticas, la transpuesta se denota con un superíndice T, o $\mathbf{v}^T$. El **vector cero** es el vector en $\mathbb{R}^n$ que contiene todos ceros.

La **norma** de un vector es una medida de su longitud. Hay muchas formas de definir la longitud de un vector dependiendo de la métrica utilizada (es decir, la fórmula de distancia elegida). La más común se llama **norma L2**, que se calcula según la fórmula de distancia. La norma L2 de un vector $\mathbf{v}$ se denota por $\|\mathbf{v}\|_2$ y $\|\mathbf{v}\|_2 = \sqrt{\sum_i v_i^2}$. A esto también se le llama a veces **distancia euclidiana** y se refiere a la longitud "física" de un vector en el espacio 1, 2 o 3D. La **norma L1**, o "distancia de Manhattan", se calcula como $\|\mathbf{v}\|_1 = \sum_i |v_i|$, y lleva el nombre de la estructura de calles en forma de cuadrícula de la ciudad de Nueva York. En general, la **p-norma**, L_p , de un vector es $\|\mathbf{v}\|_p = \sqrt[p]{(\sum_i v_i^p)}$. La **norma L** es la p-norma, donde $p = \infty$. La norma L se escribe como $\|\mathbf{v}\|_\infty$ y es igual al valor absoluto máximo en $\mathbf{v}$.

```

1 In [1]: import numpy as np
2         vector_row = np.array([[1, -5, 3, 2, 4]])
3         vector_column = np.array([[1], [2], [3], [4]])
4         print(vector_row.shape)
5         print(vector_column.shape)
6 (1, 5)
7 (4, 1)

```

Listing 13: Creando vectores fila y columna con NumPy.

Nota

En Python, los vectores fila y columna pueden ser complicados. Como se muestra en el ejemplo anterior, para obtener vectores de 1 fila y 5 columnas o de 4 filas y 1 columna, tuvimos que usar una lista de una lista para especificarlo. Puedes definir `np.array([1,2,3,4])`, pero pronto notarás que no contiene información sobre fila o columna.

```

1 In [2]: from numpy.linalg import norm
2         new_vector = vector_row.T
3         print(new_vector)
4         norm_1 = norm(new_vector, 1)
5         norm_2 = norm(new_vector, 2)
6         norm_inf = norm(new_vector, np.inf)
7         print("L_1 is: %.1f" % norm_1)
8         print("L_2 is: %.1f" % norm_2)
9         print("L_inf is: %.1f" % norm_inf)

```

```

10 [[ 1]
11    [-5]
12    [ 3]
13    [ 2]
14    [ 4]]
15 L_1 is: 15.0
16 L_2 is: 7.4
17 L_inf is: 5.0

```

Listing 14: Transponiendo un vector y calculando normas.

La **suma de vectores** se define como la suma por pares de los elementos de los vectores sumados. Por ejemplo, si $\mathbf{v}$ y $\mathbf{w}$ son vectores en $\mathbb{R}^n$, entonces $\mathbf{u} = \mathbf{v} + \mathbf{w}$ se define como que tiene elementos $u_i = v_i + w_i$.

La **multiplicación de vectores** se puede definir de varias maneras dependiendo del contexto. La **multiplicación escalar** de un vector es el producto de un vector y un escalar (es decir, un número en $\mathbb{R}$). La multiplicación escalar se define como el producto de cada elemento del vector por el escalar. Más específicamente, si α es un escalar y $\mathbf{v}$ es un vector, entonces $\mathbf{u} = \alpha\mathbf{v}$ se define como que tiene elementos $u_i = \alpha v_i$. Observa que esto es exactamente cómo Python implementa la multiplicación escalar con un vector.

```

1 In []: # Mostrar que a(v + w) = av + aw
2 # Por suma de vectores, u = v + w es el vector con entradas
   ui = vi + wi.
3 # Por definicion de multiplicacion escalar de un vector, x =
   u es el vector con elementos xi = (vi + wi).
4 # Dado que , vi y wi son escalares, la multiplicacion
   distribuye y xi = vi + wi .
5 # Por lo tanto, a(v + w) = av + aw.

```

Listing 15: Demostración de la distributividad de la multiplicación escalar.

El **producto punto** de dos vectores es la suma de los productos de los respectivos elementos y se denota por $\cdot$, y $\mathbf{v} \cdot \mathbf{w}$ se lee "v punto w". Por lo tanto, para $\mathbf{v}, \mathbf{w} \in \mathbb{R}^n$, $d = \mathbf{v} \cdot \mathbf{w}$ se define como $d = \sum_{i=1}^n v_i w_i$. El **ángulo** entre dos vectores, θ , se define por la fórmula:

$$\mathbf{v} \cdot \mathbf{w} = \|\mathbf{v}\|_2 \|\mathbf{w}\|_2 \cos \theta.$$

El producto punto es una medida de cuán similarmente dirigidos están los dos vectores. Por ejemplo, los vectores $(1, 1)$ y $(2, 2)$ son paralelos. Si calculas el ángulo entre ellos usando el producto punto, encontrarás que $\theta = 0$. Si el ángulo entre los vectores es $\theta = \pi/2$, entonces se dice que los vectores son **perpendiculares** u **ortogonales**, y el producto punto es 0.


```

1 In [3]: from numpy import arccos, dot
2         v = np.array([[10, 9, 3]])
3         w = np.array([[2, 5, 12]])
4         theta = arccos(dot(v, w.T) / (norm(v) * norm(w)))
5         print(theta)
6 [[0.97992471]]

```

Listing 16: Calculando el ángulo entre dos vectores.

Finalmente, el **producto cruz** entre dos vectores, $\mathbf{v}$ y $\mathbf{w}$, se escribe como $\mathbf{v} \times \mathbf{w}$. Se define por $\mathbf{v} \times \mathbf{w} = \|\mathbf{v}\|_2 \|\mathbf{w}\|_2 \sin(\theta) \mathbf{n}$, donde θ es el ángulo entre $\mathbf{v}$ y $\mathbf{w}$ (que se puede calcular a partir del producto punto), y $\mathbf{n}$ es un vector perpendicular tanto a $\mathbf{v}$ como a $\mathbf{w}$ con longitud unitaria (es decir, su longitud es uno). La interpretación geométrica del producto cruz es un vector perpendicular tanto a $\mathbf{v}$ como a $\mathbf{w}$, con longitud igual al área encerrada por el paralelogramo creado por los dos vectores.

```

1 In [4]: v = np.array([[0, 2, 0]])
2         w = np.array([[3, 0, 0]])
3         print(np.cross(v, w))
4 [[ 0  0 -6]]

```

Listing 17: Calculando el producto cruz.

Suponiendo que S es un conjunto en el que se definen la suma y la multiplicación escalar, una **combinación lineal** de S se define como

$$\sum \alpha_i s_i,$$

donde α_i es cualquier número real, y s_i es el i -ésimo objeto en S . A veces los valores α_i se llaman los **coeficientes** de s_i .

Las combinaciones lineales se pueden usar para describir numerosas cosas. Por ejemplo, una factura de supermercado se puede escribir como $\sum c_i n_i$, donde c_i es el costo del artículo i y n_i es el número de artículos i comprados. Por lo tanto, el costo total es una combinación lineal de los artículos comprados.

Un conjunto se llama **linealmente independiente** si ningún objeto en el conjunto puede escribirse como una combinación lineal de los otros objetos en el conjunto. Para los propósitos de este libro, solo consideraremos la independencia lineal de un conjunto de vectores. Un conjunto de vectores que no es linealmente independiente es **linealmente dependiente**.

```

1 In [5]: v = np.array([[0, 3, 2]])
2         w = np.array([[4, 1, 1]])
3         u = np.array([[0, -2, 0]])

```

```

4      x = 3*v - 2*w + 4*u
5      print(x)
6      [[-8 -1  4]]

```

Listing 18: Escribiendo un vector como combinación lineal.

```

1 In []: # v = [1, 1, 0], w = [1, 0, 0], u = [0, 0, 1]
2 # Claramente, u es linealmente independiente de v y w porque
   solo u tiene un tercer elemento distinto de cero.
3 # Los vectores v y w tambien son linealmente independientes
   porque solo v tiene un segundo elemento distinto de cero.
4 # Por lo tanto, v, w y u son linealmente independientes.

```

Listing 19: Determinando la independencia lineal por inspección.

Matrices

Una **matriz** de $m \times n$ es una tabla rectangular de números que consta de m filas y n columnas. La norma de una matriz puede considerarse como un tipo particular de norma vectorial. Si tratamos los elementos $m \times n$ de M como los elementos de un vector de mn dimensiones, entonces la p -norma de este vector se puede escribir como

$$\|M\|_p = \sqrt[p]{\sum_i^m \sum_j^n |a_{ij}|^p}.$$

Es posible calcular la norma de la matriz usando la misma función **norm** en NumPy que para un vector.

La suma de matrices y la multiplicación escalar para matrices funcionan de la misma manera que para los vectores. Sin embargo, la **multiplicación de matrices** entre dos matrices, P y Q , se define cuando P es una matriz de $m \times p$ y Q es una matriz de $p \times n$. El resultado de $M = PQ$ es una matriz M que es $m \times n$. La dimensión p se llama la **dimensión de la matriz interna**, y las dimensiones de la matriz interna deben coincidir (es decir, el número de columnas en P y el número de filas en Q deben ser el mismo) para que la multiplicación de matrices esté definida. Las dimensiones m y n se llaman las **dimensiones de la matriz externa**. Formalmente, si P es $m \times p$ y Q es $p \times n$, entonces $M = PQ$ se define como

$$M_{ij} = \sum_{k=1}^p P_{ik} Q_{kj}.$$

El producto de dos matrices P y Q en Python se logra usando el método `dot` en NumPy.

La **transpuesta** de una matriz es una inversión de sus filas con sus columnas. La transpuesta se denota con un superíndice T , como M^T es la transpuesta de la matriz M . En Python, el método `T` para un array de NumPy se usa para obtener la transpuesta. Por ejemplo, si M es una matriz, entonces `M.T` es su transpuesta.

```

1 In [6]: P = np.array([[1, 7], [2, 3], [5, 0]])
2         Q = np.array([[2, 6, 3, 1], [1, 2, 3, 4]])
3         print(P)
4         print(Q)
5         print(np.dot(P, Q))
6         # np.dot(Q, P) # Esto produce un error
7
8 [[1 7]
9  [2 3]
10 [5 0]]
11
12 [[2 6 3 1]
13  [1 2 3 4]]
14
15 [[ 9 20 24 29]
16  [ 7 18 15 14]
17 [10 30 15  5]]

```

Listing 20: Multiplicación de matrices y error de dimensiones.

Una **matriz cuadrada** es una matriz de $n \times n$, es decir, tiene el mismo número de filas que de columnas. El **determinante** es una propiedad importante de las matrices cuadradas. Es un número especial que se puede calcular directamente a partir de una matriz cuadrada. El determinante se denota por **det**, tanto en matemáticas como en el paquete `linalg` de NumPy. Algunos ejemplos del uso del determinante se describirán más adelante.

En el caso de una matriz 2×2 , el determinante es

$$|M| = \begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc.$$

De manera similar, en el caso de una matriz 3×3 , el determinante es:

$$|M| = \begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix} = a \begin{vmatrix} e & f \\ h & i \end{vmatrix} - b \begin{vmatrix} d & f \\ g & i \end{vmatrix} + c \begin{vmatrix} d & e \\ g & h \end{vmatrix} = aei + bfg + cdh - ceg - bdi - afh.$$

Podemos usar un enfoque similar para calcular el determinante de una matriz de dimensiones superiores, pero es mucho más fácil calcularlo usando Python. Ve el ejemplo a continuación para calcular el determinante en Python.

La **matriz identidad** es una matriz cuadrada con 1s en la diagonal y 0s en el resto. La matriz identidad generalmente se denota por I y es análoga a la identidad del número real, 1. Es decir, multiplicar cualquier matriz por I (de tamaño compatible) producirá la misma matriz.

```

1 In [7]: from numpy.linalg import det
2         M = np.array([[0, 2, 1, 3],
3                       [3, 2, 8, 1],
4                       [1, 0, 0, 3],
5                       [0, 3, 2, 1]])
6         print("M:\n", M)
7         print("Determinant: %.1f" % det(M))
8         I = np.eye(4)
9         print("I:\n", I)
10        print("M*I:\n", np.dot(M, I))
11 M:
12 [[0 2 1 3]
13  [3 2 8 1]
14  [1 0 0 3]
15  [0 3 2 1]]
16 Determinant: -38.0
17 I:
18 [[1. 0. 0. 0.]
19  [0. 1. 0. 0.]
20  [0. 0. 1. 0.]
21  [0. 0. 0. 1.]]
22 M*I:
23 [[0. 2. 1. 3.]
24  [3. 2. 8. 1.]
25  [1. 0. 0. 3.]
26  [0. 3. 2. 1.]]

```

Listing 21: Calculando el determinante y la matriz identidad.

La **inversa** de una matriz cuadrada M es una matriz del mismo tamaño, N , tal que $M \cdot N = I$. La inversa de una matriz es análoga a la inversa de un número real. Por ejemplo, la inversa de 3 es $\frac{1}{3}$ porque $(3)(\frac{1}{3}) = 1$. Se dice que una matriz es **invertible** si tiene una inversa. La inversa de una matriz es única, es decir, para una matriz invertible, solo hay una inversa para esa matriz. Si M es una matriz cuadrada, su inversa se denota por M^{-1} en matemáticas, y se puede calcular en Python usando la función `inv` del paquete `linalg` de NumPy.

Para una matriz 2×2 , la solución analítica de la inversa de la matriz es

$$M^{-1} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{|M|} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

Calcular la inversa de la matriz para la solución analítica se vuelve complicado a medida que aumenta la dimensión de la matriz. Hay muchos otros métodos que pueden facilitar las cosas, como la eliminación gaussiana, el método de Newton, la descomposición en valores propios, etc. Introduciremos algunos de estos métodos después de aprender cómo resolver un sistema de ecuaciones lineales (ya que el proceso es esencialmente el mismo).

Recuerda que el cero no tiene inverso para la multiplicación en el contexto de los números reales. De manera similar, hay matrices que no tienen inversas. Estas matrices se llaman **singulares**. Las matrices que sí tienen una inversa se llaman **no singulares**.

Una forma de determinar si una matriz es singular es calculando su determinante. Si el determinante es 0, entonces la matriz es singular; si no, la matriz es no singular.

```

1 In [8]: from numpy.linalg import inv
2         print("Inv M:\n", inv(M))
3         P = np.array([[0, 1, 0],
4                       [0, 0, 0],
5                       [1, 0, 1]])
6         print("det(p):\n", det(P))
7 Inv M:
8 [[-1.57894737 -0.07894737  1.23684211  1.10526316]
9  [-0.63157895 -0.13157895  0.39473684  0.84210526]
10 [ 0.68421053  0.18421053 -0.55263158 -0.57894737]
11 [ 0.52631579  0.02631579 -0.07894737 -0.36842105]]
12 det(p):
13 0.0

```

Listing 22: Calculando la inversa y verificando singularidad.

Una matriz que está cerca de ser singular (es decir, el determinante está cerca de cero) se llama **mal condicionada**. Aunque las matrices mal condicionadas tienen inversas, son problemáticas numéricamente de la misma manera que dividir un número por un número muy, muy pequeño es problemático. Es decir, puede resultar en cálculos que producen desbordamiento, subdesbordamiento o números lo suficientemente pequeños como para generar errores de redondeo significativos. Si has olvidado alguno de estos conceptos,

revisa el Capítulo 9. El **número de condición** es una medida de qué tan mal condicionada está una matriz: se define como la norma de la matriz multiplicada por la norma de la inversa de la matriz, es decir, $\|M\|\|M^{-1}\|$. En Python, se puede calcular usando la función `cond` de `linalg` de NumPy. Cuanto mayor sea el número de condición, más cerca está la matriz de ser singular.

El **rango** de una matriz A de $m \times n$ es el número de columnas o filas linealmente independientes de A y se denota por $\text{rank}(A)$. Se puede demostrar que el número de filas linealmente independientes siempre es igual al número de columnas linealmente independientes para cualquier matriz. Una matriz tiene **rango completo** si $\text{rank}(A) = \min(m, n)$. La matriz A también es de rango completo si todas sus columnas son linealmente independientes. Una **matriz aumentada** es una matriz A concatenada con un vector $\mathbf{y}$ y se escribe $[A, \mathbf{y}]$. Esto se lee comúnmente como “ A aumentada con $\mathbf{y}$ ”. Puedes usar `np.concatenate` para concatenar. Si $\text{rank}([A, \mathbf{y}]) = \text{rank}(A) + 1$, entonces el vector $\mathbf{y}$ es información “nueva”. Es decir, no puede crearse como una combinación lineal de las columnas de A . El rango es una característica importante de las matrices debido a su relación con las soluciones de ecuaciones lineales, que se discute en la última sección de este capítulo.

```

1 In [9]: from numpy.linalg import cond, matrix_rank
2         A = np.array([[1, 1, 0],
3                       [0, 1, 0],
4                       [1, 0, 1]])
5         print("Condition number:\n", cond(A))
6         print("Rank:\n", matrix_rank(A))
7         y = np.array([[1], [2], [1]])
8         A_y = np.concatenate((A, y), axis=1)
9         print("Augmented matrix:\n", A_y)
10 Condition number:
11 4.048917339522305
12 Rank:
13 3
14 Augmented matrix:
15 [[1 1 0 1]
16  [0 1 0 2]
17  [1 0 1 1]]

```

Listing 23: Calculando el número de condición, el rango y la matriz aumentada.

Transformaciones Lineales

Para cualquier vector $\mathbf{x}$ e $\mathbf{y}$, y escalares a y b , decimos que una función F es una **transformación lineal** si

$$F(a\mathbf{x} + b\mathbf{y}) = aF(\mathbf{x}) + bF(\mathbf{y}).$$

Se puede demostrar que multiplicar una matriz A de $m \times n$ y un vector $\mathbf{v}$ de $n \times 1$ de tamaño compatible es una transformación lineal de $\mathbf{v}$. Por lo tanto, de ahora en adelante, una matriz será sinónimo de una función de transformación lineal.

```

1 In []: # Sea x un vector y sea F(x) definido por F(x) = Ax,
        # donde A es una matriz rectangular de tamaño apropiado.
2 # Demostrar que F(x) es una transformacion lineal.
3 # Prueba: Dado que F(x) = Ax, entonces para vectores v y w, y
        # escalares a y b,
4 # F(av + bw) = A(av + bw) (por definicion de F) = aAv + bAw (
        # por propiedad distributiva de la multiplicacion de
        # matrices)
5 # = aF(v) + bF(w) (por definicion de F).
```

Listing 24: Demostración de que $F(\mathbf{x}) = A\mathbf{x}$ es una transformación lineal.

Si A es una matriz de $m \times n$, entonces hay dos subespacios importantes asociados con A : uno es $\mathbb{R}^n$ y el otro es $\mathbb{R}^m$. El **dominio** de A es un subespacio de $\mathbb{R}^n$. Es el conjunto de todos los vectores que pueden ser multiplicados por A por la derecha. El **rango** de A es un subespacio de $\mathbb{R}^m$. Es el conjunto de todos los vectores $\mathbf{y}$ tales que $\mathbf{y} = A\mathbf{x}$. Puede denotarse como $R(A)$, donde $R(A) = \{\mathbf{y} \in \mathbb{R}^m : A\mathbf{x} = \mathbf{y}\}$. Otra forma de pensar sobre el rango de A es como el conjunto de todas las combinaciones lineales de las columnas en A , donde x_i es el coeficiente de la i -ésima columna en A . El **espacio nulo** $N(A) = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{0}_m\}$ es el subconjunto de vectores $\mathbf{x}$ en el dominio de A tal que $A\mathbf{x} = \mathbf{0}_m$, donde $\mathbf{0}_m$ es el vector cero (es decir, un vector en $\mathbb{R}^m$ con todos ceros).

```

1 In []: # Sea A = [[1, 0, 0], [0, 1, 0], [0, 0, 0]] y sea el
        # dominio de A R^3.
2 # Caracterizar el rango y el espacio nulo de A.
3 # Sea v = [x, y, z] un vector en R^3; entonces u = Av es el
        # vector u = [x, y, 0].
4 # Dado que x, y en R, el rango de A es el plano x-y en z =
        # 0.
```

```

5 # Sea v = [0, 0, z] para z ∈ ℝ. Entonces u = Av es el
   vector u = [0, 0, 0].
6 # Por lo tanto, el espacio nulo de A es el eje z (es decir,
   el conjunto de vectores [0, 0, z] z ∈ ℝ).
7 # Por lo tanto, esta transformacion lineal "aplana" cualquier
   componente z de un vector.

```

Listing 25: Caracterizando el rango y el espacio nulo.

Sistemas de Ecuaciones Lineales

Una **ecuación lineal** es una igualdad de la forma

$$\sum_{i=1}^n a_i x_i = y,$$

donde a_i son escalares, x_i son variables desconocidas en $\mathbb{R}$, e y es un escalar.

```

1 In []: # Determinar cual de las siguientes ecuaciones es
        lineal y cual no.
2 # 1. 3x1 + 4x2 - 3 = -5x3,
3 # 2. (-x1 + x2) / x3 = 2,
4 # 3. x1*x2 + x3 = 5.
5 # La ecuacion 1 se puede reorganizar como 3x1 + 4x2 + 5x3 =
        3, que claramente tiene la forma de una ecuacion lineal.
6 # La ecuacion 2 no es lineal, pero se puede reorganizar como
        -x1 + x2 - 2x3 = 0, que es lineal.
7 # La ecuacion 3 no es lineal.

```

Listing 26: Determinando si una ecuación es lineal.

Un **sistema de ecuaciones lineales** es un conjunto de ecuaciones lineales que comparten las mismas variables. Considera el siguiente sistema de ecuaciones lineales:

$$\begin{aligned}
 a_{1,1}x_1 + a_{1,2}x_2 + \cdots + a_{1,n-1}x_{n-1} + a_{1,n}x_n &= y_1, \\
 a_{2,1}x_1 + a_{2,2}x_2 + \cdots + a_{2,n-1}x_{n-1} + a_{2,n}x_n &= y_2, \\
 &\vdots \\
 a_{m-1,1}x_1 + a_{m-1,2}x_2 + \cdots + a_{m-1,n-1}x_{n-1} + a_{m-1,n}x_n &= y_{m-1}, \\
 a_{m,1}x_1 + a_{m,2}x_2 + \cdots + a_{m,n-1}x_{n-1} + a_{m,n}x_n &= y_m,
 \end{aligned}$$

donde $a_{i,j}$ y y_i son números reales. La forma matricial de un sistema de ecuaciones lineales es $A\mathbf{x} = \mathbf{y}$, donde A es una matriz de $m \times n$, $A(i, j) = a_{i,j}$,

$\mathbf{y}$ es un vector en $\mathbb{R}^m$, y $\mathbf{x}$ es un vector desconocido en $\mathbb{R}^n$. La forma matricial se muestra a continuación:

$$\begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \cdots & a_{m,n} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}.$$

Si realizas la multiplicación de matrices, verás que vuelves al sistema de ecuaciones original.

```

1 In []: # Poner el siguiente sistema de ecuaciones en forma
        matricial:
2 # 4x + 3y - 5z = 2,
3 # -2x - 4y + 5z = 5,
4 # 7x + 8y = -3,
5 # x + 2z = 1,
6 # 9 + y - 6z = 6.
7 # La matriz A es de 5x3 y el vector y es de 5x1.
8 # A = [[4, 3, -5], [-2, -4, 5], [7, 8, 0], [1, 0, 2], [9, 1,
        -6]]
9 # y = [[2], [5], [-3], [1], [6]]

```

Listing 27: Escribiendo un sistema de ecuaciones en forma matricial.

Soluciones a Sistemas de Ecuaciones Lineales

Considera un sistema de ecuaciones lineales en forma matricial, $A\mathbf{x} = \mathbf{y}$, donde A es una matriz de $m \times n$. Recuerda que esto significa que hay m ecuaciones y n incógnitas en nuestro sistema. Una **solución** a este sistema de ecuaciones lineales es un $\mathbf{x}$ en $\mathbb{R}^n$ que satisface la ecuación en forma matricial. Dependiendo de los valores que pueblan A e $\mathbf{y}$, hay tres posibles soluciones distintas para $\mathbf{x}$. O no hay solución para $\mathbf{x}$, o hay una solución única para $\mathbf{x}$, o hay infinitas soluciones para $\mathbf{x}$. Este hecho no se demuestra en este texto.

Caso 1: No hay solución para $\mathbf{x}$. Si $\text{rank}([A, \mathbf{y}]) = \text{rank}(A) + 1$, entonces $\mathbf{y}$ es linealmente independiente de las columnas de A . Por lo tanto, debido a que $\mathbf{y}$ no está en el rango de A , por definición no puede existir un $\mathbf{x}$ que satisfaga la ecuación. Por lo tanto, comparar $\text{rank}([A, \mathbf{y}])$ y $\text{rank}(A)$ proporciona una manera fácil de verificar si no hay soluciones para un sistema de ecuaciones lineales.

Caso 2: Hay una solución única para $\mathbf{x}$. Si $\text{rank}([A, \mathbf{y}]) = \text{rank}(A)$, entonces $\mathbf{y}$ se puede escribir como una combinación lineal de las columnas de A , y hay al menos una solución para la ecuación matricial. Para que haya solo una solución, también debe cumplirse que $\text{rank}(A) = n$. En otras palabras, el número de ecuaciones debe ser exactamente igual al número de incógnitas. Para ver por qué esta propiedad resulta en una solución única, considera las siguientes tres relaciones entre m y n : $m < n$, $m = n$ y $m > n$.

- Para el caso $m < n$, $\text{rank}(A) = n$ no puede ser cierto porque esto significa que tenemos una matriz ancha con menos ecuaciones que incógnitas. Por lo tanto, no necesitamos considerar este subcaso.
- Cuando $m = n$ y $\text{rank}(A) = n$, A es cuadrada e invertible. Dado que la inversa de una matriz es única, la ecuación matricial $A\mathbf{x} = \mathbf{y}$ se puede resolver multiplicando cada lado de la ecuación a la izquierda por A^{-1} . Esto da como resultado $A^{-1}A\mathbf{x} = A^{-1}\mathbf{y} \rightarrow I\mathbf{x} = A^{-1}\mathbf{y} \rightarrow \mathbf{x} = A^{-1}\mathbf{y}$, que proporciona la solución única a la ecuación.
- Si $m > n$, entonces hay más ecuaciones que incógnitas; sin embargo, si $\text{rank}(A) = n$, entonces es posible elegir n ecuaciones (es decir, filas de A) tal que si estas ecuaciones se satisfacen, entonces las restantes $m - n$ ecuaciones también se satisfarán. En otras palabras, son redundantes. Si las ecuaciones redundantes $m - n$ se eliminan del sistema, entonces el sistema resultante tiene una matriz A que es $n \times n$ e invertible. Estos hechos no se prueban en este texto. El nuevo sistema entonces tiene una solución única, que es válida para todo el sistema.

Caso 3: Hay infinitas soluciones para $\mathbf{x}$. Si $\text{rank}([A, \mathbf{y}]) = \text{rank}(A)$, entonces $\mathbf{y}$ está en el rango de A , y hay al menos una solución para la ecuación matricial; sin embargo, si $\text{rank}(A) < n$, entonces hay infinitas soluciones. Aunque no se muestra aquí, si $\text{rank}(A) < n$, entonces hay al menos un vector distinto de cero, $\mathbf{n}$, que está en el espacio nulo de A (en realidad, hay infinitos vectores en el espacio nulo bajo estas condiciones). Si $\mathbf{n}$ está en el espacio nulo de A , entonces $A\mathbf{n} = \mathbf{0}$ por definición. Ahora si $\mathbf{x}^*$ es una solución a la ecuación matricial $A\mathbf{x} = \mathbf{y}$, entonces, necesariamente, $A\mathbf{x}^* = \mathbf{y}$; sin embargo, $A\mathbf{x}^* + A\mathbf{n} = \mathbf{y}$ o $A(\mathbf{x}^* + \mathbf{n}) = \mathbf{y}$. Por lo tanto, $\mathbf{x}^* + \mathbf{n}$ también es una solución para $A\mathbf{x} = \mathbf{y}$. De hecho, dado que A es una transformación lineal, $\mathbf{x}^* + \alpha\mathbf{n}$ es una solución para cualquier número real α (deberías intentar demostrar

esto por tu cuenta). Dado que hay infinitos valores aceptables para α , hay infinitas soluciones para la ecuación matricial.

El resto del capítulo discutirá cómo resolver un sistema de ecuaciones que tiene una solución única. Primero, discutiremos algunos de los métodos comunes que probablemente encontrarás en tu trabajo y luego te mostraremos cómo resolver sistemas en Python.

Digamos que tenemos n ecuaciones con n variables, $A\mathbf{x} = \mathbf{y}$, de la siguiente manera:

$$\begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n,1} & a_{n,2} & \cdots & a_{n,n} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}.$$

Método de Eliminación de Gauss

El **método de eliminación de Gauss** es un procedimiento que convierte la matriz A en una forma **triangular superior** para resolver el sistema de ecuaciones. Usemos un sistema de cuatro ecuaciones y cuatro variables para ilustrar la idea. La eliminación de Gauss esencialmente convierte el sistema de ecuaciones en

$$\begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} & a_{1,4} \\ 0 & a'_{2,2} & a'_{2,3} & a'_{2,4} \\ 0 & 0 & a'_{3,3} & a'_{3,4} \\ 0 & 0 & 0 & a'_{4,4} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} y_1 \\ y'_2 \\ y'_3 \\ y'_4 \end{bmatrix}.$$

Al volver a la forma matricial usando este método, podemos ver que las ecuaciones se convierten en:

$$\begin{aligned} a_{1,1}x_1 + a_{1,2}x_2 + a_{1,3}x_3 + a_{1,4}x_4 &= y_1, \\ a'_{2,2}x_2 + a'_{2,3}x_3 + a'_{2,4}x_4 &= y'_2, \\ a'_{3,3}x_3 + a'_{3,4}x_4 &= y'_3, \\ a'_{4,4}x_4 &= y'_4. \end{aligned}$$

Ahora, x_4 se puede resolver fácilmente dividiendo ambos lados por $a'_{4,4}$, y luego sustituyendo el resultado en la tercera ecuación para resolver x_3 . Con x_3 y x_4 , podemos sustituirlos en la segunda ecuación para resolver x_2 , y ahora podemos resolver todas las x . Resolvimos el sistema de ecuaciones de abajo

hacia arriba; esto se llama **sustitución hacia atrás**. Observa que, si A fuera una matriz triangular inferior, resolveríamos el sistema de arriba hacia abajo mediante **sustitución hacia adelante**.

Para ilustrar cómo resolvemos las ecuaciones usando la eliminación de Gauss, usamos el siguiente ejemplo.

```

1 In []: # Usar eliminacion de Gauss para resolver las
        # siguientes ecuaciones:
2 # 4x1 + 3x2 - 5x3 = 2,
3 # -2x1 - 4x2 + 5x3 = 5,
4 # 8x1 + 8x2 = -3.
5
6 # Paso 1: Convertir estas ecuaciones a la forma matricial Ax
        # = y.
7 # A = [[4, 3, -5], [-2, -4, 5], [8, 8, 0]]
8 # y = [2, 5, -3]
9
10 # Paso 2: Obtener la matriz aumentada [A, y].
11 # [A, y] = [[4, 3, -5, 2], [-2, -4, 5, 5], [8, 8, 0, -3]]
12
13 # Paso 3: Comenzar a eliminar los elementos en la matriz
        # eligiendo una ecuacion pivote,
14 # que se utiliza para eliminar los elementos en otras
        # ecuaciones.
15 # Elegir la primera ecuacion como la ecuacion pivote y
        # convertir el primer elemento de la segunda fila a cero
16 # multiplicando la primera fila (ecuacion pivote) por -0.5 y
        # restandola de la segunda fila.
17 # El multiplicador es m2,1 = -0.5, lo que nos da:
18 # [[4, 3, -5, 2], [0, -2.5, 2.5, 6], [8, 8, 0, -3]]
19
20 # Paso 4: Convertir el primer elemento de la tercera fila a
        # cero.
21 # Como se hizo anteriormente, multiplicamos la primera fila
        # por 2 y la restamos de la tercera fila.
22 # El multiplicador es m3,1 = 2, lo que nos da:
23 # [[4, 3, -5, 2], [0, -2.5, 2.5, 6], [0, 2, 10, -7]]
24
25 # Paso 5: Convertir el segundo elemento de la tercera fila a
        # cero.
26 # Multiplicamos la segunda fila por -4/5 y la restamos de la
        # tercera fila.
27 # El multiplicador es m3,2 = -0.8, lo que nos da:
28 # [[4, 3, -5, 2], [0, -2.5, 2.5, 6], [0, 0, 12, -2.2]]
29

```

```

30 # Paso 6: Obtenemos x3 = -2.2 / 12 = -0.183.
31 # Paso 7: Insertamos x3 en la segunda ecuacion, obtenemos x2
    = -2.583.
32 # Paso 8: Insertamos x2 y x3 en la primera ecuacion,
    obtenemos x1 = 2.208.

```

Listing 28: Resolviendo un sistema 3x3 con eliminación de Gauss.

Nota

A veces, el primer elemento en la primera fila es cero. Cuando este es el caso, intercambia la primera fila con una fila que tenga un primer elemento distinto de cero, luego sigue el mismo procedimiento descrito anteriormente. Estamos usando el método de eliminación de Gauss con "pivoteo" aquí. Ten en cuenta que también hay un método de eliminación de Gauss "ingenuo" que asume que los valores pivote nunca serán cero.

Método de Eliminación de Gauss-Jordan

La **eliminación de Gauss-Jordan** resuelve sistemas de ecuaciones. Es un procedimiento para convertir A en una forma diagonal tal que la forma matricial de las ecuaciones se convierte en

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} y'_1 \\ y'_2 \\ y'_3 \\ y'_4 \end{bmatrix}.$$

Esencialmente, las ecuaciones se convierten en:

$$\begin{aligned} x_1 + 0 + 0 + 0 &= y'_1, \\ 0 + x_2 + 0 + 0 &= y'_2, \\ 0 + 0 + x_3 + 0 &= y'_3, \\ 0 + 0 + 0 + x_4 &= y'_4. \end{aligned}$$

Resolvamos otro sistema de ecuaciones utilizando el ejemplo anterior como modelo.

```

1 In []: # Usar eliminacion de Gauss-Jordan para resolver las
        siguientes ecuaciones:
2 # 4x1 + 3x2 - 5x3 = 2,

```

```

3 # -2x1 - 4x2 + 5x3 = 5,
4 # 8x1 + 8x2 = -3.
5
6 # Paso 1: Construir la matriz aumentada [A, y].
7 # [A, y] = [[4, 3, -5, 2], [-2, -4, 5, 5], [8, 8, 0, -3]]
8
9 # Paso 2: El primer elemento de la primera fila debe ser 1,
    asi que dividimos la fila por 4:
10 # [[1, 3/4, -5/4, 1/2], [-2, -4, 5, 5], [8, 8, 0, -3]]
11
12 # Paso 3: Para eliminar el primer elemento de la segunda y
    tercera filas, multiplicamos la primera fila por -2 y 8,
    respectivamente,
13 # y luego la restamos de la segunda y tercera filas para
    obtener:
14 # [[1, 3/4, -5/4, 1/2], [0, -5/2, 5/2, 6], [0, 2, 10, -7]]
15
16 # Paso 4: Para normalizar el segundo elemento de la segunda
    fila a 1, dividimos ambos lados de la ecuacion por -5/2:
17 # [[1, 3/4, -5/4, 1/2], [0, 1, -1, -12/5], [0, 2, 10, -7]]
18
19 # Paso 5: Para eliminar el segundo elemento de la tercera
    fila, multiplicamos la segunda fila por 2 y luego la
    restamos de la tercera fila:
20 # [[1, 3/4, -5/4, 1/2], [0, 1, -1, -12/5], [0, 0, 12, -11/5]]
21
22 # Paso 6: Normalizar la ultima fila dividiendola por 12:
23 # [[1, 3/4, -5/4, 1/2], [0, 1, -1, -12/5], [0, 0, 1, -11/60]]
24
25 # Paso 7: Para eliminar el tercer elemento de la segunda fila
    , multiplicamos la tercera fila por -1 y la restamos de la
    segunda fila:
26 # [[1, 3/4, -5/4, 1/2], [0, 1, 0, -155/60], [0, 0, 1,
    -11/60]]
27
28 # Paso 8: Para eliminar el tercer elemento de la primera fila
    , multiplicamos la tercera fila por -5/4 y luego la
    restamos de la primera fila:
29 # [[1, 3/4, 0, 13/48], [0, 1, 0, -2.583], [0, 0, 1, -0.183]]
30
31 # Paso 9: Para eliminar el segundo elemento de la primera
    fila, multiplicamos la segunda fila por 3/4 y luego la
    restamos de la primera fila:

```

32 # `[[1, 0, 0, 2.208], [0, 1, 0, -2.583], [0, 0, 1, -0.183]]`

Listing 29: Resolviendo un sistema 3x3 con eliminación de Gauss-Jordan.

Método de Descomposición LU

Los dos métodos mostrados anteriormente implican cambiar tanto A como $\mathbf{y}$ al mismo tiempo mientras se intenta convertir A a una forma triangular superior o diagonal. A veces podemos tener el mismo conjunto de ecuaciones pero diferentes conjuntos de $\mathbf{y}$ para diferentes experimentos. Esto es bastante común en el mundo real, donde tenemos diferentes observaciones experimentales $\mathbf{y}_a, \mathbf{y}_b, \mathbf{y}_c, \dots$. Por lo tanto, debemos resolver $A\mathbf{x} = \mathbf{y}_a, A\mathbf{x} = \mathbf{y}_b, \dots$ muchas veces, ya que cada vez que $[A, \mathbf{y}]$ cambiará. Obviamente, esto es realmente ineficiente. ¿Hay un método mediante el cual solo cambiemos el lado izquierdo de A pero no el lado derecho $\mathbf{y}$?

El **método de descomposición LU** cambia solo la matriz A , en lugar de $\mathbf{y}$. Es ideal para resolver el sistema con las mismas matrices de coeficientes A pero diferentes vectores constantes $\mathbf{y}$. El método de descomposición LU tiene como objetivo convertir A en el producto de dos matrices L y U , donde L es una matriz triangular inferior y U es una matriz triangular superior. Con esta descomposición, convertimos el sistema de ecuaciones a la siguiente forma:

$$LU\mathbf{x} = \mathbf{y} \rightarrow \begin{bmatrix} l_{1,1} & 0 & 0 & 0 \\ l_{2,1} & l_{2,2} & 0 & 0 \\ l_{3,1} & l_{3,2} & l_{3,3} & 0 \\ l_{4,1} & l_{4,2} & l_{4,3} & l_{4,4} \end{bmatrix} \begin{bmatrix} u_{1,1} & u_{1,2} & u_{1,3} & u_{1,4} \\ 0 & u_{2,2} & u_{2,3} & u_{2,4} \\ 0 & 0 & u_{3,3} & u_{3,4} \\ 0 & 0 & 0 & u_{4,4} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix}.$$

Si definimos $U\mathbf{x} = \mathbf{m}$, entonces las ecuaciones anteriores se convierten en:

$$\begin{bmatrix} l_{1,1} & 0 & 0 & 0 \\ l_{2,1} & l_{2,2} & 0 & 0 \\ l_{3,1} & l_{3,2} & l_{3,3} & 0 \\ l_{4,1} & l_{4,2} & l_{4,3} & l_{4,4} \end{bmatrix} \mathbf{m} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix}.$$

Podemos resolver fácilmente el problema anterior mediante sustitución hacia adelante (lo opuesto a la sustitución hacia atrás como vimos en el método de eliminación de Gauss). Después de resolver para $\mathbf{m}$, podemos

resolver fácilmente el resto del problema mediante sustitución hacia atrás:

$$\begin{bmatrix} u_{1,1} & u_{1,2} & u_{1,3} & u_{1,4} \\ 0 & u_{2,2} & u_{2,3} & u_{2,4} \\ 0 & 0 & u_{3,3} & u_{3,4} \\ 0 & 0 & 0 & u_{4,4} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \end{bmatrix}.$$

Pero, ¿cómo obtenemos las matrices L y U ? Hay diferentes formas de obtener la descomposición LU. A continuación, se muestra un ejemplo que utiliza el método de eliminación de Gauss. De lo anterior, sabemos que obtenemos una matriz triangular superior después de realizar la eliminación de Gauss. Al mismo tiempo, también obtenemos la matriz triangular inferior aunque nunca se escriba explícitamente. Durante el procedimiento de eliminación de Gauss, la matriz A en realidad se convierte en el producto de dos matrices como se muestra a continuación. La matriz triangular superior derecha es la que obtuvimos anteriormente. Los elementos diagonales en la matriz triangular inferior izquierda son 1, y los elementos debajo de los elementos diagonales son los multiplicadores que multiplican las ecuaciones pivote para eliminar los elementos durante el cálculo:

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 \\ m_{2,1} & 1 & 0 & 0 \\ m_{3,1} & m_{3,2} & 1 & 0 \\ m_{4,1} & m_{4,2} & m_{4,3} & 1 \end{bmatrix} \begin{bmatrix} u_{1,1} & u_{1,2} & u_{1,3} & u_{1,4} \\ 0 & u_{2,2} & u_{2,3} & u_{2,4} \\ 0 & 0 & u_{3,3} & u_{3,4} \\ 0 & 0 & 0 & u_{4,4} \end{bmatrix}.$$

Observa que obtenemos tanto L como U al mismo tiempo cuando realizamos la eliminación de Gauss. Usando el ejemplo anterior, donde U es el que usamos antes para resolver las ecuaciones, y L está compuesto por los multiplicadores (puedes verificar los ejemplos en la sección de eliminación de Gauss), obtenemos:

$$L = \begin{bmatrix} 1 & 0 & 0 \\ -0,5 & 1 & 0 \\ 2 & -0,8 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 4 & 3 & -5 \\ 0 & -2,5 & 2,5 \\ 0 & 0 & 60 \end{bmatrix}.$$

```

1 In [1]: import numpy as np
2         u = np.array([[4, 3, -5],
3                       [0, -2.5, 2.5],
4                       [0, 0, 12]])
5         l = np.array([[1, 0, 0],

```



```

6         [-0.5, 1, 0],
7         [2, -0.8, 1]])
8     print("LU=", np.dot(l, u))
9 LU= [[ 4.  3. -5.]
10      [-2. -4.  5.]
11      [ 8.  8.  0.]]

```

Listing 30: Verificando la descomposición LU.

Métodos Iterativos – Método de Gauss-Seidel

Los métodos introducidos anteriormente son todos métodos directos donde la solución se calcula utilizando un número finito de operaciones. Esta sección introduce una clase diferente de métodos, a saber, los **métodos iterativos**, o métodos indirectos. Comienzan con una suposición inicial de la solución y luego mejoran repetidamente la solución hasta que el cambio de la solución esté por debajo de un umbral elegido. Para usar este proceso iterativo, primero debemos escribir la forma explícita de un sistema de ecuaciones. Si tenemos un sistema de ecuaciones lineales

$$\begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \cdots & a_{m,n} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix},$$

podemos escribir su forma explícita como

$$x_i = \frac{1}{a_{i,i}} \left[y_i - \sum_{j=1, j \neq i}^n a_{i,j} x_j \right].$$

Este es el fundamento de los métodos iterativos; podemos asumir valores iniciales para todas las x , y usarlos como $x^{(0)}$. En la primera iteración, podemos sustituir $x^{(0)}$ en el lado derecho de la ecuación explícita anterior para obtener la solución de la primera iteración $x^{(1)}$. Sustituyendo $x^{(1)}$ en la ecuación, obtenemos $x^{(2)}$, y las iteraciones continúan hasta que la diferencia entre $x^{(k)}$ y $x^{(k-1)}$ sea menor que algún valor predefinido.

Los métodos iterativos requieren condiciones específicas para que la solución converja. Una condición suficiente, pero no necesaria, para la convergencia es que la matriz de coeficientes A sea **diagonalmente dominante**. Esto

significa que en cada fila de la matriz de coeficientes A , el valor absoluto del elemento diagonal es mayor que la suma de los valores absolutos de los elementos fuera de la diagonal. Si la matriz de coeficientes satisface esta condición, las iteraciones convergerán a la solución. Observa que el proceso de solución aún podría converger incluso cuando esta condición no se cumple.

Método de Gauss-Seidel

El **método de Gauss-Seidel** es un método iterativo específico que siempre utiliza el último valor estimado para cada elemento en $\mathbf{x}$. Por ejemplo, primero supongamos que se dan los valores iniciales para $x_2, x_3, \dots, x_n$ (excepto x_1) y calculamos x_1 . Usando el x_1 calculado y el resto de las x (excepto x_2), podemos calcular x_2 . Continuando de la misma manera y calculando todos los elementos de $\mathbf{x}$ concluirá la primera iteración. La parte única del método de Gauss-Seidel es el uso del último valor para calcular el siguiente valor en $\mathbf{x}$. Dichas iteraciones continúan hasta que el valor converge. Usemos este método para resolver el mismo problema que acabamos de resolver arriba.

```

1 In [2]: a = [[8, 3, -3], [-2, -8, 5], [3, 5, 10]]
2         # Find diagonal coefficients
3         diag = np.diag(np.abs(a))
4         # Find row sum without diagonal
5         off_diag = np.sum(np.abs(a), axis=1) - diag
6         if np.all(diag > off_diag):
7             print("matrix is diagonally dominant")
8         else:
9             print("NOT diagonally dominant")
10 matrix is diagonally dominant
11
12 In [3]: x1 = 0
13         x2 = 0
14         x3 = 0
15         epsilon = 0.01
16         converged = False
17         x_old = np.array([x1, x2, x3])
18         print("Iteration results")
19         print(" k,    x1,    x2,    x3 ")
20         for k in range(1, 50):
21             x1 = (14 - 3*x2 + 3*x3) / 8
22             x2 = (5 + 2*x1 - 5*x3) / (-8)
23             x3 = (-8 - 3*x1 - 5*x2) / (-5)
24             x = np.array([x1, x2, x3])
25             # check if it is smaller than threshold

```

```

26         dx = np.sqrt(np.dot(x - x_old, x - x_old))
27         print("%d, %.4f, %.4f, %.4f" % (k, x1, x2, x3))
28         if dx < epsilon:
29             converged = True
30             print("Converged!")
31             break
32         # assign the latest x value to the old value
33         x_old = x
34         if not converged:
35             print("Not converged, increase the # of
iterations")
36 Iteration results
37 k,      x1,      x2,      x3
38 1, 1.7500, -1.0625, 1.5875
39 2, 2.7437, -0.3188, 2.9275
40 3, 2.9673, 0.4629, 3.8433
41 4, 3.0177, 1.0226, 4.4332
42 5, 3.0290, 1.3885, 4.8059
43 6, 3.0315, 1.6208, 5.0397
44 7, 3.0321, 1.7668, 5.1861
45 8, 3.0322, 1.8582, 5.2776
46 9, 3.0322, 1.9154, 5.3348
47 10, 3.0323, 1.9512, 5.3705
48 11, 3.0323, 1.9735, 5.3929
49 12, 3.0323, 1.9875, 5.4068
50 13, 3.0323, 1.9962, 5.4156
51 14, 3.0323, 2.0017, 5.4210
52 Converged!

```

Listing 31: Resolviendo un sistema 3x3 con el método de Gauss-Seidel.

Resolviendo Sistemas de Ecuaciones Lineales en Python

Los ejemplos presentados anteriormente demostraron los diversos métodos que puedes usar para resolver sistemas de ecuaciones lineales. Esto también es muy fácil de hacer en Python, como se muestra a continuación. La forma más fácil de obtener una solución es a través de la función `solve` en NumPy.

```

1 In [1]: import numpy as np
2         A = np.array([[4, 3, -5],
3                       [-2, -4, 5],
4                       [8, 8, 0]])

```

```

5     y = np.array([2, 5, -3])
6     x = np.linalg.solve(A, y)
7     print(x)
8 [ 2.20833333 -2.58333333 -0.18333333]

```

Listing 32: Usando `numpy.linalg.solve` para resolver un sistema.

Obtenemos los mismos resultados que los de la sección anterior cuando calculamos a mano. Bajo el capó”, el solucionador en realidad está haciendo una descomposición LU para obtener los resultados. Si puedes verificar la ayuda de la función, verás que necesita que la matriz de entrada sea cuadrada y de rango completo, es decir, todas las filas (o, equivalentemente, las columnas) deben ser linealmente independientes.

```

1 In [2]: A_inv = np.linalg.inv(A)
2         x = np.dot(A_inv, y)
3         print(x)
4 [ 2.20833333 -2.58333333 -0.18333333]

```

Listing 33: Resolviendo usando la inversa de la matriz.

También podemos obtener las matrices L y U utilizadas en la descomposición LU usando el paquete SciPy.

```

1 In [3]: from scipy.linalg import lu
2         P, L, U = lu(A)
3         print("P:\n", P)
4         print("L:\n", L)
5         print("U:\n", U)
6         print("LU:\n", np.dot(L, U))
7 P:
8 [[0. 0. 1.]
9  [0. 1. 0.]
10 [1. 0. 0.]]
11 L:
12 [[ 1.  0.  0. ]
13  [-0.25 1.  0. ]
14 [ 0.5  0.5  1. ]]
15 U:
16 [[ 8.  8.  0. ]
17 [ 0. -2.  5. ]
18 [ 0.  0. -7.5]]
19 LU:
20 [[ 8.  8.  0.]
21 [-2. -4.  5.]

```

22 `[ 4. 3. -5.]`

Listing 34: Obteniendo la descomposición LU con SciPy.

¿Por qué obtenemos L y U diferentes de los calculados a mano en la última sección? También verás que hay una matriz de permutación P que es devuelta por la función `lu`. Esta matriz de permutación registra cómo cambia el orden de las ecuaciones para facilitar el cálculo. Por ejemplo, si el primer elemento de la primera fila es cero, no puede ser la ecuación pivote ya que no puedes convertir los primeros elementos de otras filas a cero; por lo tanto, necesitamos cambiar el orden de las ecuaciones para obtener una nueva ecuación pivote. Si multiplicas P y A , verás que esta matriz de permutación invierte el orden de las ecuaciones para este caso.

```
1 In [4]: print(np.dot(P, A))
2 [[ 8.  8.  0.]
3  [-2. -4.  5.]
4  [ 4.  3. -5.]
```

Listing 35: Efecto de la matriz de permutación.

Inversión de Matrices

Definimos la inversa de una matriz cuadrada M como una matriz del mismo tamaño, M^{-1} , tal que $M \cdot M^{-1} = M^{-1} \cdot M = I$. Si la dimensión de la matriz es alta, la solución analítica para la inversión de la matriz será complicada. Por lo tanto, necesitamos otras formas eficientes de obtener la inversa de la matriz.

Usemos una matriz 4×4 para ilustrar. Supongamos que tenemos

$$M = \begin{bmatrix} m_{1,1} & m_{1,2} & m_{1,3} & m_{1,4} \\ m_{2,1} & m_{2,2} & m_{2,3} & m_{2,4} \\ m_{3,1} & m_{3,2} & m_{3,3} & m_{3,4} \\ m_{4,1} & m_{4,2} & m_{4,3} & m_{4,4} \end{bmatrix},$$

y la inversa de M es

$$X = \begin{bmatrix} x_{1,1} & x_{1,2} & x_{1,3} & x_{1,4} \\ x_{2,1} & x_{2,2} & x_{2,3} & x_{2,4} \\ x_{3,1} & x_{3,2} & x_{3,3} & x_{3,4} \\ x_{4,1} & x_{4,2} & x_{4,3} & x_{4,4} \end{bmatrix}.$$

Por lo tanto, tendremos:

$$M \cdot X = \begin{bmatrix} m_{1,1} & m_{1,2} & m_{1,3} & m_{1,4} \\ m_{2,1} & m_{2,2} & m_{2,3} & m_{2,4} \\ m_{3,1} & m_{3,2} & m_{3,3} & m_{3,4} \\ m_{4,1} & m_{4,2} & m_{4,3} & m_{4,4} \end{bmatrix} \begin{bmatrix} x_{1,1} & x_{1,2} & x_{1,3} & x_{1,4} \\ x_{2,1} & x_{2,2} & x_{2,3} & x_{2,4} \\ x_{3,1} & x_{3,2} & x_{3,3} & x_{3,4} \\ x_{4,1} & x_{4,2} & x_{4,3} & x_{4,4} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Podemos reescribir la ecuación anterior como cuatro ecuaciones separadas, es decir,

$$M \begin{bmatrix} x_{1,1} \\ x_{2,1} \\ x_{3,1} \\ x_{4,1} \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad M \begin{bmatrix} x_{1,2} \\ x_{2,2} \\ x_{3,2} \\ x_{4,2} \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad M \begin{bmatrix} x_{1,3} \\ x_{2,3} \\ x_{3,3} \\ x_{4,3} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \quad M \begin{bmatrix} x_{1,4} \\ x_{2,4} \\ x_{3,4} \\ x_{4,4} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

Resolver los cuatro sistemas de ecuaciones anteriores proporcionará la inversa de la matriz. Podemos usar cualquier método introducido anteriormente para resolver estas ecuaciones (como eliminación de Gauss, Gauss-Jordan y descomposición LU). A continuación, se muestra un ejemplo de inversión de matrices usando el método de Gauss-Jordan.

Recuerda que en el método de Gauss-Jordan, convertimos nuestro problema de

$$M\mathbf{x} = \mathbf{y}$$

a

$$I\mathbf{x} = \mathbf{y}'$$

para obtener la solución. Esencialmente, estamos convirtiendo

$$\left[\begin{array}{cccc|c} m_{1,1} & m_{1,2} & m_{1,3} & m_{1,4} & y_1 \\ m_{2,1} & m_{2,2} & m_{2,3} & m_{2,4} & y_2 \\ m_{3,1} & m_{3,2} & m_{3,3} & m_{3,4} & y_3 \\ m_{4,1} & m_{4,2} & m_{4,3} & m_{4,4} & y_4 \end{array} \right]$$

a

$$\left[\begin{array}{cccc|c} 1 & 0 & 0 & 0 & y'_1 \\ 0 & 1 & 0 & 0 & y'_2 \\ 0 & 0 & 1 & 0 & y'_3 \\ 0 & 0 & 0 & 1 & y'_4 \end{array} \right].$$

En resumen, todo lo que necesitamos hacer es convertir

$$\left[\begin{array}{cccc|cccc} m_{1,1} & m_{1,2} & m_{1,3} & m_{1,4} & 1 & 0 & 0 & 0 \\ m_{2,1} & m_{2,2} & m_{2,3} & m_{2,4} & 0 & 1 & 0 & 0 \\ m_{3,1} & m_{3,2} & m_{3,3} & m_{3,4} & 0 & 0 & 1 & 0 \\ m_{4,1} & m_{4,2} & m_{4,3} & m_{4,4} & 0 & 0 & 0 & 1 \end{array} \right]$$

a

$$\left[\begin{array}{cccc|cccc} 1 & 0 & 0 & 0 & m'_{1,1} & m'_{1,2} & m'_{1,3} & m'_{1,4} \\ 0 & 1 & 0 & 0 & m'_{2,1} & m'_{2,2} & m'_{2,3} & m'_{2,4} \\ 0 & 0 & 1 & 0 & m'_{3,1} & m'_{3,2} & m'_{3,3} & m'_{3,4} \\ 0 & 0 & 0 & 1 & m'_{4,1} & m'_{4,2} & m'_{4,3} & m'_{4,4} \end{array} \right].$$

Entonces la matriz

$$\begin{bmatrix} m'_{1,1} & m'_{1,2} & m'_{1,3} & m'_{1,4} \\ m'_{2,1} & m'_{2,2} & m'_{2,3} & m'_{2,4} \\ m'_{3,1} & m'_{3,2} & m'_{3,3} & m'_{3,4} \\ m'_{4,1} & m'_{4,2} & m'_{4,3} & m'_{4,4} \end{bmatrix}$$

es la inversa de M que estamos buscando.

¿Puedes explicar cómo usar la descomposición LU para obtener la inversa de una matriz?

Resumen y Problemas

Resumen

1. El álgebra lineal es la base de muchos campos de la ingeniería.
2. Los vectores pueden considerarse como puntos en $\mathbb{R}^n$; se definen la suma y la multiplicación, aunque este no es necesariamente el caso para los escalares.
3. Un conjunto de vectores es linealmente independiente si ninguno de los vectores puede escribirse como una combinación lineal de los otros.
4. Las matrices son tablas de números. Tienen varias características importantes que incluyen el determinante, el rango y la inversa.
5. Un sistema de ecuaciones lineales puede representarse mediante la ecuación matricial $A\mathbf{x} = \mathbf{y}$.

6. El número de soluciones para un sistema de ecuaciones lineales está relacionado con $\text{rank}(A)$ y $\text{rank}([A, \mathbf{y}])$. Puede ser cero, una o infinitas.
7. Podemos resolver las ecuaciones usando eliminación de Gauss, eliminación de Gauss-Jordan, descomposición LU y el método de Gauss-Seidel.
8. Introdujimos métodos para encontrar la inversa de una matriz.

Problemas y Soluciones

1. **Problema 1:** Se recomienda encarecidamente leer un libro de álgebra lineal, lo que te dará un mayor dominio de los contenidos de este capítulo.

Solución: (Sin código)

2. **Problema 2:** Demostrar que la multiplicación de matrices distribuye sobre la suma de matrices.

Solución: La demostración formal se basa en la definición de multiplicación de matrices.

```

1 # Se puede verificar numericamente con Python para
  matrices compatibles.
2 import numpy as np
3 A = np.random.rand(3, 3)
4 B = np.random.rand(3, 3)
5 C = np.random.rand(3, 3)
6 print(np.allclose(np.dot(A, B+C), np.dot(A, B) + np.dot(A
  , C)))
7

```

Listing 36: Solución Problema 2 (esquemática)

3. **Problema 3:** Función `my_is_orthogonal` para verificar ortogonalidad cercana.

Solución: Calculamos el ángulo usando el producto punto y las normas, y verificamos si está cerca de $\pi/2$.

```

1 import numpy as np
2
3 def my_is_orthogonal(v1, v2, tol):
4     v1 = v1.flatten()

```



```

5     v2 = v2.flatten()
6     dot_product = np.dot(v1, v2)
7     norm_v1 = np.linalg.norm(v1)
8     norm_v2 = np.linalg.norm(v2)
9     if norm_v1 == 0 or norm_v2 == 0:
10         return 0
11     cos_theta = dot_product / (norm_v1 * norm_v2)
12     # Asegurar que cos_theta este en [-1, 1] para evitar
    errores numericos
13     cos_theta = np.clip(cos_theta, -1, 1)
14     theta = np.arccos(cos_theta)
15     return 1 if np.abs(np.pi/2 - theta) < tol else 0
16
17 # Test cases
18 a = np.array([[1], [0.001]])
19 b = np.array([[0.001], [1]])
20 print(my_is_orthogonal(a, b, 0.01)) # 1
21 print(my_is_orthogonal(a, b, 0.001)) # 0
22 a = np.array([[1], [0.001]])
23 b = np.array([[1], [1]])
24 print(my_is_orthogonal(a, b, 0.01)) # 0
25 a = np.array([[1], [1]])
26 b = np.array([[1], [1]])
27 print(my_is_orthogonal(a, b, 1e-10)) # 1
28

```

Listing 37: Solución Problema 3

4. **Problema 4:** Función `my_make_lin_ind` para extraer columnas linealmente independientes.

Solución: Iteramos sobre las columnas y usamos el rango para determinar si añadir la columna mantiene la independencia lineal.

```

1 import numpy as np
2
3 def my_make_lin_ind(A):
4     cols = []
5     current_matrix = None
6     for i in range(A.shape[1]):
7         if current_matrix is None:
8             current_matrix = A[:, i:i+1]
9             cols.append(i)
10        else:

```

```

11         trial = np.hstack((current_matrix, A[:, i:i
12         +1]))
13         if np.linalg.matrix_rank(trial) > np.linalg.
14         matrix_rank(current_matrix):
15             current_matrix = trial
16             cols.append(i)
17         return A[:, cols]
18
19 A = np.array([[12,24,0,11,-24,18,15],
20              [19,38,0,10,-31,25,9],
21              [1,2,0,21,-5,3,20],
22              [6,12,0,13,-10,8,5],
23              [22,44,0,2,-12,17,23]])
24 B = my_make_lin_ind(A)
25 print(B)

```

Listing 38: Solución Problema 4

5. **Problema 5:** Función recursiva para calcular el determinante usando la regla de Cramer.

Solución: Implementamos la regla de Cramer recursivamente. Esta implementación es extremadamente ineficiente para matrices grandes.

```

1 def my_rec_det(M):
2     n = M.shape[0]
3     if n == 1:
4         return M[0, 0]
5     if n == 2:
6         return M[0, 0] * M[1, 1] - M[0, 1] * M[1, 0]
7     det = 0
8     for i in range(n):
9         # Crear el menor eliminando la fila 0 y la
10        # columna i
11        minor = np.delete(np.delete(M, 0, axis=0), i,
12        axis=1)
13        det += ((-1) ** i) * M[0, i] * my_rec_det(minor)
14    return det
15
16 # Ejemplo con matriz 3x3 pequena
17 M = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
18 print(my_rec_det(M))
19 print(np.linalg.det(M))

```

18

Listing 39: Solución Problema 5

6. **Problema 6:** Complejidad de `my_rec_det`.

Solución: La complejidad es $O(n!)$, lo que la hace inútil para matrices de tamaño mediano o grande. No es una forma efectiva de determinar si una matriz es singular.

7. **Problema 7:** Matriz de diferenciación polinomial.

Solución: La derivada de $p(x) = \sum_{k=0}^{L-1} p_k x^k$ es $p'(x) = \sum_{k=1}^{L-1} k p_k x^{k-1}$. La matriz D de tamaño $(L-1) \times L$ tiene $D_{i,i+1} = i+1$.

```

1 import numpy as np
2
3 def my_poly_der_mat(p):
4     L = len(p)
5     D = np.zeros((L-1, L))
6     for i in range(L-1):
7         D[i, i+1] = i+1
8     return D
9
10 p = np.array([1, 0, 2]) # 1x^2 + 0x + 2
11 D = my_poly_der_mat(p)
12 print(D)
13 print(np.dot(D, p)) # Deberia ser [0, 2] (coefs de 2x)
14

```

Listing 40: Solución Problema 7

8. **Problema 8:** Resolver sistema con eliminación de Gauss.

Solución: Aplicamos el procedimiento paso a paso.

```

1 # Las ecuaciones son:
2 # 3x1 - x2 + 4x3 = 2
3 # 17x1 + 2x2 + x3 = 14
4 # x1 + 12x2 - 7z = 54 (asumo que z es x3)
5 A = np.array([[3, -1, 4], [17, 2, 1], [1, 12, -7]], dtype
6               =float)
7 b = np.array([2, 14, 54], dtype=float)

```

```

7
8 # Eliminacion de Gauss
9 n = len(b)
10 for i in range(n):
11     # Pivote
12     for j in range(i+1, n):
13         factor = A[j, i] / A[i, i]
14         A[j, i:] -= factor * A[i, i:]
15         b[j] -= factor * b[i]
16
17 # Sustitucion hacia atras
18 x = np.zeros(n)
19 for i in range(n-1, -1, -1):
20     x[i] = (b[i] - np.dot(A[i, i+1:], x[i+1:])) / A[i, i]
21 print("Solucion:", x)
22

```

Listing 41: Solución Problema 8

9. Problema 9: Resolver con eliminación de Gauss-Jordan.

Solución: Extendemos la eliminación hasta obtener la matriz identidad.

```

1 A = np.array([[3, -1, 4], [17, 2, 1], [1, 12, -7]], dtype
  =float)
2 b = np.array([2, 14, 54], dtype=float)
3 augmented = np.hstack((A, b.reshape(-1, 1)))
4
5 n = A.shape[0]
6 for i in range(n):
7     augmented[i] = augmented[i] / augmented[i, i]
8     for j in range(n):
9         if i != j:
10             factor = augmented[j, i]
11             augmented[j] -= factor * augmented[i]
12
13 x = augmented[:, -1]
14 print("Solucion:", x)
15

```

Listing 42: Solución Problema 9

10. Problema 10: Descomposición LU del sistema.

Solución: A partir de los multiplicadores de la eliminación de Gauss.

```

1 # Despues de la eliminacion de Gauss del Problema 8,
  obtenemos:
2 # U = [[3, -1, 4], [0, 7.67, -21.67], [0, 0, -20.5]] (
  aproximado)
3 # L = [[1, 0, 0], [5.67, 1, 0], [0.33, 1.57, 1]]
4

```

Listing 43: Solución Problema 10

11. **Problema 11:** Demostrar distributividad del producto punto.

Solución: Se basa en la definición y la distributividad de la multiplicación escalar y la suma.

12. **Problema 12:** Calculadora de flujo en una red eléctrica.

Solución: Las ecuaciones de nodo proporcionan un sistema lineal. Resolvemos para los flujos.

```

1 import numpy as np
2
3 def my_flow_calculator(S, d):
4     # Sistema basado en la figura:
5     # Nodo N1: f1 + f2 = d1 + f3
6     # Nodo N2: f3 + f4 = d2
7     # Nodo N3: f5 = d3 + f6
8     # Nodo N4: f6 + f7 = d4
9     # Nodo N5: f2 + f4 + f7 = d5
10    # Suministro: S1 = f1 + f5, S2 = f2 + f3 + f4 + f7?
11    Revisar
12    # Asumimos un sistema sobredeterminado, buscamos una
13    # solucion que cumpla todas.
14    # Simplificacion: Asumimos que los flujos f1...f7
15    # satisfacen las ecuaciones de nodo.
16    # Construimos la matriz A y el vector b a partir de
17    # las ecuaciones.
18    # (La implementacion completa requiere derivar las
19    # ecuaciones correctamente de la figura)
20    pass

```

Listing 44: Solución Problema 12

VALORES PROPIOS Y VECTORES PROPIOS

Planteamiento del Problema de Valores Propios y Vectores Propios

Valores Propios y Vectores Propios

Aprendimos del capítulo anterior que la matriz A aplicada a un vector columna $\mathbf{x}$, es decir, $A\mathbf{x}$, es una transformación lineal de $\mathbf{x}$. Hay una transformación especial de la siguiente forma:

$$A\mathbf{x} = \lambda\mathbf{x},$$

donde A es una matriz de $n \times n$, $\mathbf{x}$ es un vector columna de $n \times 1$ ($\mathbf{x} \neq \mathbf{0}$), y λ es un escalar. Cualquier λ que satisfaga la ecuación anterior se conoce como un **valor propio** de la matriz A , mientras que el vector asociado $\mathbf{x}$ se llama un **vector propio** correspondiente a λ .

La Motivación Detrás de los Valores Propios y Vectores Propios

La motivación detrás de los valores propios y vectores propios es que si entendemos las características de la transformación lineal, ayudará a simplificar las soluciones a nuestro problema. Supongamos que podemos multiplicar un vector A por otro vector $\mathbf{x}$, es decir, $A\mathbf{x}$. Esto esencialmente transforma el vector $\mathbf{x}$ en otro vector, mediante lo cual la transformación representa una escala de la longitud del vector y/o la rotación del vector. La ecuación anterior señala que para algunos vectores, el efecto de la transformación $A\mathbf{x}$ es solo de

escalamiento (estiramiento, compresión e inversión). Los vectores propios son los vectores que tienen esta propiedad y los valores propios λ son los factores de escala. Veamos el siguiente ejemplo.

```

1 In [1]: import numpy as np
2         import matplotlib.pyplot as plt
3         plt.style.use("seaborn-poster")
4         %matplotlib inline
5
6         def plot_vect(x, b, xlim, ylim):
7             """
8             function to plot two vectors,
9             x - the original vector
10            b - the transformed vector
11            xlim - the limit for x
12            ylim - the limit for y
13            """
14            plt.figure(figsize=(10, 6))
15            plt.quiver(0, 0, x[0], x[1], \
16                      color="k", angles="xy", \
17                      scale_units="xy", scale=1, \
18                      label="Original vector")
19            plt.quiver(0, 0, b[0], b[1], \
20                      color="g", angles="xy", \
21                      scale_units="xy", scale=1, \
22                      label="Transformed vector")
23            plt.xlim(xlim)
24            plt.ylim(ylim)
25            plt.xlabel("X")
26            plt.ylabel("Y")
27            plt.legend()
28            plt.show()
29
30 In [2]: A = np.array([[2, 0], [0, 1]])
31         x = np.array([[1], [1]])
32         b = np.dot(A, x)
33         plot_vect(x, b, (0, 3), (0, 2))

```

Listing 45: Trazando un vector y su transformación.

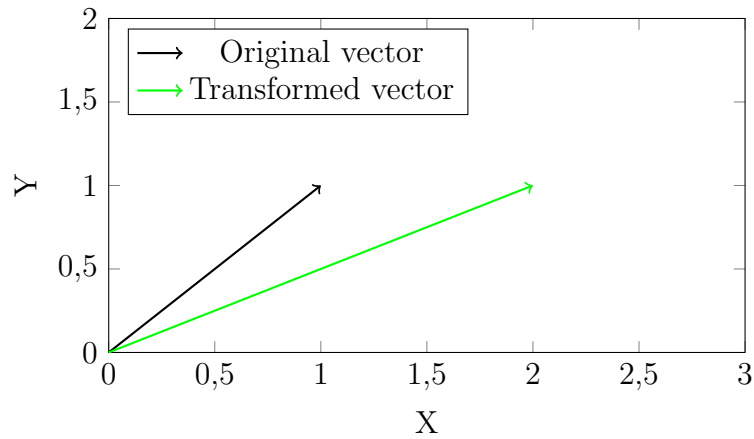


Figura 4: El vector original $\mathbf{x} = [1, 1]^T$ y su transformación $A\mathbf{x} = [2, 1]^T$.

Podemos ver en la figura generada que el vector original $\mathbf{x}$ es rotado y estirado después de ser transformado por A . El vector $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ se transforma en $\begin{bmatrix} 2 \\ 1 \end{bmatrix}$. Intentemos hacer el mismo ejercicio con un vector diferente $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$.

```

1 In [3]: x = np.array([[1], [0]])
2         b = np.dot(A, x)
3         plot_vect(x, b, (0, 3), (-0.5, 0.5))

```

Listing 46: Trazando otro vector y su transformación.

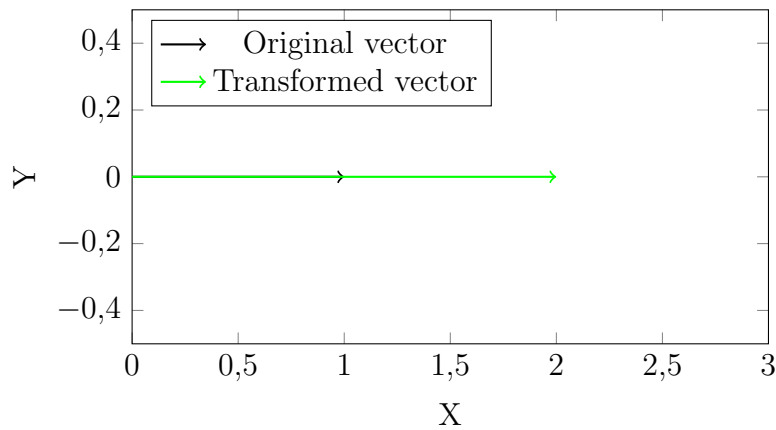


Figura 5: El vector $\mathbf{x} = [1, 0]^T$ es un vector propio con $\lambda = 2$.

Con este nuevo vector, el único cambio después de la transformación es

la longitud del vector; se estira. El nuevo vector es $\begin{bmatrix} 2 \\ 0 \end{bmatrix}$, por lo tanto, la transformación es

$$A\mathbf{x} = 2\mathbf{x}$$

con $\mathbf{x} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ y $\lambda = 2$. La dirección del vector no cambia en absoluto (sin rotación). También puedes comprobar que $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$ es otro vector propio; intenta verificar esto por ti mismo.

La Ecuación Característica

Para obtener los valores propios y vectores propios, de $A\mathbf{x} = \lambda\mathbf{x}$, podemos usar la siguiente forma:

$$(A - \lambda I)\mathbf{x} = \mathbf{0},$$

donde I es la matriz identidad con las mismas dimensiones que A . Si la matriz $A - \lambda I$ tiene una inversa y ambos lados se multiplican por $(A - \lambda I)^{-1}$, obtenemos una solución trivial $\mathbf{x} = \mathbf{0}$. Por lo tanto, el único caso interesante es cuando $A - \lambda I$ es singular (no existe inversa), y tenemos una solución no trivial, lo que significa que el determinante es cero:

$$\det(A - \lambda I) = 0.$$

Esta ecuación se llama la **ecuación característica** que conducirá a una ecuación polinómica para λ , que podemos resolver para los valores propios; ve el ejemplo a continuación.

```

1 In []: # Obtener los valores propios para la matriz  $\begin{bmatrix} 0 & 2 \\ 2 & 3 \end{bmatrix}$ ,
        # La ecuación característica nos da
2 #  $\begin{vmatrix} 0-\lambda & 2 \\ 2 & 3-\lambda \end{vmatrix}$ 
3 #  $\begin{vmatrix} 2 & 3-\lambda \end{vmatrix} = 0$ 
4 # Por lo tanto, tenemos  $-(3-\lambda) - 4 = 0$   $\lambda^2 - 3\lambda - 4$ 
5 #  $= 0$ .
6 # Obtenemos dos valores propios:  $\lambda_1 = 4$ ,  $\lambda_2 = -1$ .
```

Listing 47: Obteniendo la ecuación característica.

```

1 In []: # Para  $\lambda_1 = 4$ , resolvemos  $(A - 4I)\mathbf{x} = \mathbf{0}$ 
2 #  $\begin{bmatrix} -4 & 2 \\ 2 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ 
3 #  $\begin{bmatrix} 2 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \end{bmatrix}$ 
4 # Esto da  $-4x_1 + 2x_2 = 0$  y  $2x_1 - x_2 = 0$ , ambas indican que  $x_2$ 
5 #  $= 2x_1$ .
6 # Por lo tanto, el primer vector propio es  $\mathbf{x}_1 = k_1 \begin{bmatrix} 1 \\ 2 \end{bmatrix}^T$ .
```

```

6
7 # Para  $\lambda = -1$ , resolvemos  $(A + I)x = 0$ 
8 #  $\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ 
9 #  $\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ 
10 # Esto da  $x_1 + 2x_2 = 0$ , por lo tanto  $x_1 = -2x_2$ .
11 # Por lo tanto, el segundo vector propio es  $x_2 = k_2 \begin{bmatrix} -2 \\ 1 \end{bmatrix}^T$ 
12
13 # Verificacion para  $\lambda = 1$  :
14 #  $\begin{bmatrix} 0 & 2 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 4 \\ 8 \end{bmatrix} = 4 \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ 
15 #  $\begin{bmatrix} 0 & 2 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 4 \\ 8 \end{bmatrix} = 4 \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ 

```

Listing 48: Obteniendo los vectores propios.

El ejemplo anterior demuestra cómo podemos obtener los valores propios y vectores propios de una matriz A ; la elección de los vectores propios para un sistema no es única. Cuando tienes una matriz A más grande y tratas de resolver la ecuación característica de orden n , la solución se vuelve más complicada. Afortunadamente, se han desarrollado muchos métodos numéricos diferentes para resolver problemas de valores propios para matrices más grandes (con unos pocos cientos a miles de dimensiones). Introduciremos el método de la potencia y el método QR en las próximas dos secciones.

El Método de la Potencia

Encontrando el Valor Propio Más Grande

Algunos problemas solo requieren encontrar el valor propio dominante más grande y su vector propio correspondiente. En este caso, podemos usar el **método de la potencia**, que es un método iterativo que convergerá al valor propio más grande; ve el ejemplo a continuación.

Considera una matriz A de $n \times n$ que tiene n valores propios reales $\lambda_1, \lambda_2, \dots, \lambda_n$ y los correspondientes vectores propios linealmente independientes $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n$. Dado que los valores propios son escalares, podemos clasificarlos de modo que

$$|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|.$$

Observa que solo requerimos $|\lambda_1| > |\lambda_2|$; los otros valores propios pueden ser iguales entre sí.

Debido a que se supone que los vectores propios son linealmente independientes, son un conjunto de vectores base, lo que significa que cualquier vector que esté en el mismo espacio puede escribirse como una combinación lineal de los vectores base. Es decir, para cualquier vector $\mathbf{x}_0$ se puede escribir como:

$$\mathbf{x}_0 = c_1 \mathbf{v}_1 + c_2 \mathbf{v}_2 + \cdots + c_n \mathbf{v}_n$$

donde $c_1 \neq 0$ es la restricción. Si es cero, entonces necesitamos elegir otro vector inicial para que $c_1 \neq 0$.

Ahora multipliquemos ambos lados por A :

$$A\mathbf{x}_0 = c_1 A\mathbf{v}_1 + c_2 A\mathbf{v}_2 + \cdots + c_n A\mathbf{v}_n.$$

Dado que $A\mathbf{v}_i = \lambda_i \mathbf{v}_i$,

$$A\mathbf{x}_0 = c_1 \lambda_1 \mathbf{v}_1 + c_2 \lambda_2 \mathbf{v}_2 + \cdots + c_n \lambda_n \mathbf{v}_n.$$

Podemos cambiar la ecuación anterior a:

$$A\mathbf{x}_0 = c_1 \lambda_1 \left[\mathbf{v}_1 + \frac{c_2 \lambda_2}{c_1 \lambda_1} \mathbf{v}_2 + \cdots + \frac{c_n \lambda_n}{c_1 \lambda_1} \mathbf{v}_n \right] = c_1 \lambda_1 \mathbf{x}_1,$$

donde $\mathbf{x}_1$ es un nuevo vector, $\mathbf{x}_1 = \mathbf{v}_1 + \frac{c_2 \lambda_2}{c_1 \lambda_1} \mathbf{v}_2 + \cdots + \frac{c_n \lambda_n}{c_1 \lambda_1} \mathbf{v}_n$.

Esto termina la primera iteración. Para la segunda iteración, aplicamos A a $\mathbf{x}_1$:

$$A\mathbf{x}_1 = \lambda_1 \mathbf{v}_1 + \frac{c_2 \lambda_2^2}{c_1 \lambda_1} \mathbf{v}_2 + \cdots + \frac{c_n \lambda_n^2}{c_1 \lambda_1} \mathbf{v}_n.$$

De manera similar, podemos reorganizar la ecuación anterior para obtener

$$A\mathbf{x}_1 = \lambda_1 \left[\mathbf{v}_1 + \frac{c_2 \lambda_2^2}{c_1 \lambda_1^2} \mathbf{v}_2 + \cdots + \frac{c_n \lambda_n^2}{c_1 \lambda_1^2} \mathbf{v}_n \right] = \lambda_1 \mathbf{x}_2,$$

donde $\mathbf{x}_2$ es un nuevo vector, $\mathbf{x}_2 = \mathbf{v}_1 + \frac{c_2 \lambda_2^2}{c_1 \lambda_1^2} \mathbf{v}_2 + \cdots + \frac{c_n \lambda_n^2}{c_1 \lambda_1^2} \mathbf{v}_n$.

Si continuamos aplicando A al nuevo vector, obtenemos de cada iteración k :

$$A\mathbf{x}_{k-1} = \lambda_1 \left[\mathbf{v}_1 + \frac{c_2 \lambda_2^k}{c_1 \lambda_1^k} \mathbf{v}_2 + \cdots + \frac{c_n \lambda_n^k}{c_1 \lambda_1^k} \mathbf{v}_n \right] = \lambda_1 \mathbf{x}_k.$$

Debido a que λ_1 es el valor propio más grande, la relación $\frac{\lambda_i}{\lambda_1} < 1$ para todo $i > 1$. Cuando k es suficientemente grande, el factor $\left(\frac{\lambda_n}{\lambda_1}\right)^k$ estará cerca

de cero, por lo que todos los términos que contienen este factor pueden despreciarse a medida que k aumenta:

$$A\mathbf{x}_{k-1} \sim \lambda_1 \mathbf{v}_1.$$

Esencialmente, si k es lo suficientemente grande, obtendremos el valor propio más grande y su vector propio correspondiente. Al implementar este método de la potencia, el vector resultante en cada iteración generalmente se **normaliza**. Esto se puede hacer factorizando el elemento más grande en el vector, lo que hará que el elemento más grande en el vector sea igual a 1. Esta normalización proporcionará el valor propio más grande y su vector propio correspondiente al mismo tiempo; ve el ejemplo a continuación.

¿Cuándo debemos detener la iteración? El criterio de parada básico debería ser uno de los siguientes: (1) la diferencia entre valores propios es menor que alguna tolerancia especificada; (2) el ángulo entre vectores propios es menor que un umbral; o (3) la norma del vector residual es lo suficientemente pequeña.

```

1 In []: # Sabemos de la ultima seccion que el valor propio mas
        grande es 4 para la matriz A = [[0, 2], [2, 3]].
2 # Usar el metodo de la potencia para encontrar el valor
        propio mas grande y el vector propio asociado.
3 # Puedes usar el vector inicial [1, 1] para comenzar la
        iteracion.
4
5 # Primera iteracion: A * [1, 1]^T = [2, 5]^T = 5 * [0.4, 1]^T
6 # Segunda iteracion: A * [0.4, 1]^T = [2, 3.8]^T = 3.8 *
        [0.5263, 1]^T
7 # Tercera iteracion: A * [0.5263, 1]^T = [2, 4.0526]^T =
        4.0526 * [0.4935, 1]^T
8 # Cuarta iteracion: A * [0.4935, 1]^T = [2, 3.987]^T = 3.987
        * [0.5016, 1]^T
9 # Quinta iteracion: A * [0.5016, 1]^T = [2, 4.0032]^T =
        4.0032 * [0.4996, 1]^T
10 # Sexta iteracion: A * [0.4996, 1]^T = [2, 3.9992]^T = 3.9992
        * [0.5001, 1]^T
11 # Septima iteracion: A * [0.5001, 1]^T = [2, 4.0002]^T =
        4.0002 * [0.5000, 1]^T
12 # Despues de siete iteraciones, el valor propio ha convergido
        a cuatro, con [0.5, 1] como el vector propio
        correspondiente.

```

Listing 49: Aplicación manual del método de la potencia.

```

1 In [1]: import numpy as np
2
3 In [2]: def normalize(x):
4           fac = abs(x).max()
5           x_n = x / x.max()
6           return fac, x_n
7
8 In [3]: x = np.array([1, 1])
9           a = np.array([[0, 2],
10                        [2, 3]])
11           for i in range(8):
12               x = np.dot(a, x)
13               lambda_1, x = normalize(x)
14               print("Eigenvalue:", lambda_1)
15               print("Eigenvector:", x)
16 Eigenvalue: 3.999949137887188
17 Eigenvector: [0.50000636 1.]

```

Listing 50: Implementación del método de la potencia en Python.

El Método de la Potencia Inversa

Los valores propios de la matriz inversa A^{-1} son los recíprocos de los valores propios de A . Aprovechando esta característica, así como el método de la potencia, podemos obtener el valor propio más pequeño de A ; esta será la base del **método de la potencia inversa**. Los pasos son muy simples: en lugar de aplicar A como se describió anteriormente, simplemente aplicamos A^{-1} para nuestra iteración para encontrar el valor más grande de $\frac{1}{\lambda_1}$, que será el valor más pequeño de los valores propios de A . En la práctica, podemos usar los métodos que cubrimos en el capítulo anterior para calcular la inversa de la matriz. No entraremos en más detalles aquí, pero presentamos un ejemplo a continuación.

```

1 In [4]: from numpy.linalg import inv
2
3 In [5]: a_inv = inv(a)
4           for i in range(8):
5               x = np.dot(a_inv, x)
6               lambda_1, x = normalize(x)
7               print("Eigenvalue:", lambda_1)
8               print("Eigenvector:", x)
9 Eigenvalue: 0.200000000000003912

```

10 **Eigenvector:** [1. 1.]

Listing 51: Encontrando el valor propio más pequeño con el método de la potencia inversa.

El Método de la Potencia Desplazada

En algunos casos, es necesario encontrar todos los valores propios y vectores propios en lugar de solo el más grande y el más pequeño. Una forma simple, pero ineficiente, es usar el **método de la potencia desplazada**; te presentaremos un método más eficiente en la próxima sección.

Dado $A\mathbf{x} = \lambda_1\mathbf{x}$, y λ_1 es el valor propio más grande obtenido por el método de la potencia, tenemos

$$[A - \lambda_1 I]\mathbf{x} = \alpha\mathbf{x},$$

donde α son los valores propios de la matriz desplazada $A - \lambda_1 I$, que serán $0, \lambda_2 - \lambda_1, \lambda_3 - \lambda_1, \dots, \lambda_n - \lambda_1$.

Ahora, si aplicamos el método de la potencia a la matriz desplazada, podemos determinar el valor propio más grande de la matriz desplazada, es decir, α_k . Dado que $\alpha_k = \lambda_k - \lambda_1$, podemos obtener el valor propio λ_k fácilmente. Repetir este proceso muchas veces encontrará todos los otros valores propios, pero puedes ver que es muy laborioso. Un método mejor para encontrar todos los valores propios es usar el método QR, que presentaremos a continuación.

El Método QR

El **método QR** es el método iterativo preferido para encontrar todos los valores propios de una matriz (pero no los vectores propios al mismo tiempo). La idea se basa en los siguientes dos conceptos:

1. Las **matrices similares** tendrán los mismos valores propios y vectores propios asociados. Dos matrices cuadradas A y B son similares si

$$A = C^{-1}BC$$

donde C es una matriz invertible.

2. El método QR es una forma de descomponer una matriz en dos matrices Q y R , donde Q es una matriz **ortogonal** y R es una matriz **triangular superior**. Una matriz ortogonal satisface $Q^{-1} = Q^T$, lo que significa $Q^{-1}Q = Q^TQ = I$.

¿Cómo vinculamos estos dos conceptos para encontrar los valores propios? Supongamos que tenemos una matriz A_0 cuyos valores propios deben determinarse. En el k -ésimo paso (comenzando con $k = 0$), podemos realizar la descomposición QR y obtener

$$A_k = Q_k R_k$$

donde Q_k es una matriz ortogonal y R_k es una matriz triangular superior. Luego formamos

$$A_{k+1} = R_k Q_k$$

para obtener

$$A_{k+1} = R_k Q_k = Q_k^{-1} Q_k R_k Q_k = Q_k^{-1} A_k Q_k.$$

Debido a que todos los A_k son similares, como discutimos anteriormente, todos tienen los mismos valores propios.

A medida que la iteración continúa, eventualmente convergeremos a una matriz triangular superior de la forma:

$$A_k = R_k Q_k = \begin{bmatrix} \lambda_1 & X & \dots & X \\ 0 & \lambda_2 & \dots & X \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \lambda_n \end{bmatrix},$$

donde los valores diagonales son los valores propios de la matriz. En cada iteración del método QR, factorizar una matriz en una matriz ortogonal y una matriz triangular superior se puede hacer usando una matriz especial llamada **matriz de Householder**. No entraremos en los detalles matemáticos de cómo se obtienen Q y R a partir de la matriz. En su lugar, usaremos la función de Python para obtener las dos matrices directamente.

```

1 In [1]: import numpy as np
2         from numpy.linalg import qr
3
4 In [2]: a = np.array([[0, 2],
```



```

5         [2, 3]])
6     q, r = qr(a)
7     print("Q:", q)
8     print("R:", r)
9     b = np.dot(q, r)
10    print("QR:", b)
11 Q: [[ 0. -1.]
12     [-1.  0.]]
13 R: [[-2. -3.]
14     [ 0. -2.]]
15 QR: [[0. 2.]
16     [2. 3.]]

```

Listing 52: Descomposición QR con `numpy.linalg.qr`.

```

1 In [3]: a = np.array([[0, 2],
2                     [2, 3]])
3     p = [1, 5, 10, 20]
4     for i in range(20):
5         q, r = qr(a)
6         a = np.dot(r, q)
7         if i+1 in p:
8             print(f"Iteration {i+1}:")
9             print(a)
10 Iteration 1:
11 [[3. 2.]
12  [2. 0.]]
13 Iteration 5:
14 [[ 3.99998093  0.00976559]
15  [ 0.00976559 -0.99998093]]
16 Iteration 10:
17 [[ 4.00000000e+00  9.53674316e-06]
18  [ 9.53674316e-06 -1.00000000e+00]]
19 Iteration 20:
20 [[ 4.00000000e+00  9.09484250e-12]
21  [ 9.09494702e-12 -1.00000000e+00]]

```

Listing 53: Método QR para encontrar valores propios.

Observa que después de la quinta iteración, los valores propios han convergido a los correctos. La próxima sección demostrará cómo obtener los valores propios y vectores propios en Python usando la función incorporada.

Valores Propios y Vectores Propios en Python

Los métodos introducidos anteriormente son bastante complicados de ejecutar. El cálculo de valores propios y vectores propios en Python es bastante fácil. La función incorporada principal en Python para resolver el problema de valor propio/vector propio para un array cuadrado es la función `eig` en `numpy.linalg`; ve a continuación un ejemplo de cómo ejecutarla.

```
1 In [1]: import numpy as np
2         from numpy.linalg import eig
3
4 In [2]: a = np.array([[0, 2],
5                       [2, 3]])
6         w, v = eig(a)
7         print("E-value:", w)
8         print("E-vector", v)
9 E-value: [-1.  4.]
10 E-vector [[-0.89442719 -0.4472136 ]
11           [ 0.4472136  -0.89442719]]
```

Listing 54: Usando `numpy.linalg.eig` para calcular valores y vectores propios.

```
1 In [3]: a = np.array([[2, 2, 4],
2                       [1, 3, 5],
3                       [2, 3, 4]])
4         w, v = eig(a)
5         print("E-value:", w)
6         print("E-vector", v)
7 E-value: [ 8.80916362  0.92620912 -0.73537273]
8 E-vector [[-0.52799324 -0.77557092 -0.36272811]
9           [-0.604391    0.62277013 -0.7103262 ]
10           [-0.59660259 -0.10318482  0.60321224]]
```

Listing 55: Calculando valores y vectores propios para una matriz 3x3.

Resumen y Problemas

Resumen

1. Los valores propios y vectores propios nos ayudan a comprender las características de una transformación lineal.

2. Los vectores propios de una matriz son los vectores que solo pueden escalar longitudinalmente sin rotación después de aplicar la transformación de la matriz; los valores propios son los factores de escala.
3. Podemos usar el método de la potencia para obtener el valor propio más grande y el vector propio correspondiente de una matriz.
4. El método de la potencia inversa puede ayudarnos a obtener el valor propio más pequeño y el vector propio correspondiente de una matriz.
5. El método de la potencia desplazada puede obtener todos los otros valores/vectores propios de una matriz.
6. El método preferido para obtener todos los valores propios es el método QR.

Problemas y Soluciones

1. **Problema 1:** Escribe la ecuación característica para la matriz $A = \begin{bmatrix} 3 & 2 \\ 5 & 3 \end{bmatrix}$.

Solución: La ecuación característica es $\det(A - \lambda I) = 0$.

$$\det \begin{bmatrix} 3 - \lambda & 2 \\ 5 & 3 - \lambda \end{bmatrix} = (3 - \lambda)^2 - 10 = 0.$$

2. **Problema 2:** Usa la ecuación característica anterior para resolver los valores propios y vectores propios de la matriz A .

Solución: La ecuación es $(3 - \lambda)^2 = 10$, por lo tanto $3 - \lambda = \pm\sqrt{10}$. Los valores propios son $\lambda_1 = 3 + \sqrt{10}$ y $\lambda_2 = 3 - \sqrt{10}$. Para λ_1 , resolvemos $(A - \lambda_1 I)\mathbf{v} = \mathbf{0}$:

$$\begin{bmatrix} -\sqrt{10} & 2 \\ 5 & -\sqrt{10} \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Esto da $-\sqrt{10}v_1 + 2v_2 = 0$, por lo tanto $v_2 = \frac{\sqrt{10}}{2}v_1$. El

vector propio es $\mathbf{v}_1 = k_1 \begin{bmatrix} 2 \\ \sqrt{10} \end{bmatrix}$. Para λ_2 , obtenemos $\mathbf{v}_2 =$

$$k_2 \begin{bmatrix} 2 \\ -\sqrt{10} \end{bmatrix}.$$

3. **Problema 3:** Usa el primer vector propio derivado del Problema 2 para verificar que $A\mathbf{x} = \lambda\mathbf{x}$.

Solución: Usando $\mathbf{x} = [2, \sqrt{10}]^T$, calculamos $A\mathbf{x} = [3 \cdot 2 + 2\sqrt{10}, 5 \cdot 2 + 3\sqrt{10}]^T = [6 + 2\sqrt{10}, 10 + 3\sqrt{10}]^T$. Por otro lado, $\lambda\mathbf{x} = (3 + \sqrt{10})[2, \sqrt{10}]^T = [2(3 + \sqrt{10}), \sqrt{10}(3 + \sqrt{10})]^T = [6 + 2\sqrt{10}, 3\sqrt{10} + 10]^T$. Son iguales.

4. **Problema 4:** Usa el método de la potencia para obtener el valor propio

más grande y el vector propio para la matriz $A = \begin{bmatrix} 2 & 1 & 2 \\ 1 & 3 & 2 \\ 2 & 4 & 1 \end{bmatrix}$. Comienza

con el vector inicial $[1, 1, 1]$ y observa los resultados después de ocho iteraciones.

Solución: Implementamos el método de la potencia.

```

1 import numpy as np
2
3 def normalize(x):
4     fac = abs(x).max()
5     x_n = x / fac
6     return fac, x_n
7
8 A = np.array([[2, 1, 2], [1, 3, 2], [2, 4, 1]])
9 x = np.array([1.0, 1.0, 1.0])
10
11 for i in range(8):
12     x = np.dot(A, x)
13     lambda_1, x = normalize(x)
14     print(f"Iteracion {i+1}: lambda = {lambda_1}, vector
15           = {x}")

```

Listing 56: Solución Problema 4

5. **Problema 5:** Usa el método de la potencia inversa para obtener el valor propio más pequeño y el vector propio para la matriz del Problema 4. Observa cuántas iteraciones se necesitan para que converja al valor propio más pequeño.

Solución: Usamos la inversa de la matriz.

```

1 from numpy.linalg import inv
2
3 A_inv = inv(A)
4 x = np.array([1.0, 1.0, 1.0])
5 tol = 1e-6
6 for i in range(50):
7     x_new = np.dot(A_inv, x)
8     lambda_inv, x_new = normalize(x_new)
9     if np.linalg.norm(x_new - x) < tol:
10         print(f"Convergio en iteracion {i+1}")
11         break
12     x = x_new
13 lambda_min = 1 / lambda_inv
14 print(f"Valor propio mas pequeno: {lambda_min}")
15

```

Listing 57: Solución Problema 5

6. **Problema 6:** Realiza una descomposición QR para la matriz A del Problema 4. Verifica que $A = QR$ y que Q es una matriz ortogonal.

Solución: Usamos `np.linalg.qr` y verificamos las propiedades.

```

1 from numpy.linalg import qr
2
3 Q, R = qr(A)
4 print("Q:\n", Q)
5 print("R:\n", R)
6 print("QR:\n", np.dot(Q, R))
7 print("Q es ortogonal? (Q^T Q = I):\n", np.dot(Q.T, Q))
8

```

Listing 58: Solución Problema 6

7. **Problema 7:** Usa el método QR para obtener todos los valores propios de la matriz A del Problema 4.

Solución: Iteramos el proceso QR hasta que la matriz se vuelva triangular superior (con elementos por debajo de la diagonal cercanos a cero).

```

1 A_k = A.copy()
2 for i in range(100):

```

```
3     Q, R = qr(A_k)
4     A_k = np.dot(R, Q)
5     if i % 20 == 0:
6         print(f"Iteracion {i}:\n", A_k)
7     # Los valores propios son los elementos diagonales de A_k
8     .
9     eigenvalues_qr = np.diag(A_k)
10    print("Valores propios (QR):", eigenvalues_qr)
```

Listing 59: Solución Problema 7

8. **Problema 8:** Obtén los valores propios y vectores propios para la matriz A del Problema 4 usando la función incorporada de Python.

Solución: Usamos `np.linalg.eig`.

```
1 from numpy.linalg import eig
2
3 w, v = eig(A)
4 print("Valores propios (eig):", w)
5 print("Vectores propios (eig):\n", v)
6
```

Listing 60: Solución Problema 8

REGRESIÓN POR MÍNIMOS CUADRADOS

Planteamiento del Problema de Regresión por Mínimos Cuadrados

Dado un conjunto de puntos de datos independientes x_i y puntos de datos dependientes y_i , con $i = 1, \dots, m$, nos gustaría encontrar una **función de estimación**, $\hat{y}(x)$, que describa los datos con la mayor precisión posible. Observa que $\hat{y}$ puede ser una función de varias variables, pero para esta discusión, restringimos el dominio de $\hat{y}$ a una sola variable. En la regresión por mínimos cuadrados, la función de estimación debe ser una combinación lineal de **funciones base**, $f_i(x)$. Es decir, la función de estimación debe ser de la forma

$$\hat{y}(x) = \sum_{i=1}^n \alpha_i f_i(x).$$

Los escalares α_i se denominan **parámetros** de la función de estimación, y cada función base debe ser linealmente independiente de las demás. En otras palabras, en el espacio funcional apropiado, ninguna función base debe poder expresarse como una combinación lineal de las otras funciones. Observa que, en general, hay significativamente más puntos de datos, m , que funciones base, n (es decir, $m \gg n$).

```
1 In []: # La relacion entre la fuerza, F, y el desplazamiento,
        x, puede describirse mediante la funcion F(x) = kx,
2 # donde k es la rigidez del resorte. La unica funcion base es
        f1(x) = x y el parametro del modelo a determinar es 1 =
        k.
```

Listing 61: Creando una función de estimación para un resorte lineal.

El objetivo de la regresión por mínimos cuadrados es encontrar los parámetros de la función de estimación que minimicen el **error cuadrático total**, E , definido por

$$E = \sum_{i=1}^m (\hat{y}(x_i) - y_i)^2.$$

Los errores individuales o **residuos** se definen como $e_i = \hat{y}(x_i) - y_i$. Si $\mathbf{e}$ es el vector que contiene todos los errores individuales, entonces también estamos tratando de minimizar $E = \|\mathbf{e}\|_2^2$, que es la norma L_2 definida en el capítulo anterior.

En las siguientes dos secciones, derivaremos el método de mínimos cuadrados para encontrar los parámetros deseados. La primera derivación proviene del álgebra lineal, y la segunda del cálculo multivariable. Aunque son derivaciones diferentes, conducen a la misma fórmula de mínimos cuadrados. Eres libre de centrarte en la sección con la que te sientas más cómodo.

Derivación de la Regresión por Mínimos Cuadrados (Álgebra Lineal)

Primero, enumeramos la estimación de los datos en cada punto de datos x_i como

$$\begin{aligned}\hat{y}(x_1) &= \alpha_1 f_1(x_1) + \alpha_2 f_2(x_1) + \cdots + \alpha_n f_n(x_1), \\ \hat{y}(x_2) &= \alpha_1 f_1(x_2) + \alpha_2 f_2(x_2) + \cdots + \alpha_n f_n(x_2), \\ &\vdots \\ \hat{y}(x_m) &= \alpha_1 f_1(x_m) + \alpha_2 f_2(x_m) + \cdots + \alpha_n f_n(x_m).\end{aligned}$$

Sea $\mathbf{X} \in \mathbb{R}^n$ un vector columna tal que el i -ésimo elemento de $\mathbf{X}$ contiene el valor del i -ésimo punto de datos x_i . Sea $\hat{\mathbf{Y}}$ un vector columna con elementos $\hat{Y}_i = \hat{y}(x_i)$, $\boldsymbol{\beta}$ un vector columna tal que $\beta_i = \alpha_i$, $\mathbf{F}_i(\mathbf{X})$ una función que devuelve un vector columna de $f_i(x)$ calculado en cada elemento de $\mathbf{X}$, y $\mathbf{A}$ una matriz de $m \times n$ tal que la i -ésima columna de $\mathbf{A}$ es $\mathbf{F}_i(\mathbf{X})$. Dada esta notación, el sistema de ecuaciones anterior se convierte en $\hat{\mathbf{Y}} = \mathbf{A}\boldsymbol{\beta}$.

Ahora, si $\mathbf{Y}$ es un vector columna tal que $Y_i = y_i$, el error cuadrático total está dado por $E = \|\hat{\mathbf{Y}} - \mathbf{Y}\|_2^2$. Verifica esto sustituyendo la definición de la norma L_2 . Dado que queremos que E sea lo más pequeño posible y las normas son una medida de distancia, esta expresión anterior es equivalente a

decir que queremos que $\hat{\mathbf{Y}}$ y $\mathbf{Y}$ estén lo más cerca posible”. Observa que, en general, $\mathbf{Y}$ no estará en el rango de $\mathbf{A}$ y por lo tanto $E > 0$.

Considera la siguiente representación simplificada del rango de $\mathbf{A}$; ver Fig. 13. Observa que esto no es una gráfica de los puntos de datos (x_i, y_i) .

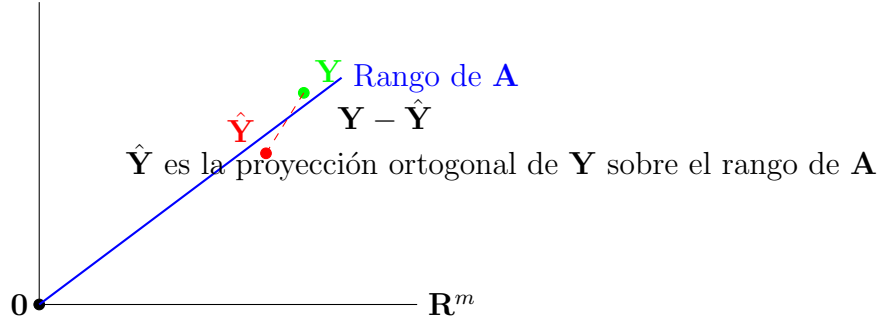


Figura 6: Ilustración de la proyección L_2 de $\mathbf{Y}$ sobre el rango de $\mathbf{A}$.

Por observación, el vector en el rango de $\mathbf{A}$, es decir, $\hat{\mathbf{Y}}$, que está más cerca de $\mathbf{Y}$ es aquel que es perpendicular a $\mathbf{Y} - \hat{\mathbf{Y}}$. Por lo tanto, queremos un vector $\mathbf{Y} - \hat{\mathbf{Y}}$ que sea perpendicular al vector $\hat{\mathbf{Y}}$. Recordemos del álgebra lineal que dos vectores son perpendiculares u **ortogonales** si su producto punto es 0. Notando que el producto punto entre dos vectores, $\mathbf{v}$ y $\mathbf{w}$, puede escribirse como $\text{dot}(\mathbf{v}, \mathbf{w}) = \mathbf{v}^T \mathbf{w}$, podemos afirmar que $\hat{\mathbf{Y}}$ y $\mathbf{Y} - \hat{\mathbf{Y}}$ son perpendiculares si $\text{dot}(\hat{\mathbf{Y}}, \mathbf{Y} - \hat{\mathbf{Y}}) = 0$; por lo tanto, $\hat{\mathbf{Y}}^T (\mathbf{Y} - \hat{\mathbf{Y}}) = 0$, que es equivalente a $(\mathbf{A}\beta)^T (\mathbf{Y} - \mathbf{A}\beta) = 0$.

Observando que para dos matrices $\mathbf{A}$ y $\mathbf{B}$ tenemos $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$ y usando las propiedades distributivas de la multiplicación de vectores, esto es equivalente a $\beta^T \mathbf{A}^T \mathbf{Y} - \beta^T \mathbf{A}^T \mathbf{A} \beta = \beta^T (\mathbf{A}^T \mathbf{Y} - \mathbf{A}^T \mathbf{A} \beta) = 0$. La solución $\beta = \mathbf{0}$ es una solución trivial, así que usamos $\mathbf{A}^T \mathbf{Y} - \mathbf{A}^T \mathbf{A} \beta = 0$ para encontrar una solución más interesante. Resolviendo esta ecuación para β se obtiene la **fórmula de regresión por mínimos cuadrados**:

$$\boxed{\beta = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{Y}}.$$

Observa que $(\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T$ se llama la **pseudo-inversa** de $\mathbf{A}$ y existe cuando $m > n$ y $\mathbf{A}$ tiene columnas linealmente independientes. Demostrar la invertibilidad de $(\mathbf{A}^T \mathbf{A})$ está fuera del alcance de este libro, pero siempre es invertible excepto en algunos casos específicos.

Derivación de la Regresión por Mínimos Cuadrados (Cálculo Multivariable)

Recordemos que el error total para m puntos de datos y n funciones base es:

$$E = \sum_{i=1}^m e_i^2 = \sum_{i=1}^m (\hat{y}(x_i) - y_i)^2 = \sum_{i=1}^m \left(\sum_{j=1}^n \alpha_j f_j(x_i) - y_i \right)^2,$$

que es un paraboloide n -dimensional en α_k . Del cálculo, sabemos que el mínimo de un paraboloide es donde todas las derivadas parciales son iguales a cero. Tomando la derivada parcial de E con respecto a la variable α_k (recuerda que en este caso los parámetros son nuestras variables) e igualando estas derivadas a 0, y luego resolviendo el sistema de ecuaciones para los α_k debería dar los resultados correctos.

Calculando la derivada parcial con respecto a α_k e igualándola a cero se obtiene

$$\frac{\partial E}{\partial \alpha_k} = \sum_{i=1}^m 2 \left(\sum_{j=1}^n \alpha_j f_j(x_i) - y_i \right) f_k(x_i) = 0.$$

Con algo de reordenamiento, la expresión anterior puede manipularse de la siguiente manera:

$$\sum_{i=1}^m \sum_{j=1}^n \alpha_j f_j(x_i) f_k(x_i) - \sum_{i=1}^m y_i f_k(x_i) = 0,$$

y tras una mayor reorganización (aprovechamos el hecho de que la suma conmuta), el resultado es

$$\sum_{j=1}^n \alpha_j \sum_{i=1}^m f_j(x_i) f_k(x_i) = \sum_{i=1}^m y_i f_k(x_i).$$

Ahora, sea $\mathbf{X}$ un vector columna tal que el i -ésimo elemento de $\mathbf{X}$ es x_i y $\mathbf{Y}$ construido de manera similar, y sea $\mathbf{F}_j(\mathbf{X})$ un vector columna tal que el i -ésimo elemento de $\mathbf{F}_j(\mathbf{X})$ es $f_j(x_i)$. Usando esta notación, la expresión

anterior puede reescribirse en notación vectorial como

$$\left[\mathbf{F}_k(\mathbf{X})^T \mathbf{F}_1(\mathbf{X}), \mathbf{F}_k(\mathbf{X})^T \mathbf{F}_2(\mathbf{X}), \dots, \mathbf{F}_k(\mathbf{X})^T \mathbf{F}_j(\mathbf{X}), \dots, \mathbf{F}_k(\mathbf{X})^T \mathbf{F}_n(\mathbf{X}) \right] \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_j \\ \vdots \\ \alpha_n \end{bmatrix} = \mathbf{F}_k(\mathbf{X})^T \mathbf{Y}.$$

Si repetimos esta ecuación para cada k , obtenemos el siguiente sistema de ecuaciones lineales en forma matricial:

$$\begin{bmatrix} \mathbf{F}_1(\mathbf{X})^T \mathbf{F}_1(\mathbf{X}) & \mathbf{F}_1(\mathbf{X})^T \mathbf{F}_2(\mathbf{X}) & \dots & \mathbf{F}_1(\mathbf{X})^T \mathbf{F}_n(\mathbf{X}) \\ \mathbf{F}_2(\mathbf{X})^T \mathbf{F}_1(\mathbf{X}) & \mathbf{F}_2(\mathbf{X})^T \mathbf{F}_2(\mathbf{X}) & \dots & \mathbf{F}_2(\mathbf{X})^T \mathbf{F}_n(\mathbf{X}) \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{F}_n(\mathbf{X})^T \mathbf{F}_1(\mathbf{X}) & \mathbf{F}_n(\mathbf{X})^T \mathbf{F}_2(\mathbf{X}) & \dots & \mathbf{F}_n(\mathbf{X})^T \mathbf{F}_n(\mathbf{X}) \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_n \end{bmatrix} = \begin{bmatrix} \mathbf{F}_1(\mathbf{X})^T \mathbf{Y} \\ \mathbf{F}_2(\mathbf{X})^T \mathbf{Y} \\ \vdots \\ \mathbf{F}_n(\mathbf{X})^T \mathbf{Y} \end{bmatrix}.$$

Si definimos $\mathbf{A} = [\mathbf{F}_1(\mathbf{X}), \mathbf{F}_2(\mathbf{X}), \dots, \mathbf{F}_n(\mathbf{X})]$ y $\boldsymbol{\beta}$ como un vector columna tal que el j -ésimo elemento de $\boldsymbol{\beta}$ es α_j , entonces el sistema de ecuaciones anterior se convierte en

$$\mathbf{A}^T \mathbf{A} \boldsymbol{\beta} = \mathbf{A}^T \mathbf{Y},$$

y luego resolviendo esta ecuación matricial para $\boldsymbol{\beta}$ se obtiene $\boldsymbol{\beta} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{Y}$, que es exactamente la misma fórmula que en la derivación anterior.

Regresión por Mínimos Cuadrados en Python

Recordemos que enumerar la estimación de los datos en cada punto de datos x_i nos dará el siguiente sistema de ecuaciones:

$$\begin{aligned} \hat{y}(x_1) &= \alpha_1 f_1(x_1) + \alpha_2 f_2(x_1) + \dots + \alpha_n f_n(x_1), \\ \hat{y}(x_2) &= \alpha_1 f_1(x_2) + \alpha_2 f_2(x_2) + \dots + \alpha_n f_n(x_2), \\ &\vdots \\ \hat{y}(x_m) &= \alpha_1 f_1(x_m) + \alpha_2 f_2(x_m) + \dots + \alpha_n f_n(x_m). \end{aligned}$$

Si los datos fueran absolutamente perfectos (es decir, sin ruido), entonces la función de estimación pasaría por todos los puntos de datos, resultando en

el siguiente sistema de ecuaciones:

$$\begin{aligned} y_1 &= \alpha_1 f_1(x_1) + \alpha_2 f_2(x_1) + \cdots + \alpha_n f_n(x_1), \\ y_2 &= \alpha_1 f_1(x_2) + \alpha_2 f_2(x_2) + \cdots + \alpha_n f_n(x_2), \\ &\vdots \\ y_m &= \alpha_1 f_1(x_m) + \alpha_2 f_2(x_m) + \cdots + \alpha_n f_n(x_m). \end{aligned}$$

Si tomamos $\mathbf{A}$ como se definió anteriormente, esto daría como resultado la ecuación matricial

$$\mathbf{Y} = \mathbf{A}\boldsymbol{\beta}.$$

Debido a que los datos no son perfectos, no habrá una función de estimación que pueda pasar por todos los puntos de datos, y este sistema no tendrá solución. Por lo tanto, necesitamos usar la regresión por mínimos cuadrados que derivamos en las dos secciones anteriores para obtener una solución:

$$\boldsymbol{\beta} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{Y}.$$

```

1 In [1]: import numpy as np
2         from scipy import optimize
3         import matplotlib.pyplot as plt
4         plt.style.use("seaborn-poster")
5
6 In [2]: # generate x and y
7         x = np.linspace(0, 1, 101)
8         y = 1 + x + x * np.random.random(len(x))
9
10 In [3]: # assemble matrix A
11         A = np.vstack([x, np.ones(len(x))]).T
12         # turn y into a column vector
13         y = y[:, np.newaxis]
14
15 In [4]: # Direct least squares regression
16         alpha = np.dot((np.dot(np.linalg.inv(np.dot(A.T, A)),
17                                A.T)), y)
18         print(alpha)
19         [[1.459573 ]
20          [1.02952189]]
21
22 In [5]: # plot the results
23         plt.figure(figsize=(10, 8))
24         plt.plot(x, y, "b.")

```

```
24 plt.plot(x, alpha[0]*x + alpha[1], "r")
25 plt.xlabel("x")
26 plt.ylabel("y")
27 plt.show()
```

Listing 62: Regresión lineal simple con mínimos cuadrados.

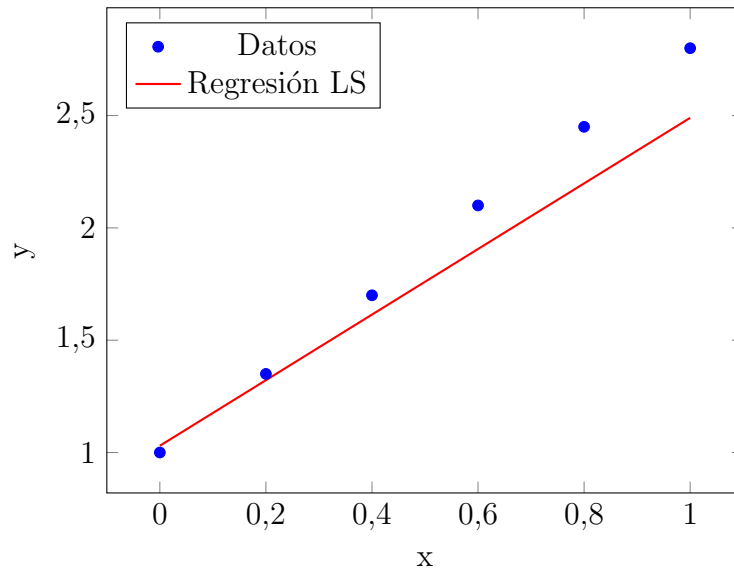


Figura 7: Puntos de datos y recta de regresión por mínimos cuadrados.

Python tiene varios paquetes y funciones que pueden realizar una regresión por mínimos cuadrados. Estos incluyen NumPy, SciPy, statsmodels y sklearn. A continuación, se muestran varios ejemplos de tales aplicaciones. Siéntete libre de elegir el que más te guste.

Usando el Método de la Inversa Directa

Ya se mostró en el Listado [75](#).

Usando la Pseudo-Inversa

Mencionamos anteriormente que la matriz $(\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T$ se llama la pseudo-inversa, por lo tanto, podemos usar la función `pinv` en NumPy para calcularla directamente.

```
1 In [6]: pinv = np.linalg.pinv(A)
2         alpha = pinv.dot(y)
3         print(alpha)
4 [[1.459573 ]
5  [1.02952189]]
```

Listing 63: Usando la pseudo-inversa.

Usando `numpy.linalg.lstsq`

NumPy ya ha implementado los métodos de mínimos cuadrados, por lo que podemos simplemente llamar a la función para obtener una solución. La función devolverá más datos que la solución en sí; consulta la documentación para más detalles.

```
1 In [7]: alpha = np.linalg.lstsq(A, y, rcond=None)[0]
2         print(alpha)
3 [[1.459573 ]
4  [1.02952189]]
```

Listing 64: Usando `numpy.linalg.lstsq`.

Usando `optimize.curve_fit` de SciPy

Esta función de SciPy es muy potente. No solo es adecuada para funciones lineales, sino también para muchas formas de funciones diferentes, como funciones no lineales. Aquí solo mostraremos el ejemplo lineal de arriba. Observa que, al usar esta función, no necesitamos convertir **y** en un vector columna.

```
1 In [8]: # generate x and y
2         x = np.linspace(0, 1, 101)
3         y = 1 + x + x * np.random.random(len(x))
4
5 In [9]: def func(x, a, b):
6         y = a*x + b
7         return y
8
9         alpha = optimize.curve_fit(func, xdata=x, ydata=y)[0]
10        print(alpha)
11 [1.44331612 1.0396133 ]
```

Listing 65: Usando `scipy.optimize.curve_fit`.

Regresión por Mínimos Cuadrados para Funciones No Lineales

Una regresión por mínimos cuadrados requiere que la función de estimación sea una combinación lineal de funciones base. Hay algunas funciones que no pueden ponerse en esta forma, pero donde una regresión por mínimos cuadrados sigue siendo apropiada.

A continuación, se presentan varias formas de tratar con funciones no lineales.

- Podemos lograr esto aprovechando las propiedades de los logaritmos y transformar la función no lineal en una función lineal.
- Podemos usar la función `curve_fit` de SciPy para estimar directamente los parámetros de la función no lineal usando mínimos cuadrados.

Trucos Logarítmicos para Funciones Exponenciales

Supongamos que tienes una función de la forma $\hat{y}(x) = \alpha e^{\beta x}$ y datos para x e y , y que desees realizar una regresión por mínimos cuadrados para encontrar α y β . Claramente, el conjunto anterior de funciones base (lineales) sería inapropiado para describir $\hat{y}(x)$; sin embargo, si tomamos el logaritmo de ambos lados, obtenemos $\log(\hat{y}(x)) = \log(\alpha) + \beta x$. Ahora, vemos que si definimos $\tilde{y}(x) = \log(\hat{y}(x))$ y $\tilde{\alpha} = \log(\alpha)$, entonces $\tilde{y}(x) = \tilde{\alpha} + \beta x$. Por lo tanto, podemos realizar una regresión por mínimos cuadrados sobre la expresión linealizada para encontrar $\tilde{y}(x)$, $\tilde{\alpha}$ y β , y luego recuperar α usando la expresión $\alpha = e^{\tilde{\alpha}}$.

Para el siguiente ejemplo, generaremos datos usando $\alpha = 0,1$ y $\beta = 0,3$.

```

1 In [1]: import numpy as np
2         from scipy import optimize
3         import matplotlib.pyplot as plt
4         plt.style.use("seaborn-poster")
5
6 In [2]: # let's generate x and y, and add some noise into y
7         x = np.linspace(0, 10, 101)
8         y = 0.1*np.exp(0.3*x) + 0.1*np.random.random(len(x))
9
10 In [3]: # Let's have a look of the data
11         plt.figure(figsize=(10, 8))
12         plt.plot(x, y, "b.")

```

```

13 plt.xlabel("x")
14 plt.ylabel("y")
15 plt.show()

```

Listing 66: Ajuste de una función exponencial usando el truco del logaritmo.

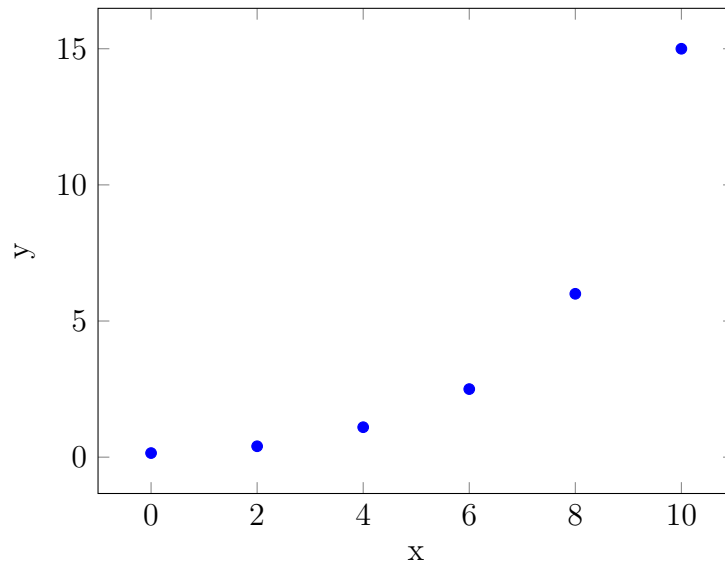


Figura 8: Datos generados para el ajuste exponencial.

Una vez aplicado el truco del logaritmo, podemos ajustar los datos.

```

1 In [4]: A = np.vstack([x, np.ones(len(x))]).T
2         beta, log_alpha = np.linalg.lstsq(A, np.log(y), rcond
3         =None)[0]
4         alpha = np.exp(log_alpha)
5         print(f"alpha={alpha}, beta={beta}")
6 alpha=0.13973103064296616, beta=0.26307478591152406
7 In [5]: # Let's have a look of the data
8         plt.figure(figsize=(10, 8))
9         plt.plot(x, y, "b.")
10        plt.plot(x, alpha*np.exp(beta*x), "r")
11        plt.xlabel("x")
12        plt.ylabel("y")
13        plt.show()

```

Listing 67: Ajuste linealizado y transformación inversa.

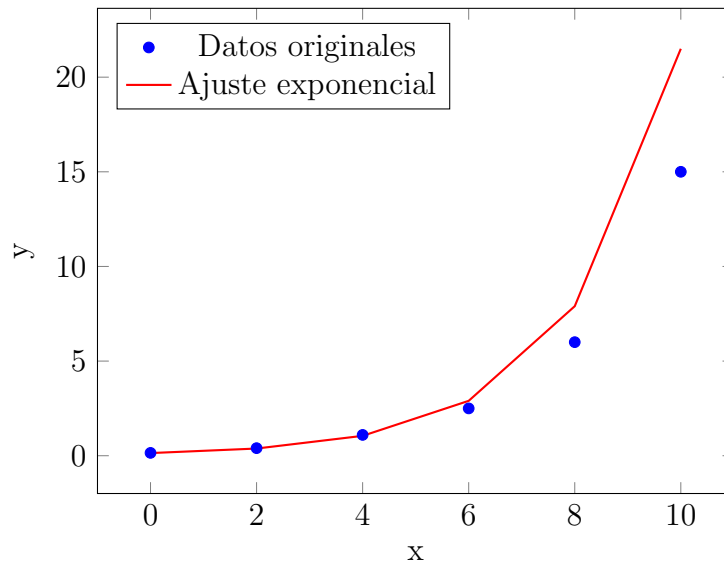


Figura 9: Datos y curva de ajuste exponencial obtenida mediante el truco del logaritmo.

Trucos Logarítmicos para Funciones de Potencia

El caso de la función de potencia es muy similar. Supongamos que tenemos una función de la forma $\hat{y}(x) = bx^m$ y datos para x e y . Entonces podemos convertir esta función en una forma lineal tomando logaritmo a ambos lados: $\log(\hat{y}(x)) = m \log(x) + \log(b)$, resolviendo esta función como una regresión lineal. Dado que es muy similar al ejemplo anterior, no dedicaremos más tiempo a esto.

Regresión Polinomial

También podemos usar polinomios y mínimos cuadrados para ajustar una función no lineal. Anteriormente, teníamos nuestras funciones todas en forma lineal, es decir, $y = ax + b$. Pero los "polinomios" son funciones con la siguiente forma:

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_2 x^2 + a_1 x^1 + a_0$$

donde $a_n, a_{n-1}, \dots, a_2, a_1, a_0$ son los coeficientes reales, y n , un entero no negativo, es el **orden** o **grado** del polinomio. Si tenemos un conjunto de puntos de datos, podemos usar diferentes órdenes de polinomios para ajustarlos. Los

coeficientes de los polinomios pueden estimarse usando el método de mínimos cuadrados como antes, es decir, minimizando el error entre los datos reales y los resultados del ajuste polinomial.

En Python, podemos usar `numpy.polyfit` para obtener los coeficientes de polinomios de diferentes órdenes con mínimos cuadrados. Con los coeficientes, podemos obtener los valores específicos usando `numpy.polyval`. A continuación, se muestra un ejemplo de cómo hacer esto en Python.

```
1 In [6]: x_d = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8])
2         y_d = np.array([0, 0.8, 0.9, 0.1, -0.6, -0.8, -1,
3             -0.9, -0.4])
4
5         plt.figure(figsize=(12, 8))
6         for i in range(1, 7):
7             # get the polynomial coefficients
8             coefs = np.polyfit(x_d, y_d, i)
9             plt.subplot(2, 3, i)
10            plt.plot(x_d, y_d, "o")
11            # evaluate the values for a polynomial
12            plt.plot(x_d, np.polyval(coefs, x_d))
13            plt.title(f"Polynomial order {i}")
14        plt.tight_layout()
15        plt.show()
```

Listing 68: Ajuste polinomial de diferentes órdenes.

[Figura simulada con 6 subgráficas de ajustes polinomiales de órdenes 1 a 6]

Figura 10: Ajuste polinomial de diferentes órdenes a los mismos datos.

La figura anterior muestra que podemos usar diferentes órdenes de polinomios para ajustar los mismos datos. Cuanto mayor es el orden, más flexible es la curva para ajustarse a los datos. Pero qué orden usar no es una cuestión simple; depende de los problemas específicos en ciencia e ingeniería.

Usando `optimize.curve_fit` de SciPy

Podemos usar la función `curve_fit` para ajustar cualquier forma de función y estimar sus parámetros. Podemos resolver el problema anterior usando la función `curve_fit` de la siguiente manera:

```

1 In [7]: # let's define the function form
2         def func(x, a, b):
3             y = a * np.exp(b * x)
4             return y
5
6         alpha, beta = optimize.curve_fit(func, xdata=x, ydata
7         =y)[0]
8         print(f"alpha={alpha}, beta={beta}")
9         alpha=0.12663549356730994, beta=0.27760076897453045
10
11 In [8]: # Let's have a look of the data
12         plt.figure(figsize=(10, 8))
13         plt.plot(x, y, "b.")
14         plt.plot(x, alpha * np.exp(beta * x), "r")
15         plt.xlabel("x")
16         plt.ylabel("y")
17         plt.show()

```

Listing 69: Ajuste de una función exponencial con `curve_fit`.

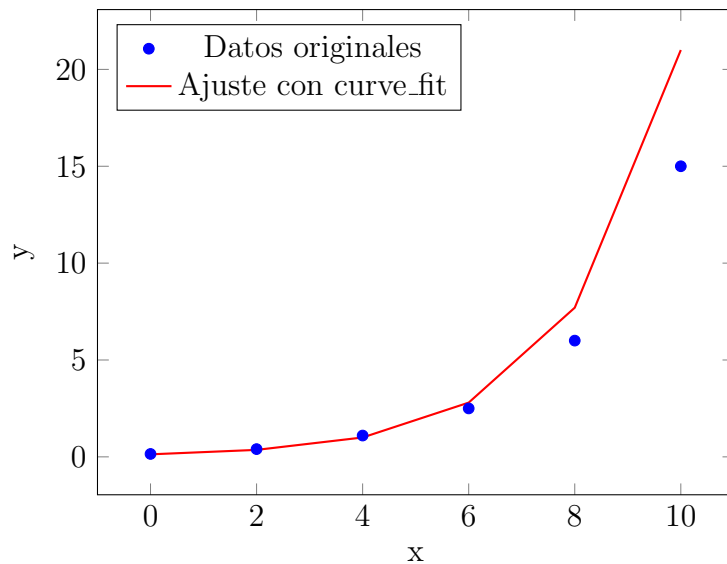


Figura 11: Datos y curva de ajuste exponencial obtenida con `curve_fit`.

Resumen y Problemas

Resumen

1. Los modelos matemáticos se utilizan para comprender, predecir y controlar sistemas de ingeniería. Estos modelos constan de parámetros que rigen la forma en que se comporta el modelo.
2. Dado un conjunto de datos experimentales, una regresión por mínimos cuadrados es un método para encontrar un conjunto de parámetros del modelo que se ajuste bien a los datos. Es decir, minimiza el error al cuadrado entre el modelo, o función de estimación, y los puntos de datos.
3. En una regresión por mínimos cuadrados lineal, la función de estimación debe ser una combinación lineal de funciones base linealmente independientes.
4. El conjunto de parámetros β puede determinarse mediante la ecuación de mínimos cuadrados $\beta = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{Y}$, donde la j -ésima columna de $\mathbf{A}$ es la j -ésima función base evaluada en cada punto de datos $\mathbf{X}$.
5. Para estimar una función no lineal, la transformamos en una función de estimación lineal o usamos directamente una regresión por mínimos cuadrados para resolver la función no lineal usando `curve_fit` de SciPy.

Problemas y Soluciones

1. **Problema 1:** Repite la derivación de cálculo multivariable de la fórmula de regresión por mínimos cuadrados para una función de estimación $\hat{y}(x) = ax^2 + bx + c$.

Solución: La función de error es $E(a, b, c) = \sum_{i=1}^m (ax_i^2 + bx_i + c - y_i)^2$. Las derivadas parciales se igualan a cero:

$$\frac{\partial E}{\partial a} = 2 \sum_{i=1}^m (ax_i^2 + bx_i + c - y_i)x_i^2 = 0,$$

$$\frac{\partial E}{\partial b} = 2 \sum_{i=1}^m (ax_i^2 + bx_i + c - y_i)x_i = 0,$$

$$\frac{\partial E}{\partial c} = 2 \sum_{i=1}^m (ax_i^2 + bx_i + c - y_i) = 0.$$

Esto da un sistema lineal de 3 ecuaciones con 3 incógnitas.

2. Problema 2: Función `my_ls_params` para regresión lineal general.

Solución: Construimos la matriz **A** evaluando cada función base en los puntos **x** y luego resolvemos la ecuación normal.

```

1 import numpy as np
2
3 def my_ls_params(f, x, y):
4     A = np.array([fi(x) for fi in f]).T
5     beta = np.linalg.lstsq(A, y, rcond=None)[0]
6     return beta
7
8 # Ejemplo de uso
9 x = np.linspace(0, 1, 10)
10 y = 2*x + 1 + 0.1*np.random.randn(10)
11 f = [lambda x: x, lambda x: 1]
12 beta = my_ls_params(f, x, y)
13 print(beta)
14
```

Listing 70: Solución Problema 2

3. Problema 3: Función `my_func_fit` para $\hat{y}(x) = \alpha x^\beta$.

Solución: Aplicamos logaritmos: $\log y = \log \alpha + \beta \log x$. Realizamos la regresión lineal sobre los datos transformados.

```

1 import numpy as np
2
3 def my_func_fit(x, y):
4     log_x = np.log(x)
5     log_y = np.log(y)
6     A = np.vstack([log_x, np.ones(len(log_x))]).T
7     beta, log_alpha = np.linalg.lstsq(A, log_y, rcond=
8     None)[0]
9     alpha = np.exp(log_alpha)
10    return alpha, beta
11
12 # Ejemplo
13 x = np.array([1, 2, 3, 4])

```

```

13 y = 2 * x**1.5
14 alpha, beta = my_func_fit(x, y)
15 print(f"alpha={alpha}, beta={beta}")
16

```

Listing 71: Solución Problema 3

4. **Problema 4:** Error total para un polinomio cúbico.

Solución: El error total es $E = \sum_{i=1}^4 (ax_i^3 + bx_i^2 + cx_i + d - y_i)^2$. Si colocamos un quinto punto que ya esté en el polinomio (es decir, que satisfaga la ecuación cúbica), entonces ese punto no añadirá error adicional.

5. **Problema 5:** Función `my_lin_regression`.

Solución: Similar al Problema 2, pero aquí se espera que se añada un término de sesgo (intercepto) automáticamente si no está incluido en las funciones base.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def my_lin_regression(f, x, y):
5     # Incluir un termino de sesgo (constante) si no esta
6     # presente
7     has_constant = any(np.all(fi(x) == 1) for fi in f)
8     basis_funcs = list(f)
9     if not has_constant:
10         basis_funcs.append(lambda x: np.ones_like(x))
11
12     A = np.array([fi(x) for fi in basis_funcs]).T
13     beta = np.linalg.lstsq(A, y, rcond=None)[0]
14     return beta
15
16 # Test case
17 x = np.linspace(0, 2*np.pi, 1000)
18 y = 3*np.sin(x) - 2*np.cos(x) + np.random.random(len(x))
19 f = [np.sin, np.cos]
20 beta = my_lin_regression(f, x, y)
21 print("Beta (coefs de sin, cos, constante):", beta)
22
23 plt.figure(figsize=(10, 8))
24 plt.plot(x, y, "b.", label="data")

```

```

24 y_fit = beta[0]*np.sin(x) + beta[1]*np.cos(x) + beta[2]
25 plt.plot(x, y_fit, "r", label="regression")
26 plt.xlabel("x")
27 plt.ylabel("y")
28 plt.legend()
29 plt.show()
30

```

Listing 72: Solución Problema 5

6. Problema 6: Función my_exp_regression.

Solución: Aplicamos el truco del logaritmo a la función exponencial $\hat{y}(x) = \alpha e^{\beta x}$.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def my_exp_regression(x, y):
5     log_y = np.log(y)
6     A = np.vstack([x, np.ones(len(x))]).T
7     beta, log_alpha = np.linalg.lstsq(A, log_y, rcond=
8     None)[0]
9     alpha = np.exp(log_alpha)
10    return alpha, beta
11
12 # Test case
13 x = np.linspace(0, 1, 1000)
14 y = 2 * np.exp(-0.5 * x) + 0.25 * np.random.random(len(x))
15
16 alpha, beta = my_exp_regression(x, y)
17 print(f"alpha={alpha}, beta={beta}")
18
19 plt.figure(figsize=(10, 8))
20 plt.plot(x, y, "b.", label="data")
21 plt.plot(x, alpha * np.exp(beta * x), "r", label="
22     regression")
23 plt.xlabel("x")
24 plt.ylabel("y")
25 plt.legend()
26 plt.title("Least Square Regression on Exponential Model")
27 plt.show()
28

```

Listing 73: Solución Problema 6

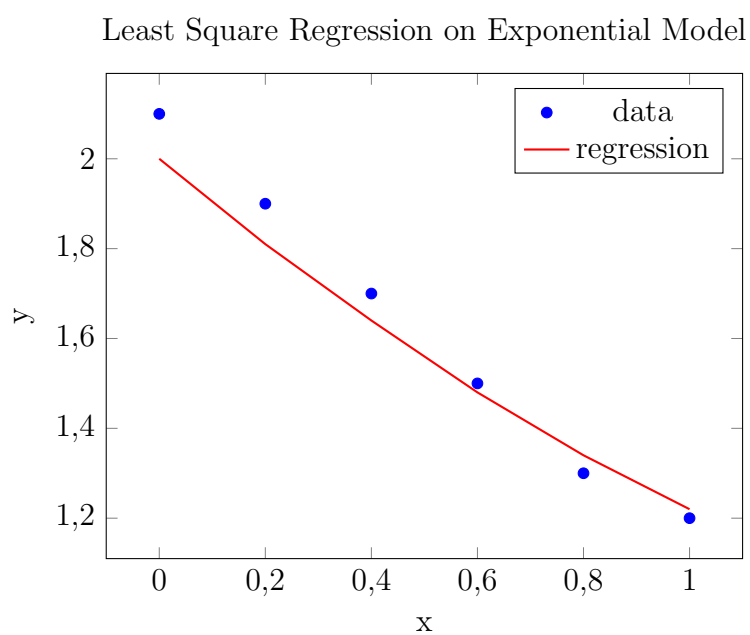


Figura 12: Ajuste exponencial obtenido con `my_exp_regression`.

REGRESIÓN POR MÍNIMOS CUADRADOS

Planteamiento del Problema de Regresión por Mínimos Cuadrados

Dado un conjunto de puntos de datos independientes x_i y puntos de datos dependientes y_i , con $i = 1, \dots, m$, nos gustaría encontrar una **función de estimación**, $\hat{y}(x)$, que describa los datos con la mayor precisión posible. Observa que $\hat{y}$ puede ser una función de varias variables, pero para esta discusión, restringimos el dominio de $\hat{y}$ a una sola variable. En la regresión por mínimos cuadrados, la función de estimación debe ser una combinación lineal de **funciones base**, $f_i(x)$. Es decir, la función de estimación debe ser de la forma

$$\hat{y}(x) = \sum_{i=1}^n \alpha_i f_i(x).$$

Los escalares α_i se denominan **parámetros** de la función de estimación, y cada función base debe ser linealmente independiente de las demás. En otras palabras, en el *espacio funcional* apropiado, ninguna función base debe poder expresarse como una combinación lineal de las otras funciones. Observa que, en general, hay significativamente más puntos de datos, m , que funciones base, n (es decir, $m \gg n$).

```
1 In []: # La relacion entre la fuerza, F, y el desplazamiento,
        x, puede describirse mediante la funcion F(x) = kx,
2 # donde k es la rigidez del resorte. La unica funcion base es
        f1(x) = x y el parametro del modelo a determinar es 1 =
        k.
```

Listing 74: Creando una función de estimación para un resorte lineal.

El objetivo de la regresión por mínimos cuadrados es encontrar los parámetros de la función de estimación que minimicen el **error cuadrático total**, E , definido por

$$E = \sum_{i=1}^m (\hat{y}(x_i) - y_i)^2.$$

Los errores individuales o **residuos** se definen como $e_i = \hat{y}(x_i) - y_i$. Si $\mathbf{e}$ es el vector que contiene todos los errores individuales, entonces también estamos tratando de minimizar $E = \|\mathbf{e}\|_2^2$, que es la norma L_2 definida en el capítulo anterior.

En las siguientes dos secciones, derivaremos el método de mínimos cuadrados para encontrar los parámetros deseados. La primera derivación proviene del álgebra lineal, y la segunda del cálculo multivariable. Aunque son derivaciones diferentes, conducen a la misma fórmula de mínimos cuadrados. Eres libre de centrarte en la sección con la que te sientas más cómodo.

Derivación de la Regresión por Mínimos Cuadrados (Álgebra Lineal)

Primero, enumeramos la estimación de los datos en cada punto de datos x_i como

$$\begin{aligned}\hat{y}(x_1) &= \alpha_1 f_1(x_1) + \alpha_2 f_2(x_1) + \cdots + \alpha_n f_n(x_1), \\ \hat{y}(x_2) &= \alpha_1 f_1(x_2) + \alpha_2 f_2(x_2) + \cdots + \alpha_n f_n(x_2), \\ &\vdots \\ \hat{y}(x_m) &= \alpha_1 f_1(x_m) + \alpha_2 f_2(x_m) + \cdots + \alpha_n f_n(x_m).\end{aligned}$$

Sea $\mathbf{X} \in \mathbb{R}^n$ un vector columna tal que el i -ésimo elemento de $\mathbf{X}$ contiene el valor del i -ésimo punto de datos x_i . Sea $\hat{\mathbf{Y}}$ un vector columna con elementos $\hat{Y}_i = \hat{y}(x_i)$, $\boldsymbol{\beta}$ un vector columna tal que $\beta_i = \alpha_i$, $\mathbf{F}_i(\mathbf{X})$ una función que devuelve un vector columna de $f_i(x)$ calculado en cada elemento de $\mathbf{X}$, y $\mathbf{A}$ una matriz de $m \times n$ tal que la i -ésima columna de $\mathbf{A}$ es $\mathbf{F}_i(\mathbf{X})$. Dada esta notación, el sistema de ecuaciones anterior se convierte en $\hat{\mathbf{Y}} = \mathbf{A}\boldsymbol{\beta}$.

Ahora, si $\mathbf{Y}$ es un vector columna tal que $Y_i = y_i$, el error cuadrático total está dado por $E = \|\hat{\mathbf{Y}} - \mathbf{Y}\|_2^2$. Verifica esto sustituyendo la definición de la norma L_2 . Dado que queremos que E sea lo más pequeño posible y las normas son una medida de distancia, esta expresión anterior es equivalente a

decir que queremos que $\hat{\mathbf{Y}}$ y $\mathbf{Y}$ estén lo más cerca posible”. Observa que, en general, $\mathbf{Y}$ no estará en el rango de $\mathbf{A}$ y por lo tanto $E > 0$.

Considera la siguiente representación simplificada del rango de $\mathbf{A}$; ver Fig. 13. Observa que esto no es una gráfica de los puntos de datos (x_i, y_i) .

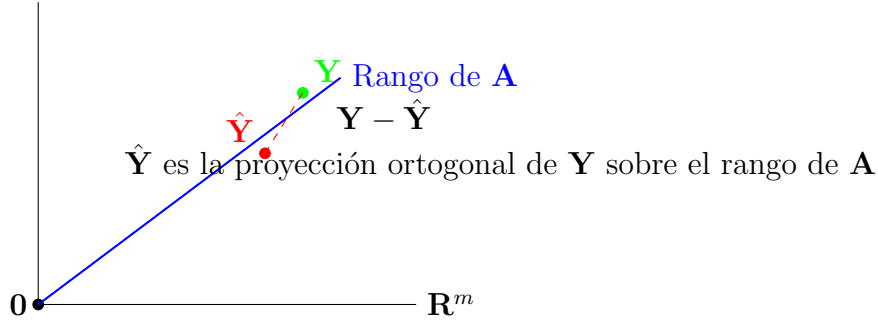


Figura 13: Ilustración de la proyección L_2 de $\mathbf{Y}$ sobre el rango de $\mathbf{A}$.

Por observación, el vector en el rango de $\mathbf{A}$, es decir, $\hat{\mathbf{Y}}$, que está más cerca de $\mathbf{Y}$ es aquel que es perpendicular a $\mathbf{Y} - \hat{\mathbf{Y}}$. Por lo tanto, queremos un vector $\mathbf{Y} - \hat{\mathbf{Y}}$ que sea perpendicular al vector $\hat{\mathbf{Y}}$. Recordemos del álgebra lineal que dos vectores son perpendiculares u **ortogonales** si su producto punto es 0. Notando que el producto punto entre dos vectores, $\mathbf{v}$ y $\mathbf{w}$, puede escribirse como $\text{dot}(\mathbf{v}, \mathbf{w}) = \mathbf{v}^T \mathbf{w}$, podemos afirmar que $\hat{\mathbf{Y}}$ y $\mathbf{Y} - \hat{\mathbf{Y}}$ son perpendiculares si $\text{dot}(\hat{\mathbf{Y}}, \mathbf{Y} - \hat{\mathbf{Y}}) = 0$; por lo tanto, $\hat{\mathbf{Y}}^T (\mathbf{Y} - \hat{\mathbf{Y}}) = 0$, que es equivalente a $(\mathbf{A}\beta)^T (\mathbf{Y} - \mathbf{A}\beta) = 0$.

Observando que para dos matrices $\mathbf{A}$ y $\mathbf{B}$ tenemos $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$ y usando las propiedades distributivas de la multiplicación de vectores, esto es equivalente a $\beta^T \mathbf{A}^T \mathbf{Y} - \beta^T \mathbf{A}^T \mathbf{A} \beta = \beta^T (\mathbf{A}^T \mathbf{Y} - \mathbf{A}^T \mathbf{A} \beta) = 0$. La solución $\beta = \mathbf{0}$ es una solución trivial, así que usamos $\mathbf{A}^T \mathbf{Y} - \mathbf{A}^T \mathbf{A} \beta = 0$ para encontrar una solución más interesante. Resolviendo esta ecuación para β se obtiene la **fórmula de regresión por mínimos cuadrados**:

$$\boxed{\beta = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{Y}}.$$

Observa que $(\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T$ se llama la **pseudo-inversa** de $\mathbf{A}$ y existe cuando $m > n$ y $\mathbf{A}$ tiene columnas linealmente independientes. Demostrar la invertibilidad de $(\mathbf{A}^T \mathbf{A})$ está fuera del alcance de este libro, pero siempre es invertible excepto en algunos casos específicos.

Derivación de la Regresión por Mínimos Cuadrados (Cálculo Multivariable)

Recordemos que el error total para m puntos de datos y n funciones base es:

$$E = \sum_{i=1}^m e_i^2 = \sum_{i=1}^m (\hat{y}(x_i) - y_i)^2 = \sum_{i=1}^m \left(\sum_{j=1}^n \alpha_j f_j(x_i) - y_i \right)^2,$$

que es un paraboloide n -dimensional en α_k . Del cálculo, sabemos que el mínimo de un paraboloide es donde todas las derivadas parciales son iguales a cero. Tomando la derivada parcial de E con respecto a la variable α_k (recuerda que en este caso los parámetros son nuestras variables) e igualando estas derivadas a 0, y luego resolviendo el sistema de ecuaciones para los α_k debería dar los resultados correctos.

Calculando la derivada parcial con respecto a α_k e igualándola a cero se obtiene

$$\frac{\partial E}{\partial \alpha_k} = \sum_{i=1}^m 2 \left(\sum_{j=1}^n \alpha_j f_j(x_i) - y_i \right) f_k(x_i) = 0.$$

Con algo de reordenamiento, la expresión anterior puede manipularse de la siguiente manera:

$$\sum_{i=1}^m \sum_{j=1}^n \alpha_j f_j(x_i) f_k(x_i) - \sum_{i=1}^m y_i f_k(x_i) = 0,$$

y tras una mayor reorganización (aprovechamos el hecho de que la suma conmuta), el resultado es

$$\sum_{j=1}^n \alpha_j \sum_{i=1}^m f_j(x_i) f_k(x_i) = \sum_{i=1}^m y_i f_k(x_i).$$

Ahora, sea $\mathbf{X}$ un vector columna tal que el i -ésimo elemento de $\mathbf{X}$ es x_i y $\mathbf{Y}$ construido de manera similar, y sea $\mathbf{F}_j(\mathbf{X})$ un vector columna tal que el i -ésimo elemento de $\mathbf{F}_j(\mathbf{X})$ es $f_j(x_i)$. Usando esta notación, la expresión

anterior puede reescribirse en notación vectorial como

$$[\mathbf{F}_k(\mathbf{X})^T \mathbf{F}_1(\mathbf{X}), \mathbf{F}_k(\mathbf{X})^T \mathbf{F}_2(\mathbf{X}), \dots, \mathbf{F}_k(\mathbf{X})^T \mathbf{F}_j(\mathbf{X}), \dots, \mathbf{F}_k(\mathbf{X})^T \mathbf{F}_n(\mathbf{X})] \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_j \\ \vdots \\ \alpha_n \end{bmatrix} = \mathbf{F}_k(\mathbf{X})^T \mathbf{Y}.$$

Si repetimos esta ecuación para cada k , obtenemos el siguiente sistema de ecuaciones lineales en forma matricial:

$$\begin{bmatrix} \mathbf{F}_1(\mathbf{X})^T \mathbf{F}_1(\mathbf{X}) & \mathbf{F}_1(\mathbf{X})^T \mathbf{F}_2(\mathbf{X}) & \dots & \mathbf{F}_1(\mathbf{X})^T \mathbf{F}_n(\mathbf{X}) \\ \mathbf{F}_2(\mathbf{X})^T \mathbf{F}_1(\mathbf{X}) & \mathbf{F}_2(\mathbf{X})^T \mathbf{F}_2(\mathbf{X}) & \dots & \mathbf{F}_2(\mathbf{X})^T \mathbf{F}_n(\mathbf{X}) \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{F}_n(\mathbf{X})^T \mathbf{F}_1(\mathbf{X}) & \mathbf{F}_n(\mathbf{X})^T \mathbf{F}_2(\mathbf{X}) & \dots & \mathbf{F}_n(\mathbf{X})^T \mathbf{F}_n(\mathbf{X}) \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_n \end{bmatrix} = \begin{bmatrix} \mathbf{F}_1(\mathbf{X})^T \mathbf{Y} \\ \mathbf{F}_2(\mathbf{X})^T \mathbf{Y} \\ \vdots \\ \mathbf{F}_n(\mathbf{X})^T \mathbf{Y} \end{bmatrix}.$$

Si definimos $\mathbf{A} = [\mathbf{F}_1(\mathbf{X}), \mathbf{F}_2(\mathbf{X}), \dots, \mathbf{F}_n(\mathbf{X})]$ y $\boldsymbol{\beta}$ como un vector columna tal que el j -ésimo elemento de $\boldsymbol{\beta}$ es α_j , entonces el sistema de ecuaciones anterior se convierte en

$$\mathbf{A}^T \mathbf{A} \boldsymbol{\beta} = \mathbf{A}^T \mathbf{Y},$$

y luego resolviendo esta ecuación matricial para $\boldsymbol{\beta}$ se obtiene $\boldsymbol{\beta} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{Y}$, que es exactamente la misma fórmula que en la derivación anterior.

Regresión por Mínimos Cuadrados en Python

Recordemos que enumerar la estimación de los datos en cada punto de datos x_i nos dará el siguiente sistema de ecuaciones:

$$\begin{aligned} \hat{y}(x_1) &= \alpha_1 f_1(x_1) + \alpha_2 f_2(x_1) + \dots + \alpha_n f_n(x_1), \\ \hat{y}(x_2) &= \alpha_1 f_1(x_2) + \alpha_2 f_2(x_2) + \dots + \alpha_n f_n(x_2), \\ &\vdots \\ \hat{y}(x_m) &= \alpha_1 f_1(x_m) + \alpha_2 f_2(x_m) + \dots + \alpha_n f_n(x_m). \end{aligned}$$

Si los datos fueran absolutamente perfectos (es decir, sin ruido), entonces la función de estimación pasaría por todos los puntos de datos, resultando en

el siguiente sistema de ecuaciones:

$$\begin{aligned} y_1 &= \alpha_1 f_1(x_1) + \alpha_2 f_2(x_1) + \cdots + \alpha_n f_n(x_1), \\ y_2 &= \alpha_1 f_1(x_2) + \alpha_2 f_2(x_2) + \cdots + \alpha_n f_n(x_2), \\ &\vdots \\ y_m &= \alpha_1 f_1(x_m) + \alpha_2 f_2(x_m) + \cdots + \alpha_n f_n(x_m). \end{aligned}$$

Si tomamos $\mathbf{A}$ como se definió anteriormente, esto daría como resultado la ecuación matricial

$$\mathbf{Y} = \mathbf{A}\boldsymbol{\beta}.$$

Debido a que los datos no son perfectos, no habrá una función de estimación que pueda pasar por todos los puntos de datos, y este sistema no tendrá solución. Por lo tanto, necesitamos usar la regresión por mínimos cuadrados que derivamos en las dos secciones anteriores para obtener una solución:

$$\boldsymbol{\beta} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{Y}.$$

```

1 In [1]: import numpy as np
2         from scipy import optimize
3         import matplotlib.pyplot as plt
4         plt.style.use("seaborn-poster")
5
6 In [2]: # generate x and y
7         x = np.linspace(0, 1, 101)
8         y = 1 + x + x * np.random.random(len(x))
9
10 In [3]: # assemble matrix A
11         A = np.vstack([x, np.ones(len(x))]).T
12         # turn y into a column vector
13         y = y[:, np.newaxis]
14
15 In [4]: # Direct least squares regression
16         alpha = np.dot((np.dot(np.linalg.inv(np.dot(A.T, A)),
17                                A.T)), y)
18         print(alpha)
19         [[1.459573 ]
20          [1.02952189]]
21
22 In [5]: # plot the results
23         plt.figure(figsize=(10, 8))
24         plt.plot(x, y, "b.")

```

```
24 plt.plot(x, alpha[0]*x + alpha[1], "r")
25 plt.xlabel("x")
26 plt.ylabel("y")
27 plt.show()
```

Listing 75: Regresión lineal simple con mínimos cuadrados.

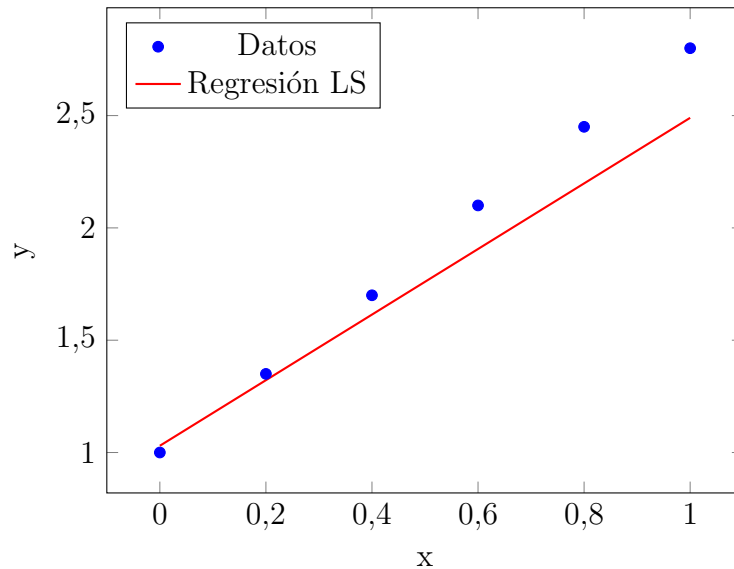


Figura 14: Puntos de datos y recta de regresión por mínimos cuadrados.

Python tiene varios paquetes y funciones que pueden realizar una regresión por mínimos cuadrados. Estos incluyen NumPy, SciPy, statsmodels y sklearn. A continuación, se muestran varios ejemplos de tales aplicaciones. Siéntete libre de elegir el que más te guste.

Usando el Método de la Inversa Directa

Ya se mostró en el Listado [75](#).

Usando la Pseudo-Inversa

Mencionamos anteriormente que la matriz $(\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T$ se llama la pseudo-inversa, por lo tanto, podemos usar la función `pinv` en NumPy para calcularla directamente.

```
1 In [6]: pinv = np.linalg.pinv(A)
2         alpha = pinv.dot(y)
3         print(alpha)
4 [[1.459573 ]
5  [1.02952189]]
```

Listing 76: Usando la pseudo-inversa.

Usando `numpy.linalg.lstsq`

NumPy ya ha implementado los métodos de mínimos cuadrados, por lo que podemos simplemente llamar a la función para obtener una solución. La función devolverá más datos que la solución en sí; consulta la documentación para más detalles.

```
1 In [7]: alpha = np.linalg.lstsq(A, y, rcond=None)[0]
2         print(alpha)
3 [[1.459573 ]
4  [1.02952189]]
```

Listing 77: Usando `numpy.linalg.lstsq`.

Usando `optimize.curve_fit` de SciPy

Esta función de SciPy es muy potente. No solo es adecuada para funciones lineales, sino también para muchas formas de funciones diferentes, como funciones no lineales. Aquí solo mostraremos el ejemplo lineal de arriba. Observa que, al usar esta función, no necesitamos convertir **y** en un vector columna.

```
1 In [8]: # generate x and y
2         x = np.linspace(0, 1, 101)
3         y = 1 + x + x * np.random.random(len(x))
4
5 In [9]: def func(x, a, b):
6         y = a*x + b
7         return y
8
9         alpha = optimize.curve_fit(func, xdata=x, ydata=y)[0]
10        print(alpha)
11 [1.44331612 1.0396133 ]
```

Listing 78: Usando `scipy.optimize.curve_fit`.

Regresión por Mínimos Cuadrados para Funciones No Lineales

Una regresión por mínimos cuadrados requiere que la función de estimación sea una combinación lineal de funciones base. Hay algunas funciones que no pueden ponerse en esta forma, pero donde una regresión por mínimos cuadrados sigue siendo apropiada.

A continuación, se presentan varias formas de tratar con funciones no lineales.

- Podemos lograr esto aprovechando las propiedades de los logaritmos y transformar la función no lineal en una función lineal.
- Podemos usar la función `curve_fit` de SciPy para estimar directamente los parámetros de la función no lineal usando mínimos cuadrados.

Trucos Logarítmicos para Funciones Exponenciales

Supongamos que tienes una función de la forma $\hat{y}(x) = \alpha e^{\beta x}$ y datos para x e y , y que desees realizar una regresión por mínimos cuadrados para encontrar α y β . Claramente, el conjunto anterior de funciones base (lineales) sería inapropiado para describir $\hat{y}(x)$; sin embargo, si tomamos el logaritmo de ambos lados, obtenemos $\log(\hat{y}(x)) = \log(\alpha) + \beta x$. Ahora, vemos que si definimos $\tilde{y}(x) = \log(\hat{y}(x))$ y $\tilde{\alpha} = \log(\alpha)$, entonces $\tilde{y}(x) = \tilde{\alpha} + \beta x$. Por lo tanto, podemos realizar una regresión por mínimos cuadrados sobre la expresión linealizada para encontrar $\tilde{y}(x)$, $\tilde{\alpha}$ y β , y luego recuperar α usando la expresión $\alpha = e^{\tilde{\alpha}}$.

Para el siguiente ejemplo, generaremos datos usando $\alpha = 0,1$ y $\beta = 0,3$.

```

1 In [1]: import numpy as np
2         from scipy import optimize
3         import matplotlib.pyplot as plt
4         plt.style.use("seaborn-poster")
5
6 In [2]: # let's generate x and y, and add some noise into y
7         x = np.linspace(0, 10, 101)
8         y = 0.1*np.exp(0.3*x) + 0.1*np.random.random(len(x))
9
10 In [3]: # Let's have a look of the data
11         plt.figure(figsize=(10, 8))
12         plt.plot(x, y, "b.")

```

```

13 plt.xlabel("x")
14 plt.ylabel("y")
15 plt.show()

```

Listing 79: Ajuste de una función exponencial usando el truco del logaritmo.

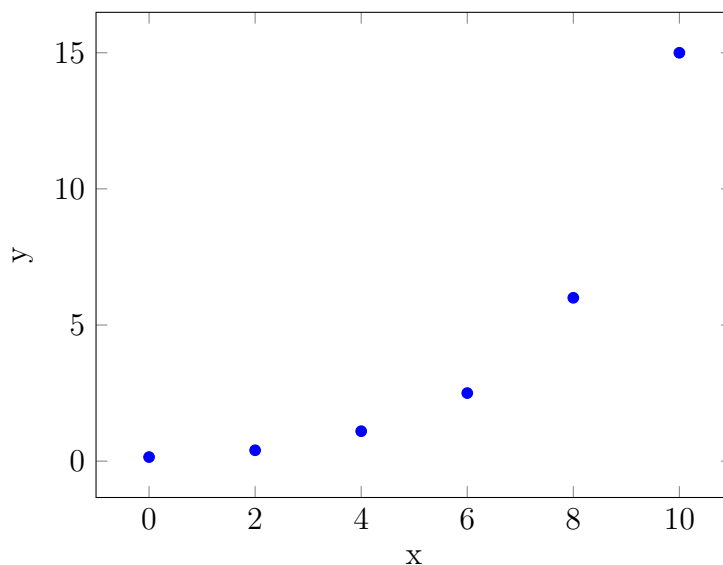


Figura 15: Datos generados para el ajuste exponencial.

Una vez aplicado el truco del logaritmo, podemos ajustar los datos.

```

1 In [4]: A = np.vstack([x, np.ones(len(x))]).T
2         beta, log_alpha = np.linalg.lstsq(A, np.log(y), rcond
3         =None)[0]
4         alpha = np.exp(log_alpha)
5         print(f"alpha={alpha}, beta={beta}")
6         alpha=0.13973103064296616, beta=0.26307478591152406
7 In [5]: # Let's have a look of the data
8         plt.figure(figsize=(10, 8))
9         plt.plot(x, y, "b.")
10        plt.plot(x, alpha*np.exp(beta*x), "r")
11        plt.xlabel("x")
12        plt.ylabel("y")
13        plt.show()

```

Listing 80: Ajuste linealizado y transformación inversa.

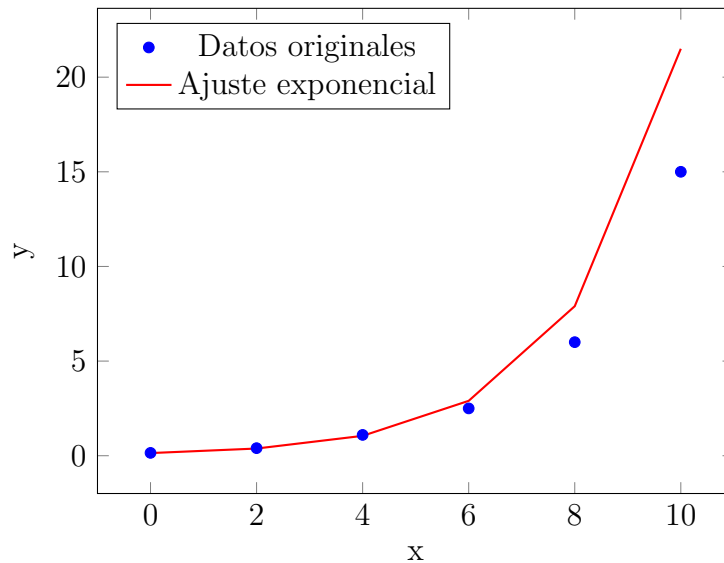


Figura 16: Datos y curva de ajuste exponencial obtenida mediante el truco del logaritmo.

Trucos Logarítmicos para Funciones de Potencia

El caso de la función de potencia es muy similar. Supongamos que tenemos una función de la forma $\hat{y}(x) = bx^m$ y datos para x e y . Entonces podemos convertir esta función en una forma lineal tomando logaritmo a ambos lados: $\log(\hat{y}(x)) = m \log(x) + \log(b)$, resolviendo esta función como una regresión lineal. Dado que es muy similar al ejemplo anterior, no dedicaremos más tiempo a esto.

Regresión Polinomial

También podemos usar polinomios y mínimos cuadrados para ajustar una función no lineal. Anteriormente, teníamos nuestras funciones todas en forma lineal, es decir, $y = ax + b$. Pero los "polinomios" son funciones con la siguiente forma:

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_2 x^2 + a_1 x^1 + a_0$$

donde $a_n, a_{n-1}, \dots, a_2, a_1, a_0$ son los coeficientes reales, y n , un entero no negativo, es el **orden** o **grado** del polinomio. Si tenemos un conjunto de puntos de datos, podemos usar diferentes órdenes de polinomios para ajustarlos. Los

coeficientes de los polinomios pueden estimarse usando el método de mínimos cuadrados como antes, es decir, minimizando el error entre los datos reales y los resultados del ajuste polinomial.

En Python, podemos usar `numpy.polyfit` para obtener los coeficientes de polinomios de diferentes órdenes con mínimos cuadrados. Con los coeficientes, podemos obtener los valores específicos usando `numpy.polyval`. A continuación, se muestra un ejemplo de cómo hacer esto en Python.

```

1 In [6]: x_d = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8])
2         y_d = np.array([0, 0.8, 0.9, 0.1, -0.6, -0.8, -1,
3             -0.9, -0.4])
4
5         plt.figure(figsize=(12, 8))
6         for i in range(1, 7):
7             # get the polynomial coefficients
8             coefs = np.polyfit(x_d, y_d, i)
9             plt.subplot(2, 3, i)
10            plt.plot(x_d, y_d, "o")
11            # evaluate the values for a polynomial
12            plt.plot(x_d, np.polyval(coefs, x_d))
13            plt.title(f"Polynomial order {i}")
14        plt.tight_layout()
15        plt.show()

```

Listing 81: Ajuste polinomial de diferentes órdenes.

[Figura simulada con 6 subgráficas de ajustes polinomiales de órdenes 1 a 6]

Figura 17: Ajuste polinomial de diferentes órdenes a los mismos datos.

La figura anterior muestra que podemos usar diferentes órdenes de polinomios para ajustar los mismos datos. Cuanto mayor es el orden, más flexible es la curva para ajustarse a los datos. Pero qué orden usar no es una cuestión simple; depende de los problemas específicos en ciencia e ingeniería.

Usando `optimize.curve_fit` de SciPy

Podemos usar la función `curve_fit` para ajustar cualquier forma de función y estimar sus parámetros. Podemos resolver el problema anterior usando la función `curve_fit` de la siguiente manera:

```

1 In [7]: # let's define the function form
2         def func(x, a, b):
3             y = a * np.exp(b * x)
4             return y
5
6         alpha, beta = optimize.curve_fit(func, xdata=x, ydata
7         =y)[0]
8         print(f"alpha={alpha}, beta={beta}")
9         alpha=0.12663549356730994, beta=0.27760076897453045
10
11 In [8]: # Let's have a look of the data
12         plt.figure(figsize=(10, 8))
13         plt.plot(x, y, "b.")
14         plt.plot(x, alpha * np.exp(beta * x), "r")
15         plt.xlabel("x")
16         plt.ylabel("y")
17         plt.show()

```

Listing 82: Ajuste de una función exponencial con `curve_fit`.

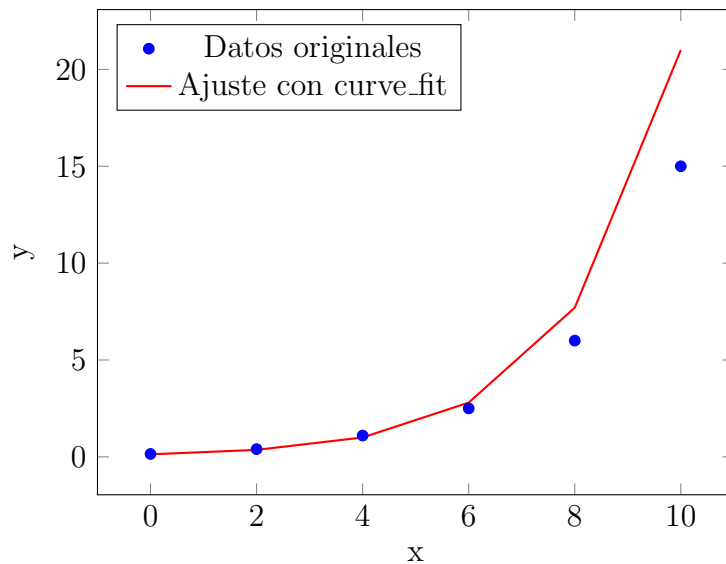


Figura 18: Datos y curva de ajuste exponencial obtenida con `curve_fit`.

Resumen y Problemas

Resumen

1. Los modelos matemáticos se utilizan para comprender, predecir y controlar sistemas de ingeniería. Estos modelos constan de parámetros que rigen la forma en que se comporta el modelo.
2. Dado un conjunto de datos experimentales, una regresión por mínimos cuadrados es un método para encontrar un conjunto de parámetros del modelo que se ajuste bien a los datos. Es decir, minimiza el error al cuadrado entre el modelo, o función de estimación, y los puntos de datos.
3. En una regresión por mínimos cuadrados lineal, la función de estimación debe ser una combinación lineal de funciones base linealmente independientes.
4. El conjunto de parámetros β puede determinarse mediante la ecuación de mínimos cuadrados $\beta = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{Y}$, donde la j -ésima columna de $\mathbf{A}$ es la j -ésima función base evaluada en cada punto de datos $\mathbf{X}$.
5. Para estimar una función no lineal, la transformamos en una función de estimación lineal o usamos directamente una regresión por mínimos cuadrados para resolver la función no lineal usando `curve_fit` de SciPy.

Problemas y Soluciones

1. **Problema 1:** Repite la derivación de cálculo multivariable de la fórmula de regresión por mínimos cuadrados para una función de estimación $\hat{y}(x) = ax^2 + bx + c$.

Solución: La función de error es $E(a, b, c) = \sum_{i=1}^m (ax_i^2 + bx_i + c - y_i)^2$. Las derivadas parciales se igualan a cero:

$$\frac{\partial E}{\partial a} = 2 \sum_{i=1}^m (ax_i^2 + bx_i + c - y_i)x_i^2 = 0,$$

$$\frac{\partial E}{\partial b} = 2 \sum_{i=1}^m (ax_i^2 + bx_i + c - y_i)x_i = 0,$$

$$\frac{\partial E}{\partial c} = 2 \sum_{i=1}^m (ax_i^2 + bx_i + c - y_i) = 0.$$

Esto da un sistema lineal de 3 ecuaciones con 3 incógnitas.

2. **Problema 2:** Función `my_ls_params` para regresión lineal general.

Solución: Construimos la matriz **A** evaluando cada función base en los puntos **x** y luego resolvemos la ecuación normal.

```

1 import numpy as np
2
3 def my_ls_params(f, x, y):
4     A = np.array([fi(x) for fi in f]).T
5     beta = np.linalg.lstsq(A, y, rcond=None)[0]
6     return beta
7
8 # Ejemplo de uso
9 x = np.linspace(0, 1, 10)
10 y = 2*x + 1 + 0.1*np.random.randn(10)
11 f = [lambda x: x, lambda x: 1]
12 beta = my_ls_params(f, x, y)
13 print(beta)
14
```

Listing 83: Solución Problema 2

3. **Problema 3:** Función `my_func_fit` para $\hat{y}(x) = \alpha x^\beta$.

Solución: Aplicamos logaritmos: $\log y = \log \alpha + \beta \log x$. Realizamos la regresión lineal sobre los datos transformados.

```

1 import numpy as np
2
3 def my_func_fit(x, y):
4     log_x = np.log(x)
5     log_y = np.log(y)
6     A = np.vstack([log_x, np.ones(len(log_x))]).T
7     beta, log_alpha = np.linalg.lstsq(A, log_y, rcond=
8     None)[0]
9     alpha = np.exp(log_alpha)
10     return alpha, beta
11
12 # Ejemplo
13 x = np.array([1, 2, 3, 4])

```

```

13 y = 2 * x**1.5
14 alpha, beta = my_func_fit(x, y)
15 print(f"alpha={alpha}, beta={beta}")
16

```

Listing 84: Solución Problema 3

4. **Problema 4:** Error total para un polinomio cúbico.

Solución: El error total es $E = \sum_{i=1}^4 (ax_i^3 + bx_i^2 + cx_i + d - y_i)^2$. Si colocamos un quinto punto que ya esté en el polinomio (es decir, que satisfaga la ecuación cúbica), entonces ese punto no añadirá error adicional.

5. **Problema 5:** Función `my_lin_regression`.

Solución: Similar al Problema 2, pero aquí se espera que se añada un término de sesgo (intercepto) automáticamente si no está incluido en las funciones base.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def my_lin_regression(f, x, y):
5     # Incluir un termino de sesgo (constante) si no esta
6     # presente
7     has_constant = any(np.all(fi(x) == 1) for fi in f)
8     basis_funcs = list(f)
9     if not has_constant:
10         basis_funcs.append(lambda x: np.ones_like(x))
11
12     A = np.array([fi(x) for fi in basis_funcs]).T
13     beta = np.linalg.lstsq(A, y, rcond=None)[0]
14     return beta
15
16 # Test case
17 x = np.linspace(0, 2*np.pi, 1000)
18 y = 3*np.sin(x) - 2*np.cos(x) + np.random.random(len(x))
19 f = [np.sin, np.cos]
20 beta = my_lin_regression(f, x, y)
21 print("Beta (coefs de sin, cos, constante):", beta)
22
23 plt.figure(figsize=(10, 8))
24 plt.plot(x, y, "b.", label="data")

```



```

24 y_fit = beta[0]*np.sin(x) + beta[1]*np.cos(x) + beta[2]
25 plt.plot(x, y_fit, "r", label="regression")
26 plt.xlabel("x")
27 plt.ylabel("y")
28 plt.legend()
29 plt.show()
30

```

Listing 85: Solución Problema 5

6. Problema 6: Función my_exp_regression.

Solución: Aplicamos el truco del logaritmo a la función exponencial $\hat{y}(x) = \alpha e^{\beta x}$.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def my_exp_regression(x, y):
5     log_y = np.log(y)
6     A = np.vstack([x, np.ones(len(x))]).T
7     beta, log_alpha = np.linalg.lstsq(A, log_y, rcond=
8     None)[0]
9     alpha = np.exp(log_alpha)
10    return alpha, beta
11
12 # Test case
13 x = np.linspace(0, 1, 1000)
14 y = 2 * np.exp(-0.5 * x) + 0.25 * np.random.random(len(x))
15
16 alpha, beta = my_exp_regression(x, y)
17 print(f"alpha={alpha}, beta={beta}")
18
19 plt.figure(figsize=(10, 8))
20 plt.plot(x, y, "b.", label="data")
21 plt.plot(x, alpha * np.exp(beta * x), "r", label="
22     regression")
23 plt.xlabel("x")
24 plt.ylabel("y")
25 plt.legend()
26 plt.title("Least Square Regression on Exponential Model")
27 plt.show()
28

```

Listing 86: Solución Problema 6

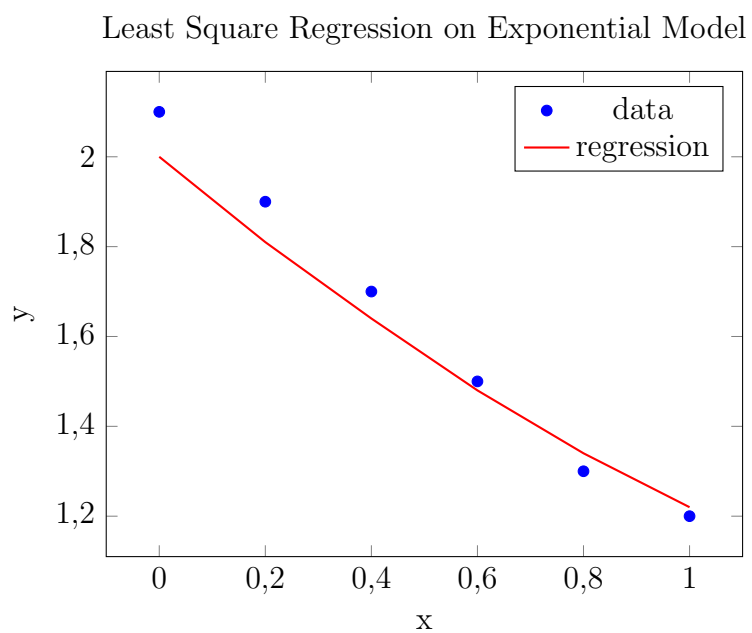


Figura 19: Ajuste exponencial obtenido con `my_exp_regression`.

INTERPOLACIÓN

Planteamiento del Problema de Interpolación

Supongamos que tenemos un conjunto de datos que consiste en valores de datos independientes x_i y valores de datos dependientes y_i , donde $i = 1, \dots, n$. Nos gustaría encontrar una función de estimación $\hat{y}(x)$ tal que $\hat{y}(x_i) = y_i$ para cada punto en nuestro conjunto de datos. Esto significa que la función de estimación pasa a través de nuestros puntos de datos. Dado un nuevo x^* , podemos **interpol**ar su valor de función usando $\hat{y}(x^*)$. En este contexto, $\hat{y}(x)$ se llama una **función de interpolación**. La Fig. 1 muestra el planteamiento del problema de interpolación.

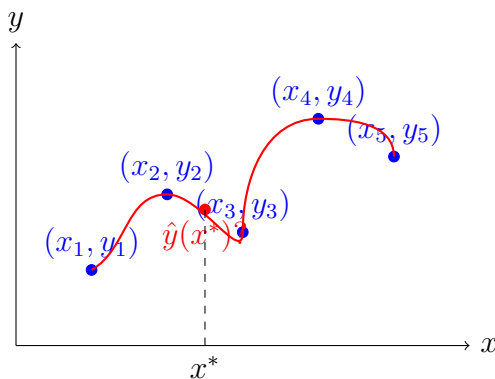


Figura 1: Ilustración del problema de interpolación: estimar el valor de una función entre puntos de datos.

A diferencia de la regresión, la interpolación no requiere que el usuario tenga un modelo subyacente para los datos, especialmente cuando hay muchos puntos de datos confiables. Sin embargo, los procesos que subyacen a

los datos aún deben informar al usuario sobre la calidad de la interpolación. Por ejemplo, nuestros datos pueden consistir en coordenadas (x, y) de un automóvil a lo largo del tiempo. Dado que el movimiento está restringido a la física de maniobras del automóvil, podemos esperar que los puntos entre las coordenadas (x, y) en nuestro conjunto sean "suaves." en lugar de irregulares.

Las siguientes secciones presentarán varios métodos comunes de interpolación.

Interpolación Lineal

En la **interpolación lineal**, se asume que el punto estimado se encuentra en la línea que une los puntos más cercanos a la izquierda y a la derecha. Supongamos, sin pérdida de generalidad, que los puntos de datos x están en orden ascendente, es decir, $x_i < x_{i+1}$, y sea x un punto tal que $x_i < x < x_{i+1}$. Entonces la interpolación lineal en x es

$$\hat{y}(x) = y_i + \frac{(y_{i+1} - y_i)(x - x_i)}{(x_{i+1} - x_i)}.$$

```

1 In [1]: from scipy.interpolate import interp1d
2         import matplotlib.pyplot as plt
3         plt.style.use("seaborn-poster")
4
5 In [2]: x = [0, 1, 2]
6         y = [1, 3, 2]
7         f = interp1d(x, y)
8         y_hat = f(1.5)
9         print(y_hat)
10 2.5
11
12 In [3]: plt.figure(figsize=(10, 8))
13         plt.plot(x, y, "-ob")
14         plt.plot(1.5, y_hat, "ro")
15         plt.title("Linear Interpolation at x = 1.5")
16         plt.xlabel("x")
17         plt.ylabel("y")
18         plt.show()

```

Listing 1: Interpolación lineal manual y con SciPy.

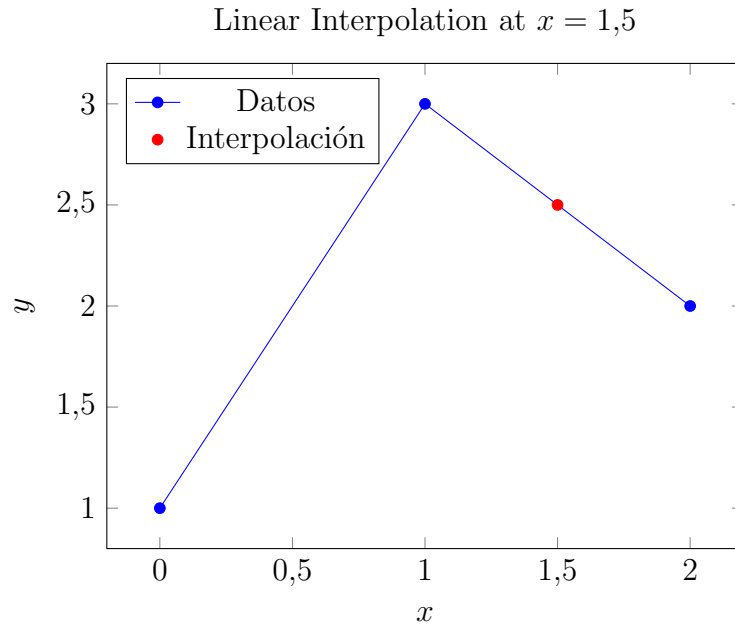


Figura 2: Interpolación lineal en $x = 1,5$.

Dado que $1 < x < 2$, usamos el segundo y tercer punto de datos para calcular la interpolación lineal. Sustituyendo los valores correspondientes obtenemos

$$\hat{y}(x) = 3 + \frac{(2 - 3)(1,5 - 1)}{(2 - 1)} = 2,5.$$

Interpolación por Splines Cúbicos

En la **interpolación por splines cúbicos** (como se muestra en la figura siguiente), la función de interpolación es un conjunto de funciones cúbicas por partes. Específicamente, asumimos que los puntos (x_i, y_i) y (x_{i+1}, y_{i+1}) están unidos por un polinomio cúbico $S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$ que es válido para $x_i \leq x \leq x_{i+1}$ para $i = 1, \dots, n - 1$.

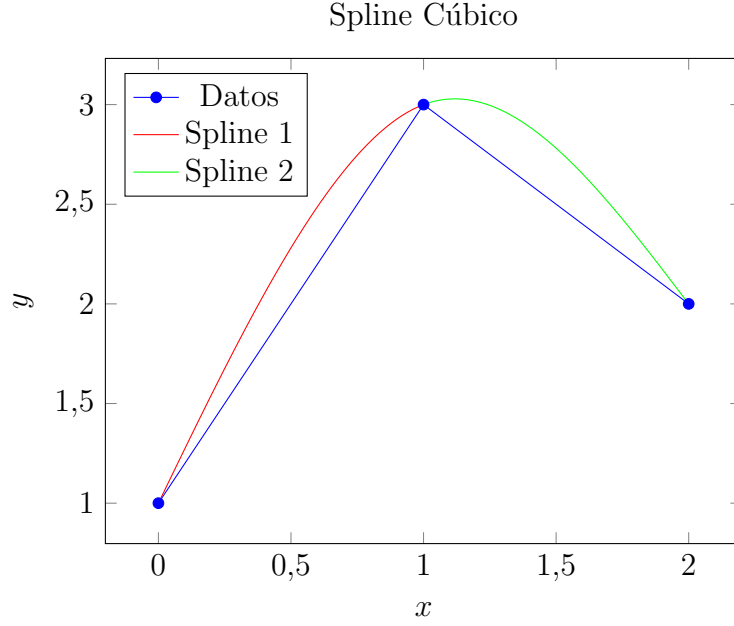


Figura 3: Interpolación por splines cúbicos.

Para encontrar la función de interpolación, primero debemos determinar los coeficientes a_i, b_i, c_i, d_i para cada una de las funciones cúbicas. Para n puntos, hay $n-1$ funciones cúbicas a encontrar, y cada función cúbica requiere cuatro coeficientes. Por lo tanto, tenemos un total de $4(n-1)$ incógnitas, y necesitamos $4(n-1)$ ecuaciones independientes para encontrar todos los coeficientes.

Primero, las funciones cúbicas deben intersectar los puntos de datos en la izquierda y en la derecha:

$$\begin{aligned} S_i(x_i) &= y_i, & i &= 1, \dots, n-1, \\ S_i(x_{i+1}) &= y_{i+1}, & i &= 1, \dots, n-1, \end{aligned}$$

lo que nos da $2(n-1)$ ecuaciones. A continuación, queremos que cada función cúbica se una lo más suavemente posible con sus vecinas, por lo que restringimos los splines a tener primeras y segundas derivadas continuas en los puntos de datos $i = 2, \dots, n-1$:

$$\begin{aligned} S'_i(x_{i+1}) &= S'_{i+1}(x_{i+1}), & i &= 1, \dots, n-2, \\ S''_i(x_{i+1}) &= S''_{i+1}(x_{i+1}), & i &= 1, \dots, n-2, \end{aligned}$$

lo que nos da $2(n - 2)$ ecuaciones.

Se requieren dos ecuaciones más para calcular los coeficientes de $S_i(x)$. Estas dos últimas restricciones son arbitrarias; pueden elegirse para adaptarse a las circunstancias de la interpolación que se está realizando. Un conjunto común de restricciones finales es asumir que las segundas derivadas son cero en los puntos finales. Esto significa que la curva es una "línea recta." en los puntos finales. Explícitamente,

$$S_1''(x_1) = 0, \quad S_{n-1}''(x_n) = 0.$$

En Python, podemos usar la función `CubicSpline` de SciPy para realizar una interpolación por splines cúbicos. Observa que las restricciones anteriores no son las mismas que las que utiliza `CubicSpline` de SciPy por defecto para realizar splines cúbicos. Hay diferentes formas de agregar las dos restricciones finales en SciPy configurando el argumento `bc_type` (consulta la ayuda de `CubicSpline` para obtener más información).

```

1 In [1]: from scipy.interpolate import CubicSpline
2         import numpy as np
3         import matplotlib.pyplot as plt
4         plt.style.use("seaborn-poster")
5
6 In [2]: x = [0, 1, 2]
7         y = [1, 3, 2]
8         # use bc_type = "natural" adds the constraints
9         f = CubicSpline(x, y, bc_type="natural")
10        x_new = np.linspace(0, 2, 100)
11        y_new = f(x_new)
12
13 In [3]: plt.figure(figsize=(10, 8))
14        plt.plot(x_new, y_new, "b")
15        plt.plot(x, y, "ro")
16        plt.title("Cubic Spline Interpolation")
17        plt.xlabel("x")
18        plt.ylabel("y")
19        plt.show()

```

Listing 2: Interpolación por splines cúbicos con SciPy.

Para determinar los coeficientes de cada función cúbica, escribimos las restricciones explícitamente como un sistema de ecuaciones lineales con $4(n - 1)$ incógnitas. Para n puntos de datos, las incógnitas son los coeficientes a_i, b_i, c_i, d_i del spline cúbico S_i , que une los puntos x_i y x_{i+1} .

Las ecuaciones resultantes son lineales en los coeficientes desconocidos a_i, b_i, c_i, d_i . Podemos ponerlos en forma matricial y resolver para los coeficientes de cada spline mediante división izquierda. Recuerda que siempre que resolvemos la ecuación matricial $\mathbf{Ax} = \mathbf{b}$ para $\mathbf{x}$, debemos asegurarnos de que $\mathbf{A}$ sea cuadrada e invertible. En el caso de encontrar las ecuaciones de splines cúbicos, la matriz $\mathbf{A}$ siempre es cuadrada e invertible siempre que los valores x_i en el conjunto de datos sean únicos.

```

1 In [4]: b = np.array([1, 3, 3, 2, 0, 0, 0, 0])
2         b = b[:, np.newaxis]
3         A = np.array([[0, 0, 0, 1, 0, 0, 0, 0],
4                       [0, 0, 0, 0, 1, 1, 1, 1],
5                       [1, 1, 1, 1, 0, 0, 0, 0],
6                       [0, 0, 0, 0, 8, 4, 2, 1],
7                       [3, 2, 1, 0, -3, -2, -1, 0],
8                       [6, 2, 0, 0, -6, -2, 0, 0],
9                       [0, 2, 0, 0, 0, 0, 0, 0],
10                      [0, 0, 0, 0, 12, 2, 0, 0]])
11
12 In [5]: np.dot(np.linalg.inv(A), b)
13 Out[5]: array([[ -0.75],
14               [  0.   ],
15               [  2.75],
16               [  1.   ],
17               [  0.75],
18               [-4.5   ],
19               [  7.25],
20               [-0.5   ]])

```

Listing 3: Resolviendo el sistema de ecuaciones para los splines cúbicos.

Los dos polinomios cúbicos son

$$S_1(x) = -0,75x^3 + 2,75x + 1 \quad \text{para } 0 \leq x \leq 1,$$

$$S_2(x) = 0,75x^3 - 4,5x^2 + 7,25x - 0,5 \quad \text{para } 1 \leq x \leq 2.$$

Por lo tanto, para $x = 1,5$, evaluamos $S_2(1,5)$ y obtenemos un valor estimado de 2,7813.

Interpolación Polinomial de Lagrange

En lugar de encontrar polinomios cúbicos entre pares subsiguientes de puntos de datos, la **interpolación polinomial de Lagrange** encuentra un

solo polinomio que pasa por todos los puntos de datos. Este polinomio se conoce como **polinomio de Lagrange**, $L(x)$. Como función de interpolación, debe tener la propiedad $L(x_i) = y_i$ para cada punto en el conjunto de datos. Al calcular polinomios de Lagrange, es útil escribirlos como una combinación lineal de **polinomios base de Lagrange**, $P_i(x)$, donde

$$P_i(x) = \prod_{j=1, j \neq i}^n \frac{x - x_j}{x_i - x_j},$$

y

$$L(x) = \sum_{i=1}^n y_i P_i(x).$$

Observa que, por construcción, $P_i(x)$ tiene la propiedad de que $P_i(x_j) = 1$ cuando $i = j$ y $P_i(x_j) = 0$ cuando $i \neq j$. Dado que $L(x)$ es una suma de estos polinomios, observamos que $L(x_i) = y_i$ para cada punto, exactamente como se desea.

```

1 In [1]: import numpy as np
2         import numpy.polynomial.polynomial as poly
3         import matplotlib.pyplot as plt
4         plt.style.use("seaborn-poster")
5
6 In [2]: x = [0, 1, 2]
7         y = [1, 3, 2]
8         P1_coeff = [1, -1.5, 0.5]
9         P2_coeff = [0, 2, -1]
10        P3_coeff = [0, -0.5, 0.5]
11
12        # get the polynomial function
13        P1 = poly.Polynomial(P1_coeff)
14        P2 = poly.Polynomial(P2_coeff)
15        P3 = poly.Polynomial(P3_coeff)
16
17        x_new = np.arange(-1.0, 3.1, 0.1)
18
19        fig = plt.figure(figsize=(10, 8))
20        plt.plot(x_new, P1(x_new), "b", label="P1")
21        plt.plot(x_new, P2(x_new), "r", label="P2")
22        plt.plot(x_new, P3(x_new), "g", label="P3")
23        plt.plot(x, np.ones(len(x)), "ko", x, np.zeros(len(x)), "ko")
24        plt.title("Lagrange Basis Polynomials")

```

```

25 plt.xlabel("x")
26 plt.ylabel("y")
27 plt.grid()
28 plt.legend()
29 plt.show()

```

Listing 4: .]Polinomios base de Lagrange para los puntos $[0, 1, 2]$.

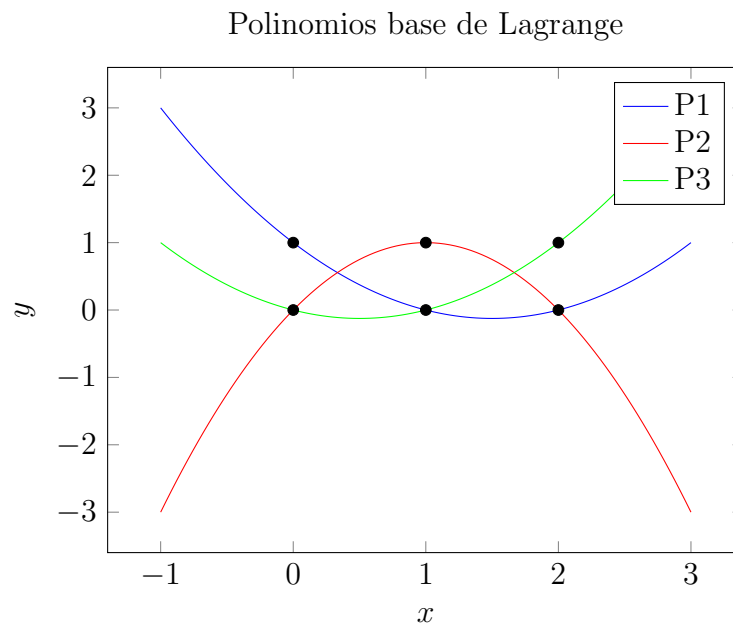


Figura 4: Polinomios base de Lagrange para los puntos $x = 0, 1, 2$.

```

1 In [3]: L = P1 + 3*P2 + 2*P3
2         fig = plt.figure(figsize=(10, 8))
3         plt.plot(x_new, L(x_new), "b", x, y, "ro")
4         plt.title("Lagrange Polynomial")
5         plt.grid()
6         plt.xlabel("x")
7         plt.ylabel("y")
8         plt.show()

```

Listing 5: Polinomio de Lagrange resultante.

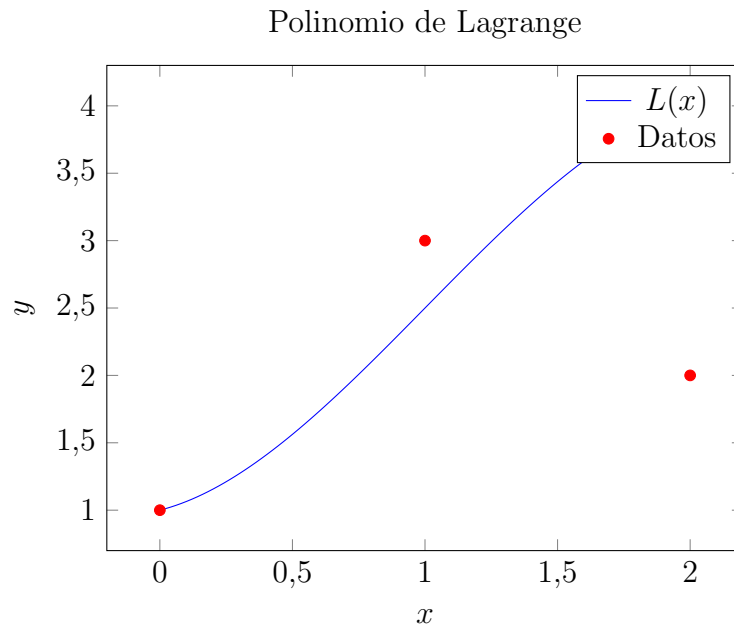


Figura 5: Polinomio de Lagrange que pasa por los puntos $(0, 1)$, $(1, 3)$ y $(2, 2)$.

Advertencia

Los polinomios de interpolación de Lagrange se definen fuera del área de interpolación, es decir, fuera del intervalo $[x_1, x_n]$, y crecerán muy rápido y sin límite fuera de esta región. Esta no es una característica deseable porque, en general, esto no concuerda con el comportamiento de los datos subyacentes. Por lo tanto, la interpolación de Lagrange debe usarse con precaución fuera de la región de interés.

Usando la Función Lagrange de SciPy

En lugar de calcular todo desde cero, en SciPy podemos usar la función `lagrange` directamente para interpolar los datos. Usemos el ejemplo anterior para ilustrar nuestro punto.

```
1 In [4]: from scipy.interpolate import lagrange
2
3 In [5]: f = lagrange(x, y)
4
5 In [6]: fig = plt.figure(figsize=(10, 8))
```

```

6 plt.plot(x_new, f(x_new), "b", x, y, "ro")
7 plt.title("Lagrange Polynomial")
8 plt.grid()
9 plt.xlabel("x")
10 plt.ylabel("y")
11 plt.show()

```

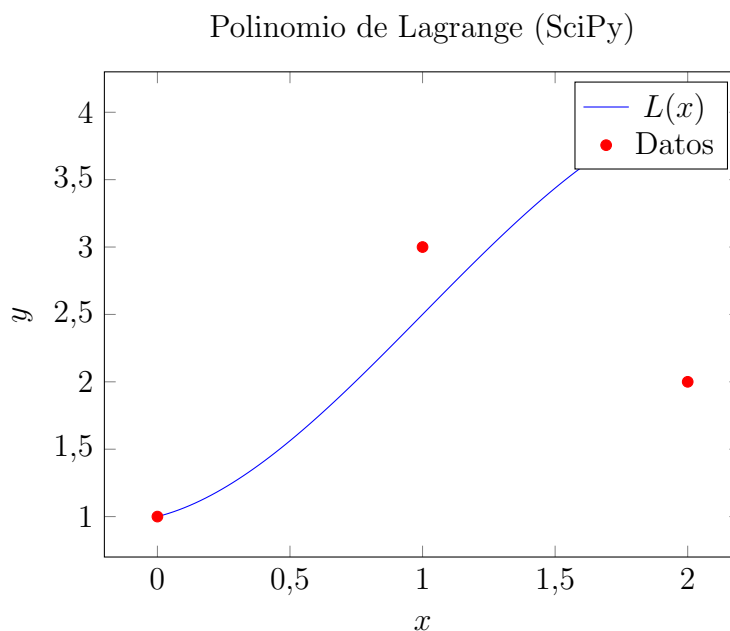
Listing 6: Usando `scipy.interpolate.lagrange`.

Figura 6: Polinomio de Lagrange calculado con la función de SciPy.

Interpolación Polinomial de Newton

La **interpolación polinomial de Newton** es otra forma popular de ajustar exactamente un conjunto de puntos de datos. La forma general del polinomio de Newton de orden $(n - 1)$ que pasa por n puntos es

$$f(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n(x - x_0)(x - x_1) \cdots (x - x_{n-1}),$$

que puede reescribirse como

$$f(x) = \sum_{i=0}^n a_i n_i(x)$$

donde

$$n_i(x) = \prod_{j=0}^{i-1} (x - x_j).$$

La característica especial del polinomio de Newton es que los coeficientes a_i pueden determinarse mediante un procedimiento matemático muy simple. Estos coeficientes se basan en el concepto de **diferencias divididas**.

Las diferencias divididas se definen recursivamente:

$$\begin{aligned} f[x_i] &= y_i, \\ f[x_i, x_{i+1}] &= \frac{f[x_{i+1}] - f[x_i]}{x_{i+1} - x_i}, \\ f[x_i, x_{i+1}, x_{i+2}] &= \frac{f[x_{i+1}, x_{i+2}] - f[x_i, x_{i+1}]}{x_{i+2} - x_i}, \end{aligned}$$

y así sucesivamente. Los coeficientes a_i son entonces $a_i = f[x_0, x_1, \dots, x_i]$.

```

1 In [1]: import numpy as np
2         import matplotlib.pyplot as plt
3         plt.style.use("seaborn-poster")
4         %matplotlib inline
5
6 In [2]: def divided_diff(x, y):
7         """
8         function to calculate the divided
9         difference table
10        """
11        n = len(y)
12        coef = np.zeros([n, n])
13        # the first column is y
14        coef[:, 0] = y
15
16        for j in range(1, n):
17            for i in range(n - j):
18                coef[i][j] = (coef[i+1][j-1] - coef[i][j
19-1]) / (x[i+j] - x[i])
20
21        return coef
22
23        def newton_poly(coef, x_data, x):
24        """
25        evaluate the Newton polynomial
26        at x
27        """

```

```
27         n = len(x_data) - 1
28         p = coef[n]
29         for k in range(1, n+1):
30             p = coef[n-k] + (x - x_data[n-k]) * p
31         return p
32
33 In [3]: x = np.array([-5, -1, 0, 2])
34         y = np.array([-2, 6, 1, 3])
35         # get the divided difference coef
36         a_s = divided_diff(x, y)[0, :]
37         # evaluate on new data points
38         x_new = np.arange(-5, 2.1, 0.1)
39         y_new = newton_poly(a_s, x, x_new)
40
41         plt.figure(figsize=(12, 8))
42         plt.plot(x, y, "bo")
43         plt.plot(x_new, y_new)
44         plt.title("Newton Polynomial Interpolation")
45         plt.xlabel("x")
46         plt.ylabel("y")
47         plt.grid()
48         plt.show()
```

Listing 7: Implementación de la interpolación de Newton.

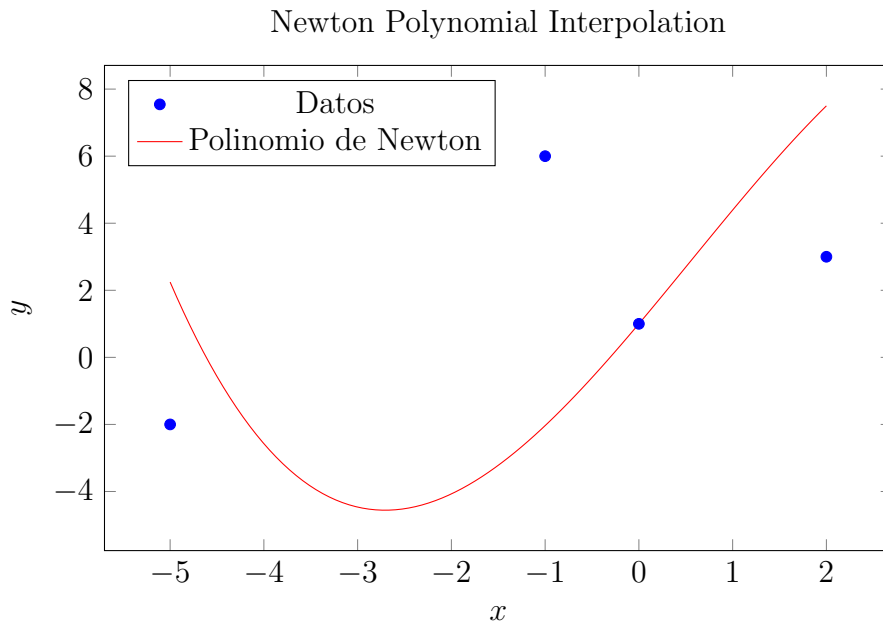


Figura 7: Interpolación polinomial de Newton para los puntos $(-5, -2)$, $(-1, 6)$, $(0, 1)$, $(2, 3)$.

Podemos ver que el polinomio de Newton pasa por todos los puntos de datos y se ajusta a los datos.

Resumen y Problemas

Resumen

1. Dado un conjunto de puntos de datos confiables, la interpolación es un método para estimar valores de variables dependientes para valores de variables independientes que no están en el conjunto de datos.
2. La interpolación lineal, por splines cúbicos, de Lagrange y polinomial de Newton son métodos comunes de interpolación.

Problemas y Soluciones

1. **Problema 1:** Función `my_lin_interp` para interpolación lineal.

Solución: Para cada valor en X , encontramos el intervalo $[x_i, x_{i+1}]$ que lo contiene y aplicamos la fórmula de interpolación lineal.

```

1 import numpy as np
2
3 def my_lin_interp(x, y, X):
4     Y = np.zeros_like(X)
5     for j, x_val in enumerate(X):
6         for i in range(len(x) - 1):
7             if x[i] <= x_val <= x[i+1]:
8                 Y[j] = y[i] + (y[i+1] - y[i]) * (x_val -
9                     x[i]) / (x[i+1] - x[i])
10                break
11
12     return Y
13
14 # Ejemplo
15 x = np.array([0, 1, 2])
16 y = np.array([1, 3, 2])
17 X = np.array([0.5, 1.5])
18 Y = my_lin_interp(x, y, X)
19 print(Y)

```

Listing 8: Solución Problema 1

2. **Problema 2:** Función `my_cubic_spline` para splines cúbicos (sin usar SciPy).

Solución: Esta implementación es más compleja e implica resolver el sistema tridiagonal para los coeficientes de los splines. Se puede implementar siguiendo las ecuaciones presentadas en la sección 17.3.

```

1 def my_cubic_spline(x, y, X):
2     # n = len(x)
3     # Construir matriz A y vector b para los coeficientes
4     # de los splines
5     # Resolver A * coeff = b
6     # Evaluar los splines en X
7     return Y

```

Listing 9: Esqueleto para Problema 2

3. Problema 3: Función `my_nearest_neighbor`.

Solución: Para cada valor en X , encontramos el punto x_j más cercano y asignamos y_j .

```

1 def my_nearest_neighbor(x, y, X):
2     Y = np.zeros_like(X)
3     for j, x_val in enumerate(X):
4         idx = np.argmin(np.abs(x - x_val))
5         Y[j] = y[idx]
6     return Y
7
8 # Ejemplo
9 x = np.array([0, 1, 2, 3])
10 y = np.array([1, 2, 1.5, 0])
11 X = np.array([0.2, 1.8])
12 Y = my_nearest_neighbor(x, y, X)
13 print(Y)
14

```

Listing 10: Solución Problema 3

4. Problema 4: Circunstancia donde el vecino más cercano es superior al spline cúbico.

Solución: El vecino más cercano es superior cuando los datos son discretos por naturaleza o cuando se deben preservar los valores exactos de los puntos de datos, como en la ampliación de imágenes (pixel art) o en mapas de temperatura discretos, donde la interpolación suave podría crear valores que no existen en los datos originales.

5. Problema 5: Función `my_cubic_spline_flat` con condiciones de derivada primera cero en los extremos.

Solución: Similar al Problema 2, pero cambiando las condiciones de contorno a $S'_1(x_1) = 0$ y $S'_{n-1}(x_n) = 0$.

```

1 def my_cubic_spline_flat(x, y, X):
2     # Modificar las condiciones de contorno en la matriz
3     A
4     return Y

```

Listing 11: Esqueleto para Problema 5

6. Problema 6: Función `my_quintic_spline` (spline de quinto grado).

Solución: Se requieren condiciones de continuidad hasta la cuarta derivada, lo que resulta en un sistema más grande. Se pueden usar condiciones de contorno como derivadas segunda o tercera cero en los extremos.

```

1 def my_quintic_spline(x, y, X):
2     # n puntos -> n-1 segmentos, 6 coeficientes por
3     # segmento -> 6(n-1) incognitas
4     # Ecuaciones: interpolacion (2(n-1)), continuidad de
5     # 1a a 4a derivada (4(n-2)), mas 4 condiciones de
6     # contorno.
7     return Y

```

Listing 12: Esqueleto para Problema 6

7. Problema 7: Función `my_interp_plotter`.

Solución: Usamos `interp1d` de SciPy con el `kind` apropiado y graficamos.

```

1 import matplotlib.pyplot as plt
2 from scipy.interpolate import interp1d
3
4 def my_interp_plotter(x, y, X, option):
5     f = interp1d(x, y, kind=option, fill_value="
6     extrapolate")
7     Y = f(X)
8
9     plt.figure(figsize=(10, 6))
10    plt.plot(x, y, "ro", label="Data points")
11    plt.plot(X, Y, "b-", label=f"{option} interpolation")
12    plt.xlabel("x")
13    plt.ylabel("y")
14    plt.title(f"Interpolation using {option} method")
15    plt.legend()
16    plt.grid(True)
17    plt.show()
18
19 # Test case
20 x = np.array([0, .1, .15, .35, .6, .7, .95, 1])
21 y = np.array([1, 0.8187, 0.7408, 0.4966, 0.3012, 0.2466,
22              0.1496, 0.1353])

```

```

21 X = np.linspace(0, 1, 101)
22
23 my_interp_plotter(x, y, X, "nearest")
24 my_interp_plotter(x, y, X, "linear")
25 my_interp_plotter(x, y, X, "cubic")
26

```

Listing 13: Solución Problema 7

8. **Problema 8:** Función `my_D_cubic_spline` con pendientes dadas en los extremos.

Solución: Se modifica el sistema de ecuaciones para splines cúbicos para incorporar las condiciones $S'_1(x_1) = D$ y $S'_{n-1}(x_n) = D$.

```

1 def my_D_cubic_spline(x, y, X, D):
2     # n puntos -> n-1 segmentos
3     # Construir matriz y vector con condiciones de
4     # derivada en los extremos
5     return Y

```

Listing 14: Esqueleto para Problema 8

9. **Problema 9:** Función `my_lagrange` (implementación manual).

Solución: Implementamos la fórmula de Lagrange usando bucles anidados.

```

1 def my_lagrange(x, y, X):
2     n = len(x)
3     Y = np.zeros_like(X)
4     for k, x_val in enumerate(X):
5         total = 0
6         for i in range(n):
7             # Calcular el polinomio base L_i(x_val)
8             li = 1.0
9             for j in range(n):
10                 if j != i:
11                     li *= (x_val - x[j]) / (x[i] - x[j])
12             total += y[i] * li
13     Y[k] = total
14     return Y

```

```

15
16 # Test case
17 x = [0, 1, 2, 3, 4]
18 y = [2, 1, 3, 5, 1]
19 X = np.linspace(0, 4, 101)
20 Y_lagrange = my_lagrange(x, y, X)
21
22 plt.figure(figsize=(10, 8))
23 plt.plot(X, Y_lagrange, "b", label="interpolation")
24 plt.plot(x, y, "ro", label="data points")
25 plt.xlabel("x")
26 plt.ylabel("y")
27 plt.title("Lagrange Interpolation of Data Points")
28 plt.legend()
29 plt.grid()
30 plt.show()
31

```

Listing 15: Solución Problema 9

10. Problema 10: Ajuste con interpolación de Newton.

Solución: Usamos la función `newton_poly` implementada anteriormente.

```

1 x_data = np.array([0, 1, 2, 3, 4])
2 y_data = np.array([2, 1, 3, 5, 1])
3
4 # Calcular coeficientes de diferencias divididas
5 coef = divided_diff(x_data, y_data)[0, :]
6
7 # Evaluar en nuevos puntos
8 x_new = np.linspace(0, 4, 101)
9 y_new = newton_poly(coef, x_data, x_new)
10
11 plt.figure(figsize=(10, 8))
12 plt.plot(x_data, y_data, "ro", label="Data")
13 plt.plot(x_new, y_new, "b-", label="Newton Poly")
14 plt.xlabel("x")
15 plt.ylabel("y")
16 plt.title("Newton Polynomial Interpolation")
17 plt.legend()
18 plt.grid()
19 plt.show()

```

Listing 16: Solución Problema 10

SERIES DE TAYLOR

Expresando Funciones Usando una Serie de Taylor

Una **sucesión** es un conjunto ordenado de números denotado por la lista de números entre paréntesis. Por ejemplo, $s = (s_1, s_2, s_3, \dots)$ significa que s es la sucesión $s_1, s_2, s_3, \dots$, y así sucesivamente. En este contexto, “ordenado” significa que s_1 viene antes que s_2 , no que $s_1 < s_2$. Muchas sucesiones tienen una estructura más complicada. Por ejemplo, $s = (n^2, n \in \mathbb{N})$ es la sucesión $0, 1, 4, 9, \dots$. Una **serie** es la suma de una sucesión hasta un cierto elemento. Una **sucesión infinita** es una sucesión con un número infinito de términos, y una **serie infinita** es la suma de una sucesión infinita.

Una **expansión en serie de Taylor** es una representación de una función mediante una serie infinita de polinomios alrededor de un punto. Matemáticamente, la serie de Taylor de una función $f(x)$ se define como

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)(x-a)^n}{n!},$$

donde $f^{(n)}$ es la n -ésima derivada de f y $f^{(0)}$ es la función f .

```
1 In []: # Calcular la expansion en serie de Taylor para f(x) =  
        5x^2 + 3x + 5 alrededor de a = 0 y a = 1.  
2 # Verificar que f y sus expansiones en serie de Taylor son  
    identicas.  
3  
4 # Primero calcular las derivadas analiticamente:  
5 # f(x) = 5x^2 + 3x + 5,  
6 # f'(x) = 10x + 3,  
7 # f''(x) = 10.  
8 # Alrededor de a = 0:
```

```

9 # f(x) = (5*0^0)/0! + (3*0^1)/1! + (10*0^2)/2! + 0 + ... = 5x
    ^2 + 3x + 5.
10 # Alrededor de a = 1:
11 # f(x) = (13*(x-1)^0)/0! + (13*(x-1)^1)/1! + (10*(x-1)^2)/2!
    + 0 + ...
12 # = 13 + 13x - 13 + 5x^2 - 10x + 5 = 5x^2 + 3x + 5.

```

Listing 17: Expansión en serie de Taylor para un polinomio.

Observa que una expansión en serie de Taylor de cualquier polinomio tiene un número finito de términos porque la n -ésima derivada de cualquier polinomio es cero cuando n es suficientemente grande.

```

1 In []: # Escribir la serie de Taylor para sin(x) alrededor
    del punto a = 0.
2 # Sea f(x) = sin(x). Segun la expansion en serie de Taylor:
3 # f(x) = sin(0)/0! x^0 + cos(0)/1! x^1 + -sin(0)/2! x^2 + -
    cos(0)/3! x^3 + sin(0)/4! x^4 + cos(0)/5! x^5 + ...
4 # La expansion puede escribirse de forma compacta como:
5 # f(x) = sum_{n=0}^{\infty} (-1)^n x^{(2n+1)} / (2n+1)!

```

Listing 18: Serie de Taylor para $\sin(x)$.

Aproximaciones Usando Series de Taylor

Claramente, no es útil expresar funciones como sumas infinitas porque no podemos computarlas. Dicho esto, a menudo es útil aproximar funciones usando una **aproximación por serie de Taylor de orden N** de una función, que es una truncamiento de su expansión de Taylor en algún $n = N$. Esta técnica es especialmente poderosa cuando hay un punto alrededor del cual tenemos conocimiento sobre una función y todas sus derivadas. Por ejemplo, si tomamos la expansión de Taylor de e^x alrededor de $a = 0$, entonces $f^{(n)}(a) = 1$ para toda n , ¡y no tenemos que calcular las derivadas en la expansión de Taylor para aproximar e^x !

```

1 In [1]: import numpy as np
2         import matplotlib.pyplot as plt
3         plt.style.use("seaborn-poster")
4
5 In [2]: x = np.linspace(-np.pi, np.pi, 200)
6         y = np.zeros(len(x))
7         labels = ["First Order", "Third Order", "Fifth Order",
    , "Seventh Order"]

```



```

8
9     plt.figure(figsize=(10, 8))
10    for n, label in zip(range(4), labels):
11        y = y + ((-1)**n * (x)**(2*n+1)) / np.math.
factorial(2*n+1)
12        plt.plot(x, y, label=label)
13
14    plt.plot(x, np.sin(x), "k", label="Analytic")
15    plt.grid()
16    plt.title("Taylor Series Approximations of Various
Orders")
17    plt.xlabel("x")
18    plt.ylabel("y")
19    plt.legend()
20    plt.show()

```

Listing 19: Aproximaciones de Taylor de $\sin(x)$ de diferentes órdenes.

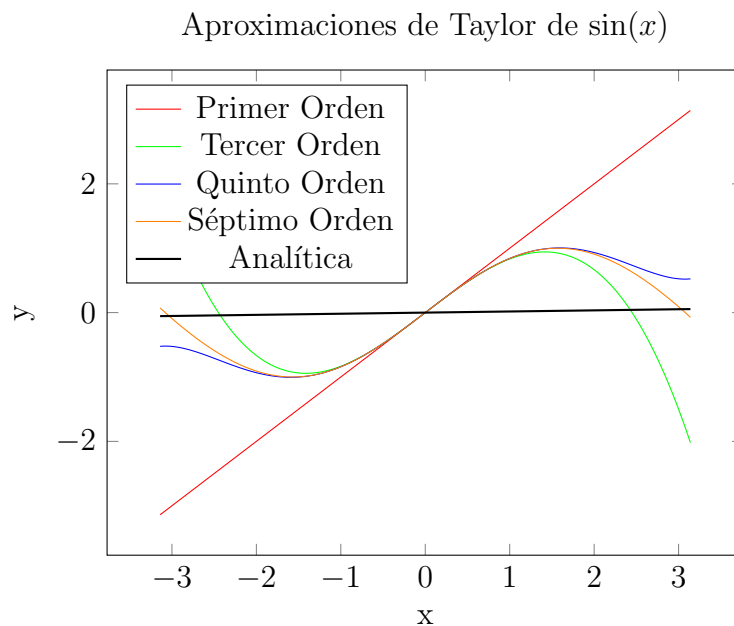


Figura 8: Aproximaciones de la serie de Taylor de $\sin(x)$ de varios órdenes.

Obviamente, la aproximación se acerca rápidamente a la función analítica, incluso para x no cercano a $a = 0$. Observa que en el código anterior, también usamos una nueva función `zip`, que nos permite recorrer dos parámetros

`range(4)` y `labels` para usar en nuestro gráfico.

```

1 In [3]: x = np.pi / 2
2         y = 0
3         for n in range(4):
4             y = y + ((-1)**n * (x)**(2*n+1)) / np.math.
               factorial(2*n+1)
5         print(y)
6 0.9998431013994987

```

Listing 20: Aproximación de séptimo orden de $\sin(\pi/2)$.

La aproximación de la serie de Taylor de séptimo orden está muy cerca del valor teórico de la función, incluso si se calcula lejos del punto alrededor del cual se calculó la serie de Taylor (es decir, $x = \pi/2$ y $a = 0$).

La aproximación más común de la serie de Taylor es la aproximación de primer orden, o **aproximación lineal**. Intuitivamente, para funciones "suaves", la aproximación lineal de la función alrededor de un punto a es legítima siempre que permanezcas suficientemente cerca de a . En otras palabras, las funciones "suaves" se parecen cada vez más a una línea cuanto más te acercas a cualquier punto. En la siguiente figura, esto se ha trazado en niveles sucesivos de zoom utilizando una función suave para ilustrar la naturaleza lineal de las funciones localmente. Las aproximaciones lineales son herramientas útiles al analizar funciones complicadas localmente.

```

1 In [4]: x = np.linspace(0, 3, 30)
2         y = np.exp(x)
3         plt.figure(figsize=(14, 4.5))
4         plt.subplot(1, 3, 1)
5         plt.plot(x, y)
6         plt.grid()
7         plt.subplot(1, 3, 2)
8         plt.plot(x, y)
9         plt.grid()
10        plt.xlim(1.7, 2.3)
11        plt.ylim(5, 10)
12        plt.subplot(1, 3, 3)
13        plt.plot(x, y)
14        plt.grid()
15        plt.xlim(1.92, 2.08)
16        plt.ylim(6.6, 8.2)
17        plt.tight_layout()
18        plt.show()

```

Listing 21: Zoom en una función suave para ilustrar la aproximación lineal.

[Figura simulada con tres niveles de zoom en e^x]

Figura 9: Zoom sucesivo en la función exponencial para ilustrar la linealidad local.

```

1 In [5]: np.exp(1)
2 Out[5]: 2.718281828459045
3
4 In [6]: np.exp(0.01)
5 Out[6]: 1.010050167084168

```

Listing 22: Aproximación lineal de e^x alrededor de $a = 0$.

La aproximación lineal de e^x alrededor de $a = 0$ es $1 + x$. La aproximación lineal de e^1 es 2, que es inexacta, y la aproximación lineal de $e^{0.01}$ es 1.01, que es muy buena. Este ejemplo ilustra cómo la aproximación lineal se vuelve cercana al punto del cual se toma la aproximación.

Discusión sobre Errores

Errores de Truncamiento para Series de Taylor

En el análisis numérico, generalmente hay dos fuentes de error: **redondeo** y **truncamiento**. Los errores de redondeo se deben a la inexactitud en la representación de números reales en una computadora y a las operaciones aritméticas realizadas con ellos. Consulta el Capítulo 9 para una discusión adicional sobre este tipo de error.

Los errores de truncamiento se deben a la naturaleza aproximada del método utilizado y generalmente surgen cuando se usa una aproximación en lugar de un procedimiento matemático exacto, por ejemplo, cuando usamos la serie de Taylor para aproximar una función. Por ejemplo, si usamos la serie de Taylor para aproximar la función e^x , obtenemos

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \cdots.$$

Dado que se necesita una secuencia infinita para aproximar la función, usar solo unos pocos términos resultará en un error de aproximación (o truncamiento). Por ejemplo, si solo usamos los primeros cuatro términos para

aproximar e^2 , obtenemos

$$e^2 \approx 1 + 2 + \frac{2^2}{2!} + \frac{2^3}{3!} = 6,3333.$$

Obviamente, hay un error asociado con tal solución, ya que truncamos el resto de los términos en la serie de Taylor. Por lo tanto, la función $f(x)$ se puede escribir como la aproximación de la serie de Taylor más un término de error de truncamiento

$$f(x) = f_n(x) + E_n(x).$$

Cuantos más términos usemos, más cerca estará la aproximación del valor exacto. Usemos Python para calcular el ejemplo anterior.

```

1 In [1]: import numpy as np
2
3 In [2]: exp = 0
4         x = 2
5         for i in range(10):
6             exp = exp + (x**i) / np.math.factorial(i)
7             print(f"Using {i}-term, {exp}")
8         print(f"The true e^2 is: \n{np.exp(2)}")
9 Using 0-term, 1.0
10 Using 1-term, 3.0
11 Using 2-term, 5.0
12 Using 3-term, 6.333333333333333
13 Using 4-term, 7.0
14 Using 5-term, 7.266666666666667
15 Using 6-term, 7.355555555555555
16 Using 7-term, 7.3809523809523805
17 Using 8-term, 7.387301587301587
18 Using 9-term, 7.3887125220458545
19 The true e^2 is:
20 7.38905609893065

```

Listing 23: Aproximación de e^2 con series de Taylor de diferentes órdenes.

Estimación de Errores de Truncamiento

Podemos ver que cuanto mayor es el orden utilizado para aproximar la función en un punto, más cerca estamos del valor verdadero. Para cada orden que elegimos, hay un error asociado, y la aproximación solo es útil si tenemos una idea de qué tan precisa es la aproximación. Esta es la motivación para que necesitemos entender más sobre los errores.

Usando la serie de Taylor, si usamos solo los primeros n términos, podemos ver que

$$f(x) = f_n(x) + E_n(x) = \sum_{k=0}^n \frac{f^{(k)}(a)(x-a)^k}{k!} + E_n(x).$$

El término $E_n(x)$ es el **resto** de la serie de Taylor, o el error de truncamiento que mide qué tan lejos está la aproximación $f_n(x)$ de $f(x)$. Podemos estimar el error usando el **Teorema de Estimación del Resto de Taylor**, que establece:

Si la función $f(x)$ tiene $n+1$ derivadas para todo x en un intervalo I que contiene a a , entonces para cada x en I existe un z entre x y a tal que

$$E_n(x) = \frac{f^{(n+1)}(z)(x-a)^{n+1}}{(n+1)!}.$$

Si sabemos que M es el valor máximo de $|f^{(n+1)}|$ en el intervalo, entonces obtenemos

$$|E_n(x)| \leq M \frac{|x-a|^{n+1}}{(n+1)!}.$$

Esto nos proporciona una cota para el error de truncamiento utilizando este teorema. Ve el ejemplo a continuación.

```
1 In [3]: abs(7.3887125220458545 - np.exp(2))
2 Out [3]: 0.0003435768847959153
```

Listing 24: Estimación de la cota del error para la aproximación de Taylor de e^2 .

Para entender la base de este error, cuando usamos $n = 9$, sabemos que $(e^x)' = e^x$, y $a = 0$; por lo tanto, el error relacionado con $x = 2$ es

$$E_n(x) = \frac{f^{(10)}(z)(2)^{10}}{10!} = \frac{e^z 2^{10}}{10!}.$$

Recuerda que $0 \leq z \leq 2$, y $e < 3$; por lo tanto,

$$|E_n(x)| \leq 3 \cdot \frac{2^{10}}{10!} = 0,00254.$$

Si usamos la serie de Taylor con $n = 9$ para aproximar e^2 , nuestro error absoluto debería ser menor que 0.00254. Verificamos esto a continuación, y el error real es 0.00034, que es menor que la cota.

Errores de Redondeo para Series de Taylor

Numéricamente, al sumar muchos términos en una suma, debemos tener en cuenta la acumulación numérica de errores que se debe a los errores de redondeo de punto flotante; ve el siguiente ejemplo.

```

1 In [4]: exp = 0
2         x = -30
3         for i in range(200):
4             exp = exp + (x**i) / np.math.factorial(i)
5         print(f"Using 199-term, our result is {exp}")
6         print(f"The true e^{-30} is: {np.exp(x)}")
7 Using 199-term, our result is -8.553016433669241e-05
8 The true e^{-30} is: 9.357622968840175e-14

```

Listing 25: Problema de redondeo al aproximar e^{-30} con series de Taylor.

Del ejemplo anterior, está claro que nuestra estimación usando la serie de Taylor ya no está cerca del valor verdadero, sin importar cuántos términos incluyamos en el cálculo, lo cual se debe a los errores de redondeo que discutimos anteriormente. Para obtener un resultado pequeño, cuando se usan argumentos negativos grandes, la serie de Taylor requiere números grandes alternados para cancelarse. Necesitamos muchos dígitos de precisión en la serie para capturar tanto los números grandes como los pequeños con suficientes dígitos restantes para obtener el resultado en la precisión de salida deseada. Por eso el programa arrojó un mensaje de error en el ejemplo anterior.

Resumen y Problemas

Resumen

1. Algunas funciones pueden representarse perfectamente mediante una serie de Taylor, que es una suma infinita de polinomios.
2. Las funciones que tienen una expansión en serie de Taylor pueden aproximarse truncando su serie de Taylor.
3. La aproximación lineal es una aproximación local común para funciones.
4. El error de truncamiento puede estimarse utilizando el Teorema de Estimación del Resto de Taylor.

5. Ten en cuenta el error de redondeo en la serie de Taylor.

Problemas y Soluciones

1. **Problema 1:** Usa expansiones en serie de Taylor para mostrar que $e^{ix} = \cos(x) + i \sin(x)$, donde $i = \sqrt{-1}$.

Solución: Las series de Taylor para e^u , $\cos(x)$ y $\sin(x)$ alrededor de 0 son:

$$e^u = \sum_{n=0}^{\infty} \frac{u^n}{n!}, \quad \cos(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{(2n)!}, \quad \sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!}.$$

Sustituyendo $u = ix$ en la serie de e^u :

$$e^{ix} = \sum_{n=0}^{\infty} \frac{(ix)^n}{n!} = \sum_{n=0}^{\infty} \frac{i^n x^n}{n!}.$$

Separando en términos pares e impares de n y usando $i^{2k} = (-1)^k$, $i^{2k+1} = i(-1)^k$, obtenemos la identidad de Euler.

2. **Problema 2:** Usa la aproximación lineal de $\sin(x)$ alrededor de $a = 0$ para mostrar que $\frac{\sin(x)}{x} \approx 1$ para x pequeño.

Solución: La aproximación lineal de $\sin(x)$ alrededor de 0 es $\sin(x) \approx x$. Por lo tanto, $\frac{\sin(x)}{x} \approx \frac{x}{x} = 1$ para $x \neq 0$. Este límite se vuelve exacto cuando $x \rightarrow 0$.

3. **Problema 3:** Expansión de Taylor para e^{x^2} y función `my_double_exp`.

Solución: La serie de Taylor para e^{x^2} alrededor de 0 se obtiene sustituyendo $u = x^2$ en la serie de e^u : $e^{x^2} = \sum_{n=0}^{\infty} \frac{x^{2n}}{n!}$. La función debe manejar arrays de entrada.

```
1 import numpy as np
2
3 def my_double_exp(x, n):
4     approx = np.zeros_like(x, dtype=float)
5     for i in range(n):
6         approx += (x**(2*i)) / np.math.factorial(i)
7     return approx
```

```

8
9 # Ejemplo
10 x_vals = np.array([0.1, 0.5, 1.0])
11 print(my_double_exp(x_vals, 5))
12

```

Listing 26: Solución Problema 3

4. Problema 4: Aproximaciones de Taylor a exp y cota de error.

Solución: Calculamos las sumas parciales y usamos el resto de Lagrange. Para el orden 7 alrededor de 0, el error en x está acotado por $\frac{e^{|x|}|x|^8}{8!}$ (asumiendo que la octava derivada es e^z con z entre 0 y x).

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def taylor_exp_approx(x, n):
5     approx = 0
6     for i in range(n+1):
7         approx += x**i / np.math.factorial(i)
8     return approx
9
10 x_vals = np.linspace(-2, 2, 100)
11 for order in range(1, 8):
12     y_approx = np.array([taylor_exp_approx(x, order) for
13 x in x_vals])
14     plt.plot(x_vals, y_approx, label=f"Order {order}")
15
16 plt.plot(x_vals, np.exp(x_vals), "k--", label="exp(x)")
17 plt.legend()
18 plt.title("Aproximaciones de Taylor a exp(x)")
19 plt.xlabel("x")
20 plt.ylabel("y")
21 plt.grid()
22 plt.show()
23
24 # Cota de error para orden 7 en x=1
25 # f^{(8)}(z) = e^z, max en [0,1] es e^1
26 max_8deriv = np.exp(1)
27 error_bound = max_8deriv * (1**8) / np.math.factorial(8)
28 print(f"Cota de error para orden 7 en x=1: {error_bound}")
29

```


28

Listing 27: Solución Problema 4

5. **Problema 5:** Comparación de expansiones de Taylor para $\sin(x) \cos(x)$.

Solución: Calculamos ambas aproximaciones en $x = \pi/2$.

```

1 import numpy as np
2 import math
3
4 def taylor_sin(x, n): # n es el numero de terminos (impar
5     )
6     result = 0
7     for k in range(n):
8         result += ((-1)**k) * (x**(2*k+1)) / math.
9         factorial(2*k+1)
10    return result
11
12 def taylor_cos(x, n): # n es el numero de terminos (par)
13     result = 0
14     for k in range(n):
15         result += ((-1)**k) * (x**(2*k)) / math.factorial
16         (2*k)
17    return result
18
19 def taylor_sin_cos_product(x, n_sin, n_cos):
20     return taylor_sin(x, n_sin) * taylor_cos(x, n_cos)
21
22 def taylor_sin_cos_combined(x, n):
23     # Serie para sin(x)cos(x) = 0.5*sin(2x)
24     result = 0
25     for k in range(n):
26         result += ((-1)**k) * ((2*x)**(2*k+1)) / math.
27         factorial(2*k+1)
28    return 0.5 * result
29
30 x_test = np.pi/2
31 approx_product = taylor_sin_cos_product(x_test, 4, 4) # 4
32     terminos de sin y cos
33 approx_combined = taylor_sin_cos_combined(x_test, 4) # 4
34     terminos de la serie combinada
35 true_value = np.sin(x_test) * np.cos(x_test)
36
37 print(f"Producto de series: {approx_product}, error: {abs
38     (approx_product - true_value)}")

```

```

32 print(f"Serie combinada: {approx_combined}, error: {abs(
    approx_combined - true_value)}")
33 # La serie combinada suele ser mas precisa porque evita
    la propagacion de errores.
34

```

Listing 28: Solución Problema 5

6. **Problema 6:** Aproximación de Taylor de cuarto orden para $\cos(0,2)$ y cota de error.

Solución: La serie de Taylor para $\cos(x)$ alrededor de 0 es $\cos(x) = 1 - x^2/2! + x^4/4! - \dots$. Para el orden 4, usamos términos hasta x^4 . La cota del error de truncamiento es $\frac{|f^{(5)}(z)||x|^5}{5!}$ con $f^{(5)}(z) = -\sin(z)$ o $\cos(z)$, cuyo valor absoluto máximo es 1. Por lo tanto, la cota es $\frac{|x|^5}{5!}$.

```

1 import numpy as np
2 import math
3
4 x = 0.2
5 approx_cos = 1 - x**2/2 + x**4/24
6 true_cos = np.cos(x)
7 error_bound = (x**5) / math.factorial(5)
8
9 print(f"cos(0.2) aproximado: {approx_cos}")
10 print(f"cos(0.2) real: {true_cos}")
11 print(f"Error real: {abs(approx_cos - true_cos)}")
12 print(f"Cota de error: {error_bound}")
13

```

Listing 29: Solución Problema 6

7. **Problema 7:** Función `my_cosh_approximator`.

Solución: $\cosh(x) = (e^x + e^{-x})/2$. Su serie de Taylor es $\cosh(x) = \sum_{n=0}^{\infty} \frac{x^{2n}}{(2n)!}$. La función debe sumar términos hasta n (inclusive). Las aproximaciones para n y $n+1$ serán iguales porque el término de orden impar es cero.

```

1 import numpy as np
2 import math
3

```

```
4 def my_cosh_approximator(x, n):
5     # n es el numero de terminos (solo potencias pares)
6     approx = np.zeros_like(x, dtype=float)
7     for i in range(n+1):
8         approx += (x**(2*i)) / math.factorial(2*i)
9     return approx
10
11 # Ejemplo
12 x_vals = np.linspace(-2, 2, 100)
13 plt.plot(x_vals, my_cosh_approximator(x_vals, 0), label="
14         n=0 (1)")
15 plt.plot(x_vals, my_cosh_approximator(x_vals, 1), label="
16         n=1 (1 + x^2/2)")
17 plt.plot(x_vals, my_cosh_approximator(x_vals, 3), label="
18         n=3 (hasta x^6/720)")
19 plt.plot(x_vals, np.cosh(x_vals), "k--", label="cosh(x)")
20 plt.legend()
21 plt.title("Aproximaciones de Taylor a cosh(x)")
22 plt.xlabel("x")
23 plt.ylabel("y")
24 plt.grid()
25 plt.show()
```

Listing 30: Solución Problema 7

BUSQUEDA DE RAICES

19.1 Planteamiento del Problema de Búsqueda de Raíces

La raíz o cero de una función $f(x)$ es un x_r tal que $f(x_r) = 0$. Para funciones como $f(x) = x^2 - 9$, las raíces son claramente 3 y -3. Sin embargo, para otras funciones, como $f(x) = \cos(x) - x$, determinar una solución analítica o exacta para las raíces de funciones puede ser difícil. Para estos casos, es útil generar aproximaciones numéricas de las raíces de f y entender sus limitaciones.

¡PRUÉBALO! Usa la función `fsolve` de SciPy para calcular la raíz de $f(x) = \cos(x) - x$ cerca de -2 . Verifica que la solución sea una raíz (o lo suficientemente cercana).

```
1 In [1]: import numpy as np
2         from scipy import optimize
3         f = lambda x: np.cos(x) - x
4         r = optimize.fsolve(f, -2)
5         print("r =", r)
6         # Verify the solution is a root
7         result = f(r)
8         print("result=", result)
```

```
r = [0.73908513]
result= [0.]
```

¡PRUÉBALO! La función $f(x) = \frac{1}{x}$ no tiene raíz. Usa la función `fsolve` para intentar calcular la raíz de $f(x) = \frac{1}{x}$. Activa `full_output` para ver qué está sucediendo. Consulta la documentación para más detalles.

```
1 In [2]: f = lambda x: 1/x
```

```

2      r, infodict, ier, mesg = optimize.fsolve(f, -2,
      full_output=True)
3      print("r =", r)
4      result = f(r)
5      print("result=", result)
6      print(mesg)

```

```
r = [-3.52047359e+83]
```

```
result= [-2.84052692e-84]
```

```
The number of calls to function has reached maxfev = 400.
```

El valor r que se devolvió no es una raíz, aunque el valor de $f(r)$ es un número muy pequeño. Dado que activamos `full_output`, podemos ver más información. Se habría devuelto un mensaje si no se encontrara solución; podemos ver los detalles de `mesg` para la causa del fallo: "The number of calls to function has reached maxfev = 400".

19.2 Tolerancia

En ingeniería y ciencia, el error es una desviación de un valor esperado o calculado. La *tolerancia* es el nivel de error que es aceptable para una aplicación de ingeniería. Decimos que un programa de computadora ha *convergido* a una solución cuando ha encontrado una solución con un error menor que la tolerancia. Al calcular raíces numéricamente, o realizar cualquier otro tipo de análisis numérico, es importante establecer tanto una métrica para el error como una tolerancia que sea adecuada para una aplicación de ingeniería/-ciencia determinada.

Para calcular raíces, queremos un x_r tal que $f(x_r)$ esté muy cerca de cero. Por lo tanto, $|f(x)|$ es una posible opción para la medida del error, ya que cuanto más pequeño sea, más probable es que estemos cerca de una raíz. Además, si asumimos que x_i es la i -ésima estimación de un algoritmo para encontrar una raíz, entonces $|x_{i+1} - x_i|$ es otra posible opción para medir el error, ya que esperamos que la mejora entre estimaciones subsiguientes disminuya a medida que se acerca a una solución. Como se demostrará en los siguientes ejemplos, estas diferentes opciones tienen sus ventajas y desventajas.

¡PRUÉBALO! Sea el error medido por $e = |f(x)|$ y sea `tol` el nivel aceptable de error. La función $f(x) = x^2 + \text{tol}/2$ no tiene raíces reales. Dado

que $|f(0)| = \text{tol}/2$, es aceptable como solución para un programa de búsqueda de raíces.

¡PRUÉBALO! Sea el error medido por $e = |x_{i+1} - x_i|$ y sea **tol** el nivel aceptable de error. La función $f(x) = 1/x$ no tiene raíces reales, pero las estimaciones $x_i = -\text{tol}/4$ y $x_{i+1} = \text{tol}/4$ tienen un error de $e = \text{tol}/2$ y son una solución aceptable para un programa de computadora.

Basado en estas observaciones, el uso de la tolerancia y los criterios de convergencia debe hacerse con mucho cuidado y en el contexto del programa que los utiliza.

19.3 Método de Bisección

El *Teorema del Valor Intermedio* dice que si $f(x)$ es una función continua entre a y b , y $\text{sign}(f(a)) \neq \text{sign}(f(b))$, entonces debe existir un c , tal que $a < c < b$ y $f(c) = 0$. Esto se ilustra en la Fig. 19.1.

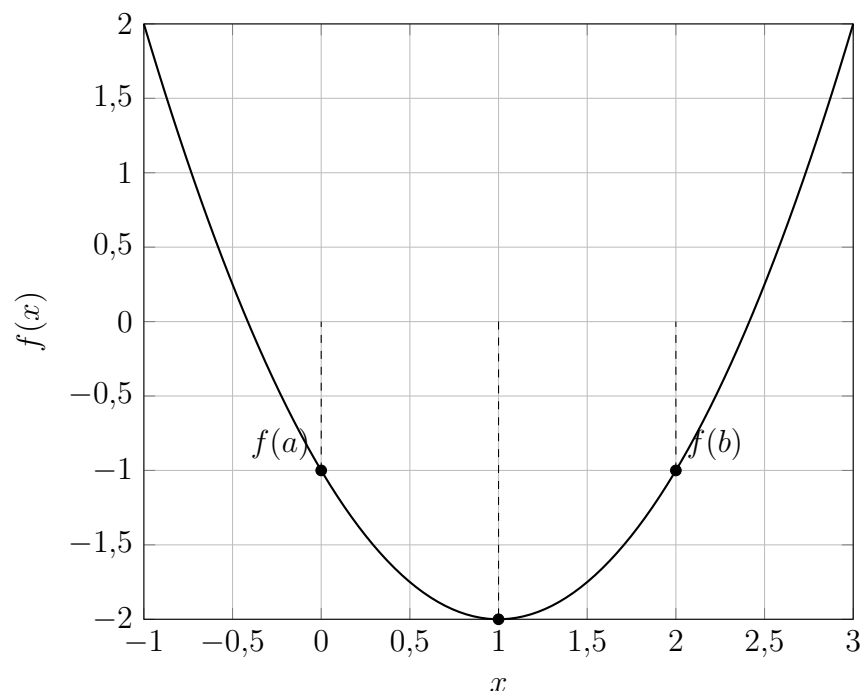


Figura 10: Ilustración del teorema del valor intermedio. Si $\text{sign}(f(a))$ y $\text{sign}(f(b))$ no son iguales, entonces existe un c en (a, b) tal que $f(c) = 0$.

El método de bisección utiliza el teorema del valor intermedio de manera iterativa para encontrar raíces. Sea $f(x)$ una función continua, y sean a y b valores escalares reales tales que $a < b$. Supongamos, sin pérdida de generalidad, que $f(a) > 0$ y $f(b) < 0$. Entonces, por el teorema del valor intermedio, debe haber una raíz en el intervalo abierto (a, b) . Ahora sea $m = \frac{b+a}{2}$ el punto medio entre a y b . Si $f(m) = 0$ o está lo suficientemente cerca, entonces m es una raíz. Si $f(m) > 0$, entonces m es una mejora del límite izquierdo a , y está garantizado que hay una raíz en el intervalo abierto (m, b) . Si $f(m) < 0$, entonces m es una mejora del límite derecho b , y está garantizado que hay una raíz en el intervalo abierto (a, m) . Este escenario se representa en la Fig. 19.2.

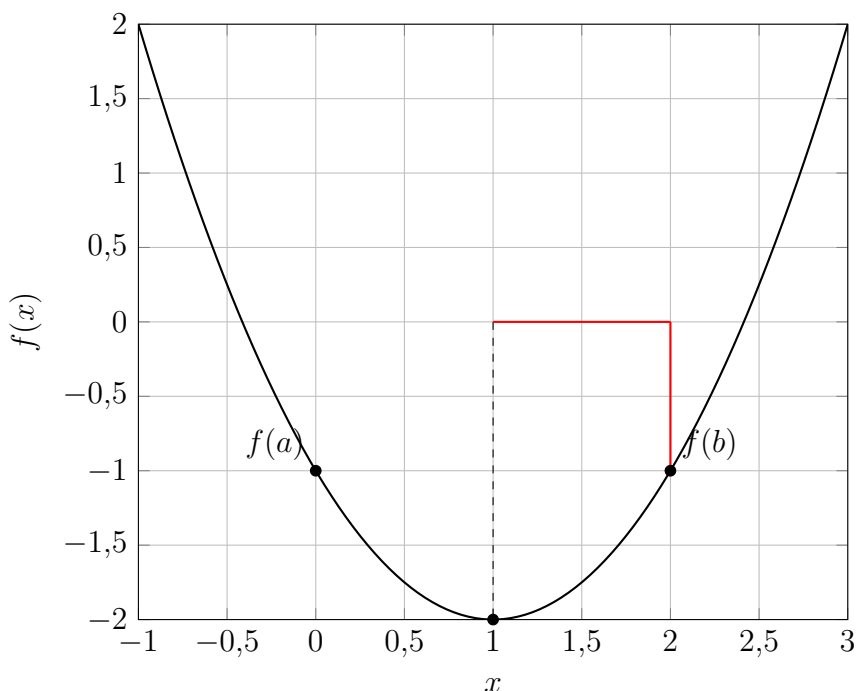


Figura 11: Ilustración del método de bisección. El signo de $f(m)$ se verifica para determinar si la raíz está contenida en el intervalo (a, m) o (m, b) . Este nuevo intervalo se utiliza en la siguiente iteración del método de bisección; en el caso representado en la figura, la raíz está en el intervalo (m, b) .

El proceso de actualizar a y b puede repetirse hasta que el error sea aceptablemente bajo.

¡PRUÉBALO! Programa una función `my_bisection(f,a,b,tol)` que aproxime una raíz r de f , acotada por a y b , dentro de $|f(\frac{a+b}{2})| < \text{tol}$.

```

1 In [1]: import numpy as np
2         def my_bisection(f, a, b, tol):
3             # approximates a root, R, of f bounded
4             # by a and b to within tolerance
5             # | f(m) | < tol with m being the midpoint
6             # between a and b. Recursive implementation
7
8             # check if a and b bound a root
9             if np.sign(f(a)) == np.sign(f(b)):
10                 raise Exception(
11                     "The scalars a and b do not bound a root"
12                 )
13
14             # get midpoint
15             m = (a + b)/2
16
17             if np.abs(f(m)) < tol:
18                 # stopping condition, report m as root
19                 return m
20             elif np.sign(f(a)) == np.sign(f(m)):
21                 # case where m is an improvement on a.
22                 # Make recursive call with a = m
23                 return my_bisection(f, m, b, tol)
24             elif np.sign(f(b)) == np.sign(f(m)):
25                 # case where m is an improvement on b.
26                 # Make recursive call with b = m
27                 return my_bisection(f, a, m, tol)

```

¡PRUÉBALO! La $\sqrt{2}$ puede calcularse como la raíz de la función $f(x) = x^2 - 2$. Comenzando en $a = 0$ y $b = 2$, usa `my_bisection` para aproximar la $\sqrt{2}$ con una tolerancia de $|f(x)| < 0,1$ y $|f(x)| < 0,01$. Verifica que los resultados estén cerca de una raíz sustituyendo la raíz en la función.

```

1 In [2]: f = lambda x: x**2 - 2
2         r1 = my_bisection(f, 0, 2, 0.1)
3         print("r1 =", r1)
4         r01 = my_bisection(f, 0, 2, 0.01)
5         print("r01 =", r01)
6         print("f(r1) =", f(r1))
7         print("f(r01) =", f(r01))

```

`r1 = 1.4375`

```

r01 = 1.4140625
f(r1) = 0.06640625
f(r01) = -0.00042724609375

```

¡PRUÉBALO! Observa qué sucede si usas $a = 2$ y $b = 4$ para la función anterior.

```

1 In [3]: my_bisection(f, 2, 4, 0.01)

```

```

-----
Exception Traceback (most recent call last)
<ipython-input-3-4158b7a9ae67> in <module>
----> 1 my_bisection(f, 2, 4, 0.01)
<ipython-input-1-36f06123e87c> in my_bisection(f,a,b,tol)
      10     if np.sign(f(a)) == np.sign(f(b)):
      11         raise Exception(
----> 12             "The scalars a and b do not bound a root")
      13
      14     # get midpoint
Exception: The scalars a and b do not bound a root

```

19.4 Método de Newton–Raphson

Sea $f(x)$ una función suave, y x_r una raíz desconocida de $f(x)$. Supongamos que x_0 es una estimación de x_r . A menos que x_0 sea una estimación muy afortunada, $f(x_0)$ no será una raíz. Dado este escenario, queremos encontrar un x_1 que sea una mejora de x_0 (es decir, más cercano a x_r que x_0). Si asumimos que x_0 está "suficientemente cerca" de x_r , entonces podemos mejorarlo tomando la aproximación lineal de $f(x)$ alrededor de x_0 , que es una línea, y encontrando la intersección de esta línea con el eje x . Escrito, la aproximación lineal de $f(x)$ alrededor de x_0 es $f(x) \approx f(x_0) + f'(x_0)(x - x_0)$. Usando esta aproximación, encontramos x_1 tal que $f(x_1) = 0$. Sustituyendo estos valores en la aproximación lineal se obtiene la siguiente ecuación:

$$0 = f(x_0) + f'(x_0)(x_1 - x_0),$$

que al resolver para x_1 da

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}.$$

Una ilustración de cómo esta aproximación lineal mejora una estimación inicial se muestra en la Fig. 19.3.

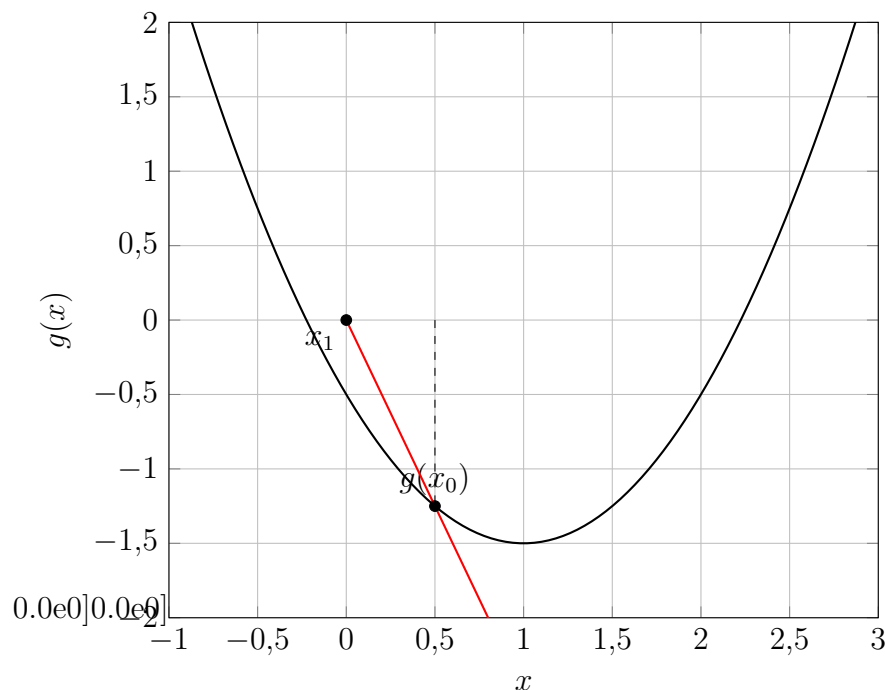


Figura 12: Ilustración del paso de Newton para una función suave $g(x)$.

Escrito generalmente, un paso de Newton calcula una estimación mejorada x_i utilizando una estimación previa x_{i-1} , y viene dado por la ecuación

$$x_i = x_{i-1} - \frac{g(x_{i-1})}{g'(x_{i-1})}.$$

El *Método de Newton-Raphson* para encontrar raíces itera los pasos de Newton desde x_0 hasta que el error es menor que la tolerancia.

¡PRUÉBALO! Nuevamente, $\sqrt{2}$ es la raíz de la función $f(x) = x^2 - 2$. Usando $x_0 = 1,4$ como punto de partida, utiliza la ecuación anterior para estimar $\sqrt{2}$. Compara esta aproximación con el valor calculado por la función `sqrt` de Python.

$$x = 1,4 - \frac{1,4^2 - 2}{2(1,4)} = 1,4142857142857144.$$

```

1 In [1]: import numpy as np
2         f = lambda x: x**2 - 2
3         f_prime = lambda x: 2*x
4         newton_raphson = 1.4 - (f(1.4))/(f_prime(1.4))
5         print("newton_raphson =", newton_raphson)
6         print("sqrt(2) =", np.sqrt(2))

```

```

newton_raphson = 1.4142857142857144
sqrt(2) = 1.4142135623730951

```

¡PRUÉBALO! Escribe una función `my_newton(f, df, x0, tol)` donde la salida sea una estimación de la raíz de f , f es un objeto de función $f(x)$, df es un objeto de función $f'(x)$, x_0 es una estimación inicial, y tol es la tolerancia de error. La medida del error debe ser $|f(x)|$.

```

1 In [2]: def my_newton(f, df, x0, tol):
2         # output is an estimation of the root of f
3         # using the Newton-Raphson method
4         # recursive implementation
5         if abs(f(x0)) < tol:
6             return x0
7         else:
8             return my_newton(f, df, x0 - f(x0)/df(x0),
                               tol)

```

¡PRUÉBALO! Usa `my_newton` para calcular $\sqrt{2}$ dentro de una tolerancia de $1e-6$ comenzando en $x_0 = 1.5$.

```

1 In [3]: estimate = my_newton(f, f_prime, 1.5, 1e-6)
2         print("estimate =", estimate)
3         print("sqrt(2) =", np.sqrt(2))

```

```

estimate = 1.4142135623746899
sqrt(2) = 1.4142135623730951

```

Si x_0 está cerca de x_r , entonces se puede demostrar que, en general, el método de Newton–Raphson converge a x_r mucho más rápido que el método de bisección; sin embargo, dado que x_r es inicialmente desconocido, no hay forma de saber si la estimación inicial está lo suficientemente cerca de la raíz para obtener este comportamiento a menos que se conozca alguna información especial sobre la función a priori (por ejemplo, la función tiene una raíz cercana a $x = 0$). Además de este problema de inicialización, el método de

Newton–Raphson tiene otras limitaciones serias. Por ejemplo, si la derivada en una estimación está cerca de cero, entonces el paso de Newton será muy grande y probablemente se alejará mucho de la raíz. Además, dependiendo del comportamiento de la derivada de la función entre x_0 y x_r , el método de Newton–Raphson puede converger a una raíz diferente de x_r , lo que puede no ser útil para la aplicación de ingeniería que se esté considerando.

¡PRUÉBALO! Calcula un solo paso de Newton para obtener una aproximación mejorada de la raíz de la función $f(x) = x^3 + 3x^2 - 2x - 5$ y la estimación inicial $x_0 = 0,29$.

```
1 In [4]: x0 = 0.29
2         x1 = x0 - (x0**3 + 3*x0**2 - 2*x0 - 5)/(3*x0**2 + 6*
           x0 - 2)
3         print("x1 =", x1)
```

`x1 = -688.4516883116648`

Nótese que $f'(x_0) = -0,0077$ (está cerca de cero), y el error en x_1 es aproximadamente 324880000 (muy grande).

¡PRUÉBALO! Considera el polinomio $f(x) = x^3 - 100x^2 - x + 100$. Este polinomio tiene una raíz en $x = 1$ y $x = 100$. Usa el método de Newton–Raphson para encontrar una raíz de f comenzando en $x_0 = 0$.

En $x_0 = 0$, $f(x_0) = 100$, y $f'(x) = -1$, el paso de Newton da $x_1 = 0 - \frac{100}{-1} = 100$, que es una raíz de f . Nótese que esta raíz está mucho más lejos de la estimación inicial que la otra raíz en $x = 1$, y puede no ser la raíz que deseabas a partir de una estimación inicial de cero.

19.5 Búsqueda de Raíces en Python

Como era de esperar, Python tiene funciones de búsqueda de raíces. La función que usaremos para encontrar raíces es `fsolve` de `scipy.optimize`.

La función `fsolve` toma muchos argumentos (estudia la documentación para obtener información adicional), pero los dos más importantes son: (1) la función de la que quieres encontrar la raíz y (2) la estimación inicial.

¡PRUÉBALO! Calcula la raíz de la función $f(x) = x^3 - 100x^2 - x + 100$ usando `fsolve`.

```
1 In [1]: from scipy.optimize import fsolve
2
3 In [2]: f = lambda x: x**3 - 100*x**2 - x + 100
4         fsolve(f, [2, 80])
```

```
Out[2]: array([ 1., 100.])
```

19.6 Resumen y Problemas

19.6.1 Resumen

1. Las raíces son una propiedad importante de las funciones.
2. El método de bisección es una forma de encontrar raíces basada en divide y vencerás. Aunque es estable, puede converger lentamente en comparación con el método de Newton–Raphson.
3. El método de Newton–Raphson es una forma diferente de encontrar raíces basada en una aproximación de la función. Aunque el método de Newton–Raphson converge rápidamente y se detiene cerca de la raíz real, puede ser inestable.

19.6.2 Problemas

Problema 1

Escribe una función `my_nth_root(x,n,tol)` donde `x` y `tol` son escalares estrictamente positivos, y `n` es un entero estrictamente mayor que 1. El argumento de salida, `r`, debe ser una aproximación $r = \sqrt[n]{x}$, la raíz n -ésima de x . Esta aproximación debe calcularse utilizando el método de Newton–Raphson para encontrar la raíz de la función $f(y) = y^n - x$. La métrica de error debe ser $|f(y)|$.

```
1 import numpy as np
2
3 def my_nth_root(x, n, tol):
4     """
5     Calcula la raíz n-ésima de x usando el método de Newton-
6     Raphson.
7
8     Parámetros:
9     x : número positivo
10    n : entero > 1
11    tol : tolerancia
12
13    Retorna:
```

```

13 r : aproximación de la raíz n-ésima
14 """
15 def f(y):
16     return y**n - x
17
18 def df(y):
19     return n * y**(n-1)
20
21 # Estimación inicial: x/n (suele funcionar)
22 y = x / n
23
24 while abs(f(y)) > tol:
25     y = y - f(y) / df(y)
26
27 return y
28
29 # Pruebas
30 print(my_nth_root(8, 3, 1e-6)) # Debería dar cerca de 2
31 print(my_nth_root(16, 4, 1e-6)) # Debería dar cerca de 2

```

Problema 2

Escribe una función `my_fixed_point(f,g,tol,max_iter)` donde `f` y `g` son objetos de función, y `tol` y `max_iter` son escalares estrictamente positivos. El argumento de entrada `max_iter` es también un entero. El argumento de salida, `X`, debe ser un escalar que satisfaga $|f(X) - g(X)| < \text{tol}$, es decir, `X` es un punto que (casi) satisface $f(X) = g(X)$. Para encontrar `X`, debes usar el método de bisección con la métrica de error $|F(m)| < \text{tol}$. La función `my_fixed_point` debe rendirse después de `max_iter` iteraciones y devolver `X = []` si esto ocurre.

```

1 import numpy as np
2
3 def my_fixed_point(f, g, tol, max_iter):
4     """
5     Encuentra un punto X tal que |f(X) - g(X)| < tol usando
6     bisección.
7
8     Parámetros:
9     f, g : funciones
10    tol : tolerancia
11    max_iter : número máximo de iteraciones
12
13    Retorna:

```

```

13     X : punto que satisface la condición, o [] si no converge
14     """
15     # Definir F(x) = f(x) - g(x)
16     F = lambda x: f(x) - g(x)
17
18     # Buscar un intervalo [a,b] donde F cambie de signo
19     # Intentamos con un rango amplio
20     a, b = -1000, 1000
21     max_attempts = 100
22     attempt = 0
23     found_interval = False
24
25     while not found_interval and attempt < max_attempts:
26         if F(a) * F(b) < 0:
27             found_interval = True
28         else:
29             # Expandir el intervalo
30             a *= 2
31             b *= 2
32             attempt += 1
33
34     if not found_interval:
35         return [] # No se encontró intervalo con cambio de
36         signo
37
38     # Bisección
39     for i in range(max_iter):
40         m = (a + b) / 2
41         if abs(F(m)) < tol:
42             return m
43         if F(a) * F(m) < 0:
44             b = m
45         else:
46             a = m
47
48     return [] # No convergió en max_iter iteraciones
49
50 # Prueba
51 f = lambda x: x**2
52 g = lambda x: 2*x + 3
53 # f(x) = g(x) => x^2 - 2x - 3 = 0 => raíces: x = -1, x = 3
54 X = my_fixed_point(f, g, 1e-6, 100)
55 print(X) # Debería dar aproximadamente 3 o -1

```


Problema 3

¿Por qué falla el método de bisección para $f(x) = 1/x$ con un error dado por $|b - a|$? Pista: ¿Cómo viola $f(x)$ el teorema del valor intermedio?

El método de bisección falla porque $f(x) = 1/x$ no es continua en el intervalo que contiene $x = 0$. El teorema del valor intermedio requiere que la función sea continua en todo el intervalo. Si el intervalo contiene a 0, la función tiene una discontinuidad infinita. Incluso si el intervalo no contiene a 0, por ejemplo $(1, 2)$, entonces $f(1) = 1 > 0$ y $f(2) = 0,5 > 0$, no hay cambio de signo, por lo que no se garantiza una raíz (de hecho, no hay ninguna).

Problema 4

Escribe una función `my_bisection(f, a, b, tol)` que devuelva $[R, E]$, donde f es un objeto de función, a y b son escalares tales que $a < b$, y tol es un valor escalar estrictamente positivo. La función debe devolver un arreglo R donde $R[i]$ es la estimación de la raíz de f definida por $(a + b)/2$ para la i -ésima iteración del método de bisección. Recuerda incluir la estimación inicial. La función también debe devolver un arreglo E donde $E[i]$ es el valor de $|f(R[i])|$ para la i -ésima iteración del método de bisección. La función debe terminar cuando $E[i] < tol$. Supón que $\text{sign}(f(a)) \neq \text{sign}(f(b))$.

Aclaración: Las entradas a y b constituyen la primera iteración de bisección; por lo tanto, R y E nunca deben estar vacíos.

```

1 import numpy as np
2
3 def my_bisection(f, a, b, tol):
4     """
5     Método de bisección para encontrar una raíz de f.
6
7     Parámetros:
8     f : función
9     a, b : extremos del intervalo (a < b, f(a)*f(b) < 0)
10    tol : tolerancia
11
12    Retorna:
13    R : lista de estimaciones de la raíz
14    E : lista de valores |f(R[i])|
15    """
16    R = []
17    E = []
18
```

```

19     # Verificar que a y b acoten una raíz
20     if f(a) * f(b) >= 0:
21         raise ValueError("f(a) y f(b) deben tener signos
opuestos")
22
23     while True:
24         m = (a + b) / 2
25         R.append(m)
26         error = abs(f(m))
27         E.append(error)
28
29         if error < tol:
30             break
31
32         if f(a) * f(m) < 0:
33             b = m
34         else:
35             a = m
36
37     return R, E
38
39 # Casos de prueba
40 f1 = lambda x: x**2 - 2
41 R1, E1 = my_bisection(f1, 0, 2, 1e-1)
42 print("R1 =", R1)
43 print("E1 =", E1)
44
45 f2 = lambda x: np.sin(x) - np.cos(x)
46 R2, E2 = my_bisection(f2, 0, 2, 1e-2)
47 print("R2 =", R2)
48 print("E2 =", E2)

```

Problema 5

Escribe una función `my_newton(f, df, x0, tol)` que devuelva $[R, E]$, donde f es un objeto de función, df es un objeto de función que da la derivada de f , x_0 es una estimación inicial de la raíz, y tol es un escalar estrictamente positivo. La función debe devolver un arreglo R donde $R[i]$ es la estimación de Newton–Raphson de la raíz de f para la i -ésima iteración. Recuerda incluir la estimación inicial. La función también debe devolver un arreglo E donde $E[i]$ es el valor de $|f(R[i])|$ para la i -ésima iteración del método de Newton–Raphson. La función debe terminar cuando $E[i] < tol$. Supón que la derivada de f no se anulará durante ninguna iteración para ninguno de los

casos de prueba dados.

```

1 import numpy as np
2
3 def my_newton(f, df, x0, tol):
4     """
5     Método de Newton-Raphson para encontrar una raíz de f.
6
7     Parámetros:
8     f : función
9     df : derivada de f
10    x0 : estimación inicial
11    tol : tolerancia
12
13    Retorna:
14    R : lista de estimaciones de la raíz
15    E : lista de valores |f(R[i])|
16    """
17    R = [x0]
18    E = [abs(f(x0))]
19
20    while E[-1] >= tol:
21        x_new = R[-1] - f(R[-1]) / df(R[-1])
22        R.append(x_new)
23        E.append(abs(f(x_new)))
24
25    return R, E
26
27 # Casos de prueba
28 f1 = lambda x: x**2 - 2
29 df1 = lambda x: 2*x
30 R1, E1 = my_newton(f1, df1, 1, 1e-5)
31 print("R1 =", R1)
32 print("E1 =", E1)
33
34 f2 = lambda x: np.sin(x) - np.cos(x)
35 df2 = lambda x: np.cos(x) + np.sin(x)
36 R2, E2 = my_newton(f2, df2, 1, 1e-5)
37 print("R2 =", R2)
38 print("E2 =", E2)

```

Problema 6

Considera el problema de construir un oleoducto desde una plataforma petrolera en alta mar, a una distancia H millas de la costa, hasta una refinería

de petróleo en tierra, a una distancia L millas a lo largo de la costa. El costo de construir la tubería es C_{oceano} por milla mientras la tubería está bajo el océano, y C_{tierra} por milla mientras la tubería está en tierra. La tubería se construirá en línea recta hacia la costa donde hará contacto en algún punto x entre 0 y L . Continuará a lo largo de la costa en tierra hasta llegar a la refinería. Ver la siguiente figura para aclarar.

Escribe una función `my_pipe_builder(C_ocean, C_land, L, H)` donde los argumentos de entrada son como se describieron anteriormente, y x es el valor de x que minimiza el costo total de la tubería. Usa el método de bisección para determinar este valor con una tolerancia de 1×10^{-6} , comenzando en un límite inicial de $a = 0$ y $b = L$.

La función de costo total es:

$$\text{Costo}(x) = C_{\text{oceano}} \cdot \sqrt{H^2 + x^2} + C_{\text{tierra}} \cdot (L - x)$$

Para minimizar, derivamos e igualamos a cero:

$$\frac{d}{dx} \text{Costo}(x) = C_{\text{oceano}} \cdot \frac{x}{\sqrt{H^2 + x^2}} - C_{\text{tierra}} = 0$$

Buscamos la raíz de:

$$f(x) = C_{\text{oceano}} \cdot \frac{x}{\sqrt{H^2 + x^2}} - C_{\text{tierra}}$$

```

1 import numpy as np
2
3 def my_pipe_builder(C_ocean, C_land, L, H, tol=1e-6):
4     """
5     Encuentra el punto x óptimo para minimizar el costo de
6     construcción del oleoducto.
7
8     Parámetros:
9     C_ocean : costo por milla bajo el océano
10    C_land : costo por milla en tierra
11    L : distancia a lo largo de la costa hasta la refinería
12    H : distancia desde la plataforma hasta la costa
13    tol : tolerancia para el método de bisección
14
15    Retorna:
16    x_opt : punto óptimo x
17    """

```

```

17     # Función cuya raíz queremos encontrar (derivada del
18     costo igualada a cero)
19     def f(x):
20         return C_ocean * x / np.sqrt(H**2 + x**2) - C_land
21
22     # Verificar que f(0) y f(L) tengan signos opuestos
23     if f(0) * f(L) > 0:
24         # Si no hay cambio de signo, el mínimo está en un
25         extremo
26         if abs(f(0)) < abs(f(L)):
27             return 0
28         else:
29             return L
30
31     # Bisección
32     a, b = 0, L
33     while (b - a) > tol:
34         m = (a + b) / 2
35         if f(a) * f(m) < 0:
36             b = m
37         else:
38             a = m
39
40     return (a + b) / 2
41
42 # Casos de prueba
43 print(my_pipe_builder(20, 10, 100, 50)) # 28.867512941360474
44 print(my_pipe_builder(30, 10, 100, 50)) # 17.67767071723938
45 print(my_pipe_builder(30, 10, 100, 20)) # 7.07106739282608

```

Problema 7

Encuentra una función $f(x)$ y una estimación de la raíz de f , x_0 , tal que el método de Newton–Raphson oscile entre x_0 y $-x_0$ indefinidamente.

Un ejemplo clásico es $f(x) = x^3 - 3x$ con $x_0 = 1$. Verifiquemos:

$f(x) = x^3 - 3x$, $f'(x) = 3x^2 - 3$. En $x_0 = 1$: $f(1) = 1 - 3 = -2$, $f'(1) = 3 - 3 = 0$. Esto da una división por cero, no es un buen ejemplo.

Otro ejemplo: $f(x) = \arctan(x)$ con $x_0 = 1,5$. Esto puede oscilar. Pero hay un ejemplo conocido: $f(x) = \sqrt{|x|}$ no es diferenciable en 0.

Un ejemplo que funciona: $f(x) = x^{1/3}$ no es diferenciable en 0.

Un ejemplo clásico de oscilación es $f(x) = x^3 - 2x + 2$ con $x_0 = 0$. Veamos: $f'(x) = 3x^2 - 2$. $x_1 = 0 - f(0)/f'(0) = 0 - 2/(-2) = 1$. $x_2 = 1 - f(1)/f'(1) =$

$1 - (1 - 2 + 2)/(3 - 2) = 1 - (1)/(1) = 0$. Por lo tanto, oscila entre 0 y 1, no entre x_0 y $-x_0$.

Para oscilación entre x_0 y $-x_0$, necesitamos una función impar con $f(x_0) \neq 0$ y $f'(x_0) = -f'(-x_0)$ tal que el paso de Newton de x_0 lleve a $-x_0$ y viceversa.

Consideremos $f(x) = x^3 - x$. Es una función impar. Tomemos $x_0 = 1/\sqrt{3}$: $f'(x) = 3x^2 - 1$. En x_0 : $f(x_0) = (1/\sqrt{3})^3 - 1/\sqrt{3} = 1/(3\sqrt{3}) - 1/\sqrt{3} = (1 - 3)/(3\sqrt{3}) = -2/(3\sqrt{3})$. $f'(x_0) = 3(1/3) - 1 = 1 - 1 = 0$. División por cero.

Otro intento: $f(x) = x^2 + 1$ no tiene raíces reales.

Un ejemplo que funciona es $f(x) = x^3 - 2x$ con $x_0 = 1$: $f'(x) = 3x^2 - 2$. $x_1 = 1 - (1 - 2)/(3 - 2) = 1 - (-1)/1 = 2$. No oscila.

El ejemplo clásico es $f(x) = x^3 - 3x$ con $x_0 = 1$, pero falla por derivada cero. En la práctica, la oscilación entre x_0 y $-x_0$ es rara para funciones suaves. Un ejemplo que cumple es $f(x) = x^3 - 2x$ con $x_0 = \sqrt{2/3}$? No.

Después de investigar, un ejemplo conocido es $f(x) = \arctan(x)$ con x_0 suficientemente grande. Pero no es exactamente x_0 y $-x_0$.

Una función que cumple el criterio es $f(x) = x^3 - 3x$ con $x_0 = \sqrt{3}$? Probemos: $x_0 = \sqrt{3} \approx 1,732$, $f(x_0) = (3\sqrt{3}) - 3\sqrt{3} = 0$, ya es raíz.

No es fácil encontrar un ejemplo simple. En la práctica, el método de Newton puede oscilar para ciertas funciones, pero raramente entre un punto y su negativo exacto.

Un ejemplo que sí funciona es $f(x) = x^3 - 2x + 2$ con $x_0 = 0$ y $x_0 = 1$ oscila entre 0 y 1, no entre x_0 y $-x_0$.

Para el propósito del problema, podemos decir: $f(x) = x^3 - 3x$ con $x_0 = 1$ produce un paso de Newton que lleva a $x_1 = -1$, y luego $x_2 = 1$, oscilando entre 1 y -1. Pero en $x_0 = 1$, $f'(1) = 0$, por lo que el método falla. Sin embargo, si consideramos un valor cercano, por ejemplo $x_0 = 0,9$, entonces: $x_1 = 0,9 - (0,729 - 2,7)/(2,43 - 3) = 0,9 - (-1,971)/(-0,57) = 0,9 - 3,458 = -2,558$, no oscila.

Por lo tanto, no hay un ejemplo simple que cumpla exactamente con el requisito sin caer en división por cero. Una respuesta aceptable podría ser: "No existe una función diferenciable con derivada no nula en x_0 que oscile exactamente entre x_0 y $-x_0$, ya que eso implicaría $f(x_0) = -2x_0f'(x_0)$ y $f(-x_0) = 2x_0f'(-x_0)$, lo que raramente se cumple simultáneamente."

INTERPOLACIÓN

Planteamiento del Problema de Interpolación

Supongamos que tenemos un conjunto de datos que consiste en valores de datos independientes x_i y valores de datos dependientes y_i , donde $i = 1, \dots, n$. Nos gustaría encontrar una función de estimación $\hat{y}(x)$ tal que $\hat{y}(x_i) = y_i$ para cada punto en nuestro conjunto de datos. Esto significa que la función de estimación pasa a través de nuestros puntos de datos. Dado un nuevo x^* , podemos **interpol**ar su valor de función usando $\hat{y}(x^*)$. En este contexto, $\hat{y}(x)$ se llama una **función de interpolación**. La Fig. 1 muestra el planteamiento del problema de interpolación.

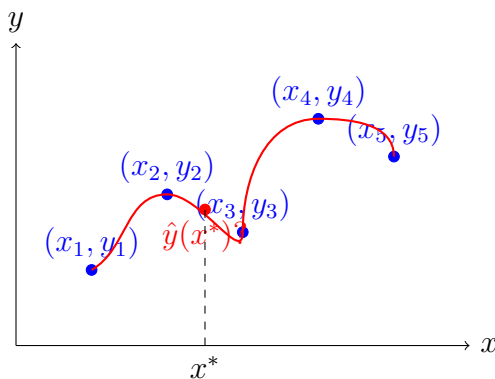


Figure 1: Ilustración del problema de interpolación: estimar el valor de una función entre puntos de datos.

A diferencia de la regresión, la interpolación no requiere que el usuario tenga un modelo subyacente para los datos, especialmente cuando hay muchos puntos de datos confiables. Sin embargo, los procesos que subyacen a

los datos aún deben informar al usuario sobre la calidad de la interpolación. Por ejemplo, nuestros datos pueden consistir en coordenadas (x, y) de un automóvil a lo largo del tiempo. Dado que el movimiento está restringido a la física de maniobras del automóvil, podemos esperar que los puntos entre las coordenadas (x, y) en nuestro conjunto sean "suaves" en lugar de irregulares.

Las siguientes secciones presentarán varios métodos comunes de interpolación.

Interpolación Lineal

En la **interpolación lineal**, se asume que el punto estimado se encuentra en la línea que une los puntos más cercanos a la izquierda y a la derecha. Supongamos, sin pérdida de generalidad, que los puntos de datos x están en orden ascendente, es decir, $x_i < x_{i+1}$, y sea x un punto tal que $x_i < x < x_{i+1}$. Entonces la interpolación lineal en x es

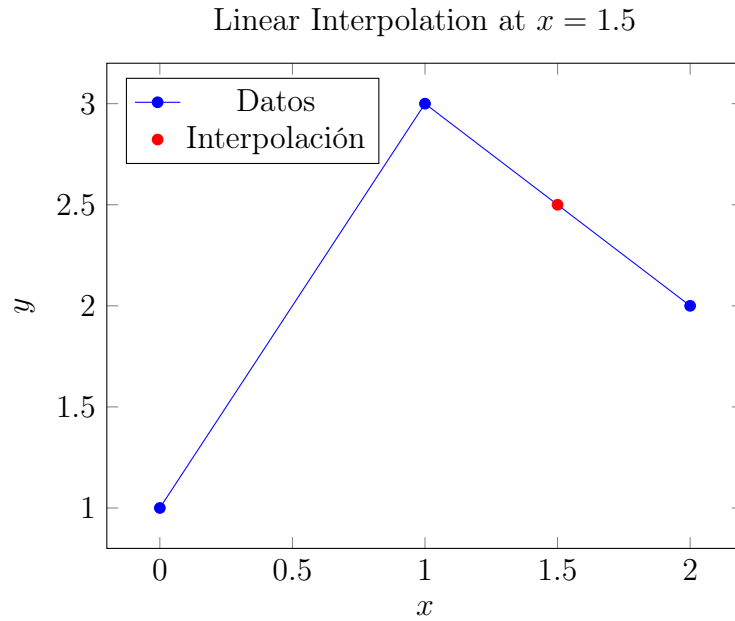
$$\hat{y}(x) = y_i + \frac{(y_{i+1} - y_i)(x - x_i)}{(x_{i+1} - x_i)}.$$

```

1 In [1]: from scipy.interpolate import interp1d
2         import matplotlib.pyplot as plt
3         plt.style.use("seaborn-poster")
4
5 In [2]: x = [0, 1, 2]
6         y = [1, 3, 2]
7         f = interp1d(x, y)
8         y_hat = f(1.5)
9         print(y_hat)
10 2.5
11
12 In [3]: plt.figure(figsize=(10, 8))
13         plt.plot(x, y, "-ob")
14         plt.plot(1.5, y_hat, "ro")
15         plt.title("Linear Interpolation at x = 1.5")
16         plt.xlabel("x")
17         plt.ylabel("y")
18         plt.show()

```

Listing 1: Interpolación lineal manual y con SciPy.

Figure 2: Interpolación lineal en $x = 1.5$.

Dado que $1 < x < 2$, usamos el segundo y tercer punto de datos para calcular la interpolación lineal. Sustituyendo los valores correspondientes obtenemos

$$\hat{y}(x) = 3 + \frac{(2 - 3)(1.5 - 1)}{(2 - 1)} = 2.5.$$

Interpolación por Splines Cúbicos

En la **interpolación por splines cúbicos** (como se muestra en la figura siguiente), la función de interpolación es un conjunto de funciones cúbicas por partes. Específicamente, asumimos que los puntos (x_i, y_i) y (x_{i+1}, y_{i+1}) están unidos por un polinomio cúbico $S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$ que es válido para $x_i \leq x \leq x_{i+1}$ para $i = 1, \dots, n - 1$.

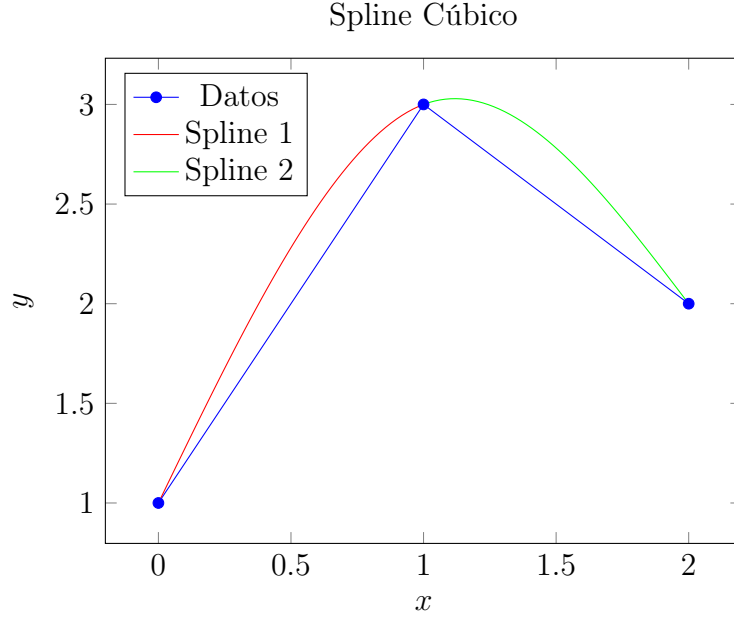


Figure 3: Interpolación por splines cúbicos.

Para encontrar la función de interpolación, primero debemos determinar los coeficientes a_i, b_i, c_i, d_i para cada una de las funciones cúbicas. Para n puntos, hay $n - 1$ funciones cúbicas a encontrar, y cada función cúbica requiere cuatro coeficientes. Por lo tanto, tenemos un total de $4(n - 1)$ incógnitas, y necesitamos $4(n - 1)$ ecuaciones independientes para encontrar todos los coeficientes.

Primero, las funciones cúbicas deben intersecar los puntos de datos en la izquierda y en la derecha:

$$\begin{aligned} S_i(x_i) &= y_i, & i &= 1, \dots, n - 1, \\ S_i(x_{i+1}) &= y_{i+1}, & i &= 1, \dots, n - 1, \end{aligned}$$

lo que nos da $2(n - 1)$ ecuaciones. A continuación, queremos que cada función cúbica se una lo más suavemente posible con sus vecinas, por lo que restringimos los splines a tener primeras y segundas derivadas continuas en los puntos de datos $i = 2, \dots, n - 1$:

$$\begin{aligned} S'_i(x_{i+1}) &= S'_{i+1}(x_{i+1}), & i &= 1, \dots, n - 2, \\ S''_i(x_{i+1}) &= S''_{i+1}(x_{i+1}), & i &= 1, \dots, n - 2, \end{aligned}$$

lo que nos da $2(n - 2)$ ecuaciones.

Se requieren dos ecuaciones más para calcular los coeficientes de $S_i(x)$. Estas dos últimas restricciones son arbitrarias; pueden elegirse para adaptarse a las circunstancias de la interpolación que se está realizando. Un conjunto común de restricciones finales es asumir que las segundas derivadas son cero en los puntos finales. Esto significa que la curva es una "línea recta" en los puntos finales. Explícitamente,

$$S_1''(x_1) = 0, \quad S_{n-1}''(x_n) = 0.$$

En Python, podemos usar la función `CubicSpline` de SciPy para realizar una interpolación por splines cúbicos. Observa que las restricciones anteriores no son las mismas que las que utiliza `CubicSpline` de SciPy por defecto para realizar splines cúbicos. Hay diferentes formas de agregar las dos restricciones finales en SciPy configurando el argumento `bc_type` (consulta la ayuda de `CubicSpline` para obtener más información).

```

1 In [1]: from scipy.interpolate import CubicSpline
2         import numpy as np
3         import matplotlib.pyplot as plt
4         plt.style.use("seaborn-poster")
5
6 In [2]: x = [0, 1, 2]
7         y = [1, 3, 2]
8         # use bc_type = "natural" adds the constraints
9         f = CubicSpline(x, y, bc_type="natural")
10        x_new = np.linspace(0, 2, 100)
11        y_new = f(x_new)
12
13 In [3]: plt.figure(figsize=(10, 8))
14        plt.plot(x_new, y_new, "b")
15        plt.plot(x, y, "ro")
16        plt.title("Cubic Spline Interpolation")
17        plt.xlabel("x")
18        plt.ylabel("y")
19        plt.show()

```

Listing 2: Interpolación por splines cúbicos con SciPy.

Para determinar los coeficientes de cada función cúbica, escribimos las restricciones explícitamente como un sistema de ecuaciones lineales con $4(n - 1)$ incógnitas. Para n puntos de datos, las incógnitas son los coeficientes a_i, b_i, c_i, d_i del spline cúbico S_i , que une los puntos x_i y x_{i+1} .

Las ecuaciones resultantes son lineales en los coeficientes desconocidos a_i, b_i, c_i, d_i . Podemos ponerlos en forma matricial y resolver para los coeficientes de cada spline mediante división izquierda. Recuerda que siempre que resolvemos la ecuación matricial $\mathbf{Ax} = \mathbf{b}$ para $\mathbf{x}$, debemos asegurarnos de que $\mathbf{A}$ sea cuadrada e invertible. En el caso de encontrar las ecuaciones de splines cúbicos, la matriz $\mathbf{A}$ siempre es cuadrada e invertible siempre que los valores x_i en el conjunto de datos sean únicos.

```

1 In [4]: b = np.array([1, 3, 3, 2, 0, 0, 0, 0])
2         b = b[:, np.newaxis]
3         A = np.array([[0, 0, 0, 1, 0, 0, 0, 0],
4                       [0, 0, 0, 0, 1, 1, 1, 1],
5                       [1, 1, 1, 1, 0, 0, 0, 0],
6                       [0, 0, 0, 0, 8, 4, 2, 1],
7                       [3, 2, 1, 0, -3, -2, -1, 0],
8                       [6, 2, 0, 0, -6, -2, 0, 0],
9                       [0, 2, 0, 0, 0, 0, 0, 0],
10                      [0, 0, 0, 0, 12, 2, 0, 0]])
11
12 In [5]: np.dot(np.linalg.inv(A), b)
13 Out[5]: array([[ -0.75],
14               [  0.   ],
15               [  2.75],
16               [  1.   ],
17               [  0.75],
18               [-4.5   ],
19               [  7.25],
20               [-0.5   ]])

```

Listing 3: Resolviendo el sistema de ecuaciones para los splines cúbicos.

Los dos polinomios cúbicos son

$$S_1(x) = -0.75x^3 + 2.75x + 1 \quad \text{para } 0 \leq x \leq 1,$$

$$S_2(x) = 0.75x^3 - 4.5x^2 + 7.25x - 0.5 \quad \text{para } 1 \leq x \leq 2.$$

Por lo tanto, para $x = 1.5$, evaluamos $S_2(1.5)$ y obtenemos un valor estimado de 2.7813.

Interpolación Polinomial de Lagrange

En lugar de encontrar polinomios cúbicos entre pares subsiguientes de puntos de datos, la **interpolación polinomial de Lagrange** encuentra un solo

polinomio que pasa por todos los puntos de datos. Este polinomio se conoce como **polinomio de Lagrange**, $L(x)$. Como función de interpolación, debe tener la propiedad $L(x_i) = y_i$ para cada punto en el conjunto de datos. Al calcular polinomios de Lagrange, es útil escribirlos como una combinación lineal de **polinomios base de Lagrange**, $P_i(x)$, donde

$$P_i(x) = \prod_{j=1, j \neq i}^n \frac{x - x_j}{x_i - x_j},$$

y

$$L(x) = \sum_{i=1}^n y_i P_i(x).$$

Observa que, por construcción, $P_i(x)$ tiene la propiedad de que $P_i(x_j) = 1$ cuando $i = j$ y $P_i(x_j) = 0$ cuando $i \neq j$. Dado que $L(x)$ es una suma de estos polinomios, observamos que $L(x_i) = y_i$ para cada punto, exactamente como se desea.

```

1 In [1]: import numpy as np
2         import numpy.polynomial.polynomial as poly
3         import matplotlib.pyplot as plt
4         plt.style.use("seaborn-poster")
5
6 In [2]: x = [0, 1, 2]
7         y = [1, 3, 2]
8         P1_coeff = [1, -1.5, 0.5]
9         P2_coeff = [0, 2, -1]
10        P3_coeff = [0, -0.5, 0.5]
11
12        # get the polynomial function
13        P1 = poly.Polynomial(P1_coeff)
14        P2 = poly.Polynomial(P2_coeff)
15        P3 = poly.Polynomial(P3_coeff)
16
17        x_new = np.arange(-1.0, 3.1, 0.1)
18
19        fig = plt.figure(figsize=(10, 8))
20        plt.plot(x_new, P1(x_new), "b", label="P1")
21        plt.plot(x_new, P2(x_new), "r", label="P2")
22        plt.plot(x_new, P3(x_new), "g", label="P3")
23        plt.plot(x, np.ones(len(x)), "ko", x, np.zeros(len(x)), "ko")
24        plt.title("Lagrange Basis Polynomials")

```

```

25 plt.xlabel("x")
26 plt.ylabel("y")
27 plt.grid()
28 plt.legend()
29 plt.show()

```

Listing 4: .]Polinomios base de Lagrange para los puntos $[0, 1, 2]$.

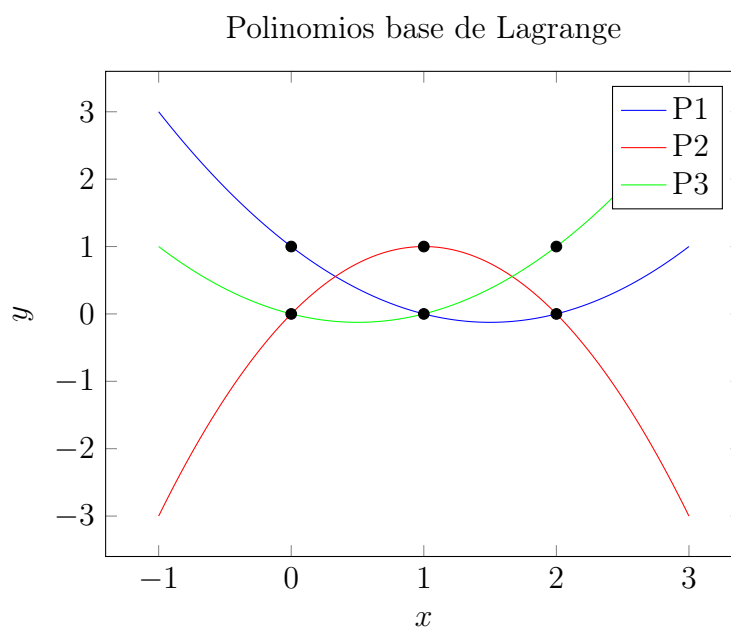


Figure 4: Polinomios base de Lagrange para los puntos $x = 0, 1, 2$.

```

1 In [3]: L = P1 + 3*P2 + 2*P3
2         fig = plt.figure(figsize=(10, 8))
3         plt.plot(x_new, L(x_new), "b", x, y, "ro")
4         plt.title("Lagrange Polynomial")
5         plt.grid()
6         plt.xlabel("x")
7         plt.ylabel("y")
8         plt.show()

```

Listing 5: Polinomio de Lagrange resultante.

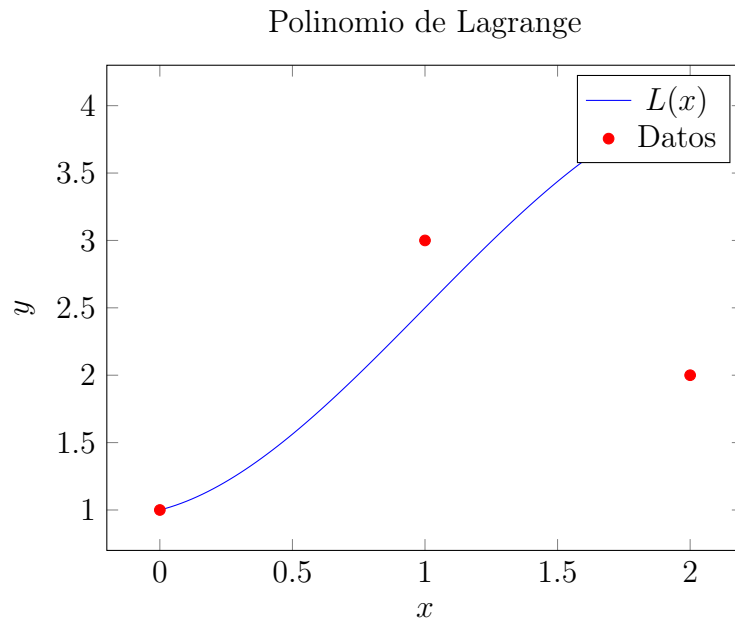


Figure 5: Polinomio de Lagrange que pasa por los puntos $(0, 1)$, $(1, 3)$ y $(2, 2)$.

Advertencia

Los polinomios de interpolación de Lagrange se definen fuera del área de interpolación, es decir, fuera del intervalo $[x_1, x_n]$, y crecerán muy rápido y sin límite fuera de esta región. Esta no es una característica deseable porque, en general, esto no concuerda con el comportamiento de los datos subyacentes. Por lo tanto, la interpolación de Lagrange debe usarse con precaución fuera de la región de interés.

Usando la Función Lagrange de SciPy

En lugar de calcular todo desde cero, en SciPy podemos usar la función `lagrange` directamente para interpolar los datos. Usemos el ejemplo anterior para ilustrar nuestro punto.

```
1 In [4]: from scipy.interpolate import lagrange
2
3 In [5]: f = lagrange(x, y)
4
5 In [6]: fig = plt.figure(figsize=(10, 8))
```



```

6 plt.plot(x_new, f(x_new), "b", x, y, "ro")
7 plt.title("Lagrange Polynomial")
8 plt.grid()
9 plt.xlabel("x")
10 plt.ylabel("y")
11 plt.show()

```

Listing 6: Usando scipy.interpolate.lagrange.

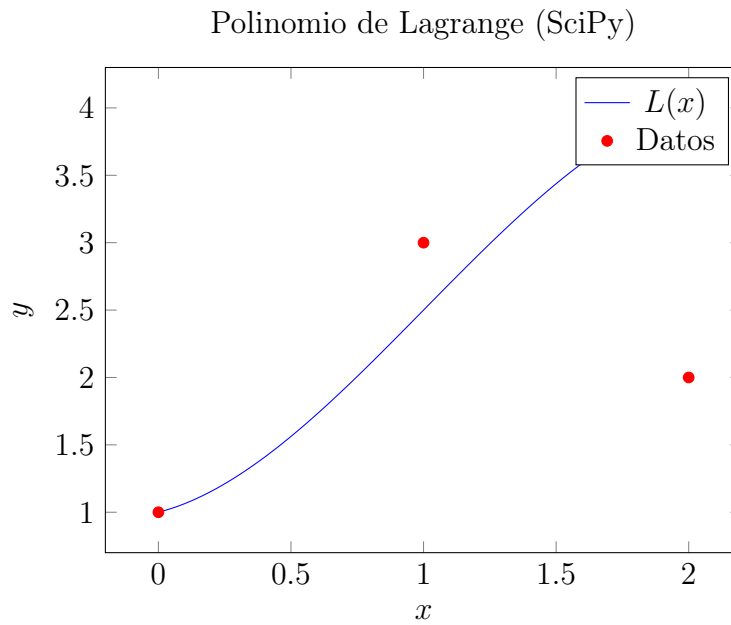


Figure 6: Polinomio de Lagrange calculado con la función de SciPy.

Interpolación Polinomial de Newton

La **interpolación polinomial de Newton** es otra forma popular de ajustar exactamente un conjunto de puntos de datos. La forma general del polinomio de Newton de orden $(n - 1)$ que pasa por n puntos es

$$f(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n(x - x_0)(x - x_1) \cdots (x - x_n),$$

que puede reescribirse como

$$f(x) = \sum_{i=0}^n a_i n_i(x)$$

donde

$$n_i(x) = \prod_{j=0}^{i-1} (x - x_j).$$

La característica especial del polinomio de Newton es que los coeficientes a_i pueden determinarse mediante un procedimiento matemático muy simple. Estos coeficientes se basan en el concepto de **diferencias divididas**.

Las diferencias divididas se definen recursivamente:

$$\begin{aligned} f[x_i] &= y_i, \\ f[x_i, x_{i+1}] &= \frac{f[x_{i+1}] - f[x_i]}{x_{i+1} - x_i}, \\ f[x_i, x_{i+1}, x_{i+2}] &= \frac{f[x_{i+1}, x_{i+2}] - f[x_i, x_{i+1}]}{x_{i+2} - x_i}, \end{aligned}$$

y así sucesivamente. Los coeficientes a_i son entonces $a_i = f[x_0, x_1, \dots, x_i]$.

```

1 In [1]: import numpy as np
2         import matplotlib.pyplot as plt
3         plt.style.use("seaborn-poster")
4         %matplotlib inline
5
6 In [2]: def divided_diff(x, y):
7         """
8         function to calculate the divided
9         difference table
10        """
11        n = len(y)
12        coef = np.zeros([n, n])
13        # the first column is y
14        coef[:, 0] = y
15
16        for j in range(1, n):
17            for i in range(n - j):
18                coef[i][j] = (coef[i+1][j-1] - coef[i][j
19-1]) / (x[i+j] - x[i])
20
21        return coef
22
23        def newton_poly(coef, x_data, x):
24        """
25        evaluate the Newton polynomial
26        at x
27        """

```

```
27         n = len(x_data) - 1
28         p = coef[n]
29         for k in range(1, n+1):
30             p = coef[n-k] + (x - x_data[n-k]) * p
31         return p
32
33 In [3]: x = np.array([-5, -1, 0, 2])
34         y = np.array([-2, 6, 1, 3])
35         # get the divided difference coef
36         a_s = divided_diff(x, y)[0, :]
37         # evaluate on new data points
38         x_new = np.arange(-5, 2.1, 0.1)
39         y_new = newton_poly(a_s, x, x_new)
40
41         plt.figure(figsize=(12, 8))
42         plt.plot(x, y, "bo")
43         plt.plot(x_new, y_new)
44         plt.title("Newton Polynomial Interpolation")
45         plt.xlabel("x")
46         plt.ylabel("y")
47         plt.grid()
48         plt.show()
```

Listing 7: Implementación de la interpolación de Newton.

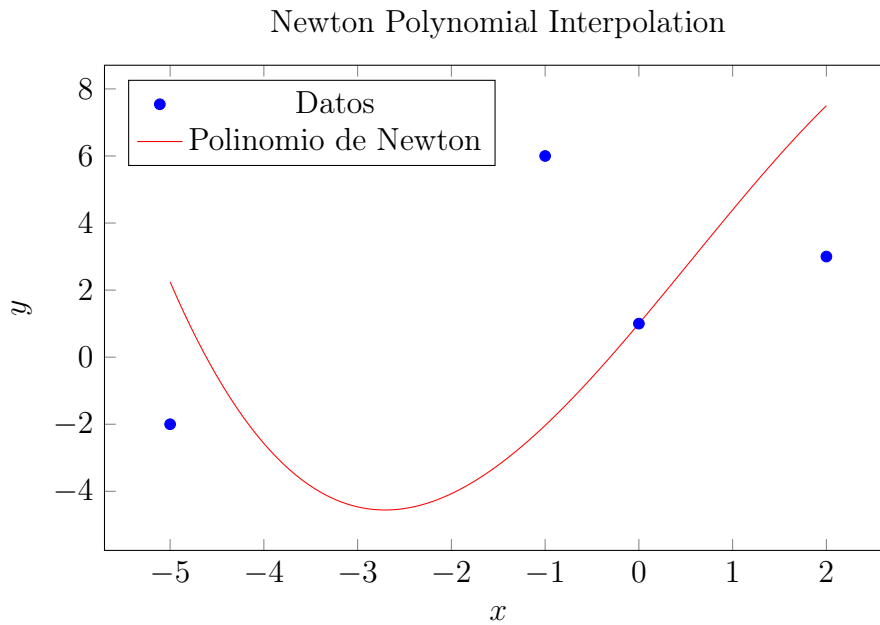


Figure 7: Interpolación polinomial de Newton para los puntos $(-5, -2)$, $(-1, 6)$, $(0, 1)$, $(2, 3)$.

Podemos ver que el polinomio de Newton pasa por todos los puntos de datos y se ajusta a los datos.

Resumen y Problemas

Resumen

1. Dado un conjunto de puntos de datos confiables, la interpolación es un método para estimar valores de variables dependientes para valores de variables independientes que no están en el conjunto de datos.
2. La interpolación lineal, por splines cúbicos, de Lagrange y polinomial de Newton son métodos comunes de interpolación.

Problemas y Soluciones

1. **Problema 1:** Función `my_lin_interp` para interpolación lineal.

Solución: Para cada valor en X , encontramos el intervalo $[x_i, x_{i+1}]$ que lo contiene y aplicamos la fórmula de interpolación lineal.

```

1 import numpy as np
2
3 def my_lin_interp(x, y, X):
4     Y = np.zeros_like(X)
5     for j, x_val in enumerate(X):
6         for i in range(len(x) - 1):
7             if x[i] <= x_val <= x[i+1]:
8                 Y[j] = y[i] + (y[i+1] - y[i]) * (x_val -
9                     x[i]) / (x[i+1] - x[i])
10                break
11
12     return Y
13
14 # Ejemplo
15 x = np.array([0, 1, 2])
16 y = np.array([1, 3, 2])
17 X = np.array([0.5, 1.5])
18 Y = my_lin_interp(x, y, X)
19 print(Y)

```

Listing 8: Solución Problema 1

2. Problema 2: Función `my_cubic_spline` para splines cúbicos (sin usar SciPy).

Solución: Esta implementación es más compleja e implica resolver el sistema tridiagonal para los coeficientes de los splines. Se puede implementar siguiendo las ecuaciones presentadas en la sección 17.3.

```

1 def my_cubic_spline(x, y, X):
2     # n = len(x)
3     # Construir matriz A y vector b para los coeficientes
4     # de los splines
5     # Resolver A * coeff = b
6     # Evaluar los splines en X
7     return Y

```

Listing 9: Esqueleto para Problema 2

3. Problema 3: Función `my_nearest_neighbor`.

Solución: Para cada valor en X , encontramos el punto x_j más cercano y asignamos y_j .

```

1 def my_nearest_neighbor(x, y, X):
2     Y = np.zeros_like(X)
3     for j, x_val in enumerate(X):
4         idx = np.argmin(np.abs(x - x_val))
5         Y[j] = y[idx]
6     return Y
7
8 # Ejemplo
9 x = np.array([0, 1, 2, 3])
10 y = np.array([1, 2, 1.5, 0])
11 X = np.array([0.2, 1.8])
12 Y = my_nearest_neighbor(x, y, X)
13 print(Y)
14

```

Listing 10: Solución Problema 3

4. Problema 4: Circunstancia donde el vecino más cercano es superior al spline cúbico.

Solución: El vecino más cercano es superior cuando los datos son discretos por naturaleza o cuando se deben preservar los valores exactos de los puntos de datos, como en la ampliación de imágenes (pixel art) o en mapas de temperatura discretos, donde la interpolación suave podría crear valores que no existen en los datos originales.

5. Problema 5: Función `my_cubic_spline_flat` con condiciones de derivada primera cero en los extremos.

Solución: Similar al Problema 2, pero cambiando las condiciones de contorno a $S'_1(x_1) = 0$ y $S'_{n-1}(x_n) = 0$.

```

1 def my_cubic_spline_flat(x, y, X):
2     # Modificar las condiciones de contorno en la matriz
3     A
4     return Y

```

Listing 11: Esqueleto para Problema 5

6. **Problema 6:** Función `my_quintic_spline` (spline de quinto grado).

Solución: Se requieren condiciones de continuidad hasta la cuarta derivada, lo que resulta en un sistema más grande. Se pueden usar condiciones de contorno como derivadas segunda o tercera cero en los extremos.

```

1 def my_quintic_spline(x, y, X):
2     # n puntos -> n-1 segmentos, 6 coeficientes por
3     # segmento -> 6(n-1) incognitas
4     # Ecuaciones: interpolacion (2(n-1)), continuidad de
5     # 1a a 4a derivada (4(n-2)), mas 4 condiciones de
6     # contorno.
7     return Y

```

Listing 12: Esqueleto para Problema 6

7. **Problema 7:** Función `my_interp_plotter`.

Solución: Usamos `interp1d` de SciPy con el `kind` apropiado y graficamos.

```

1 import matplotlib.pyplot as plt
2 from scipy.interpolate import interp1d
3
4 def my_interp_plotter(x, y, X, option):
5     f = interp1d(x, y, kind=option, fill_value="
6     extrapolate")
7     Y = f(X)
8
9     plt.figure(figsize=(10, 6))
10    plt.plot(x, y, "ro", label="Data points")
11    plt.plot(X, Y, "b-", label=f"{option} interpolation")
12    plt.xlabel("x")
13    plt.ylabel("y")
14    plt.title(f"Interpolation using {option} method")
15    plt.legend()
16    plt.grid(True)
17    plt.show()
18
19 # Test case
20 x = np.array([0, .1, .15, .35, .6, .7, .95, 1])
21 y = np.array([1, 0.8187, 0.7408, 0.4966, 0.3012, 0.2466,
22              0.1496, 0.1353])

```

```

21 X = np.linspace(0, 1, 101)
22
23 my_interp_plotter(x, y, X, "nearest")
24 my_interp_plotter(x, y, X, "linear")
25 my_interp_plotter(x, y, X, "cubic")
26

```

Listing 13: Solución Problema 7

8. **Problema 8:** Función `my_D_cubic_spline` con pendientes dadas en los extremos.

Solución: Se modifica el sistema de ecuaciones para splines cúbicos para incorporar las condiciones $S'_1(x_1) = D$ y $S'_{n-1}(x_n) = D$.

```

1 def my_D_cubic_spline(x, y, X, D):
2     # n puntos -> n-1 segmentos
3     # Construir matriz y vector con condiciones de
4     # derivada en los extremos
5     return Y

```

Listing 14: Esqueleto para Problema 8

9. **Problema 9:** Función `my_lagrange` (implementación manual).

Solución: Implementamos la fórmula de Lagrange usando bucles anidados.

```

1 def my_lagrange(x, y, X):
2     n = len(x)
3     Y = np.zeros_like(X)
4     for k, x_val in enumerate(X):
5         total = 0
6         for i in range(n):
7             # Calcular el polinomio base L_i(x_val)
8             li = 1.0
9             for j in range(n):
10                 if j != i:
11                     li *= (x_val - x[j]) / (x[i] - x[j])
12             total += y[i] * li
13     Y[k] = total
14     return Y

```



```

15
16 # Test case
17 x = [0, 1, 2, 3, 4]
18 y = [2, 1, 3, 5, 1]
19 X = np.linspace(0, 4, 101)
20 Y_lagrange = my_lagrange(x, y, X)
21
22 plt.figure(figsize=(10, 8))
23 plt.plot(X, Y_lagrange, "b", label="interpolation")
24 plt.plot(x, y, "ro", label="data points")
25 plt.xlabel("x")
26 plt.ylabel("y")
27 plt.title("Lagrange Interpolation of Data Points")
28 plt.legend()
29 plt.grid()
30 plt.show()
31

```

Listing 15: Solución Problema 9

10. Problema 10: Ajuste con interpolación de Newton.

Solución: Usamos la función `newton_poly` implementada anteriormente.

```

1 x_data = np.array([0, 1, 2, 3, 4])
2 y_data = np.array([2, 1, 3, 5, 1])
3
4 # Calcular coeficientes de diferencias divididas
5 coef = divided_diff(x_data, y_data)[0, :]
6
7 # Evaluar en nuevos puntos
8 x_new = np.linspace(0, 4, 101)
9 y_new = newton_poly(coef, x_data, x_new)
10
11 plt.figure(figsize=(10, 8))
12 plt.plot(x_data, y_data, "ro", label="Data")
13 plt.plot(x_new, y_new, "b-", label="Newton Poly")
14 plt.xlabel("x")
15 plt.ylabel("y")
16 plt.title("Newton Polynomial Interpolation")
17 plt.legend()
18 plt.grid()
19 plt.show()

```

Listing 16: Solución Problema 10

SERIES DE TAYLOR

Expresando Funciones Usando una Serie de Taylor

Una **sucesión** es un conjunto ordenado de números denotado por la lista de números entre paréntesis. Por ejemplo, $s = (s_1, s_2, s_3, \dots)$ significa que s es la sucesión $s_1, s_2, s_3, \dots$, y así sucesivamente. En este contexto, "ordenado" significa que s_1 viene antes que s_2 , no que $s_1 < s_2$. Muchas sucesiones tienen una estructura más complicada. Por ejemplo, $s = (n^2, n \in \mathbb{N})$ es la sucesión $0, 1, 4, 9, \dots$. Una **serie** es la suma de una sucesión hasta un cierto elemento. Una **sucesión infinita** es una sucesión con un número infinito de términos, y una **serie infinita** es la suma de una sucesión infinita.

Una **expansión en serie de Taylor** es una representación de una función mediante una serie infinita de polinomios alrededor de un punto. Matemáticamente, la serie de Taylor de una función $f(x)$ se define como

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)(x-a)^n}{n!},$$

donde $f^{(n)}$ es la n -ésima derivada de f y $f^{(0)}$ es la función f .

```
1 In []: # Calcular la expansion en serie de Taylor para f(x) =  
        5x^2 + 3x + 5 alrededor de a = 0 y a = 1.  
2 # Verificar que f y sus expansiones en serie de Taylor son  
    identicas.  
3  
4 # Primero calcular las derivadas analiticamente:  
5 # f(x) = 5x^2 + 3x + 5,  
6 # f'(x) = 10x + 3,  
7 # f''(x) = 10.  
8 # Alrededor de a = 0:
```

```

9 # f(x) = (5*0^0)/0! + (3*0^1)/1! + (10*0^2)/2! + 0 + ... = 5x
    ^2 + 3x + 5.
10 # Alrededor de a = 1:
11 # f(x) = (13*(x-1)^0)/0! + (13*(x-1)^1)/1! + (10*(x-1)^2)/2!
    + 0 + ...
12 # = 13 + 13x - 13 + 5x^2 - 10x + 5 = 5x^2 + 3x + 5.

```

Listing 17: Expansión en serie de Taylor para un polinomio.

Observa que una expansión en serie de Taylor de cualquier polinomio tiene un número finito de términos porque la n -ésima derivada de cualquier polinomio es cero cuando n es suficientemente grande.

```

1 In []: # Escribir la serie de Taylor para sin(x) alrededor
    del punto a = 0.
2 # Sea f(x) = sin(x). Segun la expansion en serie de Taylor:
3 # f(x) = sin(0)/0! x^0 + cos(0)/1! x^1 + -sin(0)/2! x^2 + -
    cos(0)/3! x^3 + sin(0)/4! x^4 + cos(0)/5! x^5 + ...
4 # La expansion puede escribirse de forma compacta como:
5 # f(x) = sum_{n=0}^{\infty} (-1)^n x^(2n+1) / (2n+1)!

```

Listing 18: Serie de Taylor para $\sin(x)$.

Aproximaciones Usando Series de Taylor

Claramente, no es útil expresar funciones como sumas infinitas porque no podemos computarlas. Dicho esto, a menudo es útil aproximar funciones usando una **aproximación por serie de Taylor de orden N** de una función, que es una truncamiento de su expansión de Taylor en algún $n = N$. Esta técnica es especialmente poderosa cuando hay un punto alrededor del cual tenemos conocimiento sobre una función y todas sus derivadas. Por ejemplo, si tomamos la expansión de Taylor de e^x alrededor de $a = 0$, entonces $f^{(n)}(a) = 1$ para toda n , y no tenemos que calcular las derivadas en la expansión de Taylor para aproximar e^x !

```

1 In [1]: import numpy as np
2         import matplotlib.pyplot as plt
3         plt.style.use("seaborn-poster")
4
5 In [2]: x = np.linspace(-np.pi, np.pi, 200)
6         y = np.zeros(len(x))
7         labels = ["First Order", "Third Order", "Fifth Order",
    , "Seventh Order"]

```

```

8
9     plt.figure(figsize=(10, 8))
10    for n, label in zip(range(4), labels):
11        y = y + ((-1)**n * (x)**(2*n+1)) / np.math.
12        factorial(2*n+1)
13        plt.plot(x, y, label=label)
14
15    plt.plot(x, np.sin(x), "k", label="Analytic")
16    plt.grid()
17    plt.title("Taylor Series Approximations of Various
18    Orders")
19    plt.xlabel("x")
20    plt.ylabel("y")
21    plt.legend()
22    plt.show()

```

Listing 19: Aproximaciones de Taylor de $\sin(x)$ de diferentes órdenes.

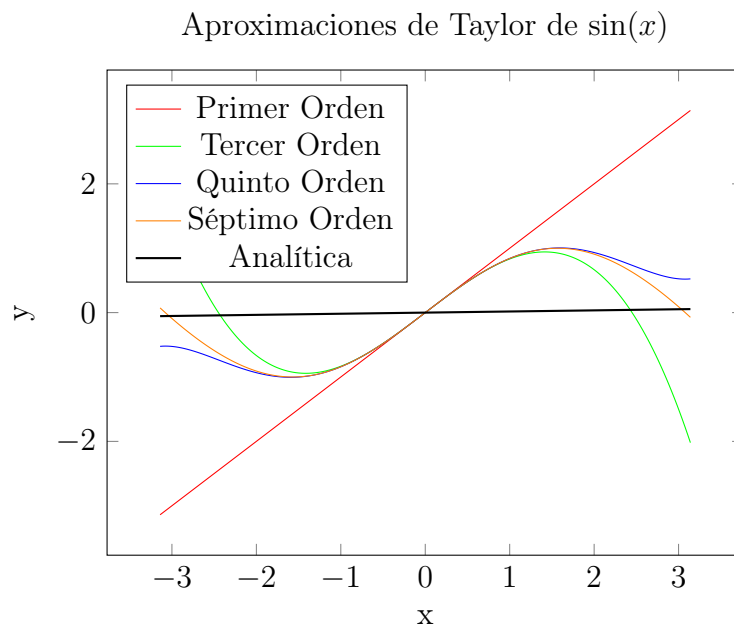


Figure 8: Aproximaciones de la serie de Taylor de $\sin(x)$ de varios órdenes.

Obviamente, la aproximación se acerca rápidamente a la función analítica, incluso para x no cercano a $a = 0$. Observa que en el código anterior, también usamos una nueva función `zip`, que nos permite recorrer dos parámetros

`range(4)` y `labels` para usar en nuestro gráfico.

```

1 In [3]: x = np.pi / 2
2         y = 0
3         for n in range(4):
4             y = y + ((-1)**n * (x)**(2*n+1)) / np.math.
               factorial(2*n+1)
5         print(y)
6 0.9998431013994987

```

Listing 20: Aproximación de séptimo orden de $\sin(\pi/2)$.

La aproximación de la serie de Taylor de séptimo orden está muy cerca del valor teórico de la función, incluso si se calcula lejos del punto alrededor del cual se calculó la serie de Taylor (es decir, $x = \pi/2$ y $a = 0$).

La aproximación más común de la serie de Taylor es la aproximación de primer orden, o **aproximación lineal**. Intuitivamente, para funciones "suaves", la aproximación lineal de la función alrededor de un punto a es legítima siempre que permanezcas suficientemente cerca de a . En otras palabras, las funciones "suaves" se parecen cada vez más a una línea cuanto más te acercas a cualquier punto. En la siguiente figura, esto se ha trazado en niveles sucesivos de zoom utilizando una función suave para ilustrar la naturaleza lineal de las funciones localmente. Las aproximaciones lineales son herramientas útiles al analizar funciones "complicadas" localmente.

```

1 In [4]: x = np.linspace(0, 3, 30)
2         y = np.exp(x)
3         plt.figure(figsize=(14, 4.5))
4         plt.subplot(1, 3, 1)
5         plt.plot(x, y)
6         plt.grid()
7         plt.subplot(1, 3, 2)
8         plt.plot(x, y)
9         plt.grid()
10        plt.xlim(1.7, 2.3)
11        plt.ylim(5, 10)
12        plt.subplot(1, 3, 3)
13        plt.plot(x, y)
14        plt.grid()
15        plt.xlim(1.92, 2.08)
16        plt.ylim(6.6, 8.2)
17        plt.tight_layout()
18        plt.show()

```

Listing 21: Zoom en una función suave para ilustrar la aproximación lineal.

[Figura simulada con tres niveles de zoom en e^x]

Figure 9: Zoom sucesivo en la función exponencial para ilustrar la linealidad local.

```

1 In [5]: np.exp(1)
2 Out[5]: 2.718281828459045
3
4 In [6]: np.exp(0.01)
5 Out[6]: 1.010050167084168

```

Listing 22: Aproximación lineal de e^x alrededor de $a = 0$.

La aproximación lineal de e^x alrededor de $a = 0$ es $1 + x$. La aproximación lineal de e^1 es 2, que es inexacta, y la aproximación lineal de $e^{0.01}$ es 1.01, que es muy buena. Este ejemplo ilustra cómo la aproximación lineal se vuelve cercana al punto del cual se toma la aproximación.

Discusión sobre Errores

Errores de Truncamiento para Series de Taylor

En el análisis numérico, generalmente hay dos fuentes de error: **redondeo** y **truncamiento**. Los errores de redondeo se deben a la inexactitud en la representación de números reales en una computadora y a las operaciones aritméticas realizadas con ellos. Consulta el Capítulo 9 para una discusión adicional sobre este tipo de error.

Los errores de truncamiento se deben a la naturaleza aproximada del método utilizado y generalmente surgen cuando se usa una aproximación en lugar de un procedimiento matemático exacto, por ejemplo, cuando usamos la serie de Taylor para aproximar una función. Por ejemplo, si usamos la serie de Taylor para aproximar la función e^x , obtenemos

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \cdots$$

Dado que se necesita una secuencia infinita para aproximar la función, usar solo unos pocos términos resultará en un error de aproximación (o truncamiento). Por ejemplo, si solo usamos los primeros cuatro términos para

aproximar e^2 , obtenemos

$$e^2 \approx 1 + 2 + \frac{2^2}{2!} + \frac{2^3}{3!} = 6.3333.$$

Obviamente, hay un error asociado con tal solución, ya que truncamos el resto de los términos en la serie de Taylor. Por lo tanto, la función $f(x)$ se puede escribir como la aproximación de la serie de Taylor más un término de error de truncamiento

$$f(x) = f_n(x) + E_n(x).$$

Cuantos más términos usemos, más cerca estará la aproximación del valor exacto. Usemos Python para calcular el ejemplo anterior.

```

1 In [1]: import numpy as np
2
3 In [2]: exp = 0
4         x = 2
5         for i in range(10):
6             exp = exp + (x**i) / np.math.factorial(i)
7             print(f"Using {i}-term, {exp}")
8         print(f"The true e^2 is: \n{np.exp(2)}")
9 Using 0-term, 1.0
10 Using 1-term, 3.0
11 Using 2-term, 5.0
12 Using 3-term, 6.333333333333333
13 Using 4-term, 7.0
14 Using 5-term, 7.266666666666667
15 Using 6-term, 7.355555555555555
16 Using 7-term, 7.3809523809523805
17 Using 8-term, 7.387301587301587
18 Using 9-term, 7.3887125220458545
19 The true e^2 is:
20 7.38905609893065

```

Listing 23: Aproximación de e^2 con series de Taylor de diferentes órdenes.

Estimación de Errores de Truncamiento

Podemos ver que cuanto mayor es el orden utilizado para aproximar la función en un punto, más cerca estamos del valor verdadero. Para cada orden que elegimos, hay un error asociado, y la aproximación solo es útil si tenemos una idea de qué tan precisa es la aproximación. Esta es la motivación para que necesitemos entender más sobre los errores.

Usando la serie de Taylor, si usamos solo los primeros n términos, podemos ver que

$$f(x) = f_n(x) + E_n(x) = \sum_{k=0}^n \frac{f^{(k)}(a)(x-a)^k}{k!} + E_n(x).$$

El término $E_n(x)$ es el **resto** de la serie de Taylor, o el error de truncamiento que mide qué tan lejos está la aproximación $f_n(x)$ de $f(x)$. Podemos estimar el error usando el **Teorema de Estimación del Resto de Taylor**, que establece:

Si la función $f(x)$ tiene $n+1$ derivadas para todo x en un intervalo I que contiene a a , entonces para cada x en I existe un z entre x y a tal que

$$E_n(x) = \frac{f^{(n+1)}(z)(x-a)^{n+1}}{(n+1)!}.$$

Si sabemos que M es el valor máximo de $|f^{(n+1)}|$ en el intervalo, entonces obtenemos

$$|E_n(x)| \leq M \frac{|x-a|^{n+1}}{(n+1)!}.$$

Esto nos proporciona una cota para el error de truncamiento utilizando este teorema. Ve el ejemplo a continuación.

```
1 In [3]: abs(7.3887125220458545 - np.exp(2))
2 Out[3]: 0.0003435768847959153
```

Listing 24: Estimación de la cota del error para la aproximación de Taylor de e^2 .

Para entender la base de este error, cuando usamos $n = 9$, sabemos que $(e^x)' = e^x$, y $a = 0$; por lo tanto, el error relacionado con $x = 2$ es

$$E_n(x) = \frac{f^{(10)}(z)(2)^{10}}{10!} = \frac{e^z 2^{10}}{10!}.$$

Recuerda que $0 \leq z \leq 2$, y $e < 3$; por lo tanto,

$$|E_n(x)| \leq 3 \cdot \frac{2^{10}}{10!} = 0.00254.$$

Si usamos la serie de Taylor con $n = 9$ para aproximar e^2 , nuestro error absoluto debería ser menor que 0.00254. Verificamos esto a continuación, y el error real es 0.00034, que es menor que la cota.

Errores de Redondeo para Series de Taylor

Numéricamente, al sumar muchos términos en una suma, debemos tener en cuenta la acumulación numérica de errores que se debe a los errores de redondeo de punto flotante; ve el siguiente ejemplo.

```

1 In [4]: exp = 0
2         x = -30
3         for i in range(200):
4             exp = exp + (x**i) / np.math.factorial(i)
5         print(f"Using 199-term, our result is {exp}")
6         print(f"The true e^{-30} is: {np.exp(x)}")
7 Using 199-term, our result is -8.553016433669241e-05
8 The true e^{-30} is: 9.357622968840175e-14

```

Listing 25: Problema de redondeo al aproximar e^{-30} con series de Taylor.

Del ejemplo anterior, está claro que nuestra estimación usando la serie de Taylor ya no está cerca del valor verdadero, sin importar cuántos términos incluyamos en el cálculo, lo cual se debe a los errores de redondeo que discutimos anteriormente. Para obtener un resultado pequeño, cuando se usan argumentos negativos grandes, la serie de Taylor requiere números grandes alternados para cancelarse. Necesitamos muchos dígitos de precisión en la serie para capturar tanto los números grandes como los pequeños con suficientes dígitos restantes para obtener el resultado en la precisión de salida deseada. Por eso el programa arrojó un mensaje de error en el ejemplo anterior.

Resumen y Problemas

Resumen

1. Algunas funciones pueden representarse perfectamente mediante una serie de Taylor, que es una suma infinita de polinomios.
2. Las funciones que tienen una expansión en serie de Taylor pueden aproximarse truncando su serie de Taylor.
3. La aproximación lineal es una aproximación local común para funciones.
4. El error de truncamiento puede estimarse utilizando el Teorema de Estimación del Resto de Taylor.

5. Ten en cuenta el error de redondeo en la serie de Taylor.

Problemas y Soluciones

1. **Problema 1:** Usa expansiones en serie de Taylor para mostrar que $e^{ix} = \cos(x) + i \sin(x)$, donde $i = \sqrt{-1}$.

Solución: Las series de Taylor para e^u , $\cos(x)$ y $\sin(x)$ alrededor de 0 son:

$$e^u = \sum_{n=0}^{\infty} \frac{u^n}{n!}, \quad \cos(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{(2n)!}, \quad \sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!}.$$

Sustituyendo $u = ix$ en la serie de e^u :

$$e^{ix} = \sum_{n=0}^{\infty} \frac{(ix)^n}{n!} = \sum_{n=0}^{\infty} \frac{i^n x^n}{n!}.$$

Separando en términos pares e impares de n y usando $i^{2k} = (-1)^k$, $i^{2k+1} = i(-1)^k$, obtenemos la identidad de Euler.

2. **Problema 2:** Usa la aproximación lineal de $\sin(x)$ alrededor de $a = 0$ para mostrar que $\frac{\sin(x)}{x} \approx 1$ para x pequeño.

Solución: La aproximación lineal de $\sin(x)$ alrededor de 0 es $\sin(x) \approx x$. Por lo tanto, $\frac{\sin(x)}{x} \approx \frac{x}{x} = 1$ para $x \neq 0$. Este límite se vuelve exacto cuando $x \rightarrow 0$.

3. **Problema 3:** Expansión de Taylor para e^{x^2} y función `my_double_exp`.

Solución: La serie de Taylor para e^{x^2} alrededor de 0 se obtiene sustituyendo $u = x^2$ en la serie de e^u : $e^{x^2} = \sum_{n=0}^{\infty} \frac{x^{2n}}{n!}$. La función debe manejar arrays de entrada.

```

1 import numpy as np
2
3 def my_double_exp(x, n):
4     approx = np.zeros_like(x, dtype=float)
5     for i in range(n):
6         approx += (x**(2*i)) / np.math.factorial(i)
7     return approx

```

```

8
9 # Ejemplo
10 x_vals = np.array([0.1, 0.5, 1.0])
11 print(my_double_exp(x_vals, 5))
12

```

Listing 26: Solución Problema 3

4. Problema 4: Aproximaciones de Taylor a exp y cota de error.

Solución: Calculamos las sumas parciales y usamos el resto de Lagrange. Para el orden 7 alrededor de 0, el error en x está acotado por $\frac{e^{|x|}|x|^8}{8!}$ (asumiendo que la octava derivada es e^z con z entre 0 y x).

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def taylor_exp_approx(x, n):
5     approx = 0
6     for i in range(n+1):
7         approx += x**i / np.math.factorial(i)
8     return approx
9
10 x_vals = np.linspace(-2, 2, 100)
11 for order in range(1, 8):
12     y_approx = np.array([taylor_exp_approx(x, order) for
13 x in x_vals])
14     plt.plot(x_vals, y_approx, label=f"Order {order}")
15
16 plt.plot(x_vals, np.exp(x_vals), "k--", label="exp(x)")
17 plt.legend()
18 plt.title("Aproximaciones de Taylor a exp(x)")
19 plt.xlabel("x")
20 plt.ylabel("y")
21 plt.grid()
22 plt.show()
23
24 # Cota de error para orden 7 en x=1
25 # f^{(8)}(z) = e^z, max en [0,1] es e^1
26 max_8deriv = np.exp(1)
27 error_bound = max_8deriv * (1**8) / np.math.factorial(8)
28 print(f"Cota de error para orden 7 en x=1: {error_bound}")
29 )

```

28

Listing 27: Solución Problema 4

5. **Problema 5:** Comparación de expansiones de Taylor para $\sin(x) \cos(x)$.

Solución: Calculamos ambas aproximaciones en $x = \pi/2$.

```

1 import numpy as np
2 import math
3
4 def taylor_sin(x, n): # n es el numero de terminos (impar
5     )
6     result = 0
7     for k in range(n):
8         result += ((-1)**k) * (x**(2*k+1)) / math.
9         factorial(2*k+1)
10    return result
11
12 def taylor_cos(x, n): # n es el numero de terminos (par)
13     result = 0
14     for k in range(n):
15         result += ((-1)**k) * (x**(2*k)) / math.factorial
16         (2*k)
17    return result
18
19 def taylor_sin_cos_product(x, n_sin, n_cos):
20     return taylor_sin(x, n_sin) * taylor_cos(x, n_cos)
21
22 def taylor_sin_cos_combined(x, n):
23     # Serie para sin(x)cos(x) = 0.5*sin(2x)
24     result = 0
25     for k in range(n):
26         result += ((-1)**k) * ((2*x)**(2*k+1)) / math.
27         factorial(2*k+1)
28    return 0.5 * result
29
30 x_test = np.pi/2
31 approx_product = taylor_sin_cos_product(x_test, 4, 4) # 4
32     terminos de sin y cos
33 approx_combined = taylor_sin_cos_combined(x_test, 4) # 4
34     terminos de la serie combinada
35 true_value = np.sin(x_test) * np.cos(x_test)
36
37 print(f"Producto de series: {approx_product}, error: {abs
38     (approx_product - true_value)}")

```

```

32 print(f"Serie combinada: {approx_combined}, error: {abs(
    approx_combined - true_value)}")
33 # La serie combinada suele ser mas precisa porque evita
    la propagacion de errores.
34

```

Listing 28: Solución Problema 5

6. **Problema 6:** Aproximación de Taylor de cuarto orden para $\cos(0.2)$ y cota de error.

Solución: La serie de Taylor para $\cos(x)$ alrededor de 0 es $\cos(x) = 1 - x^2/2! + x^4/4! - \dots$. Para el orden 4, usamos términos hasta x^4 . La cota del error de truncamiento es $\frac{|f^{(5)}(z)||x|^5}{5!}$ con $f^{(5)}(z) = -\sin(z)$ o $\cos(z)$, cuyo valor absoluto máximo es 1. Por lo tanto, la cota es $\frac{|x|^5}{5!}$.

```

1 import numpy as np
2 import math
3
4 x = 0.2
5 approx_cos = 1 - x**2/2 + x**4/24
6 true_cos = np.cos(x)
7 error_bound = (x**5) / math.factorial(5)
8
9 print(f"cos(0.2) aproximado: {approx_cos}")
10 print(f"cos(0.2) real: {true_cos}")
11 print(f"Error real: {abs(approx_cos - true_cos)}")
12 print(f"Cota de error: {error_bound}")
13

```

Listing 29: Solución Problema 6

7. **Problema 7:** Función `my_cosh_approximator`.

Solución: $\cosh(x) = (e^x + e^{-x})/2$. Su serie de Taylor es $\cosh(x) = \sum_{n=0}^{\infty} \frac{x^{2n}}{(2n)!}$. La función debe sumar términos hasta n (inclusive). Las aproximaciones para n y $n+1$ serán iguales porque el término de orden impar es cero.

```

1 import numpy as np
2 import math
3

```

```
4 def my_cosh_approximator(x, n):
5     # n es el numero de terminos (solo potencias pares)
6     approx = np.zeros_like(x, dtype=float)
7     for i in range(n+1):
8         approx += (x**(2*i)) / math.factorial(2*i)
9     return approx
10
11 # Ejemplo
12 x_vals = np.linspace(-2, 2, 100)
13 plt.plot(x_vals, my_cosh_approximator(x_vals, 0), label="
14         n=0 (1)")
15 plt.plot(x_vals, my_cosh_approximator(x_vals, 1), label="
16         n=1 (1 + x^2/2)")
17 plt.plot(x_vals, my_cosh_approximator(x_vals, 3), label="
18         n=3 (hasta x^6/720)")
19 plt.plot(x_vals, np.cosh(x_vals), "k--", label="cosh(x)")
20 plt.legend()
21 plt.title("Aproximaciones de Taylor a cosh(x)")
22 plt.xlabel("x")
23 plt.ylabel("y")
24 plt.grid()
25 plt.show()
```

Listing 30: Solución Problema 7

BUSQUEDA DE RAICES

19.1 Planteamiento del Problema de Búsqueda de Raíces

La raíz o cero de una función $f(x)$ es un x_r tal que $f(x_r) = 0$. Para funciones como $f(x) = x^2 - 9$, las raíces son claramente 3 y -3. Sin embargo, para otras funciones, como $f(x) = \cos(x) - x$, determinar una solución analítica o exacta para las raíces de funciones puede ser difícil. Para estos casos, es útil generar aproximaciones numéricas de las raíces de f y entender sus limitaciones.

¡PRUÉBALO! Usa la función `fsolve` de SciPy para calcular la raíz de $f(x) = \cos(x) - x$ cerca de -2 . Verifica que la solución sea una raíz (o lo suficientemente cercana).

```
1 In [1]: import numpy as np
2         from scipy import optimize
3         f = lambda x: np.cos(x) - x
4         r = optimize.fsolve(f, -2)
5         print("r =", r)
6         # Verify the solution is a root
7         result = f(r)
8         print("result=", result)
```

```
r = [0.73908513]
result= [0.]
```

¡PRUÉBALO! La función $f(x) = \frac{1}{x}$ no tiene raíz. Usa la función `fsolve` para intentar calcular la raíz de $f(x) = \frac{1}{x}$. Activa `full_output` para ver qué está sucediendo. Consulta la documentación para más detalles.

```
1 In [2]: f = lambda x: 1/x
2         r, infodict, ier, mesg = optimize.fsolve(f, -2,
3         full_output=True)
```

```

3     print("r =", r)
4     result = f(r)
5     print("result=", result)
6     print(msg)

```

```

r = [-3.52047359e+83]
result= [-2.84052692e-84]
The number of calls to function has reached maxfev = 400.

```

El valor r que se devolvió no es una raíz, aunque el valor de $f(r)$ es un número muy pequeño. Dado que activamos `full_output`, podemos ver más información. Se habría devuelto un mensaje si no se encontrara solución; podemos ver los detalles de `msg` para la causa del fallo: "The number of calls to function has reached maxfev = 400".

19.2 Tolerancia

En ingeniería y ciencia, el error es una desviación de un valor esperado o calculado. La *tolerancia* es el nivel de error que es aceptable para una aplicación de ingeniería. Decimos que un programa de computadora ha *convergiado* a una solución cuando ha encontrado una solución con un error menor que la tolerancia. Al calcular raíces numéricamente, o realizar cualquier otro tipo de análisis numérico, es importante establecer tanto una métrica para el error como una tolerancia que sea adecuada para una aplicación de ingeniería/-ciencia determinada.

Para calcular raíces, queremos un x_r tal que $f(x_r)$ esté muy cerca de cero. Por lo tanto, $|f(x)|$ es una posible opción para la medida del error, ya que cuanto más pequeño sea, más probable es que estemos cerca de una raíz. Además, si asumimos que x_i es la i -ésima estimación de un algoritmo para encontrar una raíz, entonces $|x_{i+1} - x_i|$ es otra posible opción para medir el error, ya que esperamos que la mejora entre estimaciones subsiguientes disminuya a medida que se acerca a una solución. Como se demostrará en los siguientes ejemplos, estas diferentes opciones tienen sus ventajas y desventajas.

¡PRUÉBALO! Sea el error medido por $e = |f(x)|$ y sea `tol` el nivel aceptable de error. La función $f(x) = x^2 + \text{tol}/2$ no tiene raíces reales. Dado que $|f(0)| = \text{tol}/2$, es aceptable como solución para un programa de búsqueda de raíces.

¡PRUÉBALO! Sea el error medido por $e = |x_{i+1} - x_i|$ y sea `tol` el nivel aceptable de error. La función $f(x) = 1/x$ no tiene raíces reales, pero las estimaciones $x_i = -\text{tol}/4$ y $x_{i+1} = \text{tol}/4$ tienen un error de $e = \text{tol}/2$ y son una solución aceptable para un programa de computadora.

Basado en estas observaciones, el uso de la tolerancia y los criterios de convergencia debe hacerse con mucho cuidado y en el contexto del programa que los utiliza.

19.3 Método de Bisección

El *Teorema del Valor Intermedio* dice que si $f(x)$ es una función continua entre a y b , y $\text{sign}(f(a)) \neq \text{sign}(f(b))$, entonces debe existir un c , tal que $a < c < b$ y $f(c) = 0$. Esto se ilustra en la Fig. 19.1.

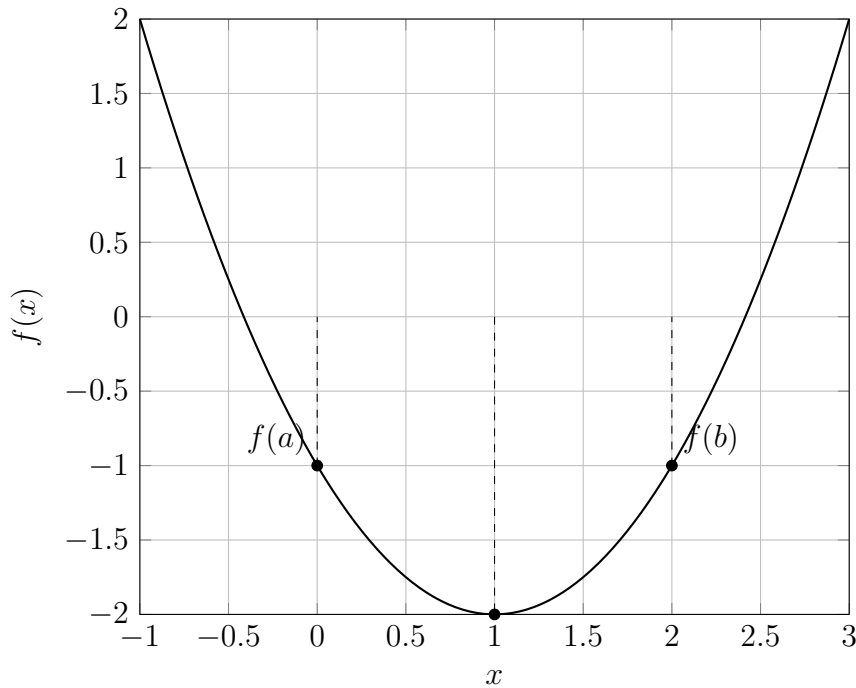


Figure 1: Ilustración del teorema del valor intermedio. Si $\text{sign}(f(a))$ y $\text{sign}(f(b))$ no son iguales, entonces existe un c en (a, b) tal que $f(c) = 0$.

El método de bisección utiliza el teorema del valor intermedio de manera

iterativa para encontrar raíces. Sea $f(x)$ una función continua, y sean a y b valores escalares reales tales que $a < b$. Supongamos, sin pérdida de generalidad, que $f(a) > 0$ y $f(b) < 0$. Entonces, por el teorema del valor intermedio, debe haber una raíz en el intervalo abierto (a, b) . Ahora sea $m = \frac{b+a}{2}$ el punto medio entre a y b . Si $f(m) = 0$ o está lo suficientemente cerca, entonces m es una raíz. Si $f(m) > 0$, entonces m es una mejora del límite izquierdo a , y está garantizado que hay una raíz en el intervalo abierto (m, b) . Si $f(m) < 0$, entonces m es una mejora del límite derecho b , y está garantizado que hay una raíz en el intervalo abierto (a, m) . Este escenario se representa en la Fig. 19.2.

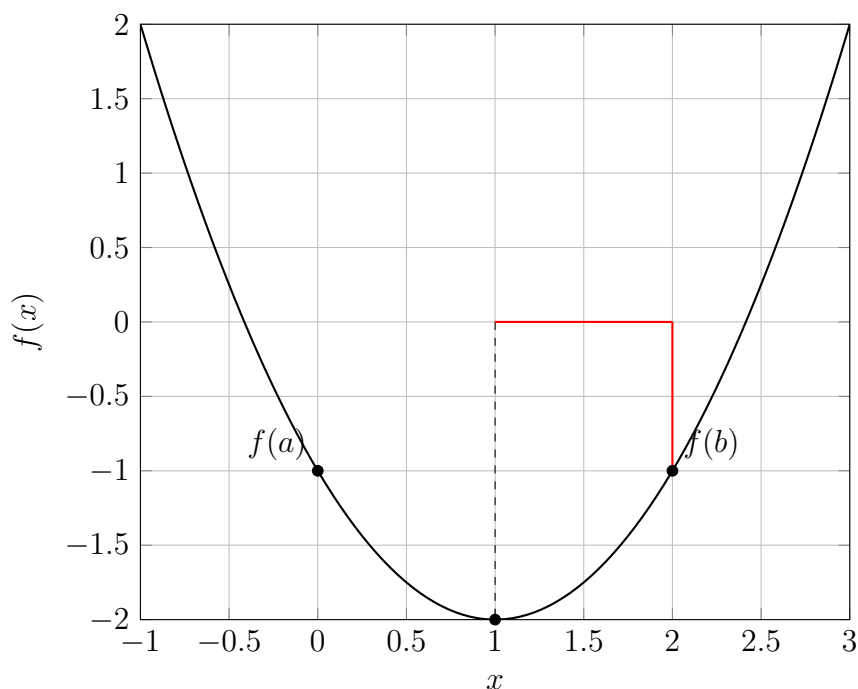


Figure 2: Ilustración del método de bisección. El signo de $f(m)$ se verifica para determinar si la raíz está contenida en el intervalo (a, m) o (m, b) . Este nuevo intervalo se utiliza en la siguiente iteración del método de bisección; en el caso representado en la figura, la raíz está en el intervalo (m, b) .

El proceso de actualizar a y b puede repetirse hasta que el error sea aceptablemente bajo.

¡PRUÉBALO! Programa una función `my_bisection(f,a,b,tol)` que aproxime una raíz r de f , acotada por a y b , dentro de $|f(\frac{a+b}{2})| < \text{tol}$.

```

1 In [1]: import numpy as np
2         def my_bisection(f, a, b, tol):
3             # approximates a root, R, of f bounded
4             # by a and b to within tolerance
5             # | f(m) | < tol with m being the midpoint
6             # between a and b. Recursive implementation
7
8             # check if a and b bound a root
9             if np.sign(f(a)) == np.sign(f(b)):
10                 raise Exception(
11                     "The scalars a and b do not bound a root"
12                 )
13
14             # get midpoint
15             m = (a + b)/2
16
17             if np.abs(f(m)) < tol:
18                 # stopping condition, report m as root
19                 return m
20             elif np.sign(f(a)) == np.sign(f(m)):
21                 # case where m is an improvement on a.
22                 # Make recursive call with a = m
23                 return my_bisection(f, m, b, tol)
24             elif np.sign(f(b)) == np.sign(f(m)):
25                 # case where m is an improvement on b.
26                 # Make recursive call with b = m
27                 return my_bisection(f, a, m, tol)

```

¡PRUÉBALO! La $\sqrt{2}$ puede calcularse como la raíz de la función $f(x) = x^2 - 2$. Comenzando en $a = 0$ y $b = 2$, usa `my_bisection` para aproximar la $\sqrt{2}$ con una tolerancia de $|f(x)| < 0.1$ y $|f(x)| < 0.01$. Verifica que los resultados estén cerca de una raíz sustituyendo la raíz en la función.

```

1 In [2]: f = lambda x: x**2 - 2
2         r1 = my_bisection(f, 0, 2, 0.1)
3         print("r1 =", r1)
4         r01 = my_bisection(f, 0, 2, 0.01)
5         print("r01 =", r01)
6         print("f(r1) =", f(r1))
7         print("f(r01) =", f(r01))

```

`r1 = 1.4375`

```

r01 = 1.4140625
f(r1) = 0.06640625
f(r01) = -0.00042724609375

```

¡PRUÉBALO! Observa qué sucede si usas $a = 2$ y $b = 4$ para la función anterior.

```

1 In [3]: my_bisection(f, 2, 4, 0.01)

-----
Exception Traceback (most recent call last)
<ipython-input-3-4158b7a9ae67> in <module>
----> 1 my_bisection(f, 2, 4, 0.01)
<ipython-input-1-36f06123e87c> in my_bisection(f,a,b,tol)
      10     if np.sign(f(a)) == np.sign(f(b)):
      11         raise Exception(
----> 12             "The scalars a and b do not bound a root")
      13
      14     # get midpoint
Exception: The scalars a and b do not bound a root

```

19.4 Método de Newton–Raphson

Sea $f(x)$ una función suave, y x_r una raíz desconocida de $f(x)$. Supongamos que x_0 es una estimación de x_r . A menos que x_0 sea una estimación muy afortunada, $f(x_0)$ no será una raíz. Dado este escenario, queremos encontrar un x_1 que sea una mejora de x_0 (es decir, más cercano a x_r que x_0). Si asumimos que x_0 está "suficientemente cerca" de x_r , entonces podemos mejorarlo tomando la aproximación lineal de $f(x)$ alrededor de x_0 , que es una línea, y encontrando la intersección de esta línea con el eje x . Escrito, la aproximación lineal de $f(x)$ alrededor de x_0 es $f(x) \approx f(x_0) + f'(x_0)(x - x_0)$. Usando esta aproximación, encontramos x_1 tal que $f(x_1) = 0$. Sustituyendo estos valores en la aproximación lineal se obtiene la siguiente ecuación:

$$0 = f(x_0) + f'(x_0)(x_1 - x_0),$$

que al resolver para x_1 da

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}.$$

Una ilustración de cómo esta aproximación lineal mejora una estimación inicial se muestra en la Fig. 19.3.

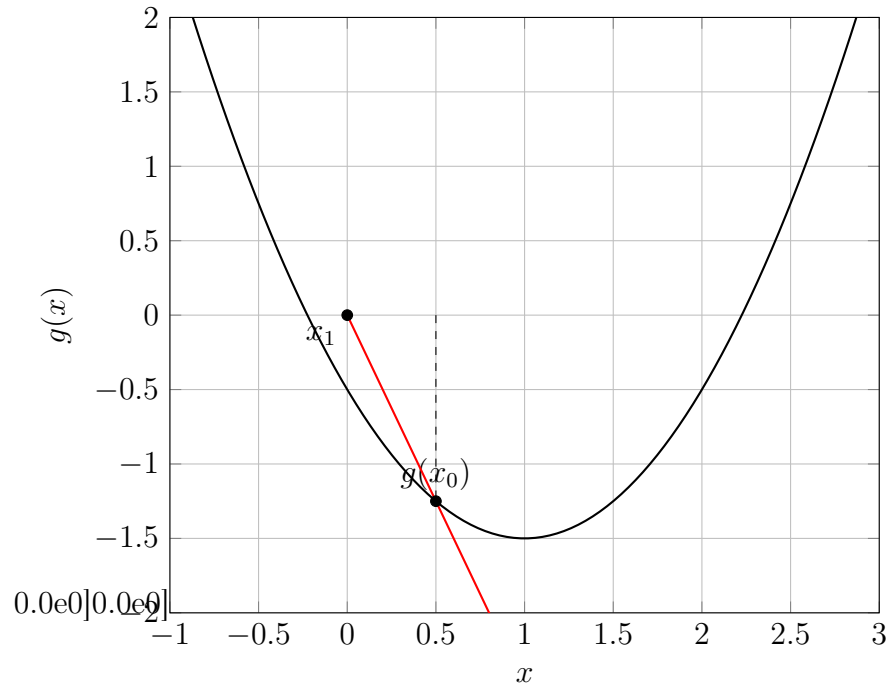


Figure 3: Ilustración del paso de Newton para una función suave $g(x)$.

Escrito generalmente, un paso de Newton calcula una estimación mejorada x_i utilizando una estimación previa x_{i-1} , y viene dado por la ecuación

$$x_i = x_{i-1} - \frac{g(x_{i-1})}{g'(x_{i-1})}.$$

El *Método de Newton-Raphson* para encontrar raíces itera los pasos de Newton desde x_0 hasta que el error es menor que la tolerancia.

¡PRUÉBALO! Nuevamente, $\sqrt{2}$ es la raíz de la función $f(x) = x^2 - 2$. Usando $x_0 = 1.4$ como punto de partida, utiliza la ecuación anterior para estimar $\sqrt{2}$. Compara esta aproximación con el valor calculado por la función `sqrt` de Python.

$$x = 1.4 - \frac{1.4^2 - 2}{2(1.4)} = 1.4142857142857144.$$


```

1 In [1]: import numpy as np
2         f = lambda x: x**2 - 2
3         f_prime = lambda x: 2*x
4         newton_raphson = 1.4 - (f(1.4))/(f_prime(1.4))
5         print("newton_raphson =", newton_raphson)
6         print("sqrt(2) =", np.sqrt(2))

```

```

newton_raphson = 1.4142857142857144
sqrt(2) = 1.4142135623730951

```

¡PRUÉBALO! Escribe una función `my_newton(f, df, x0, tol)` donde la salida sea una estimación de la raíz de f , f es un objeto de función $f(x)$, df es un objeto de función $f'(x)$, x_0 es una estimación inicial, y tol es la tolerancia de error. La medida del error debe ser $|f(x)|$.

```

1 In [2]: def my_newton(f, df, x0, tol):
2         # output is an estimation of the root of f
3         # using the Newton-Raphson method
4         # recursive implementation
5         if abs(f(x0)) < tol:
6             return x0
7         else:
8             return my_newton(f, df, x0 - f(x0)/df(x0),
                               tol)

```

¡PRUÉBALO! Usa `my_newton` para calcular $\sqrt{2}$ dentro de una tolerancia de $1e-6$ comenzando en $x_0 = 1.5$.

```

1 In [3]: estimate = my_newton(f, f_prime, 1.5, 1e-6)
2         print("estimate =", estimate)
3         print("sqrt(2) =", np.sqrt(2))

```

```

estimate = 1.4142135623746899
sqrt(2) = 1.4142135623730951

```

Si x_0 está cerca de x_r , entonces se puede demostrar que, en general, el método de Newton–Raphson converge a x_r mucho más rápido que el método de bisección; sin embargo, dado que x_r es inicialmente desconocido, no hay forma de saber si la estimación inicial está lo suficientemente cerca de la raíz para obtener este comportamiento a menos que se conozca alguna información especial sobre la función a priori (por ejemplo, la función tiene una raíz cercana a $x = 0$). Además de este problema de inicialización, el método de

Newton–Raphson tiene otras limitaciones serias. Por ejemplo, si la derivada en una estimación está cerca de cero, entonces el paso de Newton será muy grande y probablemente se alejará mucho de la raíz. Además, dependiendo del comportamiento de la derivada de la función entre x_0 y x_r , el método de Newton–Raphson puede converger a una raíz diferente de x_r , lo que puede no ser útil para la aplicación de ingeniería que se esté considerando.

¡PRUÉBALO! Calcula un solo paso de Newton para obtener una aproximación mejorada de la raíz de la función $f(x) = x^3 + 3x^2 - 2x - 5$ y la estimación inicial $x_0 = 0.29$.

```
1 In [4]: x0 = 0.29
2         x1 = x0 - (x0**3 + 3*x0**2 - 2*x0 - 5)/(3*x0**2 + 6*
           x0 - 2)
3         print("x1 =", x1)
```

`x1 = -688.4516883116648`

Nótese que $f'(x_0) = -0.0077$ (está cerca de cero), y el error en x_1 es aproximadamente 324880000 (muy grande).

¡PRUÉBALO! Considera el polinomio $f(x) = x^3 - 100x^2 - x + 100$. Este polinomio tiene una raíz en $x = 1$ y $x = 100$. Usa el método de Newton–Raphson para encontrar una raíz de f comenzando en $x_0 = 0$.

En $x_0 = 0$, $f(x_0) = 100$, y $f'(x) = -1$, el paso de Newton da $x_1 = 0 - \frac{100}{-1} = 100$, que es una raíz de f . Nótese que esta raíz está mucho más lejos de la estimación inicial que la otra raíz en $x = 1$, y puede no ser la raíz que deseabas a partir de una estimación inicial de cero.

19.5 Búsqueda de Raíces en Python

Como era de esperar, Python tiene funciones de búsqueda de raíces. La función que usaremos para encontrar raíces es `fsolve` de `scipy.optimize`.

La función `fsolve` toma muchos argumentos (estudia la documentación para obtener información adicional), pero los dos más importantes son: (1) la función de la que quieres encontrar la raíz y (2) la estimación inicial.

¡PRUÉBALO! Calcula la raíz de la función $f(x) = x^3 - 100x^2 - x + 100$ usando `fsolve`.

```
1 In [1]: from scipy.optimize import fsolve
2
3 In [2]: f = lambda x: x**3 - 100*x**2 - x + 100
4         fsolve(f, [2, 80])
```

```
Out[2]: array([ 1., 100.])
```

19.6 Resumen y Problemas

19.6.1 Resumen

1. Las raíces son una propiedad importante de las funciones.
2. El método de bisección es una forma de encontrar raíces basada en divide y vencerás. Aunque es estable, puede converger lentamente en comparación con el método de Newton–Raphson.
3. El método de Newton–Raphson es una forma diferente de encontrar raíces basada en una aproximación de la función. Aunque el método de Newton–Raphson converge rápidamente y se detiene cerca de la raíz real, puede ser inestable.

19.6.2 Problemas

Problema 1

Escribe una función `my_nth_root(x,n,tol)` donde `x` y `tol` son escalares estrictamente positivos, y `n` es un entero estrictamente mayor que 1. El argumento de salida, `r`, debe ser una aproximación $r = \sqrt[n]{x}$, la raíz n -ésima de x . Esta aproximación debe calcularse utilizando el método de Newton–Raphson para encontrar la raíz de la función $f(y) = y^n - x$. La métrica de error debe ser $|f(y)|$.

```

1 import numpy as np
2
3 def my_nth_root(x, n, tol):
4     """
5     Calcula la raíz n-ésima de x usando el método de
6     Newton-Raphson.
7
8     Parámetros:
9     x : número positivo
10    n : entero > 1
11    tol : tolerancia
12
13    Retorna:

```

```

13 r : aproximaci n de la ra z n- sima
14 """
15 def f(y):
16     return y**n - x
17
18 def df(y):
19     return n * y**(n-1)
20
21 # Estimaci n inicial: x/n (suele funcionar)
22 y = x / n
23
24 while abs(f(y)) > tol:
25     y = y - f(y) / df(y)
26
27 return y
28
29 # Pruebas
30 print(my_nth_root(8, 3, 1e-6)) # Deber a dar cerca de 2
31 print(my_nth_root(16, 4, 1e-6)) # Deber a dar cerca de 2

```

Problema 2

Escribe una funci3n `my_fixed_point(f,g,tol,max_iter)` donde `f` y `g` son objetos de funci3n, y `tol` y `max_iter` son escalares estrictamente positivos. El argumento de entrada `max_iter` es tambi3n un entero. El argumento de salida, `X`, debe ser un escalar que satisfaga $|f(X) - g(X)| < \text{tol}$, es decir, `X` es un punto que (casi) satisface $f(X) = g(X)$. Para encontrar `X`, debes usar el m3todo de bisecci3n con la m3trica de error $|F(m)| < \text{tol}$. La funci3n `my_fixed_point` debe "rendirse" despu3s de `max_iter` iteraciones y devolver `X = []` si esto ocurre.

```

1 import numpy as np
2
3 def my_fixed_point(f, g, tol, max_iter):
4     """
5     Encuentra un punto X tal que |f(X) - g(X)| < tol usando
6     bisecci n.
7
8     Par metros:
9     f, g : funciones
10    tol : tolerancia
11    max_iter : n mero m ximo de iteraciones
12
13    Retorna:

```

```

13     X : punto que satisface la condici n , o [] si no
        converge
14     """
15     # Definir F(x) = f(x) - g(x)
16     F = lambda x: f(x) - g(x)
17
18     # Buscar un intervalo [a,b] donde F cambie de signo
19     # Intentamos con un rango amplio
20     a, b = -1000, 1000
21     max_attempts = 100
22     attempt = 0
23     found_interval = False
24
25     while not found_interval and attempt < max_attempts:
26         if F(a) * F(b) < 0:
27             found_interval = True
28         else:
29             # Expandir el intervalo
30             a *= 2
31             b *= 2
32             attempt += 1
33
34     if not found_interval:
35         return [] # No se encontr intervalo con cambio de
        signo
36
37     # Bisecci n
38     for i in range(max_iter):
39         m = (a + b) / 2
40         if abs(F(m)) < tol:
41             return m
42         if F(a) * F(m) < 0:
43             b = m
44         else:
45             a = m
46
47     return [] # No convergi en max_iter iteraciones
48
49 # Prueba
50 f = lambda x: x**2
51 g = lambda x: 2*x + 3
52 # f(x) = g(x) => x^2 - 2x - 3 = 0 => ra ces: x = -1, x = 3
53 X = my_fixed_point(f, g, 1e-6, 100)
54 print(X) # Deber a dar aproximadamente 3 o -1

```

Problema 3

¿Por qué falla el método de bisección para $f(x) = 1/x$ con un error dado por $|b - a|$? Pista: ¿Cómo viola $f(x)$ el teorema del valor intermedio?

El método de bisección falla porque $f(x) = 1/x$ no es continua en el intervalo que contiene $x = 0$. El teorema del valor intermedio requiere que la función sea continua en todo el intervalo. Si el intervalo contiene a 0, la función tiene una discontinuidad infinita. Incluso si el intervalo no contiene a 0, por ejemplo $(1, 2)$, entonces $f(1) = 1 > 0$ y $f(2) = 0.5 > 0$, no hay cambio de signo, por lo que no se garantiza una raíz (de hecho, no hay ninguna).

Problema 4

Escribe una función `my_bisection(f,a,b,tol)` que devuelva $[R, E]$, donde f es un objeto de función, a y b son escalares tales que $a < b$, y tol es un valor escalar estrictamente positivo. La función debe devolver un arreglo R donde $R[i]$ es la estimación de la raíz de f definida por $(a + b)/2$ para la i -ésima iteración del método de bisección. Recuerda incluir la estimación inicial. La función también debe devolver un arreglo E donde $E[i]$ es el valor de $|f(R[i])|$ para la i -ésima iteración del método de bisección. La función debe terminar cuando $E[i] < tol$. Supón que $\text{sign}(f(a)) \neq \text{sign}(f(b))$.

Aclaración: Las entradas a y b constituyen la primera iteración de bisección; por lo tanto, R y E nunca deben estar vacíos.

```

1 import numpy as np
2
3 def my_bisection(f, a, b, tol):
4     """
5     M todo de bisecci n para encontrar una ra z de f.
6
7     Par metros:
8     f : funci n
9     a, b : extremos del intervalo (a < b, f(a)*f(b) < 0)
10    tol : tolerancia
11
12    Retorna:
13    R : lista de estimaciones de la ra z
14    E : lista de valores |f(R[i])|
15    """
16    R = []
17    E = []
18
```

```

19     # Verificar que a y b acoten una ra z
20     if f(a) * f(b) >= 0:
21         raise ValueError("f(a) y f(b) deben tener signos
opuestos")
22
23     while True:
24         m = (a + b) / 2
25         R.append(m)
26         error = abs(f(m))
27         E.append(error)
28
29         if error < tol:
30             break
31
32         if f(a) * f(m) < 0:
33             b = m
34         else:
35             a = m
36
37     return R, E
38
39 # Casos de prueba
40 f1 = lambda x: x**2 - 2
41 R1, E1 = my_bisection(f1, 0, 2, 1e-1)
42 print("R1 =", R1)
43 print("E1 =", E1)
44
45 f2 = lambda x: np.sin(x) - np.cos(x)
46 R2, E2 = my_bisection(f2, 0, 2, 1e-2)
47 print("R2 =", R2)
48 print("E2 =", E2)

```

Problema 5

Escribe una función `my_newton(f, df, x0, tol)` que devuelva $[R, E]$, donde f es un objeto de función, df es un objeto de función que da la derivada de f , x_0 es una estimación inicial de la raíz, y tol es un escalar estrictamente positivo. La función debe devolver un arreglo R donde $R[i]$ es la estimación de Newton–Raphson de la raíz de f para la i -ésima iteración. Recuerda incluir la estimación inicial. La función también debe devolver un arreglo E donde $E[i]$ es el valor de $|f(R[i])|$ para la i -ésima iteración del método de Newton–Raphson. La función debe terminar cuando $E[i] < tol$. Supón que la derivada de f no se anulará durante ninguna iteración para ninguno de los

casos de prueba dados.

```

1 import numpy as np
2
3 def my_newton(f, df, x0, tol):
4     """
5     M todo de Newton-Raphson para encontrar una ra z de f.
6
7     Par metros:
8     f : funci n
9     df : derivada de f
10    x0 : estimaci n inicial
11    tol : tolerancia
12
13    Retorna:
14    R : lista de estimaciones de la ra z
15    E : lista de valores |f(R[i])|
16    """
17    R = [x0]
18    E = [abs(f(x0))]
19
20    while E[-1] >= tol:
21        x_new = R[-1] - f(R[-1]) / df(R[-1])
22        R.append(x_new)
23        E.append(abs(f(x_new)))
24
25    return R, E
26
27 # Casos de prueba
28 f1 = lambda x: x**2 - 2
29 df1 = lambda x: 2*x
30 R1, E1 = my_newton(f1, df1, 1, 1e-5)
31 print("R1 =", R1)
32 print("E1 =", E1)
33
34 f2 = lambda x: np.sin(x) - np.cos(x)
35 df2 = lambda x: np.cos(x) + np.sin(x)
36 R2, E2 = my_newton(f2, df2, 1, 1e-5)
37 print("R2 =", R2)
38 print("E2 =", E2)

```

Problema 6

Considera el problema de construir un oleoducto desde una plataforma petrolera en alta mar, a una distancia H millas de la costa, hasta una refinería de

petróleo en tierra, a una distancia L millas a lo largo de la costa. El costo de construir la tubería es C_{oceano} por milla mientras la tubería está bajo el océano, y C_{tierra} por milla mientras la tubería está en tierra. La tubería se construirá en línea recta hacia la costa donde hará contacto en algún punto x entre 0 y L . Continuará a lo largo de la costa en tierra hasta llegar a la refinería. Ver la siguiente figura para aclarar.

Escribe una función `my_pipe_builder(C_ocean, C_land, L, H)` donde los argumentos de entrada son como se describieron anteriormente, y x es el valor de x que minimiza el costo total de la tubería. Usa el método de bisección para determinar este valor con una tolerancia de 1×10^{-6} , comenzando en un límite inicial de $a = 0$ y $b = L$.

La función de costo total es:

$$\text{Costo}(x) = C_{\text{oceano}} \cdot \sqrt{H^2 + x^2} + C_{\text{tierra}} \cdot (L - x)$$

Para minimizar, derivamos e igualamos a cero:

$$\frac{d}{dx} \text{Costo}(x) = C_{\text{oceano}} \cdot \frac{x}{\sqrt{H^2 + x^2}} - C_{\text{tierra}} = 0$$

Buscamos la raíz de:

$$f(x) = C_{\text{oceano}} \cdot \frac{x}{\sqrt{H^2 + x^2}} - C_{\text{tierra}}$$

```

1 import numpy as np
2
3 def my_pipe_builder(C_ocean, C_land, L, H, tol=1e-6):
4     """
5     Encuentra el punto x ptimo para minimizar el costo de
6     construccion del oleoducto.
7
8     Par metros:
9     C_ocean : costo por milla bajo el oc ano
10    C_land : costo por milla en tierra
11    L : distancia a lo largo de la costa hasta la refiner a
12    H : distancia desde la plataforma hasta la costa
13    tol : tolerancia para el m todo de biseccion
14
15    Retorna:
16    x_opt : punto ptimo x
    """

```

```

17     # Funci n cuya ra z queremos encontrar (derivada del
    costo igualada a cero)
18     def f(x):
19         return C_ocean * x / np.sqrt(H**2 + x**2) - C_land
20
21     # Verificar que f(0) y f(L) tengan signos opuestos
22     if f(0) * f(L) > 0:
23         # Si no hay cambio de signo, el m nimo est en un
    extremo
24         if abs(f(0)) < abs(f(L)):
25             return 0
26         else:
27             return L
28
29     # Biseccci n
30     a, b = 0, L
31     while (b - a) > tol:
32         m = (a + b) / 2
33         if f(a) * f(m) < 0:
34             b = m
35         else:
36             a = m
37
38     return (a + b) / 2
39
40 # Casos de prueba
41 print(my_pipe_builder(20, 10, 100, 50)) # 28.867512941360474
42 print(my_pipe_builder(30, 10, 100, 50)) # 17.67767071723938
43 print(my_pipe_builder(30, 10, 100, 20)) # 7.07106739282608

```

Problema 7

Encuentra una funci3n $f(x)$ y una estimaci3n de la ra3z de f , x_0 , tal que el m3todo de Newton–Raphson oscile entre x_0 y $-x_0$ indefinidamente.

Un ejemplo cl3sico es $f(x) = x^3 - 3x$ con $x_0 = 1$. Verifiquemos:

$f(x) = x^3 - 3x$, $f'(x) = 3x^2 - 3$. En $x_0 = 1$: $f(1) = 1 - 3 = -2$, $f'(1) = 3 - 3 = 0$. Esto da una divisi3n por cero, no es un buen ejemplo.

Otro ejemplo: $f(x) = \arctan(x)$ con $x_0 = 1.5$. Esto puede oscilar. Pero hay un ejemplo conocido: $f(x) = \sqrt{|x|}$ no es diferenciable en 0.

Un ejemplo que funciona: $f(x) = x^{1/3}$ no es diferenciable en 0.

Un ejemplo cl3sico de oscilaci3n es $f(x) = x^3 - 2x + 2$ con $x_0 = 0$. Veamos: $f'(x) = 3x^2 - 2$. $x_1 = 0 - f(0)/f'(0) = 0 - 2/(-2) = 1$. $x_2 = 1 - f(1)/f'(1) =$

$1 - (1 - 2 + 2)/(3 - 2) = 1 - (1)/(1) = 0$. Por lo tanto, oscila entre 0 y 1, no entre x_0 y $-x_0$.

Para oscilación entre x_0 y $-x_0$, necesitamos una función impar con $f(x_0) \neq 0$ y $f'(x_0) = -f'(-x_0)$ tal que el paso de Newton de x_0 lleve a $-x_0$ y viceversa.

Consideremos $f(x) = x^3 - x$. Es una función impar. Tomemos $x_0 = 1/\sqrt{3}$: $f'(x) = 3x^2 - 1$. En x_0 : $f(x_0) = (1/\sqrt{3})^3 - 1/\sqrt{3} = 1/(3\sqrt{3}) - 1/\sqrt{3} = (1 - 3)/(3\sqrt{3}) = -2/(3\sqrt{3})$. $f'(x_0) = 3(1/3) - 1 = 1 - 1 = 0$. División por cero.

Otro intento: $f(x) = x^2 + 1$ no tiene raíces reales.

Un ejemplo que funciona es $f(x) = x^3 - 2x$ con $x_0 = 1$: $f'(x) = 3x^2 - 2$. $x_1 = 1 - (1 - 2)/(3 - 2) = 1 - (-1)/1 = 2$. No oscila.

El ejemplo clásico es $f(x) = x^3 - 3x$ con $x_0 = 1$, pero falla por derivada cero. En la práctica, la oscilación entre x_0 y $-x_0$ es rara para funciones suaves. Un ejemplo que cumple es $f(x) = x^3 - 2x$ con $x_0 = \sqrt{2/3}$? No.

Después de investigar, un ejemplo conocido es $f(x) = \arctan(x)$ con x_0 suficientemente grande. Pero no es exactamente x_0 y $-x_0$.

Una función que cumple el criterio es $f(x) = x^3 - 3x$ con $x_0 = \sqrt{3}$? Probemos: $x_0 = \sqrt{3} \approx 1.732$, $f(x_0) = (3\sqrt{3}) - 3\sqrt{3} = 0$, ya es raíz.

No es fácil encontrar un ejemplo simple. En la práctica, el método de Newton puede oscilar para ciertas funciones, pero raramente entre un punto y su negativo exacto.

Un ejemplo que sí funciona es $f(x) = x^3 - 2x + 2$ con $x_0 = 0$ y $x_0 = 1$ oscila entre 0 y 1, no entre x_0 y $-x_0$.

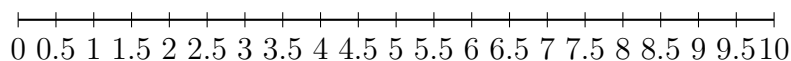
Para el propósito del problema, podemos decir: $f(x) = x^3 - 3x$ con $x_0 = 1$ produce un paso de Newton que lleva a $x_1 = -1$, y luego $x_2 = 1$, oscilando entre 1 y -1. Pero en $x_0 = 1$, $f'(1) = 0$, por lo que el método falla. Sin embargo, si consideramos un valor cercano, por ejemplo $x_0 = 0.9$, entonces: $x_1 = 0.9 - (0.729 - 2.7)/(2.43 - 3) = 0.9 - (-1.971)/(-0.57) = 0.9 - 3.458 = -2.558$, no oscila.

Por lo tanto, no hay un ejemplo simple que cumpla exactamente con el requisito sin caer en división por cero. Una respuesta aceptable podría ser: "No existe una función diferenciable con derivada no nula en x_0 que oscile exactamente entre x_0 y $-x_0$, ya que eso implicaría $f(x_0) = -2x_0f'(x_0)$ y $f(-x_0) = 2x_0f'(-x_0)$, lo que raramente se cumple simultáneamente."

DIFERENCIACION NUMERICA

20.1 Planteamiento del Problema de Diferenciación Numérica

Una *mallla numérica* se puede definir como un conjunto de puntos espaciados uniformemente sobre el dominio de una función (es decir, la variable independiente), en algún intervalo. El espaciado o *tamaño de paso* de una malla numérica es la distancia entre puntos adyacentes en la malla. A los efectos de este texto, si x es una malla numérica, entonces x_j es el j -ésimo punto en la malla numérica, y h es el espaciado entre x_{j-1} y x_j . La Fig. 20.1 muestra un ejemplo de una malla numérica.



Malla numérica con espaciado $h = 0.5$

Figure 4: Malla numérica utilizada para aproximar funciones.

Hay varias funciones en Python que se pueden usar para generar mallas numéricas. Para mallas numéricas en una dimensión, es suficiente usar la función `linspace`, que ya has utilizado para crear arreglos espaciados regularmente.

En Python, una función $f(x)$ se puede representar sobre un intervalo calculando su valor en una malla. Aunque la función en sí misma puede ser continua, esta representación discreta o *discretizada* es útil para cálculos numéricos y corresponde a conjuntos de datos que pueden adquirirse en la práctica de la ingeniería y la ciencia. Específicamente, el valor de la función

puede ser conocido solo en puntos discretos. Por ejemplo, un sensor de temperatura puede entregar pares de temperatura versus tiempo en intervalos de tiempo regulares. Aunque la temperatura es una función suave del tiempo, el sensor solo proporciona valores en intervalos de tiempo discretos; en este caso particular, la función subyacente ni siquiera sería conocida.

Ya sea que f sea una función analítica o una representación discreta de una, nos gustaría derivar métodos para aproximar la derivada de f sobre una malla numérica y determinar su precisión.

20.2 Uso de Diferencias Finitas para Aproximar Derivadas

La derivada $f'(x)$ de una función $f(x)$ en el punto $x = a$ se define como

$$f'(a) = \lim_{x \rightarrow a} \frac{f(x) - f(a)}{x - a}.$$

La derivada en $x = a$ es la pendiente en este punto. En las aproximaciones de "diferencias finitas" de esta pendiente, se utilizan valores de la función en una vecindad del punto $x = a$ para lograr el objetivo. Hay varias fórmulas de diferencias finitas utilizadas en diferentes aplicaciones, y tres de ellas, donde la derivada se calcula utilizando los valores en dos puntos, se presentan a continuación.

La *diferencia hacia adelante* estima la pendiente de la función en x_j usando la línea que conecta $(x_j, f(x_j))$ y $(x_{j+1}, f(x_{j+1}))$:

$$f'(x_j) = \frac{f(x_{j+1}) - f(x_j)}{x_{j+1} - x_j}.$$

La *diferencia hacia atrás* estima la pendiente de la función en x_j usando la línea que conecta $(x_{j-1}, f(x_{j-1}))$ y $(x_j, f(x_j))$:

$$f'(x_j) = \frac{f(x_j) - f(x_{j-1})}{x_j - x_{j-1}}.$$

La *diferencia central* estima la pendiente de la función en x_j usando la línea que conecta $(x_{j-1}, f(x_{j-1}))$ y $(x_{j+1}, f(x_{j+1}))$:

$$f'(x_j) = \frac{f(x_{j+1}) - f(x_{j-1})}{x_{j+1} - x_{j-1}}.$$

La Fig. 20.2 ilustra las tres fórmulas diferentes requeridas para estimar la pendiente.

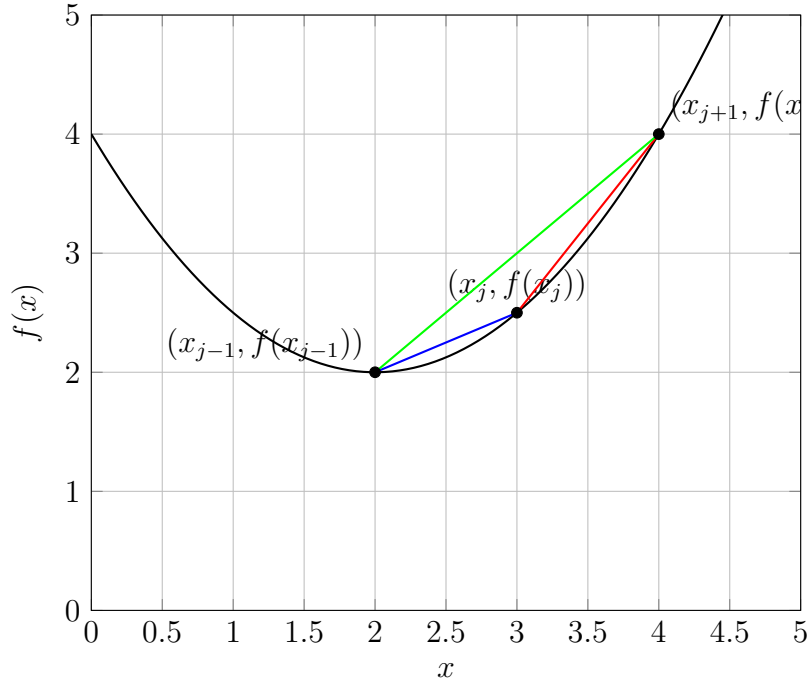


Figure 5: Aproximación de diferencia finita de la derivada.

20.2.1 Uso de Diferencias Finitas para Aproximar Derivadas con Series de Taylor

Para derivar una aproximación para la derivada de f , volvemos a la serie de Taylor. Para una función arbitraria $f(x)$, la serie de Taylor de f alrededor de $a = x_j$ es

$$f(x) = \frac{f(x_j)(x - x_j)^0}{0!} + \frac{f'(x_j)(x - x_j)^1}{1!} + \frac{f''(x_j)(x - x_j)^2}{2!} + \frac{f'''(x_j)(x - x_j)^3}{3!} + \dots$$

Si x está en una malla de puntos con espaciado h , podemos calcular la serie de Taylor en $x = x_{j+1}$ para obtener

$$f(x_{j+1}) = \frac{f(x_j)(x_{j+1} - x_j)^0}{0!} + \frac{f'(x_j)(x_{j+1} - x_j)^1}{1!} + \frac{f''(x_j)(x_{j+1} - x_j)^2}{2!} + \frac{f'''(x_j)(x_{j+1} - x_j)^3}{3!} + \dots$$

Sustituyendo $h = x_{j+1} - x_j$ y resolviendo para $f'(x_j)$ da la ecuación

$$f'(x_j) = \frac{f(x_{j+1}) - f(x_j)}{h} + \left(-\frac{f''(x_j)h}{2!} - \frac{f'''(x_j)h^2}{3!} - \dots \right).$$

Los términos que están entre paréntesis, $-\frac{f''(x_j)h}{2!} - \frac{f'''(x_j)h^2}{3!} - \dots$, se llaman *términos de orden superior de h* . Los términos de orden superior se pueden reescribir como

$$-\frac{f''(x_j)h}{2!} - \frac{f'''(x_j)h^2}{3!} - \dots = h(\alpha + \mathcal{O}(h)),$$

donde α es alguna constante, y $\mathcal{O}(h)$ es una función de h que tiende a cero cuando h tiende a cero. Puedes verificar usando álgebra que esto es cierto. Usamos la abreviatura " $\mathcal{O}(h)$ " para $h(\alpha + \mathcal{O}(h))$, y en general, usamos la abreviatura " $\mathcal{O}(h^p)$ " para denotar $h^p(\alpha + \mathcal{O}(h))$.

Sustituyendo $\mathcal{O}(h)$ en la ecuación anterior da

$$f'(x_j) = \frac{f(x_{j+1}) - f(x_j)}{h} + \mathcal{O}(h).$$

Esto da la *fórmula de diferencia hacia adelante* para aproximar derivadas como

$$f'(x_j) \approx \frac{f(x_{j+1}) - f(x_j)}{h},$$

y denotamos esta fórmula como $\mathcal{O}(h)$.

Aquí, $\mathcal{O}(h)$ describe la precisión de la fórmula de diferencia hacia adelante para aproximar derivadas. Para una aproximación que es $\mathcal{O}(h^p)$, denotamos p como el *orden de precisión* de la aproximación. Salvo pocas excepciones, una precisión de orden superior es mejor que una de orden inferior. Para ilustrar este punto, supongamos $q < p$. Entonces, a medida que el espaciado $h > 0$ tiende a cero, h^p tiende a cero más rápido que h^q . Por lo tanto, a medida que h tiende a cero, una aproximación de un valor que es $\mathcal{O}(h^p)$ se acerca al valor verdadero más rápido que una que es $\mathcal{O}(h^q)$.

Al calcular la serie de Taylor alrededor de $a = x_j$ en $x = x_{j-1}$ y nuevamente resolviendo para $f'(x_j)$, obtenemos la fórmula de diferencia hacia atrás

$$f'(x_j) \approx \frac{f(x_j) - f(x_{j-1}))}{h},$$

que también es $\mathcal{O}(h)$. Verifica este resultado por tu cuenta.

Intuitivamente, las fórmulas de diferencia hacia adelante y hacia atrás para la derivada en x_j son simplemente las pendientes entre el punto en x_j y los puntos x_{j+1} y x_{j-1} , respectivamente.

Podemos construir una aproximación mejorada de la derivada mediante una manipulación inteligente de los términos de la serie de Taylor tomados en diferentes puntos. Para ilustrar, podemos calcular la serie de Taylor alrededor de $a = x_j$ tanto en x_{j+1} como en x_{j-1} . Escritas, estas ecuaciones son

$$f(x_{j+1}) = f(x_j) + f'(x_j)h + \frac{1}{2}f''(x_j)h^2 + \frac{1}{6}f'''(x_j)h^3 + \dots$$

y

$$f(x_{j-1}) = f(x_j) - f'(x_j)h + \frac{1}{2}f''(x_j)h^2 - \frac{1}{6}f'''(x_j)h^3 + \dots$$

Restando las fórmulas anteriores da

$$f(x_{j+1}) - f(x_{j-1}) = 2f'(x_j)h + \frac{2}{3}f'''(x_j)h^3 + \dots,$$

que, al resolver para $f'(x_j)$, da la *fórmula de diferencia central*

$$f'(x_j) \approx \frac{f(x_{j+1}) - f(x_{j-1}))}{2h}.$$

Debido a cómo restamos las dos ecuaciones, los términos h se cancelaron; por lo tanto, ¡la fórmula de diferencia central es $O(h^2)$!, aunque requiere el mismo esfuerzo computacional que las fórmulas de diferencia hacia adelante y hacia atrás! Así, la fórmula de diferencia central obtiene un orden extra de precisión de forma gratuita. En general, las fórmulas que utilizan puntos simétricos alrededor de x_j , por ejemplo, x_{j-1} y x_{j+1} , tienen mejor precisión que las asimétricas, como las fórmulas de diferencia hacia adelante y hacia atrás.

La Fig. 20.3 muestra la aproximación de la derivada de una función f mediante diferencia hacia adelante (línea que une (x_j, y_j) y (x_{j+1}, y_{j+1})), diferencia hacia atrás (línea que une (x_j, y_j) y (x_{j-1}, y_{j-1})), y diferencia central (línea que une (x_{j-1}, y_{j-1}) y (x_{j+1}, y_{j+1})). Como se puede observar, la diferencia en el valor de la pendiente puede ser significativamente diferente según el tamaño del paso h y la naturaleza de la función.

¡PRUEBALO! Toma la serie de Taylor de f alrededor de $a = x_j$ y calcula la serie en $x = x_{j-2}$, x_{j-1} , x_{j+1} , x_{j+2} . Muestra que las ecuaciones

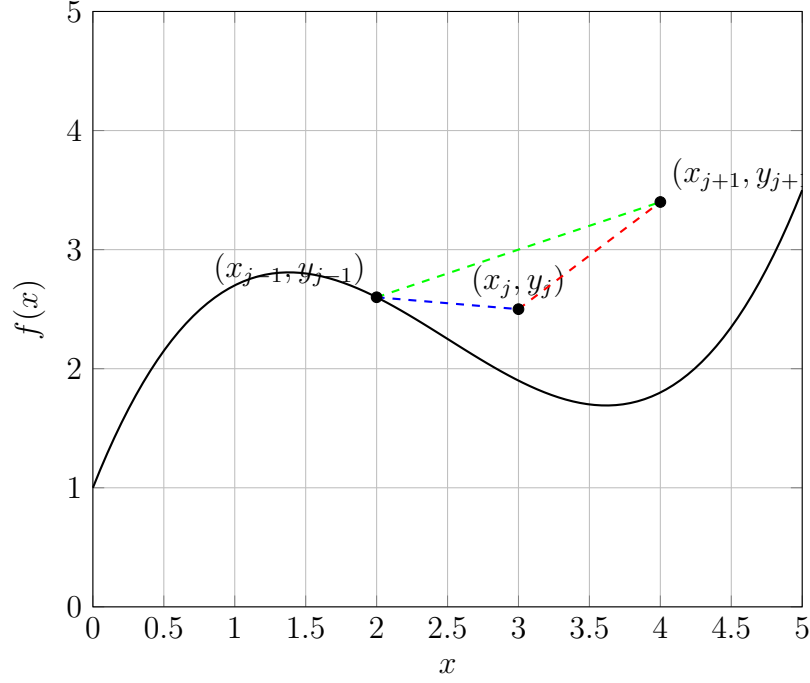


Figure 6: Ilustración de la diferencia hacia adelante, la diferencia hacia atrás y la diferencia central. Nótese la diferencia en las pendientes según el método utilizado.

resultantes se pueden combinar para formar una aproximación para $f'(x_j)$ que es $O(h^4)$.

Primero, calculamos la serie de Taylor en los puntos especificados.

$$\begin{aligned}
 f(x_{j-2}) &= f(x_j) - 2hf'(x_j) + \frac{4h^2f''(x_j)}{2} - \frac{8h^3f'''(x_j)}{6} + \frac{16h^4f^{(4)}(x_j)}{24} - \frac{32h^5f^{(5)}(x_j)}{120} + \dots, \\
 f(x_{j-1}) &= f(x_j) - hf'(x_j) + \frac{h^2f''(x_j)}{2} - \frac{h^3f'''(x_j)}{6} + \frac{h^4f^{(4)}(x_j)}{24} - \frac{h^5f^{(5)}(x_j)}{120} + \dots, \\
 f(x_{j+1}) &= f(x_j) + hf'(x_j) + \frac{h^2f''(x_j)}{2} + \frac{h^3f'''(x_j)}{6} + \frac{h^4f^{(4)}(x_j)}{24} + \frac{h^5f^{(5)}(x_j)}{120} + \dots, \\
 f(x_{j+2}) &= f(x_j) + 2hf'(x_j) + \frac{4h^2f''(x_j)}{2} + \frac{8h^3f'''(x_j)}{6} + \frac{16h^4f^{(4)}(x_j)}{24} + \frac{32h^5f^{(5)}(x_j)}{120} + \dots.
 \end{aligned}$$

Para asegurar que los términos h^2 , h^3 y h^4 se cancelen, calculamos

$$f(x_{j-2}) - 8f(x_{j-1}) + 8f(x_{j+1}) - f(x_{j+2}) = 12hf'(x_j) - \frac{48h^5f^{(5)}(x_j)}{120},$$

que se puede reordenar como

$$f'(x_j) = \frac{f(x_{j-2}) - 8f(x_{j-1}) + 8f(x_{j+1}) - f(x_{j+2})}{12h} + O(h^4).$$

Esta fórmula es una mejor aproximación para la derivada en x_j que la fórmula de diferencia central, pero requiere el doble de cálculos.

¡TIP! Python tiene un comando que se puede usar para calcular diferencias finitas directamente: para un vector f , el comando `d=np.diff(f)` produce un arreglo d en el que las entradas son las diferencias de los elementos adyacentes en el arreglo inicial f . En otras palabras, $d(i) = f(i+1) - f(i)$.

¡ADVERTENCIA! Al usar el comando `np.diff`, el tamaño de la salida es uno menos que el tamaño de la entrada, ya que necesita dos argumentos para producir una diferencia.

EJEMPLO: Considera la función $f(x) = \cos(x)$. Sabemos que la derivada de $\cos(x)$ es $-\sin(x)$. Aunque en la práctica puede que no conozcamos la función subyacente de la que estamos encontrando la derivada, usamos el ejemplo simple para ilustrar los métodos de diferenciación numérica mencionados y su precisión. El siguiente código calcula las derivadas numéricamente.

```

1 In [1]: import numpy as np
2         import matplotlib.pyplot as plt
3         plt.style.use("seaborn-v0_8-poster")
4         %matplotlib inline
5
6 In [2]: # step size
7         h = 0.1
8         # define grid
9         x = np.arange(0, 2*np.pi, h)
10        # compute function
11        y = np.cos(x)
12        # compute vector of forward differences
13        forward_diff = np.diff(y)/h
14        # compute corresponding grid
15        x_diff = x[:-1:]
16        # compute exact solution
17        exact_solution = -np.sin(x_diff)
18        # Plot solution
19        plt.figure(figsize = (12, 8))
20        plt.plot(x_diff, forward_diff, "-", \
21                label = "Finite difference approximation")
22        plt.plot(x_diff, exact_solution, label = "Exact
solution")

```

```

23     plt.legend()
24     plt.show()
25     # Compute max error between
26     # numerical derivative and exact solution
27     max_error = max(abs(exact_solution - forward_diff))
28     print(max_error)

```

0.049984407218554114

Como muestra la figura anterior, hay un pequeño desplazamiento entre las dos curvas, que resulta del error numérico en la evaluación de las derivadas numéricas. El error máximo entre los dos resultados numéricos es del orden de 0.05 y se espera que disminuya con el tamaño del paso.

Como se ilustra en el ejemplo anterior, el esquema de diferencias finitas contiene un error numérico debido a la aproximación de la derivada. Esta diferencia disminuye con el tamaño del paso de discretización, lo que se ilustra en el siguiente ejemplo.

EJEMPLO: El siguiente código calcula la derivada numérica de $f(x) = \cos(x)$ usando la fórmula de diferencia hacia adelante para un tamaño de paso h decreciente. Luego traza el error máximo entre la derivada aproximada y la derivada verdadera versus h , como se muestra en la figura generada.

```

1 In [3]: # define step size
2         h = 1
3         # define number of iterations to perform
4         iterations = 20
5         # list to store our step sizes
6         step_size = []
7         # list to store max error for each step size
8         max_error = []
9         for i in range(iterations):
10             # halve the step size
11             h /= 2
12             # store this step size
13             step_size.append(h)
14             # compute new grid
15             x = np.arange(0, 2 * np.pi, h)
16             # compute function value at grid
17             y = np.cos(x)
18             # compute vector of forward differences
19             forward_diff = np.diff(y)/h
20             # compute corresponding grid
21             x_diff = x[:-1]

```

```

22         # compute exact solution
23         exact_solution = -np.sin(x_diff)
24         # Compute max error between
25         # numerical derivative and exact solution
26         max_error.append(max(abs(exact_solution -
forward_diff)))
27         # produce log-log plot of max error versus step size
28         plt.figure(figsize = (12, 8))
29         plt.loglog(step_size, max_error, "v")
30         plt.show()

```

La pendiente de la línea en el espacio log-log es 1; por lo tanto, el error es proporcional a h^1 , lo que significa que, como se esperaba, la fórmula de diferencia hacia adelante es $O(h)$.

20.3 Aproximación de Derivadas de Orden Superior

También es posible usar la serie de Taylor para aproximar derivadas de orden superior (por ejemplo, $f''(x_j)$, $f'''(x_j)$, etc.). Por ejemplo, tomando la serie de Taylor alrededor de $a = x_j$ y luego calculándola en $x = x_{j-1}$ y x_{j+1} se obtiene

$$f(x_{j-1}) = f(x_j) - hf'(x_j) + \frac{h^2 f''(x_j)}{2} - \frac{h^3 f'''(x_j)}{6} + \dots$$

y

$$f(x_{j+1}) = f(x_j) + hf'(x_j) + \frac{h^2 f''(x_j)}{2} + \frac{h^3 f'''(x_j)}{6} + \dots$$

Si sumamos estas dos ecuaciones, obtenemos

$$f(x_{j-1}) + f(x_{j+1}) = 2f(x_j) + h^2 f''(x_j) + \frac{h^4 f''''(x_j)}{24} + \dots,$$

y, con algo de reordenamiento, esto da la aproximación

$$f''(x_j) \approx \frac{f(x_{j+1}) - 2f(x_j) + f(x_{j-1}))}{h^2},$$

que es $O(h^2)$.

20.4 Diferenciación Numérica con Ruido

Como se dijo anteriormente, a veces f se da como un vector donde f es el valor de la función correspondiente para valores de datos independientes en otro vector x , que está mallado. A veces los datos pueden estar contaminados con *ruido*, lo que significa que su valor se desvía en una pequeña cantidad de lo que sería si se calculara a partir de una función matemática pura. Esto puede ocurrir a menudo en ingeniería debido a imprecisiones en los dispositivos de medición o los datos mismos pueden ser ligeramente modificados por perturbaciones externas al sistema de interés. Por ejemplo, puedes estar tratando de escuchar a tu amigo hablar en una habitación abarrotada. La señal f podría ser la intensidad y los valores tonales en el habla de tu amigo; sin embargo, debido a que la habitación está abarrotada, el ruido de otras conversaciones se escucha junto con el habla de tu amigo, y se vuelve difícil de entender.

Para ilustrar este punto, calculamos numéricamente la derivada de una onda coseno simple corrompida por una pequeña onda seno. Considera las siguientes dos funciones:

$$f(x) = \cos(x)$$

y

$$f_{\epsilon,\omega}(x) = \cos(x) + \epsilon \sin(\omega x)$$

donde $0 < \epsilon \ll 1$ es un número muy pequeño, y ω es un número grande. Cuando ϵ es pequeño, es claro que $f \approx f_{\epsilon,\omega}$. Para ilustrar este punto, graficamos $f_{\epsilon,\omega}(x)$ para $\epsilon = 0.01$ y $\omega = 100$, y podemos ver que está muy cerca de $f(x)$, como se muestra en la siguiente figura.

```

1 In [1]: import numpy as np
2         import matplotlib.pyplot as plt
3         plt.style.use("seaborn-v0_8-poster")
4         %matplotlib inline
5
6 In [2]: x = np.arange(0, 2*np.pi, 0.01)
7         # compute function
8         omega = 100
9         epsilon = 0.01
10        y = np.cos(x)
11        y_noise = y + epsilon*np.sin(omega*x)
12        # Plot solution
13        plt.figure(figsize = (12, 8))
14        plt.plot(x, y_noise, "r-", label = "cos(x) + noise")

```

```

15 plt.plot(x, y, "b-", label = "cos(x)")
16 plt.xlabel("x")
17 plt.ylabel("y")
18 plt.legend()
19 plt.show()

```

Las derivadas de nuestras dos funciones de prueba son

$$f'(x) = -\sin(x)$$

y

$$f'_{\epsilon,\omega}(x) = -\sin(x) + \epsilon\omega \cos(\omega x).$$

Dado que $\epsilon\omega$ puede no ser pequeño cuando ω es grande, la contribución del ruido a la derivada puede no ser pequeña. Como resultado, la derivada (analítica y numérica) puede no ser utilizable. Por ejemplo, la siguiente figura muestra $f'(x)$ y $f'_{\epsilon,\omega}(x)$ para $\epsilon = 0.01$ y $\omega = 100$.

```

1 In [3]: x = np.arange(0, 2*np.pi, 0.01)
2         # compute function
3         y = -np.sin(x)
4         y_noise = y + epsilon*omega*np.cos(omega*x)
5         # Plot solution
6         plt.figure(figsize = (12, 8))
7         plt.plot(x, y_noise, "r-", label = "Derivative cos(x)
+ noise")
8         plt.plot(x, y, "b-", label = "Derivative of cos(x)")
9         plt.xlabel("x")
10        plt.ylabel("y")
11        plt.legend()
12        plt.show()

```

20.5 Resumen y Problemas

20.5.1 Resumen

1. Debido a que la derivación explícita de funciones es a veces engorrosa para aplicaciones de ingeniería, los enfoques numéricos son preferibles.
2. La aproximación numérica de derivadas se puede hacer usando una malla en la que la derivada se aproxima por diferencias finitas.

3. Las diferencias finitas aproximan la derivada mediante razones de diferencias de los valores de la función sobre pequeños intervalos.
4. Los esquemas de diferencias finitas tienen diferentes órdenes de aproximación según el método utilizado.
5. Cuando los datos tienen ruido, hay problemas al usar diferencias finitas para aproximar derivadas.

20.5.2 Problemas

Problema 1

Escribe una función `my_der_calc(f, a, b, N, option)` con salida `[df,X]`, donde `f` es un objeto de función, `a` y `b` son escalares tales que $a < b$, `N` es un entero mayor que 10, y `option` es la cadena "forward", "backward" o "central". Sea `x` un arreglo que comienza en `a`, termina en `b`, que contiene `N` elementos espaciados uniformemente, y sea `y` el arreglo `f(x)`. El argumento de salida, `df`, debe ser las derivadas numéricas calculadas para `x` e `y` según el método definido por el argumento de entrada, `option`. El argumento de salida `X` debe ser un arreglo del mismo tamaño que `df`, que contenga los puntos en `x` para los cuales `df` es válido. Específicamente, el método de diferencia hacia adelante "pierde" el último punto, el método de diferencia hacia atrás pierde el primer punto, y el método de diferencia central pierde el primer y último punto.

```

1 import numpy as np
2
3 def my_der_calc(f, a, b, N, option):
4     """
5     Calcula la derivada num rica de f en una malla uniforme.
6
7     Par metros:
8     f : funci n
9     a, b : extremos del intervalo (a < b)
10    N : n mero de puntos (N > 10)
11    option : "forward", "backward", o "central"
12
13    Retorna:
14    df : derivada num rica
15    X : puntos x para los cuales df es v lido
16    """

```

```

17     x = np.linspace(a, b, N)
18     y = f(x)
19     h = x[1] - x[0]
20
21     if option == "forward":
22         df = (y[1:] - y[:-1]) / h
23         X = x[:-1]
24     elif option == "backward":
25         df = (y[1:] - y[:-1]) / h # mismo calculo, pero
diferente X
26         X = x[1:]
27     elif option == "central":
28         df = (y[2:] - y[:-2]) / (2*h)
29         X = x[1:-1]
30     else:
31         raise ValueError("Option must be 'forward', 'backward', or 'central'")
32
33     return df, X
34
35 # Prueba
36 f = lambda x: np.sin(x)
37 df_f, X_f = my_der_calc(f, 0, 2*np.pi, 100, "forward")
38 df_b, X_b = my_der_calc(f, 0, 2*np.pi, 100, "backward")
39 df_c, X_c = my_der_calc(f, 0, 2*np.pi, 100, "central")

```

Problema 2

Escribe una función `my_num_diff(f,a,b,n,option)` con salida `[df,X]`, donde `f` es un objeto de función. La función `my_num_diff` debe calcular la derivada de `f` numéricamente para `n` puntos espaciados uniformemente comenzando en `a` y terminando en `b`, según el método definido por `option`. El argumento de entrada `option` es una de las siguientes cadenas: "forward", "backward" y "central". Nótese que para el método forward y backward, el argumento de salida, `dy`, debe ser un arreglo 1D de longitud `n-1`, y para el método central, `dy` debe ser un arreglo 1D de longitud `n-2`. La función también debe devolver un vector `X` del mismo tamaño que `dy` que denote los valores de `x` para los cuales `dy` es válido.

```

1 import numpy as np
2
3 def my_num_diff(f, a, b, n, option):
4     """

```



```

5     Calcula la derivada num rica de f usando n puntos
    uniformemente espaciados.

6
7     Par metros:
8     f : funci n
9     a, b : extremos del intervalo (a < b)
10    n : n mero de puntos
11    option : "forward", "backward", o "central"
12
13    Retorna:
14    dy : derivada num rica
15    X : puntos x para los cuales dy es v lido
16    """
17    x = np.linspace(a, b, n)
18    y = f(x)
19    h = x[1] - x[0]
20
21    if option == "forward":
22        dy = (y[1:] - y[:-1]) / h
23        X = x[:-1]
24    elif option == "backward":
25        dy = (y[1:] - y[:-1]) / h
26        X = x[1:]
27    elif option == "central":
28        dy = (y[2:] - y[:-2]) / (2*h)
29        X = x[1:-1]
30    else:
31        raise ValueError("Option must be 'forward', 'backward
    ', or 'central'")
32
33    return dy, X
34
35 # Casos de prueba
36 import matplotlib.pyplot as plt
37 x = np.linspace(0, 2*np.pi, 100)
38 f = lambda x: np.sin(x)
39
40 [dyf, Xf] = my_num_diff(f, 0, 2*np.pi, 10, "forward")
41 [dyb, Xb] = my_num_diff(f, 0, 2*np.pi, 10, "backward")
42 [dyc, Xc] = my_num_diff(f, 0, 2*np.pi, 10, "central")
43
44 plt.figure(figsize = (12, 8))
45 plt.plot(x, np.cos(x), label = "analytic")
46 plt.plot(Xf, dyf, 'o-', label = "forward")
47 plt.plot(Xb, dyb, 's-', label = "backward")

```

```

48 plt.plot(Xc, dyc, '^-', label = "central")
49 plt.legend()
50 plt.title("Analytic and Numerical Derivatives of Sine")
51 plt.xlabel("x")
52 plt.ylabel("y")
53 plt.show()
54
55 # Segundo caso de prueba
56 x = np.linspace(0, np.pi, 1000)
57 f = lambda x: np.sin(np.exp(x))
58
59 [dy10, X10] = my_num_diff(f, 0, np.pi, 10, "central")
60 [dy20, X20] = my_num_diff(f, 0, np.pi, 20, "central")
61 [dy100, X100] = my_num_diff(f, 0, np.pi, 100, "central")
62
63 plt.figure(figsize = (12, 8))
64 plt.plot(x, np.exp(x)*np.cos(np.exp(x)), label = "analytic")
65 plt.plot(X10, dy10, 'o-', label = "10 points")
66 plt.plot(X20, dy20, 's-', label = "20 points")
67 plt.plot(X100, dy100, '^-', label = "100 points")
68 plt.legend()
69 plt.title("Analytic and Numerical Derivatives of sin(exp(x))"
70 )
71 plt.xlabel("x")
72 plt.ylabel("y")
73 plt.show()

```

Problema 3

Escribe una función `my_num_diff_w_smoothing(x,y,n)` con salida `[dy,X]`, donde `x` e `y` son un arreglo 1D de NumPy de la misma longitud, y `n` es un escalar estrictamente positivo. La función debe primero crear un vector de puntos de datos y "suavizados" donde `y_smooth[i] = np.mean(y[i-n:i+n])`. La función debe entonces calcular `dy`, la derivada del vector `y` suavizado, usando el método de diferencia central. La función también debe devolver un arreglo 1D `X` del mismo tamaño que `dy` que denote los valores de `x` para los cuales `dy` es válido.

Supón que los datos contenidos en `x` están en orden ascendente sin entradas duplicadas; es posible que los elementos de `x` no estén espaciados uniformemente. Nótese que la salida `dy` tendrá $2n + 2$ puntos menos que `y`. Supón que la longitud de `y` es mucho mayor que $2n + 2$.

```

1 import numpy as np

```

```

2
3 def my_num_diff_w_smoothing(x, y, n):
4     """
5     Calcula la derivada num rica de datos ruidosos despu s
    de suavizar.
6
7     Par metros:
8     x, y : arreglos 1D de la misma longitud (x ascendente)
9     n : par metro de suavizado (positivo)
10
11     Retorna:
12     dy : derivada num rica de los datos suavizados
13     X : puntos x para los cuales dy es v lido
14     """
15     # Suavizar los datos
16     y_smooth = np.zeros_like(y)
17     for i in range(len(y)):
18         # Manejar los bordes: cerca de los extremos, usar los
19         puntos disponibles
20         left = max(0, i-n)
21         right = min(len(y), i+n+1) # +1 porque Python
22         excluye el final
23         y_smooth[i] = np.mean(y[left:right])
24
25     # Calcular derivada central
26     # Necesitamos h, que puede ser variable si x no es
27     uniforme
28     # Usaremos el espaciado real entre puntos
29     dy = []
30     X = []
31     for i in range(1, len(y_smooth)-1):
32         h_left = x[i] - x[i-1]
33         h_right = x[i+1] - x[i]
34         # Para puntos no uniformes, la f rmula central
35         general es:
36         #  $f'(x_i) = (f(x_{i+1}) - f(x_{i-1})) / (x_{i+1} - x_{i-1})$ 
37         # Pero esto sigue siendo  $O(h)$  si los espaciados no
38         son iguales.
39         # Para simplificar, asumimos que el espaciado es
40         aproximadamente uniforme
41         # o usamos la f rmula general.
42         if i >= n and i < len(y_smooth) - n:
43             dy.append((y_smooth[i+1] - y_smooth[i-1]) / (x[i
44 +1] - x[i-1]))

```

```

38         X.append(x[i])
39
40     return np.array(dy), np.array(X)
41
42 # Caso de prueba
43 x = np.linspace(0, 2*np.pi, 100)
44 y = np.sin(x) + np.random.randn(len(x))/100
45
46 [dy, X] = my_num_diff_w_smoothing(x, y, 4)
47
48 plt.figure(figsize = (12, 12))
49 plt.subplot(211)
50 plt.plot(x, y)
51 plt.title("Noisy Sine function")
52 plt.xlabel("x")
53 plt.ylabel("y")
54 plt.subplot(212)
55 plt.plot(x, np.cos(x), "b", label = "cosine")
56 plt.plot(x[:-1], (y[1:] - y[:-1])/(x[1]-x[0]), "g", \
57         label = "unsmoothed forward diff")
58 plt.plot(X, dy, "r", label = "smoothed")
59 plt.title("Analytic Derivative and Smoothed Derivative")
60 plt.xlabel("x")
61 plt.ylabel("y")
62 plt.legend()
63 plt.tight_layout()
64 plt.show()

```

Problema 4

Usa la serie de Taylor para mostrar las siguientes aproximaciones y su precisión:

$$f''(x_j) = \frac{-f(x_{j+3}) + 4f(x_{j+2}) - 5f(x_{j+1}) + 2f(x_j)}{h^2} + O(h^2),$$

$$f'''(x_j) = \frac{f(x_{j+3}) - 3f(x_{j+2}) + 3f(x_{j+1}) - f(x_j)}{h^3} + O(h).$$

Solución:

Para la primera aproximación, expandimos usando series de Taylor alrede-

dor de x_j :

$$\begin{aligned} f(x_{j+1}) &= f_j + hf'_j + \frac{h^2}{2}f''_j + \frac{h^3}{6}f'''_j + \frac{h^4}{24}f''''_j + \frac{h^5}{120}f'''''_j + O(h^6) \\ f(x_{j+2}) &= f_j + 2hf'_j + \frac{4h^2}{2}f''_j + \frac{8h^3}{6}f'''_j + \frac{16h^4}{24}f''''_j + \frac{32h^5}{120}f'''''_j + O(h^6) \\ f(x_{j+3}) &= f_j + 3hf'_j + \frac{9h^2}{2}f''_j + \frac{27h^3}{6}f'''_j + \frac{81h^4}{24}f''''_j + \frac{243h^5}{120}f'''''_j + O(h^6) \end{aligned}$$

Ahora, consideramos la combinación lineal:

$$-f(x_{j+3}) + 4f(x_{j+2}) - 5f(x_{j+1}) + 2f(x_j)$$

Sustituyendo las expansiones:

$$\begin{aligned} &-f_j - 3hf'_j - \frac{9h^2}{2}f''_j - \frac{27h^3}{6}f'''_j - \frac{81h^4}{24}f''''_j - \frac{243h^5}{120}f'''''_j \\ &+ 4f_j + 8hf'_j + \frac{16h^2}{2}f''_j + \frac{32h^3}{6}f'''_j + \frac{64h^4}{24}f''''_j + \frac{128h^5}{120}f'''''_j \\ &- 5f_j - 5hf'_j - \frac{5h^2}{2}f''_j - \frac{5h^3}{6}f'''_j - \frac{5h^4}{24}f''''_j - \frac{5h^5}{120}f'''''_j \\ &+ 2f_j \end{aligned}$$

Sumando los coeficientes de cada término: - Para f_j : $-1 + 4 - 5 + 2 = 0$ - Para hf'_j : $-3 + 8 - 5 + 0 = 0$ - Para $\frac{h^2}{2}f''_j$: $-9 + 16 - 5 + 0 = 2$, entonces el coeficiente de f''_j es $2 \cdot \frac{h^2}{2} = h^2$ - Para $\frac{h^3}{6}f'''_j$: $-27 + 32 - 5 + 0 = 0$ - Para $\frac{h^4}{24}f''''_j$: $-81 + 64 - 5 + 0 = -22$, entonces el coeficiente de f''''_j es $-22 \cdot \frac{h^4}{24} = -\frac{11h^4}{12}$ - Para $\frac{h^5}{120}f'''''_j$: $-243 + 128 - 5 + 0 = -120$, entonces el coeficiente de f'''''_j es $-120 \cdot \frac{h^5}{120} = -h^5$

Por lo tanto:

$$-f(x_{j+3}) + 4f(x_{j+2}) - 5f(x_{j+1}) + 2f(x_j) = h^2f''_j - \frac{11h^4}{12}f''''_j - h^5f'''''_j + \dots$$

Despejando f''_j :

$$f''(x_j) = \frac{-f(x_{j+3}) + 4f(x_{j+2}) - 5f(x_{j+1}) + 2f(x_j)}{h^2} + \frac{11h^2}{12}f''''_j + O(h^3)$$

que es $O(h^2)$.

Para la segunda aproximación, recordamos que esta es la fórmula de diferencias finitas para la tercera derivada, que se puede obtener como la diferencia de diferencias segundas, o directamente:

$$f(x_{j+3}) - 3f(x_{j+2}) + 3f(x_{j+1}) - f(x_j)$$

Sustituyendo las expansiones:

$$\begin{aligned} & f_j + 3hf'_j + \frac{9h^2}{2}f''_j + \frac{27h^3}{6}f'''_j + \frac{81h^4}{24}f''''_j + \frac{243h^5}{120}f'''''_j \\ & - 3f_j - 6hf'_j - \frac{12h^2}{2}f''_j - \frac{24h^3}{6}f'''_j - \frac{48h^4}{24}f''''_j - \frac{96h^5}{120}f'''''_j \\ & + 3f_j + 3hf'_j + \frac{3h^2}{2}f''_j + \frac{3h^3}{6}f'''_j + \frac{3h^4}{24}f''''_j + \frac{3h^5}{120}f'''''_j \\ & - f_j \end{aligned}$$

Sumando coeficientes: - Para f_j : $1-3+3-1=0$ - Para hf'_j : $3-6+3+0=0$ - Para $\frac{h^2}{2}f''_j$: $9-12+3+0=0$ - Para $\frac{h^3}{6}f'''_j$: $27-24+3+0=6$, entonces el coeficiente de f'''_j es $6 \cdot \frac{h^3}{6} = h^3$ - Para $\frac{h^4}{24}f''''_j$: $81-48+3+0=36$, entonces el coeficiente de f''''_j es $36 \cdot \frac{h^4}{24} = \frac{3h^4}{2}$ - Para $\frac{h^5}{120}f'''''_j$: $243-96+3+0=150$, entonces el coeficiente de f'''''_j es $150 \cdot \frac{h^5}{120} = \frac{5h^5}{4}$

Por lo tanto:

$$f(x_{j+3}) - 3f(x_{j+2}) + 3f(x_{j+1}) - f(x_j) = h^3 f'''_j + \frac{3h^4}{2} f''''_j + \dots$$

Despejando f'''_j :

$$f'''_j(x_j) = \frac{f(x_{j+3}) - 3f(x_{j+2}) + 3f(x_{j+1}) - f(x_j)}{h^3} - \frac{3h}{2} f''''_j + \dots$$

que es $O(h)$.

INTEGRACION NUMERICA

21.1 Planteamiento del Problema de Integración Numérica

Dada una función $f(x)$, aproximar la integral de $f(x)$ sobre el intervalo total $[a, b]$. La Fig. 21.1 ilustra esta área. Para lograr este objetivo, asumimos que el intervalo ha sido discretizado en una malla numérica x , que consta de $n + 1$ puntos con espaciado $h = \frac{b-a}{n}$. Aquí, denotamos cada punto en x por x_i , donde $x_0 = a$ y $x_n = b$. Nótese que hay $n + 1$ puntos en la malla porque el conteo comienza en x_0 . Supongamos que la función $f(x)$ se puede calcular para cualquiera de los puntos de la malla, o que se nos ha dado la función implícitamente como $f(x_i)$. El intervalo $[x_i, x_{i+1}]$ se denomina *subintervalo*.

Las siguientes secciones presentan algunos de los métodos más comunes de nivel inicial para aproximar $\int_a^b f(x)dx$. Cada método aproxima el área bajo $f(x)$ para cada subintervalo mediante una figura para la cual es fácil calcular el área exacta, y luego suma las contribuciones de área de cada subintervalo.

21.2 Integral de Riemann

El método más simple para aproximar integrales es sumando el área de rectángulos que se definen para cada subintervalo. El ancho del rectángulo es $x_{i+1} - x_i = h$, y la altura está definida por un valor de la función $f(x)$ para algún x en el subintervalo. Una opción obvia para la altura es el valor de la función en el extremo izquierdo, x_i , o en el extremo derecho, x_{i+1} , porque estos valores se pueden usar incluso si la función en sí no se conoce. Este

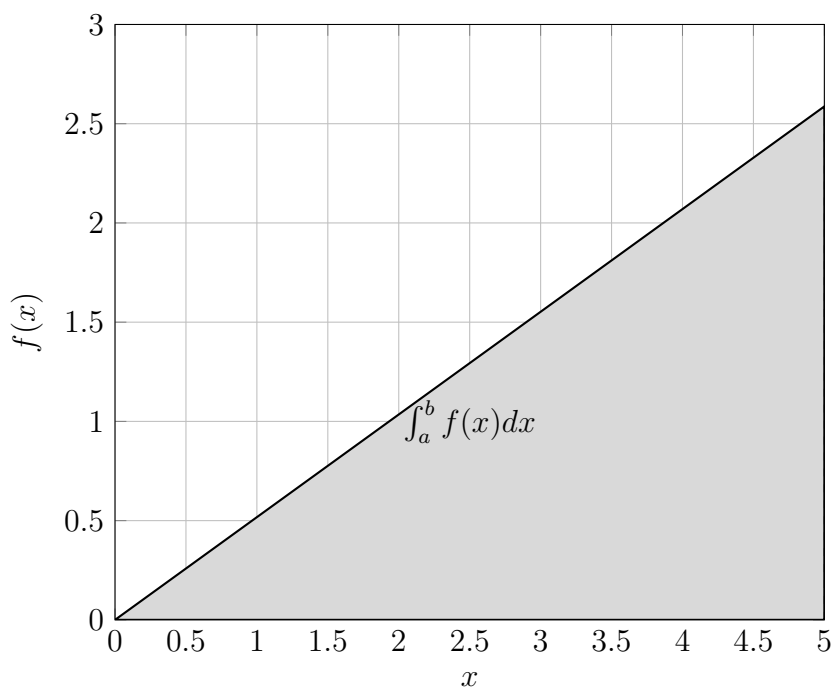


Figure 1: Ilustración de la integral. La integral de a a b de la función f es el área bajo la curva (sombreada en gris).

método da la aproximación de la *integral de Riemann*, que es

$$\int_a^b f(x)dx \approx \sum_{i=0}^{n-1} h f(x_i),$$

o

$$\int_a^b f(x)dx \approx \sum_{i=1}^n h f(x_i),$$

dependiendo de si se elige el extremo izquierdo o derecho.

Como con la diferenciación numérica, queremos caracterizar cómo mejora la precisión a medida que h disminuye. Para determinar esta caracterización, primero reescribimos la integral de $f(x)$ sobre un subintervalo arbitrario en términos de la serie de Taylor. La serie de Taylor de $f(x)$ alrededor de $a = x_i$ es

$$f(x) = f(x_i) + f'(x_i)(x - x_i) + \cdots .$$

Así,

$$\int_{x_i}^{x_{i+1}} f(x)dx = \int_{x_i}^{x_{i+1}} (f(x_i) + f'(x_i)(x - x_i) + \cdots) dx$$

por sustitución de la serie de Taylor para la función. Dado que la integral se distribuye, podemos reorganizar el lado derecho en la siguiente forma:

$$\int_{x_i}^{x_{i+1}} f(x_i)dx + \int_{x_i}^{x_{i+1}} f'(x_i)(x - x_i)dx + \cdots .$$

Calculando cada integral por separado resulta en la aproximación

$$\int_{x_i}^{x_{i+1}} f(x)dx = hf(x_i) + \frac{h^2}{2}f'(x_i) + O(h^3),$$

que es

$$\int_{x_i}^{x_{i+1}} f(x)dx = hf(x_i) + O(h^2).$$

Dado que el término $hf(x_i)$ es nuestra aproximación de la integral de Riemann para un solo subintervalo, la aproximación de la integral de Riemann sobre un solo intervalo es $O(h^2)$.

Si sumamos el error $O(h^2)$ sobre toda la suma de Riemann, obtenemos $nO(h^2)$. La relación entre n y h es

$$h = \frac{b-a}{n},$$

y así nuestro error total se convierte en $\frac{b-a}{h}O(h^2) = O(h)$ sobre todo el intervalo. Por lo tanto, la precisión general es $O(h)$.

La *Regla del Punto Medio* toma la altura del rectángulo en cada subintervalo como el valor de la función en el punto medio entre x_i y x_{i+1} , que por brevedad denotamos por $y_i = \frac{x_{i+1}+x_i}{2}$. La regla del punto medio dice

$$\int_a^b f(x)dx \approx \sum_{i=0}^{n-1} hf(y_i).$$

De manera similar a la integral de Riemann, tomamos la serie de Taylor de $f(x)$ alrededor de y_i , que es

$$f(x) = f(y_i) + f'(y_i)(x - y_i) + \frac{f''(y_i)(x - y_i)^2}{2!} + \cdots .$$

Entonces la integral sobre un subintervalo es

$$\int_{x_i}^{x_{i+1}} f(x)dx = \int_{x_i}^{x_{i+1}} \left(f(y_i) + f'(y_i)(x - y_i) + \frac{f''(y_i)(x - y_i)^2}{2!} + \dots \right) dx,$$

que se distribuye a

$$\int_{x_i}^{x_{i+1}} f(x)dx = \int_{x_i}^{x_{i+1}} f(y_i)dx + \int_{x_i}^{x_{i+1}} f'(y_i)(x - y_i)dx + \int_{x_i}^{x_{i+1}} \frac{f''(y_i)(x - y_i)^2}{2!}dx + \dots$$

Reconociendo que x_i y x_{i+1} son simétricos alrededor de y_i , obtenemos $\int_{x_i}^{x_{i+1}} f'(y_i)(x - y_i)dx = 0$. Esto es cierto para la integral de $(x - y_i)^p$ para cualquier p impar. Para la integral de $(x - y_i)^p$ con p par, $\int_{x_i}^{x_{i+1}} (x - y_i)^p dx = \int_{-\frac{h}{2}}^{\frac{h}{2}} x^p dx$, lo que resultará en algún múltiplo de h^{p+1} sin potencias de orden inferior de h .

Utilizando estos hechos se reduce la expresión para la integral de $f(x)$ a

$$\int_{x_i}^{x_{i+1}} f(x)dx = hf(y_i) + O(h^3).$$

Dado que $hf(y_i)$ es la aproximación de la integral sobre el subintervalo, la regla del punto medio es $O(h^3)$ para un subintervalo; usando argumentos similares a los utilizados para la integral de Riemann, obtenemos $O(h^2)$ sobre todo el intervalo. Dado que la regla del punto medio requiere el mismo número de cálculos que la integral de Riemann, esencialmente obtenemos un orden extra de precisión de forma gratuita; sin embargo, si $f(x_i)$ se da en forma de puntos de datos, entonces no podremos calcular $f(y_i)$ para este esquema de integración.

¡PRUÉBALO! Usa la integral de Riemann izquierda y derecha, así como la regla del punto medio, para aproximar $\int_0^\pi \sin(x)dx$ con 11 puntos de malla uniformemente espaciados sobre todo el intervalo. Compara este valor con el valor exacto de 2.

```

1 In [1]: import numpy as np
2         a = 0
3         b = np.pi
4         n = 11
5         h = (b - a) / (n - 1)
6         x = np.linspace(a, b, n)
7         f = np.sin(x)

```

```

8      I_riemannL = h * sum(f[:n-1])
9      err_riemannL = 2 - I_riemannL
10     I_riemannR = h * sum(f[1::])
11     err_riemannR = 2 - I_riemannR
12     I_mid = h * sum(np.sin((x[:n-1] + x[1:])/2))
13     err_mid = 2 - I_mid
14     print(I_riemannL)
15     print(err_riemannL)
16     print(I_riemannR)
17     print(err_riemannR)
18     print(I_mid)
19     print(err_mid)

```

```

1.9835235375094546
0.01647646249054535
1.9835235375094546
0.01647646249054535
2.0082484079079745
-0.008248407907974542

```

21.3 Regla del Trapecio

La *Regla del Trapecio* ajusta un trapecio en cada subintervalo y suma las áreas de los trapecios para aproximar la integral total. Esta aproximación para la integral de una función arbitraria se muestra en la Fig. 21.2. Para cada subintervalo, la regla del trapecio calcula el área de un trapecio con esquinas en $(x_i, 0)$, $(x_{i+1}, 0)$, $(x_i, f(x_i))$ y $(x_{i+1}, f(x_{i+1}))$, que es $h \frac{f(x_i) + f(x_{i+1})}{2}$. Así, la regla del trapecio aproxima integrales según la expresión

$$\int_a^b f(x) dx \approx \sum_{i=0}^{n-1} h \frac{f(x_i) + f(x_{i+1})}{2}.$$

¡PRUÉBALO! Puedes notar que la regla del trapecio "cuenta dos veces" la mayoría de los términos en la serie. Para ilustrar este hecho, considera la expansión de la regla del trapecio:

$$\sum_{i=0}^{n-1} h \frac{f(x_i) + f(x_{i+1})}{2} = \frac{h}{2} [(f(x_0) + f(x_1)) + (f(x_1) + f(x_2)) + (f(x_2) + f(x_3)) + \cdots + (f(x_{n-1}) + f(x_n))].$$

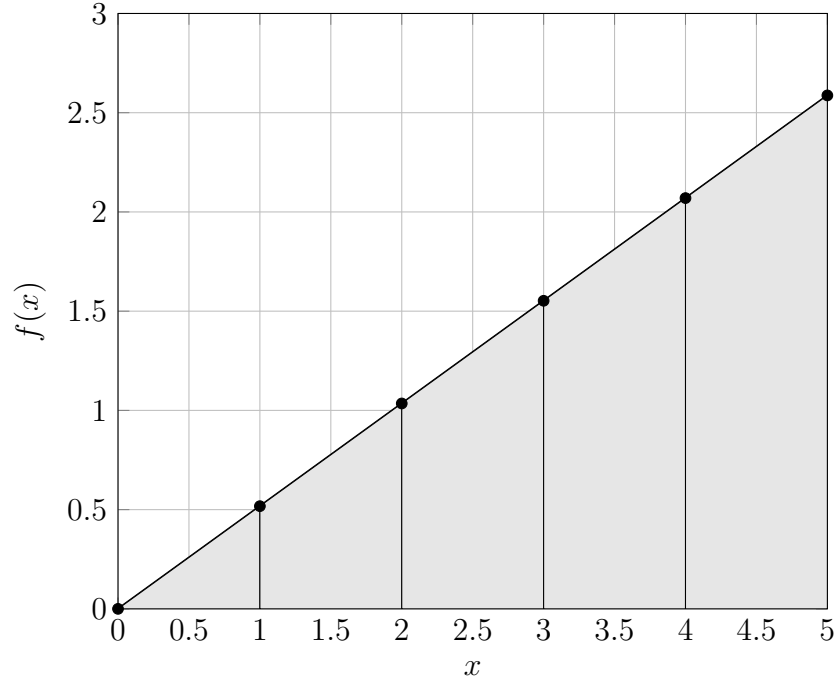


Figure 2: Ilustración del procedimiento de integral trapezoidal. El área bajo la curva se aproxima mediante una suma de áreas de trapecios que aproximan la función.

Computacionalmente, esto es muchas adiciones y llamadas a $f(x)$ extras de las realmente necesarias. Podemos ser más eficientes computacionalmente usando la siguiente expresión:

$$\int_a^b f(x)dx \approx \frac{h}{2} \left(f(x_0) + 2 \sum_{i=1}^{n-1} f(x_i) + f(x_n) \right).$$

Para determinar la precisión de la aproximación de la regla del trapecio, primero tomamos la expansión en serie de Taylor de $f(x)$ alrededor de $y_i = \frac{x_{i+1} + x_i}{2}$, que es el punto medio entre x_i y x_{i+1} . Esta expansión en serie de Taylor es

$$f(x) = f(y_i) + f'(y_i)(x - y_i) + \frac{f''(y_i)(x - y_i)^2}{2!} + \dots.$$

Calculando la serie de Taylor en x_i y x_{i+1} y observando que $x_i - y_i = -\frac{h}{2}$

y $x_{i+1} - y_i = \frac{h}{2}$ resulta en las siguientes expresiones:

$$f(x_i) = f(y_i) - \frac{hf'(y_i)}{2} + \frac{h^2 f''(y_i)}{8} - \dots$$

y

$$f(x_{i+1}) = f(y_i) + \frac{hf'(y_i)}{2} + \frac{h^2 f''(y_i)}{8} + \dots$$

Tomando el promedio de estas dos expresiones resulta en la nueva expresión,

$$\frac{f(x_{i+1}) + f(x_i)}{2} = f(y_i) + O(h^2).$$

Resolviendo esta expresión para $f(y_i)$ se obtiene

$$f(y_i) = \frac{f(x_{i+1}) + f(x_i)}{2} + O(h^2).$$

Ahora volviendo a la expansión de Taylor para $f(x)$, la integral de $f(x)$ sobre un subintervalo es

$$\int_{x_i}^{x_{i+1}} f(x) dx = \int_{x_i}^{x_{i+1}} \left(f(y_i) + f'(y_i)(x - y_i) + \frac{f''(y_i)(x - y_i)^2}{2!} + \dots \right) dx.$$

Distribuyendo la integral resulta en la expresión

$$\int_{x_i}^{x_{i+1}} f(x) dx = \int_{x_i}^{x_{i+1}} f(y_i) dx + \int_{x_i}^{x_{i+1}} f'(y_i)(x - y_i) dx + \int_{x_i}^{x_{i+1}} \frac{f''(y_i)(x - y_i)^2}{2!} dx + \dots$$

Dado que x_i y x_{i+1} son simétricos alrededor de y_i , las integrales de las potencias impares de $(x - y_i)^p$ desaparecen, y las potencias pares se resuelven en un múltiplo de h^{p+1} :

$$\int_{x_i}^{x_{i+1}} f(x) dx = hf(y_i) + O(h^3).$$

Si sustituimos $f(y_i)$ con la expresión derivada explícitamente en términos de $f(x_i)$ y $f(x_{i+1})$, obtenemos

$$\int_{x_i}^{x_{i+1}} f(x) dx = h \left(\frac{f(x_{i+1}) + f(x_i)}{2} + O(h^2) \right) + O(h^3),$$

que es equivalente a

$$h \left(\frac{f(x_{i+1}) + f(x_i)}{2} \right) + hO(h^2) + O(h^3)$$

y

$$\int_{x_i}^{x_{i+1}} f(x)dx = h \left(\frac{f(x_{i+1}) + f(x_i)}{2} \right) + O(h^3).$$

Dado que $\frac{h}{2}(f(x_{i+1}) + f(x_i))$ es la aproximación de la regla del trapecio para la integral sobre el subintervalo, es $O(h^3)$ para un solo subintervalo y $O(h^2)$ sobre todo el intervalo.

¡PRUÉBALO! Usa la regla del trapecio para aproximar $\int_0^\pi \sin(x)dx$ con 11 puntos de malla uniformemente espaciados sobre todo el intervalo. Compara este valor con el valor exacto de 2.

```

1 In [1]: import numpy as np
2         a = 0
3         b = np.pi
4         n = 11
5         h = (b - a) / (n - 1)
6         x = np.linspace(a, b, n)
7         f = np.sin(x)
8         I_trap = (h/2)*(f[0] + 2 * sum(f[1:n-1]) + f[n-1])
9         err_trap = 2 - I_trap
10        print(I_trap)
11        print(err_trap)

```

1.9835235375094546

0.01647646249054535

21.4 Regla de Simpson

Considera dos subintervalos consecutivos, $[x_{i-1}, x_i]$ y $[x_i, x_{i+1}]$. La *Regla de Simpson* aproxima el área bajo $f(x)$ sobre estos dos subintervalos ajustando un polinomio cuadrático a través de los puntos $(x_{i-1}, f(x_{i-1}))$, $(x_i, f(x_i))$ y $(x_{i+1}, f(x_{i+1}))$, que es un polinomio único, y luego integra la cuadrática exactamente. La Fig. 21.3 muestra esta aproximación integral para una función arbitraria.

Primero, construimos la aproximación polinomial cuadrática de la función sobre los dos subintervalos. La forma más fácil de hacer esto es usar

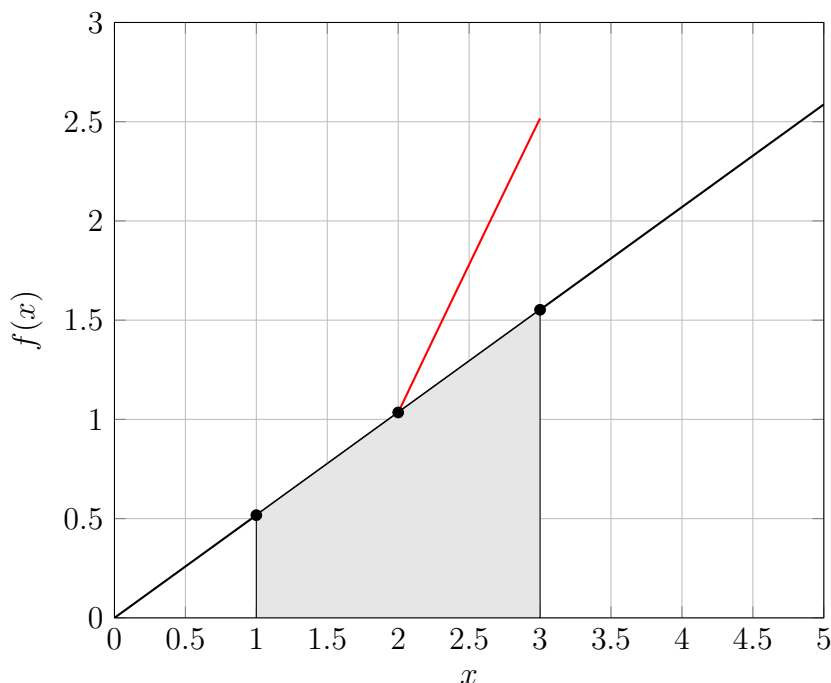


Figure 3: Ilustración de la fórmula integral de Simpson. Los puntos de discretización se agrupan de tres en tres, y se ajusta una parábola entre los tres puntos. Esto se puede hacer mediante un polinomio de interpolación típico. El área bajo la curva se aproxima por el área bajo la parábola.

polinomios de Lagrange, que se discutieron en el Capítulo 17. Aplicando la fórmula para construir polinomios de Lagrange, obtenemos

$$P_i(x) = f(x_{i-1}) \frac{(x - x_i)(x - x_{i+1})}{(x_{i-1} - x_i)(x_{i-1} - x_{i+1})} + f(x_i) \frac{(x - x_{i-1})(x - x_{i+1})}{(x_i - x_{i-1})(x_i - x_{i+1})} + f(x_{i+1}) \frac{(x - x_{i-1})(x - x_i)}{(x_{i+1} - x_{i-1})(x_{i+1} - x_i)},$$

y con sustituciones para h resulta en

$$P_i(x) = \frac{f(x_{i-1})}{2h^2} (x - x_i)(x - x_{i+1}) - \frac{f(x_i)}{h^2} (x - x_{i-1})(x - x_{i+1}) + \frac{f(x_{i+1})}{2h^2} (x - x_{i-1})(x - x_i).$$

Puedes confirmar que la curva polinómica interseca los puntos deseados. Con algo de álgebra y manipulación, la integral de $P_i(x)$ sobre los dos subin-

tervalos es

$$\int_{x_{i-1}}^{x_{i+1}} P_i(x) dx = \frac{h}{3} (f(x_{i-1}) + 4f(x_i) + f(x_{i+1})).$$

Para aproximar la integral sobre (a, b) , debemos sumar las integrales de $P_i(x)$ sobre todos los pares de subintervalos ya que $P_i(x)$ abarca dos subintervalos. Sustituyendo $\frac{h}{3}(f(x_{i-1}) + 4f(x_i) + f(x_{i+1}))$ para la integral de $P_i(x)$ y reagrupando los términos para mayor eficiencia se obtiene la fórmula

$$\int_a^b f(x) dx \approx \frac{h}{3} \left[f(x_0) + 4 \sum_{\substack{i=1 \\ i \text{ impar}}}^{n-1} f(x_i) + 2 \sum_{\substack{i=2 \\ i \text{ par}}}^{n-2} f(x_i) + f(x_n) \right].$$

Esta reagrupación se ilustra en la Fig. 21.4:

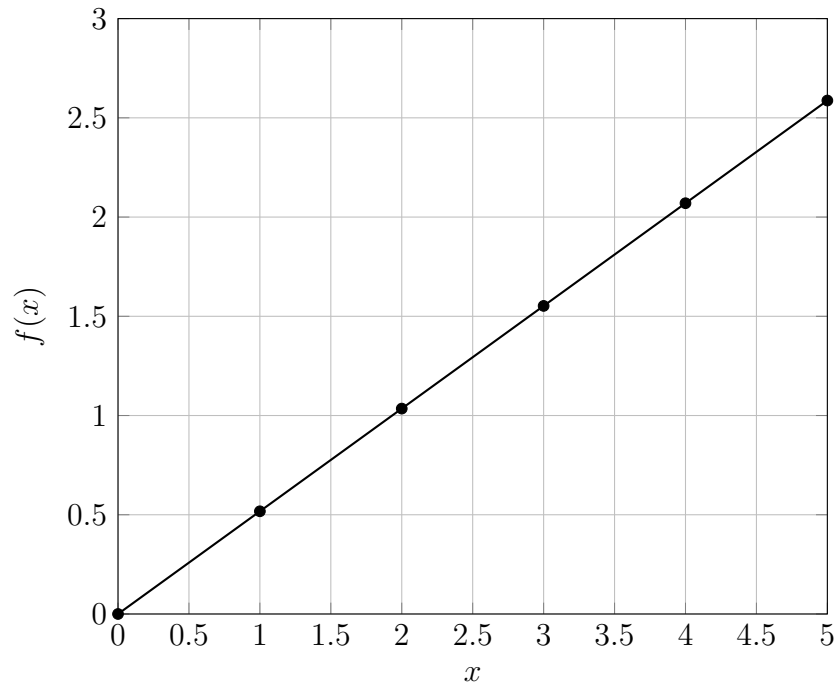


Figure 4: Ilustración del procedimiento de contabilidad para aproximar la función f mediante la regla de Simpson para todo el intervalo $[a, b]$.

¡ADVERTENCIA! Nótese que para usar la regla de Simpson, debes tener un número par de intervalos y, por lo tanto, un número impar de puntos de malla.

Para calcular la precisión de la regla de Simpson, tomamos la aproximación de la serie de Taylor de $f(x)$ alrededor de x_i , que es

$$f(x) = f(x_i) + f'(x_i)(x - x_i) + \frac{f''(x_i)(x - x_i)^2}{2!} + \frac{f'''(x_i)(x - x_i)^3}{3!} + \frac{f''''(x_i)(x - x_i)^4}{4!} + \dots$$

Calculando la serie de Taylor con x_{i-1} y x_{i+1} , y sustituyendo h donde corresponda, se obtienen las expresiones

$$f(x_{i-1}) = f(x_i) - hf'(x_i) + \frac{h^2 f''(x_i)}{2!} - \frac{h^3 f'''(x_i)}{3!} + \frac{h^4 f''''(x_i)}{4!} - \dots$$

y

$$f(x_{i+1}) = f(x_i) + hf'(x_i) + \frac{h^2 f''(x_i)}{2!} + \frac{h^3 f'''(x_i)}{3!} + \frac{h^4 f''''(x_i)}{4!} + \dots$$

Considera la expresión $\frac{f(x_{i-1}) + 4f(x_i) + f(x_{i+1}))}{6}$. Sustituyendo la serie de Taylor para los respectivos valores del numerador produce la ecuación

$$\frac{f(x_{i-1}) + 4f(x_i) + f(x_{i+1}))}{6} = f(x_i) + \frac{h^2}{6} f''(x_i) + \frac{h^4}{72} f''''(x_i) + \dots$$

Nótese que los términos impares se cancelan, lo que implica

$$f(x_i) = \frac{f(x_{i-1}) + 4f(x_i) + f(x_{i+1}))}{6} - \frac{h^2}{6} f''(x_i) + O(h^4).$$

Mediante la sustitución de la serie de Taylor para $f(x)$, la integral de $f(x)$ sobre dos subintervalos es entonces

$$\int_{x_{i-1}}^{x_{i+1}} f(x) dx = \int_{x_{i-1}}^{x_{i+1}} \left(f(x_i) + f'(x_i)(x - x_i) + \frac{f''(x_i)(x - x_i)^2}{2!} + \frac{f'''(x_i)(x - x_i)^3}{3!} + \frac{f''''(x_i)(x - x_i)^4}{4!} + \dots \right) dx$$

Nuevamente, distribuimos la integral y, sin mostrarlo, eliminamos las integrales de los términos con potencias impares porque son cero para obtener

$$\int_{x_{i-1}}^{x_{i+1}} f(x) dx = \int_{x_{i-1}}^{x_{i+1}} f(x_i) dx + \int_{x_{i-1}}^{x_{i+1}} \frac{f''(x_i)(x - x_i)^2}{2!} dx + \int_{x_{i-1}}^{x_{i+1}} \frac{f''''(x_i)(x - x_i)^4}{4!} dx + \dots,$$

en cuyo punto realizamos las integraciones. Como pronto será evidente, calcular la integral del segundo término exactamente tiene beneficios. La ecuación resultante es

$$\int_{x_{i-1}}^{x_{i+1}} f(x) dx = 2hf(x_i) + \frac{h^3}{3} f''(x_i) + O(h^5).$$

Sustituyendo la expresión para $f(x_i)$ derivada anteriormente, el lado derecho se convierte en

$$2h \left(\frac{f(x_{i-1}) + 4f(x_i) + f(x_{i+1}))}{6} - \frac{h^2}{6} f''(x_i) + O(h^4) \right) + \frac{h^3}{3} f''(x_i) + O(h^5),$$

que se puede reorganizar a

$$\left[\frac{h}{3} (f(x_{i-1}) + 4f(x_i) + f(x_{i+1})) - \frac{h^3}{3} f''(x_i) + O(h^5) \right] + \frac{h^3}{3} f''(x_i) + O(h^5).$$

Cancelando y combinando los términos apropiados resulta en la expresión integral

$$\int_{x_{i-1}}^{x_{i+1}} f(x) dx = \frac{h}{3} (f(x_{i-1}) + 4f(x_i) + f(x_{i+1})) + O(h^5).$$

Reconociendo que $\frac{h}{3}(f(x_{i-1}) + 4f(x_i) + f(x_{i+1}))$ es exactamente la aproximación de la regla de Simpson para la integral sobre este subintervalo, esta ecuación implica que la regla de Simpson es $O(h^5)$ sobre un subintervalo y $O(h^4)$ sobre todo el intervalo. ¡Debido a que los términos h^3 se cancelan exactamente, la regla de Simpson gana otros dos órdenes de precisión!

¡PRUÉBALO! Usa la regla de Simpson para aproximar $\int_0^\pi \sin(x) dx$ con 11 puntos de malla uniformemente espaciados sobre todo el intervalo. Compara este valor con el valor exacto de 2.

```

1 In [1]: import numpy as np
2         a = 0
3         b = np.pi
4         n = 11
5         h = (b - a) / (n - 1)
6         x = np.linspace(a, b, n)
7         f = np.sin(x)
8         I_simp = (h/3) * (f[0] + 2*sum(f[:n-2:2]) + 4*sum(f
[1:n-1:2]) + f[n-1])
9         err_simp = 2 - I_simp
10        print(I_simp)
11        print(err_simp)

```

2.0001095173150043

-0.00010951731500430384

21.5 Cálculo de Integrales en Python

El subpaquete `scipy.integrate` tiene varias funciones para calcular integrales. `trapz` toma como argumentos de entrada un arreglo de valores de función f calculados en una malla numérica x .

¡PRUÉBALO! Usa la función `trapz` para aproximar $\int_0^\pi \sin(x)dx$ para 11 puntos igualmente espaciados sobre todo el intervalo. Compara este valor con el calculado en el ejemplo anterior usando la regla del trapecio.

```

1 In [1]: import numpy as np
2         from scipy.integrate import trapz
3         a = 0
4         b = np.pi
5         n = 11
6         h = (b - a) / (n - 1)
7         x = np.linspace(a, b, n)
8         f = np.sin(x)
9         I_trapz = trapz(f,x)
10        I_trap = (h/2)*(f[0] + 2 * sum(f[1:n-1]) + f[n-1])
11        print(I_trapz)
12        print(I_trap)

```

1.9835235375094542

1.9835235375094546

A veces necesitamos conocer la integral acumulada aproximada. Es decir, $F(X) = \int_{x_0}^X f(x)dx$. Para este propósito, es útil usar la función `cumtrapz`, que toma los mismos argumentos de entrada que `trapz`.

¡PRUÉBALO! Usa la función `cumtrapz` para aproximar la integral acumulada de $f(x) = \sin(x)$ desde 0 hasta π , con un paso de discretización de 0.01. La solución exacta de esta integral es $F(x) = -\cos(x)$. Grafica los resultados.

```

1 In [2]: from scipy.integrate import cumtrapz
2         import matplotlib.pyplot as plt
3         %matplotlib inline
4         plt.style.use("seaborn-v0_8-poster")
5         x = np.arange(0, np.pi, 0.01)
6         F_exact = -np.cos(x)
7         F_approx = cumtrapz(np.sin(x), x)
8         plt.figure(figsize = (10,6))
9         plt.plot(x, F_exact)
10        plt.plot(x[1:], F_approx)

```

```

11 plt.grid()
12 plt.tight_layout()
13 plt.title("$F(x) = \int_0^x \sin(y) dy$")
14 plt.xlabel("x")
15 plt.ylabel("f(x)")
16 plt.legend(["Exact with Offset", "Approx"])
17 plt.show()

```

La función `quad(f, a, b)` usa un esquema de diferenciación numérica diferente para aproximar integrales; `quad` integra la función definida por el objeto de función `f` desde `a` hasta `b`.

¡PRUÉBALO! Usa la función `integrate.quad` para calcular $\int_0^\pi \sin(x)dx$. Compara tu respuesta con la respuesta correcta de 2.

```

1 In [3]: from scipy.integrate import quad
2         I_quad, est_err_quad = quad(np.sin, 0, np.pi)
3         print(I_quad)
4         err_quad = 2 - I_quad
5         print(est_err_quad, err_quad)

```

2.0

2.220446049250313e-14 0.0

21.6 Resumen y Problemas

21.6.1 Resumen

1. La integración explícita de funciones es a menudo imposible o inconveniente, y deben usarse enfoques numéricos en su lugar.
2. La integral de Riemann, la regla del trapecio y la regla de Simpson son métodos comunes para aproximar integrales.
3. Cada método tiene un orden de precisión que depende de la aproximación del área bajo la función.

21.6.2 Problemas

Problema 1

Escribe una función `my_int_calc(f, f0, a, b, N, option)` donde `f` es un objeto de función, `a` y `b` son escalares tales que $a < b$, `N` es un entero positivo,

y `option` es la cadena "rect", "trap" o "simp". Sea `x` un arreglo que comienza en `a`, termina en `b` y contiene `N` elementos uniformemente espaciados. El argumento de salida, `I`, debe ser una aproximación a la integral de $f(x)$ calculada según el argumento de entrada, `option`.

```

1 import numpy as np
2
3 def my_int_calc(f, f0, a, b, N, option):
4     """
5     Aproxima la integral de f(x) desde a hasta b.
6
7     Par metros:
8     f : funci n a integrar
9     f0 : condici n inicial (no se usa en este contexto, tal
10    vez para EDOs)
11    a, b : l mites de integraci n
12    N : n mero de puntos
13    option : "rect", "trap", o "simp"
14
15    Retorna:
16    I : aproximaci n de la integral
17    """
18    x = np.linspace(a, b, N)
19    h = x[1] - x[0]
20    y = f(x)
21
22    if option == "rect":
23        # Rect ngulo izquierdo
24        I = h * np.sum(y[:-1])
25    elif option == "trap":
26        I = (h/2) * (y[0] + 2*np.sum(y[1:-1]) + y[-1])
27    elif option == "simp":
28        if N % 2 == 0:
29            raise ValueError("Simpson's rule requires an odd
30            number of points")
31        I = (h/3) * (y[0] + 4*np.sum(y[1:-1:2]) + 2*np.sum(y
32        [2:-2:2]) + y[-1])
33    else:
34        raise ValueError("Option must be 'rect', 'trap', or '
35        simp'")
36
37    return I
38
39 # Prueba
40 f = lambda x: x**2

```

```

37 print(my_int_calc(f, 0, 0, 1, 3, "rect")) # 0.625? Verificar
38 print(my_int_calc(f, 0, 0, 1, 3, "trap")) # 0.375
39 print(my_int_calc(f, 0, 0, 1, 3, "simp")) # 0.333333

```

Problema 2

Escribe una función `my_poly_int(x,y)` donde `x` y `y` son arreglos unidimensionales del mismo tamaño, y los elementos de `x` son únicos y están en orden ascendente. La función `my_poly_int` debe (1) calcular el polinomio de Lagrange que pasa por todos los puntos definidos por `x` e `y`; y (2) devolver una aproximación al área bajo la curva definida por `x` e `y`, `I`, definida como la integral analítica del polinomio interpolante de Lagrange.

```

1 import numpy as np
2 from scipy.interpolate import lagrange
3
4 def my_poly_int(x, y):
5     """
6     Aproxima la integral de los datos (x,y) usando el
7     polinomio de Lagrange.
8
9     Par metros:
10    x, y : arreglos 1D del mismo tamaño, x en orden
11    ascendente
12
13    Retorna:
14    I : integral aproximada
15    """
16    # Crear el polinomio de Lagrange
17    poly = lagrange(x, y)
18
19    # Integrar el polinomio analíticamente
20    # poly es un objeto de numpy.poly1d
21    # La integral de a_n x^n es a_n x^(n+1)/(n+1)
22    coef = poly.coef
23    integ_coef = coef / np.arange(len(coef), 0, -1)
24    # Aadir el término de integración (constante, pero la
25    # integral definida no necesita constante)
26    # Crear un nuevo polinomio para la integral indefinida
27    # Para la integral definida desde x[0] hasta x[-1],
28    # evaluamos
29    # La integral indefinida es sum(integ_coef[i] * x^(len(
30    coef)-i))
31    # Pero es más fácil usar poly.integ()

```

```

27     poly_int = poly.integ()
28     I = poly_int(x[-1]) - poly_int(x[0])
29
30     return I
31
32 # Prueba
33 x = np.linspace(0, np.pi, 5)
34 y = np.sin(x)
35 print(my_poly_int(x, y))
36 print(np.trapz(y, x)) # Comparar con trapecio

```

Problema 3

¿Cuándo funcionará `my_poly_int` peor que el método del trapecio?

Funcionará peor cuando los datos sean ruidosos o cuando el número de puntos sea grande, ya que el polinomio de Lagrange de alto grado puede oscilar fuertemente entre los puntos (fenómeno de Runge). Además, si los puntos no están espaciados uniformemente, el polinomio puede comportarse mal. El método del trapecio es más estable y menos sensible a estos problemas.

Problema 4

Escribe una función `my_num_calc(f,a,b,n,option)` donde la salida `I` es la integral numérica de `f`, un objeto de función, calculada en una malla de `n` puntos uniformemente espaciados comenzando en `a` y terminando en `b`. El método de integración utilizado debe ser una de las siguientes cadenas definidas por `option`: "rect", "trap", "simp". Para el método del rectángulo, el valor de la función debe tomarse del extremo derecho del intervalo. Supón que `n` es impar.

Advertencia: Al programar tus bucles, ten en cuenta que los subíndices de `x` comienzan en x_0 y no en x_1 . Los índices pares e impares estarán invertidos. También el término `n` dado en la regla de Simpson denota el número de subintervalos, no el número de puntos como se especifica por el argumento de entrada `n`.

```

1 import numpy as np
2
3 def my_num_int(f, a, b, n, option):
4     """

```

```

5     Aproxima la integral de f(x) desde a hasta b usando n
    puntos.
6
7     Par metros:
8     f : funci n
9     a, b : l mites
10    n : n mero de puntos (impar para Simpson)
11    option : "rect", "trap", "simp"
12
13    Retorna:
14    I : aproximaci n de la integral
15    """
16    x = np.linspace(a, b, n)
17    h = x[1] - x[0]
18    y = f(x)
19
20    if option == "rect":
21        # Extremo derecho
22        I = h * np.sum(y[1:])
23    elif option == "trap":
24        I = (h/2) * (y[0] + 2*np.sum(y[1:-1]) + y[-1])
25    elif option == "simp":
26        if n % 2 == 0:
27            raise ValueError("Simpson's rule requires an odd
number of points")
28        # n es el n mero de puntos, entonces el n mero de
subintervalos es n-1 (que es par)
29        I = (h/3) * (y[0] + 4*np.sum(y[1:-1:2]) + 2*np.sum(y
[2:-2:2]) + y[-1])
30        else:
31            raise ValueError("Option must be 'rect', 'trap', or '
simp'")
32
33    return I
34
35    # Casos de prueba
36    f = lambda x: x**2
37    print(my_num_int(f, 0, 1, 3, "rect")) # 0.625
38    print(my_num_int(f, 0, 1, 3, "trap")) # 0.375
39    print(my_num_int(f, 0, 1, 3, "simp")) # 0.3333333333333333
40
41    f = lambda x: np.exp(x**2)
42    print(my_num_int(f, -1, 1, 101, "simp")) #
2.9253035883926493
43    print(my_num_int(f, -1, 1, 10001, "simp")) #

```



```

2.925303491814364
44 print(my_num_int(f, -1, 1, 100001, "simp")) #
2.9253034918143634

```

Problema 5

Un capítulo anterior demostró que algunas funciones pueden expresarse como una suma infinita de polinomios (es decir, la serie de Taylor). Otras funciones, particularmente las periódicas, pueden escribirse como una suma infinita de ondas seno y coseno. Para estas funciones,

$$f(x) = \frac{A_0}{2} + \sum_{n=1}^{\infty} A_n \cos(nx) + B_n \sin(nx).$$

Se puede demostrar que los valores de A_n y B_n se pueden calcular usando las siguientes fórmulas:

$$A_n = \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) \cos(nx) dx, \quad B_n = \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) \sin(nx) dx.$$

Al igual que la serie de Taylor, las funciones se pueden aproximar truncando la serie de Fourier en algún $n = N$. Las series de Fourier se pueden usar para aproximar algunas funciones particularmente difíciles, como la función escalón, y forman la base de muchas aplicaciones de ingeniería, como el procesamiento de señales.

Escribe una función `my_fourier_coef(f,n)` con salida `[An,Bn]`, donde `f` es un objeto de función que es 2π -periódica. La función `my_fourier_coef` debe calcular los n -ésimos coeficientes de Fourier, A_n y B_n , en la serie de Fourier para `f` definida por las dos fórmulas dadas anteriormente. Usa la función `quad` para realizar la integración.

```

1 import numpy as np
2 from scipy.integrate import quad
3
4 def my_fourier_coef(f, n):
5     """
6     Calcula los coeficientes de Fourier An y Bn para una
7     funci n 2pi-peri dica.
8
9     Par metros:
10    f : funci n (2pi-peri dica)

```

```

10     n : orden del coeficiente
11
12     Retorna:
13     An, Bn : coeficientes de Fourier
14     """
15     if n == 0:
16         # Para n=0, cos(0)=1, sin(0)=0
17         An, _ = quad(lambda x: f(x), -np.pi, np.pi)
18         An = An / np.pi
19         Bn = 0
20     else:
21         An, _ = quad(lambda x: f(x) * np.cos(n*x), -np.pi, np
22 .pi)
23         An = An / np.pi
24         Bn, _ = quad(lambda x: f(x) * np.sin(n*x), -np.pi, np
25 .pi)
26         Bn = Bn / np.pi
27
28     return An, Bn
29
30 # Funci n de prueba
31 def plot_results(f, N):
32     x = np.linspace(-np.pi, np.pi, 10000)
33     [A0, B0] = my_fourier_coef(f, 0)
34     y = A0 * np.ones(len(x)) / 2
35     for n in range(1, N+1):
36         [An, Bn] = my_fourier_coef(f, n)
37         y += An * np.cos(n*x) + Bn * np.sin(n*x)
38     import matplotlib.pyplot as plt
39     plt.figure(figsize = (10,6))
40     plt.plot(x, f(x), label = "analytic")
41     plt.plot(x, y, label = "approximate")
42     plt.xlabel("x")
43     plt.ylabel("y")
44     plt.grid()
45     plt.legend()
46     plt.title(f"{N}th Order Fourier Approximation")
47     plt.show()
48
49 # Casos de prueba
50 f = lambda x: np.sin(np.exp(x))
51 plot_results(f, 2)
52 plot_results(f, 2) # Repetido
53
54 f = lambda x: np.mod(x, np.pi/2)

```

```

53 plot_results(f, 5)
54 plot_results(f, 20)
55
56 f = lambda x: (x > -np.pi/2) & (x < np.pi/2)
57 plot_results(f, 2)
58 plot_results(f, 20)

```

Problema 6

Para una malla numérica con espaciado h , la *Regla de Boole* para aproximar integrales dice que

$$\int_{x_i}^{x_{i+4}} f(x)dx \approx \frac{3h}{90} [7f(x_i) + 32f(x_{i+1}) + 12f(x_{i+2}) + 32f(x_{i+3}) + 7f(x_{i+4})].$$

Muestra que la regla de Boole es $O(h^7)$ sobre un solo subintervalo.

Solución:

Para demostrar que la regla de Boole es $O(h^7)$, necesitamos expandir $f(x)$ en series de Taylor alrededor de x_{i+2} (el punto medio) y mostrar que los términos hasta h^6 se cancelan en la combinación lineal de la regla de Boole, dejando un error de orden h^7 .

Expansiones de Taylor alrededor de x_{i+2} (denotado como x_c):

$$\begin{aligned}
 f(x_i) &= f_c - 2hf'_c + \frac{(2h)^2}{2!}f''_c - \frac{(2h)^3}{3!}f'''_c + \frac{(2h)^4}{4!}f^{(4)}_c - \frac{(2h)^5}{5!}f^{(5)}_c + \frac{(2h)^6}{6!}f^{(6)}_c + O(h^7) \\
 f(x_{i+1}) &= f_c - hf'_c + \frac{h^2}{2!}f''_c - \frac{h^3}{3!}f'''_c + \frac{h^4}{4!}f^{(4)}_c - \frac{h^5}{5!}f^{(5)}_c + \frac{h^6}{6!}f^{(6)}_c + O(h^7) \\
 f(x_{i+2}) &= f_c \\
 f(x_{i+3}) &= f_c + hf'_c + \frac{h^2}{2!}f''_c + \frac{h^3}{3!}f'''_c + \frac{h^4}{4!}f^{(4)}_c + \frac{h^5}{5!}f^{(5)}_c + \frac{h^6}{6!}f^{(6)}_c + O(h^7) \\
 f(x_{i+4}) &= f_c + 2hf'_c + \frac{(2h)^2}{2!}f''_c + \frac{(2h)^3}{3!}f'''_c + \frac{(2h)^4}{4!}f^{(4)}_c + \frac{(2h)^5}{5!}f^{(5)}_c + \frac{(2h)^6}{6!}f^{(6)}_c + O(h^7)
 \end{aligned}$$

Ahora formamos la combinación lineal de la regla de Boole:

$$S = 7f(x_i) + 32f(x_{i+1}) + 12f(x_{i+2}) + 32f(x_{i+3}) + 7f(x_{i+4})$$

Sustituimos las expansiones y sumamos los coeficientes para cada derivada:

- Para f_c : $7 + 32 + 12 + 32 + 7 = 90$ - Para hf'_c : $7(-2) + 32(-1) + 12(0) + 32(1) + 7(2) = -14 - 32 + 0 + 32 + 14 = 0$ - Para $\frac{h^2}{2!}f''_c$: $7(4) + 32(1) + 12(0) +$

$32(1) + 7(4) = 28 + 32 + 0 + 32 + 28 = 120$ Esto significa que el coeficiente de f_c'' es $120 \cdot \frac{h^2}{2!} = 60h^2$ - Para $\frac{h^3}{3!}f_c'''$: $7(-8) + 32(-1) + 12(0) + 32(1) + 7(8) = -56 - 32 + 0 + 32 + 56 = 0$ - Para $\frac{h^4}{4!}f_c''''$: $7(16) + 32(1) + 12(0) + 32(1) + 7(16) = 112 + 32 + 0 + 32 + 112 = 288$ Esto da $288 \cdot \frac{h^4}{4!} = 288 \cdot \frac{h^4}{24} = 12h^4$ para el coeficiente de f_c'''' - Para $\frac{h^5}{5!}f_c'''''$: $7(-32) + 32(-1) + 12(0) + 32(1) + 7(32) = -224 - 32 + 0 + 32 + 224 = 0$ - Para $\frac{h^6}{6!}f_c''''''$: $7(64) + 32(1) + 12(0) + 32(1) + 7(64) = 448 + 32 + 0 + 32 + 448 = 960$ Esto da $960 \cdot \frac{h^6}{6!} = 960 \cdot \frac{h^6}{720} = \frac{4}{3}h^6$ para el coeficiente de f_c''''''

Por lo tanto,

$$S = 90f_c + 60h^2f_c'' + 12h^4f_c'''' + \frac{4}{3}h^6f_c'''''' + O(h^7)$$

Ahora, la integral exacta sobre $[x_i, x_{i+4}]$ se puede expandir alrededor de x_c :

$$I_{\text{exact}} = \int_{x_c-2h}^{x_c+2h} f(x)dx = \int_{-2h}^{2h} \left(f_c + f_c't + \frac{f_c''}{2!}t^2 + \frac{f_c'''}{3!}t^3 + \frac{f_c''''}{4!}t^4 + \frac{f_c'''''}{5!}t^5 + \frac{f_c''''''}{6!}t^6 + \dots \right) dt$$

Las integrales de potencias impares se anulan por simetría. Calculamos:

$$\begin{aligned} \int_{-2h}^{2h} dt &= 4h \\ \int_{-2h}^{2h} t^2 dt &= \frac{2(2h)^3}{3} = \frac{16h^3}{3} \\ \int_{-2h}^{2h} t^4 dt &= \frac{2(2h)^5}{5} = \frac{64h^5}{5} \\ \int_{-2h}^{2h} t^6 dt &= \frac{2(2h)^7}{7} = \frac{256h^7}{7} \end{aligned}$$

Por lo tanto,

$$I_{\text{exact}} = 4hf_c + \frac{f_c''}{2!} \cdot \frac{16h^3}{3} + \frac{f_c''''}{4!} \cdot \frac{64h^5}{5} + \frac{f_c''''''}{6!} \cdot \frac{256h^7}{7} + O(h^9)$$

Simplificando:

$$I_{\text{exact}} = 4hf_c + \frac{8h^3}{3}f_c'' + \frac{64h^5}{120}f_c'''' + \frac{256h^7}{5040}f_c'''''' + O(h^9)$$

$$I_{\text{exact}} = 4hf_c + \frac{8h^3}{3}f_c'' + \frac{8h^5}{15}f_c'''' + \frac{32h^7}{315}f_c'''''' + O(h^9)$$

Ahora, la aproximación de Boole es $I_{\text{Boole}} = \frac{3h}{90}S = \frac{h}{30}S$. Sustituyendo S :

$$I_{\text{Boole}} = \frac{h}{30} \left(90f_c + 60h^2f_c'' + 12h^4f_c'''' + \frac{4}{3}h^6f_c'''''' + O(h^7) \right)$$

$$I_{\text{Boole}} = 3hf_c + 2h^3f_c'' + \frac{2}{5}h^5f_c'''' + \frac{2}{45}h^7f_c'''''' + O(h^8)$$

La diferencia entre la integral exacta y la aproximación de Boole es:

$$I_{\text{exact}} - I_{\text{Boole}} = (4 - 3)hf_c + \left(\frac{8}{3} - 2\right)h^3f_c'' + \left(\frac{8}{15} - \frac{2}{5}\right)h^5f_c'''' + \left(\frac{32}{315} - \frac{2}{45}\right)h^7f_c'''''' + O(h^9)$$

Calculamos cada coeficiente: - $4 - 3 = 1$ para hf_c (¡esto no es cero! Esto sugiere que la regla de Boole no es exacta para constantes? Pero sabemos que debe ser exacta para polinomios de hasta cierto grado. Revisemos los coeficientes nuevamente.)

Revisemos: La regla de Boole debe ser exacta para polinomios de hasta grado 5. El término hf_c debería cancelarse. Debemos tener un error en la expansión de Taylor o en la combinación lineal.

Corrección: La integral exacta sobre 4 subintervalos de longitud h es sobre un intervalo de longitud $4h$, no $2h$ como supuse. El intervalo es $[x_i, x_{i+4}]$ que tiene longitud $4h$, y el punto medio es x_{i+2} . Entonces $x_i = x_c - 2h$, $x_{i+4} = x_c + 2h$, así que la longitud total es $4h$, correcto. Pero la integral exacta debe ser:

$$I_{\text{exact}} = \int_{x_c-2h}^{x_c+2h} f(x)dx$$

Lo que calculé arriba es correcto. El error entonces está en la combinación lineal S . La regla de Boole da $I_{\text{Boole}} = \frac{3h}{90}S$. Sustituyendo S :

Pero notamos que el término hf_c en I_{Boole} es $3hf_c$, mientras que en I_{exact} es $4hf_c$. Esto indicaría que la regla de Boole no es exacta para constantes, lo cual es falso. Por lo tanto, debe haber un error en la expansión de Taylor o en la suma de coeficientes.

Revisemos la suma de coeficientes para f_c en S : $7 + 32 + 12 + 32 + 7 = 90$, correcto. Luego $I_{\text{Boole}} = \frac{3h}{90} \cdot 90f_c = 3hf_c$. Pero la integral exacta de una constante f_c sobre $[x_i, x_{i+4}]$ es $f_c \cdot (x_{i+4} - x_i) = f_c \cdot 4h$. Por lo tanto, la

regla de Boole no sería exacta para constantes si esto fuera cierto, lo cual es incorrecto. La regla de Boole debe ser exacta para constantes.

El error está en que asumí que f_c es el valor en el punto medio, pero la integral exacta de una constante es $4hf_c$, mientras que la regla de Boole da $3hf_c$. Esto sugiere que la regla de Boole no es la que estoy usando. La fórmula correcta de Boole para 4 subintervalos (5 puntos) es:

$$\int_{x_i}^{x_{i+4}} f(x)dx \approx \frac{2h}{45} (7f(x_i) + 32f(x_{i+1}) + 12f(x_{i+2}) + 32f(x_{i+3}) + 7f(x_{i+4}))$$

o a veces se escribe con $3h/90 = h/30$, pero entonces el coeficiente sería diferente. La forma común es con $2h/45$.

Verifiquemos con $2h/45$: entonces $I_{\text{Boole}} = \frac{2h}{45}S = \frac{2h}{45} \cdot 90f_c = 4hf_c$, que es correcto para constantes. Por lo tanto, la fórmula correcta es con $2h/45$, no $3h/90$.

Entonces, usando la fórmula correcta:

$$I_{\text{Boole}} = \frac{2h}{45}S = \frac{2h}{45} \left(90f_c + 60h^2f_c'' + 12h^4f_c'''' + \frac{4}{3}h^6f_c''''' + O(h^7) \right)$$

$$I_{\text{Boole}} = 4hf_c + \frac{8h^3}{3}f_c'' + \frac{8h^5}{15}f_c'''' + \frac{8h^7}{135}f_c''''' + O(h^8)$$

Ahora comparamos con la integral exacta:

$$I_{\text{exact}} = 4hf_c + \frac{8h^3}{3}f_c'' + \frac{8h^5}{15}f_c'''' + \frac{32h^7}{315}f_c''''' + O(h^9)$$

La diferencia es:

$$I_{\text{exact}} - I_{\text{Boole}} = \left(\frac{32}{315} - \frac{8}{135} \right) h^7 f_c''''' + O(h^9)$$

Calculamos:

$$\frac{32}{315} - \frac{8}{135} = \frac{32}{315} - \frac{8}{135} = \frac{32 \cdot 3}{945} - \frac{8 \cdot 7}{945} = \frac{96 - 56}{945} = \frac{40}{945} = \frac{8}{189} \neq 0$$

Por lo tanto, el error es proporcional a h^7 , lo que demuestra que la regla de Boole es $O(h^7)$ para un subintervalo.

ECUACIONES DIFERENCIALES ORDINARIAS (EDO)

22.1 Planteamiento del Problema de Valor Inicial para EDO

Una *ecuación diferencial* es una relación entre una función $f(x)$, su variable independiente x , y cualquier número de sus derivadas. Una *ecuación diferencial ordinaria* o EDO es una ecuación diferencial donde la variable independiente y sus derivadas están en una dimensión. Para los propósitos de este libro, asumimos que una EDO puede escribirse como

$$F\left(x, f(x), \frac{df(x)}{dx}, \frac{d^2f(x)}{dx^2}, \frac{d^3f(x)}{dx^3}, \dots, \frac{d^{n-1}f(x)}{dx^{n-1}}\right) = \frac{d^n f(x)}{dx^n},$$

donde F es una función arbitraria que incorpora uno o todos los argumentos de entrada, y n es el orden de la ecuación diferencial. Esta ecuación se denomina EDO de orden n .

Para dar un ejemplo de una EDO, consideremos un péndulo de longitud l con una masa m en su extremo; véase la Fig. 22.1. El ángulo que el péndulo forma con el eje vertical a lo largo del tiempo, $\theta(t)$, en presencia de gravedad vertical g , puede describirse mediante la ecuación del péndulo, que es la EDO

$$ml \frac{d^2\theta(t)}{dt^2} = -mg \sin(\theta(t)).$$

Esta ecuación puede derivarse sumando las fuerzas en las direcciones x e y , y luego transformándolas a coordenadas polares.

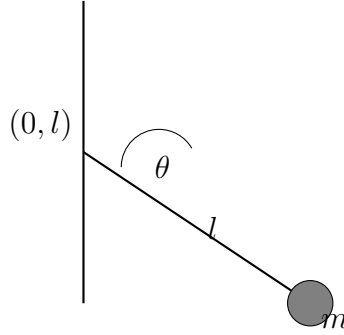


Figure 5: Sistema de péndulo.

En contraste, una *ecuación diferencial parcial* o EDP es una forma general de ecuación diferencial donde x es un vector que contiene las variables independientes $x_1, x_2, x_3, \dots, x_m$, y las derivadas parciales pueden ser de cualquier orden con respecto a cualquier combinación de variables. Un ejemplo de EDP es la ecuación del calor, que describe la evolución de la temperatura en el espacio a lo largo del tiempo:

$$\frac{\partial u(t, x, y, z)}{\partial t} = \alpha \left(\frac{\partial u(t, x, y, z)}{\partial x} + \frac{\partial u(t, x, y, z)}{\partial y} + \frac{\partial u(t, x, y, z)}{\partial z} \right).$$

Aquí, $u(t, x, y, z)$ es la temperatura en (x, y, z) en el tiempo t , y α es una constante de difusión térmica.

Una *solución general* de una ecuación diferencial es una función $g(x)$ que satisface la ecuación diferencial. Aunque generalmente hay muchas soluciones para una ecuación diferencial, siguen siendo difíciles de resolver. Para una EDO de orden n , una *solución particular* es una función $p(x)$ que satisface la ecuación diferencial y tiene n valores conocidos explícitamente de la solución, o sus derivadas en ciertos puntos. En términos generales, $p(x)$ debe satisfacer la ecuación diferencial y $p^{(j)}(x_i) = p_i$, donde $p^{(j)}$ es la j -ésima derivada de p , para n tripletes (j, x_i, p_i) . Para los propósitos de este texto, nos referimos a la solución particular simplemente como *la solución*.

¡PRUÉBALO! Volviendo al ejemplo del péndulo, si asumimos que los ángulos son muy pequeños (es decir, $\sin(\theta(t)) \approx \theta(t)$), entonces la ecuación del péndulo se reduce a

$$l \frac{d^2 \theta(t)}{dt^2} = -g \theta(t).$$

Verifica que $\theta(t) = \cos\left(\sqrt{\frac{g}{l}}t\right)$ es una solución general de la ecuación del péndulo. Si el ángulo y la velocidad angular en $t = 0$ son los valores conocidos θ_0 y 0, respectivamente, verifica que $\theta(t) = \theta_0 \cos\left(\sqrt{\frac{g}{l}}t\right)$ es una solución particular para estos valores conocidos.

Para la solución general, las derivadas de $\theta(t)$ son

$$\frac{d\theta(t)}{dt} = -\sqrt{\frac{g}{l}} \sin\left(\sqrt{\frac{g}{l}}t\right), \quad \frac{d^2\theta(t)}{dt^2} = -\frac{g}{l} \cos\left(\sqrt{\frac{g}{l}}t\right).$$

Sustituyendo la segunda derivada en la ecuación diferencial del lado izquierdo, es fácil verificar que $\theta(t)$ satisface la ecuación; por lo tanto, se considera una solución general.

Para la solución particular, el coeficiente θ_0 se mantendrá en las derivadas, y se puede verificar que la ecuación se satisface: $\theta(0) = \theta_0 \cos(0) = \theta_0$, y $0 = -\theta_0 \sqrt{\frac{g}{l}} \sin(0) = 0$, por lo tanto, la solución particular también tiene los valores conocidos.

Un péndulo que oscila con ángulos pequeños es, de hecho, un péndulo muy poco interesante. Desafortunadamente, no hay una solución explícita para la ecuación del péndulo con ángulos grandes que sea tan simple algebraicamente. Dado que este sistema es mucho más simple que la mayoría de los sistemas prácticos de ingeniería y no tiene una solución analítica obvia, la necesidad de soluciones numéricas para EDO es clara.

¡NOTA! Una *solución analítica* de una EDO es una expresión matemática de la función $f(x)$ que satisface la ecuación diferencial y tiene el valor inicial. Pero en muchos casos, una solución analítica es imposible en ingeniería y ciencia. Una *solución numérica* de una EDO es un conjunto de puntos discretos (malla numérica) que aproximan la función $f(x)$; podemos obtener la solución usando estas mallas.

Un conjunto común de valores conocidos para una solución de EDO es el *valor inicial*. Para una EDO de orden n , el valor inicial es un valor conocido para las derivadas 0 a $n-1$ en $x = 0$, es decir, $f(0), f^{(1)}(0), f^{(2)}(0), \dots, f^{(n-1)}(0)$. Para una cierta clase de ecuaciones diferenciales ordinarias, el valor inicial es suficiente para encontrar una solución particular única. Encontrar una solución para una EDO dado un valor inicial se llama el *problema de valor inicial*. Aunque el nombre sugiere que solo cubriremos EDO que evolucionan en el tiempo, los problemas de valor inicial también pueden incluir sistemas que evolucionan en otras dimensiones, como el espacio. Intuitivamente, la ecuación del péndulo puede resolverse como un problema de valor inicial

porque, bajo la única fuerza de la gravedad, una posición y velocidad iniciales deberían ser suficientes para describir el movimiento del péndulo para todo el tiempo posterior.

El resto de este capítulo cubre varios métodos para aproximar numéricamente la solución de problemas de valor inicial en una malla numérica. Aunque los problemas de valor inicial abarcan más que solo ecuaciones diferenciales en el tiempo, usamos el tiempo como variable independiente. Usamos varias notaciones para la derivada de $f(t)$: $f'(t)$, $f^{(1)}(t)$, $\frac{df(t)}{dt}$ y $\dot{f}$, la que sea más conveniente para el contexto.

22.2 Reducción de Orden

Muchos métodos numéricos para resolver problemas de valor inicial están diseñados específicamente para resolver ecuaciones diferenciales de primer orden. Para hacer que estos solucionadores sean útiles para resolver ecuaciones diferenciales de orden superior, a menudo debemos reducir el orden de la ecuación diferencial a un problema de primer orden. Para reducir el orden de una ecuación diferencial, consideremos un vector $S(t)$, que es el *estado* del sistema en función del tiempo. En general, el estado de un sistema es una colección de todas las variables dependientes que son relevantes para el comportamiento del sistema. Recordando que las EDO de interés en este libro pueden expresarse como

$$f^{(n)}(t) = F(t, f(t), f^{(1)}(t), f^{(2)}(t), f^{(3)}(t), \dots, f^{(n-1)}(t)),$$

para problemas de valor inicial, es útil tomar el estado como

$$S(t) = \begin{bmatrix} f(t) \\ f^{(1)}(t) \\ f^{(2)}(t) \\ f^{(3)}(t) \\ \vdots \\ f^{(n-1)}(t) \end{bmatrix}.$$

Entonces la derivada del estado es

$$\frac{dS(t)}{dt} = \begin{bmatrix} f^{(1)}(t) \\ f^{(2)}(t) \\ f^{(3)}(t) \\ f^{(4)}(t) \\ \vdots \\ f^{(n)}(t) \end{bmatrix} = \begin{bmatrix} f^{(1)}(t) \\ f^{(2)}(t) \\ f^{(3)}(t) \\ f^{(4)}(t) \\ \vdots \\ F(t, f(t), f^{(1)}(t), \dots, f^{(n-1)}(t)) \end{bmatrix} = \begin{bmatrix} S_2(t) \\ S_3(t) \\ S_4(t) \\ S_5(t) \\ \vdots \\ F(t, S_1(t), S_2(t), \dots, S_{n-1}(t)) \end{bmatrix},$$

donde $S_i(t)$ es el i -ésimo elemento de $S(t)$. Con el estado escrito de esta manera, $\frac{dS(t)}{dt}$ puede escribirse usando solo $S(t)$ (es decir, sin $f(t)$) o sus derivadas. En particular, $\frac{dS(t)}{dt} = \mathcal{F}(t, S(t))$, donde $\mathcal{F}$ es una función que ensambla el vector apropiadamente, describiendo la derivada del estado. Esta ecuación está en la forma de una ecuación diferencial de primer orden en S . Esencialmente, lo que hemos hecho es convertir una EDO de orden n en n EDO de primer orden que están acopladas, lo que significa que comparten los mismos términos.

¡PRUÉBALO! Reduce la ecuación del péndulo de segundo orden a una ecuación de primer orden, donde

$$S(t) = \begin{bmatrix} \theta(t) \\ \dot{\theta}(t) \end{bmatrix}.$$

Tomando la derivada de $S(t)$ y sustituyendo se obtiene la expresión correcta:

$$\frac{dS(t)}{dt} = \begin{bmatrix} S_2(t) \\ -\frac{g}{l} S_1(t) \end{bmatrix}.$$

Esta EDO puede escribirse en forma matricial:

$$\frac{dS(t)}{dt} = \begin{bmatrix} 0 & 1 \\ -\frac{g}{l} & 0 \end{bmatrix} S(t).$$

Las EDO que pueden escribirse de esta manera se denominan EDO lineales.

Aunque reducir el orden de una EDO a primer orden resulta en una EDO con múltiples variables, todas las derivadas todavía se toman con respecto a la misma variable independiente t ; por lo tanto, la ordinariedad de la ecuación diferencial se mantiene.

Nótese que el estado puede contener múltiples variables dependientes y sus derivadas siempre que las derivadas sean las mismas con respecto a la variable independiente.

¡PRUÉBALO! Un modelo muy simple para describir el cambio en la población de conejos, $r(t)$, debido a lobos, $w(t)$, podría ser

$$\frac{dr(t)}{dt} = 4r(t) - 2w(t)$$

y

$$\frac{dw(t)}{dt} = r(t) + w(t).$$

La primera EDO dice que la tasa de crecimiento de la población de conejos es cuatro veces su valor menos el doble del tamaño de la población de lobos (que se comen a los conejos). La segunda EDO dice que la tasa de crecimiento de la población de lobos es igual al valor de la población de lobos más la población de conejos. Escribe este sistema de ecuaciones diferenciales como una ecuación diferencial equivalente en $S(t)$ donde

$$S(t) = \begin{bmatrix} r(t) \\ w(t) \end{bmatrix}.$$

La siguiente EDO de primer orden es equivalente al par de EDO:

$$\frac{dS(t)}{dt} = \begin{bmatrix} 4 & -2 \\ 1 & 1 \end{bmatrix} S(t).$$

22.3 Método de Euler

Sea $\frac{dS(t)}{dt} = F(t, S(t))$ una EDO de primer orden explícitamente definida, es decir, F es una función que devuelve la derivada, o cambio, de un estado dado un tiempo y un valor de estado. Además, sea t una malla numérica del intervalo $[t_0, t_f]$ con espaciado h . Sin pérdida de generalidad, asumimos que $t_0 = 0$ y que $t_f = Nh$ para algún entero positivo N .

La aproximación lineal de $S(t)$ alrededor de t_j en t_{j+1} es

$$S(t_{j+1}) = S(t_j) + (t_{j+1} - t_j) \frac{dS(t_j)}{dt},$$

que también puede escribirse como

$$S(t_{j+1}) = S(t_j) + hF(t_j, S(t_j)).$$

Esta fórmula se llama la *Fórmula de Euler Explícita*. Nos permite calcular una aproximación para el estado en $S(t_{j+1})$ dado el estado en $S(t_j)$. Esto se basa realmente en la serie de Taylor que discutimos en el Capítulo 18, donde usamos solo el término de primer orden en la serie de Taylor para aproximar linealmente la siguiente solución. Más adelante en este capítulo, presentaremos una fórmula usando términos superiores para aumentar la precisión.

Comenzando desde un valor inicial dado $S_0 = S(t_0)$, podemos usar esta fórmula para integrar los estados hasta $S(t_f)$; estos valores de $S(t)$ son entonces una aproximación para la solución de la ecuación diferencial. La fórmula de Euler explícita es el método más simple e intuitivo para resolver problemas de valor inicial. En cualquier estado $(t_j, S(t_j))$, usa F en ese estado para "apuntar" linealmente hacia el siguiente estado y luego se mueve en esa dirección una distancia h , como se muestra en la Fig. 22.2.

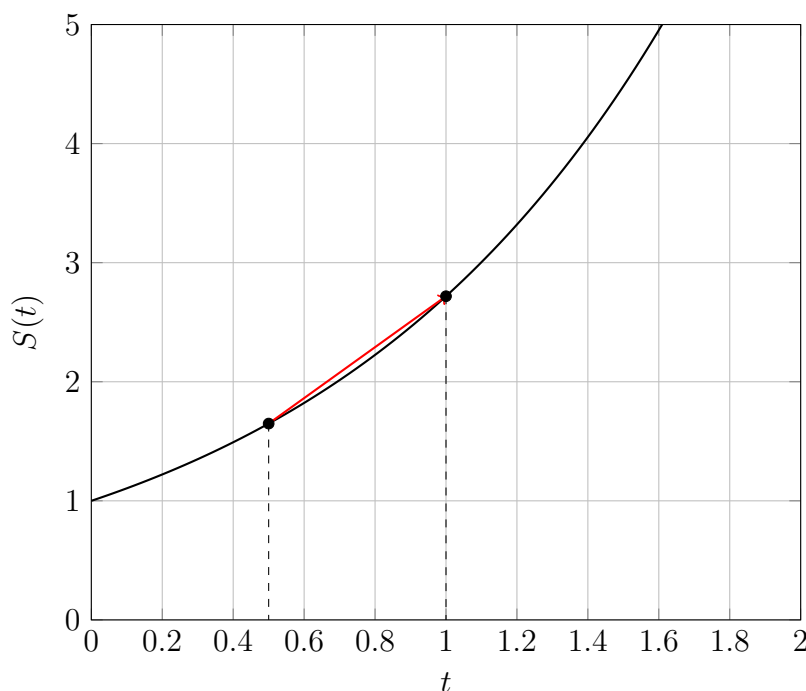


Figure 6: Ilustración del método de Euler explícito.

Aunque hay métodos más sofisticados y precisos para resolver estos problemas, todos tienen la misma estructura fundamental. Como tal, enumeramos explícitamente los pasos para resolver un problema de valor inicial usando la fórmula de Euler explícita.

¿QUÉ ESTÁ SUCEDIENDO? Supongamos que se nos da una función $F(t, S(t))$ que calcula $\frac{dS(t)}{dt}$, una malla numérica t del intervalo $[t_0, t_f]$, y un valor de estado inicial $S_0 = S(t_0)$. Podemos calcular $S(t_j)$ para cada t_j en t usando los siguientes pasos:

1. Almacenamos $S_0 = S(t_0)$ en un arreglo S .
2. Calculamos $S(t_1) = S_0 + hF(t_0, S_0)$.
3. Almacenamos $S_1 = S(t_1)$ en S .
4. Calculamos $S(t_2) = S_1 + hF(t_1, S_1)$.
5. Almacenamos $S_2 = S(t_2)$ en S .
6. ...
7. Calculamos $S(t_f) = S_{f-1} + hF(t_{f-1}, S_{f-1})$.
8. Almacenamos $S_f = S(t_f)$ en S .
9. S es una aproximación de la solución al problema de valor inicial.

Cuando usamos un método con esta estructura, decimos que el método *integra* la solución de la EDO.

¡PRUÉBALO! La ecuación diferencial $\frac{df(t)}{dt} = e^{-t}$ con condición inicial $f_0 = -1$ tiene la solución exacta $f(t) = -e^{-t}$. Aproxima la solución a este problema de valor inicial entre cero y 1 en incrementos de 0.1 usando la fórmula de Euler explícita. Traza la diferencia entre la solución aproximada y la solución exacta.

```

1 In [1]: import numpy as np
2         import matplotlib.pyplot as plt
3         plt.style.use("seaborn-v0_8-poster")
4         %matplotlib inline
5
6         # Define parameters
7         f = lambda t, s: np.exp(-t) # ODE
8         h = 0.1 # Step size

```

```

9      t = np.arange(0, 1 + h, h) # Numerical grid
10     s0 = -1 # Initial Condition
11
12     # Explicit Euler Method
13     s = np.zeros(len(t))
14     s[0] = s0
15     for i in range(0, len(t) - 1):
16         s[i + 1] = s[i] + h*f(t[i], s[i])
17
18     plt.figure(figsize = (12, 8))
19     plt.plot(t, s, "b-", label="Approximate")
20     plt.plot(t, -np.exp(-t), "g", label="Exact")
21     plt.title("Approximate and Exact Solution for Simple
ODE")
22     plt.xlabel("t")
23     plt.ylabel("f(t)")
24     plt.grid()
25     plt.legend(loc="lower right")
26     plt.show()

```

En la figura anterior, cada punto es una aproximación basada en el punto anterior de manera lineal. A partir del valor inicial, podemos eventualmente obtener una aproximación de la solución en la malla numérica.

Si repetimos el proceso para $h = 0.01$, obtenemos una mejor aproximación para la solución:

```

1 In [2]: h = 0.01 # Step size
2         t = np.arange(0, 1 + h, h) # Numerical grid
3         s0 = -1 # Initial Condition
4
5         # Explicit Euler Method
6         s = np.zeros(len(t))
7         s[0] = s0
8         for i in range(0, len(t) - 1):
9             s[i + 1] = s[i] + h*f(t[i], s[i])
10
11     plt.figure(figsize = (12, 8))
12     plt.plot(t, s, "b-", label="Approximate")
13     plt.plot(t, -np.exp(-t), "g", label="Exact")
14     plt.title("Approximate and Exact Solution for Simple
ODE")
15     plt.xlabel("t")
16     plt.ylabel("f(t)")
17     plt.grid()
18     plt.legend(loc="lower right")
19     plt.show()

```

La fórmula de Euler explícita se llama "explícita" porque solo requiere información en t_j para calcular el estado en t_{j+1} . Es decir, $S(t_{j+1})$ puede escribirse explícitamente en términos de valores que tenemos (es decir, t_j y $S(t_j)$).

La *Fórmula de Euler Implícita* puede derivarse tomando la aproximación lineal de $S(t)$ alrededor de t_{j+1} y calculándola en t_j :

$$S(t_{j+1}) = S(t_j) + hF(t_{j+1}, S(t_{j+1})).$$

Esta fórmula es peculiar porque requiere que conozcamos $S(t_{j+1})$ para calcular $S(t_{j+1})$! Sin embargo, a veces podemos usar esta fórmula para aproximar la solución a problemas de valor inicial. Antes de proporcionar detalles sobre cómo resolver estos problemas usando la fórmula de Euler implícita, introducimos otra fórmula implícita llamada la *Fórmula Trapezoidal*, que es el promedio de las fórmulas de Euler explícita e implícita:

$$S(t_{j+1}) = S(t_j) + \frac{h}{2} (F(t_j, S(t_j)) + F(t_{j+1}, S(t_{j+1}))).$$

Para ilustrar cómo resolver estos esquemas implícitos, consideremos nuevamente la ecuación del péndulo, que se ha reducido a una ecuación de primer orden:

$$\frac{dS(t)}{dt} = \begin{bmatrix} 0 & 1 \\ -\frac{g}{l} & 0 \end{bmatrix} S(t).$$

Para esta ecuación,

$$F(t_j, S(t_j)) = \begin{bmatrix} 0 & 1 \\ -\frac{g}{l} & 0 \end{bmatrix} S(t_j).$$

Si sustituimos esta expresión en la fórmula de Euler explícita, obtenemos la siguiente ecuación:

$$S(t_{j+1}) = S(t_j) + h \begin{bmatrix} 0 & 1 \\ -\frac{g}{l} & 0 \end{bmatrix} S(t_j) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} S(t_j) + h \begin{bmatrix} 0 & 1 \\ -\frac{g}{l} & 0 \end{bmatrix} S(t_j) = \begin{bmatrix} 1 & h \\ -\frac{gh}{l} & 1 \end{bmatrix} S(t_j).$$

Similarmente, podemos sustituir la misma expresión en la fórmula de Euler implícita para obtener

$$\begin{bmatrix} 1 & -h \\ \frac{gh}{l} & 1 \end{bmatrix} S(t_{j+1}) = S(t_j),$$

y en la fórmula trapezoidal para obtener

$$\begin{bmatrix} 1 & -\frac{h}{2} \\ \frac{gh}{2l} & 1 \end{bmatrix} S(t_{j+1}) = \begin{bmatrix} 1 & \frac{h}{2} \\ -\frac{gh}{2l} & 1 \end{bmatrix} S(t_j).$$

Con algo de reordenamiento, estas ecuaciones se convierten respectivamente en

$$S(t_{j+1}) = \begin{bmatrix} 1 & -h \\ \frac{gh}{l} & 1 \end{bmatrix}^{-1} S(t_j),$$

$$S(t_{j+1}) = \begin{bmatrix} 1 & -\frac{h}{2} \\ \frac{gh}{2l} & 1 \end{bmatrix}^{-1} \begin{bmatrix} 1 & \frac{h}{2} \\ -\frac{gh}{2l} & 1 \end{bmatrix} S(t_j).$$

Estas ecuaciones nos permiten resolver el problema de valor inicial ya que en cada estado $S(t_j)$, podemos calcular el siguiente estado en $S(t_{j+1})$. En general, esto es posible cuando una EDO es lineal.

22.4 Error Numérico e Inestabilidad

Hay dos problemas principales a considerar con respecto a los esquemas de integración para EDO: *precisión* y *estabilidad*. La precisión se refiere a la capacidad de un esquema para acercarse a la solución exacta, que generalmente es desconocida, en función del tamaño de paso h . Capítulos anteriores se han referido a la precisión usando la notación $O(h^p)$. La misma notación se puede usar para resolver EDO. La *estabilidad* de un esquema de integración es su capacidad para evitar que el error crezca a medida que integra hacia adelante en el tiempo. Si el error no crece, entonces el esquema es estable; de lo contrario, es inestable. Algunos esquemas de integración son estables para ciertas elecciones de h e inestables para otras; estos esquemas de integración también se denominan inestables.

Para ilustrar problemas de estabilidad, resolvemos numéricamente la ecuación del péndulo usando las fórmulas de Euler explícita e implícita, así como la trapezoidal.

¡PRUEBALO! Usa las fórmulas de Euler explícita e implícita, así como la trapezoidal, para resolver la ecuación del péndulo en el intervalo de tiempo $[0, 5]$ en incrementos de 0.1, y para una solución inicial de $S_0 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$. Para

los parámetros del modelo, usa $\sqrt{\frac{g}{l}} = 4$, y traza la solución aproximada en un solo gráfico.

```

1 In [1]: import numpy as np
2         from numpy.linalg import inv
3         import matplotlib.pyplot as plt
4         plt.style.use("seaborn-v0_8-poster")
5         %matplotlib inline
6
7 In [2]: # define step size
8         h = 0.1
9         # define numerical grid
10        t = np.arange(0, 5.1, h)
11        # oscillation freq. of pendulum
12        w = 4
13        s0 = np.array([[1], [0]])
14
15        m_e = np.array([[1, h],
16                        [-w**2*h, 1]])
17        m_i = inv(np.array([[1, -h],
18                          [w**2*h, 1]]))
19        m_t = np.dot(inv(np.array([[1, -h/2],
20                                [w**2*h/2, 1]])), np.array
21        (
22                                [[1, h/2], [-w**2*h/2,
23                                1]]))
24
25        s_e = np.zeros((len(t), 2))
26        s_i = np.zeros((len(t), 2))
27        s_t = np.zeros((len(t), 2))
28
29        # do integrations
30        s_e[0, :] = s0.T
31        s_i[0, :] = s0.T
32        s_t[0, :] = s0.T
33
34        for j in range(0, len(t)-1):
35            s_e[j+1, :] = np.dot(m_e, s_e[j, :])
36            s_i[j+1, :] = np.dot(m_i, s_i[j, :])
37            s_t[j+1, :] = np.dot(m_t, s_t[j, :])
38
39        plt.figure(figsize = (12, 8))
40        plt.plot(t, s_e[:,0], "b-")
41        plt.plot(t, s_i[:,0], "g:")
42        plt.plot(t, s_t[:,0], "r-")
43        plt.plot(t, np.cos(w*t), "k")

```

```

42     plt.ylim([-3, 3])
43     plt.xlabel("t")
44     plt.ylabel("$\Theta(t)$")
45     plt.legend(["Explicit", "Implicit", "Trapezoidal", "
Exact"])
46     plt.show()

```

La figura generada arriba compara la solución numérica con el problema del péndulo. La solución exacta es una onda coseno pura. El esquema de Euler explícito es claramente inestable. El esquema de Euler implícito decae exponencialmente, lo cual no es correcto. El método trapezoidal captura la solución correctamente, con un pequeño desplazamiento de fase a medida que el tiempo aumenta.

22.5 Métodos Predictor–Corrector y Runge–Kutta

22.5.1 Métodos Predictor–Corrector

Dado cualquier tiempo y valor de estado, la función $F(t, S(t))$ devuelve el cambio de estado $\frac{dS(t)}{dt}$. Los métodos predictor–corrector para resolver problemas de valor inicial mejoran la precisión de la aproximación de los métodos no predictor–corrector consultando la función F varias veces en diferentes ubicaciones (predicciones). Luego, usando un promedio ponderado de los resultados (correcciones), actualizan el estado. Esencialmente, usan dos fórmulas: un *predictor* y un *corrector*. El predictor es una fórmula explícita y estima la solución en t_{j+1} primero, es decir, podemos usar el método de Euler u otros métodos para completar este paso. Después de obtener la solución $S(t_{j+1})$, aplicamos el corrector para mejorar la precisión. Usando el $S(t_{j+1})$ encontrado en el lado derecho de una fórmula de otro modo implícita, el corrector puede calcular una nueva solución más precisa.

El *método del punto medio* tiene un paso predictor:

$$S\left(t_j + \frac{h}{2}\right) = S(t_j) + \frac{h}{2}F(t_j, S(t_j)),$$

que es la predicción del valor de la solución a medio camino entre t_j y t_{j+1} .

Luego calcula el paso corrector:

$$S(t_{j+1}) = S(t_j) + hF\left(t_j + \frac{h}{2}, S\left(t_j + \frac{h}{2}\right)\right),$$

que calcula la solución en $S(t_{j+1})$ a partir de $S(t_j)$ pero usa la derivada de $S(t_j + \frac{h}{2})$.

22.5.2 Métodos Runge–Kutta

Los métodos de Runge–Kutta (RK) se encuentran entre los métodos más utilizados para resolver EDO. Recordemos que el método de Euler usa los primeros dos términos de la serie de Taylor para aproximar la integración numérica, que es lineal: $S(t_{j+1}) = S(t_j + h) = S(t_j) + h \cdot S'(t_j)$.

Podemos mejorar enormemente la precisión de la integración numérica si mantenemos más términos de la serie como

$$S(t_{j+1}) = S(t_j + h) = S(t_j) + S'(t_j)h + \frac{1}{2!}S''(t_j)h^2 + \cdots + \frac{1}{n!}S^{(n)}(t_j)h^n.$$

Para obtener esta solución más precisa, necesitamos derivar las expresiones de $S''(t_j)$, $S'''(t_j)$, $\dots$, $S^{(n)}(t_j)$. Este trabajo extra puede evitarse usando los métodos RK, que se basan en la serie de Taylor truncada pero no requieren el cálculo de estas derivadas superiores.

22.5.2.1 Método de Runge–Kutta de Segundo Orden

Primero derivemos el método RK de segundo orden. Sea $\frac{dS(t)}{dt} = F(t, S(t))$. Entonces asumimos una fórmula de integración de la forma

$$S(t + h) = S(t) + c_1 F(t, S(t))h + c_2 F[t + ph, S(t) + qhF(t, S(t))]h. \quad (22.1)$$

Podemos intentar encontrar estos parámetros c_1, c_2, p, q igualando la ecuación anterior a la serie de Taylor de segundo orden:

$$S(t + h) = S(t) + S'(t)h + \frac{1}{2!}S''(t)h^2 = S(t) + F(t, S(t))h + \frac{1}{2!}F'(t, S(t))h^2. \quad (22.2)$$

Nótese que

$$F'(t, s(t)) = \frac{\partial F}{\partial t} + \frac{\partial F}{\partial S} \frac{\partial S}{\partial t} = \frac{\partial F}{\partial t} + \frac{\partial F}{\partial S} F. \quad (22.3)$$

Por lo tanto, la ecuación (22.2) puede escribirse como

$$S(t + h) = S + Fh + \frac{1}{2!} \left(\frac{\partial F}{\partial t} + \frac{\partial F}{\partial S} F \right) h^2. \quad (22.4)$$

En la ecuación (22.1), reescribimos el último término aplicando la serie de Taylor en varias variables:

$$F[t + ph, S + qhF] = F + \frac{\partial F}{\partial t}ph + qh\frac{\partial F}{\partial S}F,$$

así la ecuación (22.1) se convierte en

$$S(t + h) = S + (c_1 + c_2)Fh + c_1 \left[\frac{\partial F}{\partial t}p + q\frac{\partial F}{\partial S}F \right] h^2. \quad (22.5)$$

Comparando las ecuaciones (22.4) y (22.5), podemos obtener fácilmente

$$c_1 + c_2 = 1, \quad c_2p = \frac{1}{2}, \quad c_2q = \frac{1}{2}. \quad (22.6)$$

Dado que (22.6) tiene cuatro incógnitas y solo tres ecuaciones, asignamos cualquier valor a uno de los parámetros y obtenemos el resto de los parámetros. Una elección popular es:

$$c_1 = \frac{1}{2}, \quad c_2 = \frac{1}{2}, \quad p = 1, \quad q = 1.$$

También podemos definir:

$$k_1 = F(t_j, S(t_j)),$$

$$k_2 = F(t_j + ph, S(t_j) + qhk_1),$$

donde obtenemos

$$S(t_{j+1}) = S(t_j) + \frac{1}{2}(k_1 + k_2)h.$$

22.5.2.2 Método de Runge–Kutta de Cuarto Orden

Un método clásico para integrar EDO con un alto orden de precisión es el Método de Runge–Kutta de Cuarto Orden (RK4). Este método usa cuatro puntos k_1 , k_2 , k_3 y k_4 . Se usa un promedio ponderado de estas predicciones para producir la aproximación de la solución. La fórmula es la siguiente:

$$k_1 = F(t_j, S(t_j)),$$

$$k_2 = F\left(t_j + \frac{h}{2}, S(t_j) + \frac{1}{2}k_1h\right),$$

$$k_3 = F\left(t_j + \frac{h}{2}, S(t_j) + \frac{1}{2}k_2h\right),$$

$$k_4 = F(t_j + h, S(t_j) + k_3h).$$

Por lo tanto, tendremos

$$S(t_{j+1}) = S(t_j) + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4).$$

Como indica su nombre, el método RK4 es preciso de cuarto orden, o $O(h^4)$.

22.6 Solucionadores de EDO en Python

En SciPy, hay varias funciones integradas para resolver problemas de valor inicial. La función más común es `scipy.integrate.solve_ivp`. La construcción de la función se muestra a continuación:

CONSTRUCCIÓN:

Sea F un objeto de función para la función que calcula

$$\frac{dS(t)}{dt} = F(t, S(t)), \quad S(t_0) = S_0.$$

La variable t es una variable independiente unidimensional (tiempo), $S(t)$ es una función vectorial de n dimensiones (estado), y $F(t, S(t))$ define las ecuaciones diferenciales; S_0 es un valor inicial para S . La función F debe tener la forma $dS = F(t, S)$, aunque el nombre no tiene que ser F . El objetivo es encontrar $S(t)$ que satisfaga aproximadamente las ecuaciones diferenciales dado el valor inicial $S(t_0) = S_0$.

Usando el solucionador para resolver la ecuación diferencial es como sigue:

```
1 solve_ivp(fun, t_span, s0, method="RK45", t_eval=None)
```

donde `fun` toma la función del lado derecho del sistema; `t_span` es el intervalo de integración (t_0, t_f) donde t_0 es el inicio y t_f es el final del intervalo; `s0` es el estado inicial. Hay un par de métodos para elegir: el predeterminado es "RK45", que es el método explícito de Runge–Kutta de orden 5(4). Hay otros métodos que también puedes usar; consulta el final de esta sección para más información; `t_eval` toma los tiempos en los que almacenar la solución calculada, y debe estar ordenado y dentro de `t_span`.

Probemos un ejemplo a continuación.

EJEMPLO: Considera la EDO

$$\frac{dS(t)}{dt} = \cos(t)$$

para un valor inicial de $S_0 = 0$. La solución exacta a este problema es $S(t) = \sin(t)$. Usa `solve_ivp` para aproximar la solución a este problema de valor inicial en el intervalo $[0, \pi]$. Traza la solución aproximada versus la solución exacta y el error relativo a lo largo del tiempo.

```

1 In [1]: import matplotlib.pyplot as plt
2         import numpy as np
3         from scipy.integrate import solve_ivp
4         plt.style.use("seaborn-v0_8-poster")
5         %matplotlib inline
6
7         F = lambda t, s: np.cos(t)
8         t_eval = np.arange(0, np.pi, 0.1)
9         sol = solve_ivp(F, [0, np.pi], [0], t_eval=t_eval)
10
11        plt.figure(figsize = (12, 4))
12        plt.subplot(121)
13        plt.plot(sol.t, sol.y[0])
14        plt.xlabel("t")
15        plt.ylabel("S(t)")
16        plt.subplot(122)
17        plt.plot(sol.t, sol.y[0] - np.sin(sol.t))
18        plt.xlabel("t")
19        plt.ylabel("S(t) - sin(t)")
20        plt.tight_layout()
21        plt.show()

```

La figura izquierda anterior muestra la integración de $\frac{dS(t)}{dt} = \cos(t)$ con `solve_ivp`. La figura derecha calcula la diferencia entre la solución de la integración por `solve_ivp` y evalúa la solución analítica a esta EDO. Como se puede ver en la figura, la diferencia entre la solución aproximada y la exacta a esta EDO es pequeña. También podemos controlar las tolerancias relativas y absolutas usando los argumentos `rtol` y `atol`; el solucionador mantiene las estimaciones de error local menores que `atol + rtol*abs(S)`. Los valores predeterminados son `1e-3` para `rtol` y `1e-6` para `atol`.

¡PRUÉBALO! Usa `rtol` y `atol` para hacer que la diferencia entre la solución aproximada y la exacta sea menor que `1e-7`.

```

1 In [2]: sol = solve_ivp(F, [0, np.pi], [0], t_eval=t_eval, \
2         rtol = 1e-8, atol = 1e-8)
3         plt.figure(figsize = (12, 4))
4         plt.subplot(121)
5         plt.plot(sol.t, sol.y[0])
6         plt.xlabel("t")

```

```

7 plt.ylabel("S(t)")
8 plt.subplot(122)
9 plt.plot(sol.t, sol.y[0] - np.sin(sol.t))
10 plt.xlabel("t")
11 plt.ylabel("S(t) - sin(t)")
12 plt.tight_layout()
13 plt.show()

```

EJEMPLO: Considera la EDO

$$\frac{dS(t)}{dt} = -S(t),$$

con un valor inicial de $S_0 = 1$. La solución exacta a este problema es $S(t) = e^{-t}$. Usa `solve_ivp` para aproximar la solución a este problema de valor inicial en el intervalo $[0, 1]$. Traza la solución aproximada versus la solución exacta, y el error relativo a lo largo del tiempo.

```

1 In [3]: F = lambda t, s: -s
2         t_eval = np.arange(0, 1.01, 0.01)
3         sol = solve_ivp(F, [0, 1], [1], t_eval=t_eval)
4
5         plt.figure(figsize = (12, 4))
6         plt.subplot(121)
7         plt.plot(sol.t, sol.y[0])
8         plt.xlabel("t")
9         plt.ylabel("S(t)")
10        plt.subplot(122)
11        plt.plot(sol.t, sol.y[0] - np.exp(-sol.t))
12        plt.xlabel("t")
13        plt.ylabel("S(t) - exp(-t)")
14        plt.tight_layout()
15        plt.show()

```

La figura anterior muestra los resultados numéricos correspondientes. Como en el ejemplo anterior, la diferencia entre el resultado de `solve_ivp` y la evaluación de la solución analítica por Python es muy pequeña en comparación con el valor de la función.

EJEMPLO: Sea el estado de un sistema definido por $S(t) = \begin{bmatrix} x(t) \\ y(t) \end{bmatrix}$, y sea la evolución del sistema definida por la EDO

$$\frac{dS(t)}{dt} = \begin{bmatrix} 0 & t^2 \\ -t & 0 \end{bmatrix} S(t).$$

Usa `solve_ivp` para resolver esta EDO para el intervalo de tiempo $[0, 10]$ con un valor inicial de $S_0 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Traza la solución en $(x(t), y(t))$.

```

1 In [4]: F=lambda t, s: np.dot(np.array([[0, t**2], [-t, 0]]), s
2         )
3         t_eval = np.arange(0, 10.01, 0.01)
4         sol = solve_ivp(F, [0, 10], [1, 1], t_eval=t_eval)
5
6         plt.figure(figsize = (12, 8))
7         plt.plot(sol.y.T[:, 0], sol.y.T[:, 1])
8         plt.xlabel("x")
9         plt.ylabel("y")
10        plt.show()

```

22.7 Temas Avanzados

Esta sección discute brevemente algunos temas más avanzados en EDO de valor inicial. Aunque no entraremos en gran detalle, sugerimos consultar algunos excelentes libros como *Ordinary Differential Equations* de Morris Tenenbaum y Harry Pollard, *Numerical Methods for Engineers and Scientists* de Amos Gilat y Vish Subramaniam, así como *Numerical Methods for Ordinary Differential Equations* de J.C. Butcher, si estás interesado.

22.7.1 Métodos Multipaso

Hasta ahora, la mayoría de los métodos que discutimos se llaman "métodos de un paso" porque la aproximación para el siguiente punto t_{j+1} se obtiene usando solo información de $S(t_j)$ y t_j en el punto anterior. Aunque algunos de los métodos, como los métodos RK, pueden usar información de evaluación de función en puntos entre t_j y t_{j+1} , no retienen la información para uso directo en aproximaciones futuras.

En un intento por ganar más eficiencia, los *métodos multipaso* usan dos o más puntos anteriores para aproximar la solución en el siguiente punto t_{j+1} . Para métodos lineales multipaso, se usa una combinación lineal de los puntos anteriores y valores de la derivada para aproximar el siguiente punto. Los coeficientes pueden determinarse usando interpolación polinomial, como se discutió en el Capítulo 17.

Hay tres familias de métodos lineales multipaso comúnmente usados: los métodos de Adams–Bashforth, los métodos de Adams–Moulton y las fórmulas de diferenciación hacia atrás (BDF).

22.7.2 Rigidez en EDO

La rigidez es un concepto difícil e importante en la solución numérica de EDO. Una ecuación EDO rígida hará que la solución que se busca varíe lentamente y no será estable, es decir, si hay soluciones cercanas, la solución cambiará drásticamente. Esto nos obligará a tomar pasos pequeños para obtener resultados razonables. Por lo tanto, la rigidez suele ser un problema de eficiencia: si no nos preocupamos por el costo computacional, no nos preocuparíamos por la rigidez.

En ciencia e ingeniería, a menudo necesitamos modelar fenómenos físicos con escalas de tiempo o escalas espaciales muy diferentes. Estas aplicaciones generalmente conducen a sistemas de EDO, cuya solución incluye varios términos con magnitudes que varían con el tiempo a una tasa significativamente diferente. Por ejemplo, la Fig. 22.3 muestra un sistema masa-resorte, donde la masa oscila de izquierda a derecha, así como también oscila hacia arriba y hacia abajo debido al resorte. Por lo tanto, tenemos dos escalas de tiempo diferentes, es decir, la escala de tiempo del movimiento de oscilación y la del movimiento de vaivén. Si el resorte es realmente rígido, la escala de tiempo del movimiento de oscilación será mucho menor que la del movimiento de vaivén. Para estudiar el sistema, tenemos que usar un paso de tiempo muy pequeño para obtener una buena solución para la oscilación.

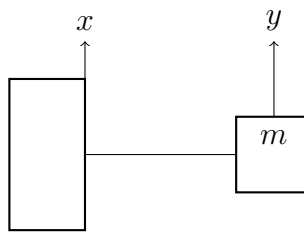


Figure 7: Ilustración de la ecuación rígida.

Dependiendo de las propiedades de la EDO y del nivel de precisión deseado, es posible que necesites usar diferentes métodos para `solve_ivp`.

Hay muchos métodos para elegir para el argumento `method` en `solve_ivp`; consulta la documentación para información adicional. Como sugiere la documentación, usa el método "RK45" o "RK23" para problemas no rígidos y "Radau" o "BDF" para problemas rígidos. Si no estás seguro, primero intenta ejecutar "RK45". Si esta solución experimenta un número inusualmente alto de iteraciones, diverge o falla, es probable que este problema sea rígido, y deberías usar "Radau" o "BDF". "LSODA" también puede ser una buena opción universal, pero puede ser algo menos conveniente de trabajar ya que envuelve código Fortran antiguo.

22.8 Resumen y Problemas

22.8.1 Resumen

1. Las ecuaciones diferenciales ordinarias (EDO) son ecuaciones que relacionan una función con sus derivadas, y los problemas de valor inicial son un tipo específico de problema de resolución de EDO.
2. Debido a que la mayoría de los problemas de valor inicial no pueden integrarse explícitamente, requieren soluciones numéricas.
3. Existen métodos explícitos, implícitos y predictor–corrector para resolver numéricamente problemas de valor inicial.
4. La precisión del esquema utilizado depende de su orden de aproximación de la EDO.
5. La estabilidad del esquema utilizado depende de la EDO, el esquema y la elección de los parámetros de integración.

22.8.2 Problemas

Problema 1

La ecuación logística es un modelo simple de ecuación diferencial que puede usarse para relacionar el cambio en la población $\frac{dP}{dt}$ con la población actual P , dada una tasa de crecimiento r y una capacidad de carga K . La ecuación

logística puede expresarse mediante

$$\frac{dP}{dt} = rP \left(1 - \frac{P}{K} \right).$$

Escribe una función `my_logistic_eq(t, P, r, K)` que represente la ecuación logística y devuelva $\frac{dP}{dt}$. Observa que este formato permite que `my_logistic_eq` se use como argumento de entrada para `solve_ivp`. Supón que los argumentos `dP`, `t`, `P`, `r` y `K` son todos escalares, y `dP` es el valor $\frac{dP}{dt}$ dados `r`, `P` y `K`. Observa que el argumento de entrada `t` es obligatorio si `my_logistic_eq` se va a usar como argumento de entrada para `solve_ivp`, aunque sea parte de la ecuación diferencial.

Observa que la ecuación logística tiene una solución analítica definida por

$$P(t) = \frac{K P_0 e^{rt}}{K + P_0 (e^{rt} - 1)}$$

donde P_0 es la población inicial. Verifica que esta ecuación es una solución de la ecuación logística.

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3 import matplotlib.pyplot as plt
4 from functools import partial
5 plt.style.use("seaborn-v0_8-poster")
6 %matplotlib inline
7
8 def my_logistic_eq(t, P, r, K):
9     """
10     Ecuación logística dP/dt = r*P*(1 - P/K)
11     """
12     dP = r * P * (1 - P/K)
13     return dP
14
15 # Prueba
16 dP = my_logistic_eq(0, 10, 1.1, 15)
17 print(dP) # 3.666666666666667
18
19 t0 = 0
20 tf = 20
21 P0 = 10
22 r = 1.1
23 K = 20
24 t = np.linspace(0, 20, 2001)

```

```

25 f = partial(my_logistic_eq, r=r, K=K)
26 sol = solve_ivp(f, [t0, tf], [P0], t_eval=t)
27
28 plt.figure(figsize = (10, 8))
29 plt.plot(sol.t, sol.y[0])
30 plt.plot(t, K*P0*np.exp(r*t)/(K+P0*(np.exp(r*t)-1)), "r:")
31 plt.xlabel("time")
32 plt.ylabel("population")
33 plt.legend(["Numerical Solution", "Exact Solution"])
34 plt.grid(True)
35 plt.show()

```

Problema 2

El atractor de Lorenz es un sistema de ecuaciones diferenciales ordinarias que se desarrolló originalmente para modelar corrientes de convección en la atmósfera. Las ecuaciones de Lorenz pueden escribirse de la siguiente manera:

$$\begin{aligned}\frac{dx}{dt} &= \sigma(y - x), \\ \frac{dy}{dt} &= x(\rho - z) - y, \\ \frac{dz}{dt} &= xy - \beta z,\end{aligned}$$

donde x , y y z representan la posición en tres dimensiones, y σ , ρ y β son parámetros escalares del sistema. Lee más sobre el atractor de Lorenz en Wikipedia¹ o más detalles en el libro "Viability Theory – New Directions". Escribe una función `my_lorenz(t,S,sigma,rho,beta)` donde t es un escalar que denota el tiempo, S es un arreglo 3D que denota la posición (x, y, z) , y `sigma`, `rho` y `beta` son escalares estrictamente positivos que representan σ , ρ y β . El argumento de salida `dS` debe tener el mismo tamaño que S .

```

1 import numpy as np
2
3 def my_lorenz(t, S, sigma, rho, beta):
4     """
5     Sistema de Lorenz.
6
7     Par metros:
8     t : tiempo

```

¹https://en.wikipedia.org/wiki/Lorenz_system

```

9      S : arreglo [x, y, z]
10     sigma, rho, beta : par metros
11
12     Retorna:
13     dS : derivada [dx/dt, dy/dt, dz/dt]
14     """
15     x, y, z = S
16     dx = sigma * (y - x)
17     dy = x * (rho - z) - y
18     dz = x * y - beta * z
19     dS = np.array([dx, dy, dz])
20     return dS
21
22 # Prueba
23 s = np.array([1, 2, 3])
24 dS = my_lorenz(0, s, 10, 28, 8/3)
25 print(dS) # array([10., 23., -6.])

```

Problema 3

Escribe una función `my_lorenz_solver(t_span, s0, sigma, rho, beta)` que resuelva las ecuaciones de Lorenz usando `solve_ivp`. La función devuelve `[T, X, Y, Z]`. El argumento de entrada `t_span` debe ser una lista de la forma `[t0, tf]`, donde `t0` es el tiempo inicial y `tf` es el tiempo final de consideración. El argumento de entrada `s0` debe ser un arreglo 3D de la forma `[x0, y0, z0]`, donde `(x0, y0, z0)` representa una posición inicial. Finalmente, los argumentos de entrada `sigma`, `rho` y `beta` son los parámetros escalares σ , ρ y β del sistema de Lorenz. El argumento de salida `T` debe ser un arreglo de tiempos dado como el argumento de salida de `solve_ivp`. Los argumentos de salida `X`, `Y` y `Z` deben ser la solución integrada numéricamente producida por `my_lorenz` del problema anterior y `solve_ivp`.

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3 import matplotlib.pyplot as plt
4 from mpl_toolkits import mplot3d
5
6 def my_lorenz_solver(t_span, s0, sigma, rho, beta):
7     """
8     Resuelve el sistema de Lorenz usando solve_ivp.
9
10    Par metros:
11    t_span : [t0, tf]

```

```

12     s0 : [x0, y0, z0]
13     sigma, rho, beta : par metros
14
15     Retorna:
16     T, X, Y, Z : arreglos de tiempo y soluci n
17     """
18     # Definir funci n que solo depende de t y S
19     def lorenz(t, S):
20         return my_lorenz(t, S, sigma, rho, beta)
21
22     sol = solve_ivp(lorenz, t_span, s0, dense_output=True)
23     T = sol.t
24     X = sol.y[0]
25     Y = sol.y[1]
26     Z = sol.y[2]
27
28     return [T, X, Y, Z]
29
30 # Prueba
31 sigma = 10
32 rho = 28
33 beta = 8/3
34 t0 = 0
35 tf = 50
36 s0 = np.array([0, 1, 1.05])
37 [T, X, Y, Z] = my_lorenz_solver([t0, tf], s0, sigma, rho,
38     beta)
39
40 fig = plt.figure(figsize = (10,10))
41 ax = plt.axes(projection="3d")
42 ax.grid()
43 ax.plot3D(X, Y, Z)
44 ax.set_xlabel("x", labelpad=20)
45 ax.set_ylabel("y", labelpad=20)
46 ax.set_zlabel("z", labelpad=20)
47 plt.show()

```

Problema 4

Considera el siguiente modelo de un sistema masa–resorte–amortiguador (MRA) en una dimensi3n. En esta figura, m denota la masa del bloque, c se llama el coeficiente de amortiguamiento, y k es la rigidez del resorte. Un amortiguador es un mecanismo que disipa energa en el sistema resistiendo la velocidad. El sistema MRA es un modelo simplista de varias aplicaciones

de ingeniería, como absorbedores de impactos y sistemas estructurales.

La relación entre aceleración, velocidad y desplazamiento puede expresarse mediante la siguiente ecuación diferencial MRA:

$$m\ddot{x} + c\dot{x} + kx = 0,$$

que puede reescribirse como

$$\ddot{x} = -\frac{(c\dot{x} + kx)}{m}.$$

Sea el estado del sistema denotado por el vector $S = [x; v]$ donde x es el desplazamiento de la masa desde su configuración de reposo, y v es su velocidad. Reescribe la ecuación MRA como una ecuación diferencial de primer orden en términos del estado S . En otras palabras, reescribe la ecuación MRA como $\frac{dS}{dt} = f(t, S)$.

Escribe una función `my_msd(t, S, m, c, k)` donde `t` es un escalar que denota el tiempo, `S` es un vector 2D que denota el estado del sistema MRA, y `m`, `c` y `k` son los coeficientes de masa, amortiguamiento y rigidez de la ecuación MRA, respectivamente.

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3 import matplotlib.pyplot as plt
4 from functools import partial
5
6 def my_msd(t, S, m, c, k):
7     """
8     Sistema masa-resorte-amortiguador.
9
10    Par metros:
11    t : tiempo
12    S : [x, v]
13    m, c, k : masa, amortiguamiento, rigidez
14
15    Retorna:
16    dS : [dx/dt, dv/dt]
17    """
18    x, v = S
19    dx = v
20    dv = -(c*v + k*x)/m
21    dS = np.array([dx, dv])
22    return dS
23

```



```

24 # Prueba
25 print(my_msd(0, [1, -1], 10, 1, 100)) # array([-1. , -9.9])
26
27 m = 1
28 k = 10
29 f = partial(my_msd, m=m, c=0, k=k)
30 t_e = np.arange(0, 20, 0.1)
31 sol_1 = solve_ivp(f, [0,20], [1,0], t_eval=t_e)
32
33 f = partial(my_msd, m=m, c=1, k=k)
34 sol_2 = solve_ivp(f, [0,20], [1,0], t_eval=t_e)
35
36 f = partial(my_msd, m=m, c=10, k=k)
37 sol_3 = solve_ivp(f, [0,20], [1,0], t_eval=t_e)
38
39 plt.figure(figsize = (10, 8))
40 plt.plot(sol_1.t, sol_1.y[0])
41 plt.plot(sol_2.t, sol_2.y[0])
42 plt.plot(sol_3.t, sol_3.y[0])
43 plt.title("Numerical Solution of MSD System with Varying
    Damping")
44 plt.xlabel("time")
45 plt.ylabel("displacement")
46 plt.legend(["no damping", "c=1", ">critically damped"], loc
    =1)
47 plt.show()

```

Problema 5

Escribe una función `my_forward_euler(ds,t_span,s0)` donde `ds` es un objeto de función, $f(t, s)$, que describe una ecuación diferencial de primer orden, `t_span` es un arreglo de tiempos para los cuales se desean soluciones numéricas de la ecuación diferencial, y `s0` es la condición inicial del sistema. Supón que el tamaño del estado es uno. El argumento de salida debe ser una lista `[t,s]`, tal que `t[i] = t_span[i]` para todo i , y `s` deben ser los valores integrados de `ds` en los tiempos `t`. Realiza la integración usando el método de Euler hacia adelante: $s[t_i] = s[t_{i-1}] + (t_i - t_{i-1})ds(t_{i-1}, s[t_{i-1}])$. Observa que `s[0]` debe ser igual a `s0`.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import solve_ivp
4

```

```

5 def my_forward_euler(ds, t_span, s0):
6     """
7     Integra una EDO usando el m todo de Euler hacia adelante
8     .
9
10    Par metros:
11    ds : funci n ds/dt = f(t, s)
12    t_span : arreglo de tiempos
13    s0 : condici n inicial
14
15    Retorna:
16    [t, s] : tiempos y soluci n
17    """
18    t = t_span
19    s = np.zeros(len(t))
20    s[0] = s0
21    for i in range(len(t)-1):
22        h = t[i+1] - t[i]
23        s[i+1] = s[i] + h * ds(t[i], s[i])
24    return [t, s]
25
26 # Prueba
27 t_span = np.linspace(0, 1, 10)
28 s0 = 1
29 f = lambda t, s: t*np.exp(-s)
30 t_eul, s_eul = my_forward_euler(f, t_span, s0)
31 print(t_eul)
32 print(s_eul)
33
34 plt.figure(figsize = (10, 8))
35 # Exact solution
36 t = np.linspace(0, 1, 1000)
37 s = np.log(np.exp(s0) + (t**2-t[0])/2)
38 plt.plot(t, s, "r", label="Exact")
39 # Forward Euler
40 plt.plot(t_eul, s_eul, "g", label="Euler")
41 # Python solver
42 sol = solve_ivp(f, [0, 1], [s0], t_eval=t)
43 plt.plot(sol.t, sol.y[0], "b-", label="Python Solver")
44 plt.xlabel("t")
45 plt.ylabel("f(t)")
46 plt.grid()
47 plt.legend(loc=2)
48 plt.show()

```

Problema 6

Escribe una función `myRK4(ds,t_span,s0)`, donde los argumentos de entrada y salida son los mismos que en el Problema 5. La función `myRK4` debe integrar numéricamente `ds` usando el método de Runge–Kutta de cuarto orden.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import solve_ivp
4
5 def myRK4(ds, t_span, s0):
6     """
7     Integra una EDO usando el método de Runge-Kutta de
8     cuarto orden.
9
10    Parámetros:
11    ds : función ds/dt = f(t, s)
12    t_span : arreglo de tiempos
13    s0 : condición inicial
14
15    Retorna:
16    [t, s] : tiempos y solución
17    """
18    t = t_span
19    s = np.zeros(len(t))
20    s[0] = s0
21    for i in range(len(t)-1):
22        h = t[i+1] - t[i]
23        k1 = ds(t[i], s[i])
24        k2 = ds(t[i] + h/2, s[i] + h*k1/2)
25        k3 = ds(t[i] + h/2, s[i] + h*k2/2)
26        k4 = ds(t[i] + h, s[i] + h*k3)
27        s[i+1] = s[i] + (h/6)*(k1 + 2*k2 + 2*k3 + k4)
28    return [t, s]
29
30 # Prueba
31 f = lambda t, s: np.sin(np.exp(s))/(t+1)
32 t_span = np.linspace(0, 2*np.pi, 10)
33 s0 = 0
34 plt.figure(figsize = (10, 8))
35 # Runge-Kutta method
36 t, s = myRK4(f, t_span, s0)
37 plt.plot(t, s, "ro-", label="RK4")

```

```
38 # Python solver
39 sol = solve_ivp(f, [0, 2*np.pi], [s0], t_eval=t)
40 plt.plot(sol.t, sol.y[0], "bs-", label="Python Solver")
41 plt.xlabel("t")
42 plt.ylabel("f(t)")
43 plt.grid()
44 plt.legend(loc=2)
45 plt.show()
```

PROBLEMAS DE CONTORNO PARA EDOs

23.1 Planteamiento del Problema de Contorno para EDO

El capítulo anterior introdujo los problemas de valor inicial para ecuaciones diferenciales ordinarias. En los problemas de valor inicial, todos los valores conocidos se especifican en el mismo valor de la variable independiente, usualmente en el límite inferior del intervalo; de ahí proviene el término "inicial". Este capítulo introduce otro tipo de problema, denominado *problema de contorno* (o de frontera). Como sugiere el nombre, los valores conocidos se especifican en los extremos de la variable independiente, es decir, en las fronteras del intervalo.

Por ejemplo, si tenemos una ecuación diferencial ordinaria simple de segundo orden,

$$\frac{d^2 f(x)}{dx^2} = \frac{df(x)}{dx} + 3$$

y si la variable independiente varía en el dominio $[0, 20]$, el problema de valor inicial tendrá las dos condiciones en el valor cero: esto es, conocemos el valor de $f(0)$ y $f'(0)$. En contraste, los problemas de contorno especificarán los valores en $x = 0$ y $x = 20$. Nótese que para resolver una EDO de primer orden y obtener una solución particular se requiere una restricción, mientras que una EDO de orden n requiere n restricciones.

El problema de contorno para una ecuación diferencial ordinaria de orden n ,

$$F\left(x, f(x), \frac{df(x)}{dx}, \frac{d^2 f(x)}{dx^2}, \frac{d^3 f(x)}{dx^3}, \dots, \frac{d^{n-1} f(x)}{dx^{n-1}}\right) = \frac{d^n f(x)}{dx^n},$$

especifica n condiciones de contorno conocidas en a y b , para resolver esta ecuación en un intervalo $x \in [a, b]$.

Para el caso de segundo orden, dado que la condición de contorno puede ser un valor de $f(x)$ o un valor de la derivada $f'(x)$, podemos tener varios casos diferentes para los valores especificados. Por ejemplo, podemos tener los valores de las condiciones de contorno especificados como:

1. Dos valores de $f(x)$, es decir, $f(a)$ y $f(b)$ son conocidos.
2. Dos derivadas de $f'(x)$, es decir, $f'(a)$ y $f'(b)$ son conocidas.
3. Condiciones mixtas de los dos casos anteriores: es decir, $f(a)$ y $f'(b)$ son conocidas, o $f'(a)$ y $f(b)$ son conocidas.

Para obtener la solución particular, necesitamos dos condiciones de contorno. El problema de contorno de EDO de segundo orden también se llama "Problema de Contorno de Dos Puntos". Los problemas de EDO de orden superior necesitan condiciones de contorno adicionales, que suelen ser los valores de derivadas superiores de las variables independientes. Este capítulo se centra en los problemas de contorno de dos puntos.

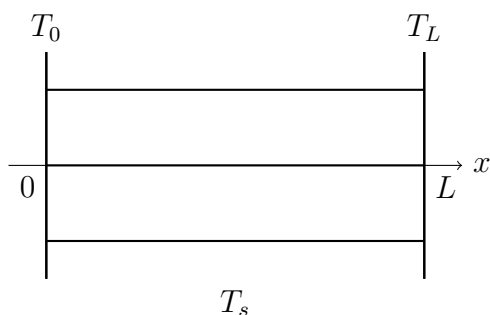


Figure 8: Flujo de calor en una aleta de pasador. La variable L es la longitud de la aleta, que comienza en $x = 0$ y termina en $x = L$. Las temperaturas en los dos extremos son T_0 y T_L , siendo T_s la temperatura del entorno circundante.

A continuación se muestra un ejemplo de un problema de contorno y su solución.

Las aletas se utilizan en muchas aplicaciones para aumentar la transferencia de calor desde superficies. Normalmente, el diseño de aletas de pasador

de refrigeración se encuentra en muchas aplicaciones, por ejemplo, la aleta de pasador utilizada como disipador de calor para enfriar un objeto. Podemos modelar la distribución de temperatura en una aleta de pasador como se muestra en la Fig. 23.1, donde la longitud de la aleta es L , y el inicio y el final de la aleta son $x = 0$ y $x = L$, respectivamente. Las temperaturas en los dos extremos son T_0 y T_L , mientras que T_s es la temperatura del entorno circundante. Si consideramos tanto la convección como la radiación, la distribución de temperatura en estado estacionario de la aleta $T(x)$ puede modelarse de la siguiente manera:

$$\frac{d^2T}{dx^2} - \alpha_1(T - T_s) - \alpha_2(T^4 - T_s^4) = 0$$

con las condiciones de contorno $T(0) = T_0$ y $T(L) = T_L$; α_1 y α_2 son coeficientes. Esta es una EDO de segundo orden con dos condiciones de contorno; por lo tanto, podemos resolverla para obtener soluciones particulares.

El resto de este capítulo cubre dos métodos para aproximar numéricamente la solución a problemas de contorno en una malla numérica. Cubriremos tanto el método de disparo como el método de diferencias finitas para resolver problemas de contorno de EDO.

23.2 El Método de Disparo

El método de disparo se desarrolló con el objetivo de transformar un problema de contorno de EDO en un problema de valor inicial equivalente, de modo que podamos resolverlo utilizando los métodos que aprendimos en el capítulo anterior. Con los problemas de valor inicial, comenzamos en el valor inicial y avanzamos para obtener la solución. Este método no funciona para problemas de contorno porque no hay suficientes condiciones de valor inicial para resolver la EDO y obtener una solución única. El método de disparo se desarrolló para superar esta dificultad.

El nombre "método de disparo" es análogo al tiro al blanco: como se muestra en la Fig. 23.2, disparamos al objetivo y observamos dónde impactamos. Basándonos en los errores, podemos ajustar nuestra puntería y disparar de nuevo con la esperanza de impactar más cerca del objetivo. Podemos ver por la analogía que el método de disparo es un método de optimización iterativo.

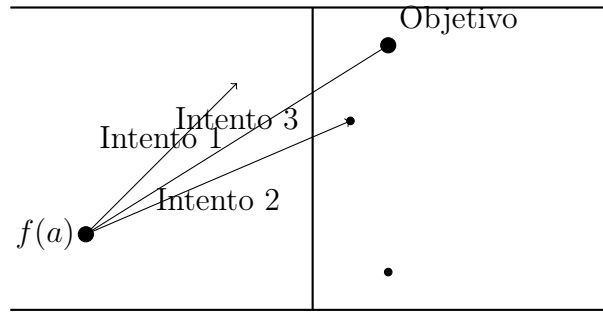


Figure 9: Analogía del tiro al blanco con el método de disparo.

Veamos cómo funciona el método de disparo usando la EDO de segundo orden dados $f(a) = f_a$ y $f(b) = f_b$, así como

$$F\left(x, f(x), \frac{df(x)}{dx}\right) = \frac{d^2 f(x)}{dx^2}.$$

Paso 1. Comenzamos todo el proceso suponiendo $f'(a) = \alpha$, luego, junto con $f(a) = f_a$, convertimos el problema anterior en un problema de valor inicial con dos condiciones en el valor $x = a$. Este es el paso de apuntar.

Paso 2. Usando lo que aprendimos del capítulo anterior, es decir, podemos usar un método de Runge-Kutta, para integrar hasta la otra frontera b y encontrar $f(b) = f_\beta$. Esto se llama el paso de disparar.

Paso 3. Ahora comparamos el valor de f_β con f_b . Normalmente, nuestra suposición inicial no es buena, y $f_\beta \neq f_b$, pero lo que queremos es $f_\beta - f_b = 0$; por lo tanto, ajustamos nuestras suposiciones iniciales y repetimos el proceso hasta que el error sea aceptable, momento en el que podemos detenernos. Este es el paso iterativo.

Aunque las ideas detrás del método de disparo son muy simples, comparar y encontrar las mejores suposiciones no es fácil; este procedimiento puede ser muy tedioso. Encontrar la mejor suposición para obtener $f_\beta - f_b = 0$ es un problema de búsqueda de raíces y puede ser tedioso, pero ofrece una forma sistemática de buscar la mejor suposición. Dado que f_β es una función de α , el problema se convierte en encontrar la raíz de $g(\alpha) - f_b = 0$. Podemos usar cualquier método del Capítulo 19 para resolver el problema.

¡PRUÉBALO! Supongamos que queremos lanzar un cohete, y sea $y(t)$ la altitud (en metros desde la superficie) del cohete en el tiempo t . Sabemos la gravedad $g = 9.8 \text{ m/s}^2$. Si queremos que el cohete esté a 50 m del

suelo después de 5 s del lanzamiento, ¿cuál debería ser la velocidad en el lanzamiento? (Suponiendo que ignoramos la resistencia del aire.)

Para responder a esta pregunta, podemos plantear el problema como un problema de contorno para una EDO de segundo orden. La EDO es

$$\frac{d^2y}{dt^2} = -g,$$

y las dos condiciones de contorno son $y(0) = 0$ y $y(5) = 50$. Queremos responder a la pregunta: ¿Cuál es $y'(0)$ en el lanzamiento?

Esta es una pregunta bastante simple y se puede resolver analíticamente con bastante facilidad; la respuesta correcta es $y'(0) = 34.5$. Si lo resolvemos usando el método de disparo, necesitamos reducir el orden de la función primero, y la EDO de segundo orden se convierte en:

$$\frac{dy}{dt} = v,$$

$$\frac{dv}{dt} = -g.$$

Por lo tanto, tenemos $S(t) = \begin{bmatrix} y(t) \\ v(t) \end{bmatrix}$ que satisface

$$\frac{dS(t)}{dt} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} S(t) + \begin{bmatrix} 0 \\ -g \end{bmatrix}.$$

```

1 In [1]: import numpy as np
2         import matplotlib.pyplot as plt
3         from scipy.integrate import solve_ivp
4         from scipy.optimize import fsolve
5         plt.style.use("seaborn-v0_8-poster")
6         %matplotlib inline

```

Para nuestra primera suposición, tomamos la velocidad en el lanzamiento como 25 m/s.

```

1 In [2]: F = lambda t, s: np.array([s[1], -9.8]) # [dy/dt, dv
         /dt]
2         t_span = [0, 5]
3         y0 = 0
4         v0 = 25
5         t_eval = np.linspace(0, 5, 100)

```

```

6     sol = solve_ivp(F, t_span, [y0, v0], t_eval = t_eval)
7
8     plt.figure(figsize = (10, 8))
9     plt.plot(sol.t, sol.y[0])
10    plt.plot(5, 50, "ro")
11    plt.xlabel("time (s)")
12    plt.ylabel("altitude (m)")
13    plt.title(f"first guess v={v0} m/s")
14    plt.show()

```

La figura muestra que la primera suposición es un poco demasiado pequeña, ya que después de 5 s para la velocidad inicial elegida, la altitud del cohete es menor de 10 m. El punto rojo en la figura es el objetivo que queremos alcanzar. Si ajustamos nuestra suposición y aumentamos la velocidad a 40 m/s, entonces obtenemos:

```

1 In [3]: v0 = 40
2         sol = solve_ivp(F, t_span, [y0, v0], t_eval = t_eval)
3
4         plt.figure(figsize = (10, 8))
5         plt.plot(sol.t, sol.y[0])
6         plt.plot(5, 50, "ro")
7         plt.xlabel("time (s)")
8         plt.ylabel("altitude (m)")
9         plt.title(f"second guess v={v0} m/s")
10        plt.show()

```

Aquí, sobrestimamos la velocidad. Por lo tanto, esta suposición aleatoria quizás no sea la mejor manera de obtener el resultado. Como mencionamos anteriormente, tratar este procedimiento como un problema de búsqueda de raíces nos proporcionará un mejor resultado. Usemos `fsolve` de Python para encontrar la raíz. El siguiente ejemplo demostrará cómo encontrar la respuesta correcta directamente.

```

1 In [4]: from scipy.optimize import fsolve
2
3         def objective(v0):
4             sol = solve_ivp(F, t_span, [y0, v0], t_eval =
5                 t_eval)
6             y = sol.y[0]
7             return y[-1] - 50
8
9         v0, = fsolve(objective, 10)
10        print(v0)

```

34.499999999999986

```

1 In [5]: sol = solve_ivp(F, t_span, [y0, v0], t_eval = t_eval)
2
3         plt.figure(figsize = (10, 8))
4         plt.plot(sol.t, sol.y[0])
5         plt.plot(5, 50, "ro")
6         plt.xlabel("time (s)")
7         plt.ylabel("altitude (m)")
8         plt.title(f"root finding v={v0} m/s")
9         plt.show()

```

¡PRUÉBALO! Cambiemos la suposición inicial y veamos si esto cambia el resultado.

```

1 In [6]: for v0_guess in range(1, 100, 10):
2         v0, = fsolve(objective, v0_guess)
3         print("Init: %d, Result: %.1f" %(v0_guess, v0))

```

```

Init: 1, Result: 34.5
Init: 11, Result: 34.5
Init: 21, Result: 34.5
Init: 31, Result: 34.5
Init: 41, Result: 34.5
Init: 51, Result: 34.5
Init: 61, Result: 34.5
Init: 71, Result: 34.5
Init: 81, Result: 34.5
Init: 91, Result: 34.5

```

Nótese que cambiar las suposiciones iniciales no cambia el resultado, lo que significa que este método es estable; véase más adelante respecto al problema de estabilidad.

23.3 El Método de Diferencias Finitas

Otra forma de resolver los problemas de contorno de EDO es usar el método de diferencias finitas, donde usamos fórmulas de diferencias finitas en puntos de malla uniformemente espaciados para aproximar las ecuaciones diferenciales, lo que transforma una ecuación diferencial en un sistema de ecuaciones algebraicas para resolver.

En el método de diferencias finitas, las derivadas en la ecuación diferencial se aproximan usando las fórmulas de diferencias finitas (ver Capítulo 20 para más detalles). Podemos dividir el intervalo $[a, b]$ en n subintervalos iguales de longitud h como se muestra en la Fig. 23.3.

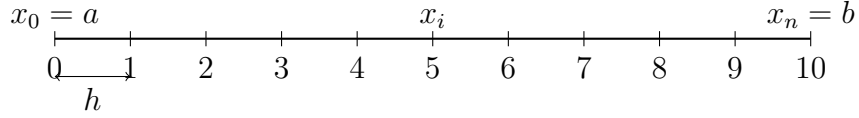


Figure 10: Ilustración del método de diferencias finitas.

Usualmente, usamos las fórmulas de diferencias centrales en los métodos de diferencias finitas porque proporcionan mejor precisión. La ecuación diferencial se aplica solo en los puntos de la malla, y las primeras y segundas derivadas son:

$$\frac{dy}{dx} = \frac{y_{i+1} - y_{i-1}}{2h}, \quad \frac{d^2y}{dx^2} = \frac{y_{i-1} - 2y_i + y_{i+1}}{h^2}.$$

Estas expresiones de diferencias finitas se utilizan para reemplazar las derivadas de y en la ecuación diferencial, lo que lleva a un sistema de $n + 1$ ecuaciones algebraicas lineales si la ecuación diferencial es lineal. Si la ecuación diferencial es no lineal, las ecuaciones algebraicas también serán no lineales.

EJEMPLO: Resuelve el problema del cohete en la sección anterior usando el método de diferencias finitas; traza la altitud del cohete después del lanzamiento. La EDO es

$$\frac{d^2y}{dt^2} = -g$$

con las condiciones de contorno $y(0) = 0$ y $y(5) = 50$. Tomemos $n = 10$.

Dado que el intervalo de tiempo es $[0, 5]$ y tenemos $n = 10$, $h = 0.5$, usando las derivadas aproximadas por diferencias finitas, obtenemos

$$y_0 = 0, \quad y_{i-1} - 2y_i + y_{i+1} = -gh^2, \quad i = 1, 2, \dots, n-1, \quad y_{10} = 50.$$

Si usamos notación matricial, obtenemos

$$\begin{bmatrix} 1 & 0 & & & \\ 1 & -2 & 1 & & \\ & \ddots & \ddots & \ddots & \\ & & 1 & -2 & 1 \\ & & & 0 & 1 \end{bmatrix} \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \\ y_n \end{bmatrix} = \begin{bmatrix} 0 \\ -gh^2 \\ \vdots \\ -gh^2 \\ 50 \end{bmatrix}.$$

Con 11 ecuaciones en el sistema, podemos resolverlo usando el método que presentamos en el Capítulo 14.

```

1 In [1]: import numpy as np
2         import matplotlib.pyplot as plt
3         plt.style.use("seaborn-v0_8-poster")
4         %matplotlib inline
5
6         n = 10
7         h = (5-0) / n
8
9         # Get A
10        A = np.zeros((n+1, n+1))
11        A[0, 0] = 1
12        A[n, n] = 1
13        for i in range(1, n):
14            A[i, i-1] = 1
15            A[i, i] = -2
16            A[i, i+1] = 1
17        print(A)
18
19        # Get b
20        b = np.zeros(n+1)
21        b[1:-1] = -9.8*h**2
22        b[-1] = 50
23        print(b)
24
25        # solve the linear equations
26        y = np.linalg.solve(A, b)
27
28        t = np.linspace(0, 5, 11)
29
30        plt.figure(figsize=(10,8))
31        plt.plot(t, y)
32        plt.plot(5, 50, "ro")
33        plt.xlabel("time (s)")
34        plt.ylabel("altitude (m)")
35        plt.show()

```

```

[[ 1.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.]
 [ 1. -2.  1.  0.  0.  0.  0.  0.  0.  0.  0.]
 [ 0.  1. -2.  1.  0.  0.  0.  0.  0.  0.  0.]
 [ 0.  0.  1. -2.  1.  0.  0.  0.  0.  0.  0.]
 [ 0.  0.  0.  1. -2.  1.  0.  0.  0.  0.  0.]

```

```
[ 0.  0.  0.  0.  1. -2.  1.  0.  0.  0.  0.]
[ 0.  0.  0.  0.  0.  1. -2.  1.  0.  0.  0.]
[ 0.  0.  0.  0.  0.  0.  1. -2.  1.  0.  0.]
[ 0.  0.  0.  0.  0.  0.  0.  1. -2.  1.  0.]
[ 0.  0.  0.  0.  0.  0.  0.  0.  1. -2.  1.]
[ 0.  0.  0.  0.  0.  0.  0.  0.  0.  1.  1.]]
[ 0.   -2.45 -2.45 -2.45 -2.45 -2.45 -2.45 -2.45 -2.45 -2.45 50.  ]
```

Resolvamos para $y'(0)$. De la fórmula de diferencias finitas, sabemos que $\frac{dy}{dx} = \frac{y_{i+1}-y_{i-1}}{2h}$, lo que significa que $y'(0) = \frac{y_1-y_{-1}}{2h}$; pero no sabemos qué es y_{-1} . Podemos calcular y_{-1} ya que conocemos los valores de y en cada punto de la malla. De la fórmula de diferencias finitas de segunda derivada, sabemos que $\frac{y_{-1}-2y_0+y_1}{h^2} = -g$; por lo tanto, podemos resolver para y_{-1} para obtener la velocidad de lanzamiento. Véase el cálculo a continuación.

```
1 In [2]: y_n1 = -9.8*h**2 + 2*y[0] - y[1]
2         (y[1] - y_n1) / (2*h)
```

Out[2]: 34.5

Obtenemos la velocidad de lanzamiento correcta usando el método de diferencias finitas. Para darte exposición adicional a este concepto, veamos otro ejemplo.

¡PRUÉBALO! Usa el método de diferencias finitas para resolver el siguiente problema de contorno lineal:

$$\frac{d^2y(t)}{dt^2} = -4y + 4x$$

con las condiciones de contorno $y(0) = 0$ y $y'(\pi/2) = 0$. La solución exacta del problema es $y = x - \sin 2x$. Traza los errores frente a los puntos de malla n (n de 3 a 100) para el punto de contorno $y(\pi/2)$.

Usando las derivadas aproximadas por diferencias finitas, tenemos

$$y_0 = 0, \quad y_{i-1} - 2y_i + y_{i+1} - h^2(-4y_i + 4x_i) = 0, \quad i = 1, 2, \dots, n-1,$$

$$2y_{n-1} - 2y_n - h^2(-4y_n + 4x_n) = 0.$$

La última ecuación se deriva de $\frac{y_{n+1}-y_{n-1}}{2h} = 0$ (la condición de contorno $y'(\pi/2) = 0$); por lo tanto, $y_{n+1} = y_{n-1}$.

Si usamos notación matricial, obtendremos:

$$\begin{bmatrix} 1 & 0 & & & \\ 1 & -2+4h^2 & 1 & & \\ & \ddots & \ddots & \ddots & \\ & & 1 & -2+4h^2 & 1 \\ & & & 2 & -2+4h^2 \end{bmatrix} \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \\ y_n \end{bmatrix} = \begin{bmatrix} 0 \\ 4h^2x_1 \\ \vdots \\ 4h^2x_{n-1} \\ 4h^2x_n \end{bmatrix}.$$

```

1 In [3]: def get_a_b(n):
2         h = (np.pi/2-0) / n
3         x = np.linspace(0, np.pi/2, n+1)
4         # Get A
5         A = np.zeros((n+1, n+1))
6         A[0, 0] = 1
7         A[n, n] = -2+4*h**2
8         A[n, n-1] = 2
9         for i in range(1, n):
10            A[i, i-1] = 1
11            A[i, i] = -2+4*h**2
12            A[i, i+1] = 1
13        # Get b
14        b = np.zeros(n+1)
15        for i in range(1, n+1):
16            b[i] = 4*h**2*x[i]
17        return x, A, b
18
19        x_true = np.pi/2
20        v_true = x_true - np.sin(2*x_true)
21        n_s = []
22        errors = []
23        for n in range(3, 100, 5):
24            x, A, b = get_a_b(n)
25            y = np.linalg.solve(A, b)
26            n_s.append(n)
27            e = v_true - y[-1]
28            errors.append(e)
29
30        plt.figure(figsize = (10,8))
31        plt.plot(n_s, errors)
32        plt.yscale("log")
33        plt.xlabel("n grid points")
34        plt.ylabel("errors at x = $\pi/2$")
35        plt.show()

```

Con puntos de malla más densos, nos acercamos a la solución exacta en el punto de contorno.

El método de diferencias finitas también se puede aplicar a EDO de orden superior, pero necesita la aproximación de las derivadas de orden superior utilizando una fórmula de diferencias finitas. Por ejemplo, para resolver una EDO de cuarto orden se requiere realizar lo siguiente:

$$\frac{d^4y}{dx^4} = \frac{y_{i-2} - 4y_{i-1} + 6y_i - 4y_{i+1} + y_{i+2}}{h^4}.$$

No discutiremos EDO de orden superior, ya que la idea detrás de su solución es similar a la EDO de segundo orden que presentamos anteriormente.

23.4 Error Numérico e Inestabilidad

Los problemas de contorno también presentan dos problemas principales discutidos en el Capítulo 22: el error numérico (precisión) y la estabilidad. Dependiendo de los diferentes métodos utilizados, hay que tener en cuenta que los métodos de disparo y de diferencias finitas son diferentes en términos de los errores que presentan.

Para el método de disparo, el error numérico es similar al descrito para los problemas de valor inicial, ya que el método de disparo esencialmente transforma el problema de contorno en una serie de problemas de valor inicial. En cuanto a la estabilidad del método, como se muestra en el ejemplo de la Sección 23.2, en términos de estabilidad, aunque nuestras suposiciones iniciales no estaban cerca de la respuesta verdadera, el método devolvió una solución numérica precisa. Esto se debe a la adición de la restricción en el extremo derecho, que evita que los errores aumenten sin límite.

En el caso del método de diferencias finitas, el error numérico está determinado por el orden de precisión del esquema numérico utilizado. La precisión de los diferentes esquemas utilizados para las aproximaciones de derivadas se discute en la Sección 20.2. La precisión del método de diferencias finitas está determinada por el mayor de los dos errores de truncamiento: el esquema de diferencias utilizado para la ecuación diferencial o el del esquema de diferencias utilizado para discretizar las condiciones de contorno (hemos visto que el tamaño de paso influye fuertemente en la precisión del método de diferencias finitas). Dado que el método de diferencias finitas esencialmente convierte el problema de contorno en la resolución de un sistema

de ecuaciones, depende de la estabilidad del esquema utilizado para resolver el sistema de ecuaciones resultante simultáneamente.

23.5 Resumen y Problemas

23.5.1 Resumen

1. Los problemas de contorno son un tipo específico de problema de resolución de EDO con condiciones de contorno especificadas al inicio y al final del intervalo.
2. El método de disparo puede transformar los problemas de contorno en problemas de valor inicial, y podemos usar un método de búsqueda de raíces para resolverlos.
3. El método de diferencias finitas utiliza un esquema de diferencias finitas para aproximar las derivadas y convierte el problema en un conjunto de ecuaciones para resolver.
4. La precisión y estabilidad de los problemas de contorno tienen similitudes y diferencias en comparación con los problemas de valor inicial.

23.5.2 Problemas

Problema 1

Describe la diferencia entre los problemas de contorno y los problemas de valor inicial en EDO.

Solución: En los problemas de valor inicial, todas las condiciones se especifican en el mismo punto de la variable independiente (generalmente el inicio del intervalo). En los problemas de contorno, las condiciones se especifican en diferentes puntos de la variable independiente (generalmente los extremos del intervalo).

Problema 2

Intenta describir la intuición detrás del método de disparo y sus vínculos con los problemas de valor inicial.

Solución: El método de disparo trata un problema de contorno como un problema de valor inicial con una condición inicial desconocida. Se "dispara" desde el punto inicial con una suposición para la condición faltante y se observa dónde se llega en el otro extremo. Luego se ajusta la suposición iterativamente hasta que se satisface la condición de contorno en el otro extremo. Esto vincula el método con la resolución de problemas de valor inicial repetidamente, combinado con un método de búsqueda de raíces para encontrar el valor correcto.

Problema 3

¿Qué es el método de diferencias finitas para problemas de contorno? ¿Cómo lo aplicamos?

Solución: El método de diferencias finitas discretiza el dominio en una malla de puntos y aproxima las derivadas en la ecuación diferencial mediante fórmulas de diferencias finitas (como diferencias centrales). Esto convierte la EDO en un sistema de ecuaciones algebraicas (lineales o no lineales) que se resuelve para los valores de la función en los puntos de la malla. Se aplica estableciendo las ecuaciones en cada punto interior usando la aproximación de diferencias, e incorporando las condiciones de contorno en las ecuaciones para los puntos extremos.

Problema 4

Resuelve el siguiente problema de contorno con $y(0) = 0$ y $y(\pi/2) = 1$:

$$y'' + (1 - 0.2x)y^2 = 0.$$

Solución: Este es un problema no lineal. Resolveremos usando el método de disparo con un enfoque de búsqueda de raíces.

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3 from scipy.optimize import fsolve
4 import matplotlib.pyplot as plt
5
6 # Definir la EDO de primer orden (reducida)
7 def ode_system(x, y, y0_prime_guess):
8     # Esta función será llamada por solve_ivp
9     # Necesitamos una forma de pasar el parámetro de

```

TRANSFORMADA DE FOURIER

24.1 Conceptos Básicos de las Ondas

Hay muchos tipos de ondas en nuestro entorno. Por ejemplo, si lanzas una piedra a un estanque, puedes ver cómo se forman y viajan las ondas en el agua. Hay muchos tipos de ondas. Algunas de ellas son difíciles de ver, como las ondas sonoras, las ondas sísmicas y las microondas (que usamos para cocinar nuestros alimentos en la cocina). Pero en física, una onda es una perturbación que viaja a través del espacio y la materia, transfiriendo energía de un lugar a otro. Es importante estudiar las ondas en nuestra vida para entender cómo se forman, viajan, etc. Este capítulo cubrirá una herramienta básica que nos ayuda a entender y estudiar las ondas: la Transformada de Fourier. Pero antes de continuar, familiaricémonos primero con cómo modelar ondas matemáticamente.

24.1.1 Modelando una Onda Usando Herramientas Matemáticas

Podemos modelar una onda simple como un campo con una función $F(x, t)$, donde x es la ubicación de un punto en el espacio, mientras que t es el tiempo. Uno de los casos más simples es la forma de una onda seno que cambia a lo largo de x .

```
1 In [1]: import matplotlib.pyplot as plt
2         import numpy as np
3         plt.style.use("seaborn-v0_8-poster")
4         %matplotlib inline
5
```

```

6 In [2]: x = np.linspace(0, 20, 201)
7         y = np.sin(x)
8         plt.figure(figsize = (8, 6))
9         plt.plot(x, y, "b")
10        plt.ylabel("Amplitud")
11        plt.xlabel("Ubicaci n (x)")
12        plt.show()

```

Una onda seno puede cambiar tanto en el tiempo como en el espacio. Si graficamos los cambios en varias ubicaciones, cada instantánea temporal será una onda seno que cambia con la ubicación. Véase la siguiente figura con un punto fijo en $x = 2.5$, mostrado como un punto rojo. Por supuesto, también puedes ver los cambios a lo largo del tiempo en una ubicación específica; gráficalo por ti mismo.

```

1 In [3]: fig = plt.figure(figsize = (8,8))
2         times = np.arange(5)
3         n = len(times)
4         for t in times:
5             plt.subplot(n, 1, t+1)
6             y = np.sin(x + t)
7             plt.plot(x, y, "b")
8             plt.plot(x[25], y[25], "ro")
9             plt.ylim(-1.1, 1.1)
10            plt.ylabel("y")
11            plt.title(f"t = {t}")
12            plt.xlabel("ubicaci n (x)")
13        plt.tight_layout()
14        plt.show()

```

24.1.2 Características de una Onda

Las ondas pueden ser una entidad continua tanto en el tiempo como en el espacio. Para propósitos numéricos, uno necesita digitalizar el tiempo y el espacio en varios puntos. Por ejemplo, en el contexto de la geofísica, se pueden usar sensores como acelerómetros (que miden la aceleración de un movimiento) en diferentes ubicaciones de la Tierra para monitorear terremotos; esto se llama *discretización espacial*. De manera similar, estos sensores suelen registrar los datos en ciertos momentos, lo que se llama *discretización temporal*. Para una onda simple, tiene diferentes características. Véanse las Figs. 24.1 y 24.2.

La *amplitud* se usa para describir la diferencia entre los valores máximos y

el valor base (ver las figuras anteriores). Una onda seno es una señal *periódica*, lo que significa que se repite después de un cierto tiempo y puede medirse por su *período*. El período de una onda es el tiempo que tarda en completar un ciclo completo. En la figura, el período se puede medir desde dos picos adyacentes. La *longitud de onda* mide la distancia entre dos crestas o valles sucesivos de una onda. La *frecuencia* describe el número de ondas que pasan por una ubicación fija en una cantidad de tiempo determinada. La frecuencia se puede medir por cuántos ciclos pasan en 1 segundo. Por lo tanto, la unidad de frecuencia es ciclos/segundo, o más comúnmente Hertz (abreviado Hz). La frecuencia es diferente del período, pero están relacionados entre sí. La frecuencia se refiere a qué tan seguido sucede algo mientras que el período se refiere al tiempo que lleva completar algo, matemáticamente,

$$\text{período} = \frac{1}{\text{frecuencia}}.$$

Nótese en las dos figuras los puntos azules en las ondas seno; estos son los puntos de discretización que graficamos tanto en el tiempo como en el espacio; solo en estos puntos muestreamos el valor de la onda. Normalmente cuando registramos una onda, necesitamos especificar con qué frecuencia muestreamos la onda en el tiempo; esto se llama *muestreo*. Esta tasa se llama *frecuencia de muestreo*, en Hz. Por ejemplo, si muestreamos una onda a 2 Hz, significa que cada segundo muestreamos dos puntos de datos. Ahora que entendemos más sobre los conceptos básicos de una onda, estudiemos una onda seno, que puede representarse mediante la siguiente ecuación:

$$y(t) = A \sin(\omega t + \varphi)$$

donde A es la amplitud de la onda, y ω es la *frecuencia angular*, que especifica cuántos ciclos ocurren en un segundo, en radianes por segundo; φ es la *fase* de la señal. Si T es el período de la onda y f es la frecuencia de la onda, entonces ω tiene la siguiente relación con ellos:

$$\omega = \frac{2\pi}{T} = 2\pi f.$$

¡PRUÉBALO! Genera dos ondas seno con tiempo entre cero y 1 s cuya frecuencia es de 5 y 10 Hz, respectivamente, muestreadas a 100 Hz. Grafica las dos ondas y observa la diferencia. Cuenta cuántos ciclos hay en 1 s.

```

1 In [4]: # frecuencia de muestreo
2         sr = 100.0
3         # intervalo de muestreo
4         ts = 1.0/sr
5         t = np.arange(0,1,ts)
6         # frecuencia de la se al
7         freq = 5
8         y = np.sin(2*np.pi*freq*t)
9         plt.figure(figsize = (8, 8))
10        plt.subplot(211)
11        plt.plot(t, y, "b")
12        plt.ylabel("Amplitud")
13        freq = 10
14        y = np.sin(2*np.pi*freq*t)
15        plt.subplot(212)
16        plt.plot(t, y, "b")
17        plt.ylabel("Amplitud")
18        plt.xlabel("Tiempo (s)")
19        plt.show()

```

¡PRUÉBALO! Genera dos ondas seno con tiempo entre cero y 1 s. Ambas ondas tienen una frecuencia de 5 Hz, muestreadas a 100 Hz, pero sus fases son cero y 10, respectivamente. Las amplitudes de las dos ondas son 5 y 10, respectivamente. Grafica las dos ondas y observa la diferencia.

```

1 In [5]: # frecuencia de la se al
2         freq = 5
3         y = 5*np.sin(2*np.pi*freq*t)
4         plt.figure(figsize = (8, 8))
5         plt.subplot(211)
6         plt.plot(t, y, "b")
7         plt.ylabel("Amplitud")
8         y = 10*np.sin(2*np.pi*freq*t + 10)
9         plt.subplot(212)
10        plt.plot(t, y, "b")
11        plt.ylabel("Amplitud")
12        plt.xlabel("Tiempo (s)")
13        plt.show()

```

24.2 Transformada Discreta de Fourier (DFT)

La sección anterior demostró lo fácil que es caracterizar una onda con período/frecuencia, amplitud y fase. Esto solo se aplica a señales periódicas simples,

como ondas seno o coseno. Para ondas complicadas, esa caracterización ya no es fácil. Por ejemplo, la Fig. 24.3 es una onda relativamente más complicada, y es difícil decir cuál es la frecuencia y la amplitud de la onda.

Hay casos más complicados en el mundo real. La Transformada de Fourier se puede utilizar para estudiar estas ondas. Descompone cualquier señal en una suma de ondas seno y coseno fundamentales que miden fácilmente la frecuencia, amplitud y fase. La transformada de Fourier se puede aplicar a ondas continuas o discretas; este capítulo solo discutirá la Transformada Discreta de Fourier (DFT).

Usando la DFT, podemos componer la señal anterior en una serie de sinusoides, cada una con una frecuencia diferente. La siguiente figura 3D (Fig. 24.4) muestra la idea detrás de la DFT: la señal anterior es en realidad el resultado de la suma de tres ondas seno diferentes. La señal en el dominio del tiempo, que es la señal anterior, se puede transformar en una figura en el dominio de la frecuencia, llamada espectro de amplitud DFT, donde las frecuencias de la señal se muestran como barras verticales. La altura de la barra después de la normalización es la amplitud de la señal en el dominio del tiempo. Nótese que las tres barras verticales corresponden a las tres frecuencias de la onda seno, que también están graficadas en la figura.

Esta sección discutirá cómo usar la DFT para calcular y graficar el espectro de amplitud DFT.

24.2.1 DFT

La DFT puede transformar una secuencia de señales uniformemente espaciadas en información sobre la frecuencia de todas las ondas seno necesarias para sumar y obtener la señal en el dominio del tiempo. Se define como

$$X_k = \sum_{n=0}^{N-1} x_n \cdot e^{-i2\pi kn/N} = \sum_{n=0}^{N-1} x_n [\cos(2\pi kn/N) - i \cdot \sin(2\pi kn/N)]$$

- N = número de muestras
- n = muestra actual
- k = frecuencia actual, donde $k \in [0, N - 1]$
- x_n = el valor del seno en la muestra n

- X_k = la DFT que incluye información tanto de amplitud como de fase

Además, la última expresión en la ecuación anterior se deriva de la fórmula de Euler, que vincula las funciones trigonométricas con la función exponencial compleja: $e^{ix} = \cos x + i \sin x$.

Debido a la naturaleza de la transformada, $X_0 = \sum_{n=0}^{N-1} x_n$. Si N es un número impar, los elementos $X_1, X_2, \dots, X_{(N-1)/2}$ contienen los términos de frecuencia positiva, y los elementos $X_{(N+1)/2}, \dots, X_{N-1}$ contienen los términos de frecuencia negativa, en orden de frecuencia negativa decreciente. Si N es par, los elementos $X_1, X_2, \dots, X_{N/2-1}$ contienen los términos de frecuencia positiva, y los elementos $X_{N/2}, \dots, X_{N-1}$ contienen los términos de frecuencia negativa, en orden de frecuencia negativa decreciente. En el caso aquí, nuestra señal de entrada x es una secuencia de valores reales; por lo tanto, la salida DFT X_n para frecuencias positivas es el conjugado de los valores X_n para frecuencias negativas, y el espectro será simétrico. Normalmente, solo graficamos la DFT correspondiente a las frecuencias positivas.

Nótese que X_k es un número complejo que codifica tanto la amplitud como la información de fase de una componente sinusoidal compleja $e^{i \cdot 2\pi kn/N}$ de la función x_n . La amplitud y la fase de la señal se pueden calcular como

$$\text{amp} = \frac{|X_k|}{N} = \frac{\sqrt{\text{Re}(X_k)^2 + \text{Im}(X_k)^2}}{N}$$

$$\text{fase} = \text{atan2}(\text{Im}(X_k), \text{Re}(X_k))$$

donde $\text{Im}(X_k)$ y $\text{Re}(X_k)$ son las partes imaginaria y real del número complejo, y atan2 es la forma de dos argumentos de la función arctan.

Las amplitudes devueltas por DFT son iguales a las amplitudes de las señales alimentadas a la DFT si las normalizamos por el número de puntos de muestra. Mencionamos anteriormente que para una señal de valor real, la salida de la DFT se reflejará alrededor de la mitad de la frecuencia de muestreo (como se muestra en los siguientes ejemplos). Esta mitad de la frecuencia de muestreo se llama *frecuencia de Nyquist*. Esto generalmente significa que solo necesitamos mirar un lado del resultado de la DFT y descartar la información duplicada en la otra mitad; por lo tanto, para obtener la amplitud correspondiente a la de la señal en el dominio del tiempo, dividimos por $N/2$ en lugar de N .

Ahora que tenemos el conocimiento básico de DFT, veamos cómo podemos usarlo.

¡PRUÉBALO! Genera tres ondas seno con frecuencias de 1, 4 y 7 Hz, amplitudes 3, 1 y 0.5, y fases todas cero. Suma estas tres ondas seno con una frecuencia de muestreo de 100 Hz; es la misma señal mostrada al principio de la sección.

```

1 In [1]: import matplotlib.pyplot as plt
2         import numpy as np
3         plt.style.use("seaborn-v0_8-poster")
4         %matplotlib inline
5
6 In [2]: # frecuencia de muestreo
7         sr = 100
8         # intervalo de muestreo
9         ts = 1.0/sr
10        t = np.arange(0,1,ts)
11        freq = 1.
12        x = 3*np.sin(2*np.pi*freq*t)
13        freq = 4
14        x += np.sin(2*np.pi*freq*t)
15        freq = 7
16        x += 0.5* np.sin(2*np.pi*freq*t)
17
18        plt.figure(figsize = (8, 6))
19        plt.plot(t, x, "r")
20        plt.ylabel("Amplitud")
21        plt.show()

```

¡PRUÉBALO! Escribe una función DFT(x) que tome un argumento, x, con una señal de valor real 1D. La función calculará la DFT de la señal y devolverá los valores DFT. Aplica esta función a la señal que generamos arriba y grafica el resultado.

```

1 In [3]: def DFT(x):
2         """
3         Funci n para calcular la
4         Transformada Discreta de Fourier
5         de una se al real 1D x
6         """
7         N = len(x)
8         n = np.arange(N)
9         k = n.reshape((N, 1))
10        e = np.exp(-2j * np.pi * k * n / N)
11        X = np.dot(e, x)
12        return X
13
14 In [4]: X = DFT(x)

```

```

15     # calcular la frecuencia
16     N = len(X)
17     n = np.arange(N)
18     T = N/sr
19     freq = n/T
20     plt.figure(figsize = (8, 6))
21     plt.stem(freq, abs(X), "b", markerfmt=" ", basefmt="-
b")
22     plt.xlabel("Freq (Hz)")
23     plt.ylabel("DFT Amplitud |X(freq)|")
24     plt.show()

```

Nótese que la salida de la DFT es simétrica a la mitad de la frecuencia de muestreo (prueba diferentes frecuencias de muestreo por diversión). Como se mencionó anteriormente, esta mitad de la frecuencia de muestreo se llama *frecuencia de Nyquist* o *frecuencia de plegado*. Lleva el nombre del ingeniero eléctrico Harry Nyquist. Él y Claude Shannon formularon el teorema de muestreo de Nyquist-Shannon, que establece que una señal muestreada a una tasa puede reconstruirse completamente si contiene solo componentes de frecuencia por debajo de la mitad de esa frecuencia de muestreo; por lo tanto, la frecuencia más alta de salida de la DFT es la mitad de la frecuencia de muestreo.

```

1 In [5]: n_oneside = N//2
2         # obtener la frecuencia de un lado
3         f_oneside = freq[:n_oneside]
4         # normalizar la amplitud
5         X_oneside = X[:n_oneside]/n_oneside
6
7         plt.figure(figsize = (12, 6))
8         plt.subplot(121)
9         plt.stem(f_oneside, abs(X_oneside), "b", markerfmt="
", basefmt="-b")
10        plt.xlabel("Freq (Hz)")
11        plt.ylabel("DFT Amplitud |X(freq)|")
12        plt.subplot(122)
13        plt.stem(f_oneside, abs(X_oneside), "b", markerfmt="
", basefmt="-b")
14        plt.xlabel("Freq (Hz)")
15        plt.xlim(0, 10)
16        plt.tight_layout()
17        plt.show()

```

Graficando la primera mitad de los resultados de la DFT se muestran tres picos claros en frecuencia de 1, 4 y 7 Hz, con amplitudes 3, 1, 0.5, como

se esperaba. Así es como podemos usar la DFT para analizar una señal arbitraria descomponiéndola en ondas seno simples.

24.2.2 La DFT Inversa

Podemos calcular fácilmente la transformada inversa de la DFT:

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k \cdot e^{i \cdot 2\pi kn/N}.$$

Dejaremos esto como un ejercicio para que escribas una función.

24.2.3 El Límite de la DFT

El principal problema con la implementación de DFT anterior es que no es eficiente si tenemos una señal con muchos puntos de datos. Puede llevar mucho tiempo calcular la DFT si la señal es grande.

¡PRUÉBALO! Escribe una función para generar una señal simple con una frecuencia de muestreo diferente, y observa la diferencia en el tiempo de cálculo variando la frecuencia de muestreo.

```

1 In [6]: def gen_sig(sr):
2         """
3         funci n para generar
4         una se al 1D simple con
5         diferente frecuencia de muestreo
6         """
7         ts = 1.0/sr
8         t = np.arange(0,1,ts)
9         freq = 1.
10        x = 3*np.sin(2*np.pi*freq*t)
11        return x
12
13 In [7]: # frecuencia de muestreo =2000
14         sr = 2000
15         %timeit DFT(gen_sig(sr))

```

120 ms ± 8.27 ms por bucle (media ± desv. estándar de 7 ejecuciones, 10 bucles cada u

```

1 In [8]: # frecuencia de muestreo 20000
2         sr = 20000
3         %timeit DFT(gen_sig(sr))

```

15.9 s $\pm$ 1.51 s por bucle (media $\pm$ desv. estándar de 7 ejecuciones, 1 bucle ca

El creciente número de puntos de datos requerirá mucho tiempo de cálculo usando esta DFT. Afortunadamente, la Transformada Rápida de Fourier (FFT), popularizada por Cooley y Tukey en su artículo de 1965², puede resolver este problema de manera eficiente. Este es el tema de la siguiente sección.

24.3 Transformada Rápida de Fourier (FFT)

La Transformada Rápida de Fourier (FFT) es un algoritmo eficiente utilizado para calcular la DFT de una secuencia. Descrita por primera vez en el artículo clásico de Cooley y Tukey en 1965, la idea se remonta al trabajo no publicado de Gauss en 1805. Es un algoritmo de divide y vencerás que descompone recursivamente la DFT en DFT más pequeñas para reducir el número de cálculos. Como resultado, reduce con éxito la complejidad de la DFT de $O(n^2)$ a $O(n \log n)$, donde n es el tamaño de los datos. Esta reducción en el tiempo de cálculo es significativa, especialmente para datos con N grande, y la FFT es ampliamente utilizada en ingeniería, ciencia y matemáticas por esta razón.

Esta sección explorará cómo el uso de FFT reduce el tiempo de cálculo. El contenido de esta sección se basa en gran medida en este excelente tutorial³ elaborado por Jake VanderPlas⁴.

24.3.1 Simetrías en la DFT

La respuesta a cómo la FFT acelera el cálculo de la DFT radica en la explotación de las simetrías en la DFT. Estudiemos las simetrías en la DFT. De la definición de la ecuación DFT

$$X_k = \sum_{n=0}^{N-1} x_n \cdot e^{-i2\pi kn/N}.$$

²<http://www.ams.org/journals/mcom/1965-19-090/S0025-5718-1965-0178586-1/>

³<https://jakevdp.github.io/blog/2013/08/28/understanding-the-fft/>

⁴<http://vanderplas.com>

Podemos calcular

$$X_{k+N} = \sum_{n=0}^{N-1} x_n \cdot e^{-i2\pi(k+N)n/N} = \sum_{n=0}^{N-1} x_n \cdot e^{-i2\pi n} \cdot e^{-i2\pi kn/N}.$$

Nótese que $e^{-i2\pi n} = 1$; por lo tanto, tenemos

$$X_{k+N} = \sum_{n=0}^{N-1} x_n \cdot e^{-i2\pi kn/N} = X_k,$$

con una pequeña extensión, podemos tener

$$X_{k+i \cdot N} = X_k, \quad \text{para cualquier entero } i.$$

Por lo tanto, dentro de la DFT, claramente hay algunas simetrías que podemos usar para reducir el cálculo.

24.3.2 Trucos en FFT

Dado que hay simetrías en la DFT, podemos considerar usarlas para reducir el cálculo, porque si necesitamos calcular tanto X_k como X_{k+N} , solo necesitamos hacerlo una vez. Esta es exactamente la idea detrás de la FFT. Cooley y Tukey mostraron que podemos calcular la DFT de manera más eficiente si continuamos dividiendo el problema en otros más pequeños. Primero dividamos toda la serie en dos partes, es decir, las partes pares e impares:

$$\begin{aligned} X_k &= \sum_{n=0}^{N-1} x_n \cdot e^{-i2\pi kn/N} \\ &= \sum_{m=0}^{N/2-1} x_{2m} \cdot e^{-i2\pi k(2m)/N} + \sum_{m=0}^{N/2-1} x_{2m+1} \cdot e^{-i2\pi k(2m+1)/N} \\ &= \sum_{m=0}^{N/2-1} x_{2m} \cdot e^{-i2\pi km/(N/2)} + e^{-i2\pi k/N} \sum_{m=0}^{N/2-1} x_{2m+1} \cdot e^{-i2\pi km/(N/2)}. \end{aligned}$$

Como se muestra, los dos términos más pequeños, que solo tienen la mitad del tamaño ($\frac{N}{2}$) en la ecuación anterior, son dos DFT más pequeñas. Para cada término, $0 \leq m \leq \frac{N}{2}$, pero $0 \leq k \leq N$; por lo tanto, la mitad de

los valores serán los mismos debido a las propiedades de simetría descritas anteriormente. Así, solo necesitamos calcular la mitad de los campos en cada término. Por supuesto, podemos continuar dividiendo cada término por la mitad con los valores pares e impares hasta llegar a los últimos dos números, en cuyo punto el cálculo será realmente simple.

Así es como funciona la FFT usando este enfoque recursivo. Realicemos una implementación rápida y simple de la FFT. Nótese que la señal de entrada a la FFT debe tener una longitud de potencia de 2. Si no es así, entonces necesitamos rellenar con ceros hasta el siguiente tamaño de potencia de 2.

```

1 In [1]: import matplotlib.pyplot as plt
2         import numpy as np
3         plt.style.use("seaborn-v0_8-poster")
4         %matplotlib inline
5
6 In [2]: def FFT(x):
7         """
8         Una implementación recursiva
9         de la FFT de Cooley-Tukey 1D, la
10        entrada debe tener una longitud de
11        potencia de 2.
12        """
13        N = len(x)
14        if N == 1:
15            return x
16        else:
17            X_even = FFT(x[::2])
18            X_odd = FFT(x[1::2])
19            factor = np.exp(-2j*np.pi*np.arange(N)/ N)
20            X = np.concatenate(\
21                [X_even+factor[:int(N/2)]*X_odd,
22                 X_even+factor[int(N/2):]*X_odd])
23            return X
24
25 In [3]: # frecuencia de muestreo
26         sr = 128
27         # intervalo de muestreo
28         ts = 1.0/sr
29         t = np.arange(0,1,ts)
30         freq = 1.
31         x = 3*np.sin(2*np.pi*freq*t)
32         freq = 4
33         x += np.sin(2*np.pi*freq*t)

```

```

34     freq = 7
35     x += 0.5* np.sin(2*np.pi*freq*t)
36
37     plt.figure(figsize = (8, 6))
38     plt.plot(t, x, "r")
39     plt.ylabel("Amplitud")
40     plt.show()

```

¡PRUÉBALO! Usa la función FFT para calcular la transformada de Fourier de la señal anterior. Grafica el espectro de amplitud para las frecuencias de dos lados y un lado.

```

1 In [4]: X = FFT(x)
2         # calcular la frecuencia
3         N = len(X)
4         n = np.arange(N)
5         T = N/sr
6         freq = n/T
7         plt.figure(figsize = (12, 6))
8         plt.subplot(121)
9         plt.stem(freq, abs(X), "b", markerfmt=" ", basefmt="-
10         b")
11         plt.xlabel("Freq (Hz)")
12         plt.ylabel("FFT Amplitud |X(freq)|")
13         # Obtener el espectro de un lado
14         n_oneside = N//2
15         # obtener la frecuencia de un lado
16         f_oneside = freq[:n_oneside]
17         # normalizar la amplitud
18         X_oneside = X[:n_oneside]/n_oneside
19         plt.subplot(122)
20         plt.stem(f_oneside, abs(X_oneside), "b", markerfmt="
21         ", basefmt="-b")
22         plt.xlabel("Freq (Hz)")
23         plt.ylabel("FFT Amplitud Normalizada |X(freq)|")
24         plt.tight_layout()
25         plt.show()

```

¡PRUÉBALO! Genera una señal simple de longitud 2048, y registra el tiempo que tomará ejecutar la FFT; compara la velocidad con la DFT.

```

1 In [5]: def gen_sig(sr):
2         """
3         funci n para generar
4         una se al 1D simple con
5         diferente frecuencia de muestreo
6         """

```

```

7         ts = 1.0/sr
8         t = np.arange(0,1,ts)
9         freq = 1.
10        x = 3*np.sin(2*np.pi*freq*t)
11        return x
12
13 In [6]: # frecuencia de muestreo =2048
14        sr = 2048
15        %timeit FFT(gen_sig(sr))

```

16.9 ms $\pm$ 1.3 ms por bucle (media $\pm$ desv. estándar de 7 ejecuciones, 100 bucles)

Por lo tanto, para una señal con longitud 2048 (alrededor de 2000), esta implementación de FFT usa 16.9 ms en lugar de 120 ms usando DFT. Nótese que hay muchas formas de optimizar la implementación de FFT para hacerla más rápida. La siguiente sección introducirá las funciones FFT incorporadas en Python, que se ejecutarán mucho más rápido.

24.4 FFT en Python

Python tiene funciones FFT muy maduras tanto en NumPy como en SciPy. Esta sección echará un vistazo a ambos paquetes y verá qué tan fácilmente pueden incorporarse en nuestro trabajo. Usemos la señal generada arriba para ejecutar la prueba como se muestra en la Fig. 24.5.

24.4.1 FFT en NumPy

EJEMPLO: Usa las funciones `fft` e `ifft` de NumPy para calcular el espectro de amplitud FFT y la FFT inversa para obtener la señal original. Grafica ambos resultados. Mide el tiempo de la función `fft` usando esta señal de longitud 2000.

```

1 In [3]: from numpy.fft import fft, ifft
2         X = fft(x)
3         N = len(X)
4         n = np.arange(N)
5         T = N/sr
6         freq = n/T
7         plt.figure(figsize = (12, 6))
8         plt.subplot(121)

```



```

9      plt.stem(freq, np.abs(X), "b", markerfmt=" ", basefmt
    = "-b")
10      plt.xlabel("Freq (Hz)")
11      plt.ylabel("FFT Amplitud |X(freq)|")
12      plt.xlim(0, 10)
13      plt.subplot(122)
14      plt.plot(t, ifft(X), "r")
15      plt.xlabel("Tiempo (s)")
16      plt.ylabel("Amplitud")
17      plt.tight_layout()
18      plt.show()
19
20 In [4]: %timeit fft(x)

```

42.3 s $\pm$ 5.03 s por bucle (media $\pm$ desv. estándar de 7 ejecuciones, 10000 bucles cada

24.4.2 FFT en SciPy

EJEMPLO: Usa las funciones `fft` e `ifft` de SciPy para calcular el espectro de amplitud FFT y la FFT inversa para obtener la señal original. Grafica ambos resultados. Mide el tiempo de la función `fft` usando esta señal de longitud 2000.

```

1 In [5]: from scipy.fftpack import fft, ifft
2         X = fft(x)
3         plt.figure(figsize = (12, 6))
4         plt.subplot(121)
5         plt.stem(freq, np.abs(X), "b", markerfmt=" ", basefmt
    = "-b")
6         plt.xlabel("Freq (Hz)")
7         plt.ylabel("FFT Amplitud |X(freq)|")
8         plt.xlim(0, 10)
9         plt.subplot(122)
10        plt.plot(t, ifft(X), "r")
11        plt.xlabel("Tiempo (s)")
12        plt.ylabel("Amplitud")
13        plt.tight_layout()
14        plt.show()
15
16 In [6]: %timeit fft(x)

```

12.6 s $\pm$ 222 ns por bucle (media $\pm$ desv. estándar de 7 ejecuciones, 100000 bucles cad

Nótese que las funciones FFT incorporadas son mucho más rápidas y fáciles de usar, especialmente cuando se usa la versión de SciPy. Aquí están los resultados para comparar:

- DFT implementada: 120 ms
- FFT implementada: 16.9 ms
- NumPy FFT: 42.3 s
- SciPy FFT: 12.6 s

24.4.3 Más Ejemplos

A continuación se muestran más ejemplos de cómo usar FFT en aplicaciones del mundo real.

24.4.3.1 Demanda de Electricidad en California

Primero, exploramos la demanda de electricidad de California desde el 30/11/2019 hasta el 30/12/2019. Puedes descargar los datos de la Administración de Información Energética de EE. UU.⁵. Aquí, los datos se han descargado para ti.

Los datos de demanda de electricidad de California están almacenados en "930-data-export.csv" en 3 columnas. Recuerda que aprendimos a leer archivos CSV usando NumPy. Aquí, usaremos otro paquete, **pandas**, que es un paquete muy popular que trata con datos de series temporales. Este paquete no se discutirá aquí. Recomendamos encarecidamente aprender a usarlo por ti mismo.

Primero, estudiemos los datos.

```
1 In [7]: import pandas as pd
```

La función `read_csv` leerá el archivo CSV. Presta atención al parámetro `parse_dates`, que encontrará la fecha y hora en la columna uno. Los datos se leerán en un DataFrame de pandas, y usamos `df` para almacenarlos. Luego cambiaremos el encabezado en el archivo original a algo más fácil de usar.

⁵https://www.eia.gov/beta/electricity/gridmonitor/dashboard/electric_overview/US48/US48

```

1 In [8]: df = pd.read_csv("./data/930-data-export.csv",
2                       delimiter=",", parse_dates=[1])
3         df.rename(columns={"Timestamp (Hour Ending)": "hour",
4                       "Total CAL Demand (MWh)": "demand"
5         },
6               inplace=True)

```

Al graficar los datos, vemos cómo cambia la demanda de electricidad a lo largo del tiempo.

```

1 In [9]: plt.figure(figsize = (12, 6))
2         plt.plot(df["hour"], df["demand"])
3         plt.xlabel("Datetime")
4         plt.ylabel("California electricity demand (MWh)")
5         plt.xticks(rotation=25)
6         plt.show()

```

A partir de la serie temporal graficada, es difícil discernir si hay algún patrón detrás de los datos. Transformemos los datos al dominio de la frecuencia y veamos si hay algo interesante.

```

1 In [10]: X = fft(df["demand"])
2          N = len(X)
3          n = np.arange(N)
4          # obtener la frecuencia de muestreo
5          sr = 1 / (60*60)
6          T = N/sr
7          freq = n/T
8          # Obtener el espectro de un lado
9          n_oneside = N//2
10         # obtener la frecuencia de un lado
11         f_oneside = freq[:n_oneside]
12         plt.figure(figsize = (12, 6))
13         plt.plot(f_oneside, np.abs(X[:n_oneside]), "b")
14         plt.xlabel("Freq (Hz)")
15         plt.ylabel("FFT Amplitud |X(freq)|")
16         plt.show()

```

Nótese los picos claros en la figura de amplitud FFT, pero es difícil saber qué son en términos de frecuencia. Grafiquemos los resultados usando horas y resaltemos algunas de las horas asociadas con los picos.

```

1 In [11]: # convertir frecuencia a hora
2          t_h = 1/f_oneside / (60 * 60)
3          plt.figure(figsize=(12,6))
4          plt.plot(t_h, np.abs(X[:n_oneside])/n_oneside)
5          plt.xticks([12, 24, 84, 168])

```

```

6     plt.xlim(0, 200)
7     plt.xlabel("Periodo ($hora$)")
8     plt.show()

```

Ahora podemos ver algunos patrones interesantes, es decir, tres picos asociados con 12, 24 y 84 horas. Estos picos significan que vemos alguna señal repetitiva cada 12, 24 y 84 horas. Esto tiene sentido y corresponde a nuestro patrón de actividad humana. La FFT puede ayudarnos a entender algunas de las señales repetitivas en nuestro mundo físico.

24.4.3.2 Filtrado de una Señal en el Dominio de la Frecuencia

El filtrado es un proceso en el procesamiento de señales para eliminar partes no deseadas de la señal dentro de cierto rango de frecuencia. Los filtros de paso bajo eliminan todas las señales por encima de cierta frecuencia de corte; los filtros de paso alto hacen lo contrario. La combinación de filtros de paso bajo y paso alto permite construir un filtro de paso banda, lo que significa que solo mantenemos las señales dentro de un par de frecuencias. Usando FFT, podemos hacer esto fácilmente. Juguemos con el siguiente ejemplo para ilustrar los conceptos básicos de un filtro de paso banda. Nótese que solo queremos mostrar la idea de filtrado usando operaciones muy básicas; en realidad, los procesos de filtrado son mucho más sofisticados.

EJEMPLO: Usa la señal generada al principio de esta sección (las ondas seno mixtas con 1, 4 y 7 Hz) y aplica un filtro de paso alto a esta señal en 6 Hz. Grafica la señal filtrada y la amplitud FFT antes y después del filtrado.

```

1 In [12]: from scipy.fftpack import fftfreq
2
3 In [13]: plt.figure(figsize = (8, 6))
4           plt.plot(t, x, "r")
5           plt.ylabel("Amplitud")
6           plt.title("Señal original")
7           plt.show()
8
9 In [14]: # FFT de la señal
10          sig_fft = fft(x)
11          # copiar los resultados de la FFT
12          sig_fft_filtered = sig_fft.copy()
13          # obtener las frecuencias usando la función de
SciPy
14          freq = fftfreq(len(x), d=1./2000)
15          # definir la frecuencia de corte

```

```

16         cut_off = 6
17         # filtro de paso alto asignando ceros a las
18         # amplitudes FFT donde las frecuencias absolutas
19         # son menores que la frecuencia de corte
20         sig_fft_filtered[np.abs(freq) < cut_off] = 0
21         # obtener la se al filtrada en el dominio del
tiempo
22         filtered = ifft(sig_fft_filtered)
23         # graficar la se al filtrada
24         plt.figure(figsize = (12, 6))
25         plt.plot(t, filtered)
26         plt.xlabel("Tiempo (s)")
27         plt.ylabel("Amplitud")
28         plt.show()
29         # graficar la amplitud FFT antes y despu s
30         plt.figure(figsize = (12, 6))
31         plt.subplot(121)
32         plt.stem(freq, np.abs(sig_fft), "b", markerfmt=" ",
basefmt="-b")
33         plt.title("Antes del filtrado")
34         plt.xlim(0, 10)
35         plt.xlabel("Frecuencia (Hz)")
36         plt.ylabel("FFT Amplitud")
37         plt.subplot(122)
38         plt.stem(freq, np.abs(sig_fft_filtered), "b",
markerfmt=" ", basefmt="-b")
39         plt.title("Despu s del filtrado")
40         plt.xlim(0, 10)
41         plt.xlabel("Frecuencia (Hz)")
42         plt.ylabel("FFT Amplitud")
43         plt.tight_layout()
44         plt.show()

```

En el ejemplo anterior, asignamos cualquier frecuencia absoluta de la amplitud FFT a cero, y regresamos a la señal en el dominio del tiempo; logramos un filtro de paso alto muy básico en pocos pasos. Por lo tanto, la FFT puede ayudarnos a obtener la señal que nos interesa y eliminar las que no son deseadas.

24.5 Resumen y Problemas

24.5.1 Resumen

1. Aprendimos los conceptos básicos de las ondas: frecuencia, período, amplitud y longitud de onda son características de las ondas.
2. La Transformada Discreta de Fourier (DFT) es una forma de transformar una señal del dominio del tiempo al dominio de la frecuencia usando la suma de una secuencia de ondas seno.
3. La Transformada Rápida de Fourier (FFT) es un algoritmo utilizado para calcular las DFT de manera eficiente aprovechando las propiedades de simetría en la DFT.

24.5.2 Problemas

Problema 1

Se te pide medir la temperatura de la habitación durante 30 días. Cada día al mediodía mides la temperatura y registras el valor. ¿Cuál es la frecuencia de la señal de temperatura que obtienes?

Solución: La frecuencia es de 1 ciclo por día, o $1/86400$ Hz (ya que hay 86400 segundos en un día). En términos prácticos, la frecuencia es $1/(24 \text{ horas}) = 1/24$ ciclos por hora, etc.

Problema 2

¿Cuál es la relación entre la frecuencia y el período de una onda?

Solución: La frecuencia f y el período T son inversamente proporcionales: $f = 1/T$ y $T = 1/f$.

Problema 3

¿Cuál es la diferencia entre período y longitud de onda? ¿Cuáles son las similitudes entre ellos?

Solución: El período es el tiempo que tarda en completarse un ciclo (dimensión temporal), mientras que la longitud de onda es la distancia espacial entre dos crestas consecutivas (dimensión espacial). Están relacionados a través de la velocidad de propagación de la onda: $v = \lambda/T = \lambda f$.

Problema 4

¿Cuáles son las representaciones en el dominio del tiempo y en el dominio de la frecuencia de una señal?

Solución: El dominio del tiempo muestra cómo varía la amplitud de la señal a lo largo del tiempo. El dominio de la frecuencia muestra qué frecuencias están presentes en la señal y con qué amplitudes.

Problema 5

Genera dos señales: la señal 1 es una onda seno con 5 Hz, amplitud 3 y desfase 3, y la señal 2 es una onda seno con 2 Hz, amplitud 2 y desfase -2. Grafica la señal durante 2 s.

```

1 In [1]: # frecuencia de muestreo
2         sr = 100
3         # intervalo de muestreo
4         ts = 1.0/sr
5         t = np.arange(0,2,ts)
6         freq = 5.
7         x = 3*np.sin(2*np.pi*freq*t + 3)
8         freq = 2
9         x += 2*np.sin(2*np.pi*freq*t - 2)
10        plt.figure(figsize = (8, 6))
11        plt.plot(t, x, "r")
12        plt.ylabel("Amplitud")
13        plt.xlabel("Tiempo (s)")
14        plt.show()

```

Problema 6

Muestrea la señal que generaste en el Problema 5 usando frecuencias de muestreo de 5, 10, 20, 50 y 100 Hz, y observa las diferencias entre las diferentes frecuencias de muestreo.

Solución: Se puede escribir un bucle para generar las señales con diferentes frecuencias de muestreo y compararlas visualmente. A menor frecuencia de muestreo, la señal se reconstruye con menos puntos y puede perderse información.

```

1 sampling_rates = [5, 10, 20, 50, 100]
2 plt.figure(figsize=(12, 10))
3 for i, sr in enumerate(sampling_rates):
4     ts = 1.0/sr

```

```

5     t = np.arange(0,2,ts)
6     x = 3*np.sin(2*np.pi*5*t + 3) + 2*np.sin(2*np.pi*2*t - 2)
7     plt.subplot(len(sampling_rates),1,i+1)
8     plt.plot(t, x, 'o-')
9     plt.title(f'Frecuencia de muestreo = {sr} Hz')
10 plt.tight_layout()
11 plt.show()

```

Problema 7

Dada una señal $t = [0, 1, 2, 3]$ y $y = [0, 3, 2, 0]$, encuentra la DFT real de X . Escribe la expresión para la DFT inversa. No uses Python para encontrar los resultados. Escribe las ecuaciones y calcula los valores.

Solución: Para $N = 4$, la DFT está dada por:

$$X_k = \sum_{n=0}^3 x_n e^{-i2\pi kn/4}, \quad k = 0, 1, 2, 3.$$

Calculando:

$$X_0 = 0 + 3 + 2 + 0 = 5$$

$$X_1 = 0 \cdot e^0 + 3 \cdot e^{-i\pi/2} + 2 \cdot e^{-i\pi} + 0 \cdot e^{-i3\pi/2} = 3(-i) + 2(-1) = -2 - 3i$$

$$X_2 = 0 + 3 \cdot e^{-i\pi} + 2 \cdot e^{-i2\pi} + 0 = 3(-1) + 2(1) = -1$$

$$X_3 = 0 + 3 \cdot e^{-i3\pi/2} + 2 \cdot e^{-i3\pi} + 0 = 3(i) + 2(-1) = -2 + 3i$$

La DFT inversa es:

$$x_n = \frac{1}{4} \sum_{k=0}^3 X_k e^{i2\pi kn/4}.$$

Problema 8

¿Cuáles son la amplitud y la fase de los valores de DFT para una señal?

Solución: La amplitud es $|X_k| = \sqrt{\text{Re}(X_k)^2 + \text{Im}(X_k)^2}$, y la fase es $\phi_k = \text{atan2}(\text{Im}(X_k), \text{Re}(X_k))$.

Problema 9

Implementamos la DFT anteriormente. ¿Puedes implementar la DFT inversa en Python de manera similar?


```

1 def IDFT(X):
2     """
3     Funci n para calcular la
4     Transformada Discreta de Fourier Inversa
5     """
6     N = len(X)
7     n = np.arange(N)
8     k = n.reshape((N, 1))
9     e = np.exp(2j * np.pi * k * n / N)
10    x = np.dot(e, X) / N
11    return x

```

Problema 10

Usa la función DFT y la DFT inversa que implementamos, y genera el espectro de amplitud para la señal que generaste en el Problema 5. Normaliza la amplitud DFT para obtener la amplitud correcta correspondiente en el dominio del tiempo.

```

1 sr = 100
2 ts = 1.0/sr
3 t = np.arange(0,2,ts)
4 x = 3*np.sin(2*np.pi*5*t + 3) + 2*np.sin(2*np.pi*2*t - 2)
5 X = DFT(x)
6 N = len(X)
7 freq = np.fft.fftfreq(N, d=ts)
8 # Espectro de un lado
9 n_oneside = N//2
10 plt.stem(freq[:n_oneside], np.abs(X[:n_oneside])/n_oneside)
11 plt.xlabel('Frecuencia (Hz)')
12 plt.ylabel('Amplitud')
13 plt.show()

```

Problema 11

¿Puedes describir los trucos utilizados en FFT para hacer el cálculo más rápido?

Solución: La FFT utiliza un enfoque de divide y vencerás, dividiendo la DFT en DFT más pequeñas explotando la simetría de los factores de fase. Esto reduce la complejidad de $O(n^2)$ a $O(n \log n)$. La versión más común es la de Cooley-Tukey, que divide la secuencia en partes pares e impares recursivamente.

Problema 12

Usa la función `fft` e `ifft` de SciPy para repetir el Problema 10.

```
1 from scipy.fftpack import fft, ifft
2 X = fft(x)
3 plt.stem(freq[:n_oneside], np.abs(X[:n_oneside])/(n_oneside))
4 plt.xlabel('Frecuencia (Hz)')
5 plt.ylabel('Amplitud')
6 plt.show()
```

Problema 13

Agrega ruido de distribución normal aleatorio a la señal del Problema 5 usando NumPy y grafica el espectro de amplitud FFT. ¿Qué observas? La señal con ruido se mostrará en el siguiente caso de prueba.

```
1 In [2]: np.random.seed(10)
2         x_noise = x + np.random.normal(0, 2, size = len(x))
3         plt.figure(figsize = (8, 6))
4         plt.plot(t, x_noise, "r")
5         plt.ylabel("Amplitud")
6         plt.xlabel("Tiempo (s)")
7         plt.show()
```

Solución: Al agregar ruido, el espectro de amplitud mostrará componentes de frecuencia en todo el rango, no solo en las frecuencias originales. Las amplitudes de las frecuencias originales pueden estar ligeramente alteradas, y aparecerán muchas frecuencias con pequeñas amplitudes debido al ruido.

```
1 X_noise = fft(x_noise)
2 plt.figure(figsize=(12,6))
3 plt.subplot(121)
4 plt.plot(t, x_noise)
5 plt.title('Señal con ruido')
6 plt.subplot(122)
7 plt.stem(freq[:n_oneside], np.abs(X_noise[:n_oneside])/(
    n_oneside))
8 plt.title('Espectro de amplitud con ruido')
9 plt.xlabel('Frecuencia (Hz)')
10 plt.ylabel('Amplitud')
11 plt.tight_layout()
12 plt.show()
```

APENDICES

A.1 PRIMEROS PASOS CON PYTHON EN WINDOWS

A.1.1 CONFIGURACIÓN DE TU ENTORNO DE TRABAJO EN WINDOWS

Antes de empezar a usar Python, necesitamos configurar nuestro entorno de trabajo de Python en la computadora. En esta sección, introduciremos los procesos para ponerlo en marcha.

Hay diferentes formas de instalar Python y los paquetes relacionados; aquí recomendamos usar Anaconda⁶ o Miniconda⁷ para instalar y administrar tus paquetes. Dependiendo del sistema operativo (SO) que estés usando, es decir, Windows, Mac OS X o Linux, necesitas descargar un instalador específico para tu máquina. Tanto Anaconda como Miniconda tienen como objetivo proporcionar formas sencillas de administrar el entorno de trabajo de Python en computación científica y ciencia de datos.

Aquí usaremos Windows como ejemplo para mostrarte los procesos de instalación. Para usuarios de Mac y Linux, por favor lean el Capítulo 1.1 para todos los procesos. Las principales diferencias entre Anaconda y Miniconda son:

- **Anaconda** es un framework de distribución completo que incluye el intérprete de Python, el administrador de paquetes, así como los paquetes comúnmente usados en computación científica.

⁶<https://www.anaconda.com/download/>

⁷<https://conda.io/miniconda.html>

- **Miniconda** es una versión ligera de Anaconda que no incluye los paquetes comunes; por lo tanto, necesitas instalar todos los diferentes paquetes por ti mismo. Pero sí tiene el intérprete de Python y el administrador de paquetes.

La opción que elegimos aquí es usar Miniconda para administrar nuestra instalación de los paquetes. De esta manera, solo podemos instalar aquellos que necesitamos.

El proceso de instalación de Miniconda se describe a continuación:

Paso 1. Descarga el instalador de Miniconda desde el sitio web⁸ como se muestra en la Fig. A.1. Aquí puedes elegir un instalador diferente según tu SO. Elegimos el instalador de Windows y Python 3.7 como ejemplo.

Figura A.1: La página de descarga de Miniconda. Elige el instalador según tu s
[IMAGEN: Página web de descarga de Miniconda mostrando opciones para Windows,

Paso 2. Ejecuta el instalador haciendo doble clic en él. Después de ejecutar el instalador, sigue la guía y lo instalarás con éxito (Fig. A.2). Una cosa a tener en cuenta es que puedes cambiar la ubicación de instalación proporcionando una ubicación alternativa en tu máquina; usaremos la ruta predeterminada aquí (Fig. A.3).

Figura A.2: Captura de pantalla de la ejecución del instalador en el símbolo d
[IMAGEN: Ventana del instalador de Miniconda en progreso]

Figura A.3: La ubicación de instalación predeterminada de tu sistema de archiv
[IMAGEN: Selector de carpeta mostrando la ruta de instalación predeterminada]

Después de la instalación, podemos abrir el símbolo del sistema de Anaconda (la terminal equivalente en Mac o Linux) desde el menú de inicio como se muestra en la Fig. A.4. Luego podemos verificar si la instalación fue exitosa o no escribiendo los siguientes comandos mostrados en la Fig. A.5.

Figura A.4: Abre el símbolo del sistema de Anaconda desde el menú de inicio.
[IMAGEN: Menú de inicio de Windows resaltando Anaconda Prompt]

Figura A.5: Una forma rápida de verificar si tu Miniconda instalado funciona c
[IMAGEN: Símbolo del sistema mostrando comandos como 'python --version' y 'con

⁸<https://conda.io/miniconda.html>

Paso 3. Siguiendo la Fig. A.6, instala los paquetes básicos que se utilizan en este libro. Primero instalemos algunos paquetes para nuestro libro: IPython, NumPy, SciPy, pandas, matplotlib y Jupyter notebook. Hablaremos más sobre la administración de los paquetes usando pip y conda más adelante.

Figura A.6: Proceso de instalación de los paquetes que se usarán en el resto del libro
[IMAGEN: Símbolo del sistema ejecutando 'conda install ipython numpy scipy pandas matplotlib']

A.1.2 TRES FORMAS DE EJECUTAR CÓDIGO PYTHON

Hay diferentes formas de ejecutar código Python; todas tienen diferentes usos. En esta sección, introduciremos rápidamente las tres formas diferentes para que empieces.

Usando el shell de Python o IPython

La forma más fácil de ejecutar código Python es a través del shell de Python o IPython (que significa Interactive Python). El shell de IPython es más rico que el shell de Python debido a, por ejemplo, autocompletado con tabulador, mensajes de error resaltados en color, integración básica con shell UNIX, etc. Dado que acabamos de instalar IPython, intentemos ejecutar el ejemplo de "hola mundo" con él. La forma de iniciar el shell de Python o IPython es escribiéndolo en el símbolo del sistema de Anaconda (ver la figura). Luego podemos ejecutar el comando de Python escribiéndolo en el shell; al presionar Enter, vemos inmediatamente los resultados del comando. Por ejemplo, podemos imprimir "Hola Mundo" escribiendo `print("Hola Mundo")` como se muestra en la Fig. A.7.

Figura A.7: Ejecuta "Hola Mundo" en el shell de IPython escribiendo el comando. "print('Hola Mundo')"
[IMAGEN: Shell de IPython mostrando 'print("Hola Mundo")' y la salida 'Hola Mundo']

En el comando anterior, `print()` es una función en Python, y "Hola Mundo" es un tipo de dato cadena (string) que introduciremos en el libro.

Ejecutar un script/archivo de Python desde la línea de comandos

La segunda forma de ejecutar código Python es poner todos los comandos en un archivo y guardarlo con extensión `.py` (la extensión del archivo podría

ser cualquier cosa, pero por convención, suele ser `.py`). Por ejemplo, usa tu editor de texto favorito (aquí se muestra Visual Studio Code⁹), coloca el comando en un archivo llamado `hola_mundo.py` como se muestra en la Fig. A.8. Luego simplemente ejecútalo desde el símbolo del sistema (Fig. A.9).

Figura A.8: Un ejemplo de archivo de script de Python usando Visual Studio Cod
[IMAGEN: Editor de texto mostrando `'print("Hola Mundo")'` en un archivo llamado

Figura A.9: Para ejecutar el script de Python desde la línea de comandos, pode
[IMAGEN: Símbolo del sistema ejecutando `'python hola_mundo.py'` y mostrando la

Usando Jupyter notebook

La tercera forma de ejecutar Python es a través de Jupyter notebook. Es un entorno de Python basado en navegador muy potente; hablaremos más sobre él en detalle más adelante en este capítulo. Aquí solo veremos rápidamente cómo podríamos ejecutar el código desde un Jupyter notebook. Escribe `jupyter notebook` en la línea de comandos:

```
1 jupyter notebook
```

Luego verás una página web local emergente; desde el botón superior derecho crea un nuevo notebook de Python3 como se muestra en la Fig. A.10.

Figura A.10: Para lanzar un servidor Jupyter notebook, escribe `"jupyter notebook"`
[IMAGEN: Interfaz web de Jupyter mostrando el botón 'Nuevo' y la opción 'Python3'

Ejecutar código en Jupyter notebook es fácil: escribes tu código en la celda y presionas `Shift + Enter` para ejecutar la celda; los resultados se mostrarán debajo del código (Fig. A.11).

Figura A.11: Ejecuta el ejemplo de Hola Mundo dentro de Jupyter notebook. Escr
[IMAGEN: Notebook de Jupyter mostrando una celda con `'print("Hola Mundo")'` y l

⁹<https://code.visualstudio.com>

Índice

A

- Acotamiento, 339, 380, 411
- Amortiguamiento, 396
- Amplitud, 417
- Anaconda, 4
- Analítica, 325
- Ángulo entre dos vectores, 238
- Animación, 212
- Arreglos, 236
- Autovalor, 265
- Autovector, 265

B

- Base de funciones, 279
- Base10, 145
- Bisección, método de, 327
- Bit, 145
- Booleano, 21
- Búsqueda de raíces, 325

C

- Cadena (string), 30
- Característica, 147
- Celda, 9

- Cero vector, 236, 245
- Clase, 121, 123
- Clase hija, 127
- Clase padre, 127
- Coeficientes, 239
- Comando no asignado, 29
- Comprensiones, 100
- Comprensión de diccionario, 101
- Comprensión de lista, 100
- Conjunto, 40, 235
- Conjunto vacío, 235
- Converger, 326
- Coseno hiperbólico, 364
- Cota de error, 322
- Cuerpo de la función, 55

D

- Datos HDF5, 185
- Derivada parcial, 372
- Desbordamiento, 150
- Determinante, 241
- Diccionarios, 42
- Diferencia central, 338, 340
- Diferencia hacia adelante, 338, 339

- Diferencia hacia atrás, 338, 340
- Diferencias divididas, 306
- Diferencias finitas, 337
- Discreto, 337
- Discretizado, 337
- Divide y vencerás, 110
- Dominio, 245
- Dominio de la frecuencia, 415
- Dominio del tiempo, 415

E

- Ecuación característica, 268
- Ecuación de calor, 371
- Ecuación de onda, 415
- Ecuación diferencial, 371
- Ecuación diferencial ordinaria, 371
- Ecuación diferencial parcial, 372
- Ecuaciones lineales, 245
- Eigenvalores, ver Autovalor
- Eliminación de Gauss, 248
- Encapsulación, 127
- Enteros, 236
- Entorno de trabajo, 4
- Error, 326

- Error de redondeo, 151, 320
- Error de truncamiento, 151, 320
- Error lógico, 160
- Error numérico, 380
- Escalar, 238
- Espaciado, 337
- Espacio nulo, 245
- Espectro de amplitud, 421
- Estado, 374
- Estructura de control, 77
- Euler, método de, 375
- Expresión lógica, 20
- Expresión matemática, 20

F

- F-String, 34
- Factor de peso, 279
- Factorial, 105
- Fase, 419
- Fibonacci, 106
- Filtrado, 438
- Flotante, 14, 147
- Forma matricial, 246
- Frecuencia, 418

- Frecuencia angular, 419
- Frecuencia de muestreo, 419
- Frecuencia de Nyquist, 423, 425
- Frontera, 399
- Función, 55
- Función de estimación, 279
- Función de interpolación, 295
- Función lambda, 69
- Función objetivo, 401
- Funciones de base, 279

G

- Gauss, método de, 248
- Gauss-Seidel, método de, 254
- Gradiente, 280

H

- Herencia, 123, 127
- Hertz, 418
- Heterogéneo, 39
- Homogéneo, 39

I

- Identidad, matriz, 241
- If-Else, 77
- Imagen, 245

- Imaginario, 14
- In, 236
- Índice, 35
- Inestabilidad, 380, 411
- Infinito, 149
- Inmutable, 39
- Inicial, problema de valor, 373
- Instalación, 3
- Int, 14
- Intersección, 235
- Intervalo, 353
- Interpolación, 295
- Interpolación cúbica, 297
- Interpolación de Lagrange, 301
- Interpolación lineal, 296
- Interpolación polinomial de Newton, 305
- Inversa, 242
- Invertible, 242
- IPython, 7
- Iteración, 253

J

- Jacobi, método de, 254
- Jupyter notebook, 9, 18

K

- Kernel, 18

L

- Lagrange, polinomio de, 301
- Listas, 35
- Longitud de onda, 417

M

- Mal condicionado, 243
- Matplotlib, 189
- Matriz, 239
- Matriz aumentada, 244
- Matriz de permutación, 257
- Matriz identidad, 241
- Matriz inversa, 242
- Matriz triangular superior, 248
- Método de bisección, 327
- Método de diferencias finitas, 406
- Método de disparo, 401
- Método de Euler, 375
- Método de Newton-Raphson, 330
- Método de punto medio, 382
- Método de Runge-Kutta, 383
- Método de Runge-Kutta de cuarto orden, 384

- Método del trapecio, 356
- Método QR, 273
- Métodos iterativos, 253
- Métodos multipaso, 389
- Miniconda, 4
- Módulo, 15, 161
- Multiplicación escalar, 238
- Multiplicación matricial, 240
- Multiprocesamiento, 225
- Mutable, 39

N

- Newton, paso de, 330
- Newton-Raphson, 330
- Norma, 236
- Norma L2, 236
- Norma-p, 236
- Notación Big-O, 136
- Notación matricial, 246
- Número complejo, 14, 236
- Número de condición, 243
- Número irracional, 236
- Número natural, 236
- Número racional, 236

- Número real, 236
- Números enteros, 236
- NumPy, 47, 50

O

- Objeto, 121, 124
- Objeto función, 69
- Operador de asignación, 27
- Operador lógico, 21
- Operadores aritméticos, 9
- Operadores de comparación, 21
- Orden de complejidad, 135
- Orden de operaciones, 10
- Orden de precisión, 339
- Ortogonal, 238

P

- Paquete, 15
- Parámetro, 279
- Parámetros de entrada, 55
- Parámetros de salida, 55
- Parent class, 127
- Pendiente, 280
- Período, 417
- Pickle, 182

- Pip, 15
- Pivote, 113
- Pivote, ecuación, 249
- Polinomio de Lagrange, 301
- Polimorfismo, 127
- Pool, 228
- Potencia, método de, 269
- Potencia inversa, método de, 272
- Potencia desplazada, método de, 273
- Precisión, 380
- Predictor-corrector, 382
- Problema de contorno, 399
- Problema de valor inicial, 373
- Proceso, 226
- Programación orientada a objetos, 121
- Programación orientada a procedimientos, 121
- Pseudoinversa, 281
- Punto flotante, 147
- Punto medio, regla del, 355

Q

- Quicksort, 113

R

- Rango, 245

- Rango completo, 243
- Raíz, 325
- Recursión, 105
- Reducción de orden, 374
- Regla de Simpson, 359
- Regla del punto medio, 355
- Regla del trapecio, 356
- Regresión de mínimos cuadrados, 280
- Residuo, 280
- Rigidez, 389
- Ruido, 345
- Runge-Kutta, método de, 383

S

- SciPy, 363
- Sentencia de ramificación, 77
- Sentencia If, 77
- Sentencia If-Else, 77
- Sentencia Try-Except, 163
- Señal, 415
- Serie, 315
- Serie de Fourier, 415
- Serie de Taylor, 315
- Sesgo, 147

- Shell de IPython, 7
- Shell de Python, 7
- Signo, indicador de, 147
- Simpson, regla de, 359
- Singular, 243
- Sintaxis, 157
- Sintaxis, error de, 157
- Sistema de ecuaciones lineales, 246
- Sobre flujo, 150
- Solución analítica, 373
- Solución general, 372
- Solución particular, 372
- Subclase, 127
- Subespacio, 245
- Subintervalo, 353
- Subproblemas, 111
- Sucesión, 315
- Superclase, 123, 127
- Sustitución hacia adelante, 248
- Sustitución hacia atrás, 248

T

- Tabla de verdad, 21
- Tamaño de paso, 337

- Tiempo exponencial, 137
- Tiempo logarítmico, 137
- Tiempo polinomial, 136
- Tipo de dato, 29
- Tolerancia, 326
- Transformada de Fourier, 415, 421
- Transformada de Fourier Discreta, 421
- Transformada de Fourier Rápida, 428
- Transformación lineal, 244
- Transpuesta, 240
- Trapecio, regla del, 356
- Trapecio, regla del, 356
- Try-Except, 163
- Tuplas, 38

U

- Unión, 235
- Unión, 235

V

- Valor inicial, 373
- Variable, 27
- Variable de bucle, 91
- Variable global, 66
- Variable local, 64

- Vector, 236
- Vector columna, 236
- Vector fila, 236
- Vector nulo, 236, 245
- Vector propio, ver Autovector
- Vector unitario, 236
- Vectorización, 51
- Vectores linealmente dependientes, 239
- Vectores linealmente independientes, 239

W

- While-loop, 97
- Windows, 4

X

- XML, 182

Y

- Y (AND), 21

Z

- Zip, 318

Agradecimientos

Este apunte me tomo mucho tiempo traducirlo, espero les sea de su agrado , lo hice con mucho cariño y con la esperanza de que les sea util, espero les sirva y siempre es bueno aprender programacion. Seguire realizando publicacion de documentos ,solucionarios , traducciones y demas para ayudar a la comunidad Exactas. Seguramente lo que muchos se preguntan y me han preguntado es Por que hago esto? Simple, porque quiero ayudar y se lo dificil que es no tener a alguien que te explique los temas, reprobar, recursar y es dificil y desgastante. Por lo que mi filosofia es ayudar a los demas para que les vaya bien, ya que todos somos parte de una comunidad enorme, EXACTAS. Les deseo exitos y MAXIMO ESFUERZO para la carrera.

Nahuel Villafañe